

类型和程序设计语言

Types and Programming Languages

Benjamin C. Pierce 著

原版：The MIT Press, 2002

ISBN: 978-0-262-16209-8

目录

译者序	x
前言	xi
第一章 引言	1
1.1 计算机科学中的类型	1
1.2 类型系统有何用处	3
1.3 类型系统与语言设计	7
1.4 简史	7
1.5 延伸阅读	10
第二章 数学预备知识	12
2.1 集合、关系与函数	12
2.2 有序集	13
2.3 序列	15
2.4 归纳法	15
2.5 背景读物	16
第一部分 无类型系统	17
第三章 无类型算术表达式	18
3.1 引言	18
3.2 语法	19
3.3 项上的归纳	22
3.4 语义风格	24
3.5 求值	25
3.6 注记	33
第四章 算术表达式的 ML 实现	34
4.1 语法	34
4.2 求值	37
4.3 其余部分	40

目录	iii
第五章 无类型 lambda 演算	41
5.1 基础	42
5.2 用 lambda 演算编程	46
5.3 形式细节	56
5.4 注记	60
第六章 项的无名表示	61
6.1 项与上下文	62
6.2 移位与替换	64
6.3 求值	66
第七章 lambda 演算的 ML 实现	67
7.1 项与上下文	67
7.2 移位与替换	72
7.3 求值	75
7.4 注记	77
第二部分 简单类型	78
第八章 带类型的算术表达式	79
8.1 类型	79
8.2 类型关系	80
8.3 安全性等于进展性加保持性	82
第九章 简单类型 lambda 演算	86
9.1 函数类型	86
9.2 类型关系	88
9.3 类型关系的性质	90
9.4 Curry-Howard 对应	94
9.5 擦除与可赋型性	97
9.6 Curry 风格与 Church 风格	98
9.7 注记	98
第十章 简单类型的 ML 实现	99
10.1 上下文	99
10.2 项与类型	102
10.3 类型检查	103
第十一章 简单扩展	106
11.1 基本类型	106
11.2 Unit 类型	107

11.3 派生形式：顺序执行与通配符	108
11.4 类型标注	109
11.5 let 绑定	111
11.6 二元对	112
11.7 元组	114
11.8 记录	114
11.9 和类型	117
11.10 变体	119
11.11 一般递归	125
11.12 列表	128
第十二章 规范化	130
12.1 简单类型的规范化	130
12.2 注记	134
第十三章 引用	135
13.1 引言	135
13.2 类型	139
13.3 求值	139
13.4 存储类型	141
13.5 安全性	145
13.6 注记	147
第十四章 异常	148
14.1 抛出异常	149
14.2 处理异常	151
14.3 携带值的异常	151
第三部分 子类型化	154
第十五章 子类型化	155
15.1 包容规则	155
15.2 子类型关系	156
15.3 子类型关系与类型关系的性质	160
15.4 Top 与 Bot 类型	162
15.5 子类型化与其他特性	163
15.6 子类型化的强制转换语义	169
15.7 交类型与并类型	172
15.8 注记	173

第十六章 子类型化的元理论	175
16.1 算法子类型关系	176
16.2 算法类型关系	179
16.3 并与交	182
16.4 算法类型关系与 Bot 类型	184
第十七章 子类型化的 ML 实现	185
17.1 语法	185
17.2 子类型关系	186
17.3 类型检查	188
第十八章 案例研究：命令式对象	192
18.1 什么是面向对象程序设计?	192
18.2 对象	194
18.3 对象生成器	195
18.4 子类型化	195
18.5 组合实例变量	195
18.6 简单的类	196
18.7 增加实例变量	197
18.8 调用超类方法	198
18.9 带 <code>self</code> 的类	199
18.10 通过 <code>self</code> 实现开放递归	200
18.11 开放递归与求值次序	201
18.12 更高效的实现	204
18.13 小结	207
18.14 注记	207
第十九章 案例研究：Featherweight Java	209
19.1 引言	209
19.2 概览	210
19.3 名义类型系统与结构类型系统	212
19.4 定义	214
19.5 性质	220
19.6 对象编码与作为基本构造的对象	221
19.7 注记	222
第四部分 递归类型	224
第二十章 递归类型	225
20.1 示例	226

20.2 形式定义	233
20.3 子类型化	236
20.4 注记	237
第二十一章 递归类型的元理论	238
21.1 归纳与余归纳	238
21.2 有限类型与无限类型	240
21.3 子类型化	242
21.4 关于传递性的题外讨论	244
21.5 成员关系检查	245
21.6 更高效的算法	248
21.7 正则树	252
21.8 μ -类型	253
21.9 子表达式计数	257
21.10 题外讨论：指数时间算法	262
21.11 同构递归类型的子类型化	263
21.12 注记	264
第五部分 多态	266
第二十二章 类型重构	267
22.1 类型变量与替换	267
22.2 理解类型变量的两种方式	268
22.3 基于约束的赋型	269
22.4 合一	274
22.5 主类型	276
22.6 隐式类型标注	277
22.7 let 多态	278
22.8 注记	282
第二十三章 全称类型	284
23.1 动机	284
23.2 多态的几种形式	284
23.3 System F	285
23.4 示例	287
23.5 基本性质	293
23.6 擦除、可赋型性与类型重构	294
23.7 擦除与求值次序	297
23.8 System F 的受限片段	298

23.9 参数性	298
23.10 非直谓性	299
23.11 注记	300
第二十四章 存在类型	301
24.1 动机	301
24.2 用存在类型实现数据抽象	305
24.3 用全称类型编码存在类型	311
24.4 注记	312
第二十五章 System F 的 ML 实现	313
25.1 类型的无名表示	313
25.2 类型移位与替换	314
25.3 项	318
25.4 求值	324
25.5 赋型	326
第二十六章 有界量化	330
26.1 动机	330
26.2 定义	331
26.3 示例	335
26.4 安全性	338
26.5 有界存在类型	343
26.6 注记	345
第二十七章 案例研究：再论命令式对象	347
第二十八章 有界量化的元理论	352
28.1 暴露	352
28.2 最小赋型	353
28.3 核 $F_{<}$ 的子类型算法	355
28.4 完全 $F_{<}$ 的子类型	357
28.5 完全 $F_{<}$ 的不可判定性	359
28.6 并与交	362
28.7 有界存在类型	365
28.8 有界量化与底类型	366
第六部分 高阶系统（建设中）	
第二十九章 类型算子与种类（建设中）	

第三十章 高阶多态（建设中）**第三十一章 高阶子类型化（建设中）****第三十二章 案例研究：纯函数式对象（建设中）****第七部分 附录 367****附录 A 习题解答选编 368**

A.1 第 3 章：无类型算术表达式	368
A.2 第 4 章：算术表达式的 ML 实现	374
A.3 第 5 章：无类型 lambda 演算	375
A.4 第 6 章：项的无名表示	381
A.5 第 8 章：带类型的算术表达式	383
A.6 第 9 章：简单类型 lambda 演算	384
A.7 第 11 章：简单扩展	387
A.8 第 12 章：规范化	396
A.9 第 13 章：引用	398
A.10 第 14 章：异常	402
A.11 第 15 章：子类型化	403
A.12 第 16 章：子类型化的元理论	407
A.13 第 17 章：子类型化的 ML 实现	415
A.14 第 18 章：案例研究——命令式对象	420
A.15 第 19 章：案例研究——Featherweight Java	423
A.16 第 20 章：递归类型	428
A.17 第 21 章：递归类型的元理论	434
A.18 第 22 章：类型重构	440
A.19 第 23 章：全称类型	449
A.20 第 24 章：存在类型	456
A.21 第 25 章：System F 的 ML 实现	459
A.22 第 26 章：有界量化	460
A.23 第 27 章：案例研究——再论命令式对象	464
A.24 第 28 章：有界量化的元理论	465

译者附录：原书未解答练习参考解答 468

第 2 章：数学预备知识	468
第 3 章：无类型算术表达式	470
第 4 章：算术表达式的 ML 实现	470
第 5 章：无类型 lambda 演算	472
第 6 章：项的无名表示	472

第 7 章：无类型 lambda 演算的 ML 实现	475
第 8 章：带类型的算术表达式	477
第 9 章：简单类型 lambda 演算	478
第 11 章：简单扩展	479
第 15 章：子类型化	480
第 16 章：子类型化的元理论	482
第 17 章：子类型化的 ML 实现	483
第 18 章：案例研究——命令式对象	497
第 19 章：案例研究——Featherweight Java	498
第 20 章：递归类型	509
第 22 章：类型重构	511
第 23 章：全称类型	525
第 24 章：存在类型	527
第 26 章：有界量化	528
第 28 章：有界量化的元理论	530

译者序



——写于 2026 年 8 月 28 日，科兴科学园，离职前

前言

类型系统研究，以及从类型论视角开展的程序语言研究，已经成为一个充满活力的领域，在软件工程、语言设计、高性能编译器实现和安全等方面有着重要应用。本书全面介绍这一领域的基本定义、主要结果和核心技术。

读者对象

本书主要面向两类读者：一类是专攻程序语言和类型论的研究生与研究人员；另一类是来自计算机科学各个方向、希望了解程序语言理论核心概念的研究生和基础扎实的本科生。对于前一类读者，本书将系统梳理整个领域，其深度足以使读者随后直接研读研究文献；对于后一类读者，本书提供了大量入门材料，以及丰富的示例、练习和案例研究。本书既可以作为程序语言入门研究生课程的主教材，也可以用于高级专题研讨课。

目标

本书的首要目标是覆盖一系列核心主题，包括基本的操作语义及相关证明技术、无类型 lambda 演算、简单类型系统、全称多态与存在多态、类型重构、子类型化、有界量化、递归类型和类型算子；此外还会较为简略地讨论许多其他主题。

第二个主要目标是注重**实用**。本书集中讨论类型系统在程序语言中的应用，为此舍弃了一些在更偏数学的带类型 lambda 演算教材中很可能会出现的主题，例如指称语义。全书底层的计算模型采用按值调用 (*call-by-value*) 的 lambda 演算；它与当今大多数程序语言相契合，也很容易在此基础上加入引用、异常等会影响程序状态或执行流程的语言机制。对于每一种语言特性，我们主要关心的是：考虑这一特性的实际动机；证明包含这一特性的语言具有安全性所需的技术；以及它所引出的实现问题，尤其是类型检查算法的设计与分析。

另一个目标是尊重本领域的**多样性**。本书覆盖了许多单独的主题和若干已经得到充分理解的组合，但并不试图把所有内容统一到一个系统之中。对于其中一些主题的子集，人们已经给出了统一的表述。例如，纯类型系统 (*pure type system*) 采用统一的记法，可以优雅而紧凑地处理多种广义函数类型 (原文称 “*arrow types*”)。然而，整个领域的发展仍然太快，尚不足以得到完整的系统化整理。

无论用于课堂教学还是自学，本书都力求**便于使用**。书中为大多数练习给出了完整解答。核心定义被整理成内容完备、便于单独查阅的图表。概念与系统之间的依赖关系也尽可能明确地标示出来。此外，本书还配有详尽的参考文献和索引。

最后一项组织原则是**求实**。除少数仅顺带提及的系统外，本书讨论的所有系统都有实现。每章都配有类型检查器和解释器，用来对示例进行机械检查。这些实现可以从本书网站获得，可用于编程练习、扩展实验和规模更大的课程项目。

为了实现上述目标，本书不得不放弃另外一些同样可取的性质。其中最重要的是覆盖面**的完备性**。想在一本书里调查程序语言和类型系统的整个领域大概是不可能的——对于教材而言尤其如此。本书着重细致地展开核心概念，同时提供了大量通向研究文献的线索，供读者继续深入学习。第二个非目标是类型检查算法的实际**效率**：本书并不是一本讲解工业级编译器或类型检查器实现的著作。

结构

本书第一部分讨论无类型系统。我们先在一种极其简单的数值与布尔值语言中介绍抽象语法、归纳定义与证明、推导规则和操作语义等基本概念，再针对无类型 lambda 演算重复这一过程。第二部分讨论简单类型 lambda 演算，以及积、和、记录、变体、引用和异常等多种基本语言特性。其中有一章先讨论带类型的算术表达式，以较为平缓的方式引入类型安全性这一核心思想；另有一章为选读内容，使用 Tait 方法证明简单类型 lambda 演算的规范化性质。第三部分讨论子类型化这一基本机制，其中包括对元理论的详细研究和两个篇幅较长的案例研究。第四部分讨论递归类型，涵盖较简单的同构递归 (*iso-recursive*) 形式和更为棘手的等递归 (*equi-recursive*) 形式。本部分两章中的第二章在余归纳 (*coinduction*) 这一数学框架下，研究一个包含等递归类型和子类型化的系统的元理论。第五部分转向多态：先讨论 ML 风格的类型重构，再讨论 System F 中的非直谓多态 (*impredicative polymorphism*)，其表达能力强于前述 ML 风格的多态；接着讨论存在量化及其与抽象数据类型的联系，最后讨论在带有有界量化的系统中如何组合多态与子类型化。第六部分讨论类型算子：第一章介绍基本概念；第二章展开 F_ω 及其元理论；第三章把类型算子与有界量化结合起来，得到 $F_{\omega<}$ ；最后一章以一个收束全书的案例研究作结。

各章之间的主要依赖关系如图P-1所示。灰色箭头表示后面的章节中只有一部分依赖前面的章节。

本书对每项语言特性的讨论都遵循一个共同模式：首先给出阐明动机的示例；然后给出形式化定义；接着证明类型安全性等基本性质；随后（通常另设一章）深入研究元理论，导出类型检查算法，并证明算法的可靠性、完备性和终止性；最后（同样另设一章）把这些算法具体实现为 OCaml (Objective Caml) 程序。

贯穿全书的一类重要示例来自对面向对象程序设计特性的分析与设计。四个案例研究章节分别详细展开了不同的方法：一种关于传统命令式对象和类的简单模型（第十八章）；基于 Java 的核心演算（第十九章）；利用有界量化对命令式对象所作的更精细刻画（第二十七章）；以及在 $F_{\omega<}$ 这一纯函数式环境中借助存在类型处理对象与类的方法（第三十二章）。

为了让本书的篇幅足以在一个学期的高级课程中讲完——同时也让普通研究生还能举得动它——我们不得不排除许多有趣而重要的主题。指称语义和公理语义的方法被完全略去；这些方法已经有优秀的专著介绍，而且在这里讨论它们会削弱本书鲜明的实用取向和实现取向。类型系统与逻辑之间丰富的联系在少数地方有所提示，但没有详细展开；它们虽然重要，

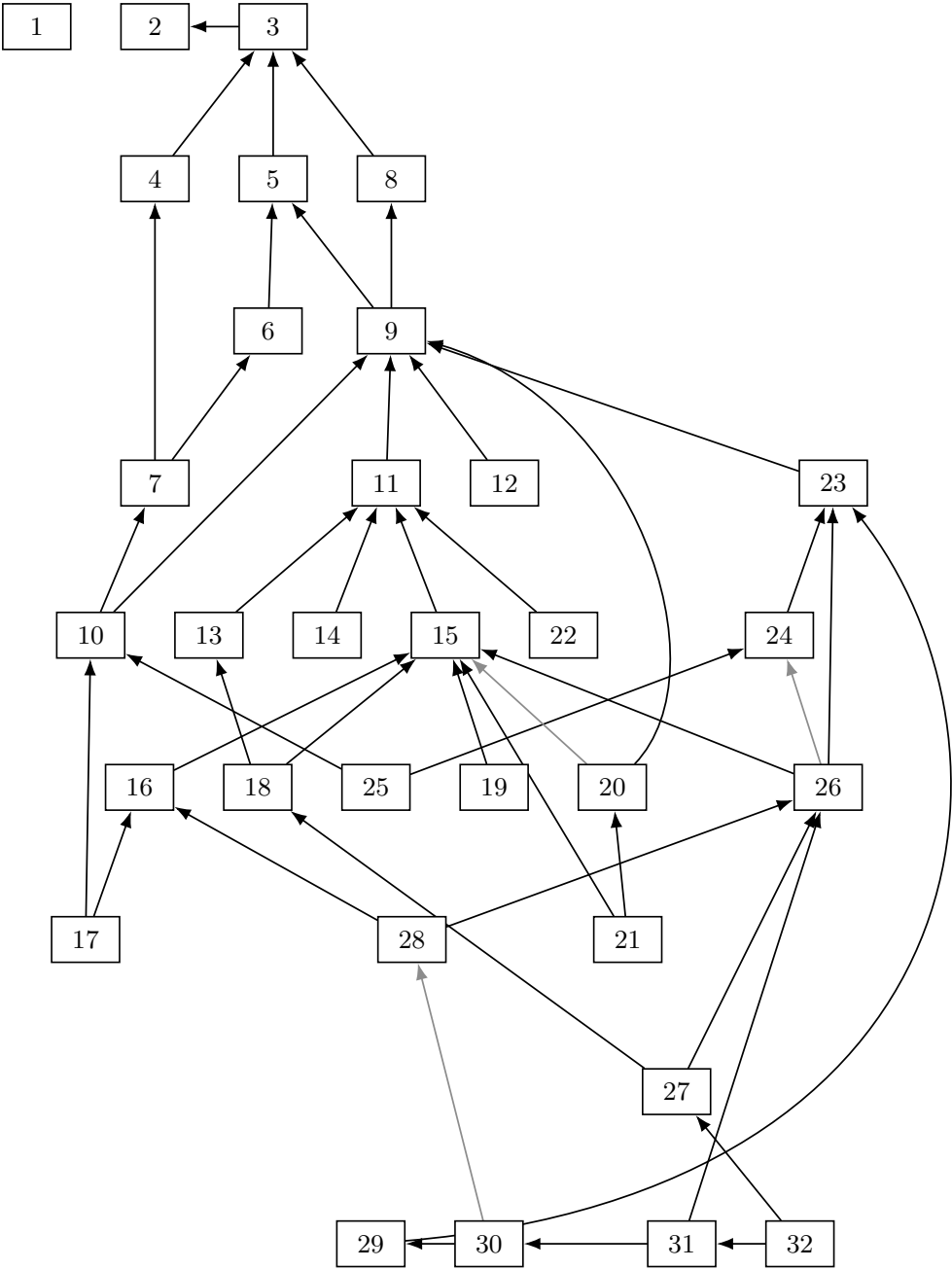


图 P-1: 章节依赖关系

却会把讨论带得太远。许多程序语言和类型系统的高级特性也只是顺带提及，例如依赖类型、交类型 (*intersection type*) 和 Curry-Howard 对应；书中关于这些主题的简短小节可以作为进一步阅读的起点。最后，除了对一个类似 Java 的核心语言所作的短暂考察 (第十九章) 之外，本书完全集中于基于 lambda 演算的系统。不过，在这一环境下发展出的概念和机制，可以直接迁移到带类型的并发语言、类型化汇编语言和专用对象演算等相关领域。

预备知识

本书不要求读者预先学习过程序语言理论，但读者应具备一定的数学素养，尤其应系统而严谨地学习过本科层次的离散数学、算法和初等逻辑课程。

读者应熟悉至少一种高阶函数式程序设计语言 (Scheme、ML、Haskell 等)，并掌握程序语言与编译器的基本概念，例如抽象语法、BNF 文法、求值和抽象机 (*abstract machine*)。许多优秀的本科教材都介绍了这些材料；我尤其喜欢 Friedman、Wand 和 Haynes 的 *Essentials of Programming Languages* (2001)，以及 Scott 的 *Programming Language Pragmatics* (1999)。在若干章节中，具有 Java (Arnold 和 Gosling, 1996) 等面向对象语言的使用经验也会有所帮助。

具体实现类型检查器的章节给出了相当数量的 OCaml (亦称 Objective Caml) 代码片段。OCaml 是 ML 的一种流行方言。事先了解 OCaml 有助于阅读这些章节，但并非绝对必要：书中只使用了该语言的一小部分，而且每种特性都会在首次出现时加以解释。这些章节形成了一条与全书其余部分相对独立的线索；读者如果愿意，也可以将其全部跳过。

译者注：原书的程序实现采用 OCaml。本译本保留原书的 OCaml 代码及相关说明，并在每段 OCaml 代码之后直接给出对应的 Rust 实现，便于对照阅读。Rust 版本属于译者增补材料，不替代原书实现；书中片段从配套代码工程的实际 Rust 源码自动抽取，完整工程按章保存在 `code/` 目录，并已在本项目规定的环境中通过编译和测试。

目前最好的 OCaml 教材是 Cousineau 和 Mauny 的著作 (1998)。OCaml 发行版附带的教程材料也很容易阅读，可从 <http://caml.inria.fr> 和 <http://www.ocaml.org> 获得。

熟悉 ML 的另一种主要方言 Standard ML 的读者，在阅读 OCaml 代码片段时应该不会遇到困难。流行的 Standard ML 教材包括 Paulson (1996) 和 Ullman (1997) 的著作。

课程安排

一门中级或高级研究生课程应当能够在一个学期内讲授本书的大部分内容。图P-2给出了宾夕法尼亚大学一门博士生高级课程的示例大纲：每周两次课，每次 90 分钟；假设学生在程序语言理论方面几乎没有预备知识，但课程推进较快。

对于本科课程或入门研究生课程，可以从本书材料中选择多条不同的讲授路径。一门面向程序设计的类型系统课程，可以集中讨论引入各种类型特性并说明其用途的章节，略去大部分元理论和实现章节。另一种选择是开设类型系统基本理论与实现课程，依次讲授前面的所有章节，但很可能跳过第十二章 (也可能跳过第十八章和第二十一章)，并相应放弃本书

讲次	主题	阅读章节
1	课程概览；历史；课程事务	1, (2)
2	预备知识：语法、操作语义	3, 4
3	lambda 演算导论	5.1, 5.2
4	lambda 演算的形式化	5.3, 6, 7
5	类型；简单类型 lambda 演算	8, 9, 10
6	简单扩展；派生形式	11
7	更多扩展	11
8	规范化	12
9	引用；异常	13, 14
10	子类型化	15
11	子类型化的元理论	16, 17
12	命令式对象	18
13	Featherweight Java	19
14	递归类型	20
15	递归类型的元理论	21
16	递归类型的元理论	21
17	类型重构	22
18	全称多态	23
19	存在多态；抽象数据类型	24, (25)
20	有界量化	26, 27
21	有界量化的元理论	28
22	类型算子	29
23	F_ω 的元理论	30
24	高阶子类型化	31
25	纯函数式对象	32
26	预留课时	

图 P-2: 高级研究生课程示例大纲

后半部分较为高级的材料。较短的课程也可以借助图P-1中的依赖关系图，选择若干感兴趣的章节来组织。

本书也适合用作更一般的程序语言理论研究生课程的主教材。这类课程可以用半个到三分之二个学期学习本书的大部分内容，把剩余时间用于其他教学单元，例如：基于 Milner 的 π 演算著作 (1999) 讨论并发理论；介绍 Hoare 逻辑和公理语义（例如 Winskel, 1993）；或者概览 续延（*continuation*）、模块系统等高级语言特性。

如果课程十分重视学期项目，可以推迟讲授部分理论材料，例如规范化，或许还有若干元理论章节。这样，学生在选择项目主题之前就能接触更为广泛的示例。

练习

大多数章节都包含大量练习：有些适合纸笔完成；有些要求 在所讨论的演算中编写程序示例；还有些涉及扩展 这些演算的 ML 实现。每道练习的预计难度采用下列等级表示：

标记	难度	预计用时
★	快速检查	30 秒至 5 分钟
★★	简单	不超过 1 小时
★★★	中等	不超过 3 小时
★★★★	有挑战性	超过 3 小时

标有 ★ 的练习用于在阅读过程中即时检查对重要概念的理解。强烈建议读者在继续阅读后续内容之前，停下来完成每一道这样的练习。每章还会选出一组规模大致相当于一次作业的练习，并标为 RECOMMENDED（推荐）。

附录 A 给出了大多数练习的完整解答。少数没有提供解答的练习会标上 ↯，以免读者徒劳地到附录中寻找。

排版约定

大多数章节先以叙述方式介绍某个类型系统的特性，然后在一幅或多幅图中把该系统形式化定义为一组推导规则。为了便于查阅，这些定义通常会完整呈现：不仅包括当前讨论特性所新增的规则，也包括构成完整演算所需的其余规则。新增部分使用灰色背景，以便清楚显示它相对于先前系统的“增量”。

本书制作过程有一个不同寻常的特点：所有示例都会在排版期间接受机械的类型检查。一个脚本会逐章提取示例，生成并编译一个包含当前所讨论特性的定制类型检查器，再把它应用到示例上，并将检查器的响应插入正文。完成其中困难部分的系统名为 TinkerType，由 Michael Levin 和我共同开发 (2001)。美国国家科学基金会通过项目 CCR-9701826 *Principled Foundations for Programming with Objects* 和 CCR-9912352 *Modular Type Systems* 为这项研究提供了资助。

电子资源

与本书配套的网站位于：

<http://www.cis.upenn.edu/~bcpierce/tapl>

网站提供的资源包括勘误表、课程项目建议、读者贡献的补充材料链接，以及本书各章所讨论演算的一组实现（类型检查器和简单解释器）。

这些实现为试验书中的示例和检验练习答案提供了环境。它们也经过整理，以便阅读和修改；我的学生已经成功地用它们完成过小型实现练习和规模更大的课程项目。这些实现使用 OCaml 编写。OCaml 编译器可从 <http://caml.inria.fr> 免费获得，并且在大多数平台上都很容易安装。

读者还应当了解 *Types Forum*，这是一个讨论类型系统及其应用各个方面的电子邮件列表。该邮件列表采用审核制，以控制邮件数量，并使公告和讨论保持较高的信噪比。邮件归档和订阅说明可在 <http://www.cis.upenn.edu/~bcpierce/types> 找到。

致谢

如果读者认为本书有所价值，那么最应当感谢的是我的四位导师：Luca Cardelli、Bob Harper、Robin Milner 和 John Reynolds。关于程序语言和类型的知识，我大多是从他们那里学到的。

其余知识主要来自合作研究。除了 Luca、Bob、Robin 和 John，我在这些研究中的合作者还包括 Martín Abadi、Gordon Plotkin、Randy Pollack、David N. Turner、Didier Rémy、Davide Sangiorgi、Adriana Compagnoni、Martin Hofmann、Giuseppe Castagna、Martin Steffen、Kim Bruce、Naoki Kobayashi、Haruo Hosoya、Atsushi Igarashi、Philip Wadler、Peter Buneman、Vladimir Gapeyev、Michael Levin、Peter Sewell、Jérôme Vouillon 和 Eijiro Sumii。这些合作不仅奠定了我对这一主题的理解，也构成了我从中获得乐趣的基础。

Thorsten Altenkirch、Bob Harper 和 John Reynolds 与我讨论教学方法，改进了本书的结构和组织。Jim Alexander、Penny Anderson、Josh Berdine、Tony Bonner、John Tang Boyland、Dave Clarke、Diego Dainese、Olivier Danvy、Matthew Davis、Vladimir Gapeyev、Bob Harper、Eric Hilsdale、Haruo Hosoya、Atsushi Igarashi、Robert Irwin、Takayasu Ito、Assaf Kfoury、Michael Levin、Vassily Litvinov、Pablo López Olivas、Dave MacQueen、Narciso Marti-Oliet、Philippe Meunier、Robin Milner、Matti Nykänen、Gordon Plotkin、John Prevost、Fermin Reig、Didier Rémy、John Reynolds、James Riely、Ohad Rodeh、Jürgen Schlegelmilch、Alan Schmitt、Andrew Schoonmaker、Olin Shivers、Perdita Stevens、Chris Stone、Eijiro Sumii、Val Tannen、Jérôme Vouillon 和 Philip Wadler 提供的订正与意见改进了正文。（如果我无意中漏掉了谁，谨此致歉。）Luca Cardelli、Roger Hindley、Dave MacQueen、John Reynolds 和 Jonathan Seldin 还就若干错综复杂的历史问题提供了亲历者的视角。

1997 年和 1998 年参加我在印第安纳大学开设的研究生研讨课、以及 1999 年和 2000 年参加我在宾夕法尼亚大学开设的研究生研讨课的学生，坚持读完了本书的早期版本；他们的

反应和意见为本书最终成形提供了至关重要的指引。MIT Press 的 Bob Prior 及其团队专业地引导书稿完成了出版过程的各个阶段。本书的设计以 Christopher Manning 为 MIT Press 开发的 L^AT_EX 宏为基础。

程序证明太过乏味，数学界检验证明所依赖的社会过程无法在这里发挥作用。¹

Proofs of programs are too boring for the social process of mathematics to work.

——Richard DeMillo、Richard Lipton 和 Alan Perlis, 1979

……所以，程序验证不应依赖这种社会过程。²

... So don't rely on social processes for verification.

——David Dill, 1999

只有当不理解形式化方法的人也能使用它们时，形式化方法才会产生重大影响。

Formal methods will never have a significant impact until they can be used by people that don't understand them.

——据传为 Tom Melham 所言

¹译者注：这里的“社会过程”，指数学共同体通过阅读、讨论、质疑和纠错，逐步建立对定理及其证明的信心。DeMillo、Lipton 与 Perlis 认为，程序证明难以形成同样的共同检验过程。

²译者注：Dill 此处回应上一则引文。他长期从事形式化验证研究；这句话可以理解为：即使程序证明难以依靠数学共同体的社会过程获得信任，也不必因此放弃程序验证，而应让验证更多地依靠可重复的机械检查。

第一章 引言

1.1 计算机科学中的类型

现代软件工程采用了种类繁多的形式化方法，来帮助确保系统的行为符合某种隐式或显式的预期行为规约 (*specification*)。在这一谱系的一端，是 Hoare 逻辑、代数规约语言、模态逻辑和指称语义等能力强大的框架。它们可以表达非常一般的正确性性质，但使用起来往往相当繁琐，并且要求程序员具备较深的专业知识。在另一端，则是能力弱得多的一些技术；也正因为能力有限，它们的自动检查器可以嵌入编译器、链接器或程序分析器，因而即使程序员不了解背后的理论也能使用。这类轻量级形式化方法 (*lightweight formal method*) 中，一个广为人知的例子是模型检查器 (*model checker*)：这类工具在芯片设计、通信协议等有限状态系统中搜索错误。另一个日益流行的例子是运行时监测 (*run-time monitoring*)，即一组使系统能够在运行过程中发现某个组件没有按规约行事的技术。不过，应用最广、发展也最成熟的轻量级形式化方法，还是本书的核心主题——类型系统。

“类型系统”这个词由许多不同群体共同使用，因此很难给出一个定义：既能涵盖程序语言设计者和实现者对它的非正式用法，又具体到足以提供实质内容。下面是一种合理的定义：

类型系统是一种可有效处理的句法方法：它按照程序短语所计算出的值的种类对这些短语进行分类，从而证明某些程序行为不会发生。

这里有几点值得说明。首先，这一定义把类型系统视为对程序进行推理的工具。这种措辞反映了本书对程序语言中类型系统的侧重。更一般地说，“类型系统”（或“类型论”）指逻辑、数学和哲学中一个范围广得多的研究领域。这一意义上的类型系统最早在 20 世纪初得到形式化，用来避开 Russell 悖论 (Russell, 1902) 等威胁数学基础的逻辑悖论。20 世纪期间，类型成为逻辑学——尤其是证明论——中的标准工具（参见 Gandy, 1976; Hindley, 1997），并渗入哲学与科学的语言。该领域的重要里程碑包括 Russell 最初的分支类型论 (*ramified theory of types*) (Whitehead 和 Russell, 1910)、Ramsey 的简单类型论 (1925)——它是 Church 的简单类型 lambda 演算 (1940) 的基础——Martin-Löf 的构造性类型论 (1973, 1984)，以及 Berardi、Terlouw 和 Barendregt 的纯类型系统 (Berardi, 1988; Terlouw, 1989; Barendregt, 1992)。

即使只看计算机科学，类型系统研究也有两条主要分支。偏实用的一支关注类型系统在程序语言中的应用，是本书的主要内容。更抽象的一支则通过 Curry-Howard 对应 (§9.4)，研究各种“纯带类型 lambda 演算”与不同逻辑之间的联系。两个研究群体使用相似的概念、记法和技术，但关注取向存在一些重要差别。例如，带类型 lambda 演算研究通常考察保证

每个良类型 (*well-typed*) 计算都终止的系统；大多数程序语言则为了递归函数定义等特性而放弃了这一性质。

上述定义的另一个重要方面，是强调按照项——也就是句法短语——执行后所计算出的值的性质，对项进行分类。可以把类型系统看作是在计算一种对程序中各项运行时行为的静态近似 (*static approximation*)。(而且，项的类型通常以组合式 (*compositionally*) 方法计算：一个表达式的类型只取决于其子表达式的类型。)

有时人们会明确加上“静态”二字，例如称“静态类型程序语言”，以便把这里讨论的编译期分析与 Scheme (Sussman 和 Steele, 1975; Kelsey、Clinger 和 Rees, 1998; Dybvig, 1996) 等语言中的动态类型 (*dynamic typing*) 或潜在类型 (*latent typing*) 区分开来；后者利用运行时类型标签，辨别堆中不同种类的数据结构。“动态类型”之类的说法严格说来名不副实，也许应当改称“动态检查”，但这种用法已经约定俗成。

类型系统既然是静态的，就必然也是保守的 (*conservative*)：它们能够确凿地证明某些不良程序行为不会发生；但如果无法作出这种证明，也不能由此断定这些行为实际会发生。因此，它们有时也不得不拒绝那些实际上在运行时表现良好的程序。例如，下面的程序会被判定为非良类型：

```
if <complex test> then 5 else <type error>
```

即使其中的 `<complex test>` 碰巧总会求值为真，它仍会被拒绝，因为静态分析无法断定事实果真如此。在类型系统设计中，保守性与表达能力之间的张力是一项根本而无法回避的事实。研究者希望通过给程序的各个部分赋予更精确的类型，让更多程序能够通过类型检查；这是推动该领域研究的主要动力。

与此相关的一点是，大多数类型系统采用的分析都相对简单：我们不能任意指定一种不希望出现的程序行为，就指望类型系统将它排除；类型系统只能保证良类型程序不会出现某些特定类别的不良行为。例如，大多数类型系统都能静态检查：基本算术运算的实参总是数值，方法调用的接收者对象总能提供所请求的方法，等等；但它们通常无法保证除法运算的第二个实参非零，也无法保证数组访问始终不越界。

在一门给定的语言中，那些本来可能在程序运行时发生、但能够由类型系统预先排除的不良行为，通常称为运行时类型错误 (*run-time type error*)。必须牢记，哪些行为属于这一集合，是每种语言各自作出的选择。不同语言对运行时类型错误的认定虽有大量重合，但原则上，每个类型系统都要自行定义它试图防止哪些行为。判断一个类型系统是否安全（或可靠），必须以它自己所定义的运行时错误集合为准。

类型分析能够检测的不良行为，并不限于调用不存在的方法这类底层故障。类型系统还可以保证程序满足更高层次的模块化要求，并保护用户定义的抽象不受破坏。违反信息隐藏同样属于运行时类型错误。例如，若某个数据值的表示本应保持抽象，程序却直接访问了它的字段，这种违反信息隐藏的行为，和把整数当作指针使用、进而导致机器崩溃一样，都被归入运行时类型错误。

类型检查器通常内置于编译器或链接器。这意味着它们必须自动完成工作，无须人工干预，也无须与程序员交互；换言之，它们所体现的分析在计算上必须切实可行。不过，程序员仍有很大的余地通过显式类型标注 (*explicit type annotation*) 来提供指引。通常，这些标注

会保持得相当简洁，以便程序更易编写和阅读。但从原则上说，也可以在类型标注中编码一份完整证明，证明程序满足某个任意规约；这时类型检查器实际上就成了证明检查器 (*proof checker*)。扩展静态检查 (*Extended Static Checking*) (Detlefs、Leino、Nelson 和 Saxe, 1998) 等技术正在探索类型系统与完整程序验证方法之间的地带：它们对若干范围广泛的正确性性质实施完全自动的检查，只需“相当简洁”的程序标注加以引导。

同理，我们最感兴趣的不只是原则上可以自动化的方法，而是确实配有高效类型检查算法的方法。不过，怎样才算高效仍有争议。即便是 ML 这种得到广泛应用的类型系统 (Damas 和 Milner, 1982)，在某些极端情况下，类型检查也可能耗时极长 (Henglein 和 Mairson, 1991)。有些语言的类型检查或类型重构问题甚至不可判定，但仍有一些可用算法能在“实际关心的大多数情形”中很快停机 (例如 Pierce 和 Turner, 2000; Nadathur 和 Miller, 1988; Pfenning, 1994)。

1.2 类型系统有何用处

检测错误

静态类型检查最明显的好处，是能够及早发现一部分程序设计错误。尽早发现的错误可以立刻修复，而不会潜伏在代码中，等到很久以后——程序员正忙于别的事情时，甚至程序已经部署以后——才暴露出来。此外，类型检查往往比运行时更能准确定位错误；到了运行时，问题开始发生后，可能要过一段时间才能看到它造成的影响。

在实践中，静态类型检查能够揭示的错误种类多得令人惊讶。使用富类型语言 (*richly typed language*) 时，程序员常说，程序一旦通过类型检查，往往就能“直接工作”，其频率甚至高得超出他们原先的合理预期。一种可能的解释是：不但微小的思维疏漏——例如对字符串开平方之前忘了把它转换成数值——常会在类型层面表现为不一致，更深层的概念错误也会如此，例如复杂分支分析中漏掉边界条件，或在科学计算中混淆计量单位。这个效果有多强，取决于类型系统的表达能力和具体的程序设计任务。处理多种数据结构的程序，例如编译器等符号处理应用，能给类型检查器提供更多可利用的线索；只涉及少数简单类型的程序，例如科学应用中的数值计算，则提供不了那么多线索。(不过，即使在后一种情形中，支持量纲分析 (*dimension analysis*) 的精细类型系统也相当有用；Kennedy, 1994。)

要让类型系统发挥最大效用，程序员通常也要投入一些心力，并愿意善用语言提供的设施。例如，一个把所有数据结构都编码成列表的复杂程序，不会像为每种结构分别定义数据类型或抽象类型的程序那样，从编译器获得充分帮助。表达能力强的类型系统提供了许多“技巧”，可以用类型来编码结构信息。

对于某些程序，类型检查器还是价值极高的维护工具。例如，程序员需要修改某个复杂数据结构的定义时，不必手工搜索大型程序中所有需要随之修改的代码。数据类型声明一经改变，这些位置都会变得类型不一致；只要运行编译器，查看类型检查失败的位置，就能把它们逐一列出。

抽象

类型系统支持程序设计过程的另一个重要方式，是强制执行有章法的程序设计。特别是在大规模软件组合中，模块语言负责封装和连接大型系统的组件，而类型系统构成了模块语言的骨架。类型出现在模块的接口中，也出现在类等相关结构的接口中；事实上，接口本身可以看作“模块的类型”，它概括了模块提供的功能，在实现者与使用者之间形成一种部分契约。

用接口清晰的模块组织大型系统，会促成一种更抽象的设计风格：接口可以独立于最终实现来设计和讨论。对接口作更抽象的思考，通常会带来更好的设计。

文档

阅读程序时，类型同样很有用。过程头部和模块接口中的类型声明构成了一种文档，为程序行为提供有价值的线索。与写在注释里的说明不同，这种文档不会过时，因为编译器每次运行都会检查它。类型的这一作用在 模块签名 (*module signature*) 中尤其重要。

语言安全性

遗憾的是，语言安全性 (*language safety*) 甚至比“类型系统”更具争议。人们见到安全语言时，通常觉得自己认得出来；但究竟什么构成语言安全性，他们的理解又深受各自所属语言群体的影响。不过，非正式地说，安全语言可以定义为这样一种语言：用它编程时，程序员不可能搬起石头砸自己的脚。

把这一直觉再说得精确一些，可以认为安全语言能够保护自身的抽象。每种高级语言都提供了对机器服务的抽象。所谓安全性，是指语言能够保证这些抽象的完整性，也能保证程序员利用语言的定义机制引入的更高层抽象的完整性。例如，语言可以提供数组及其访问、更新操作，以此抽象底层内存。使用这种语言的程序员会期待：只有显式调用数组的更新操作才能改变数组，而不能因为对另一个数据结构发生越界写入就意外改变它。同样，人们会期待 词法作用域变量 (*lexically scoped variable*) 只能在其作用域内访问，调用栈 (*call stack*) 确实按栈的方式工作，等等。在安全语言中，这些抽象可以真正作为抽象来使用；在不安全语言中则不行。要完全理解一个程序可能怎样表现或发生错误，程序员必须始终记住数据结构在内存中的布局、编译器分配它们的顺序等各种底层细节。在极端情况下，不安全语言的程序不仅能破坏自己的数据结构，甚至能破坏运行时系统的数据结构；此时结果可能完全无法预测。

语言安全性不等同于静态类型安全性 (*static type safety*)。语言安全性可以通过静态检查实现，也可以通过运行时检查实现：在无意义的操作刚要执行时将其截获，停止程序或抛出异常。例如，Scheme 没有静态类型系统，却是一门安全语言。

反过来，不安全语言常会提供“尽力而为”的静态类型检查器，帮助程序员至少消除最明显的疏漏；但按照这里的定义，这类语言仍不能称为类型安全，因为它们通常无法保证良类型程序一定表现良好。它们的类型检查器可以提示可能存在运行时类型错误——这当然比什么都不做要好——却不能证明此类错误不会发生。

	静态检查	动态检查
安全	ML、Haskell、Java 等	Lisp、Scheme、Perl、PostScript 等
不安全	C、C++ 等	

上表右下格为空，是因为一旦已经具备在运行时保证大多数操作安全的设施，再检查其余所有操作通常不会增加多少成本。（实际上，也有少数动态检查语言确实提供可读写任意内存位置的底层原语，例如某些面向操作系统极其精简的微型计算机的 Basic 方言；这些原语可能遭到误用，破坏运行时系统的完整性。）

只靠静态类型通常无法实现运行时安全性。例如，上表列出的所有安全语言实际上都在运行时检查数组边界。¹ 类似地，采用静态检查的语言有时也会提供某些操作，例如 Java 的向下类型转换运算符（见 §15.5）。这些操作的类型检查规则实际上并不可靠；因此，要保证语言安全，就必须在程序每次执行这类操作时进行动态检查。

语言安全性很少是绝对的。安全语言常会为程序员提供某种“绕过安全机制的出口”（*escape hatch*），例如调用用其他语言编写的外部函数，而那些语言可能并不安全。事实上，有时语言本身也会以受控形式提供这样的出口，例如 OCaml 的 `Obj.magic` (Leroy, 2000)、Standard ML of New Jersey 中的 `Unsafe.cast` 等。Modula-3 (Cardelli 等, 1989; Nelson, 1991) 和 C# (Wille, 2000) 更进一步，提供一种“不安全子语言”，专门用来实现垃圾收集器等底层运行时设施。只有显式标为不安全的模块才能使用这种子语言的特殊特性。

Cardelli (1996) 从稍有不同的角度讨论语言安全性。他把运行时错误分为 受控错误 (*trapped error*) 和 非受控错误 (*untrapped error*)。受控错误一旦发生，计算便会立即停止，或者抛出一个能够在程序内妥善处理的异常；发生非受控错误后，计算却可能继续，至少可能继续一段时间。例如，在 C 语言中访问数组边界之外的数据，就是一种非受控错误。依照这种观点，安全语言就是能够在运行时阻止非受控错误的语言。

还有一种观点着眼于可移植性，可以用一句口号来表达：“安全语言完全由它的程序员手册定义。”按照这种观点，一门语言的定义，就是程序员为了预测该语言中每个程序的行为而必须了解的全部事项。这样一来，C 之类语言的手册并不构成语言定义，因为有些程序——例如执行未经检查的数组访问或指针算术的程序——只有知道特定 C 编译器如何在内存中安排结构等细节，才能预测其行为；同一个程序由不同编译器生成并执行时，行为可能大不相同。相比之下，Java、Scheme 和 ML 的手册以严谨程度不一的方式，规定了语言中所有程序的确切行为。一个良类型程序在这些语言的任何正确实现下都会得到相同的结果。

效率

计算机科学中最早的一批类型系统，始于 20 世纪 50 年代的 Fortran 等语言 (Backus, 1981)；它们区分取整数值和取实数值的算术表达式，以提高数值计算的效率。这样，编译器就能使用不同的表示，并为基本操作生成适当的机器指令。在安全语言中，还可以通过消除

¹静态消除数组边界检查，是类型系统设计者长期追求的目标。原则上，所需机制——以依赖类型为基础，见 §30.5——已经得到充分理解；但要把它包装成一种可用形式，在表达能力、可预测性、类型检查的可处理性以及程序标注的复杂度之间取得平衡，仍是一项重大挑战。Xi 和 Pfenning (1998, 1999) 介绍了这一领域的一些近期进展。

许多原本用于保证安全性的动态检查来进一步提高效率，因为静态证明已经表明这些检查一定会通过。如今，大多数高性能编译器都在优化和代码生成阶段大量依赖类型检查器收集的信息。即使语言本身没有类型系统，它的编译器也会努力恢复出这种类型信息的近似。

依靠类型信息提高效率，有时会出现令人意想不到的地方。例如，近期研究表明，在并行科学计算程序中，类型分析生成的信息不仅能改善代码生成决策，还能改善指针表示。Titanium 语言 (Yelick 等, 1998) 使用类型推断技术分析指针的作用域，据此作出的决策可以显著优于程序员亲自对程序进行显式调优的结果。ML Kit 编译器采用功能强大的区域推断 (*region inference*) 算法 (Gifford、Jouvelot、Lucassen 和 Sheldon, 1987; Jouvelot 和 Gifford, 1991; Talpin 和 Jouvelot, 1992; Tofte 和 Talpin, 1994, 1997; Tofte 和 Birkedal, 1998)，以基于栈的内存管理取代大部分垃圾收集需求；在某些程序中，它甚至能取代全部垃圾收集。

更多应用

除了在程序设计和语言设计中的传统用途，类型系统如今还以许多更具体的方式，应用于计算机科学及相关学科。这里仅举几例。

类型系统一个日益重要的应用领域是计算机与网络安全。例如，静态类型构成了 Java 安全模型和网络设备 Jini “即插即用” 体系结构 (Arnold 等, 1999) 的核心，也是实现证明携带代码 (*Proof-Carrying Code*) (Necula 和 Lee, 1996, 1998; Necula, 1997) 的一项关键技术。

译者注：证明携带代码是一种安全执行不受信任代码的机制：代码提供方在交付程序时一并附上机器可检查的安全性证明，表明代码符合接收方预先规定的安全策略，例如类型安全或内存安全；接收方在执行代码前验证这份证明，验证失败便拒绝运行。这里的“证明”并不保证程序在所有方面都正确，其范围仅限于既定的安全策略。

与此同时，安全领域发展出的许多基本思想正在程序语言语境中重新得到研究，并且常表现为类型分析 (例如 Abadi、Banerjee、Heintze 和 Riecke, 1999; Abadi, 1999; Leroy 和 Rouaix, 1998 等)。反过来，人们也越来越有兴趣把程序语言理论直接应用于安全领域的问题 (例如 Abadi, 1999; Sumii 和 Pierce, 2001)。

除编译器外，许多程序分析工具也使用类型检查和类型推断算法。例如，AnnoDomini 是一个面向 Cobol 程序的“千年虫”转换工具，其基础是 ML 风格的类型推断引擎 (Eidorff 等, 1999)。类型推断技术还用于别名分析 (*alias analysis*) 工具 (O’Callahan 和 Jackson, 1997) 与异常分析 (*exception analysis*) 工具 (Leroy 和 Pessaux, 2000)。

在自动定理证明中，人们使用类型系统来表示逻辑命题和证明。这类类型系统通常建立在依赖类型之上，具有很强的表达能力。若干流行的证明助理 (*proof assistant*)，包括 Nuprl (Constable 等, 1986)、Lego (Luo 和 Pollack, 1992; Pollack, 1994)、Coq (Barras 等, 1997) 和 Alf (Magnusson 和 Nordström, 1994)，都直接以类型论为基础。Constable (1998) 和 Pfenning (1999) 讨论了这些系统的历史。

随着以文档类型定义 (*Document Type Definition*) (XML, 1998) 和其他描述 XML 结构化数据的模式——例如当时新出现的 XML Schema 标准 (XS, 2000)——为形式的“Web

元数据”迅速增长，数据库领域对类型系统的兴趣也在不断上升。用于查询和操作 XML 的新语言，直接以这些模式语言为基础，提供功能强大的静态类型系统（Hosoya 和 Pierce, 2000；Hosoya、Vouillon 和 Pierce, 2001；Hosoya 和 Pierce, 2001；Relax, 2000；Shields, 2001）。

译者注：文档类型定义通常简称 DTD，是用来描述一类 XML 文档允许采用何种结构的语法。它可以规定文档中有哪些元素和属性、元素如何嵌套，以及某类元素可以或必须出现多少次；XML 文档若符合相应的 DTD，便可据此通过结构有效性检查。

类型系统还有一种截然不同的应用，出现在计算语言学领域：带类型 lambda 演算构成了范畴语法（*categorical grammar*）等形式体系的基础（van Benthem, 1995；van Benthem 和 Meulen, 1997；Ranta, 1995 等）。

译者注：范畴语法是一类把语法范畴当作类型处理的形式语法：每个词语被指派一个或多个范畴，某些复合范畴表示“接受某类表达式作为参数，并产生另一类表达式”的函数；各成分再依照函数应用等组合规则形成短语和句子。这种类型与函数相结合的结构，解释了它为何可以建立在带类型 lambda 演算之上。

1.3 类型系统与语言设计

给一门最初设计时没有考虑类型检查的语言事后加装类型系统，可能十分棘手。理想情况下，语言设计应当与类型系统设计同步进行。

原因之一是，没有类型系统的语言——即使是安全的动态检查语言——往往会提供一些使类型检查困难甚至不可行的特性，或鼓励人们采用这样的程序设计习惯。事实上，在带类型的语言中，类型系统本身常被当作设计的基础和组织原则，语言设计的其他各个方面都要在它的统摄下加以考量。

另一个原因是，带类型语言的具体语法通常更复杂，因为语法还必须容纳类型标注。若能从一开始就把语言构造与类型标注的写法一并设计，就更容易得到简洁而易懂的语法。

主张类型应当成为程序语言不可分割的一部分，并不意味着程序员必须亲手写出所有类型标注；哪些标注应当显式给出，哪些可以由编译器推断，是另一个问题。设计良好的静态类型语言绝不会要求程序员显式而繁琐地维护海量类型信息。不过，对于多少显式类型信息才算过多，人们仍有分歧。ML 家族语言的设计者努力把标注压到最低限度，用类型推断方法恢复必要信息。包括 Java 在内的 C 家族语言，则选择了稍为繁复的风格。

1.4 简史

计算机科学中最早的类型系统只用来对数值的整数表示和浮点表示作非常简单的区分，例如 Fortran 中的类型系统。20 世纪 50 年代末到 60 年代初，这种分类扩展到结构化数据（记录数组等）和高阶函数。到了 70 年代，人们又引入了若干更丰富的概念，包括参数多态、抽象数据类型、模块系统和子类型化；类型系统也逐渐成为一个独立的研究领域。与此同时，计算机科学家开始认识到程序语言中的类型系统与数理逻辑所研究的类型系统之间存在联系，由此形成了一直延续至今的丰富互动。

图1-1仅择要列出了计算机科学类型系统发展史上的一些重要节点，难免挂一漏万。表中还以斜体列出逻辑学的相关进展，以显示该领域所作贡献的重要性。右栏的引文均可在参考文献中找到。

年代	进展	参考文献
1870 年代	形式逻辑的起源	Frege (1879)
1900 年代	数学的形式化	Whitehead 和 Russell (1910)
1930 年代	无类型 λ 演算	Church (1941)
1940 年代	简单类型 λ 演算	Church (1940); Curry 和 Feys (1958)
1950 年代	Fortran	Backus (1981)
	Algol-60	Naur 等 (1963)
1960 年代	<i>Automath</i> 项目	de Bruijn (1980)
	Simula	Birtwistle 等 (1979)
	Curry-Howard 对应	Howard (1980)
	Algol-68	Van Wijngaarden 等 (1975)
1970 年代	Pascal	Wirth (1971)
	<i>Martin-Löf</i> 类型论	Martin-Löf (1973, 1982)
	<i>System F</i> 、 F_ω	Girard (1972)
	多态 λ 演算	Reynolds (1974)
	CLU	Liskov 等 (1981)
	多态类型推断	Milner (1978); Damas 和 Milner (1982)
	ML	Gordon、Milner 和 Wadsworth (1979)
	交类型	Coppo 和 Dezani (1978)
		Coppo、Dezani 和 Sallé (1979); Pottinger (1980)
1980 年代	NuPRL 项目	Constable 等 (1986)
	子类型化	Reynolds (1980); Cardelli (1984); Mitchell (1984a)
	以存在类型表示抽象数据类型	Mitchell 和 Plotkin (1988)
	构造演算	Coquand (1985); Coquand 和 Huet (1988)
	线性逻辑	Girard (1987); Girard 等 (1989)
	有界量化	Cardelli 和 Wegner (1985)
	爱丁堡逻辑框架	Curien 和 Ghelli (1992); Cardelli 等 (1994)
	Forsythe	Harper、Honsell 和 Plotkin (1992)
	纯类型系统	Reynolds (1988)
		Terlouw (1989); Berardi (1988); Barendregt (1991)
	依赖类型与模块化	Burstall 和 Lampson (1984); MacQueen (1986)
	Quest	Cardelli (1991)
	效应系统 (<i>effect system</i>)	Gifford 等 (1987); Talpin 和 Jouvelot (1992)
	行变量 (<i>row variable</i>); 可扩展记录	Wand (1987); Rémy (1989)
		Cardelli 和 Mitchell (1991)
1990 年代	高阶子类型化	Cardelli (1990); Cardelli 和 Longo (1991)
	带类型的中间语言	Tarditi、Morrisett 等 (1996)
	对象演算	Abadi 和 Cardelli (1996)
	半透明类型 (<i>translucent type</i>) 与模块化	Harper 和 Lillibridge (1994); Leroy (1994)
	类型化汇编语言	Morrisett 等 (1998)

图 1-1: 计算机科学与逻辑学中的类型简史

1.5 延伸阅读

本书虽力求自成体系，却远非面面俱到。这个领域过于庞大，切入角度又太多，不可能靠一本书作出公允而充分的介绍。本节列出另外一些很好的入门途径。

Cardelli (1996) 和 Mitchell (1990b) 的手册文章可以帮助读者快速入门。Barendregt 的文章 (1992) 更适合数学兴趣较强的读者。Mitchell 篇幅宏大的教材 *Foundations for Programming Languages* (1996) 涵盖 lambda 演算基础、多种类型系统和语义学的许多方面，重点在语义问题而非实现问题。Reynolds 的 *Theories of Programming Languages* (1998b) 是一本面向研究生的程序语言理论综述，其中对多态、子类型化和交类型作了优美的阐述。Schmidt 的 *The Structure of Typed Programming Languages* (1994) 在语言设计的语境中展开类型系统的核心概念，其中有数章讨论传统的命令式语言。Hindley 的专著 *Basic Simple Type Theory* (1997) 汇集了简单类型 lambda 演算及其近缘系统的大量精彩结果；它追求的是深度，而不是广度。

Abadi 和 Cardelli 的 *A Theory of Objects* (1996) 展开了许多与本书相同的材料，但弱化实现方面的内容，转而集中讨论如何把这些思想用于面向对象程序设计的基础性处理。Kim Bruce 的 *Foundations of Object-Oriented Languages: Types and Semantics* (2002) 覆盖相近的内容。Palsberg 和 Schwartzbach (1994) 以及 Castagna (1997) 的著作中也有面向对象类型系统的入门材料。

译者注：此处“这些思想”具体指前文哪些内容，原文没有交代清楚；作者也在勘误中说明了这一问题。

Gunter (1992)、Winskel (1993) 和 Mitchell (1996) 的教材深入讨论了无类型语言和带类型语言的语义基础。Hennessy (1990) 还详细介绍了操作语义。许多资料从范畴论这一数学框架出发，讨论类型的语义基础，其中包括 Jacobs (1999)、Asperti 和 Longo (1991) 以及 Crole (1994) 的著作；Pierce 的 *Basic Category Theory for Computer Scientists* (1991a) 则提供了一篇简短导论。

Girard、Lafont 和 Taylor 的 *Proofs and Types* (1989) 讨论类型系统的逻辑层面，例如 Curry-Howard 对应。书中还收录了 System *F* 的创立者本人对该系统的介绍，以及一篇引入线性逻辑的附录。Pfenning 的 *Computation and Deduction* (2001) 进一步探讨了类型与逻辑之间的联系。Thompson 的 *Type Theory and Functional Programming* (1991) 和 Turner 的 *Constructive Foundations for Functional Languages* (1991) 从逻辑角度考察函数式程序设计——这里指 Haskell 或 Miranda 意义上的“纯函数式程序设计”——与构造性类型论之间的联系。Goubault-Larrecq 和 Mackie 的 *Proof Theory and Automated Deduction* (1997) 展开了证明论中若干相关主题。Constable (1998)、Wadler (2000)、Huet (1990) 和 Pfenning (1999) 的文章，Laan 的博士论文 (1997)，以及 Grattan-Guinness (2001) 和 Sommaruga (2000) 的著作，更详细地叙述了类型在逻辑学与哲学中的历史。

事实证明，要宣称一门程序语言具有类型可靠性，必须先进行大量细致分析；否则，这种断言很可能后来被证明是错误的，令人尴尬。也正因如此，类型系统的分类、描述与研究逐渐发展为一门形式化学科。

It turns out that a fair amount of careful analysis is required to avoid false and embarrassing claims of type soundness for programming languages. As a consequence, the classification, description, and study of type systems has emerged as a formal discipline.

——Luca Cardelli, 1996

第二章 数学预备知识

在正式开始之前，我们需要约定一些共同的记法，并陈述若干基本的数学事实。大多数读者只需浏览本章，在需要时回来查阅即可。

2.1 集合、关系与函数

定义 2.1.1. 集合采用标准记法：用花括号明确列出集合的元素 ($\{\dots\}$)，或通过描述法 (*set comprehension*) 从一个集合构造另一个集合 ($\{x \in S \mid \dots\}$)；空集记作 $\emptyset$ ； $S \setminus T$ 表示 S 与 T 的集合差，即 S 中不属于 T 的元素所成的集合。集合 S 的大小记作 $|S|$ 。 S 的幂集 (*powerset*)，即 S 的所有子集所成的集合，记作 $\mathcal{P}(S)$ 。

定义 2.1.2. 自然数集合 $\{0, 1, 2, 3, 4, 5, \dots\}$ 用符号 $\mathbb{N}$ 表示。若一个集合的元素可以与自然数建立一一对应，则称该集合可数 (*countable*)。

定义 2.1.3. 给定一组集合 $S_1, S_2, \dots, S_n$ ，其上的一个 n 元关系 (*n-place relation*) 是由 S_1 到 S_n 中的元素构成的元组集合

$$R \subseteq S_1 \times S_2 \times \dots \times S_n$$

若 $(s_1, \dots, s_n) \in R$ ，其中 $s_i \in S_i$ ，就说 $s_1, \dots, s_n$ 由 R 相关联。

定义 2.1.4. 集合 S 上的一元关系称为 S 上的谓词 (*predicate*)。若 $s \in P$ ，就说谓词 P 对 $s \in S$ 成立。为了突出这一直观，我们常把 $s \in P$ 写作 $P(s)$ ，将 P 看作把 S 中的元素映射为真值的函数。

定义 2.1.5. 集合 S 与 T 上的二元关系 R 称为二元关系 (*binary relation*)。我们常把 $(s, t) \in R$ 写作 $s R t$ 。当 S 与 T 是同一个集合 U 时，就说 R 是 U 上的二元关系。

定义 2.1.6. 为了便于阅读，三元及更多元的关系常使用混合缀具体语法 (*mixfix concrete syntax*) 来书写：关系中的各个元素由一串符号分隔，这些符号合在一起构成关系的名称。例如，对于第 9 章简单类型 lambda 演算中的类型关系，我们写 $\Gamma \vdash s : T$ ，表示“三元组 (Γ, s, T) 属于该类型关系”。

定义 2.1.7. 设 R 是集合 S 与 T 上的关系。 R 的定义域 (*domain*) 记作 $\text{dom}(R)$ ，它由所有满足如下条件的 $s \in S$ 构成：存在某个 t ，使 $(s, t) \in R$ 。 R 的值域 (*range*) 记作 $\text{range}(R)$ ，它由所有满足如下条件的 $t \in T$ 构成：存在某个 s ，使 $(s, t) \in R$ 。

译者注：原书此处写作“陪域或值域” (*codomain or range*)；作者勘误指出，其中 *codomain* 一词使用不当。通常把 T 称为陪域 (*codomain*)，而把这里定义的子集称为值域。译文据此采用“值域”。

定义 2.1.8. 若集合 S 与 T 上的关系 R 满足：只要 $(s, t_1) \in R$ 且 $(s, t_2) \in R$ ，就有 $t_1 = t_2$ ，则称 R 是从 S 到 T 的偏函数 (*partial function*)。如果还满足 $\text{dom}(R) = S$ ，则称 R 是从 S 到 T 的全函数 (*total function*)，或简称函数。

定义 2.1.9. 设 R 是从 S 到 T 的偏函数。若 $s \in \text{dom}(R)$ ，则称 R 在参数 $s \in S$ 上有定义，否则称其无定义。我们写 $f(x) \uparrow$ 或 $f(x) = \uparrow$ ，表示“ f 在 x 上无定义”；写 $f(x) \downarrow$ ，表示“ f 在 x 上有定义”。

在一些实现章节中，我们还需要定义可能在某些输入上失败的函数（例如图 22-2）。必须区分失败 (*failure*) 与发散 (*divergence*)：失败是正当且可观察的结果；一个可能失败的函数既可以是偏函数（即它还可能发散），也可以是全函数（即它总会返回结果或明确失败）——事实上，我们常常希望证明它是全函数。当 f 在输入 x 上返回失败结果时，记作 $f(x) = \text{fail}$ 。

形式上，一个从 S 到 T 、同时还可能失败的函数，实际上是从 S 到 $T \cup \{\text{fail}\}$ 的函数；这里假定 $\text{fail} \notin T$ 。

定义 2.1.10. 设 R 是集合 S 上的二元关系， P 是 S 上的谓词。若由 $s R s'$ 和 $P(s)$ 总能推出 $P(s')$ ，则称 P 被 R 保持 (*preserved*)。

2.2 有序集

定义 2.2.1. 设 R 是集合 S 上的二元关系。若 R 把 S 中每个元素都与自身相关联，即对所有 $s \in S$ 都有 $s R s$ （或 $(s, s) \in R$ ），则称 R 是自反的 (*reflexive*)。若对 S 中任意 s, t ， $s R t$ 都蕴含 $t R s$ ，则称 R 是对称的 (*symmetric*)。若 $s R t$ 和 $t R u$ 一起蕴含 $s R u$ ，则称 R 是传递的 (*transitive*)。若 $s R t$ 和 $t R s$ 一起蕴含 $s = t$ ，则称 R 是反对称的 (*antisymmetric*)。

定义 2.2.2. 集合 S 上自反且传递的关系 R 称为 S 上的预序 (*preorder*)。（当我们说“预序集 S ”时，总是默认心中有 S 上的某个特定预序 R 。）预序通常用 $\leq$ 或 $\sqsubseteq$ 之类的符号表示；偏序和全序是预序的特殊情形，因此也沿用这类记号。我们写 $s < t$ （“ s 严格小于 t ”），表示 $s \leq t \wedge s \neq t$ 。

集合 S 上同时还是反对称的预序称为 S 上的偏序 (*partial order*)。若偏序 $\leq$ 还满足： S 中任意两个元素都可以比较，即对任意 $s, t \in S$ ，必有 $s \leq t$ 或 $t \leq s$ ，则称其为全序 (*total order*)。换言之，全序是任意两个元素都可比较的偏序；每个全序都是偏序，反之则不一定。

定义 2.2.3. 设 $\leq$ 是集合 S 上的偏序，且 $s, t \in S$ 。如果元素 $j \in S$ 满足：

1. $s \leq j$ 且 $t \leq j$ ；
2. 对任意满足 $s \leq k$ 且 $t \leq k$ 的 $k \in S$ ，都有 $j \leq k$ ，

则称 j 是 s 与 t 的并 (*join*)，也就是最小上界 (*least upper bound*)。

类似地，如果元素 $m \in S$ 满足：

1. $m \leq s$ 且 $m \leq t$;
2. 对任意满足 $n \leq s$ 且 $n \leq t$ 的 $n \in S$, 都有 $n \leq m$,

则称 m 是 s 与 t 的交 (*meet*), 也就是最大下界 (*greatest lower bound*)。

定义 2.2.4. 集合 S 上自反、传递且对称的关系称为 S 上的 等价关系 (*equivalence*)。

定义 2.2.5. 设 R 是集合 S 上的二元关系。 R 的 自反闭包 (*reflexive closure*) 是包含 R 的最小自反关系 R' 。“最小”的意思是: 如果 R'' 是另一个包含 R 中所有有序对的自反关系, 那么 $R' \subseteq R''$ 。

类似地, R 的传递闭包 (*transitive closure*) 是包含 R 的最小传递关系, 常记作 R^+ 。 R 的 自反传递闭包 (*reflexive and transitive closure*) 是包含 R 的最小自反且传递的关系, 常记作 R^* 。

练习 2.2.6 ($\star\star, \rightarrow$). 设 R 是集合 S 上的关系。定义关系

$$R' = R \cup \{(s, s) \mid s \in S\}$$

也就是说, R' 包含 R 中的所有有序对, 外加所有形如 (s, s) 的有序对。证明 R' 是 R 的自反闭包。

[查看练习 2.2.6 的译者参考解答](#)

练习 2.2.7 ($\star\star, \rightarrow$). 下面给出关系 R 的传递闭包的一种更具构造性的定义。先定义以下关系序列:

$$\begin{aligned} R_0 &= R, \\ R_{i+1} &= R_i \cup \{(s, u) \mid \text{存在 } t, (s, t) \in R_i \text{ 且 } (t, u) \in R_i\} \end{aligned}$$

也就是说, 构造 R_{i+1} 时, 把由 R_i 中已有有序对经过“一步传递性”得到的全部有序对加入 R_i 。最后, 把关系 R^+ 定义为所有 R_i 的并:

$$R^+ = \bigcup_i R_i$$

证明这个 R^+ 确实是 R 的传递闭包, 即它满足 [定义2.2.5](#) 给出的条件。

[查看练习 2.2.7 的译者参考解答](#)

练习 2.2.8 ($\star\star, \rightarrow$). 设 R 是集合 S 上的二元关系, P 是 S 上被 R 保持的谓词。证明 P 也被 R^* 保持。

[查看练习 2.2.8 的译者参考解答](#)

定义 2.2.9. 设集合 S 上有预序 $\leq$ 。 $\leq$ 中的一条 下降链 (*decreasing chain*) 是 S 中元素构成的序列 $s_1, s_2, s_3, \dots$, 其中每个元素都严格小于前一个元素, 即对每个 i 都有 $s_{i+1} < s_i$ 。(链可以是有限的, 也可以是无限的; 我们更关心下一定义所涉及的无限链。)

定义 2.2.10. 设集合 S 上有预序 $\leq$ 。若 $\leq$ 中不存在无限下降链，则称 $\leq$ 是良基的 (*well founded*)。例如，自然数上按数值大小比较的标准次序

$$0 < 1 < 2 < 3 < \dots$$

是良基的；而整数上按数值大小比较的标准次序

$$\dots < -3 < -2 < -1 < 0 < 1 < 2 < 3 < \dots$$

不是良基的。有时我们省略对 $\leq$ 的明确说明，直接说 S 是良基集。

2.3 序列

定义 2.3.1. 书写序列时，依次列出其元素，并用逗号分隔。我们既用逗号表示在序列任意一端添加一个元素的 *cons* 操作 (*cons operation*)，也用它表示序列的拼接操作 (*append operation*)。例如，若 a 是序列 $3, 2, 1$ ， b 是序列 $5, 6$ ，那么 $0, a$ 表示 $0, 3, 2, 1$ ， $a, 0$ 表示 $3, 2, 1, 0$ ，而 b, a 表示 $5, 6, 3, 2, 1$ 。(只要不讨论由序列构成的序列，用逗号同时表示 *cons* 和拼接就不会造成混淆。)

从 1 到 n 的数列简写作 $1..n$ ，只使用两个点。序列 a 的长度记作 $|a|$ 。空序列写作 $\bullet$ ，也可以留空。若一个序列与另一个序列包含完全相同的元素，只是次序可能不同，则称前者是后者的排列 (*permutation*)。

2.4 归纳法

在程序语言理论乃至整个计算机科学中，归纳证明都随处可见。许多这样的证明建立在以下原则之一上。

公理 2.4.1 (自然数上的普通归纳原理)。设 P 是自然数上的谓词。如果

$$P(0)$$

并且对所有 i ， $P(i)$ 都蕴含 $P(i+1)$ ，那么对所有 n ， $P(n)$ 都成立。

公理 2.4.2 (自然数上的完全归纳原理)。设 P 是自然数上的谓词。如果对每个自然数 n ，在假定所有 $i < n$ 都满足 $P(i)$ 的前提下，可以证明 $P(n)$ ，那么对所有 n ， $P(n)$ 都成立。

定义 2.4.3. 自然数对上的字典序 (*lexicographic order*，也称 *dictionary order*) 定义如下： $(m, n) \leq (m', n')$ 当且仅当 $m < m'$ ，或者 $m = m'$ 且 $n \leq n'$ 。

公理 2.4.4 (字典序归纳原理)。设 P 是自然数对上的谓词。如果对每个自然数对 (m, n) ，在假定所有 $(m', n') < (m, n)$ 都满足 $P(m', n')$ 的前提下，可以证明 $P(m, n)$ ，那么 $P(m, n)$ 对所有 m, n 都成立。

字典序归纳原理是嵌套归纳 (*nested induction*) 证明的基础; 在这类证明中, 归纳证明的某个情形会 “再作一次内层归纳”。它可以推广到数的三元组、四元组等。(对数对归纳相当常见; 对三元组归纳偶尔有用; 超过三元组就很少见了。)

第 3 章的定理 3.3.4 还会介绍另一种归纳证明形式, 称为结构归纳 (*structural induction*), 尤其适合证明与项或类型推导之类树状结构有关的性质。第 21 章将更详细地讨论归纳推理的数学基础; 在那里我们会看到, 这些具体的归纳原理其实都是同一个更深层思想的实例。

2.5 背景读物

如果读者不熟悉本章概述的材料, 可以先阅读一些背景资料。相关资料很多; Winskel 的著作 (1993) 特别适合用来建立对归纳法的直观理解。Davey 和 Priestley (1990) 开篇对有序集作了很好的回顾。Halmos (1987) 则是基础集合论的优秀入门读物。

证明是一项可以重复进行的说服实验。

A proof is a repeatable experiment in persuasion.

——*Jim Horning*

第一部分

无类型系统

第三章 无类型算术表达式

为了严谨地讨论类型系统及其性质，我们首先需要形式化地处理程序语言的一些更基础方面。尤其是，我们需要清晰、精确而且便于数学处理的工具，用来表达程序的语法与语义，并对它们进行推理。

本章和下一章将为一种由数字和布尔值构成的小语言建立这些工具。这个语言简单得几乎不值一提，但它很适合用来直接引入几个基本概念：抽象语法、归纳定义与证明、求值，以及运行时错误的建模。第 5–7 章会在强大得多的无类型 lambda 演算上展开同一套内容；到那时，我们还必须处理名称绑定与替换。再往后，第 8 章将回到本章的简单语言，以它为例正式开始研究类型系统，并引入静态类型的基本概念；第 9 章再把这些概念扩展到 lambda 演算。

3.1 引言

本章使用的语言只有少数几种句法形式：布尔常量 `true` 和 `false`、条件表达式、数值常量 0、算术运算符 `succ`（后继）和 `pred`（前驱），以及测试操作 `iszero`；后者应用于 0 时返回 `true`，应用于其他数字时返回 `false`。下面的文法对这些形式作了紧凑概括。¹

$t ::=$	<code>true</code>	项：常量真
	<code>false</code>	常量假
	<code>if t then t else t</code>	条件表达式
	<code>0</code>	常量零
	<code>succ t</code>	后继
	<code>pred t</code>	前驱
	<code>iszero t</code>	零测试

这套文法所采用的约定（全书也将沿用）与标准 BNF 很接近（参见 Aho、Sethi 和 Ullman, 1986）。第一行的 $t ::=$ 声明我们正在定义项的集合，并用字母 t 指代任意项。其后每一行给出项的一种候选句法形式。符号 t 每出现一次，都可以换成任意项；右侧的斜体短语只是注释。

文法规则右侧的符号 t 称为元变量（*metavariable*）。说它是“变量”，是因为它充当某个具体项的占位符；说它是“元”变量，是因为它不是对象语言（*object language*）——也就

¹本章研究的是无类型布尔值与自然数演算（见图3-2）。相应的 OCaml 实现名为 `arith`，将在第 4 章介绍。有关下载并构建这一检查器的说明，见作者网站 <https://www.cis.upenn.edu/~bcpierce/tapl/>。

是我们正在描述其语法的这个简单程序语言——中的变量，而属于元语言 (*metalanguage*)，即我们用来给出描述的记法。(事实上，目前的对象语言里根本没有变量；第 5 章才会引入变量。) 前缀“元”来自元数学 (*metamathematics*)，这是逻辑学的一个分支，研究数学推理和逻辑推理系统 (包括程序语言) 的数学性质。由此还得到术语元理论 (*metatheory*)：它既指我们能够就某个逻辑系统 (或程序语言) 作出的真命题的集合，也进一步指对这些命题的研究。因此，本书所谓“子类型化的元理论”，可以理解为“对带有子类型化的系统之性质所作的形式研究”。

全书都用元变量 t ，以及邻近的字母 s, u, r 和 t_1, s' 等变体，表示当前讨论的对象语言中的项。随着讨论展开，我们还会引入其他字母，表示其他句法范畴中的表达式。附录 B 汇总了全部元变量约定。

目前，“项”和“表达式”两个词可以互换使用。从第 8 章开始，我们会讨论带有类型等额外句法范畴的演算；届时，“表达式”将泛指各种句法短语 (包括项表达式、类型表达式、种类表达式等)，而“项”将专门指表示计算的短语，也就是可以替换元变量 t 的短语。

为使示例简洁，我们还使用通常的阿拉伯数字；在形式表示中，数字是对 0 嵌套应用 `succ` 得到的。例如，`succ (succ (succ 0))` 简写作 3。

在当前语言中，程序就是一个由上述文法形式构成的项。下面是几个程序及其求值结果：

```
if false then 0 else 1;      ► 1
iszero (pred (succ 0));     ► true
```

全书都用符号 ► 展示示例的求值结果。(若结果显而易见或并不重要，则为了简洁而省略。) 原书排版时会用与当前形式系统对应的实现自动处理示例；这里的实现是 `arith`，显示的响应就是实现的实际输出。

在示例中，为便于阅读，`succ`、`pred` 和 `iszero` 的复合参数都放在括号里。² 项的文法只定义抽象语法，并未提到括号。当然，在眼下这个极其简单的语言里，有无括号区别不大：括号通常用于消除文法歧义，而这个文法不存在歧义——每个词法单元序列至多能解析成一个项。第 5 章的“抽象语法与具体语法”一节会回到括号与抽象语法的话题。

求值结果总具有格外简单的形式：它们不是布尔常量，就是数字，即对 0 嵌套零次或多次 `succ` 得到的项。这些项称为值 (*value*)。后面用形式化规则规定项的求值顺序时，值将用来判断一个子项是否已经求值完成，并据此决定下一步对哪一部分求值。

注意，项的语法允许构造出 `succ true` 和 `if 0 then 0 else 0` 这样看起来可疑的项。稍后我们还会继续讨论它们。事实上，从某种意义上说，正是这些项让这个微型语言值得研究：它们代表了我们希望类型系统排除的那些无意义程序。

3.2 语法

定义这一语言的语法有几种等价方式。前一节的文法已经给出了一种。实际上，该文法只是下面这个归纳定义的紧凑记法。

²从抽象语法看，本章的简单语言即使省略这些括号也不会产生歧义。不过，为使示例的写法与后面采用相似函数应用语法的演算保持一致，原书网站提供的 `arith` 实现仍规定复合参数必须加括号。因此，这些括号是该工具所采用的具体语法约定，并非消除歧义所必需。

定义 3.2.1 (项: 归纳定义). 项的集合是满足以下条件的最小集合 $\mathcal{T}$:

1. $\{\text{true}, \text{false}, 0\} \subseteq \mathcal{T}$;
2. 若 $t_1 \in \mathcal{T}$, 则 $\{\text{succ } t_1, \text{pred } t_1, \text{iszero } t_1\} \subseteq \mathcal{T}$;
3. 若 $t_1, t_2, t_3 \in \mathcal{T}$, 则 $\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \in \mathcal{T}$.

归纳定义在程序语言研究中无处不在, 因此值得停下来仔细考察这个定义。第一项告诉我们三个属于 $\mathcal{T}$ 的简单表达式。第二项和第三项给出规则, 用来判断某些复合表达式属于 $\mathcal{T}$ 。最后, “最小”一词说明, $\mathcal{T}$ 不含这三项所要求之外的其他元素。

与上一节的文法一样, 这一定义也没有说明如何用括号标记复合子项。严格地说, 我们真正做的是把 $\mathcal{T}$ 定义为树的集合, 而不是字符串的集合。例如, 在上一节的 $\text{iszero}(\text{pred}(\text{succ } 0))$ 中, 括号表明 $\text{succ } 0$ 是 pred 的参数, 而整个 $\text{pred}(\text{succ } 0)$ 又是 iszero 的参数。一般来说, 项在书中写成一行, 而其抽象语法实际上是树形结构; 示例中的括号正是用来说明二者如何对应。

同一归纳定义还可以采用逻辑系统“自然演绎式”表述中常见的二维推导规则格式:

定义 3.2.2 (项: 由推导规则定义). 项的集合由下列规则定义:

$$\begin{array}{c}
 \text{true} \in \mathcal{T} \quad \text{false} \in \mathcal{T} \quad 0 \in \mathcal{T} \\
 \\
 \frac{t_1 \in \mathcal{T}}{\text{succ } t_1 \in \mathcal{T}} \quad \frac{t_1 \in \mathcal{T}}{\text{pred } t_1 \in \mathcal{T}} \quad \frac{t_1 \in \mathcal{T}}{\text{iszero } t_1 \in \mathcal{T}} \\
 \\
 \frac{t_1 \in \mathcal{T} \quad t_2 \in \mathcal{T} \quad t_3 \in \mathcal{T}}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \in \mathcal{T}}
 \end{array}$$

前三条规则重述定义3.2.1的第一项; 后四条则刻画第 2、3 项。每条规则都读作: “如果我们已经建立横线上方列出的前提, 就可以导出横线下方的结论。” 此处没有明说 $\mathcal{T}$ 是满足这些规则的最小集合; 采用推导规则给出定义时, 通常会省略这一点。

这里有两个术语值得说明。第一, 不带前提的规则(如上面的前三条)常称为公理 (*axiom*)。本书把推导规则 (*inference rule*) 用作统称, 既包括公理, 也包括带一个或多个前提的“真正规则”。公理上方无物可放, 所以通常不画横线。第二, 如果一定要完全严谨, 我们所称的“推导规则”其实是规则模式 (*rule schema*), 因为它们的前提和结论中可能含有元变量。形式上, 每个模式代表无穷多条具体规则: 把每个元变量一致地换成相应句法范畴中的所有短语, 就得到这些规则。例如, 在上面的规则中, 要把每个 t 换成每一个可能的项。

最后, 再以略显“具体”的风格给出同一项集的另一定义; 它明确提供生成 $\mathcal{T}$ 中元素的过程。

定义 3.2.3 (项: 具体定义). 对每个自然数 i , 定义集合 S_i :

$$\begin{aligned}
 S_0 &= \emptyset, \\
 S_{i+1} &= \{\text{true}, \text{false}, 0\} \\
 &\quad \cup \{\text{succ } t_1, \text{pred } t_1, \text{iszero } t_1 \mid t_1 \in S_i\} \\
 &\quad \cup \{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \mid t_1, t_2, t_3 \in S_i\}
 \end{aligned}$$

最后令

$$S = \bigcup_i S_i$$

S_0 为空; S_1 只含常量; S_2 含常量, 以及只用一次 `succ`、`pred`、`iszero` 或 `if` 与常量构成的短语; S_3 还含对 S_2 中短语使用上述运算所能构成的全部短语; 依此类推。 S 汇集了以常量为起点、使用有限次算术和条件运算能够构成的所有短语。

练习 3.2.4 (★★). S_3 有多少个元素?

[查看原书附录 A 中的 3.2.4 解答](#)

练习 3.2.5 (★★). 证明集合 S_i 是累积的, 即对每个 i 都有 $S_i \subseteq S_{i+1}$ 。

[查看原书附录 A 中的 3.2.5 解答](#)

上述定义从不同方向刻画了同一个项集: [定义3.2.1](#)和 [定义3.2.2](#)只是把它描述为满足某些“闭包性质”的最小集合; [定义3.2.3](#)则展示如何把这个集合实际构造为一个序列的极限。

作为本节的收尾, 我们来验证这两种观点确实定义了同一个集合。下面把证明写得相当详细, 以展示各个部分如何配合。

命题 3.2.6. $\mathcal{T} = S$ 。

证明. $\mathcal{T}$ 被定义为满足某些条件的最小集合。因此, 只需证明: (a) S 满足这些条件; (b) 任何满足这些条件的集合都以 S 为子集, 即 S 的确是满足这些条件的最小集合。

先看 (a)。需要检查[定义3.2.1](#)中的三个条件对 S 都成立。首先, 因为 $S_1 = \{\text{true}, \text{false}, 0\}$, 常量显然都在 S 中。其次, 若 $t_1 \in S$, 则由 $S = \bigcup_i S_i$ 可知, 必有某个 i 使 $t_1 \in S_i$ 。根据 S_{i+1} 的定义, $\text{succ } t_1 \in S_{i+1}$, 所以 $\text{succ } t_1 \in S$; 同理, $\text{pred } t_1 \in S$ 且 $\text{iszero } t_1 \in S$ 。第三, 若 $t_1, t_2, t_3 \in S$, 通过类似论证可得 $\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \in S$ 。

再看 (b)。假设某个集合 S' 满足[定义3.2.1](#)的三个条件。我们对 i 作完全归纳, 证明每个 $S_i \subseteq S'$, 由此显然得到 $S \subseteq S'$ 。

假设对所有 $j < i$ 都有 $S_j \subseteq S'$; 需要证明 $S_i \subseteq S'$ 。 S_i 的定义分为 $i = 0$ 与 $i > 0$ 两种情形。如果 $i = 0$, 则 $S_i = \emptyset$, 显然 $\emptyset \subseteq S'$ 。否则 $i = j + 1$ 。任取 $t \in S_{j+1}$ 。由于 S_{j+1} 被定义为三个较小集合的并, t 必来自其中之一, 有三种可能:

1. 若 t 是常量, 则由条件 1 有 $t \in S'$ 。
2. 若 t 形如 $\text{succ } t_1$ 、 $\text{pred } t_1$ 或 $\text{iszero } t_1$, 其中 $t_1 \in S_j$, 则由归纳假设 $t_1 \in S'$, 再由条件 2 得 $t \in S'$ 。
3. 若 t 形如 $\text{if } t_1 \text{ then } t_2 \text{ else } t_3$, 其中 $t_1, t_2, t_3 \in S_j$, 则由归纳假设三者都在 S' 中, 再由条件 3 得 $t \in S'$ 。

所以每个 $S_i \subseteq S'$ 。根据 S 是所有 S_i 之并的定义, 得到 $S \subseteq S'$, 证明完成。 $\square$

值得注意的是，这个证明对自然数采用完全归纳，而不是更熟悉的“基础情形 / 归纳情形”形式。对于每个 i ，我们假设所需谓词对所有严格小于 i 的数都成立，再证明它对 i 也成立。实质上，这里的每一步都是归纳步骤；当 $i = 0$ 时，不存在满足 $j < i$ 的自然数 j ，因此这一步不需要使用任何归纳假设。这正是 $i = 0$ 与其他归纳步骤唯一不同的地方。全书大多数归纳证明——尤其是“结构归纳”证明——也都适用这一说明。

3.3 项上的归纳

命题3.2.6对项集 $\mathcal{T}$ 的明确刻画，为我们推理其中的元素提供了一条重要原则。如果 $t \in \mathcal{T}$ ，那么 t 必属于下列三种情形之一：(1) t 是常量；(2) t 形如 $\text{succ } t_1$ 、 $\text{pred } t_1$ 或 $\text{iszero } t_1$ ，其中 t_1 是较小的项；(3) t 形如 $\text{if } t_1 \text{ then } t_2 \text{ else } t_3$ ，其中 t_1, t_2, t_3 是较小的项。我们可以从两方面利用这一观察：既可以归纳地定义项集上的函数，也可以归纳地证明项的性质。下面先给出一个简单的归纳定义，把每个项 t 映射为 t 中使用的常量集合。

定义 3.3.1. 项 t 中出现的常量集合记作 $\text{Consts}(t)$ ，定义如下：

$$\begin{aligned} \text{Consts}(\text{true}) &= \{\text{true}\}, & \text{Consts}(\text{false}) &= \{\text{false}\}, \\ \text{Consts}(0) &= \{0\}, & \text{Consts}(\text{succ } t_1) &= \text{Consts}(t_1), \\ \text{Consts}(\text{pred } t_1) &= \text{Consts}(t_1), & \text{Consts}(\text{iszero } t_1) &= \text{Consts}(t_1), \\ \text{Consts}(\text{if } t_1 \text{ then } t_2 \text{ else } t_3) &= \text{Consts}(t_1) \cup \text{Consts}(t_2) \\ &\quad \cup \text{Consts}(t_3) \end{aligned}$$

项的大小是另一个可以用归纳定义计算的性质。

定义 3.3.2. 项 t 的大小记作 $\text{size}(t)$ ，定义如下：

$$\begin{aligned} \text{size}(\text{true}) &= 1, & \text{size}(\text{false}) &= 1, \\ \text{size}(0) &= 1, & \text{size}(\text{succ } t_1) &= \text{size}(t_1) + 1, \\ \text{size}(\text{pred } t_1) &= \text{size}(t_1) + 1, & \text{size}(\text{iszero } t_1) &= \text{size}(t_1) + 1, \\ \text{size}(\text{if } t_1 \text{ then } t_2 \text{ else } t_3) &= \text{size}(t_1) + \text{size}(t_2) + \text{size}(t_3) + 1 \end{aligned}$$

也就是说， t 的大小就是其抽象语法树中的节点数。

类似地，项 t 的深度记作 $\text{depth}(t)$ ，定义如下：

$$\begin{aligned} \text{depth}(\text{true}) &= 1, & \text{depth}(\text{false}) &= 1, \\ \text{depth}(0) &= 1, & \text{depth}(\text{succ } t_1) &= \text{depth}(t_1) + 1, \\ \text{depth}(\text{pred } t_1) &= \text{depth}(t_1) + 1, & \text{depth}(\text{iszero } t_1) &= \text{depth}(t_1) + 1, \\ \text{depth}(\text{if } t_1 \text{ then } t_2 \text{ else } t_3) &= \max\{\text{depth}(t_1), \text{depth}(t_2), \\ &\quad \text{depth}(t_3)\} + 1 \end{aligned}$$

等价地，按照定义3.2.3，从 S_0 开始逐层构造项时， t 最早出现在哪个集合 S_i 中， $\text{depth}(t)$ 就等于这个最小的层号 i 。例如， $\text{pred}(\text{succ } 0)$ 最早出现在 S_3 中，因此其深度为 3。

下面归纳证明一个简单事实，把项中常量的个数与项的大小联系起来。（性质本身当然显而易见；有意思的是归纳证明的形式，我们之后会反复看到它。）

引理 3.3.3. 项 t 中不同常量的数目不大于 t 的大小，即 $|\text{Consts}(t)| \leq \text{size}(t)$ 。

证明. 对 t 的深度作归纳。假设所需性质对所有小于 t 的项都成立，需要证明它对 t 本身成立。分三种情形。

情形： t 是常量。 立即有 $|\text{Consts}(t)| = |\{t\}| = 1 = \text{size}(t)$ 。

情形： $t = \text{succ } t_1$ 、 $\text{pred } t_1$ 或 $\text{iszero } t_1$ 。 由归纳假设， $|\text{Consts}(t_1)| \leq \text{size}(t_1)$ 。于是

$$|\text{Consts}(t)| = |\text{Consts}(t_1)| \leq \text{size}(t_1) < \text{size}(t)$$

情形： $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$ 。 归纳假设分别给出 $|\text{Consts}(t_i)| \leq \text{size}(t_i)$ ($i = 1, 2, 3$)。于是

$$\begin{aligned} |\text{Consts}(t)| &= |\text{Consts}(t_1) \cup \text{Consts}(t_2) \cup \text{Consts}(t_3)| \\ &\leq |\text{Consts}(t_1)| + |\text{Consts}(t_2)| + |\text{Consts}(t_3)| \\ &\leq \text{size}(t_1) + \text{size}(t_2) + \text{size}(t_3) \\ &< \text{size}(t) \end{aligned}$$

□

上述证明采用的并不只是针对这个引理的特殊技巧，而是一种通用的归纳方法。略去其中关于常量个数和项的大小的具体内容，就可以提炼出下面这条项上的归纳原理。我们顺带再列出两条证明项的性质时常用的类似原则。

定理 3.3.4 (项上的归纳原理). 设 P 是项上的谓词。

对深度归纳。 如果对每个项 s ，在假定所有满足 $\text{depth}(r) < \text{depth}(s)$ 的 r 都有 $P(r)$ 的前提下，可以证明 $P(s)$ ，那么 $P(s)$ 对所有 s 都成立。

对大小归纳。 如果对每个项 s ，在假定所有满足 $\text{size}(r) < \text{size}(s)$ 的 r 都有 $P(r)$ 的前提下，可以证明 $P(s)$ ，那么 $P(s)$ 对所有 s 都成立。

结构归纳。 如果对每个项 s ，在假定 s 的所有直接子项 r 都有 $P(r)$ 的前提下，可以证明 $P(s)$ ，那么 $P(s)$ 对所有 s 都成立。

证明. 留作练习 (★★)。

□

查看原书附录 A 中的 3.3.4 解答

对项的深度或大小作归纳，类似于自然数上的完全归纳（公理2.4.2）。普通结构归纳则对应普通的自然数归纳原理（公理2.4.1）：其中的归纳步骤只要求根据 $P(n)$ 建立 $P(n+1)$ 。

与自然数归纳的不同风格一样，选择哪一种项归纳原理，取决于哪一种能让手头证明的结构更简单；形式上它们可以彼此推导。对于简单证明，对大小、深度还是结构归纳通常差别不大。作为一种行文风格，只要可能，通常都采用结构归纳，因为它直接作用于项，避免绕道自然数。

大多数项上的归纳证明具有相似结构。归纳的每一步都给定一个项 t ，要求在假设所有子项（或所有较小项）都满足 P 的前提下证明 $P(t)$ 。为此，分别考察 t 的每种可能形式（`true`、`false`、条件式、`0` 等），并在每种情形下论证该形式的任意 t 都满足 P 。不同归纳证明之间，只有各个情形的具体论证不同，所以通常省略不变的部分，把证明写成：

证明. 对 t 作归纳。

情形： $t = \text{true}$ 。 ... 证明 $P(\text{true})$...

情形： $t = \text{false}$ 。 ... 证明 $P(\text{false})$...

情形： t 是条件式。 若 $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$ ，利用 $P(t_1), P(t_2), P(t_3)$ 证明 $P(t)$...

其他句法形式同理。

□

对于许多归纳论证（包括[引理3.3.3](#)的证明），甚至没有必要写这么多：在基础情形中，项 t 没有子项， $P(t)$ 立即成立；在归纳情形中，只需对子项应用归纳假设，再以完全显然的方式组合结果，就得到 $P(t)$ 。读者一边查看文法、一边记住归纳假设，现场重建证明，实际上比核对写出的论证更容易。在这种情况下，只写“对 t 作归纳”就是完全可以接受的证明。

3.4 语义风格

严谨地给出语言语法之后，下一步是同样精确地定义项如何求值，也就是语言的语义。形式化语义有三种基本方法。

1. 操作语义 (*operational semantics*) 通过为程序语言定义一台简单的抽象机来规定其行为。这里的“抽象”是说，机器代码直接采用该语言的项，而不是某种底层微处理器指令集。对于简单语言，机器状态就是一个项；机器行为由转移函数定义：对每个状态，或者在项上执行一步化简并给出下一状态，或者宣告机器停机。项 t 的含义可以取为：以 t 为初始状态启动机器后最终到达的状态。³

有时，为同一种语言给出两种或更多种操作语义很有用：有些更抽象，机器状态接近程序员书写的项；另一些则更接近实际解释器或编译器操纵的结构。若能证明这些不同的机器在执行同一个程序时，其行为在某种适当意义下相互对应，那么也就证明了其中较具体的机器正确实现了该语言。

³严格地说，这里描述的是操作语义的小步风格 (*small-step style*)，有时也称结构操作语义 (*structural operational semantics*) (Plotkin, 1981)。[练习3.5.17](#)会介绍另一种大步风格，也称自然语义 (Kahn, 1987)；在这种风格中，抽象机的一次转移就把项求值到最终结果。

2. 指称语义 (*denotational semantics*) 对“含义”采取更抽象的观点：项的含义不再只是一串机器状态，而是数字、函数之类的数学对象。为语言给出指称语义，就是找出一组语义域 (*semantic domain*)，再定义一个解释函数，把项映射到这些域中的元素。寻找适合模拟各种语言特性的语义域，形成了丰富而优美的研究领域——域理论 (*domain theory*)。

指称语义的一项主要优点，是它抽去求值的繁琐细节，突出语言的本质概念。此外，所选语义域的性质可以用来推导强大的程序行为推理定律，例如证明两个程序的行为完全相同，或证明某个程序的行为满足某项规约。最后，从语义域的性质中，往往可以立即看出某些事情（无论可取与否）在该语言中不可能发生。

3. 公理语义 (*axiomatic semantics*) 更直接地处理这些定律：它不先用操作语义或指称语义定义程序行为，再由定义导出定律，而是直接把定律本身作为语言的定义。一个项的含义，就是能够证明它具有的性质。

公理方法的优美之处，在于它把注意力集中到程序推理过程上。正是这一思路为计算机科学带来了不变式等强大观念。

在 20 世纪 60、70 年代，人们通常认为操作语义不如另外两种风格：它适合快速而粗略地定义语言特性，却不够优雅，数学能力也较弱。但到了 80 年代，更抽象的方法开始遇到越来越棘手的技术问题，⁴ 相比之下，操作方法的简单性与灵活性日益显得有吸引力。与此同时，一批研究者取得了新进展：Plotkin 的结构操作语义 (1981)、Kahn 的自然语义 (1987)，以及 Milner 关于 CCS 的工作 (1980、1989、1999)，都提出了更优雅的形式体系，并展示如何在指称语义环境中发展出的许多强大数学技术迁移到操作语境。操作语义由此成为一个活跃的独立研究领域，也常常成为定义程序语言并研究其性质时的首选方法。本书只使用操作语义。

3.5 求值

暂且把数字放到一边，先从布尔表达式的操作语义开始。图3-1汇总了这一定义，下面逐项考察。

B (无类型)		
语法	求值规则 $t \longrightarrow t'$	
$t ::= \text{true}$	项: 常量真	$\text{if true then } t_2 \text{ else } t_3 \longrightarrow t_2$ E-IFTRUE
false	常量假	$\text{if false then } t_2 \text{ else } t_3 \longrightarrow t_3$ E-IFFALSE
$\text{if } t \text{ then } t \text{ else } t$	条件式	$\frac{t_1 \longrightarrow t'_1}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3}$ E-IF
$v ::= \text{true}$	值: 真值	
false	假值	

图 3-1: 布尔值 (B)

⁴指称语义最难对付的问题是非确定性与并发；公理语义最难对付的则是过程。

图3-1左栏的文法定义了两个表达式集合。第一个只是为方便而重复项的语法；第二个定义项的一个子集，称为值，它们是求值可能得到的最终结果。这里的值只有常量 `true` 和 `false`。全书都用元变量 v 表示值。

右栏定义项上的求值关系 (*evaluation relation*)，⁵ 写作 $t \longrightarrow t'$ ，读作“ t 单步求值为 t' ”。直观上，如果抽象机当前处于状态 t ，那么它可以执行一步计算，把状态变为 t' 。这个关系由三条推导规则定义；也可以说是两条公理加一条规则，因为前两条没有前提。

第一条规则 E-IFTRUE 说：若正在求值的项是一个条件式，而且守卫就是常量 `true`，机器便可丢弃条件结构，把 `then` 分支 t_2 留作新的机器状态，即下一个待求值项。同样，E-IFFALSE 说，守卫就是 `false` 的条件式单步求值为 `else` 分支 t_3 。规则名中的 E- 表示它们属于求值关系；其他关系的规则会使用不同前缀。

第三条规则 E-IF 更有意思：若守卫 t_1 求值为 t'_1 ，那么整个条件式 `if  $t_1$  then  $t_2$  else  $t_3$` 求值为 `if  $t'_1$  then  $t_2$  else  $t_3$` 。用抽象机的语言说，如果一台只以 t_1 为状态的机器可以走到 t'_1 ，那么处于 `if  $t_1$  then  $t_2$  else  $t_3$` 状态的机器就可以走到 `if  $t'_1$  then  $t_2$  else  $t_3$` 。

这些规则没有说什么，与它们说了什么同样重要。常量 `true` 和 `false` 不求值为任何东西，因为它们没有出现在任何规则的左侧。此外，也没有规则允许在对 `if` 本身求值之前，先对其 `then` 或 `else` 子表达式求值。例如

`if true then (if false then false else false) else true`

不能求值为 `if true then false else true`。唯一的选择是先用 E-IFTRUE 对外层条件式求值。规则之间的这种配合，为条件式确定了一种特定的求值策略，与常见程序语言中熟悉的次序相同：求值条件式时，必须先求守卫；如果守卫本身也是条件式，又必须先求它的守卫；依此类推。E-IFTRUE 和 E-IFFALSE 告诉我们，当这一过程到达终点、遇到守卫已经求值为布尔值的条件式时该做什么。从某种意义上说，前两条规则完成求值的实际工作，而 E-IF 帮助确定工作发生的位置。为了突出规则性质的差异，常把前两条称为计算规则 (*computation rule*)，把 E-IF 称为同余规则 (*congruence rule*)。

译者注：这里的“同余”与数论中的同余无关。在操作语义中，它表示求值关系与相应的语法上下文相容：如果一个子项能够走一步，那么把它放进该上下文后，整个项也能相应地走一步。以 E-IF 为例，令

$$C[\Box] = \text{if } \Box \text{ then } t_2 \text{ else } t_3,$$

则 $t_1 \longrightarrow t'_1$ 可以推出 $C[t_1] \longrightarrow C[t'_1]$ 。因此，E-IF 本身并不选择求值结果，而是把守卫的一次单步求值传递到整个条件式；直观上可以把它理解为一“上下文传播规则”。

下面把这些直观说法表述得更精确。

定义 3.5.1. 对一条推导规则中的每个元变量分别选取一个项，并在结论与所有前提（如果有）中一致地替换该元变量的全部出现，就得到该规则的一个实例 (*instance*)。

例如

`if true then true else (if false then false else false)  $\longrightarrow$  true`

⁵有些专家更愿意把这个关系称为归约 (*reduction*)，把“求值”一词留给 练习3.5.17中的“大步”变体；后者直接把项映射到最终值。

是 E-IFTRUE 的一个实例： t_2 的两处出现都被替换成 true， t_3 则被替换成

if false then false else false

定义 3.5.2. 在本节中，把式子 $t \rightarrow t'$ 看作有序对 (t, t') 。考察一条规则的任意一个实例：如果它的所有前提所表示的有序对均属于关系 R ，那么其结论所表示的有序对也必须属于 R 。如果这项要求对该规则的每个实例都成立，就说关系 R 满足 (*satisfied*) 这条规则。

译者注：可以暂时把关系 R 想成一张“允许的单步求值表”：若 $(t, t') \in R$ ，这张表就允许项 t 单步求值为 t' 。规则 E-IF 要求：只要表中已经允许 $t_1 \rightarrow t'_1$ ，就还必须允许

if t_1 then t_2 else $t_3 \rightarrow$ if t'_1 then t_2 else t_3

换成有序对的说法就是：只要 $(t_1, t'_1) \in R$ ，上面这个条件式求值步骤所对应的有序对也必须属于 R 。若 $(t_1, t'_1) \notin R$ ，这个规则实例便不对 R 提出进一步要求。E-IFTRUE 和 E-IFFALSE 没有前提，因此它们每个实例的结论都必须出现在这张表中。

定义 3.5.3. 单步求值关系 (*one-step evaluation relation*) $\rightarrow$ 收录 图3-1 三条规则能够推出的全部单步求值判断，并且不收录其他项对。形式地说，它是项上满足这三条规则的最小二元关系。当有序对 (t, t') 属于该关系时，就说判断 (*judgment*) $t \rightarrow t'$ 可导出。

译者注：这里的“最小”按关系所包含的项对多少来比较。仅仅要求一个关系满足三条规则，还不能唯一确定它：例如，包含所有项对的关系也满足这些规则，因为任何规则实例的结论都在其中；但它还会包含 $(\text{true}, \text{false})$ 这样没有求值规则作为根据的项对。取满足规则的最小关系，正是为了排除这些多余项对，只留下规则所要求的单步求值判断。

换句话说，判断 $t \rightarrow t'$ 可导出，当且仅当三条规则能够为它提供根据：它或者是公理 E-IFTRUE、E-IFFALSE 之一的实例，或者是 E-IF 某个实例的结论，而且该实例的前提可导出。要说明一个判断可导出，可以展示一棵推导树 (*derivation tree*)：叶节点标记为 E-IFTRUE 或 E-IFFALSE 的实例，内部节点标记为 E-IF 的实例。例如，为了不让后面的公式超出页面宽度，先作如下简写：

$$\begin{aligned} s &\stackrel{\text{def}}{=} \text{if true then false else false,} \\ t &\stackrel{\text{def}}{=} \text{if } s \text{ then true else true,} \\ u &\stackrel{\text{def}}{=} \text{if false then true else true} \end{aligned}$$

以下推导树证明了判断

if t then false else false $\rightarrow$ if u then false else false

可以由求值规则导出：

$$\frac{\frac{s \rightarrow \text{false} \text{ E-IFTRUE}}{t \rightarrow u} \text{ E-IF}}{\text{if } t \text{ then false else false} \rightarrow \text{if } u \text{ then false else false}} \text{ E-IF}$$

译者注：读推导树时，横线下方是要证明的结论，横线上方是使用右侧规则所需的前提。可以从最下面向上检查：最下面的 E-If 要求先证明 $t \rightarrow u$ ；中间的 E-If 又把这个问题化为证明 $s \rightarrow \text{false}$ ；而根据 s 的定义，最上面的判断直接由 E-IfTrue 得到。

这里推导树有三层，并不表示整项连续求值了三步。根节点仍然只断言一次单步求值。两次 E-If 都是同余规则：它们只是把叶节点中的同一个单步计算 $s \rightarrow \text{false}$ 逐层传递到外面的两个条件式中。推导树的高度表示证明这一次求值需要展开多少层规则，而不是程序执行了多少步。

称这种结构为树或许显得奇怪，因为它没有分支。事实上，用来证明求值判断可导出的推导树总是如此纤细：没有求值规则带有一个以上的前提，所以无法构造出分支推导树。等到考察类型关系等其他归纳定义的关系时，有些规则会有多个前提，届时这个术语就显得自然了。

求值判断 $t \rightarrow t'$ 可导出，当且仅当存在一棵以它为根节点标签的推导树。这个事实经常用于推理求值关系的性质，尤其是它直接导出一种称为 对推导归纳 (*induction on derivations*) 的证明技术。下面的定理展示了这种技术。

定理 3.5.4 (单步求值的确定性). 若 $t \rightarrow t'$ 且 $t \rightarrow t''$ ，则 $t' = t''$ 。

证明. 对 $t \rightarrow t'$ 的推导作归纳。归纳的每一步都假定所需结论对所有更小的推导成立，再对当前推导根部使用的求值规则分情形讨论。（这里不是对求值序列的长度归纳：我们只考察单步求值。也可以说我们对 t 的结构归纳，因为“求值推导”的结构直接跟随被归约项的结构；同样也可以对 $t \rightarrow t''$ 的推导归纳。）

若 $t \rightarrow t'$ 推导的最后一条规则是 E-IfTrue，则我们知道 t 形如 $\text{if } t_1 \text{ then } t_2 \text{ else } t_3$ ，其中 $t_1 = \text{true}$ 。此时， $t \rightarrow t''$ 推导的最后一条规则不可能是 E-IfFalse，因为同一个 t_1 不可能既等于 true 又等于 false 。此外，第二个推导的最后一条规则也不可能是 E-If，因为该规则的前提要求存在某个 t'_1 使 $t_1 \rightarrow t'_1$ ，而我们已经看到 true 不求值为任何东西。因此，第二个推导的最后一条规则只能是 E-IfTrue，立即得到 $t' = t''$ 。

类似地，若 $t \rightarrow t'$ 推导的最后一条规则是 E-IfFalse，则 $t \rightarrow t''$ 推导的最后一条规则也必须相同，结论立即成立。

最后，若 $t \rightarrow t'$ 推导的最后一条规则是 E-If，则规则形式说明 $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$ ，并有某个 t'_1 满足 $t_1 \rightarrow t'_1$ 。由上面相同的理由， $t \rightarrow t''$ 推导的最后一条规则只能也是 E-If；这说明 t 仍形如 $\text{if } t_1 \text{ then } t_2 \text{ else } t_3$ （这一点我们已经知道），并有某个 t''_1 满足 $t_1 \rightarrow t''_1$ 。归纳假设现在适用，因为 $t_1 \rightarrow t'_1$ 与 $t_1 \rightarrow t''_1$ 的推导分别是原来两个推导的子推导，所以 $t'_1 = t''_1$ 。于是

$$t' = \text{if } t'_1 \text{ then } t_2 \text{ else } t_3 = \text{if } t''_1 \text{ then } t_2 \text{ else } t_3 = t'',$$

正是所需结论。 □

练习 3.5.5 (★). 仿照定理3.3.4，明确写出前一证明所用的归纳原理。

查看原书附录 A 中的 3.5.5 解答

单步求值关系展示抽象机在求值项时怎样从一个状态走到下一个状态。但作为程序员，我们同样关心求值的最终结果，即机器无法再迈出一步的状态。

定义 3.5.6. 如果没有求值规则可以应用于项 t ，即不存在 t' 使 $t \longrightarrow t'$ ，则称 t 处于范式 (*normal form*)。 (有时简称 “ t 是一个范式”。)

我们已经观察到，当前系统中的 `true` 和 `false` 都是范式：所有求值规则左侧的最外层构造子都是 `if`，显然无法实例化成 `true` 或 `false`。可以把这一观察重述成关于值的一般事实。

定理 3.5.7. 每个值都处于范式。

以后用算术表达式以及其他构造扩充系统时，我们始终会保证 [定理3.5.7](#) 继续成立。“值”表示求值已经完成的结果，因此处于范式是值的必要性质；若某个语言定义不具备这条性质，它就从根本上出了问题。

在当前系统中，[定理3.5.7](#) 的逆命题也成立：每个范式都是值。一般而言则未必如此。事实上，等到本节稍后讨论算术表达式时，范式但非值的项将在运行时错误分析中扮演关键角色。

定理 3.5.8. 若 t 处于范式，则 t 是值。

证明. 假设 t 不是值。容易对 t 作结构归纳，证明它不处于范式。

因为 t 不是值，所以它必形如 `if  $t_1$  then  $t_2$  else  $t_3$` 。考察 t_1 的可能形式。若 $t_1 = \text{true}$ ，则 t 与 `E-IFTRUE` 左侧匹配，显然不是范式； $t_1 = \text{false}$ 时同理。若 t_1 既非 `true` 又非 `false`，它就不是值。归纳假设说明 t_1 不是范式，即存在 t'_1 使 $t_1 \longrightarrow t'_1$ 。于是可用 `E-IF` 导出

$$t \longrightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3,$$

所以 t 也不是范式。 □

有时把许多步求值视作一次大的状态转移会很方便。为此，定义多步求值关系，把一个项与从它出发经过零次或多次单步求值能够导出的所有项联系起来。

定义 3.5.9. 多步求值关系 (*multi-step evaluation relation*) $\longrightarrow^*$ 是单步求值的自反传递闭包，即满足以下条件的最小关系：

1. 若 $t \longrightarrow t'$ ，则 $t \longrightarrow^* t'$ ；
2. 对所有 t ，都有 $t \longrightarrow^* t$ ；
3. 若 $t \longrightarrow^* t'$ 且 $t' \longrightarrow^* t''$ ，则 $t \longrightarrow^* t''$ 。

练习 3.5.10 (★). 把[定义3.5.9](#)改写成一组推导规则。

[查看原书附录 A 中的 3.5.10 解答](#)

有了明确的多步求值记法，下面这样的事实就很容易陈述。

定理 3.5.11 (范式的唯一性). 若 $t \longrightarrow^* u$ 且 $t \longrightarrow^* u'$ ，并且 u, u' 都处于范式，则 $u = u'$ 。

证明. 它是单步求值确定性 ([定理3.5.4](#)) 的推论。 □

转向算术表达式之前，最后考察求值的另一个性质：每个项都能求值到一个值。显然，在带有递归函数定义等特性的更丰富语言中，这条性质未必成立。即使成立，它的证明通常也比下面的证明微妙得多。第 12 章会重新讨论这一点，展示类型系统如何成为某些语言终止性证明的骨架。

计算机科学中的大多数终止性证明都有同样的基本形式。⁶ 首先选择某个良基集 S ，并给出把“机器状态”（这里就是项）映射到 S 的函数 f 。然后证明，只要机器状态 t 能走一步到 t' ，就有 $f(t') < f(t)$ 。于是，从 t 开始的无限求值序列可以通过 f 映射为 S 中的一条无限下降链。由于 S 良基，不存在这样的无限下降链，因此也不存在无限求值序列。函数 f 常称为求值关系的终止性度量（*termination measure*）。

定理 3.5.12 (求值的终止性). 对每个项 t ，都存在某个范式 t' ，使 $t \rightarrow^* t'$ 。

证明. 注意每次单步求值都会减小项的大小；自然数上的通常次序良基，所以项的大小就是一个终止性度量。□

练习 3.5.13 (推荐, ★★). 1. 假设在图3-1的规则中加入

$$\text{if true then } t_2 \text{ else } t_3 \rightarrow t_3 \quad (\text{E-FUNNY1})$$

上述定理3.5.4、3.5.7、3.5.8、3.5.11和3.5.12中，哪些仍然成立？

2. 换成加入以下规则：

$$\frac{t_2 \rightarrow t'_2}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t_1 \text{ then } t'_2 \text{ else } t_3} \quad (\text{E-FUNNY2})$$

现在上述哪些定理仍然成立？是否有证明需要修改？

查看原书附录 A 中的 3.5.13 解答

下一项工作是把求值定义扩展到算术表达式。图3-2概括了定义中新加的部分。（图右上角的说明提醒我们：该图是图3-1的扩展，而不是独立成篇的语言。）

项的定义仍只是重复第3.1节中的语法。值的定义稍有意思，因为需要引入“数值”这一新句法范畴。直观上，算术表达式的最终求值结果可以是数字：数字要么是 0，要么是某个数字的后继，但不能是任意值的后继；我们希望说 `succ true` 是错误，而不是值。

图3-2右栏的求值规则沿用图3-1中的模式。四条计算规则 E-PREDZERO、E-PREDSUCC、E-ISZEROZERO 和 E-ISZEROSUCC 说明 `pred` 与 `iszero` 应用于数字时的行为；三条同余规则 E-SUCC、E-PRED 与 E-ISZERO 则规定：对这些复合项求值时，应先让其参数 t_1 进行一次单步求值。

译者注：这里的不对称源于 `succ` 与 `pred` 的不同作用。`succ` 用来构造数值：当其参数成为数值 nv 后，`succ nv` 本身已经是数值，无须继续计算；因此 E-SUCC 只负责把求值引向参数。`pred` 则用于观察并拆解数值：它先通过 E-PRED 求值参数，随后还必须根据所得数值是零值还是后继值，分别使用 E-PREDZERO 或 E-PREDSUCC 得出结果。

⁶第 12 章会看到结构略复杂一些的终止性证明。

$\mathcal{NB}$ (无类型)扩展 $\mathcal{B}$ (图 3-1)

新增句法形式

$t ::= \dots$	项
0	常量零
$\text{succ } t$	后继
$\text{pred } t$	前驱
$\text{iszero } t$	零测试
$v ::= \dots$	值
nv	数值
$nv ::= 0$	数值
$\text{succ } nv$	后继值

新增求值规则 $t \longrightarrow t'$

$\frac{t_1 \longrightarrow t'_1}{\text{succ } t_1 \longrightarrow \text{succ } t'_1}$	E-SUCC
$\text{pred } 0 \longrightarrow 0$	E-PREDZERO
$\text{pred } (\text{succ } nv_1) \longrightarrow nv_1$	E-PREDSUCC
$\frac{t_1 \longrightarrow t'_1}{\text{pred } t_1 \longrightarrow \text{pred } t'_1}$	E-PRED
$\text{iszero } 0 \longrightarrow \text{true}$	E-ISZEROZERO
$\text{iszero } (\text{succ } nv_1) \longrightarrow \text{false}$	E-ISZEROSUCC
$\frac{t_1 \longrightarrow t'_1}{\text{iszero } t_1 \longrightarrow \text{iszero } t'_1}$	E-ISZERO

图 3-2: 算术表达式 ($\mathcal{NB}$)

换言之，E-PREDSUCC 拆掉的是由 `succ` 构造出的最外层结构，并不是一条关于 `succ` 自身的求值规则。

严格地说，现在应当重述定义 3.5.3：“算术表达式上的单步求值关系，是满足图 3-1 和图 3-2 全部规则实例的最小关系……”为了避免把篇幅浪费在这种样板文字上，常见做法是把推导规则本身直接看作关系的定义，默认补上“包含所有规则实例的最小关系……”。

数值句法范畴 nv 在这些规则中很重要。例如，在 E-PREDSUCC 中，左侧写的是 `pred (succ  $nv_1$ )`，而不是 `pred (succ  $t_1$ )`。因此，该规则不能把 `pred (succ (pred 0))` 求值成 `pred 0`，因为这要求把元变量 nv_1 实例化为 `pred 0`，而它不是数值。这个项唯一的下一求值步骤具有如下推导树：

$$\frac{\frac{\text{pred } 0 \longrightarrow 0 \text{ E-PREDZERO}}{\text{succ } (\text{pred } 0) \longrightarrow \text{succ } 0} \text{ E-SUCC}}{\text{pred } (\text{succ } (\text{pred } 0)) \longrightarrow \text{pred } (\text{succ } 0)} \text{ E-PRED}$$

练习 3.5.14 (★★). 证明定理 3.5.4 对算术表达式上的求值关系仍然成立：若 $t \longrightarrow t'$ 且 $t \longrightarrow t''$ ，则 $t' = t''$ 。

查看原书附录 A 中的 3.5.14 解答

形式化语言的操作语义，迫使我们规定所有项的行为；在这里包括 `pred 0` 和 `succ false` 这样的项。根据图 3-2 的规则，0 的前驱被定义为 0。另一方面，`false` 的后继没有定义为求值到任何项，也就是说它是范式。我们把这样的项称为卡住 (*stuck*)。

定义 3.5.15. 若项处于范式但不是值，就称它卡住。

“卡住”为这台简单机器提供了一种简单的运行时错误概念。直观上，它刻画这样的情况：程序到达“无意义状态”，操作语义不知道接下来该做什么。在语言更具体的实现中，这些状态可能对应各种机器故障，如段错误、执行非法指令等。这里把所有此类不良行为归并为“卡住状态”这一个概念。

练习 3.5.16 (推荐, ★★). 形式化抽象机无意义状态的另一种方式, 是引入新项 `wrong`, 并为操作语义添加规则: 在当前语义会卡住的所有情形下, 显式产生 `wrong`。具体来说, 引入两个新句法范畴:

$$\begin{array}{lcl} & \text{wrong} & \text{运行时错误} \\ \text{badnat} ::= & \left| \begin{array}{l} \text{true} \quad \text{常量真} \\ \text{false} \quad \text{常量假} \end{array} \right. & \left. \vphantom{\begin{array}{l} \text{true} \\ \text{false} \end{array}} \right\} \text{非数值范式} \\ \text{badbool} ::= & \left| \begin{array}{l} \text{wrong} \quad \text{运行时错误} \\ nv \quad \text{数值} \end{array} \right. & \left. \vphantom{\begin{array}{l} \text{wrong} \\ nv \end{array}} \right\} \text{非布尔范式} \end{array}$$

并为求值关系添加规则:

$$\begin{array}{l} \text{if badbool then } t_1 \text{ else } t_2 \longrightarrow \text{wrong} \quad (\text{E-IF-WRONG}) \\ \text{succ badnat} \longrightarrow \text{wrong} \quad (\text{E-SUCC-WRONG}) \\ \text{pred badnat} \longrightarrow \text{wrong} \quad (\text{E-PRED-WRONG}) \\ \text{iszero badnat} \longrightarrow \text{wrong} \quad (\text{E-ISZERO-WRONG}) \end{array}$$

证明这两种处理运行时错误的方式相互吻合: (1) 先精确说明这里的“相互吻合”究竟是什么意思; (2) 再证明这一精确表述。证明程序语言性质时经常如此: 真正棘手的是准确表述需要证明的命题, 证明本身应当很直接。

[查看原书附录 A 中的 3.5.16 解答](#)

练习 3.5.17 (推荐, ★★). 常用的操作语义有两种风格。本书采用 小步语义 (*small-step semantics*): 求值关系的定义说明怎样用单个计算步骤逐点改写一个项, 直至它最终成为值; 再在其上定义多步求值关系, 讨论项如何经过多步求值为值。另一种风格称为 大步语义 (*big-step semantics*), 有时也称 自然语义 (*natural semantics*); 它直接形式化“这个项求值为那个最终值”这一概念, 写作 $t \Downarrow v$ 。布尔值与算术表达式语言的大步求值规则如下:

$$\begin{array}{c} v \Downarrow v \quad (\text{B-VALUE}) \\[10pt] \frac{t_1 \Downarrow \text{true} \quad t_2 \Downarrow v_2}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow v_2} \quad (\text{B-IFTTRUE}) \qquad \frac{t_1 \Downarrow \text{false} \quad t_3 \Downarrow v_3}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \Downarrow v_3} \quad (\text{B-IFFALSE}) \\[10pt] \frac{t_1 \Downarrow nv_1}{\text{succ } t_1 \Downarrow \text{succ } nv_1} \quad (\text{B-SUCC}) \qquad \frac{t_1 \Downarrow 0}{\text{pred } t_1 \Downarrow 0} \quad (\text{B-PREDZERO}) \\[10pt] \frac{t_1 \Downarrow \text{succ } nv_1}{\text{pred } t_1 \Downarrow nv_1} \quad (\text{B-PREDSUCC}) \\[10pt] \frac{t_1 \Downarrow 0}{\text{iszero } t_1 \Downarrow \text{true}} \quad (\text{B-ISZEROZERO}) \qquad \frac{t_1 \Downarrow \text{succ } nv_1}{\text{iszero } t_1 \Downarrow \text{false}} \quad (\text{B-ISZEROSUCC}) \end{array}$$

证明这个语言的小步语义与大步语义一致, 即

$$t \longrightarrow^* v \iff t \Downarrow v$$

[查看原书附录 A 中的 3.5.17 解答](#)

练习 3.5.18 ($\star\star, \Rightarrow$). 假设要改变语言的求值策略, 使 if 表达式先按顺序求值 then 分支和 else 分支, 再求值守卫。说明应如何修改求值规则才能达到这一效果。

[查看练习 3.5.18 的译者参考解答](#)

3.6 注记

抽象语法、具体语法、解析等思想, 在许多编译器教材中都有介绍。Winskel (1993) 和 Hennessy (1990) 更详细地讨论了归纳定义、推导规则系统与归纳证明。

这里采用的操作语义风格可以追溯到 Plotkin 的技术报告 (1981)。大步风格 ([练习 3.5.17](#)) 由 Kahn (1987) 发展出来。更详细的论述可参见 Astesiano (1991) 和 Hennessy (1990)。

Burstall (1969) 把结构归纳引入了计算机科学。

问: 为什么还要费心证明程序语言的性质? 如果定义正确, 这些证明几乎总是很乏味。

答: 因为定义几乎总是错的。

Q: Why bother doing proofs about programming languages? They are almost always boring if the definitions are right.

A: The definitions are almost always wrong.

——佚名

第四章 算术表达式的 ML 实现

把上一章那样的形式定义与具体实现联系起来，让定义背后的直观“落到实处”，往往会使这些定义更容易处理。本章介绍布尔表达式与算术表达式语言实现中的关键部分。（不打算亲自动手使用后续类型检查器实现的读者，可以跳过本章，以及以后章名中带有“ML 实现”的所有章节。）

本章代码（以及全书各实现章节的代码）采用 ML 家族（Gordon、Milner 和 Wadsworth, 1979）的一种流行语言 Objective Caml，简称 OCaml（Leroy, 2000；Cousineau 和 Mauny, 1998）。这里只用到完整 OCaml 语言的一小部分，因此把示例移植到其他多数语言应该不难。最重要的要求是自动存储管理（垃圾回收），以及能够方便地对结构化数据类型进行模式匹配、定义递归函数。Standard ML（Milner、Tofte、Harper 和 MacQueen, 1997）、Haskell（Hudak 等, 1992；Thompson, 1999），以及带有某种模式匹配扩展的 Scheme（Kelsey、Clinger 和 Rees, 1998；Dybvig, 1996）等函数式语言都很合适。具有垃圾回收却没有模式匹配的语言，如 Java（Arnold 和 Gosling, 1996）和纯 Scheme，对于我们要写的程序略显笨重；既没有垃圾回收也没有模式匹配的语言，如 C（Kernighan 和 Ritchie, 1988），就更不合适。¹

本章代码可以在作者网站的 `arith` 实现中找到，其中也附有下载和构建说明。

译者注：下文完整保留原书的 OCaml 程序，并在每段代码之后直接给出相应的 Rust 对照实现。书中片段来自参与编译和测试的实际 Rust 源码；可运行的完整算术语言工程位于 `code/arith/`，具体工具链、语义覆盖范围、已知差异与验证命令见该目录的 README。为便于复制和运行，两种语言的代码块都保留 ASCII 源码字符。

4.1 语法

第一项工作，是定义一种表示项的 OCaml 值类型。OCaml 的数据类型定义机制使这件事很直接：下面的声明几乎逐字转写了第 3 章开头的项文法。

¹当然，每个人对语言的品味不同，优秀程序员用手边任何工具都能完成工作；读者完全可以使用自己喜欢的语言。不过需要提醒的是：类型检查器所需的符号处理若要手工管理存储，尤其繁琐而且容易出错。

```

type term =
  TmTrue of info
| TmFalse of info
| TmIf of info * term * term * term
| TmZero of info
| TmSucc of info * term
| TmPred of info * term
| TmIsZero of info * term

```

Rust 对照实现 (译者增补)

```

#[derive(Clone, Copy, Debug, Default, PartialEq, Eq)]
pub struct Info {
    pub line: usize,
    pub column: usize,
}

const DUMMY_INFO: Info = Info { line: 0, column: 0 };

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    True(Info),
    False(Info),
    If(Info, Box<Term>, Box<Term>, Box<Term>),
    Zero(Info),
    Succ(Info, Box<Term>),
    Pred(Info, Box<Term>),
    IsZero(Info, Box<Term>),
}

```

从 `TmTrue` 到 `TmIsZero` 的构造子，分别命名 `term` 类型抽象语法树中的不同节点；每个 `of` 后面的类型则说明该类节点会连接多少棵子树。

每个抽象语法树节点还带有一个 `info` 类型的值，记录该节点来自哪个源文件、其中哪个字符位置。解析器扫描输入文件时生成这些信息；打印函数用它指出发生错误的位置。若只想理解求值、类型检查等基本算法，完全可以省略这些信息。这里之所以保留，是为了让希望亲手试验实现的读者看到与书中讨论完全一致的代码形式。

定义求值关系时，需要判断一个项是否为数值：

```
let rec isnumericval t = match t with
  | TmZero(_) -> true
  | TmSucc(_, t1) -> isnumericval t1
  | _ -> false
```

Rust 对照实现 (译者增补)

```
pub fn is_numeric_value(term: &Term) -> bool {
  match term {
    Term::Zero(_) => true,
    Term::Succ(_, inner) => is_numeric_value(inner),
    _ => false,
  }
}
```

这是 OCaml 中由模式匹配定义递归函数的典型例子。`isnumericval` 应用于 `TmZero` 时返回 `true`；应用于带有子树 `t1` 的 `TmSucc` 时递归调用自身，检查 `t1` 是否为数值；应用于其他任意项时返回 `false`。模式中的下划线是“不关心”项，可以匹配对应位置上的任何内容：前两条分支用它忽略 `info` 标注，最后一条则用它匹配任意项。`rec` 关键字告诉编译器这是递归函数定义，也就是说，函数体中的 `isnumericval` 指向正在定义的函数，而不是同名的旧绑定。

注意，书中的 ML 代码在排版时做了少量“美化”，以便阅读，并与 lambda 演算示例保持一致。例如，原书印刷版使用真正的箭头符号，而不是两个字符 `->`。作者网站列出了所有这类排版替换。

判断项是否为值的函数与此类似：

```
let rec isval t = match t with
  | TmTrue(_) -> true
  | TmFalse(_) -> true
  | t when isnumericval t -> true
  | _ -> false
```

Rust 对照实现 (译者增补)

```
pub fn is_value(term: &Term) -> bool {
    matches!(term, Term::True(_) | Term::False(_)) || is_numeric_value(term)
}
```

第三条分支是一个条件模式 (*conditional pattern*): 它可以匹配任意项 t , 但仅当布尔表达式 `isnumericval t` 得到 `true` 时才算匹配成功。

4.2 求值

求值关系的实现紧密跟随图3-1 和图3-2 中的单步求值规则。前面已经看到, 这些规则定义了一个偏函数: 应用于尚未成为值的项时, 它给出该项的单步求值结果; 应用于值时, 求值函数没有结果。把求值规则翻译为 OCaml 时, 需要决定如何处理后一种情况。一种直接办法, 是让单步求值函数 `eval1` 在没有任何求值规则适用于给定项时抛出异常。(另一种同样可行的办法, 是让单步求值器返回 `term option`: 它说明求值是否成功, 若成功还携带结果项; 不过这会增加一点簿记工作。) 先定义无规则可用时抛出的异常:

```
exception NoRuleApplies
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct NoRuleApplies;

pub type StepResult = Result<Term, NoRuleApplies>;
```

现在可以写出单步求值器:

```
let rec eval1 t = match t with
  TmIf(_, TmTrue(_), t2, t3) ->
    t2
  | TmIf(_, TmFalse(_), t2, t3) ->
    t3
  | TmIf(fi, t1, t2, t3) ->
    let t1' = eval1 t1 in
    TmIf(fi, t1', t2, t3)
  | TmSucc(fi, t1) ->
    let t1' = eval1 t1 in
    TmSucc(fi, t1')
  | TmPred(_, TmZero(_)) ->
```



```

    TmZero(dummyinfo)
| TmPred(_, TmSucc(_, nv1)) when (isnumericval nv1) ->
    nv1
| TmPred(fi, t1) ->
    let t1' = eval1 t1 in
    TmPred(fi, t1')
| TmIsZero(_, TmZero(_)) ->
    TmTrue(dummyinfo)
| TmIsZero(_, TmSucc(_, nv1)) when (isnumericval nv1) ->
    TmFalse(dummyinfo)
| TmIsZero(fi, t1) ->
    let t1' = eval1 t1 in
    TmIsZero(fi, t1')
| _ ->
    raise NoRuleApplies

```

Rust 对照实现 (译者增补)

```

pub fn eval1(term: &Term) -> StepResult {
    match term {
        Term::If(_, guard, then_term, _) if matches!(guard.as_ref(),
↪ Term::True(_)) => {
            Ok((*then_term).clone())
        }
        Term::If(_, guard, _, else_term) if matches!(guard.as_ref(),
↪ Term::False(_)) => {
            Ok((*else_term).clone())
        }
        Term::If(info, guard, then_term, else_term) => Ok(Term::If(
            *info,
            Box::new(eval1(guard)?),
            then_term.clone(),
            else_term.clone(),
        )),
        Term::Succ(info, inner) => Ok(Term::Succ(*info, Box::new(eval1(inner)?))),
        Term::Pred(_, inner) if matches!(inner.as_ref(), Term::Zero(_)) => {
            Ok(Term::Zero(DUMMY_INFO))
        }
        Term::Pred(_, inner) => {
            if let Term::Succ(_, numeric) = inner.as_ref()
                && is_numeric_value(numeric)
            {
                return Ok((*numeric).clone());
            }
        }
    }
}

```

```

    Ok(Term::Pred(DUMMY_INFO, Box::new(eval1(inner)?)))
  }
  Term::IsZero(_, inner) if matches!(inner.as_ref(), Term::Zero(_)) => {
    Ok(Term::True(DUMMY_INFO))
  }
  Term::IsZero(_, inner) => {
    if let Term::Succ(_, numeric) = inner.as_ref()
      && is_numeric_value(numeric)
    {
      return Ok(Term::False(DUMMY_INFO));
    }
    Ok(Term::IsZero(DUMMY_INFO, Box::new(eval1(inner)?)))
  }
  Term::True(_) | Term::False(_) | Term::Zero(_) => Err(NoRuleApplies),
}
}

```

其中有几处从零开始构造新项，而不是重新组织已有项。由于这些新项并不存在于用户原始源文件中，它们的 `info` 标注没有用处；常量 `dummyinfo` 就用作这类项的标注。变量名 `fi`（即“file information”）则一贯用来匹配模式中的 `info` 标注。

`eval1` 定义中还有一点值得注意：模式里显式使用 `when` 子句，以刻画图3-1和图3-2的求值关系中 `v`、`nv` 之类元变量名所隐含的限制。例如，在对 `TmPred(_, TmSucc(_, nv1))` 求值的分支中，OCaml 模式语义本来允许 `nv1` 匹配任意项，这不是我们想要的；加上 `when (isnumericval nv1)`，就把规则限制为只有在 `nv1` 实际匹配数值时才能触发。

如果愿意，也可以把原来的推导规则改写成与 ML 模式相同的风格，把元变量命名所隐含的限制变成规则的显式旁条件：

$$\frac{t_1 \text{ 是数值}}{\text{pred}(\text{succ } t_1) \longrightarrow t_1} \quad (\text{E-PREDSUCC}),$$

代价是记法不再那么紧凑易读。

最后，函数 `eval` 接受一个项，通过反复调用 `eval1` 找到其范式。每当 `eval1` 返回新项 `t'`，就递归调用 `eval`，从 `t'` 继续求值。当 `eval1` 最终到达无规则可用的位置时，它抛出 `NoRuleApplies`，使 `eval` 跳出循环并返回序列中的最后一项。²

²为求简单，这里把 `eval` 写成这样；不过，在递归循环里设置 `try` 处理器并不是良好的 ML 风格。

```
let rec eval t =
  try let t' = eval1 t
    in eval t'
  with NoRuleApplies -> t
```

Rust 对照实现 (译者增补)

```
pub fn eval(term: Term) -> Term {
  match eval1(&term) {
    Ok(next) => eval(next),
    Err(NoRuleApplies) => term,
  }
}
```

练习 4.2.1 (**). 为什么这不是良好的风格? 怎样重写 eval 更好?

[查看原书附录 A 中的 4.2.1 解答](#)

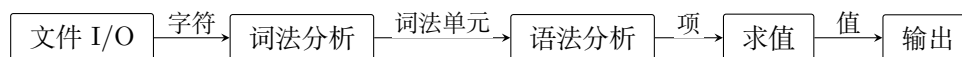
显然, 这个简单求值器的目标是便于与数学上的求值定义逐项比较, 而不是尽快找出范式。从[练习3.5.17](#)的大步求值规则出发, 可以得到效率稍高的算法。

练习 4.2.2 (推荐, ***, $\Rightarrow$). 把 arith 实现中的 eval 函数改写为 [练习3.5.17](#)所介绍的大步风格。

[查看练习 4.2.2 的译者参考解答](#)

4.3 其余部分

当然, 即使是非常简单的解释器或编译器, 除了这里明确讨论的内容以外, 也还有许多组成部分。真正开始求值之前, 项只是文件中的字符序列: 程序必须从文件系统读入字符, 由词法分析器处理成词法单元流, 再由解析器处理成抽象语法树, 之后才能交给前面看到的函数求值。求值结束后, 还需要打印结果。



建议感兴趣的读者查看作者网站上这个完整解释器的 OCaml 代码。

第五章 无类型 lambda 演算

概述

本章回顾无类型 lambda 演算（也称纯 lambda 演算）的定义与若干基本性质。它构成了本书后续大多数类型系统的底层计算基础。

20 世纪 60 年代中期，Peter Landin 指出，可以用以下方式来理解一种复杂的程序语言：先用一个小型核心演算刻画这门语言的本质机制，再加入一组便于使用的派生形式；要理解这些派生形式的行为，只需把它们翻译成核心演算中的形式（Landin, 1964、1965、1966；另见 Tennent, 1981）。Landin 使用的核心语言是 lambda 演算——Alonzo Church 在 20 世纪 20 年代发明的一套形式系统（Church, 1936、1941），其中所有计算都归结为函数定义与函数应用两种基本操作。继 Landin 的洞见以及 John McCarthy 关于 Lisp 的开创性工作（1959、1981）之后，lambda 演算被广泛用于程序语言特性的规约、语言设计与实现，以及类型系统研究。它的重要性来自双重身份：既是一种可以描述计算的简单程序语言，又是一个能够对其作出严格数学论断的数学对象。

用于类似目的的核心演算很多，lambda 演算只是其中之一。Milner、Parrow 和 Walker 的 π 演算（1992、1991）已经成为定义基于消息的并发语言语义的一种常用核心语言；Abadi 和 Cardelli 的对象演算（1996）则提炼了面向对象语言的核心特性。我们为 lambda 演算发展的大多数概念和技术，都能相当直接地迁移到这些演算。第 19 章会展开一个这样的案例研究。¹

扩展 lambda 演算有多种方式。首先，可以为数字、元组、记录等特性增添专门的具体语法，使它们更便于使用；这些特性原本就能在核心语言中模拟。更值得注意的是，还可以加入可变引用单元、非局部异常处理等复杂特性。若要在核心语言中模拟这些特性，就必须经过一套相当复杂的转换。沿着这些方向不断扩展，最终便会得到 ML（Gordon、Milner 和 Wadsworth, 1979；Milner、Tofte 和 Harper, 1990；Weis、Aponte、Laville、Mauny 和 Suárez, 1989；Milner、Tofte、Harper 和 MacQueen, 1997）、Haskell（Hudak 等, 1992）或 Scheme（Sussman 和 Steele, 1975；Kelsey、Clinger 和 Rees, 1998）这样的语言。后面各章会看到，核心语言的扩展往往也伴随着类型系统的扩展。

¹本章示例或者属于纯无类型 lambda 演算 λ （图5-3），或者属于用布尔值与算术运算扩展后的 λ_{NB} （图3-2）。相应的 OCaml 实现是 `fulluntyped`。

5.1 基础

过程抽象 (*procedural abstraction*) (或函数抽象) 是几乎所有程序语言的关键特性。我们不必一遍遍写同一项计算, 而是写一个过程或函数, 用一个或多个具名参数对计算作一般描述, 再按需实例化该函数, 每次为参数提供具体值。例如, 程序员会很自然地把冗长重复的表达式

$$(5 * 4 * 3 * 2 * 1) + (7 * 6 * 5 * 4 * 3 * 2 * 1) - (3 * 2 * 1)$$

改写为

$$factorial(5) + factorial(7) - factorial(3),$$

其中

$$factorial(n) = \text{if } n = 0 \text{ then } 1 \text{ else } n * factorial(n - 1)$$

对每个非负数 n , 用参数 n 实例化函数 $factorial$, 就得到 n 的阶乘。如果用 “ $\lambda n. \dots$ ” 表示 “以 n 为参数并返回……的函数”, 就可以把定义重写成

$$factorial = \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n * factorial(n - 1)$$

于是 $factorial(0)$ 的意思是 “把函数 $\lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } \dots$ 应用于参数 0”; 也就是 “在函数体 $\text{if } n = 0 \text{ then } 1 \text{ else } \dots$ 中, 用 0 替换参数变量 n 后得到的值”; 也就是 $\text{if } 0 = 0 \text{ then } 1 \text{ else } \dots$; 也就是 1。

lambda 演算 (或 λ 演算) 以最纯粹的形式体现这种函数定义与应用。在 lambda 演算中, 一切都是函数: 函数接受的参数本身是函数, 返回的结果仍是函数。

lambda 演算的语法只有三类项。² 变量 x 本身是项; 以 x 为参数、以项 t_1 为函数体构成的 lambda 抽象 $\lambda x. t_1$ 也是项; 把项 t_1 应用于另一个项 t_2 , 写作 $t_1 t_2$, 仍是项。这三种构造方式概括为:

$$\begin{array}{ll} t ::= x & \text{项: 变量} \\ \quad | \lambda x. t & \text{抽象} \\ \quad | t t & \text{应用} \end{array}$$

下面几个小节讨论这个定义中的一些细节。

抽象语法与具体语法

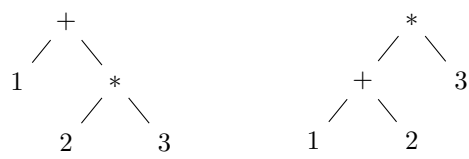
讨论程序语言语法时, 把结构分成两个层次很有用。³ 语言的具体语法 (*concrete syntax*) (或表层语法) 指程序员直接读写的字符串; 抽象语法 (*abstract syntax*) 则是在内部把程序表示为带标签的树, 这种树称为抽象语法树 (*abstract syntax tree, AST*)。树表示使项

² “lambda 项” 泛指 lambda 演算中的任意项; 以 λ 开头的 lambda 项常称为 lambda 抽象。

³ 完整语言的定义有时使用更多层次。例如, 沿用 Landin 的思路, 常把某些语言构造定义为 派生形式 (*derived form*), 通过翻译成其他更基本特性的组合来规定其行为。只包含这些核心特性的受限子语言称为 内部语言 (*internal language, IL*), 包含全部派生形式的完整语言称为 外部语言 (*external language, EL*)。从 EL 到 IL 的变换至少在概念上是解析之后的独立阶段。第 11.3 节讨论派生形式。

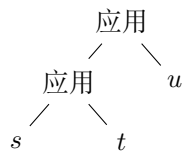
的结构一目了然，因此非常适合严格的语言定义、关于语言的证明，以及编译器和解释器内部的复杂变换。

从具体语法到抽象语法的变换分两步。首先，词法分析器 (*lexical analyzer, lexer*) 把程序员书写的字符串转换成词法单元序列，包括标识符、关键字、常量、标点等。词法分析器会删除注释，处理空白、大小写约定，以及数值和字符串常量的格式。随后，解析器 (*parser*) 把词法单元序列转换成抽象语法树。解析时，运算符优先级和结合性等约定减少了表层程序为明确复合表达式结构而使用括号的需要。例如， $*$ 比 $+$ 结合得更紧，所以解析器把不带括号的 $1 + 2 * 3$ 解释成左边这棵抽象语法树，而不是右边那棵：

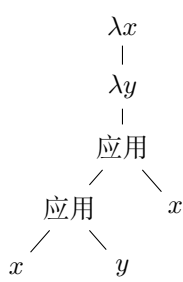


本书关注抽象语法，而不是具体语法。上面的 lambda 项文法应理解为描述合法树结构，而不是词法单元或字符组成的字符串。当然，在示例、定义、定理和证明中，我们需要把项写成从左到右排列的一串符号，也就是具体的线性记法；不过在理解这些符号时，我们始终着眼于它们底层的抽象语法树。

为了用这种写法时少写一些括号，我们采用两项约定。第一，应用向左结合，也就是说， $st u$ 与 $(st) u$ 表示同一棵树：



第二，在没有括号明确截断时，lambda 抽象的函数体包含句点右侧的整个表达式。例如， $\lambda x. \lambda y. x y x$ 与 $\lambda x. (\lambda y. ((x y) x))$ 表示同一棵树：



变量与元变量

上面的语法定义还有一个容易混淆之处：元变量的用法。为了写出适用于任意项的一般性陈述，我们需要使用一些占位符来代表具体的语法对象；这样的占位符称为元变量。和前面一样，本章用 t, s, u （均可带下标）作为元变量，泛指任意项。⁴

⁴在本章中， t 泛指任意 lambda 项，而不再代表前面讨论的算术表达式。本书各章都会用 t 泛指该章所讨论演算中的任意项；每章首页的脚注会说明具体是哪一种演算。

在语法规则和一般性论述中, x (以及 y, z) 也用作元变量, 但它们泛指的不是任意项, 而是任意一个变量名。例如, 若一条一般性规则使用元变量 x , 那么在使用这条规则时, 可以用对象语言中的变量 a, b 或其他任意变量来代替它。

容易混淆的是, 可供使用的简短字母不多, 而在具体的 lambda 项中, 我们同样会用 x, y 等字母作为实际的变量名。不过, 根据上下文总能分清这些字母扮演的是哪一种角色。例如, 我们可以说: “项 $\lambda x. \lambda y. xy$ 具有 $\lambda z. s$ 的形式, 其中 $z = x$, 并且 $s = \lambda y. xy$ 。” 这里, 语法模式 $\lambda z. s$ 中的 z, s 是元变量; 具体项中的 x, y 则是对象语言实际使用的变量名。

作用域

最后还必须说明: lambda 抽象中的参数声明对哪些变量出现有效, 这就是变量的作用域问题。

在 lambda 抽象 $\lambda x. t$ 中, λx 声明参数 x , 函数体是 t 。 λx 称为绑定子 (*binder*), 它的作用域就是函数体 t 。 变量 x 在这个作用域内的出现称为 受约束的 (*bound*); 也就是说, 这些出现被 λx 绑定。 如果 x 的某次出现不在任何相应的 λx 的作用域内, 就称这次出现是自由的 (*free*)。

例如, xy 中的 x 是自由的; 在 $\lambda y. xy$ 中, y 被 λy 绑定, 而 x 仍然自由。 在 $\lambda x. x$ 以及 $\lambda z. \lambda x. \lambda y. x(yz)$ 中, x 都是受约束的。 在 $(\lambda x. x)x$ 中, 函数体里的 x 受约束, 括号右侧作为参数的 x 则是自由的。

没有自由变量的项称为闭项 (*closed term*); 闭项也称为 组合子 (*combinator*)。 最简单的组合子是恒等函数:

$$id = \lambda x. x$$

它什么也不做, 只把参数原样返回。

操作语义

纯 lambda 演算没有内建常量或原始运算符: 没有数字、算术运算、条件式、记录、循环、顺序、I/O 等。 项进行“计算”的唯一方式, 就是把函数应用于参数 (参数本身也是函数)。 每一步计算都改写一个形如 $(\lambda x. t_{12}) t_2$ 的函数应用: 将右侧参数 t_2 代入左侧抽象的函数体 t_{12} , 也就是用 t_2 替换其中由 λx 绑定的 x 。 写作

$$(\lambda x. t_{12}) t_2 \longrightarrow [x \mapsto t_2] t_{12},$$

其中 $[x \mapsto t_2] t_{12}$ 表示“用 t_2 替换 t_{12} 中 x 的所有自由出现后得到的项”。 例如, $(\lambda x. x)y$ 求值为 y , 而 $(\lambda x. x(\lambda x. x))(ur)$ 求值为 $ur(\lambda x. x)$ 。

译者注: 这里的“自由出现”是相对于函数体 t_{12} 本身而言。 在完整的抽象 $\lambda x. t_{12}$ 中, 这些 x 受到外层 λx 的绑定; 进行函数应用时, 外层绑定子被消去, 并由参数 t_2 替换这些出现。 因此, 在 $(\lambda x. x)y$ 中, 函数体 x 里的 x 会被 y 替换。

沿用 Church 的术语, 形如 $(\lambda x. t_{12}) t_2$ 的项称为 可归约式 (*redex, reducible expression*); 按照上述规则改写可归约式的操作称为 β 归约 (*beta-reduction*)。

程序语言设计者与理论研究者多年来研究了 lambda 演算的多种求值策略。每种策略都规定一个项中的哪些可归约式能够在下一步触发。⁵

- 在完全 β 归约 (*full beta-reduction*) 下, 任何可归约式随时都可归约。每一步从当前项中任意位置选取一个可归约式并归约。例如

$$(\lambda x. x) ((\lambda x. x) (\lambda z. (\lambda x. x) z))$$

可以更易读地写成 $id(id(\lambda z. id z))$, 其中含有三个可归约式; 分别标出如下:

$$\begin{aligned} & id(id(\lambda z. id z)) \\ & id(id(\lambda z. id z)) \\ & id(id(\lambda z. id z)) \end{aligned}$$

完全 β 归约可以先选最内层, 再选中间, 最后选最外层:

$$\begin{aligned} id(id(\lambda z. id z)) &\longrightarrow id(id(\lambda z. z)) \\ id(id(\lambda z. z)) &\longrightarrow id(\lambda z. z) \\ id(\lambda z. z) &\longrightarrow \lambda z. z \not\rightarrow \end{aligned}$$

译者注: 求值序列末尾的 $\not\rightarrow$ 表示: 按照当前求值策略, 左侧的项已经无法再迈出归约一步。后面各求值序列末尾的同一记号含义相同; 它并不表示该项在其他求值策略下也一定不能继续归约。

- 在正规序策略 (*normal-order strategy*) 下, 总是先归约最左、最外层的可归约式。上面的项会按如下方式归约:

$$\begin{aligned} id(id(\lambda z. id z)) &\longrightarrow id(\lambda z. id z) \\ id(\lambda z. id z) &\longrightarrow \lambda z. id z \\ \lambda z. id z &\longrightarrow \lambda z. z \not\rightarrow \end{aligned}$$

在这种策略以及下面几种策略下, 求值关系实际上都是偏函数: 每个项 t 至多能单步求值为一个 t' 。

- 按名调用 (*call-by-name*) 策略限制更强, 不允许在抽象内部归约。从同一个项出发, 前两步与正规序相同, 但会在最后一步之前停止, 把 $\lambda z. id z$ 视为范式:

$$\begin{aligned} id(id(\lambda z. id z)) &\longrightarrow id(\lambda z. id z) \\ id(\lambda z. id z) &\longrightarrow \lambda z. id z \not\rightarrow \end{aligned}$$

一些著名程序语言采用了按名调用的变体, 尤其是 Algol-60 (Naur 等, 1963) 和 Haskell (Hudak 等, 1992)。Haskell 实际采用称为 按需调用 (*call-by-need*) 的优化版本 (Wadsworth, 1971; Ariola 等, 1995): 与每次用到参数时都重新求值不同, 按需

⁵有人把“归约”和“求值”当作同义词; 也有人只把涉及某种“值”概念的策略称为求值, 其他策略则称为归约。

调用只在第一次真正需要参数时对它求值，并用所得的值替换该参数的所有出现，从而避免以后的重复计算。该策略要求项的运行时表示保留一定程度的共享；实际上，它是抽象语法图而不是抽象语法树上的归约关系。

- 大多数语言使用按值调用策略。与按名调用一样，这种策略不进入 lambda 抽象的函数体归约，只归约抽象之外的最外层可归约式。而且，只有当可归约式的参数——也就是应用右侧的项——已经归约为值时，才归约该可归约式。在这里对闭程序求值的非正式讨论中，值是已经完成计算、按当前策略不能再继续归约的闭项。⁶

译者注：官方勘误在这里把“项”修正为“闭项”，同时提醒读者：后续形式定义中的值不一定是闭项。例如， $\lambda x. y$ 含有自由变量 y ，却仍符合“lambda 抽象是值”的形式定义。因此，“闭项”只限定这里对闭程序求值的非正式说明；值的正式范围以后续语法定义为准。

上例按此策略归约为：

$$\begin{aligned} id(id(\lambda z. id z)) &\longrightarrow id(\lambda z. id z) \\ id(\lambda z. id z) &\longrightarrow \lambda z. id z \not\rightarrow \end{aligned}$$

按值调用是严格的 (*strict*)：函数参数无论是否会在函数体中使用，都总要先求值。相反，按名调用与按需调用等非严格 (*non-strict*) (或惰性) 策略，只求值实际使用的参数。

讨论类型系统时，选择哪种求值策略其实差别不大。各种类型特性的动机，以及处理这些动机的技术，在各策略下大致相同。本书采用按值调用，一方面因为多数著名语言都采用它，另一方面因为它最容易扩展引用 (第 13 章)、异常 (第 14 章) 等特性。

5.2 用 lambda 演算编程

lambda 演算的定义虽然极其精简，其表达能力却远比表面看来强大。本节将介绍若干使用 lambda 演算编程的典型示例。这些示例并不是要说明 lambda 演算本身应当被视为一门成熟的程序语言；对于同样的任务，所有得到广泛使用的高级语言都有更清晰、更高效的实现方式。它们只是一些热身练习，用来帮助读者熟悉这个系统。

多个参数

首先注意，lambda 演算没有内建的多参数函数。当然，加入这种特性并不难，但利用返回函数的高阶函数实现同样效果更容易。假设项 s 含有两个自由变量 x, y ，我们想写一个函数 f ：对每对参数 (v, w) ，它都返回在 s 中以 v 替换 x 、以 w 替换 y 的结果。在拥有更多内建造的程序语言中，我们也许会写成 $f = \lambda(x, y). s$ ；而在这里则写成

$$f = \lambda x. \lambda y. s$$

⁶在这个最精简的演算中，只有 lambda 抽象是值。更丰富的演算还会有其他值，如数值常量与布尔常量、字符串、值的元组、值的记录、值的列表等。

也就是说, 给 f 一个作为 x 值的 v , 它返回另一个函数; 再给该函数一个作为 y 值的 w , 就得到所需结果。参数逐个应用, 写作 $f v w$, 即 $(f v) w$ 。它先归约为 $(\lambda y. [x \mapsto v] s) w$, 再归约为 $[y \mapsto w][x \mapsto v] s$ 。这种把多参数函数变换为高阶函数的做法, 为纪念与 Church 同时代的 Haskell Curry, 称为 柯里化 (*currying*)。

Church 布尔值

布尔值与条件式也很容易在 lambda 演算中编码。定义

$$tru = \lambda t. \lambda f. t,$$

$$fls = \lambda t. \lambda f. f$$

(使用缩写名, 是为了避免与第 3 章的原始布尔常量 `true`、`false` 混淆。)

tru 与 fls 可以视为表示布尔值“真”和“假”: 它们可以执行测试布尔值真假的操作。具体来说, 可以用应用定义组合子 $test$, 使 $test b v w$ 在 b 为 tru 时归约为 v , 在 b 为 fls 时归约为 w :

$$test = \lambda l. \lambda m. \lambda n. l m n$$

$test$ 实际上没有做多少事: $test b v w$ 只是归约为 $b v w$ 。布尔值 b 自己就是条件式: 它接受两个参数, 并在自身为 tru 时选择第一个, 为 fls 时选择第二个。例如:

$$\begin{aligned} test\ tru\ v\ w &= (\lambda l. \lambda m. \lambda n. l m n)\ tru\ v\ w && \text{按定义} \\ &\longrightarrow (\lambda m. \lambda n. tru\ m\ n)\ v\ w && \text{归约带下划线的可归约式} \\ &\longrightarrow (\lambda n. tru\ v\ n)\ w && \text{归约带下划线的可归约式} \\ &\longrightarrow tru\ v\ w && \text{归约带下划线的可归约式} \\ &= (\lambda t. \lambda f. t)\ v\ w && \text{按定义} \\ &\longrightarrow (\lambda f. v)\ w && \text{归约带下划线的可归约式} \\ &\longrightarrow v && \text{归约带下划线的可归约式} \end{aligned}$$

逻辑合取等布尔运算也可以定义为函数:

$$and = \lambda b. \lambda c. b c fls$$

也就是说, 给 and 两个布尔值 b, c : 若 b 是 tru , 返回 c ; 若 b 是 fls , 返回 fls 。所以仅当二者都是 tru 时, $and\ b\ c$ 才得到 tru :

$$\begin{aligned} and\ tru\ tru &\blacktriangleright (\lambda t. \lambda f. t) \\ and\ tru\ fls &\blacktriangleright (\lambda t. \lambda f. f) \end{aligned}$$

练习 5.2.1 (★). 定义逻辑或函数和逻辑非函数。

查看原书附录 A 中的 5.2.1 解答

译者注：读到这里，可以看出 Church 编码最关键的思想：一个数据不一定要由某种内建结构或标签表示，也可以由它被使用时表现出的行为来表示。这里的 *tru* 与 *fls* 不是等待条件式检查的两个被动标签；它们本身就是选择器，分别从两个参数中选择第一个或第二个。后面的序对与 Church 数也遵循同一思路：序对通过把两个成分交给使用者来表现自己，自然数 n 则通过把某个操作重复 n 次来表现自己。

Church 最初发展 lambda 演算，并不是为了设计程序语言，而是为了研究函数的一般性质，并尝试以函数为基础建立形式逻辑体系。在这种观点下，函数首先是一种规则或操作，可以接收函数、返回函数；计算则归结为函数抽象、函数应用和替换。Church 最初设想的完整无类型逻辑体系未能保持一致，但其中纯粹的无类型 lambda 演算保留了下来，并在 Church、Kleene、Rosser 等人的工作中逐渐显露出与一般可计算性之间的深刻联系。参见 Church (1936、1941)。

序对

利用布尔值，可以把值的序对编码为项：

$$\begin{aligned} pair &= \lambda f. \lambda s. \lambda b. b f s, \\ fst &= \lambda p. p \text{ tru}, \\ snd &= \lambda p. p \text{ fls} \end{aligned}$$

也就是说， $pair\ v\ w$ 是一个函数；把它应用于布尔值 b ，就把 b 应用于 v, w 。根据布尔值的定义，当 b 是 *tru* 时得到 v ，为 *fls* 时得到 w ，所以只需提供相应布尔值，就能实现第一、第二投影 *fst*、*snd*。验证 $fst(pair\ v\ w) \longrightarrow^* v$ ：

$$\begin{aligned} fst(pair\ v\ w) &= fst((\lambda f. \lambda s. \lambda b. b f s)\ v\ w) && \text{按定义} \\ &\longrightarrow fst((\lambda s. \lambda b. b\ v\ s)\ w) && \text{归约带下划线的可归约式} \\ &\longrightarrow fst(\lambda b. b\ v\ w) && \text{归约带下划线的可归约式} \\ &= (\lambda p. p\ \text{tru})(\lambda b. b\ v\ w) && \text{按定义} \\ &\longrightarrow (\lambda b. b\ v\ w)\ \text{tru} && \text{归约带下划线的可归约式} \\ &\longrightarrow \text{tru}\ v\ w && \text{归约带下划线的可归约式} \\ &\longrightarrow^* v && \text{同前} \end{aligned}$$

Church 数

用 lambda 项表示数字，只比前面的编码稍微复杂一些。定义 Church 数 $c_0, c_1, c_2, \dots$ ：

$$\begin{aligned} c_0 &= \lambda s. \lambda z. z, \\ c_1 &= \lambda s. \lambda z. s\ z, \\ c_2 &= \lambda s. \lambda z. s\ (s\ z), \\ c_3 &= \lambda s. \lambda z. s\ (s\ (s\ z)), \\ &\vdots \end{aligned}$$

数字 n 由组合子 c_n 表示；它接受两个参数 s, z （分别代表“后继”和“零”），把 s 对 z 应用 n 次。和布尔值、序对一样，这种编码也通过行为来表示数据：数字 n 本身就是一个把给定

操作执行 n 次的函数。可以把它看作一种“主动式”的一进制表示 (*unary numeral*): 普通的一进制表示用 n 个重复记号表示 n , 而这里的函数会主动执行 n 次操作。

读者或许已经发现, c_0 和 fls 实际是同一个项。在汇编语言中也常见这种“双关”: 同一个位模式根据解释方式不同, 可以表示整数、浮点数、地址或四个字符等; 低级语言 C 同样把 0 与假等同起来。

Church 数的后继函数可以定义为:

$$scc = \lambda n. \lambda s. \lambda z. s (n s z)$$

scc 是接受 Church 数 n 并返回另一个 Church 数的组合子, 也就是返回一个接受 s, z 并把 s 反复应用于 z 的函数。先把 s, z 传给 n , 再把 s 显式多应用一次, 就得到正确的应用次数。

练习 5.2.2 (★★). 换一种方式定义 Church 数的后继函数。

[查看原书附录 A 中的 5.2.2 解答](#)

类似地, Church 数加法由项 $plus$ 完成。它接受两个 Church 数 m, n , 返回另一个 Church 数; 返回的函数接受 s, z , 先让 n 把 s 对 z 迭代 n 次, 再让 m 把 s 多迭代 m 次:

$$plus = \lambda m. \lambda n. \lambda s. \lambda z. m s (n s z)$$

乘法的定义使用了另一个技巧。函数 $plus$ 依次接受两个参数, 因此先把 n 传给它, 便得到一个新函数 $plus n$; 这个函数把任意 Church 数 k 变为 $n + k$ 。接下来, 以 c_0 为初始值, 让 Church 数 m 把这个“加 n ”的函数重复应用 m 次。结果就是从零开始连续加上 m 个 n , 也就是 $m \times n$:

$$times = \lambda m. \lambda n. m (plus n) c_0$$

练习 5.2.3 (★★). 能否不借助 $plus$ 定义 Church 数乘法?

[查看原书附录 A 中的 5.2.3 解答](#)

练习 5.2.4 (推荐, ★★). 定义一个计算一个数的另一个数次幂的项。

[查看原书附录 A 中的 5.2.4 解答](#)

Church 数 m 按定义接受迭代函数和初始值两个参数。为了利用这一结构判断 m 是否为零, 我们需要为这两个参数分别选取具体值 ss 和 zz , 使 $m ss zz$ 在 $m = c_0$ 时得到 tru , 在 $m > 0$ 时得到 fls 。当 $m = c_0$ 时, ss 一次也不会应用, 结果就是初始值 zz , 所以取 $zz = tru$ 。当 $m > 0$ 时, ss 至少会应用一次, 因此可将 ss 定义为一个忽略参数、始终返回 fls 的函数:

$$iszro = \lambda m. m (\lambda x. fls) tru$$

$$iszro c_1 \quad \blacktriangleright \quad (\lambda t. \lambda f. f)$$

$$iszro (times c_0 c_2) \quad \blacktriangleright \quad (\lambda t. \lambda f. t)$$

令人意外的是，Church 数减法比加法困难得多。下面这个颇有技巧的“前驱函数”可以完成它：参数为 c_0 时返回 c_0 ，参数为 c_{i+1} 时返回 c_i ：

$$\begin{aligned} zz &= \text{pair } c_0 \ c_0, \\ ss &= \lambda p. \text{pair}(\text{snd } p) (\text{plus } c_1 (\text{snd } p)), \\ prd &= \lambda m. \text{fst}(m \ ss \ zz) \end{aligned}$$

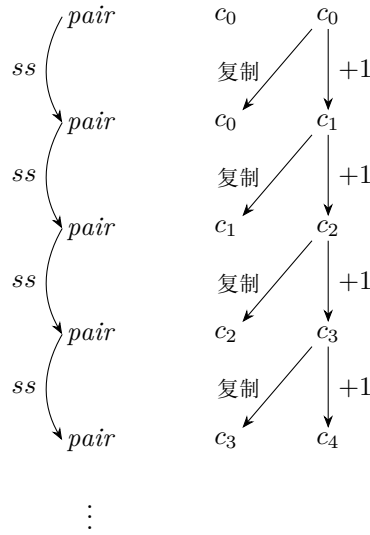


图 5-1: 前驱函数的“内层循环”

该定义把 m 当作函数,从初始值 zz 出发应用 m 份 ss 。每份 ss 都接受一个数对 $\text{pair } c_i \ c_j$, 返回 $\text{pair } c_j \ c_{j+1}$ (见图5-1)。所以把 ss 对 $\text{pair } c_0 \ c_0$ 应用 m 次: 当 $m = 0$ 时得到 $\text{pair } c_0 \ c_0$; 当 $m > 0$ 时得到 $\text{pair } c_{m-1} \ c_m$ 。两种情况下, m 的前驱都在第一分量。

练习 5.2.5 (★★). 用 prd 定义减法函数。

查看原书附录 A 中的 5.2.5 解答

练习 5.2.6 (★★). 计算 $prd \ c_n$ 大约需要多少步求值? 请给出关于 n 的函数。

查看原书附录 A 中的 5.2.6 解答

练习 5.2.7 (★★). 写一个函数 $equal$, 测试两个数是否相等并返回 Church 布尔值。例如:

$$\begin{aligned} equal \ c_3 \ c_3 &\blacktriangleright (\lambda t. \lambda f. t) \\ equal \ c_3 \ c_2 &\blacktriangleright (\lambda t. \lambda f. f) \end{aligned}$$

查看原书附录 A 中的 5.2.7 解答

列表、树、数组、变体和记录等其他常见数据类型,也可以用类似技术编码。

练习 5.2.8 (推荐, ★★★). 列表可以在 lambda 演算中编码成一个执行 右折叠 (*right fold*) 的函数 (OCaml 将其称为 `fold_right`, 有时也称为 `reduce`)。对列表 $[x, y, z]$ 做右折叠时, 需要提供一个二元函数 c 和一个初始值 n , 结果为

$$\text{fold_right } c \ [x, y, z] \ n = c \ x \ (c \ y \ (c \ z \ n))$$

因此, 可以把列表 $[x, y, z]$ 本身编码为

$$\lambda c. \lambda n. c \ x \ (c \ y \ (c \ z \ n))$$

空表 nil 应如何编码? 写一个函数 $cons$: 它接受元素 h 和一个已经采用上述方式编码的列表 t , 返回把 h 添加到 t 开头后所得列表的同类编码。再写 $isnil$ 和 $head$, 它们各接受一个列表参数。最后, 为这种列表编码写出 $tail$ 函数; 这要难得多, 需要使用类似数值前驱函数 prd 的技巧。

[查看原书附录 A 中的 5.2.8 解答](#)

扩充演算

我们已经看到, 布尔值、数字及其运算都能在纯 lambda 演算中编码。严格说来, 所有编程工作都可以不离开纯系统。不过, 处理示例时, 加入原始布尔值和数字 (也许还有其他数据类型) 往往更方便。需要明确所用系统时, 我们用 λ 表示图5-3定义的纯 lambda 演算, 用 λ_{NB} 表示加入图3-1 和图3-2 的布尔与算术表达式后的系统。

在 λ_{NB} 中, 写程序时有两套布尔值实现和两套数字实现可供选择: 原始值与本章的编码。它们之间很容易相互转换。要把 Church 布尔值变成原始布尔值, 只需把它应用于 `true, false`:

$$\text{realbool} = \lambda b. b \ \text{true} \ \text{false}$$

反方向则使用 `if` 表达式:

$$\text{churchbool} = \lambda b. \text{if } b \ \text{then } \text{tru} \ \text{else } \text{fls}$$

这些转换还可以嵌入更高层运算。例如, 下面的 Church 数相等函数返回原始布尔值:

$$\text{realeq} = \lambda m. \lambda n. (\text{equal } m \ n) \ \text{true} \ \text{false}$$

类似地, 可以定义一个转换函数 realnat , 把 Church 数变成对应的原始数字。Church 数 m 会从给定的初始值开始, 把给定函数重复应用它所表示的次数; 因此, 只要以原始数字 0 为初始值, 并让每一步都执行原始的后继操作, 最终就会得到由相应层数的 `succ` 包围 0 的原始数字:

$$\text{realnat} = \lambda m. m \ (\lambda x. \text{succ } x) \ 0$$

这里必须把后继操作写成函数 $\lambda x. \text{succ } x$ 。按照算术表达式的语法, 只有 `succ t` 才是合法的项; 单独的 `succ` 不是可以作为参数传递的函数。因此, 不能直接写 $m \ \text{succ } 0$, 而要先用 lambda 抽象把后继操作包装成一个真正的函数。

原始布尔值和数字之所以方便示例，主要与求值次序有关。例如考察 $scc\ c_1$ 。按照前文，可能以为它会求值为 Church 数 c_2 ，实际却得到：

$$scc\ c_1 \quad \blacktriangleright \quad \lambda s. \lambda z. s\ ((\lambda s'. \lambda z'. s'\ z')\ s\ z)$$

该项包含一个可归约式，再归约两步就会得到 c_2 ；但按值调用规则此时不允许归约它，因为它位于 lambda 抽象之下。

这并不存在根本问题。所得项显然在行为上等价于 c_2 ：把它们分别应用于任意一对参数 v, w ，都会得到相同结果。不过，残留的计算让人不易检查 scc 是否按预期工作。算术计算更复杂时，问题更加严重。例如， $times\ c_2\ c_2$ 求值后不是 c_4 ，而是下面这个庞然大物：

```
(lambda s.
  lambda z.
    (lambda s'. lambda z'. s' (s' z')) s
      ((lambda s'.
        lambda z'.
          (lambda s''. lambda z''. s'' (s'' z'')) s'
            ((lambda s''. lambda z''. z'') s' z'))
        s
      z))
```

检查它是否表现为 c_4 的一种办法，是测试两者是否相等：

$$equal\ c_4\ (times\ c_2\ c_2) \quad \blacktriangleright \quad \lambda t. \lambda f. t$$

更直接的办法，是把 $times\ c_2\ c_2$ 转换为原始数字：

$$realnat\ (times\ c_2\ c_2) \quad \blacktriangleright \quad 4$$

转换过程向 $times\ c_2\ c_2$ 提供了它尚在等待的两个参数，从而触发函数体中尚未进行的计算。

递归

回忆一下，不能在求值关系下迈出一步的项称为范式。有趣的是，有些项不能求值到范式。例如，发散组合子

$$omega = (\lambda x. x\ x)\ (\lambda x. x\ x)$$

只含一个可归约式，而归约它恰好又得到 $omega$ 自己！没有范式的项称为发散 (*diverge*)。

$omega$ 有一个有用的推广，称为 不动点组合子 (*fixed-point combinator*)；⁷ 它可以帮助定义阶乘等递归函数：

$$fix = \lambda f. (\lambda x. f\ (\lambda y. x\ x\ y))\ (\lambda x. f\ (\lambda y. x\ x\ y))$$

⁷它常称为按值调用的 Y 组合子。Plotkin (1975) 称它为 Z 。

8

与 *omega* 一样, *fix* 的结构精细而重复, 单看定义很难理解。建立直观的最好方式, 或许是观察它如何作用于具体示例。⁹ 假设要写形如

$$h = \underbrace{h_{\text{body}}}_{\text{其中含有 } h}$$

的递归函数定义, 也就是等号右侧的项要使用正在定义的函数本身, 如 第5.1节开头的阶乘定义。我们希望递归函数在每次调用自身的位置继续展开其定义。以阶乘为例, 直观上相当于:

```
if n = 0 then 1
else n * (if n - 1 = 0 then 1
          else (n - 1) * (if n - 2 = 0 then 1
                          else (n - 2) * ...))
```

换成 Church 数则是:

```
if realeq n c0 then c1
else times n (if realeq (prd n) c0 then c1
               else times (prd n) (if realeq (prd (prd n)) c0 then c1
                                   else times (prd (prd n)) ...))
```

使用 *fix* 可以达到这一效果: 先定义

$$g = \lambda f. \underbrace{h_{\text{body}}}_{\text{其中原来出现 } h \text{ 的位置改为 } f},$$

再令 $h = \text{fix } g$ 。例如:

```
g = λfct. λn. if realeq n c0 then c1 else times n (fct (prd n)),
factorial = fix g
```

图5-2展示 *factorial* c_3 的求值。这段求值之所以能够按预期展开, 关键在于 $\text{fct } n \rightarrow^* g \text{ fct } n$ 。也就是说, *fct* 是一种“自我复制器”: 应用于参数时, 它把自身和 n 一起作为参数传给 g 。无论 g 的函数体在哪里使用第一个参数, 都会得到另一份 *fct*; 这份 *fct* 再应用于参数时, 又会把自身与参数传给 g , 依此类推。每次用 *fct* 进行递归调用, 都会再展开一份 g 的函数体, 并在其中放入新的 *fct*, 准备下一次展开。

⁸更简单的按名调用不动点组合子

$$Y = \lambda f. (\lambda x. f (x x)) (\lambda x. f (x x))$$

在按值调用环境下没有用, 因为对任何 g , 表达式 $Y g$ 都发散。

⁹也可以从第一原则推导 *fix* 的定义 (例如 Friedman 和 Felleisen, 1996, 第 9 章), 但这种推导同样相当复杂。

为缩短公式，记

$$M_i = \lambda n. c_i (plus\ n)\ c_0, \quad M'_i = \lambda n. c'_i (plus\ n)\ c_0$$

$$\begin{aligned} factorial\ c_3 &= fix\ g\ c_3 \\ &\longrightarrow h\ h\ c_3 && \text{其中 } h = \lambda x. g\ (\lambda y. x\ x\ y) \\ &\longrightarrow g\ fct\ c_3 && \text{其中 } fct = \lambda y. h\ h\ y \\ &\longrightarrow (\lambda n. \text{if } realeq\ n\ c_0 \text{ then } c_1 \text{ else } times\ n\ (fct\ (prd\ n)))\ c_3 \\ &\longrightarrow \text{if } realeq\ c_3\ c_0 \text{ then } c_1 \text{ else } times\ c_3\ (fct\ (prd\ c_3)) \\ &\longrightarrow^* times\ c_3\ (fct\ (prd\ c_3)) \\ &\longrightarrow M_3\ (fct\ (prd\ c_3)) \\ &\longrightarrow^* M_3\ (fct\ c'_2) \\ &\longrightarrow^* M_3\ (g\ fct\ c'_2) \\ &\longrightarrow^* M_3\ (M'_2\ (g\ fct\ c'_1)) \\ &\longrightarrow^* M_3\ (M'_2\ (M'_1\ (g\ fct\ c'_0))) \\ &\longrightarrow^* M_3\ (M'_2\ (M'_1\ (\text{if } realeq\ c'_0\ c_0 \text{ then } c_1 \text{ else } \dots))) \\ &\longrightarrow^* M_3\ (M'_2\ (M'_1\ c_1)) \\ &\longrightarrow^* c'_6 \end{aligned}$$

这里每个 c'_i 都与 c_i 行为等价。

图 5-2: $factorial\ c_3$ 的求值

译者注：作者勘误指出，原图从 $times\ c_3\ (fct\ (prd\ c_3))$ 开始的求值序列不符合按值调用：必须先把 $times\ c_3$ 归约为函数值。本图已据此修正； M_i 与 M'_i 只是为了清楚展示修正后序列而引入的缩写。

译者注：理解不动点组合子，可以先把递归定义拆成两个部分：函数 g 描述阶乘每展开一层要做的计算； fix 则把其中预留的递归调用位置重新接回同一个定义。正文将 g 定义为

$$g = \lambda \underbrace{fct}_{\text{形式参数}}. \lambda n. \text{if } realeq\ n\ c_0 \text{ then } c_1 \text{ else } times\ n\ (fct\ (prd\ n))$$

这里的 fct 是形式参数，暂时代表在递归位置应当调用的函数。只要给 g 提供这样一个函数，它就能构造出阶乘的一层函数体。若把最终的阶乘函数记为 $factorial$ ，递归定义要求它在行为上满足

$$factorial = g\ factorial$$

fix 的任务正是构造出满足这一关系的函数。

下面直接展开 $fix\ g$ 。把 g 作为实参传给 fix 最外层的 λf ，会把函数体中自由出现的 f 都替换为 g 。为缩短结果，先记

$$h = \lambda x. g\ (\lambda y. x\ x\ y)$$

于是第一步就是

$$fix\ g \longrightarrow h\ h$$

接着展开左侧的 h ：

$$\begin{aligned} h\ h &= (\lambda x. g\ (\lambda y. x\ x\ y))\ h \\ &\longrightarrow g\ (\lambda y. h\ h\ y) \end{aligned}$$

括号中的 lambda 抽象才是实际传给 g 的递归函数。为避免把它与 g 定义中的形式参数 fct 混为一谈，这里暂时称它为 R ：

$$R = \lambda y. h\ h\ y$$

因此，整个过程可以写成

$$fix\ g \longrightarrow hh \longrightarrow g\ R$$

在 $g\ R$ 这一步中，实际参数 R 会替换 g 函数体中的形式参数 $fact$ 。原书图5-2沿用形式参数的名字，把这里的 R 也记作 $fact$ ；两者分别是实参和形参，只是在函数应用后对应到了一起。

现在可以看出递归如何发生。假设 n 已经是值，则

$$\begin{aligned} R\ n &= (\lambda y. h\ h\ y)\ n \\ &\longrightarrow h\ h\ n \\ &\longrightarrow g\ R\ n \end{aligned}$$

也就是说，每次调用 R ，都会重新得到带有同一个递归入口 R 的 $g\ R$ ，从而再展开一层阶乘函数体。

这里真正起作用的是自应用 (*self-application*) $h\ h$ 。不过，按值调用会在传参前先求参数的值；若把 $h\ h$ 直接传给 g ，它就会立刻反复展开。本节的组合子把它包成 $R = \lambda y. h\ h\ y$ 。lambda 抽象本身已经是值，可以先安全地传给 g ；只有 R 真正收到递归参数时，其中的 $h\ h$ 才继续求值。这层 lambda 正是按值调用版本所需的延迟。

从这个角度看， fix 的作用就是把“已经描述好一层计算、但仍等待递归入口”的函数闭合起来，使它能够通过反复调用由自己生成的同一个函数。带有递归定义的程序语言会把这条自我连接隐藏在函数名或 `let rec` 背后；不动点组合子则仅用函数抽象与函数应用，把这条连接显式构造出来。

练习 5.2.9 (★). 为什么 g 的定义使用原始 `if`，而不使用 Church 布尔值上的 `test`？说明如何改用 `test` 定义阶乘函数。

[查看原书附录 A 中的 5.2.9 解答](#)

练习 5.2.10 (★★). 定义函数 `churchnat`，把原始自然数转换为相应的 Church 数。

[查看原书附录 A 中的 5.2.10 解答](#)

练习 5.2.11 (推荐, ★★). 使用 fix 与练习5.2.8的列表编码，写一个对 Church 数列表求和的函数。

[查看原书附录 A 中的 5.2.11 解答](#)

表示

结束示例、进入 lambda 演算的形式定义之前，还应当问最后一个问题：“Church 数表示通常自然数”究竟是什么意思？

要回答它，先回忆通常自然数是什么。自然数有许多等价定义；这里采用的定义（图3-2）给出：

- 常量 0；
- 把数字映射为布尔值的操作 `iszero`；
- 把数字映射为数字的两个操作 `succ` 与 `pred`。

这些算术操作的行为由图3-2中的求值规则定义。例如,规则告诉我们 3 是 2 的后继,且 `iszero 0` 为 `true`。

Church 编码把上述每个成分表示为 lambda 项,即函数:

- 项 c_0 表示数字 0。

正如前面的 `succ c1` 示例所见,数字也可以由不同但行为等价的项表示;这类项称为“非规范表示”。例如, $\lambda s. \lambda z. (\lambda x. x) z$ 与 c_0 行为等价,因此也表示 0。

- 项 `scc` 与 `prd` 分别表示算术操作 `succ`、`pred`: 若项 t 表示数字 n , 则 `scc t` 求值到 $n+1$ 的一种表示; `prd t` 求值到 $n-1$ 的一种表示 (若 $n=0$, 则是 0 的表示)。
- 项 `iszro` 表示 `iszero`: 若 t 表示 0, 则 `iszro t` 求值为真;¹⁰ 若 t 表示非零数, 则求值为假。

把这些事实合在一起,假设有一个完整程序,用数字进行复杂计算,最终产生布尔结果。若把所有数字与算术操作都换成相应的 lambda 项表示,再对程序求值,会得到相同结果。因此,就它们对程序总体结果的影响而言,真实数字与 Church 数表示之间没有可观察的差别。

5.3 形式细节

本章余下部分更详细地考察 lambda 演算的语法与操作语义。所需结构大多与第 3 章非常相似;为了避免逐字重复,我们这里只处理不带布尔值、数字等装饰的纯 lambda 演算。不过,用一个项替换变量这一操作包含一些出人意料的细节。

语法

与第 3 章一样,本章前面给出的项的抽象文法应读作抽象语法树集合的归纳定义。

定义 5.3.1 (项). 设 $\mathcal{V}$ 是可数的变量名集合。项集 $\mathcal{T}$ 是满足以下条件的最小集合:

1. 对每个 $x \in \mathcal{V}$, 都有 $x \in \mathcal{T}$;
2. 若 $t_1 \in \mathcal{T}$ 且 $x \in \mathcal{V}$, 则 $\lambda x. t_1 \in \mathcal{T}$;
3. 若 $t_1, t_2 \in \mathcal{T}$, 则 $t_1 t_2 \in \mathcal{T}$ 。

项 t 的大小可以完全照定义3.3.2对算术表达式的做法定义。更有意思的是,可以简单地归纳定义 lambda 项中自由出现的变量集合。

定义 5.3.2. 项 t 的自由变量集合记作 $\text{FV}(t)$, 定义如下:

$$\begin{aligned} \text{FV}(x) &= \{x\}, \\ \text{FV}(\lambda x. t_1) &= \text{FV}(t_1) \setminus \{x\}, \\ \text{FV}(t_1 t_2) &= \text{FV}(t_1) \cup \text{FV}(t_2) \end{aligned}$$

¹⁰严格地说,按照我们的定义, `iszro t` 求值为另一个表示真的项。这里为简化讨论而略去差别。可以用同样方式说明 Church 布尔值在何种意义上表示原始布尔值。

练习 5.3.3 (★★). 仔细证明：对每个项 t ，都有 $|\text{FV}(t)| \leq \text{size}(t)$ 。

[查看原书附录 A 中的 5.3.3 解答](#)

替换

细究起来，替换操作相当棘手。本书实际会采用两个不同定义，分别针对不同目的优化。本节引入的第一个定义紧凑直观，适合示例、数学定义和证明；第 6 章发展的第二个定义记法更重，它采用另一种“de Bruijn 表示”，用数值索引取代具名变量，但更适合后续章节的具体 ML 实现。

下面先考察两种看似自然却不正确的定义，并通过分析它们的问题，逐步得到正确的替换定义。第一种是最直接的方案：按照参数项 t 的结构递归定义替换函数 $[x \mapsto s]$ ：

$$\begin{aligned} [x \mapsto s]x &= s, \\ [x \mapsto s]y &= y && \text{若 } x \neq y, \\ [x \mapsto s](\lambda y. t_1) &= \lambda y. [x \mapsto s]t_1, \\ [x \mapsto s](t_1 t_2) &= ([x \mapsto s]t_1) ([x \mapsto s]t_2) \end{aligned}$$

这个定义对多数示例都能正常工作。例如：

$$[x \mapsto (\lambda z. z w)](\lambda y. x) = \lambda y. \lambda z. z w,$$

这个结果正是我们期望替换操作得到的。然而，这一定义能否正常工作，竟会取决于绑定变量采用什么名字。例如：

$$[x \mapsto y](\lambda x. x) = \lambda x. y$$

译者注：这里的 x 在 $\lambda x. x$ 中受外层 λx 约束，并不是自由出现。因此，正确的替换不应改变这个项，结果仍应是 $\lambda x. x$ 。上述错误定义却继续进入抽象的函数体，把受约束的 x 也替换成了 y ，从而得到忽略参数、固定返回 y 的函数 $\lambda x. y$ ；这正是该定义失效之处。

这违背函数抽象的一项基本直观：只要把 lambda 后面的变量名和函数体中由它约束的所有出现一起改名，项的含义就不会改变。因此， $\lambda x. x$ 、 $\lambda y. y$ 和 $\lambda \text{franz}. \text{franz}$ 都表示同一个恒等函数。如果这三种写法经过替换后会产生不同结果，它们在归约时也会表现不同；这就与它们本应表示同一函数相矛盾。

朴素定义的第一个错误，是没有区分项 t 中变量 x 的自由出现与受约束出现。替换时应当替换自由出现，而不能替换受约束出现。递归遍历项 t 时，如果遇到形如 $\lambda x. t_1$ 的子项，其中的 x 已经被外层的 λx 绑定，不属于替换范围；因此，替换操作不应继续进入 t_1 。于是得到第二次尝试：

$$\begin{aligned} [x \mapsto s]x &= s, \\ [x \mapsto s]y &= y && \text{若 } y \neq x, \\ [x \mapsto s](\lambda y. t_1) &= \begin{cases} \lambda y. t_1 & \text{若 } y = x, \\ \lambda y. [x \mapsto s]t_1 & \text{若 } y \neq x, \end{cases} \\ [x \mapsto s](t_1 t_2) &= ([x \mapsto s]t_1) ([x \mapsto s]t_2) \end{aligned}$$

这已经好一些，但仍不完全正确。例如，把项 z 替换到项 $\lambda z. x$ 中的变量 x ：

$$[x \mapsto z](\lambda z. x) = \lambda z. z$$

这次犯了几乎相反的错误：把常函数 $\lambda z. x$ 变成了恒等函数！问题仍然来自绑定变量的具体名称：这里恰好使用了 z ，导致替换进去的自由变量 z 被 lambda 抽象绑定。这说明定义仍有缺陷。

如果把项 s 代入项 t 时没有处理变量名冲突， s 中原本自由的变量可能会被 t 中的 lambda 抽象绑定；这种现象称为 变量捕获 (*variable capture*)。要避免捕获，必须保证 t 的受约束变量名与 s 的自由变量名彼此不同。正确做到这一点的替换称为 避免捕获的替换 (*capture-avoiding substitution*)；不加以限制地说“替换”时，几乎总是指它。只需在抽象情形的第二个分支中再增加一项限制，要求 $y \notin \text{FV}(s)$ ：

$$\begin{aligned} [x \mapsto s]x &= s, \\ [x \mapsto s]y &= y && \text{若 } y \neq x, \\ [x \mapsto s](\lambda y. t_1) &= \begin{cases} \lambda y. t_1 & \text{若 } y = x, \\ \lambda y. [x \mapsto s]t_1 & \text{若 } y \neq x \text{ 且 } y \notin \text{FV}(s), \end{cases} \\ [x \mapsto s](t_1 t_2) &= ([x \mapsto s]t_1) ([x \mapsto s]t_2) \end{aligned}$$

现在离正确的定义只差一步。这套规则一旦适用，得到的替换结果就符合预期；但新加入的限制使替换成为偏操作 (*partial operation*)（只对部分输入有定义的操作）。例如，对 $[x \mapsto yz](\lambda y. xy)$ ，抽象 $\lambda y. xy$ 绑定的变量 y 不等于 x ，所以抽象情形的第一个分支不适用；同时， y 又在待代入项 yz 中自由出现，第二个分支要求的 $y \notin \text{FV}(yz)$ 也不成立。由于没有任何分支适用，这一定义无法给出替换结果。

Church 将这种同时改写变量的绑定处及其所约束出现的操作称为 α 变换 (*alpha-conversion*)。这一术语在类型系统和 lambda 演算文献中沿用至今。

约定 5.3.4. 对受约束变量作一致的重命名不会改变项；重命名前后的项可以在所有上下文中互换。

实际使用时，这意味着可以在需要时更改 lambda 绑定的变量名，只要把函数体中由它约束的所有出现一并改名。例如，计算 $[x \mapsto yz](\lambda y. xy)$ 时，先把 $\lambda y. xy$ 改写为 $\lambda w. xw$ ，再计算 $[x \mapsto yz](\lambda w. xw)$ ，得到 $\lambda w. yzw$ 。

有了这项约定，替换虽然在形式上仍是偏操作，在实际使用中却总能得到结果：如果一组参数不满足定义中的限制，可以先适当重命名受约束变量，使这些限制成立。采用该约定后，替换定义还能写得更简洁。抽象情形的第一个分支可以删去，因为总能假设（必要时先重命名）绑定变量 y 与 x 以及 s 中的自由变量都不同。最终定义如下。

定义 5.3.5 (替换).

$$\begin{aligned}
 [x \mapsto s]x &= s, \\
 [x \mapsto s]y &= y && \text{若 } y \neq x, \\
 [x \mapsto s](\lambda y. t_1) &= \lambda y. [x \mapsto s]t_1 && \text{若 } y \neq x \text{ 且 } y \notin \text{FV}(s), \\
 [x \mapsto s](t_1 t_2) &= ([x \mapsto s]t_1) ([x \mapsto s]t_2)
 \end{aligned}$$

操作语义

图5-3汇总 lambda 项的操作语义。这里的值集比算术表达式时更有意思。按值调用求值到达 lambda 抽象就停止，因此值可以是任意 lambda 抽象。

λ (无类型)

语法	求值 $t \longrightarrow t'$
$t ::= x$ 项: 变量	$\frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2} \quad \text{E-APP1}$
$ \lambda x. t$ 抽象	$\frac{t_2 \longrightarrow t'_2}{v_1 t_2 \longrightarrow v_1 t'_2} \quad \text{E-APP2}$
$ t t$ 应用	$(\lambda x. t_{12}) v_2 \longrightarrow [x \mapsto v_2]t_{12} \quad \text{E-APPABS}$
$v ::= \lambda x. t$ 值: 抽象值	

图 5-3: 无类型 lambda 演算 (λ)

求值关系位于图的右栏。与算术表达式的求值一样,规则分成两类:计算规则 E-APPABS, 以及同余规则 E-APP1、E-APP2。

注意, 这些规则中的元变量选择有助于控制求值次序。由于 v_2 只取值, E-APPABS 的左侧可以匹配右侧已经是值的应用。E-APP1 则适用于左侧尚非值的应用: t_1 可以匹配任意项, 但前提还要求它能够迈出一小步。E-APP2 必须等到应用左侧成为值、能够绑定给值元变量 v_1 后才能触发。三条规则合在一起, 完全确定应用 $t_1 t_2$ 的求值次序: 先用 E-APP1 把 t_1 归约为值, 再用 E-APP2 把 t_2 归约为值, 最后用 E-APPABS 执行应用本身。

练习 5.3.6 ($\star\star$). 调整这些规则, 描述其他三种求值策略: 完全 β 归约、正规序与惰性求值。

[查看原书附录 A 中的 5.3.6 解答](#)

注意, 在纯 lambda 演算中, lambda 抽象是唯一可能的值。因此, 如果到达 E-APP1 已经成功把 t_1 归约为值的状态, 这个值必为 lambda 抽象。当然, 给语言加入原始布尔值等其他构造后, 这一观察就失效了, 因为那些构造会引入抽象以外的值形式。

练习 5.3.7 ($\star\star, \rightarrow$). 练习3.5.16给出了布尔值与算术表达式操作语义的另一种表述: 卡住的项求值为特殊常量 `wrong`。把这种语义扩展到 λ_{NB} 。

[查看练习 5.3.7 的译者参考解答](#)

练习 5.3.8 ($\star\star$). 练习3.5.17为算术表达式引入了“大步”求值风格, 其基本关系是“项 t 求值为最终结果 v ”。说明如何用大步风格表述 lambda 项的求值规则。

[查看原书附录 A 中的 5.3.8 解答](#)

5.4 注记

Church 及其合作者在 20 世纪 20、30 年代发展了无类型 lambda 演算 (Church, 1941)。全面介绍无类型 lambda 演算各方面内容的标准著作是 Barendregt (1984); Hindley 和 Seldin (1986) 不及前者全面, 但更容易入门。Barendregt 在 *Handbook of Theoretical Computer Science* 中的文章 (1990) 是一篇紧凑综述。许多函数式程序设计教材 (如 Abelson 和 Sussman, 1985; Friedman、Wand 和 Haynes, 2001; Peyton Jones 和 Lester, 1992) 以及程序语言语义教材 (如 Schmidt, 1986; Gunter, 1992; Winskel, 1993; Mitchell, 1996) 也介绍 lambda 演算。Böhm 和 Berarducci (1985) 提出了一种将种类广泛的数据结构编码为 lambda 项的系统方法。

尽管“柯里化”以 Curry 命名, 他本人否认发明过这个想法。通常认为它来自 Schönfinkel (1924), 但底层思想早已为 Frege、Cantor 等多位 19 世纪数学家所熟知。

事实上, 这个系统除了用作逻辑之外, 或许还有其他用途。

There may, indeed, be other applications of the system than its use as a logic.

——Alonzo Church, 1932

第六章 项的无名表示

上一章处理项时，我们允许把一个受约束变量连同它所约束的所有出现一起改名。替换过程中，如果原来的名字会造成变量捕获，就先换成一个不会冲突的新名字；在其他场合，也可以仅仅为了书写方便而改名。换言之，在这些数学讨论中，重要的是变量的每次出现由哪个 λ 绑定，而不是变量具体叫什么。这项约定很适合讨论基本概念和清晰地陈述证明；但是在程序实现中，每个项都必须采用一种确定的表示，尤其要明确如何记录变量的每次出现。这里有不只一种选择。

1. 仍像前面一样用符号表示变量，但不再默许受约束变量可以重命名；替换时若有必要，则显式地把受约束变量改成一个不与相关变量冲突的新名字，以避免变量捕获。
2. 仍用符号表示变量，但作出一项全局约定：所有受约束变量的名字彼此不同，并且也不同于可能用到的任何自由变量。这项约定有时称为 Barendregt 约定 (*Barendregt convention*)。它比本书此前的约定更严格，因为它不允许在任意时刻“就地”重命名。然而，它在替换（以及 β 归约）下并不稳定：替换会复制被代入的项，很容易构造出某些结果项，其中几个 λ 抽象使用了同一个受约束变量名。因此，每次涉及替换的求值步骤之后，还必须重命名以恢复这项不变量。
3. 为变量和项设计某种规范表示 (*canonical representation*)，使其无需重命名。
4. 借助显式替换 (*explicit substitution*) 等机制，完全避免元语言层面的替换 (Abadi、Cardelli、Curien 和 Lévy, 1991a)。
5. 完全避开变量，直接使用基于组合子的语言，例如 组合逻辑 (*combinatory logic*) (Curry 和 Feys, 1958; Barendregt, 1984) ——它是以组合子而非过程抽象为基础的 λ 演算变体——或 Backus 的函数式语言 FP (1978)。

每种方案都有拥护者，如何取舍在一定程度上属于个人偏好。严肃的编译器实现还要考虑性能，不过这不在本章范围内。我们选择第三种方案；根据我们的经验，它更能应对后续实现中不断增长的复杂度。主要原因在于：这种方案若实现错误，往往会显著地而不是隐蔽地失败，因此错误能较早暴露并得到修正。相比之下，使用具名变量的实现中，有些缺陷会在引入数月甚至数年后才显现出来。这里采用 Nicolas de Bruijn (1972) 提出的一项著名技术。¹

¹本章研究的系统是纯无类型 λ 演算 λ (图5-3)。相应的 OCaml 实现是 `fulluntyped`。

6.1 项与上下文

de Bruijn 的想法是：让变量的每次出现直接指向自己的绑定子，而不再通过名字引用绑定子，就能以更直接——尽管可读性稍差——的方式表示项。具体做法是用自然数替代具名变量，其中数字 k 表示“从这次变量出现的位置由内向外数（从 0 起），第 k 个包围它的 λ 所约束的变量”。例如，具名项 $\lambda x. x$ 对应无名项 $\lambda. 0$ ，而 $\lambda x. \lambda y. x (y x)$ 对应 $\lambda. \lambda. 1 (0 1)$ 。无名项也称为 de Bruijn 项 (*de Bruijn term*)，其中表示变量的数字称为 de Bruijn 索引 (*de Bruijn index*)。² 编译器领域也把同一概念称为静态距离 (*static distance*)。

练习 6.1.1 (★). 请写出下列各组合子所对应的无名项：

$$c_0 = \lambda s. \lambda z. z;$$

$$c_2 = \lambda s. \lambda z. s (s z);$$

$$plus = \lambda m. \lambda n. \lambda s. \lambda z. m s (n s z);$$

$$fix = \lambda f. (\lambda x. f (\lambda y. (x x) y)) (\lambda x. f (\lambda y. (x x) y));$$

$$foo = (\lambda x. (\lambda x. x)) (\lambda x. x)$$

查看原书附录 A 中的 6.1.1 解答

译者注：原书初版在本练习的 *plus* 中印作 $m s (n z s)$ ，这并不是第五章定义的 Church 数加法。作者勘误将其视为错误；此处按正确项 $m s (n s z)$ 修正。

形式上，无名项的语法与具名项的语法（定义 5.3.1）几乎完全相同。唯一的差别是：还必须说明一个无名项可在多长的外部变量上下文中使用，这样才能判断其中的索引是否都有所指。为此，我们把不需要外部变量的项称为 0-项，把至多使用一个外部变量位置的项称为 1-项，依此类推。

定义 6.1.2 (项). 令 T 为满足下列条件的最小集合族 $\{T_0, T_1, T_2, \dots\}$ ：

1. 若 $0 \leq k < n$ ，则以 k 为 de Bruijn 索引的变量项属于 T_n （简写为 $k \in T_n$ ）；
2. 若 $t_1 \in T_{n+1}$ ，则 $\lambda. t_1 \in T_n$ 。函数体可以使用当前 lambda 绑定的变量，因此比整个抽象多一个可用变量位置；
3. 若 $t_1 \in T_n$ 且 $t_2 \in T_n$ ，则应用项 $(t_1 t_2) \in T_n$ 。应用不引入新的变量绑定，因此两个子项与整个应用项使用同一个上下文。

这里仍是标准的归纳定义，只不过定义的是以自然数为索引的一族集合，而非单个集合。每个 T_n 的元素称为 n -项。

T_n 中的元素可以在含有 n 个外部变量位置的上下文中使用；其中自由变量的索引只能介于 0 与 $n-1$ 之间。一个给定的 T_n 元素不必使用所有这些位置，甚至可以完全不使用自由变量。例如，闭项属于每一个 T_n 。

²关于读音：“de Bruijn” 的第二个音节，最接近的英语读音是 “brown”，不是 “broyn”。

对于任意闭的具名项，每个变量出现都有唯一确定的绑定者，因此其 de Bruijn 表示也唯一确定。两个闭的具名项得到同一个 de Bruijn 表示，当且仅当它们只在受约束变量的命名上不同。

为了处理含自由变量的项，需要引入命名上下文 (*naming context*)。例如，设要把 $\lambda x. yx$ 表示成无名项。我们知道怎样处理 x ，却看不到 y 的绑定子，因而无从判断它“距离多远”，也就不知道该给它什么编号。解决办法是一次性选定一套从自由变量到 de Bruijn 索引的指派 (即命名上下文)，此后凡是需要给自由变量选择编号，都一致地使用这套指派。例如，假定采用

$$\Gamma = x \mapsto 4, \quad y \mapsto 3, \quad z \mapsto 2, \quad a \mapsto 1, \quad b \mapsto 0,$$

那么 $x(yz)$ 表示成 $4(32)$ ， $\lambda w. yw$ 表示成 $\lambda. 40$ ，而 $\lambda w. \lambda a. x$ 表示成 $\lambda. \lambda. 6$ 。

译者注：最后一个例子中， x 在 Γ 中的索引原本是 4。进入 λw 的函数体后，新绑定的 w 占据索引 0， x 因而变成 5；再进入 λa 的函数体， x 又变成 6。一般来说，如果一个外部变量原来的索引为 k ，把它放到不绑定它的 r 层新 lambda 之下后，其索引就变为 $k + r$ 。§6.2 将把这种调整形式化为“移位”。

由于变量在 Γ 中的顺序已经决定其数字索引，可以把上下文简写为一个序列。

定义 6.1.3. 设 $x_0, \dots, x_n$ 是变量集合 V 中的名字。命名上下文

$$\Gamma = x_n, x_{n-1}, \dots, x_1, x_0$$

把 de Bruijn 索引 i 指派给 x_i 。序列最右侧的变量获得索引 0；这与把具名项转换为无名形式时从右向左计数 λ 绑定子的方式一致。记 $\text{dom}(\Gamma)$ 为 Γ 中出现的变量名集合 $\{x_n, \dots, x_0\}$ 。

练习 6.1.4 ($\star\star\star, \rightarrow$)。参照定义 3.2.3 的风格，给出 n -项集合的另一种构造，并像命题 3.2.6 那样证明它与上面的定义等价。

查看练习 6.1.4 的译者参考解答

练习 6.1.5 (推荐, $\star\star\star$)。1. 定义函数 $\text{removenames}_\Gamma(t)$ ：它接受命名上下文 Γ 和满足 $\text{FV}(t) \subseteq \text{dom}(\Gamma)$ 的具名项 t ，返回对应的无名项。

2. 定义函数 $\text{restorenames}_\Gamma(t)$ ：它接受无名项 t 和命名上下文 Γ ，返回一个具名项。为此，需要为 t 中各抽象所约束的变量“造出”名字。可以假定 Γ 中的名字两两不同，而且变量名集合 V 带有序，从而“选择第一个不在 $\text{dom}(\Gamma)$ 中的变量名”是有意义的。

这对函数应满足：对任意无名项 t ，

$$\text{removenames}_\Gamma(\text{restorenames}_\Gamma(t)) = t;$$

对任意具名项 t ，把它转换成无名项再恢复名称，所得结果应能通过 α 变换还原为 t ：

$$\text{restorenames}_\Gamma(\text{removenames}_\Gamma(t)) = t$$

查看原书附录 A 中的 6.1.5 解答

严格说来，“某个 $t \in T$ ”并没有意义——总要说明 t 是在哪一种长度的外部变量上下文中使用的，也就是它属于哪个 T_n 。不过在实际使用中，通常已经固定了命名上下文 Γ 。为简化记法，我们仍写 $t \in T$ ，其含义是 $t \in T_n$ ，其中 n 是 Γ 的长度。

6.2 移位与替换

接下来要在无名项上定义替换操作 $[k \mapsto s]t$ 。为此需要一项辅助操作，称为移位 (*shifting*)，它重新编号项中自由变量的索引。

待代入项 s 中的自由变量索引，是按照 lambda 外层的上下文编号的。以 $[1 \mapsto s](\lambda.2)$ 为例：若 x 在外层上下文中的索引为 1，该式就对应具名替换 $[x \mapsto s](\lambda y.x)$ 。替换递归进入 λy 的函数体后，当前 lambda 绑定的 y 会加入上下文并占据索引 0，所以函数体中的上下文比外层多一个变量位置。为了让 s 中的自由变量仍然指向同一个外部变量，必须把它们的索引加一。但这要谨慎进行：不能把 s 中的每个索引都加一，否则可能连 s 内部的受约束变量也被移位。例如，若 $s = 2(\lambda.0)$ （即在外层上下文中 z 的索引为 2 时， $s = z(\lambda w.w)$ ），应当移动 2，却不能移动 0。

下面的移位函数带有一个控制哪些变量应当移动的截点 (*cutoff*) 参数 c 。处理最外层的待移位项时， c 从 0 开始；递归每进入一层 lambda 抽象， c 就加一。因此，计算 $\uparrow_c^d(t)$ 时， c 表示从最外层待移位项走到当前子项 t 的过程中，已经进入了多少层 lambda 抽象。当前子项中满足 $k < c$ 的索引由这些 lambda 绑定，不应移动；满足 $k \geq c$ 的索引则指向这些 lambda 之外，应当移动。

定义 6.2.1 (移位). 项 t 在截点 c 之上的 d 位移位，记作 $\uparrow_c^d(t)$ ，定义如下：

$$\begin{aligned}\uparrow_c^d(k) &= \begin{cases} k, & k < c, \\ k + d, & k \geq c; \end{cases} \\ \uparrow_c^d(\lambda. t_1) &= \lambda. \uparrow_{c+1}^d(t_1); \\ \uparrow_c^d(t_1 t_2) &= \uparrow_c^d(t_1) \uparrow_c^d(t_2)\end{aligned}$$

用 $\uparrow^d(t)$ 简写 $\uparrow_0^d(t)$ 。

译者注：这里的位移量 d 是整数，可以为负。 $d > 0$ 使需要移动的索引增大， $d < 0$ 则使其减小；负移位只有在所有移动后的索引仍为非负数时才得到合法的无名项。后面的 β 归约会用到负移位：一层 lambda 绑定被消去后，指向其外层的变量与目标之间少了一层距离，相关索引也要相应减一。

练习 6.2.2 ($\star$). 1. $\uparrow^2(\lambda. \lambda. 1(02))$ 是什么？

2. $\uparrow^2(\lambda. 01(\lambda. 012))$ 是什么？

[查看原书附录 A 中的 6.2.2 解答](#)

练习 6.2.3 ($\star\star, \rightarrow$). 证明：若 t 是 n -项，且当 $d < 0$ 时 t 中每个自由变量的索引均不小于 $|d|$ ，则顶层移位 $\uparrow^d(t) = \uparrow_0^d(t)$ 是 $\max(n + d, 0)$ -项。

[查看练习 6.2.3 的译者参考解答](#)

译者注：原书初版把练习 6.2.3 无条件写成“若 t 是 n -项，则 $\uparrow_c^d(t)$ 是 $(n + d)$ -项”。作者勘误补上了 $d < 0$ 时的自由变量前提，并把结论改为 $\max(n + d, 0)$ -项，但仍保留任意截点 c 。按字面理解，后一陈述仍有反

例：取 $t = 1 \in T_2$ 、 $d = -1$ 、 $c = 2$ ，则 $\uparrow_2^{-1}(1) = 1 \notin T_1$ 。本题实际使用的是顶层移位；这里把结论明确写成 $\uparrow^d = \uparrow_0^d$ 。任意截点的正确推广需要同时记录截点所处的绑定深度。

现在可以定义替换算子 $[j \mapsto s]t$ 。 β 归约开始替换时，目标总是索引 0，因为 lambda 抽象的参数在其函数体中编号为 0。但是，替换需要递归遍历项；每进入一层新的 lambda 抽象，该抽象绑定的变量就会占据索引 0，原来的目标变量在函数体中的编号因而增加 1。例如，计算 $[0 \mapsto s](\lambda.1)$ 时，递归进入函数体后需要替换的是编号 1 的变量。所以，即使替换的最外层调用通常取 $j = 0$ ，其递归定义也必须能够处理任意编号 j 。

定义 6.2.4 (替换). 用项 s 替换项 t 中编号为 j 的变量，记作 $[j \mapsto s]t$ ，定义如下：

$$[j \mapsto s]k = \begin{cases} s, & k = j, \\ k, & \text{其他情形;} \end{cases}$$

$$[j \mapsto s](\lambda. t_1) = \lambda. [j + 1 \mapsto \uparrow^1(s)]t_1;$$

$$[j \mapsto s](t_1 t_2) = ([j \mapsto s]t_1) ([j \mapsto s]t_2)$$

练习 6.2.5 ($\star$). 假定全局上下文为 $\Gamma = a, b$ 。把下面各次替换转换为无名形式，再按上述定义算出结果。这些答案是否与 §5.3 中对具名项的原始替换定义相符？

1. $[b \mapsto a](b(\lambda x. \lambda y. b))$
2. $[b \mapsto a(\lambda z. a)](b(\lambda x. b))$
3. $[b \mapsto a](\lambda b. b a)$
4. $[b \mapsto a](\lambda a. b a)$

[查看原书附录 A 中的 6.2.5 解答](#)

练习 6.2.6 ($\star\star, \rightarrow$). 证明：若 s, t 都是 n -项且 $j \leq n$ ，则 $[j \mapsto s]t$ 是 n -项。

[查看练习 6.2.6 的译者参考解答](#)

练习 6.2.7 ($\star, \rightarrow$). 拿出一张纸，不看上面的替换与移位定义，自己重新推导出它们。

[查看练习 6.2.7 的译者参考解答](#)

练习 6.2.8 (推荐, $\star\star\star$). 无名项上的替换定义应当与具名项上的非形式化替换定义一致。

1. 要严格论证这种对应，需要证明什么定理？
2. 证明这个定理。

[查看原书附录 A 中的 6.2.8 解答](#)

6.3 求值

要在无名项上定义求值关系，只需重新表述图5-3中的三条求值规则。其中，E-APP1 与 E-APP2 只规定应用两个子项的求值次序，不涉及变量名，因而可以原样保留。

E-APPABS 描述的是 β 归约：它既提到具体的变量名，又执行按名称进行的替换，因此需要改写为使用新的无名替换操作。

这里有一个需要特别处理的细节：归约一个可归约式会“用掉”受约束变量。把 $(\lambda x. t_{12}) v_2$ 归约为 $[x \mapsto v_2]t_{12}$ 时，受约束变量 x 随之消失。因此必须重新编号替换结果中的变量，以反映 x 已经不在上下文中。例如：

$$(\lambda. 102)(\lambda. 0) \longrightarrow 0(\lambda. 0)1 \quad (\text{而不是 } 1(\lambda. 0)2)。$$

同样，在把 v_2 替换进 t_{12} 之前，还必须把 v_2 中的变量向上移动一位，因为 t_{12} 所处的上下文比 v_2 的上下文大。综合这两点， β 归约规则为：

$$\frac{}{(\lambda. t_{12}) v_2 \longrightarrow \uparrow^{-1} ([0 \mapsto \uparrow^1 (v_2)]t_{12})} \text{E-APPABS}$$

其余规则与图5-3 完全相同。

练习 6.3.1 (★). 这条规则中的负位移会不会产生含负索引的病态项？我们是否需要担心？

查看原书附录 A 中的 6.3.1 解答

练习 6.3.2 (★★★). de Bruijn 的原始论文其实提出了两种项的无名表示：本章介绍的 de Bruijn 索引从内向外给 lambda 绑定子编号；de Bruijn 层级 (*de Bruijn level*) 则从外向内编号。例如，项

$$\lambda x. (\lambda y. x y) x$$

使用 de Bruijn 索引表示为 $\lambda. (\lambda. 10) 0$ ，使用 de Bruijn 层级表示为 $\lambda. (\lambda. 01) 0$ 。请严格定义后一种变体，并证明同一个项的索引表示与层级表示同构，也就是说，任一种表示都能唯一地恢复出另一种。

查看原书附录 A 中的 6.3.2 解答

第七章 lambda 演算的 ML 实现

本章以第四章的算术表达式解释器，以及第六章对变量绑定和替换的处理为基础，为无类型 lambda 演算构造一个解释器。

只要把前面的定义直接翻译成 OCaml，就能得到一个可执行的无类型 lambda 项求值器。与第四章一样，这里只展示核心算法，暂且忽略词法分析、解析、打印等问题。¹

译者注：原书的 OCaml 程序完整保留在正文中，每段代码之后直接给出相应的 Rust 对照实现。可运行的完整工程位于 `code/untyped/`，覆盖 de Bruijn 索引、上下文长度一致性检查、移位、替换、按值调用求值、解析与打印；其语义覆盖和与作者工具的具体语法差异见该目录的 README。

7.1 项与上下文

把定义6.1.2 直接转写为数据类型，就得到表示项的抽象语法树：

```
type term =  
  TmVar of int  
| TmAbs of term  
| TmApp of term * term
```

Rust 对照实现（译者增补）

```
#[derive(Clone, Debug, PartialEq, Eq)]  
pub enum Term {  
  Var(usize),  
  Abs(Box<Term>),  
  App(Box<Term>, Box<Term>),  
}
```

变量表示为一个数字，即它的 de Bruijn 索引。抽象只携带表示函数体的子项；应用则携带被应用的两个子项。

不过，实际实现使用的定义还会多携带一些信息。首先，像以前一样，给每个项标注一个 `info` 类型的元素很有用。它记录该项最初出现的文件位置，从而让错误报告过程能把用户（甚至用户的文本编辑器）自动引导到发生错误的确切位置。

¹本章大部分内容研究纯无类型 lambda 演算（图5-3），相应实现为 `untyped`。`fulluntyped` 实现还包含数字和布尔值等扩展。

```
type term =
  TmVar of info * int
| TmAbs of info * term
| TmApp of info * term * term
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct Info;

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
  Var(Info, usize),
  Abs(Info, Box<Term>),
  App(Info, Box<Term>, Box<Term>),
}
```

其次, 为了调试, 最好在每个变量结点上再携带一个数字, 作为一致性检查。约定这个数字始终等于该变量出现处的上下文总长度 (即该位置的当前上下文中共有多少个变量绑定)。

```
type term =
  TmVar of info * int * int
| TmAbs of info * term
| TmApp of info * term * term
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
  Var(Info, usize, usize),
  Abs(Info, Box<Term>),
  App(Info, Box<Term>, Box<Term>),
}
```

每次打印变量时, 都要验证这个数字是否等于当前上下文的实际大小; 若不相等, 就说明某处漏做了一次移位操作。

最后一项改进也与打印有关。尽管内部用 de Bruijn 索引表示项, 显然不应把这种形式直接呈现给用户: 解析时, 应当从普通表示转换成无名项; 打印时, 则应转换回普通形式。这本身并不困难, 但也不能简单地把所有变量重新命名。例如, 如果打印时一律为变量生成与原程序无关的新名字, 那么输出项中的受约束变量名就会与原程序里的名字毫无关系。解决办法是在每个抽象上标注一个字符串, 作为受约束变量名的提示:


```

type term =
  TmVar of info * int * int
| TmAbs of info * string * term
| TmApp of info * term * term

```

Rust 对照实现 (译者增补)

```

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    Var(Info, usize, usize),
    Abs(Info, String, Box<Term>),
    App(Info, Box<Term>, Box<Term>),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum EvalError {
    // 当前项已经没有可用的求值规则。
    NoRuleApplies,
    // 移位后的索引或上下文长度会变成负数, 因而不是合法的 de Bruijn 表示。
    NegativeShift,
    // 项中记录的上下文长度与打印时实际提供的上下文长度不一致。
    BadContextLength { recorded: usize, actual: usize },
    // 变量索引超出了当前上下文的范围。
    UnboundIndex { index: usize, context_len: usize },
}

```

项上的基本操作, 特别是替换, 不会对这些字符串做任何复杂处理: 它们只会原样随项传递, 不检查名字冲突、变量捕获等问题。打印过程需要为受约束变量确定一个当前上下文尚未使用的名字。它会先尝试使用给定提示; 若提示与当前上下文中已经使用的名字冲突, 就尝试相近的名字, 不断添加撇号, 直到找到当前尚未使用的名字。这样, 除了可能多几个撇号之外, 打印出的项会与用户预期的形式相近。

打印过程本身如下:

```

let rec printtm ctx t = match t with
| TmAbs(fi,x,t1) ->
    let (ctx',x') = pickfreshname ctx x in
    pr "(lambda "; pr x'; pr ". "; printtm ctx' t1; pr ")"
| TmApp(fi,t1,t2) ->
    pr "("; printtm ctx t1; pr " "; printtm ctx t2; pr ")"
| TmVar(fi,x,n) ->
    if ctxlength ctx = n then
        pr (index2name fi ctx x)

```



```

else
  pr "[bad index]"

```

Rust 对照实现 (译者增补)

```

pub fn print_term(context: &Context, term: &Term) -> Result<String, EvalError> {
  match term {
    Term::Abs(_, hint, body) => {
      let (extended, name) = pick_fresh_name(context, hint);
      Ok(format!("(lambda {name}. {})", print_term(&extended, body)?))
    }
    Term::App(_, function, argument) => Ok(format!(
      "({} {})",
      print_term(context, function)?,
      print_term(context, argument)?
    )),
    Term::Var(_, index, context_len) => {
      if *context_len == context.len() {
        index_to_name(context, *index)
      } else {
        Err(EvalError::BadContextLength {
          recorded: *context_len,
          actual: context.len(),
        })
      }
    }
  }
}

```

它使用数据类型 context:

```

type context = (string * binding) list

```

Rust 对照实现 (译者增补)

```

pub type Context = Vec<(String, Binding)>;

```

上下文就是由字符串及其关联绑定组成的列表。眼下绑定本身极为简单:

```
type binding = NameBind
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Binding {
    Name,
}
```

其中没有携带任何有意义的信息。稍后在第十章, 我们会为 `binding` 类型加入其他分支, 用于记录变量对应的类型假设以及类似信息。

打印函数还依赖若干底层函数: `pr` 把字符串送往标准输出流; `ctxlength` 返回上下文长度; `index2name` 根据变量索引查找其字符串名字。其中最有意思的是 `pickfreshname`。它接受上下文 `ctx` 和字符串提示 `x`, 找出一个与 `x` 相近且尚未列入 `ctx` 的名字 `x'`, 把 `x'` 加进 `ctx` 形成新上下文 `ctx'`, 再把 `ctx'` 与 `x'` 成对返回。

Rust 辅助实现 (译者增补)

```
// 优先沿用原项提供的名字提示; 若发生冲突, 就不断添加撇号。
fn pick_fresh_name(context: &Context, hint: &str) -> (Context, String) {
    let mut name = if hint.is_empty() {
        "x".to_owned()
    } else {
        hint.to_owned()
    };
    while context.iter().any(|(used, _)| used == &name) {
        name.push('\''');
    }
    let mut extended = context.clone();
    extended.insert(0, (name.clone(), Binding::Name));
    (extended, name)
}

// de Bruijn 索引从 0 开始计数, 直接对应上下文中的位置。
fn index_to_name(context: &Context, index: usize) -> Result<String, EvalError> {
    context
        .get(index)
        .map(|(name, _)| name.clone())
        .ok_or(EvalError::UnboundIndex {
            index,
            context_len: context.len(),
        })
}
```

本书网站上 `untyped` 实现中的实际打印函数还要复杂一些，它额外处理两个问题。第一，它遵循应用左结合、抽象函数体尽量向右延伸的约定，尽可能省略括号。第二，它为一个底层的美观打印模块（OCaml 的 `Format` 库）生成格式化指令，由该模块决定怎样换行和缩进。

7.2 移位与替换

移位的定义 6.2.1 几乎可以逐符号翻译成 OCaml：

```
let termShift d t =
  let rec walk c t = match t with
    | TmVar(fi,x,n) ->
      if x >= c then TmVar(fi,x+d,n+d)
      else TmVar(fi,x,n+d)
    | TmAbs(fi,x,t1) ->
      TmAbs(fi,x,walk (c+1) t1)
    | TmApp(fi,t1,t2) ->
      TmApp(fi,walk c t1,walk c t2)
  in walk 0 t
```

Rust 对照实现（译者增补）

```
// 索引使用 usize，而位移量可能为负。checked_add_signed 会拒绝小于零或
// 超出 usize 范围的结果；调用处的 `?` 会继续向外传播这个错误。
fn shifted(value: usize, distance: isize) -> Result<usize, EvalError> {
  value
    .checked_add_signed(distance)
    .ok_or(EvalError::NegativeShift)
}

pub fn term_shift(distance: isize, term: &Term) -> Result<Term, EvalError> {
  fn walk(distance: isize, cutoff: usize, term: &Term) -> Result<Term,
  ↪ EvalError> {
    match term {
      Term::Var(info, index, context_len) => Ok(Term::Var(
        *info,
        if *index >= cutoff {
          shifted(*index, distance)?
        } else {
          *index
        },
        shifted(*context_len, distance)?,
      )),
      Term::Abs(info, hint, body) => Ok(Term::Abs(
```

```

        *info,
        hint.clone(),
        Box::new(walk(distance, cutoff + 1, body)?),
    )),
    Term::App(info, function, argument) => Ok(Term::App(
        *info,
        Box::new(walk(distance, cutoff, function)?),
        Box::new(walk(distance, cutoff, argument)?),
    )),
}

walk(distance, 0, term)
}

```

内部移位 $\uparrow_c^d(t)$ 在这里表示成对内部函数 `walk c t` 的一次调用。由于 d 始终不变，不必在每次调用 `walk` 时都传递它；在变量分支需要 d 时，直接使用外层绑定即可。顶层移位 $\uparrow^d(t)$ 表示成 `termShift d t`。注意，`termShift` 本身没有标成递归函数，因为它只调用一次 `walk`。

同样，替换函数也几乎直接来自[定义6.2.4](#)：

```

let termSubst j s t =
  let rec walk c t = match t with
    | TmVar(fi,x,n) ->
      if x=j+c then termShift c s
      else TmVar(fi,x,n)
    | TmAbs(fi,x,t1) ->
      TmAbs(fi,x,walk (c+1) t1)
    | TmApp(fi,t1,t2) ->
      TmApp(fi,walk c t1,walk c t2)
  in walk 0 t

```

Rust 对照实现 (译者增补)

```

pub fn term_substitute(
    variable: usize,
    replacement: &Term,
    term: &Term,
) -> Result<Term, EvalError> {
    fn walk(
        variable: usize,
        replacement: &Term,
        cutoff: usize,
    ) -> Result<Term, EvalError> {
        match term {
            Term::Var(x) => {
                if x == variable {
                    Ok(replacement.clone())
                } else {
                    Ok(Term::Var(x))
                }
            }
            Term::Abs(x, t1) => {
                Ok(Term::Abs(x, walk(x, replacement, cutoff + 1)?))
            }
            Term::App(t1, t2) => {
                Ok(Term::App(walk(x, replacement, cutoff)?, walk(x, replacement, cutoff)?))
            }
        }
    }
    walk(variable, replacement, 0)
}

```

```

    term: &Term,
  ) -> Result<Term, EvalError> {
    match term {
      Term::Var(_, index, _) if *index == variable + cutoff => {
        // 替换项将被放到已经进入的 `cutoff` 层抽象之下；先上移其中的
        // 自由变量，避免它们被这几层抽象意外捕获。
        term_shift(usize::try_from(cutoff).expect("cutoff fits"),
↪ replacement)
      }
      Term::Var(info, index, context_len) => Ok(Term::Var(*info, *index,
↪ *context_len)),
      Term::Abs(info, hint, body) => Ok(Term::Abs(
        *info,
        hint.clone(),
        Box::new(walk(variable, replacement, cutoff + 1, body)?),
      )),
      Term::App(info, function, argument) => Ok(Term::App(
        *info,
        Box::new(walk(variable, replacement, cutoff, function)?),
        Box::new(walk(variable, replacement, cutoff, argument)?),
      )),
    }
  }

  walk(variable, replacement, 0, term)
}

```

用项 s 替换项 t 中编号为 j 的变量，即 $[j \mapsto s]t$ ，这里写成 `termSubst j s t`。与原始替换定义唯一的差别是：这里到达 `TmVar` 分支时一次完成 s 所需的全部移位，而不是每穿过一个绑定子就把 s 上移一位。因此，`walk` 的每次调用中参数 j 都相同，可以从内部定义中省去。

读者可能已经注意到，`termShift` 与 `termSubst` 的定义非常相似，只有到达变量时采取的动作不同。本书网站提供的 `untyped` 实现利用了这一点，把移位和替换都表述成一个更一般函数 `tmmmap` 的特例。给定项 t 和函数 `onvar`，`tmmmap onvar t` 会遍历 t ，并保持它的整体结构不变；每遇到一个变量结点，就把该结点的信息传给 `onvar`，再用 `onvar` 返回的项取代原变量结点。在更大型的演算中，这种记法技巧能省去大量乏味的重复；§25.2 会进一步说明。

在 lambda 演算的操作语义中，唯一使用替换的地方是 β 归约规则。如前所述，这条规则包含三个步骤：先把将要代入函数体的项 s 上移一位，再用它替换受约束变量；替换完成后，将整个结果下移一位，删除该变量原先占用的上下文位置。下面的定义封装了这一系列步骤：

```
let termSubstTop s t =
  termShift (-1) (termSubst 0 (termShift 1 s) t)
```

Rust 对照实现 (译者增补)

```
pub fn term_substitute_top(replacement: &Term, body: &Term) -> Result<Term,
↳ EvalError> {
  // 为即将消去的参数绑定腾出一个索引位置。
  let lifted = term_shift(1, replacement)?;
  let substituted = term_substitute(0, &lifted, body)?;
  // 替换完成后, 移除这个参数绑定占用的索引位置。
  term_shift(-1, &substituted)
}
```

7.3 求值

与第四章一样, 求值函数依赖一个辅助谓词 isval:

```
let rec isval ctx t = match t with
  TmAbs(_,_,_) -> true
  | _ -> false
```

Rust 对照实现 (译者增补)

```
pub const fn is_value(_context: &Context, term: &Term) -> bool {
  matches!(term, Term::Abs(_, _, _))
}
```

单步求值函数直接转写求值规则, 只是把上下文 ctx 与项一同传入。当前的 eval1 并不使用这个参数, 但后面一些更复杂的求值器需要它。

```
let rec eval1 ctx t = match t with
  TmApp(fi, TmAbs(_, x, t12), v2) when isval ctx v2 ->
    termSubstTop v2 t12
  | TmApp(fi, v1, t2) when isval ctx v1 ->
    let t2' = eval1 ctx t2 in
    TmApp(fi, v1, t2')
  | TmApp(fi, t1, t2) ->
    let t1' = eval1 ctx t1 in
    TmApp(fi, t1', t2)
  | _ ->
    raise NoRuleApplies
```

Rust 对照实现 (译者增补)

```

pub fn eval1(context: &Context, term: &Term) -> Result<Term, EvalError> {
    match term {
        Term::App(_, function, argument)
            if matches!(function.as_ref(), Term::Abs(_, _, _)) &&
↪ is_value(context, argument) =>
        {
            let Term::Abs(_, _, body) = function.as_ref() else {
                unreachable!();
            };
            term_substitute_top(argument, body)
        }
        Term::App(info, function, argument) if is_value(context, function) =>
↪ Ok(Term::App(
            *info,
            function.clone(),
            Box::new(eval1(context, argument)?),
        )),
        Term::App(info, function, argument) => Ok(Term::App(
            *info,
            Box::new(eval1(context, function)?),
            argument.clone(),
        )),
        Term::Var(_, _, _) | Term::Abs(_, _, _) => Err(EvalError::NoRuleApplies),
    }
}

```

多步求值函数仍与以前一样，只是多了 `ctx` 参数：

```

let rec eval ctx t =
    try
        let t' = eval1 ctx t in
        eval ctx t'
    with NoRuleApplies -> t

```

Rust 对照实现 (译者增补)

```
pub fn eval(context: &Context, term: Term) -> Result<Term, EvalError> {
    match eval1(context, &term) {
        Ok(next) => eval(context, next),
        Err(EvalError::NoRuleApplies) => Ok(term),
        Err(error) => Err(error),
    }
}
```

练习 7.3.1 (推荐, ★★★, ⇨). 把这个实现改为使用[练习5.3.8](#)引入的“大步”求值风格。

[查看练习 7.3.1 的译者参考解答](#)

7.4 注记

本章给出的替换处理足以满足全书需要,但绝不是关于这个问题的最终结论。尤其是,我们的求值器采用“急切”替换:执行 β 归约时,它会立即遍历函数体,把其中由当前抽象绑定的参数变量的每一处出现都替换为参数值。注重速度而非简洁性的函数式语言解释器(和编译器)会采用另一种策略:不预先改写函数体,而是在一个名为环境(*environment*)的辅助数据结构中记录受约束变量名与参数值之间的关联,并在求值时让环境随项一起传递;当实际需要计算函数体中的某个变量引用时,才从当前环境中取出与它关联的值。

这种基于环境的策略也可以形式化为显式替换。具体做法是扩展对象语言(这里指正在研究的 lambda 演算)的项语法,使“一个项连同尚未执行的替换”本身也成为一种项。这样,替换就不再只是元语言(这里指用来描述或实现该演算的数学记法与实现语言)施加于项的外部操作,而改由对象语言自身的求值规则逐步处理。Abadi、Cardelli、Curien 和 Lévy (1991a) 最早研究了显式替换;此后,它已经成为一个活跃的研究领域。

能把一样东西实现出来,并不代表你真正理解了它。

Just because you've implemented something doesn't mean you understand it.

——Brian Cantwell Smith

第二部分

简单类型

第八章 带类型的算术表达式

第三章曾用一种简单的布尔与算术表达式语言，介绍精确描述语法和求值的基本工具。现在我们回到这个简单语言，为它增添静态类型。这里的类型系统本身仍然简单得近乎平凡，却很适合用来引入全书将反复出现的概念。

8.1 类型

先回顾算术表达式的语法：

$t ::=$	<code>true</code>	项：常量真
	<code>false</code>	常量假
	<code>if t then t else t</code>	条件表达式
	<code>0</code>	常量零
	<code>succ t</code>	后继
	<code>pred t</code>	前驱
	<code>iszero t</code>	零测试

第三章已经说明，项的求值可能得到一个值

$v ::=$	<code>true</code>	值：真值
	<code>false</code>	假值
	<code>nv</code>	数值
$nv ::=$	<code>0</code>	数值：零值
	<code>succ nv</code>	后继值

也可能在某个阶段卡住，例如到达 `pred false`，此时没有任何求值规则适用。¹

卡住的项对应无意义或错误的程序。因此，我们希望不必真正求值，就能判断某个项的求值必定不会卡住。为此，需要区分结果为数值的项——只有它们才能作为 `pred`、`succ` 和 `iszero` 的参数——与结果为布尔值的项——只有它们才能充当条件表达式的守卫。我们引入 `Nat` 和 `Bool` 两种类型，按这种方式对项分类。全书将用元变量 S, T, U 等表示类型。

说“项 t 具有类型 T ”（或“ t 属于 T ”“ t 是 T 的元素”），意思是仅凭静态检查就能看出： t 的求值结果将具有类型 T 所规定的形式。也就是说，无需实际求值 t 即可作出这一判断。例如，`if true then false else true` 具有类型 `Bool`，而 `pred (succ (pred (succ 0)))` 具有类型 `Nat`。

¹本章研究的是带类型的布尔值与自然数演算（见图8-2），相应的 OCaml 实现名为 `tyarith`。

不过，这套类型分析是保守的：它只检查项的静态结构，不会先对项求值，以判断哪些分支实际会被执行。只要无法仅凭静态信息确认一个项不会卡住，类型系统就不会为它给出类型，即使它实际求值时不会卡住。例如，`if (iszero 0) then 0 else false`，甚至 `if true then 0 else false` 的求值事实上都不会卡住，但按照这套类型规则，我们仍然无法为它们推导出类型。

8.2 类型关系

算术表达式的类型关系 (*typing relation*) 写作 $t : T$ ，由一组为项指派类型的推导规则定义，汇总于图8-1 和 图8-2。² 与第三章一样，我们把布尔值与自然数的规则分列在两幅图中，因为后文有时需要分别引用它们。

B (带类型)

扩展 B (图3-1)

新增句法形式

$T ::= \text{Bool}$ 布尔值类型

新增类型规则

判断形式： $t : T$ 项 t 具有类型 T

$$\begin{array}{c} \frac{}{\text{true} : \text{Bool}} \text{T-TRUE} \qquad \frac{}{\text{false} : \text{Bool}} \text{T-FALSE} \\[10pt] \frac{t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \text{T-IF} \end{array}$$

图 8-1: 布尔值的类型规则 (B)

N (带类型)

扩展 NB (图3-2) 与 B

新增句法形式

$T ::= \dots$ 类型
| Nat 自然数类型

新增类型规则

判断形式： $t : T$ 项 t 具有类型 T

$$\begin{array}{c} \frac{}{0 : \text{Nat}} \text{T-ZERO} \qquad \frac{t_1 : \text{Nat}}{\text{succ } t_1 : \text{Nat}} \text{T-SUCC} \\[10pt] \frac{t_1 : \text{Nat}}{\text{pred } t_1 : \text{Nat}} \text{T-PRED} \qquad \frac{t_1 : \text{Nat}}{\text{iszero } t_1 : \text{Bool}} \text{T-ISZERO} \end{array}$$

图 8-2: 自然数的类型规则 (NB)

图8-1 中的 T-TRUE 和 T-FALSE 把类型 `Bool` 指派给布尔常量 `true` 和 `false`。T-IF 根据条件表达式各个子表达式的类型，为整个表达式指派类型：守卫 t_1 必须求值为布尔值， t_2 与 t_3 则必须求值为同一类型的值。规则两次使用同一个元变量 T ，表达了两项约束：条件表达式的结果类型就是 `then` 与 `else` 分支的类型，并且这个类型可以是任意类型——目前可以是 `Nat` 或 `Bool`，以后面对类型集合更丰富的演算时，也可以是其他类型。

图8-2 中自然数的规则形式相似。T-ZERO 为常量 `0` 指派类型 `Nat`。只要 t_1 具有类型 `Nat`，T-SUCC 就为 `succ  $t_1$` 指派类型 `Nat`。类似地，T-PRED 与 T-ISZERO 说明：参数具有类型 `Nat` 时，`pred` 产生 `Nat`，而 `iszero` 产生 `Bool`。

²符号 $\in$ 也经常代替冒号使用。

定义 8.2.1. 形式地说，算术表达式的类型关系是项与类型之间满足图8-1 和 图8-2 中所有规则实例的最小二元关系。若存在某个 T 使得 $t : T$ ，则称项 t 可赋型 (*typable*) (或良类型)。

对类型关系进行推理时，我们常会用到这样的事实：“若项 $\text{succ } t_1$ 能够通过类型检查，那么它的类型只能是 Nat ，并且其参数 t_1 也必须具有 Nat 类型。”下面的反演引理 (*inversion lemma*) 汇集了这一类基本事实；其中每一项都可以直接从相应类型规则的形式看出。

引理 8.2.2 (类型关系的反演).

1. 若 $\text{true} : R$ ，则 $R = \text{Bool}$ 。
2. 若 $\text{false} : R$ ，则 $R = \text{Bool}$ 。
3. 若 $\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : R$ ，则 $t_1 : \text{Bool}$ 、 $t_2 : R$ 且 $t_3 : R$ 。
4. 若 $0 : R$ ，则 $R = \text{Nat}$ 。
5. 若 $\text{succ } t_1 : R$ ，则 $R = \text{Nat}$ 且 $t_1 : \text{Nat}$ 。
6. 若 $\text{pred } t_1 : R$ ，则 $R = \text{Nat}$ 且 $t_1 : \text{Nat}$ 。
7. 若 $\text{iszero } t_1 : R$ ，则 $R = \text{Bool}$ 且 $t_1 : \text{Nat}$ 。

证明. 由类型关系的定义立即得到。 □

反演引理有时也称为类型关系的生成引理 (*generation lemma*): 给定一个已经成立的类型判断，它可以反推出该判断的类型推导最后可能使用了哪条规则，以及该规则的前提必须满足什么条件。反演引理直接导出一个递归算法来计算项的类型，因为它针对每一种句法形式，都告诉我们怎样从子项的类型计算整个项的类型（如果存在）。第九章会详细回到这一点。

练习 8.2.3 ($\star, \rightarrow$). 证明：良类型项的每个子项都是良类型的。

查看练习 8.2.3 的译者参考解答

§3.5 引入了求值推导的概念。类似地，类型推导 (*typing derivation*) 是由类型规则实例构成的树。类型关系中的每一对 (t, T) 都有一棵以 $t : T$ 为结论的类型推导树作为依据。例如，类型判断 $\text{if } (\text{iszero } 0) \text{ then } 0 \text{ else } (\text{pred } 0) : \text{Nat}$ 的推导树是：

$$\begin{array}{c}
 \frac{}{0 : \text{Nat}} \text{ T-ZERO} \quad \frac{}{0 : \text{Nat}} \text{ T-ZERO} \\
 \frac{}{\text{iszero } 0 : \text{Bool}} \text{ T-ISZERO} \quad \frac{}{0 : \text{Nat}} \text{ T-ZERO} \quad \frac{}{\text{pred } 0 : \text{Nat}} \text{ T-PRED} \\
 \hline
 \text{if } (\text{iszero } 0) \text{ then } 0 \text{ else } (\text{pred } 0) : \text{Nat} \quad \text{T-IF}
 \end{array}$$

换言之，类型判断是关于程序类型的形式断言，类型规则表达类型判断之间的蕴涵关系，而类型推导树记录以这些规则为依据的演绎过程。

定理 8.2.4 (类型的唯一性). 每个项 t 至多具有一种类型。也就是说, 如果 t 可赋型, 它的类型是唯一的。此外, 由图 8-1 和图 8-2 中的推理规则构成、以这一类型判断为结论的类型推导树也是唯一的。

证明. 对 t 作直接的结构归纳。每种情形使用反演引理的相应条目和归纳假设。 $\square$

在本章这个简单类型系统里, 每个项只具有一种类型 (如果它确实具有类型), 而且证明该类型判断的推导树也只有一棵。以后, 这两项性质都会放宽。例如, 到 第十五章 讨论带子类型化的类型系统时, 同一个项可能具有多种类型, 而同一个类型判断通常也可能有多棵不同的推导树。

类型关系的性质常常通过对推导树作归纳来证明, 正如求值关系的性质通常通过对求值推导作归纳来证明。从下一节开始, 我们会看到许多对类型推导作归纳的例子。

8.3 安全性等于进展性加保持性

这个类型系统——以及其他任何类型系统——最基本的性质是 安全性 (*safety*) (也称可靠性 (*soundness*)): 良类型项不会“出错”。我们已经选定了“出错”的形式含义: 到达一个不是最终值、求值规则却没有说明下一步该怎么做的“卡住状态” (定义 3.5.15)。因此, 我们希望证明良类型项不会卡住。证明分为两个步骤, 通常称为进展性 (*progress*) 定理和保持性 (*preservation*) 定理。³

进展性

良类型项不会卡住: 它要么是值, 要么能按求值规则迈进一步。

保持性

良类型项经过一次单步求值后, 所得的项仍然是良类型的。⁴

两项性质合起来告诉我们: 良类型项在求值过程中永远不会到达卡住状态。

证明进展性定理之前, 先说明 `Bool` 与 `Nat` 类型的良类型值分别只能采用哪些形式; 这样的良类型值称为相应类型的典范形式 (*canonical forms*)。

引理 8.3.1 (典范形式).

1. 若 v 是类型为 `Bool` 的值, 则 v 是 `true` 或 `false`。
2. 若 v 是类型为 `Nat` 的值, 则 v 是 图 3-2 文法所定义的数值。

³“安全性等于进展性加保持性”(并使用一个典范形式引理)这一口号由 Harper 明确提出; Wright 和 Felleisen (1994) 提出了一个变体。

⁴在本书考察的大多数类型系统中, 求值不仅保持良类型性, 也保持项的精确类型。不过, 有些系统允许类型在求值期间改变。例如, 在带子类型化的系统 (第十五章) 中, 项的类型可能在求值时变得更小, 也就是包含更多信息。

证明. 对第 1 项, 根据图3-1 和图3-2 的文法, 这个语言中的值可能具有四种形式: `true`、`false`、`0` 和 `succ nv`, 其中 nv 为数值。前两种立即给出所需结论。后两种不可能出现, 因为已经假定 v 的类型是 `Bool`, 而反演引理第 4、5 项告诉我们, `0` 和 `succ nv` 只能具有类型 `Nat`, 不能具有类型 `Bool`。第 2 项类似。□

定理 8.3.2 (进展性). 设 t 是良类型项, 即对某个 T 有 $t : T$ 。那么, t 要么是值, 要么存在某个 t' 使 $t \rightarrow t'$ 。

证明. 对 $t : T$ 的推导作归纳。T-TRUE、T-FALSE 和 T-ZERO 的情形立即成立, 因为这些情形下 t 是值。其余情形如下。

情形 T-IF:

$$t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \quad t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T$$

整个项 t 是条件表达式, 因而不是值; 所以这里只需证明它能够迈进一步。对前提 $t_1 : \text{Bool}$ 的类型推导使用归纳假设可知, t_1 要么是值, 要么存在 t'_1 使 $t_1 \rightarrow t'_1$ 。若 t_1 是值, 典范形式引理 (引理8.3.1) 保证它必为 `true` 或 `false`。前一种情况下, E-IFTRUE 给出 $t \rightarrow t_2$; 后一种情况下, E-IFFALSE 给出 $t \rightarrow t_3$ 。若 $t_1 \rightarrow t'_1$, 则由 E-IF,

$$\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3$$

两种情形下, 整个项 t 都能迈进一步, 因此进展性成立。

情形 T-SUCC:

$$t = \text{succ } t_1 \quad t_1 : \text{Nat}$$

由归纳假设, t_1 要么是值, 要么存在 t'_1 使 $t_1 \rightarrow t'_1$ 。若 t_1 是值, 典范形式引理说明它必为数值, 于是 t 也是数值。另一方面, 若 $t_1 \rightarrow t'_1$, 则由 E-SUCC, $\text{succ } t_1 \rightarrow \text{succ } t'_1$ 。

情形 T-PRED:

$$t = \text{pred } t_1 \quad t_1 : \text{Nat}$$

由归纳假设, t_1 要么是值, 要么存在 t'_1 使 $t_1 \rightarrow t'_1$ 。若 t_1 是值, 典范形式引理说明它必为数值, 即要么是 `0`, 要么对某个 nv_1 是 `succ nv_1`; 于是 E-PREDZERO 或 E-PREDSUCC 适用于 t 。另一方面, 若 $t_1 \rightarrow t'_1$, 则由 E-PRED, $\text{pred } t_1 \rightarrow \text{pred } t'_1$ 。

情形 T-ISZERO: 类似。□

对这个系统来说, 求值保持类型的证明也相当直接。

定理 8.3.3 (保持性). 若 $t : T$ 且 $t \rightarrow t'$, 则 $t' : T$ 。

证明. 对 $t : T$ 的推导作归纳。在归纳的每一步, 假定所需性质对所有子推导成立 (即只要 $s : S$ 是由当前推导的某个子推导证明的, 而且 $s \rightarrow s'$, 就有 $s' : S$), 再对推导的最后一条规则作分类讨论。下面只列出一部分情形; 其余类似。

情形 T-TRUE:

$$t = \text{true} \quad T = \text{Bool}$$

若最后一条规则是 T-TRUE, 由该规则可知 t 必为常量 `true`, T 必为 `Bool`。但 t 是值, 所以对任何 t' 都不可能由 $t \rightarrow t'$; 定理要求因此空泛成立。

情形 T-IF:

$$t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3 \quad t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T$$

查看左端为条件表达式的求值规则 (图3-1), 发现 $t \rightarrow t'$ 可能由三条规则导出: E-IFTRUE、E-IFFALSE 与 E-IF。分别讨论如下; 省略与 E-IFTRUE 类似的 E-IFFALSE 情形。

子情形 E-IFTRUE: $t_1 = \text{true}$ 且 $t' = t_2$ 。当前类型推导的最后一步是 T-IF, 因此该规则的第二个前提给出 $t_2 : T$ 。又因 $t' = t_2$, 所以 $t' : T$, 证明完成。

子情形 E-IF: 当前类型推导的最后一步是 T-IF, 因此该规则的三个前提给出 $t_1 : \text{Bool}$ 、 $t_2 : T$ 与 $t_3 : T$ 。在这个子情形中, E-IF 说明: 对某个 t'_1 , 有 $t_1 \rightarrow t'_1$, 并且 $t' = \text{if } t'_1 \text{ then } t_2 \text{ else } t_3$ 。把归纳假设用于 $t_1 : \text{Bool}$ 和 $t_1 \rightarrow t'_1$, 得到 $t'_1 : \text{Bool}$ 。两个分支没有改变, 仍有 $t_2 : T$ 与 $t_3 : T$; 再次应用 T-IF, 便得到 $\text{if } t'_1 \text{ then } t_2 \text{ else } t_3 : T$ 。这个条件表达式正是 t' , 所以 $t' : T$ 。

情形 T-ZERO:

$$t = 0 \quad T = \text{Nat}$$

不可能发生, 理由与上面的 T-TRUE 情形相同。

情形 T-SUCC:

$$t = \text{succ } t_1 \quad T = \text{Nat} \quad t_1 : \text{Nat}$$

当前类型推导的最后一步是 T-SUCC, 因此它的前提给出 $t_1 : \text{Nat}$, 而结论中的类型 T 就是 `Nat`。另一方面, 定理假设 $t \rightarrow t'$ 。检查 图3-2 的求值规则可知, 只有 E-SUCC 能让 $\text{succ } t_1$ 迈进一步。因此, 对某个 t'_1 , 必有 $t_1 \rightarrow t'_1$, 并且 $t' = \text{succ } t'_1$ 。

把归纳假设用于 $t_1 : \text{Nat}$ 和 $t_1 \rightarrow t'_1$, 得到 $t'_1 : \text{Nat}$ 。再次应用 T-SUCC, 便有 $\text{succ } t'_1 : \text{Nat}$ 。这个项正是 t' , 而 $\text{Nat} = T$, 所以 $t' : T$ 。□

练习 8.3.4 (★★, $\rightarrow$)。重组上述证明, 使其对求值推导而非类型推导作归纳。

查看练习 8.3.4 的译者参考解答

保持性定理常称为主体归约 (*subject reduction*) (或 主体求值 (*subject evaluation*))。其直觉是把类型判断 $t : T$ 看成句子 “ t 具有类型 T ”; 项 t 是这个句子的主体。主体归约性质说明, 把 t 归约为 t' 后, 相应的类型判断 $t' : T$ 仍然成立。

类型的唯一性在一些类型系统中成立, 在另一些系统中不成立; 相比之下, 进展性与保持性将是本书所考察的所有类型系统的基本要求。⁵

练习 8.3.5 (★)。求值规则 E-PREDZERO (图3-2) 有些违反直觉: 我们可能觉得零的前驱应当没有定义, 而不是定义为零。只需从单步求值的定义中删去这条规则, 就能达到这个目的吗?

⁵有些语言不满足这些性质, 却仍然可以视为类型安全。例如, 若采用小步风格形式化 Java 的操作语义 (Flatt、Krishnamurthi 和 Felleisen, 1998a; Igarashi、Pierce 和 Wadler, 1999), 这里给出的保持性形式会失效 (详见第十九章)。不过, 这应视为形式化方式造成的现象, 而不是语言本身的缺陷; 例如改用大步语义呈现时, 它便会消失。

[查看原书附录 A 中的 8.3.5 解答](#)

练习 8.3.6 (推荐, ★★). 看到主体归约性质之后, 自然会问相反的性质——主体扩张 (*subject expansion*) ——是否也成立。若 $t \rightarrow t'$ 且 $t' : T$, 是否总有 $t : T$? 若是, 请证明; 若不是, 请给出反例。

[查看原书附录 A 中的 8.3.6 解答](#)

练习 8.3.7 (推荐, ★★). 假设求值关系采用[练习3.5.17](#) 那样的大步风格定义。类型安全性的直觉性质应如何形式化?

[查看原书附录 A 中的 8.3.7 解答](#)

练习 8.3.8 (推荐, ★★). 假设按照[练习3.5.16](#), 在求值关系中加入把无意义项归约到显式 `wrong` 状态的规则。此时类型安全性又应如何形式化?

[查看原书附录 A 中的 8.3.8 解答](#)

从无类型宇宙通向有类型宇宙的道路, 已在许多不同领域走过许多次, 而且大体都是出于同样的理由。

The road from untyped to typed universes has been followed many times, in many different fields, and largely for the same reasons.

Luca Cardelli 与 Peter Wegner (1985)

第九章 简单类型 lambda 演算

本章介绍本书余下部分将研究的有类型语言家族中最基本的成员：Church（1940）与 Curry（1958）的简单类型 lambda 演算（*simply typed lambda-calculus*）。

9.1 函数类型

第八章为算术表达式引入了一个简单的静态类型系统，其中有两种类型：Bool 对求值结果为布尔值的项分类，Nat 对求值结果为数字的项分类。不属于任何一种类型的“病类型”项，既包括求值时到达卡住状态的所有项，例如 `if 0 then 1 else 2`，也包括一些求值行为其实正常、只是静态分类过于保守而无法接受的项，例如 `if true then 0 else false`。

现在设想为一种语言构造类似的类型系统：它把布尔值与纯 lambda 演算的基本构件组合起来；为简洁起见，本章忽略自然数。也就是说，我们要为变量、抽象与应用引入类型规则，并要求这些规则：

- (a) 维持类型安全性，即满足 进展性定理8.3.2和 类型保持性定理8.3.3；
- (b) 不至于过分保守，即应当能为我们真正关心的大多数程序指派类型。

但是，纯 lambda 演算能够表达任意计算，其中也包括可能永不终止的计算。因此，一个自身保证终止的静态类型检查器，不可能对由这些基本构件组成的所有程序都作出完全精确的类型判断；它只能采用保守的近似分析。例如，不实际运行下面程序中的漫长而棘手的计算，观察它究竟产生 `true` 还是 `false`，就无法可靠判断整个程序产生的是布尔值还是函数：

`if <漫长而棘手的计算> then true else ( $\lambda x.x$ )`

此外，这段漫长而棘手的计算甚至可能永不终止（即发散）；如果类型检查器试图通过实际运行它来精确预测结果，类型检查器本身也会一直运行下去。¹

为了把函数纳入布尔值的类型系统，显然需要增加一种类型，对求值结果为函数的项分类。作为第一次近似，姑且把它记作 $\rightarrow$ 。若加入类型规则

$\lambda x.t : \rightarrow$

给每个 lambda 抽象都指派类型 $\rightarrow$ ，那么无论是 $\lambda x.x$ 这样的简单项，还是下面这样的复合项，都能归为产生函数的项：

`if true then ( $\lambda x.\text{true}$ ) else ( $\lambda x.\lambda y.y$ )`

¹本章研究的是带布尔值（图8-1）的简单类型 lambda 演算（图9-1）。相应的 OCaml 实现为 `fullsimple`。

$\lambda_{\rightarrow}$ [带类型]基于 λ (图5-3)

$t ::= x$	项: 变量	$\frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2}$ E-APP1
$\lambda x:T. t$	抽象	
$t t$	应用	
$v ::= \lambda x:T. t$	值: 抽象值	$\frac{t_2 \longrightarrow t'_2}{v_1 t_2 \longrightarrow v_1 t'_2}$ E-APP2
$T ::= T \rightarrow T$	类型: 函数类型	
$\Gamma ::= \emptyset$	上下文: 空上下文	$\frac{}{(\lambda x:T_{11}. t_{12}) v_2 \longrightarrow [x \mapsto v_2] t_{12}}$ E-APPAbs
$\Gamma, x:T$	项变量绑定	
$\Gamma \vdash t:T$ 判断形式: 在上下文 Γ 下, 项 t 具有类型 T		
$\frac{x:T \in \Gamma}{\Gamma \vdash x:T}$ T-VAR $\frac{\Gamma, x:T_1 \vdash t_2:T_2}{\Gamma \vdash \lambda x:T_1. t_2:T_1 \rightarrow T_2}$ T-ABS		
$\frac{\Gamma \vdash t_1:T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2:T_{11}}{\Gamma \vdash t_1 t_2:T_{12}}$ T-APP		

浅灰区域 表示相较于图5-3 新增或修改的部分。

图 9-1: 纯简单类型 lambda 演算 ($\lambda_{\rightarrow}$)

但这种粗略分析显然太保守: $\lambda x.\text{true}$ 和 $\lambda x.\lambda y.y$ 被混入同一种类型 $\rightarrow$, 忽略了前者应用于 true 得到布尔值, 而后者应用于 true 得到另一个函数。通常, 为了给函数应用的结果确定一个有用的类型, 仅知道被应用的项是函数还不够; 我们还必须知道该函数返回什么类型。为了确保调用时不会因参数类型不符而出错, 还必须记录它接受什么类型的参数。为此, 我们不再用单独的类型 $\rightarrow$ 统称所有函数, 而把函数类型写成 $T_1 \rightarrow T_2$, 表示接受 T_1 型参数、返回 T_2 型结果的函数。由于 T_1 和 T_2 本身也可以是函数类型, 这种构造可以无限嵌套。

定义 9.1.1. 以类型 Bool 为基础生成的简单类型集合由以下文法定义:

$$\begin{aligned}
 T &::= \text{Bool} && \text{类型: 布尔值的类型} \\
 &| T \rightarrow T && \text{函数类型}
 \end{aligned}$$

类型构造子 $\rightarrow$ 右结合, 即 $T_1 \rightarrow T_2 \rightarrow T_3$ 表示 $T_1 \rightarrow (T_2 \rightarrow T_3)$ 。

例如, $\text{Bool} \rightarrow \text{Bool}$ 是把布尔参数映射到布尔结果的函数类型。而

$$(\text{Bool} \rightarrow \text{Bool}) \rightarrow (\text{Bool} \rightarrow \text{Bool}),$$

等价地,

$$(\text{Bool} \rightarrow \text{Bool}) \rightarrow \text{Bool} \rightarrow \text{Bool},$$

则是以“布尔到布尔”的函数为参数、又把这种函数作为结果返回的函数类型。

9.2 类型关系

要为 $\lambda x.t$ 这样的抽象指派类型, 就需要计算这个抽象应用于某个参数时会发生什么。接下来的问题是: 怎样知道它预期接收什么类型的参数? 有两种回答。可以直接在 lambda 抽象上标注预期的参数类型; 也可以分析抽象体中怎样使用参数, 据此推断参数应有的类型。眼下我们选择第一种。我们不再只写 $\lambda x.t$, 而写 $\lambda x : T_1. t_2$, 受约束变量上的标注表示应当假定参数具有类型 T_1 。

一般地, 若语言在项中使用类型标注来引导类型检查器, 就称为 显式类型 (*explicitly typed*) 语言。若要求类型检查器推断或重建这些信息, 就称为隐式类型 (*implicitly typed*) 语言; 在 lambda 演算文献中, 这类隐式类型系统也称为 类型指派系统 (*type-assignment system*)。本书主要研究显式类型语言; 第二十二章将考察隐式类型。

一旦知道抽象的参数类型, 函数的结果类型显然就是抽象体 t_2 的类型, 其中 t_2 中出现的 x 被假定为表示 T_1 型的项。这一关系可以写成下面的类型规则:

$$\frac{x : T_1 \vdash t_2 : T_2}{\vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2} \text{ T-ABS}$$

lambda 抽象可以层层嵌套。检查内层抽象体时, 往往需要同时记录多个处于作用域中的变量及其类型。例如, 在 $\lambda x : T_1. \lambda y : T_2. t$ 中, 检查 t 时必须同时假定 $x : T_1$ 与 $y : T_2$ 。因此, 类型关系由二元关系 $t : T$ 扩展为三元关系 $\Gamma \vdash t : T$, 其中 Γ 集中记录 t 中自由变量的类型假设。

形式地说, 类型上下文 (*typing context*) (也称 类型环境 (*type environment*)) Γ 是变量及其类型的序列; 逗号运算符在右侧加入新绑定来扩展 Γ 。空上下文有时写作 $\emptyset$, 但通常直接省略; 例如 $\vdash t : T$ 表示“闭项 t 在空假设集下具有类型 T ”。

为了避免新绑定与 Γ 中已有的绑定混淆, 要求所选的名字 x 不同于 Γ 已经绑定的变量。根据约定, lambda 抽象所绑定的变量可以在方便时重命名, 所以需要时总能通过重命名受约束变量满足这个条件。因此, 可以把 Γ 看作从变量到类型的有限函数, 并用 $\text{dom}(\Gamma)$ 表示 Γ 所绑定的变量集合。

抽象的类型规则一般写作:

$$\frac{\Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2} \text{ T-ABS},$$

与结论相比, 前提的类型上下文多了假设 $x : T_1$: 检查抽象体 t_2 时, 需要暂时假定参数 x 具有类型 T_1 。

变量的类型规则也直接由上述讨论得到: 变量具有当前假定它具有的类型。

$$\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \text{ T-VAR}$$

前提 $x : T \in \Gamma$ 读作“ Γ 中为 x 假定的类型是 T ”。

最后还需要应用的类型规则：

$$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}} \text{ T-APP}$$

第一个前提表示：在 Γ 记录的自由变量类型假设下， t_1 具有函数类型 $T_{11} \rightarrow T_{12}$ ；第二个前提表示，在同样的假设下， t_2 具有类型 T_{11} 。因此，应用 $t_1 t_2$ 具有类型 T_{12} 。

布尔常量与条件表达式的类型规则与以前相同（图8-1）。不过，T-IF 中的元变量 T 现在可以实例化为任意函数类型，所以分支为函数的条件表达式也能赋型：

$$\begin{aligned} & \text{if true then } (\lambda x : \text{Bool}. x) \text{ else } (\lambda x : \text{Bool}. \text{not } x) \\ & \quad \blacktriangleright (\lambda x : \text{Bool}. x) : \text{Bool} \rightarrow \text{Bool} \end{aligned}$$

2

上述规则汇总在图9-1 中；为求完整，图中也列出了语法与求值规则。图中高亮区域表示相较于无类型 lambda 演算新增或修改的部分：有些是全新的规则，有些是在原有规则中加入的内容。与布尔值和自然数一样，完整演算的定义被拆成两部分：一部分是图中没有任何基本类型的纯简单类型 lambda 演算；另一部分是图8-1 已给出的布尔值规则——当然，必须为该图的每个类型判断都加上上下文 Γ 。

本书常用符号 $\lambda \rightarrow$ 泛指简单类型 lambda 演算；即使具体系统选用的基本类型不同，也仍沿用这一符号。

练习 9.2.1 ($\star$). 完全没有基本类型的纯简单类型 lambda 演算实际上是退化的，因为它根本没有良类型项。为什么？

[查看原书附录 A 中的 9.2.1 解答](#)

$\lambda \rightarrow$ 的类型规则实例可以像带类型的算术表达式一样组合成推导树。例如，下面的推导说明 $(\lambda x : \text{Bool}. x) \text{ true}$ 在空上下文具有类型 Bool ：

$$\frac{\frac{\frac{x : \text{Bool} \in x : \text{Bool}}{\Gamma \vdash x : \text{Bool}} \text{ T-VAR} \quad \frac{}{\Gamma \vdash \text{true} : \text{Bool}} \text{ T-TRUE}}{\Gamma \vdash \lambda x : \text{Bool}. x : \text{Bool} \rightarrow \text{Bool}} \text{ T-ABS} \quad \frac{}{\Gamma \vdash (\lambda x : \text{Bool}. x) \text{ true} : \text{Bool}} \text{ T-APP}$$

练习 9.2.2 ($\star, \rightarrow$). 画出推导树，证明下列项具有所标类型：

1. $f : \text{Bool} \rightarrow \text{Bool} \vdash f \text{ (if false then true else false)} : \text{Bool}$;
2. $f : \text{Bool} \rightarrow \text{Bool} \vdash \lambda x : \text{Bool}. f \text{ (if } x \text{ then false else } x) : \text{Bool} \rightarrow \text{Bool}$.

[查看练习 9.2.2 的译者参考解答](#)

练习 9.2.3 ($\star$). 找出一个上下文 Γ ，使得 $f x y$ 具有类型 Bool 。能否简洁描述所有这类上下文组成的集合？

[查看原书附录 A 中的 9.2.3 解答](#)

²从这里开始，展示实现交互的例子会同时给出结果及其类型；显而易见时有时会省略。

9.3 类型关系的性质

与第八章一样，证明类型安全性之前需要建立几个基本引理。其中大部分与前面类似：只需在类型关系中加入上下文，并在每个证明中增加对 lambda 抽象、应用和变量的处理。唯一实质上新增的要求，是类型关系的替换引理（引理9.3.8）。

首先，反演引理记录类型推导怎样构成：针对每一种句法形式，它说明“若这种形式的项是良类型的，那么它的子项必定具有如下形式的类型……”

引理 9.3.1 (类型关系的反演).

1. 若 $\Gamma \vdash x : R$ ，则 $x : R \in \Gamma$ 。
2. 若 $\Gamma \vdash \lambda x : T_1. t_2 : R$ ，则存在某个 R_2 ，使 $R = T_1 \rightarrow R_2$ ，且 $\Gamma, x : T_1 \vdash t_2 : R_2$ 。
3. 若 $\Gamma \vdash t_1 t_2 : R$ ，则存在某个类型 T_{11} ，使 $\Gamma \vdash t_1 : T_{11} \rightarrow R$ ，且 $\Gamma \vdash t_2 : T_{11}$ 。
4. 若 $\Gamma \vdash \text{true} : R$ ，则 $R = \text{Bool}$ 。
5. 若 $\Gamma \vdash \text{false} : R$ ，则 $R = \text{Bool}$ 。
6. 若 $\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : R$ ，则 $\Gamma \vdash t_1 : \text{Bool}$ ，而且 $\Gamma \vdash t_2 : R$ 与 $\Gamma \vdash t_3 : R$ 。

证明. 由类型关系的定义立即得到。 □

练习 9.3.2 (推荐, ★★★). 是否存在上下文 Γ 和类型 T ，使 $\Gamma \vdash x x : T$ ？若存在，请给出 Γ 和 T ，并画出该类型判断的推导树；若不存在，请证明。

[查看原书附录 A 中的 9.3.2 解答](#)

§9.2 为了简化类型检查选择了显式类型的演算呈现方式。为此，我们只给函数抽象中的受约束变量添加类型标注，其他地方均未标注。这在什么意义上“已经足够”？一种回答来自类型唯一性定理：良类型项与其类型推导一一对应——可以从项唯一地恢复类型推导，反之当然也可以。事实上，这种对应如此直接，以至于从某种意义上说，项与推导几乎没有差别。

定理 9.3.3 (类型的唯一性). 固定类型上下文 Γ 。若项 t 的所有自由变量都属于 $\text{dom}(\Gamma)$ ，则 t 在 Γ 下至多具有一种类型。也就是说，如果 $\Gamma \vdash t : T$ 可以导出，那么 T 是唯一的；而且，依照本节类型规则构造出的这棵类型推导树也是唯一的。

证明. 留作练习（推荐，三星）。证明直接得几乎无话可说；不过，写出部分细节很适合练习如何“搭起”关于类型关系的证明。 □

[查看原书附录 A 中的 9.3.3 解答](#)

本书以后会遇到许多不再具有这种简单对应的类型系统：同一个项会被指派多种类型，而每一种类型又会由许多类型推导证明。在这些系统中，有效地从项恢复类型推导往往需要相当多的工作。

下面的典范形式引理根据一个值的类型，限定它可能是哪一种值：布尔类型的值只能是布尔常量，函数类型的值只能是 lambda 抽象。

引理 9.3.4 (典范形式).

1. 若 v 是类型为 **Bool** 的值, 则 v 是 **true** 或 **false**。
2. 若 v 是类型为 $T_1 \rightarrow T_2$ 的值, 则 $v = \lambda x : T_1. t_2$ 。

证明. 直接得到, 与 [算术表达式的典范形式引理8.3.1](#) 的证明类似。 $\square$

借助典范形式引理, 可以证明与[定理8.3.2](#)对应的进展性定理。定理陈述需要稍作改变: 这里只关心没有自由变量的闭项。对开项而言, 进展性确实不成立, 例如 $f \text{ true}$ 是范式, 却不是值。不过, 这并不代表语言有缺陷, 因为我们真正关心求值的完整程序总是闭项。

定理 9.3.5 (进展性). 设 t 是闭的良类型项, 即对某个 T 有 $\vdash t : T$ 。那么, t 要么是值, 要么存在某个 t' 使 $t \rightarrow t'$ 。

证明. 对类型推导作直接归纳。布尔常量和条件表达式的情形, 与带类型算术表达式的进展性证明 ([定理8.3.2](#)) 完全相同。变量情形不可能发生: 若类型推导最后使用 T-VAR, 则 t 本身必为某个自由变量 x , 这与 t 是闭项的假设矛盾。抽象情形立即成立, 因为抽象是值。

唯一有意思的是应用情形:

$$t = t_1 t_2, \quad \vdash t_1 : T_{11} \rightarrow T_{12}, \quad \vdash t_2 : T_{11}$$

由归纳假设, t_1 要么是值, 要么能迈进一步; t_2 也一样。若 t_1 能迈进一步, 规则 E-APP1 适用于 t 。若 t_1 是值而 t_2 能迈进一步, 规则 E-APP2 适用。最后, 若二者都是值, 典范形式引理说明 t_1 形如 $\lambda x : T_{11}. t_{12}$, 于是规则 E-APPABS 适用。 $\square$

接下来要证明求值保持类型。先陈述类型关系的两个“结构引理”。它们本身并不特别有趣, 但能让后续证明对类型推导作若干有用的变换。

第一个结构引理说明, 可以按需要置换上下文元素, 而不会改变在该上下文中能够导出的类型判断。回想上文的要求: 上下文中的所有绑定都必须有不同名字; 每次为上下文加入绑定时, 都默认新名字不同于已经绑定的全部名字, 必要时按照 [约定5.3.4](#) 重命名。

引理 9.3.6 (置换). 若 $\Gamma \vdash t : T$, 且 Δ 是 Γ 的一个置换, 则 $\Delta \vdash t : T$ 。此外, 后一个推导与前一个推导深度相同。

证明. 对类型推导作直接归纳。 $\square$

第二个结构引理是弱化 (*weakening*): 向上下文加入未被使用的新假设, 不会使原有的类型判断失效。

引理 9.3.7 (弱化). 若 $\Gamma \vdash t : T$, 且 $x \notin \text{dom}(\Gamma)$, 则 $\Gamma, x : S \vdash t : T$ 。此外, 后一个推导与前一个推导深度相同。

证明. 对类型推导作直接归纳。 $\square$

借助这些辅助引理, 可以证明类型关系的一项关键性质: 用类型适当的项替换变量时, 仍然保持良类型性。类似引理在程序语言的安全性证明中无处不在, 因而经常简称为“替换引理”。

引理 9.3.8 (替换保持类型). 若 $\Gamma, x : S \vdash t : T$, 且 $\Gamma \vdash s : S$, 则 $\Gamma \vdash [x \mapsto s]t : T$ 。

证明. 对类型判断 $\Gamma, x : S \vdash t : T$ 的推导深度作归纳, 并按推导最后使用的类型规则分类讨论。³最有意思的是变量与抽象情形。

情形 T-VAR:

$$t = z, \quad z : T \in (\Gamma, x : S)$$

要根据 z 是 x 还是另一个变量分成两个子情形。

若 $z = x$, 那么绑定 $z : T$ 就是上下文中新加入的绑定 $x : S$, 故 $T = S$ 。又因为 $[x \mapsto s]x = s$, 所需结论 $\Gamma \vdash [x \mapsto s]x : T$ 化为 $\Gamma \vdash s : S$, 这正是引理的第二个假设。

若 $z \neq x$, 那么绑定 $z : T$ 只能来自原上下文 Γ , 所以 $z : T \in \Gamma$ 。此时 $[x \mapsto s]z = z$, 再使用 T-VAR 即得所需结论 $\Gamma \vdash [x \mapsto s]z : T$ 。

情形 T-ABS:

$$t = \lambda y : T_2. t_1, \quad T = T_2 \rightarrow T_1,$$

$$\Gamma, x : S, y : T_2 \vdash t_1 : T_1$$

必要时, 先把抽象的受约束变量 y 及其在 t_1 中受该抽象约束的出现一同作 α 重命名。根据[约定5.3.4](#), 总能为它选取一个新名字, 使 $x \neq y$ 且 $y \notin \text{FV}(s)$ 。条件 $x \neq y$ 排除了这个抽象本身绑定 x 的情形; 在那种情形中, 函数体里的 x 是受约束变量, 不属于此次替换要处理的自由出现。条件 $y \notin \text{FV}(s)$ 则保证替换后, s 中的自由变量不会被这个抽象捕获。

为了把归纳假设用于函数体 t_1 , 令 $\Delta = \Gamma, y : T_2$ 。归纳假设要求先有

$$\Delta, x : S \vdash t_1 : T_1 \quad \text{和} \quad \Delta \vdash s : S$$

展开 Δ 后, 这两个前提分别是

$$\Gamma, y : T_2, x : S \vdash t_1 : T_1 \quad \text{和} \quad \Gamma, y : T_2 \vdash s : S$$

当前关于函数体的子推导给出的是 $\Gamma, x : S, y : T_2 \vdash t_1 : T_1$ 。使用置换引理调换后两个绑定的顺序, 就得到所需的第一个前提 $\Gamma, y : T_2, x : S \vdash t_1 : T_1$ 。引理的另一个假设是 $\Gamma \vdash s : S$; 使用弱化引理加入未被 s 使用的新绑定 $y : T_2$, 就得到所需的第二个前提 $\Gamma, y : T_2 \vdash s : S$ 。

函数体的类型推导是整个 T-ABS 推导的直接子推导, 深度严格更小; 置换引理又保证调换上下文顺序不会改变推导深度, 所以可以对该子推导使用归纳假设。这里, 归纳假设处理的项是 t_1 , 其类型是 T_1 , 基础上下文就是前面设定的 $\Delta = \Gamma, y : T_2$ 。上面得到的两个类型判断正好是归纳假设的两个前提, 因而可以推出

$$\Gamma, y : T_2 \vdash [x \mapsto s]t_1 : T_1$$

³等价地, 也可按 t 最外层的句法构造子分类, 因为每个句法构造子恰好只有一条类型规则。

再由 T-ABS,

$$\Gamma \vdash \lambda y : T_2. [x \mapsto s]t_1 : T_2 \rightarrow T_1$$

根据替换定义以及上面两个关于变量名不冲突的条件,

$$[x \mapsto s](\lambda y : T_2. t_1) = \lambda y : T_2. [x \mapsto s]t_1$$

再结合 $t = \lambda y : T_2. t_1$ 和 $T = T_2 \rightarrow T_1$, 这恰是所需结论 $\Gamma \vdash [x \mapsto s]t : T$ 。

情形 T-APP:

$$t = t_1 t_2, \quad \Gamma, x : S \vdash t_1 : T_2 \rightarrow T_1,$$

$$\Gamma, x : S \vdash t_2 : T_2, \quad T = T_1$$

这里 T_2 是参数类型, T_1 是函数的结果类型, 因此整个应用的类型 T 就是 T_1 。本情形需要证明

$$\Gamma \vdash [x \mapsto s](t_1 t_2) : T$$

上面关于 t_1 和 t_2 的两个类型判断来自整个 T-APP 推导的两个直接子推导, 深度都严格更小。因此可以分别使用归纳假设, 得到

$$\Gamma \vdash [x \mapsto s]t_1 : T_2 \rightarrow T_1 \quad \text{和} \quad \Gamma \vdash [x \mapsto s]t_2 : T_2$$

替换后的函数部分仍接受 T_2 型参数并返回 T_1 型结果, 而替换后的参数部分仍具有类型 T_2 。对这两个判断使用 T-APP, 得到

$$\Gamma \vdash ([x \mapsto s]t_1) ([x \mapsto s]t_2) : T_1$$

由于 $T = T_1$, 上式的结果类型也就是 T 。另一方面, 根据替换对应用项的定义,

$$[x \mapsto s](t_1 t_2) = ([x \mapsto s]t_1) ([x \mapsto s]t_2)$$

所以刚刚由 T-APP 得到的判断正是所需结论 $\Gamma \vdash [x \mapsto s](t_1 t_2) : T$ 。

情形 T-TRUE: $t = \text{true}$ 且 $T = \text{Bool}$ 。此时 $[x \mapsto s]t = \text{true}$ 。由 T-TRUE 可得 $\Gamma \vdash \text{true} : \text{Bool}$, 也就是所需结论 $\Gamma \vdash [x \mapsto s]t : T$ 。

情形 T-FALSE: $t = \text{false}$ 且 $T = \text{Bool}$ 。替换不改变布尔常量, 所以 $[x \mapsto s]t = \text{false}$ 。由 T-FALSE 可得 $\Gamma \vdash \text{false} : \text{Bool}$, 也就是所需结论。

情形 T-IF:

$$t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3,$$

$$\Gamma, x : S \vdash t_1 : \text{Bool}, \quad \Gamma, x : S \vdash t_2 : T, \quad \Gamma, x : S \vdash t_3 : T$$

三次使用归纳假设, 得到

$$\Gamma \vdash [x \mapsto s]t_1 : \text{Bool},$$

$$\Gamma \vdash [x \mapsto s]t_2 : T,$$

$$\Gamma \vdash [x \mapsto s]t_3 : T$$

对这三个判断使用 T-If, 得到

$$\Gamma \vdash \text{if } [x \mapsto s]t_1 \text{ then } [x \mapsto s]t_2 \text{ else } [x \mapsto s]t_3 : T$$

替换会分别进入条件项的三个子项, 因此上式正是 $\Gamma \vdash [x \mapsto s]t : T$ 。 $\square$

借助替换引理, 可以证明类型安全性的另一半: 求值保持良类型性。

定理 9.3.9 (保持性). 若 $\Gamma \vdash t : T$, 且 $t \longrightarrow t'$, 则 $\Gamma \vdash t' : T$ 。

证明. 留作练习 (推荐, 三星)。其结构与算术表达式的类型保持性证明 (定理8.3.3) 非常相似, 区别是这里要使用替换引理。 $\square$

[查看原书附录 A 中的 9.3.9 解答](#)

练习 9.3.10 (推荐, $\star\star\star$). 练习8.3.6 考察了简单的带类型算术表达式演算是否具有主体扩张性质。该性质对简单类型 lambda 演算的“函数部分”成立吗? 也就是说, 假设 t 不含条件表达式, $t \longrightarrow t'$ 与 $\Gamma \vdash t' : T$ 是否蕴含 $\Gamma \vdash t : T$?

[查看原书附录 A 中的 9.3.10 解答](#)

9.4 Curry–Howard 对应

类型构造子 $\rightarrow$ 配有两类类型规则:

1. 引入规则 (*introduction rule*) T-ABS, 描述如何构造这种类型的元素;
2. 消去规则 (*elimination rule*) T-APP, 描述如何使用这种类型的元素。

当引入形式 λ 是消去形式 (应用) 的直接子项时, 结果就是一个可归约式, 也就是一次计算机会。

讨论类型系统时, 引入形式与消去形式这套术语常常很有用。后面研究更复杂的系统时会看到: 对每个类型构造子, 都会出现类似的相互配合的引入规则和消去规则。

练习 9.4.1 ($\star$). 图8-1 中 Bool 类型的哪些规则是引入规则, 哪些是消去规则? 图8-2 中 Nat 的规则又如何?

[查看原书附录 A 中的 9.4.1 解答](#)

引入 / 消去术语来自类型论与逻辑之间一种称为 Curry–Howard 对应 (*Curry–Howard correspondence*) 或 Curry–Howard 同构 (*Curry–Howard isomorphism*) 的联系 (Curry 与 Feys, 1958; Howard, 1980)。简言之, 在构造逻辑中, 命题 P 的证明由支持 P 的具体证据构成。⁴ Curry 与 Howard 注意到, 这类证据可以用程序来表示和操作。在逻辑记法中, $P \supset Q$

⁴经典逻辑与构造逻辑的典型区别, 是后者省略了排中律一类证明规则。排中律说, 对每个命题 Q , 要么 Q 成立, 要么 $\neg Q$ 成立。在构造逻辑中证明 $Q \vee \neg Q$, 必须给出支持 Q 或支持 $\neg Q$ 的证据。

表示“ P 蕴含 Q ”，也就是“如果 P ，那么 Q ”。它的证明可以看作一个过程：给它一个 P 的证明，它就构造一个 Q 的证明。换一种说法，它把 P 的证明作为抽象参数，函数体则构造 Q 的证明。类似地， $P \wedge Q$ 表示“ P 且 Q ”，它的证明由 P 的证明与 Q 的证明共同构成。由此得到以下对应：

逻辑	程序语言
命题	类型
命题 $P \supset Q$	类型 $P \rightarrow Q$
命题 $P \wedge Q$	类型 $P \times Q$ (见 §11.6)
命题 P 的证明	类型为 P 的项 t
命题 P 可证明	类型 P 有居留元 (<i>inhabited</i>) (即存在某个具有类型 P 的项)

译者注：阅读本节所需的最少逻辑背景。命题是可以讨论其是否成立的断言，例如“2 是偶数”。本节用大写字母 P, Q 代表任意命题，就像前文用 T 代表任意类型。几个逻辑符号的读法如下：

- $P \supset Q$: P 蕴含 Q ，即“如果 P ，那么 Q ”；
- $P \wedge Q$: P 且 Q ，证明它必须同时证明两边；
- $P \vee Q$: P 或 Q ，构造性证明必须选择并证明其中一边；
- $\neg P$: 非 P 。要证明它，必须说明：即使暂且假设 P 成立，也一定会推出矛盾。这是在作条件推理，并没有承认 P 确实为真；
- $\vdash p : P$: 不依赖任何未说明的假设， p 是命题 P 的证明。

表中逻辑一栏的符号 $\supset$ 表示蕴含，而不是集合的包含关系。作者在逻辑一侧写 $P \supset Q$ ，在程序语言一侧写函数类型 $P \rightarrow Q$ ，正是为了把两边的记号区分开来；Curry-Howard 对应说明，这两种结构彼此对应。

“命题即类型”究竟在说什么。这里并不是简单地把“命题”改名为“类型”，而是说同一个形式判断可以从两边来读。例如

$$\Gamma \vdash t : P$$

在程序语言一侧，它读作“在类型上下文 Γ 下，项 t 具有类型 P ”；在逻辑一侧，它读作“在 Γ 所列假设下， t 是命题 P 的证明”。若

$$\Gamma = x_1 : P_1, \dots, x_n : P_n$$

那么逻辑读法就是：手头已经分别有 $P_1, \dots, P_n$ 的证据 $x_1, \dots, x_n$ ，利用这些证据构造出了 P 的证据 t 。在给定上下文 Γ 中，只要存在某个项 t 满足 $\Gamma \vdash t : P$ ，就称类型 P 在该上下文中有居留元；这样的 t 称为类型 P 的居留元 (*inhabitant*)。当上下文非空时， t 可以使用其中声明的变量，因而不一定是闭项。

当上下文为空时， $\vdash p : P$ 表示不借助任何额外假设便得到 P 的证明，此时 p 必然是闭项。因此，“命题 P 可证明”恰好对应“类型 P 在空上下文中有居留元”，也等价于“存在一个类型为 P 的闭项”。从这个角度看，类型检查器验证 $t : P$ ，就像证明检查器验证 t 确实构成命题 P 的证明。

三条类型规则如何成为证明规则。

- T-VAR: 若上下文中已经有 $x : P$ ，就可以使用这份现成的证据，得到 $\Gamma \vdash x : P$ 。逻辑上，这只是“使用一项假设”。
- T-ABS: 若在额外假定 $x : P$ 时，项 t 能证明 Q ，即 $\Gamma, x : P \vdash t : Q$ ，那么把这段推理封装成 $\lambda x : P. t$ ，就得到类型 $P \rightarrow Q$ 的函数。逻辑上，这表示：从一份 P 的证明出发能够构造 Q 的证明，因而证明了 $P \supset Q$ 。
- T-APP: 若 f 证明了 $P \supset Q$ ，而 p 证明了 P ，那么函数应用 $f p$ 就证明了 Q 。程序语言一侧正好是： $f : P \rightarrow Q$ 、 $p : P$ ，所以 $f p : Q$ 。

以 $P \supset P$ 为例看证明怎样成为程序。这里要证明的不是命题 P 本身，而是条件命题“如果 P 成立，那么 P 成立”。证明可以逐步写成：

1. 为了讨论“如果 P 成立”这一条件，先暂时设想已经得到一份 P 的证明，并把它命名为 x 。上下文中的 $x : P$ 记录的正是这项临时假设；它并没有证明 P 无条件成立。
2. 在上下文 $x : P$ 中，T-VAR 允许直接使用这份已有的证明，所以得到 $x : P \vdash x : P$ 。
3. T-ABS 把上述推理封装成函数 $\lambda x : P. x$ ：它接受一份 P 的证明，再原样返回这份证明。原来的临时假设由此变成函数参数，函数外部不再需要预先拥有 P 的证明。

完整推导可以写成

$$\frac{x : P \vdash x : P}{\vdash (\lambda x : P. x) : P \rightarrow P} \text{ T-ABS}$$

横线上方说明函数体 x 在临时假设下具有类型 P ；横线下说明整个 lambda 抽象不依赖外部假设，具有类型 $P \rightarrow P$ 。关键不在于只要写出一个 lambda 表达式就算完成了证明，而在于类型规则确实能够推出这个表达式具有 $P \rightarrow P$ 类型。于是， $\lambda x : P. x$ 既是恒等函数，也是命题 $P \supset P$ 的证明；“程序”和“证明”是同一个形式对象的两种读法。

其他逻辑联结词也以类似方式对应数据类型。 $P \wedge Q$ 的证明必须同时携带 P 与 Q 的证明，因而对应数对类型 $P \times Q$ ；析取 $P \vee Q$ 的构造性证明必须明确选择左边或右边，并携带相应一侧的证明，因而对应带标签的分支；存在命题的构造性证明则必须给出一个满足条件的具体取值（称为见证 (*witness*)），并证明该取值确实满足条件。

为什么这里使用构造逻辑。经典逻辑可以直接使用排中律 $P \vee \neg P$ ，而构造逻辑要求证明者实际给出 P 的证据，或者给出 $\neg P$ 的证据。换成程序语言的说法，证明不能只声称“两种结果必有一种”，而必须构造出带有明确分支和值的结果。这种对具体证据的要求，正是逻辑证明能够对应为程序的原因。

计算为什么对应证明化简。归约

$$(\lambda x : P. t) s \longrightarrow [x \mapsto s]t$$

把作为参数传入的证明 s 直接代入使用假设 x 的证明 t 。在逻辑中，这相当于消去一层“先证明 P ，再借助 P 证明其他结论”的中间环节，使证明变得更直接；后文所说的割消去正是在形式上刻画这种证明化简。

现阶段可以把整套观点压缩为四句话：命题对应类型；逻辑假设对应类型上下文；证明对应良类型项；证明的化简对应程序的求值。

从这种观点看，简单类型 lambda 演算的项就是其类型所对应之逻辑命题的证明。计算——lambda 项的归约——对应于通过割消去 (*cut elimination*) 简化证明这一逻辑操作。Curry–Howard 对应也称为命题即类型 (*propositions as types*) 类比。Girard、Lafont 与 Taylor (1989), Gallier (1993), Sørensen 与 Urzyczyn (1998), Pfenning (2001), Goubault-Larrecq 与 Mackie (1997), 以及 Simmons (2000) 等文献都对此有深入讨论。

Curry–Howard 对应之美在于，它并不局限于某一个类型系统和某一种相关逻辑；相反，它可以扩展到种类繁多的类型系统与逻辑。例如，参数多态涉及对类型量化的 System F (第二十三章)，精确对应于允许对命题量化的二阶构造逻辑；System F_ω (第三十章) 对应于高阶逻辑。事实上，人们经常借助这种对应，把一个领域的新成果引入另一个领域。Girard 的线性逻辑 (1987) 催生了线性类型系统的思想 (Wadler, 1990、1991; Turner、Wadler 与 Mossin, 1995; Hodas, 1992; Mackie, 1994; Chirimar、Gunter 与 Riecke, 1996; Kobayashi、Pierce 与 Turner, 1996; 等等)；模态逻辑则被用于帮助设计部分求值与运行时代码生成的框架 (参见 Davies 与 Pfenning, 1996; Wickline、Lee、Pfenning 与 Davies, 1998, 以及其中引用的其他资料)。

9.5 擦除与可赋型性

图9-1 直接在简单类型项上定义了求值关系。类型标注在求值中不起任何作用——我们不做任何运行时检查来保证函数应用于适当类型的参数——但在求值项时仍会让这些标注随项一起传递。

实际的完整程序语言编译器大多避免在运行时携带标注：标注用于类型检查（在更复杂的编译器中也用于代码生成），却不出现在程序的编译结果中。实际上，程序求值前会被转换回无类型形式。这种语义风格可以用擦除函数（*erasure function*）形式化：它把简单类型项映射为对应的无类型项。

定义 9.5.1. 简单类型项 t 的擦除定义如下：

$$\begin{aligned}\text{erase}(x) &= x, \\ \text{erase}(\lambda x : T_1. t_2) &= \lambda x. \text{erase}(t_2), \\ \text{erase}(t_1 t_2) &= \text{erase}(t_1) \text{erase}(t_2)\end{aligned}$$

当然，我们预期简单类型演算的两种语义呈现方式实际上相符：直接求值有类型项，或先擦除类型再求值底层无类型项，不会造成真正的差别。下面的定理形式化了这项预期。其口号是“求值与擦除可交换”：无论先求值再擦除，还是先擦除再求值，都会到达同一个项。

定理 9.5.2.

1. 若有类型求值关系下 $t \longrightarrow t'$ ，则 $\text{erase}(t) \longrightarrow \text{erase}(t')$ 。
2. 若无类型求值关系下 $\text{erase}(t) \longrightarrow m'$ ，则存在一个简单类型项 t' ，使 $t \longrightarrow t'$ ，且 $\text{erase}(t') = m'$ 。

证明. 对求值推导作直接归纳。 □

这里所考察的“编译”太直接，因此定理9.5.2 几乎显然到了平凡的程度。不过，对更有意思的语言和编译器，它会成为十分重要的性质：它说明，直接以程序员所写语言表述的“高层”语义，与语言实现实际采用的另一种低层求值策略一致。

擦除函数还带来另一个有意思的问题：给定无类型 lambda 项 m ，能否找到一个擦除后得到 m 的简单类型项 t ？

定义 9.5.3. 若存在某个简单类型项 t 、类型 T 和上下文 Γ ，使 $\text{erase}(t) = m$ 且 $\Gamma \vdash t : T$ ，则称无类型 lambda 演算中的项 m 在 $\lambda_{\rightarrow}$ 中可赋型。

译者注：这里的“可赋型”是说：能否在不改变无类型项原有程序结构的前提下，为其中的绑定变量补上适当的类型标注，并为自由变量选择适当的上下文，使所得项通过简单类型检查。例如， $\lambda x. x$ 可以补成 $\lambda x : \text{Bool}. x$ ，因而可赋型；而 $\lambda x. x x$ 无法在简单类型系统中补出一致的类型，因而不可赋型。这个定义只要求存在一种可行的赋型，不要求这种赋型唯一。

第二十二章讨论与此密切相关的 $\lambda_{\rightarrow}$ 类型重构时，还会更详细地回到这个问题。

9.6 Curry 风格与 Church 风格

我们已经看到两种表述简单类型 lambda 演算语义的方式：直接在简单类型演算的语法上定义求值关系，或者编译到无类型演算，再在无类型项上定义求值关系。两者有一个重要共同点：它们都先为全部语法合法的项规定求值规则。因此，即使某个项没有通过类型检查，我们仍能按照这些规则判断它会怎样求值——当然，它也可能最终卡住。这种语言定义形式常称为 Curry 风格 (*Curry-style*)。先定义项，再用语义说明项的行为，然后给出类型系统，拒绝一部分我们不喜歡其行为的项。语义先于类型。

另一种颇为不同的组织方式，是先定义项，再识别良类型项，最后只为良类型项给出语义。在所谓 Church 风格 (*Church-style*) 系统中，类型先于语义；我们甚至根本不问“病类型项的行为是什么”。严格地说，Church 风格系统真正求值的是类型推导，而不是项；§15.6 会给出一个例子。

历史上，lambda 演算的隐式类型呈现通常采用 Curry 风格，而 Church 风格一般只见于显式类型系统。这也造成了术语上的一些混乱：“Church 风格”有时被用来指显式类型语法，“Curry 风格”则用来指隐式类型语法。

9.7 注记

Hindley 与 Seldin (1986) 研究了简单类型 lambda 演算；Hindley 的专著 (1997) 给出了更为详尽的论述。

良类型程序不会“出错”。

Well-typed programs cannot “go wrong.”

Robin Milner (1978)

译者注：这里的“出错”具有特定的技术含义：通过类型检查的闭项在求值过程中不会落入“既不是值、又无法继续求值”的卡住状态。进展性保证良类型闭项要么是值，要么还能继续求值；保持性保证它经过一次单步求值后仍为良类型。两者合在一起，排除了类型系统所针对的运行时错误，但并不保证程序一定终止、计算结果符合预期，或不会发生资源耗尽等其他问题。

第十章 简单类型的 ML 实现

把 $\lambda \rightarrow$ 具体实现为 ML 程序，所遵循的思路与第七章实现无类型 lambda 演算时相同。主要增加的是函数 `typeof`，它在给定上下文中计算给定项的类型。不过在介绍它之前，还需要一些操纵上下文的底层机制。

译者注：原书的 OCaml 程序完整保留在正文中，每段代码之后直接给出译者增补的 Rust 对照实现。Rust 版本不追求逐行照译，而是保持相同的对象语言语法、类型检查规则和求值行为。

10.1 上下文

回想 §7.1，上下文是一个列表；列表中的每一项都是一个二元组，包含变量名及其绑定信息：

```
type context = (string * binding) list
```

Rust 对照实现（译者增补）

```
pub type Context = Vec<(String, Binding)>;
```

在第七章中，上下文只服务于两项工作：解析时把项从有名形式转换为无名形式，打印时再把无名形式还原为有名形式。为此，我们只需记录变量名；`binding` 类型被定义成一个只有单个构造子的平凡数据类型，不携带任何信息：

```
type binding = NameBind
```

Rust 对照实现（译者增补）

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Binding {
    Name,
}
```

实现类型检查器时¹，需要用上下文携带关于变量的类型假设。因此，我们为 `binding` 类型加入一个名为 `VarBind` 的新构造子：

¹这里描述的实现对应带布尔值（图8-1）的简单类型 lambda 演算（图9-1）。本章代码可在作者网站仓库的 `simplebool` 实现中找到。


```
type binding =
    NameBind
  | VarBind of ty
```

Rust 对照实现 (译者增补)

```
// `Type` 的完整定义见第 10.2 节。
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Binding {
    Name,
    Var(Type),
}
```

每个 VarBind 构造子都携带相应变量的类型假设。旧的 NameBind 仍然保留，供不关心类型假设的打印和解析函数使用。（另一种实现策略是定义两套完全不同的上下文类型：一套用于解析和打印，另一套用于类型检查。）

typeof 使用 addbinding 函数，把新的变量绑定 (x,bind) 加入上下文 ctx。上下文由列表表示，所以 addbinding 实质上就是列表的 cons 操作：

```
let addbinding ctx x bind = (x,bind)::ctx
```

Rust 对照实现 (译者增补)

```
pub fn add_binding(mut context: Context, name: String, binding: Binding) ->
    Context {
    context.insert(0, (name, binding));
    context
}
```

反过来，函数 getTypeFromContext 从上下文 ctx 中提取与特定变量 i 关联的类型假设。若 i 越界，则使用文件位置信息 fi 打印错误消息：

```
let getTypeFromContext fi ctx i =
    match getbinding fi ctx i with
    | VarBind(tyT) -> tyT
    | _ -> error fi
        ("getTypeFromContext: Wrong kind of binding for variable "
         ^ (index2name fi ctx i))
```

Rust 对照实现 (译者增补)

```
pub fn get_type_from_context(
    info: Info,
    context: &Context,
    index: usize,
) -> Result<Type, TypeError> {
    match get_binding(info, context, index)? {
        Binding::Var(term_type) => Ok(term_type.clone()),
        Binding::Name => Err(TypeError::WrongBinding { info, index }),
    }
}
```

这个模式匹配还提供了一项内部一致性检查：正常情况下，调用 `getTypeFromContext` 时，上下文中的第 `i` 个绑定应当确实是 `VarBind`。不过，后续章节会加入其他形式的绑定，特别是类型变量的绑定。届时，如果传入的索引指向的不是 `VarBind`，`getTypeFromContext` 就无法从中取得项变量的类型，于是会调用底层的 `error` 函数报告这种内部不一致；传入的 `info` 用于指出错误发生的文件位置。

```
val error : info -> string -> 'a
```

Rust 对照实现 (译者增补)

```
pub fn error(info: Info, message: &str) -> ! {
    panic!("{}", message, info.line, info.column)
}
```

`error` 的结果类型是 类型变量 `'a`，可以实例化为任意 ML 类型。这是合理的，因为该函数无论如何也不会返回：它打印消息后便终止程序。这里必须假定 `error` 的结果是 `ty`，因为模式匹配的另一分支返回的正是 `ty`。

译者注：Rust 对照实现把 `error` 的返回类型写成 `!`，表示该函数永远不会正常返回。因此，对 `error` 的调用也可以出现在需要任何其他结果类型的位置；在这里，`!` 与 OCaml 中可以实例化为任意类型的 `'a` 起到相同作用。

注意，类型假设是按索引查找的，因为项在内部采用无名形式表示，变量表示为数字索引。函数 `getbinding` 只是查找给定上下文中的第 `i` 个绑定：


```
val getbinding : info -> context -> int -> binding
```

Rust 对照实现 (译者增补)

```
pub fn get_binding(info: Info, context: &Context, index: usize) ->
↳ Result<&Binding, TypeError> {
    context
        .get(index)
        .map(|(_, binding)| binding)
        .ok_or(TypeError::BadIndex { info, index })
}
```

其定义可在本书网站的 `simplebool` 实现中找到。

10.2 项与类型

类型语法直接从图8-1 与图9-1 的抽象语法转写为 ML 数据类型:

```
type ty =
    TyBool
  | TyArr of ty * ty
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Type {
    Bool,
    Arrow(Box<Type>, Box<Type>),
}
```

项的表示与无类型 lambda 演算所用的形式相同 (见 §7.1), 只是在 `TmAbs` 分支中加入类型标注:

```
type term =
    TmTrue of info
  | TmFalse of info
  | TmIf of info * term * term * term
  | TmVar of info * int * int
  | TmAbs of info * string * ty * term
  | TmApp of info * term * term
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    True(Info),
    False(Info),
    If(Info, Box<Term>, Box<Term>, Box<Term>),
    Var(Info, usize, usize),
    Abs(Info, String, Type, Box<Term>),
    App(Info, Box<Term>, Box<Term>),
}
```

10.3 类型检查

类型检查函数 `typeof` 可以看作 $\lambda \rightarrow$ 类型规则 (图8-1 与图9-1) 的直接翻译; 更准确地说, 它是 反演引理9.3.1的转写。后一种说法更准确, 是因为反演引理针对每一种句法形式, 精确说明这种形式的项成为良类型项必须满足什么条件。类型规则只说明特定形式的项在特定条件下是良类型的; 单看一条类型规则, 永远不能断定某个项不是良类型的, 因为总可能有另一条规则能为这个项赋型。(目前这看起来也许只是没有实质区别的措辞差异, 因为反演引理直接来自类型规则。但在后面的系统中, 证明反演引理会比在 $\lambda \rightarrow$ 中费力得多, 这项区别便会变得重要。)

```
let rec typeof ctx t =
  match t with
  | TmTrue(fi) ->
    TyBool
  | TmFalse(fi) ->
    TyBool
  | TmIf(fi,t1,t2,t3) ->
    if (=) (typeof ctx t1) TyBool then
      let tyT2 = typeof ctx t2 in
      if (=) tyT2 (typeof ctx t3) then tyT2
      else error fi "arms of conditional have different types"
    else error fi "guard of conditional not a boolean"
  | TmVar(fi,i,_) ->
    getTypeFromContext fi ctx i
  | TmAbs(fi,x,tyT1,t2) ->
    let ctx' = addbinding ctx x (VarBind(tyT1)) in
    let tyT2 = typeof ctx' t2 in
    TyArr(tyT1, tyT2)
  | TmApp(fi,t1,t2) ->
    let tyT1 = typeof ctx t1 in
```

```

let tyT2 = typeof ctx t2 in
(match tyT1 with
  TyArr(tyT11,tyT12) ->
    if (==) tyT2 tyT11 then tyT12
    else error fi "parameter type mismatch"
  | _ -> error fi "arrow type expected")

```

Rust 对照实现 (译者增补)

```

pub fn type_of(context: &Context, term: &Term) -> Result<Type, TypeError> {
  match term {
    Term::True(_) | Term::False(_) => Ok(Type::Bool),
    Term::If(info, guard, then_term, else_term) => {
      if type_of(context, guard)? != Type::Bool {
        return Err(TypeError::ExpectedBoolean { info: *info });
      }
      let then_type = type_of(context, then_term)?;
      if then_type == type_of(context, else_term)? {
        Ok(then_type)
      } else {
        Err(TypeError::BranchMismatch { info: *info })
      }
    }
    Term::Var(info, index, _) => get_type_from_context(*info, context,
↳ *index),
    Term::Abs(_, name, parameter_type, body) => {
      let extended = add_binding(
        context.clone(),
        name.clone(),
        Binding::Var(parameter_type.clone()),
      );
      Ok(Type::Arrow(
        Box::new(parameter_type.clone()),
        Box::new(type_of(&extended, body)?),
      ))
    }
    Term::App(info, function, argument) => {
      let function_type = type_of(context, function)?;
      let argument_type = type_of(context, argument)?;
      match function_type {
        Type::Arrow(parameter_type, result_type) => {
          if argument_type == *parameter_type {
            Ok(*result_type)
          } else {

```

```
      Err(TypeError::ParameterMismatch { info: *info })
    }
  }
Type::Bool => Err(TypeError::ExpectedArrow { info: *info }),
}
}
```

这里有两个 OCaml 细节值得一提。第一，等号运算符 `=` 被写在括号中，因为这里把它放在前缀位置使用，而不是通常的中缀位置。这样更方便与后续版本的 `typeof` 比较；到那时，比较类型需要比简单相等更精细的操作。

第二，等号运算符计算复合值的结构相等，而不是指针相等。也就是说：

```
let t = TmApp(t1,t2) in
let t' = TmApp(t1,t2) in
(=) t t'
```

虽然绑定到 `t` 与 `t'` 的两个 `TmApp` 实例是在不同时刻分别分配的，内存地址也不同，上述比较仍会得到 `true`。

译者注：Rust 不需要为这个例子另写相等性函数。Term 已经派生 PartialEq 与 Eq；因此，对两个分别构造但内容相同的项直接使用 `t == t'`，就会递归比较其结构和内容。这与上面的 OCaml 表达式 `(=) t t'` 作用相同。

第十一章 简单扩展

简单类型 lambda 演算已经包含了足够丰富的形式构造，值得研究其理论性质；但作为一门程序语言，它仍然相当简陋。本章加入若干熟悉的语言特性；它们都很容易纳入现有的类型系统，从而开始缩小这一演算与常见语言之间的距离。贯穿全章的一个重要主题是派生形式。

11.1 基本类型

每种程序语言都会提供多种基本类型 (*base type*) ——由数字、布尔值或字符等简单而没有内部结构的值构成的集合——以及操纵这些值的适当基本操作。我们已经详细考察过自然数和布尔值；语言设计者还可以用完全相同的方式加入任意多种其他基本类型。

除 Bool 与 Nat 外，本书其余部分偶尔还会用基本类型 String (包含 "hello" 一类元素) 和 Float (包含 3.14159 一类元素)，让例子稍丰富一些。

为研究理论性质，把特定基本类型及其运算的细节抽象掉往往很有用。我们可以只假定语言配有某个基本类型集合 $\mathcal{A}$ 。对于其中的类型，我们不规定它们包含哪些具体值，也不提供作用于这些值的基本操作；这样的类型称为 未解释的基本类型 (*uninterpreted base type*)。只需把 $\mathcal{A}$ 的元素纳入类型集合，即可做到这一点，如图11-1所示。在这条语法规则中，我们用元变量 A 表示任意基本类型，而不用 B ，以免与原书表示系统含有布尔值的特性标记 B 混淆。 A 也可以理解为 原子类型 (*atomic type*) 的缩写；从类型系统的角度看，这些类型没有内部结构，所以基本类型也常用这个名字。¹

A

扩展 $\lambda_{\rightarrow}$ (图9-1)

$$\begin{array}{l} T ::= \dots \text{ 类型} \\ \quad | \quad A \quad \text{基本类型}(A \in \mathcal{A}) \end{array}$$

图 11-1: 未解释的基本类型

具体的基本类型可以命名为 A, B, C 等。注意，这里和此前处理变量时一样，既把 A 用作某个基本类型的名称，也把它用作遍历基本类型的元变量；具体含义依上下文判断。

¹本章研究纯简单类型 lambda 演算 (图9-1) 的多种扩展。相应的 OCaml 实现 `fullsimple` 包含所有这些扩展。

未解释的类型是否毫无用处？当然不是。虽然我们无法直接写出这种类型的具体值，但仍可以引入这种类型的变量。例如：

$$\lambda x : A. x \quad \blacktriangleright \quad \langle \text{fun} \rangle : A \rightarrow A$$

是 A 的元素上的恒等函数，无论这些元素实际是什么。类似地，

$$\lambda x : B. x \quad \blacktriangleright \quad \langle \text{fun} \rangle : B \rightarrow B$$

是 B 上的恒等函数，而

$$\lambda f : A \rightarrow A. \lambda x : A. f(f x) \quad \blacktriangleright \quad \langle \text{fun} \rangle : (A \rightarrow A) \rightarrow A \rightarrow A$$

接受函数 f 和参数 x ，把 f 的行为重复两次。²

11.2 Unit 类型

另一种有用的基本类型是只有一个值的 Unit 类型，它尤其常见于 ML 家族的语言，见图11-2。与上一节的未解释基本类型不同，我们明确规定 Unit 只有一个值，即项常量 unit（小写 u ），并用一条类型规则规定 unit 的类型为 Unit。我们还把 unit 列为一种值；因此，Unit 类型的表达式求值后只能得到 unit。

Unit

扩展 $\lambda \rightarrow$ (图9-1)

$t ::= \dots$	项	
unit	常量 unit	
$v ::= \dots$	值	
unit	unit 值	
$T ::= \dots$	类型	
Unit	unit 类型	

$\frac{}{\Gamma \vdash \text{unit} : \text{Unit}} \text{ T-UNIT}$

图 11-2: Unit 类型

即使在纯函数式语言中，Unit 类型也有一定的研究价值³；不过，它主要用于带副作用的语言，例如可以给引用单元赋值的语言；第十三章会回到这个话题。在这类语言中，我们关心的往往是表达式的副作用而非结果；Unit 正适合作为这类表达式的结果类型。

这种用法类似 C 与 Java 等语言中的 void 类型。void 这个名字似乎暗示它与空类型 Bot（名称取自英文单词 *bottom*，即类型层次最底部的类型）有关（参见 §15.4），但其实际用法更接近这里的 Unit。

练习 11.2.1 ($\star \star \star$). 在只有基本类型 Unit 的简单类型 lambda 演算中，能否构造项序列 $t_1, t_2, \dots$ ，使每个 t_n 的大小至多为 $O(n)$ ，却至少需要 $O(2^n)$ 步求值才能到达范式？

查看原书附录 A 中的 11.2.1 解答

²从现在开始，显示求值结果时，为节省空间会省略 lambda 抽象的函数体，只写 $\langle \text{fun} \rangle$ 。

³读者也许会喜欢这个小谜题：在只有基本类型 Unit 的简单类型 lambda 演算中，能构造多少个不同的闭范式？[参考解答见译者附录](#)。

11.3 派生形式：顺序执行与通配符

在带副作用的语言中，依次求值两个或更多表达式常常很有用。顺序执行记法 $t_1; t_2$ 先求值 t_1 ，丢弃其无关紧要的结果，再继续求值 t_2 。

形式化顺序执行有两种不同方法。一种沿用其他句法形式的处理模式：把 $t_1; t_2$ 作为项语法的新候选形式，再加入两条求值规则和一条类型规则：

$$\frac{t_1 \longrightarrow t'_1}{t_1; t_2 \longrightarrow t'_1; t_2} \text{ E-SEQ} \quad \frac{}{v_1; t_2 \longrightarrow t_2} \text{ E-SEQNEXT}$$

$$\frac{\Gamma \vdash t_1 : \text{Unit} \quad \Gamma \vdash t_2 : T_2}{\Gamma \vdash t_1; t_2 : T_2} \text{ T-SEQ}$$

译者注：原书初版把 E-SEQNEXT 的左端写成 $\text{unit}; t_2$ ，这与按值调用的应用规则不能完全对应。作者勘误改为任意值 $v_1; t_2 \longrightarrow t_2$ ；本译文采用勘误后的规则。

另一种方法，是直接把 $t_1; t_2$ 视为项 $(\lambda x : \text{Unit}. t_2) t_1$ 的缩写，其中另取变量 x ，要求 $x \notin \text{FV}(t_2)$ ，即 x 不在 t_2 中自由出现。为避免名称冲突而这样另取的变量称为新鲜变量 (*fresh variable*)。

直觉上，这两种顺序执行呈现方式对程序员来说效果相同：可以从 $t_1; t_2$ 的上述缩写，导出顺序执行的高层类型规则和求值规则。更形式地说，类型与求值都和缩写展开“可交换”：展开前后，项具有相同的类型；而先在外语语言中进行一次单步求值再展开，与先展开后在内部语言中求值，所得结果彼此对应。

定理 11.3.1 (顺序执行是一种派生形式). 以 λ^E (E 表示外部语言) 表示带 Unit、顺序执行构造及规则 E-SEQ、E-SEQNEXT、T-SEQ 的简单类型 *lambda* 演算；以 λ^I (I 表示内部语言) 表示只带 Unit 的简单类型 *lambda* 演算。设精化函数 (*elaboration function*) $e \in \lambda^E \rightarrow \lambda^I$ 把外部语言翻译为内部语言：将每处 $t_1; t_2$ 替换为 $(\lambda x : \text{Unit}. t_2) t_1$ ，每次都为相应的 t_2 另取一个不在其中自由出现的变量 x 。那么，对每个 λ^E 中的项 t ，都有：

$$\begin{aligned} e(t) \longrightarrow_I u &\implies \text{存在 } t' \in \lambda^E, u = e(t') \text{ 且 } t \longrightarrow_E t', \\ t \longrightarrow_E t' &\implies e(t) \longrightarrow_I e(t'), \\ \Gamma \vdash_E t : T &\iff \Gamma \vdash_I e(t) : T \end{aligned}$$

下标 E 与 I 分别标明使用哪一种语言的求值关系和类型关系。

证明. 每个方向都对 t 的结构作直接归纳。 □

定理 11.3.1 说明顺序执行构造的类型和求值行为可以从更基本的抽象与应用操作导出，因此“派生形式”这个名字名副其实。把顺序执行一类特性作为派生形式，而不作为完全独立的语言构造，其好处是：可以扩充表层语法（程序员实际编写程序所使用的语言），却不增加内部语言的复杂性；类型安全性一类定理只需对内部语言证明。这种分解语言特性描述的方式，在 Algol 60 报告 (Naur 等, 1963) 中已经出现，许多较新的语言定义也大量采用，尤其是

Standard ML 定义 (Milner、Tofte 与 Harper, 1990; Milner、Tofte、Harper 与 MacQueen, 1997)。

沿用 Landin 的说法, 派生形式常称为语法糖 (*syntactic sugar*); 把派生形式换成其低层定义, 称为去糖 (*desugaring*)。

另一个在后面例子中很有用的派生形式, 是变量绑定子的“通配符”约定。我们经常需要写一个“哑元”lambda 抽象, 其参数变量在抽象体中并未实际使用, 例如顺序执行去糖产生的项。此时还要显式选择受约束变量的名字很烦琐; 可以用通配绑定子 $_$ 代替。也就是说, $\lambda_ : S. t$ 是 $\lambda x : S. t$ 的缩写, 其中 x 是 t 中未出现的某个变量。

练习 11.3.2 (*). 给出通配符抽象的类型规则与求值规则, 并证明它们可以从上述缩写导出。

[查看原书附录 A 中的 11.3.2 解答](#)

11.4 类型标注

以后还会经常用到另一项简单特性: 为给定项显式标注某一类型, 即在程序文本中记下一项断言, 声明该项具有该类型。我们用 $t \text{ as } T$ 表示“项 t , 并为它标注类型 T ”。这个构造的类型规则 T-ASCRIBE (见图11-3) 只验证所标注的 T 确实是 t 的类型。求值规则同样直接: 丢掉标注, 让 t 照常求值。

as

扩展 $\lambda_{\rightarrow}$ (图9-1)

$$\begin{array}{c}
 \begin{array}{lcl}
 t & ::= & \dots \quad \text{项} \\
 & | & t \text{ as } T \quad \text{类型标注}
 \end{array}
 \end{array}
 \qquad
 \begin{array}{c}
 \frac{t_1 \longrightarrow t'_1}{t_1 \text{ as } T \longrightarrow t'_1 \text{ as } T} \text{ E-ASCRIBE1} \\
 \\
 \frac{}{v_1 \text{ as } T \longrightarrow v_1} \text{ E-ASCRIBE} \\
 \\
 \frac{\Gamma \vdash t_1 : T}{\Gamma \vdash t_1 \text{ as } T : T} \text{ T-ASCRIBE}
 \end{array}$$

图 11-3: 类型标注

类型标注在编程中有多种用途。常见用途之一是文档说明。大型复合表达式中, 读者有时很难始终追踪每个子表达式的类型; 适当加入类型标注, 会使程序容易理解得多。类似地, 面对格外复杂的表达式, 作者自己也未必清楚所有子表达式的类型; 加入几处类型标注很适合帮助程序员理清思路。事实上, 它有时还能帮助定位令人费解的类型错误究竟来自哪里。

另一个用途是控制复杂类型的打印。本书用来检查示例的类型检查器, 以及名称以 `full` 开头的配套 OCaml 实现, 都提供一种简单机制, 为冗长或复杂的类型表达式引入缩写; 其他实现为便于阅读和修改, 省略了这项机制。例如声明

```
UU = Unit->Unit;
```


使 $\mathbb{U}$ 在后文中成为 $\text{Unit} \rightarrow \text{Unit}$ 的缩写；看到 $\mathbb{U}$ 的地方，都理解成后一个类型。因此可以写：

$$(\lambda f : \mathbb{U}. f \text{ unit})(\lambda x : \text{Unit}. x)$$

类型检查时会按需自动展开缩写；反过来，类型检查器也会尽可能把类型折叠成缩写。具体地，每次算出子项的类型时，它都会检查这个类型是否与当前定义的某一缩写完全匹配；若是，就用缩写替换该类型。这通常能给出合理结果，但偶尔我们会希望类型以不同形式打印。可能是简单匹配策略错过了折叠缩写的机会（例如记录类型的字段可置换时，它无法认出 $\{a : \text{Bool}, b : \text{Nat}\}$ 与 $\{b : \text{Nat}, a : \text{Bool}\}$ 可以互换），也可能出于其他原因。例如：

$$\lambda f : \text{Unit} \rightarrow \text{Unit}. f \quad \blacktriangleright \quad \langle \text{fun} \rangle : (\text{Unit} \rightarrow \text{Unit}) \rightarrow \mathbb{U}$$

中，函数的结果类型折叠成了 $\mathbb{U}$ ，参数类型却没有。要把它打印为 $\mathbb{U} \rightarrow \mathbb{U}$ ，可以把抽象的类型注解改成：

$$\lambda f : \mathbb{U}. f \quad \blacktriangleright \quad \langle \text{fun} \rangle : \mathbb{U} \rightarrow \mathbb{U},$$

也可以给整个抽象加上类型标注：

$$(\lambda f : \text{Unit} \rightarrow \text{Unit}. f) \text{ as } \mathbb{U} \rightarrow \mathbb{U} \quad \blacktriangleright \quad \langle \text{fun} \rangle : \mathbb{U} \rightarrow \mathbb{U}$$

处理 $t \text{ as } T$ 时，类型检查器会展开 T 中的缩写，检查 t 确实具有类型 T ，但随后以原样写出的 T 作为标注表达式的类型。这种借助类型标注控制类型打印结果的做法，主要是本书配套实现中的一项设计选择。在功能完整的程序语言中，类型缩写和类型打印机制要么根本没有必要——例如 Java 按构造就用短名字表示全部类型，参见第十九章——要么会更紧密地集成到语言中——例如 OCaml，参见 Rémy 与 Vouillon (1998) 及 Vouillon (2000)。

类型标注的最后一种用途，是充当抽象机制；§15.5 会详细讨论。在同一项 t 可能具有许多不同类型的系统中，例如带子类型化的系统，可以用类型标注“隐藏”其中一些类型，要求类型检查器把 t 当作仿佛只具有一个更小的类型集合。§15.5 也会讨论类型标注与类型转换的关系。

练习 11.4.1 (推荐, ★★). 1. 说明如何把类型标注表述成派生形式。证明这里给出的“正式”类型规则与求值规则在适当意义上对应于你的定义。

2. 假设不用 E-ASCRIBE 与 E-ASCRIBE1 这一对规则，而改用下面这条“急切”规则：求值器一遇到类型标注，就立即将它去掉

$$t \text{ as } T \longrightarrow t$$

此时类型标注还能视为派生形式吗？

查看原书附录 A 中的 11.4.1 解答

11.5 let 绑定

编写复杂表达式时，为一些子表达式命名既可避免重复，也能提高可读性。大多数语言都提供一种或多种做到这一点的方式。例如，ML 中写 `let  $x = t_1$  in  $t_2$` ，意指“求值 t_1 ，把名字 x 绑定到所得的值，再求值 t_2 ”。

图11-4 汇总的 let 绑定与 ML 一样采用按值调用的求值次序：必须先把被 let 绑定的项求值为值，才能开始求值 let 体。类型规则 T-LET 告诉我们：先计算被绑定项的类型，以该类型的绑定扩展上下文，再在扩展后的上下文中计算体的类型；体的类型就是整个 let 表达式的类型。

$$\begin{array}{c}
 \text{let} \qquad \qquad \qquad \text{扩展 } \lambda_{\rightarrow} \text{ (图 9-1)} \\
 \\
 \begin{array}{c}
 \frac{}{\text{let } x = v_1 \text{ in } t_2 \longrightarrow [x \mapsto v_1]t_2} \text{E-LETV} \\
 \frac{t_1 \longrightarrow t'_1}{\text{let } x = t_1 \text{ in } t_2 \longrightarrow \text{let } x = t'_1 \text{ in } t_2} \text{E-LET}
 \end{array} \\
 \\
 \frac{\Gamma \vdash t_1 : T_1 \quad \Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : T_2} \text{T-LET}
 \end{array}$$

图 11-4: let 绑定

练习 11.5.1 (推荐, ★★). 本书网站上的 `letexercise` 类型检查器对 let 表达式的实现并不完整：已经提供基本解析和打印函数，但 `eval1` 与 `typeof` 中缺少处理 `TmLet` 的分支；占位分支会匹配所有输入，再以断言失败使程序崩溃。请完成这个实现。

查看原书附录 A 中的 11.5.1 解答

let 也能定义为派生形式吗？Landin 已经指出，可以用 lambda 抽象和函数应用表达 let；不过，具体处理比顺序执行和类型标注稍微复杂一些。最直接的编码是：

$$\text{let } x = t_1 \text{ in } t_2 \triangleq (\lambda x : T_1. t_2) t_1$$

不过，这种展开产生了一个问题：展开后的 lambda 抽象需要为参数 x 标注类型 T_1 ，原来的 let 表达式却没有提供这一信息。若设想编译器在解析阶段对派生形式进行去糖，就必须问：解析器怎样知道，应当在去糖后的内部语言项中为 lambda 参数生成类型标注 T_1 ？

答案当然是：信息来自类型检查器。只需计算 t_1 的类型，就能发现所需的类型注解。更形式地说，这说明 let 构造与此前的派生形式略有不同：不应把它看成作用于项的去糖变换，而应看成作用于类型推导的变换（也可以理解为作用于被类型检查器的分析结果装饰过的项），将含 let 的推导

$$\frac{\Gamma \vdash t_1 : T_1 \quad \Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : T_2} \text{T-LET}$$

映射为使用抽象与应用的推导：

$$\frac{\frac{\Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2} \text{T-ABS} \quad \Gamma \vdash t_1 : T_1}{\Gamma \vdash (\lambda x : T_1. t_2) t_1 : T_2} \text{T-APP}$$

因此，let 并不像前面的派生形式那样，能够在类型检查前通过简单去糖完全消除。求值时，可以把它展开为 lambda 抽象和函数应用；但进行类型检查时，仍需专门处理 let，不能只依赖这一展开。

第二十二章会看到另一项不把 let 当作派生形式的理由：在具有 Hindley–Milner 多态的语言中，类型检查器会通过合一（*unification*）算法求解类型约束，并对 let 绑定作特殊处理：在这里泛化所绑定定义的类型，使该定义在不同使用处可以取得不同的类型实例。仅靠普通 lambda 抽象和函数应用无法获得这种多态类型。

练习 11.5.2 (★★). 把 let 定义为派生形式的另一种办法，也许是立即“执行”它，即把 $\text{let } x = t_1 \text{ in } t_2$ 当作替换后的体 $[x \mapsto t_1]t_2$ 的缩写。这种做法是否妥当？

[查看原书附录 A 中的 11.5.2 解答](#)

11.6 二元对

大多数程序语言都提供多种构造复合数据结构的方法。最简单的是由两个值组成的二元对（*pair*），更一般的形式则是元组。本节讨论二元对，§11.7 和 §11.8 再讨论一般元组和带标签的记录。⁴

二元对的形式化简单得几乎不值得讨论。读到这里，直接阅读图11-5 中的规则，应该已经不比阅读表达同一内容的文字说明更困难。不过，为强调共同模式，还是简要看看定义的几个部分。

在简单类型 lambda 演算中加入二元对，需要加入两个新的项形式——用于构造二元对的 $\{t_1, t_2\}$ ，以及投影 $t.1$ 与 $t.2$ ——和一个新的类型构造子 $T_1 \times T_2$ ，称为 T_1 与 T_2 的积（*product*）（有时也称笛卡儿积（*cartesian product*））。这里用花括号书写二元对，是为了强调它和第11.8节讨论的记录之间的联系。⁵

求值需要几条新规则。E-PAIRBETA1 与 E-PAIRBETA2 说明：当二元对的两个分量都已成为值时，对它作第一或第二投影，会得到相应分量。E-PROJ1 与 E-PROJ2 允许在被投影项尚未成为值时，在投影之下继续归约。E-PAIR1 与 E-PAIR2 求值二元对的分量：先求值左侧，再在左侧出现值之后求值右侧。

⁴fullsimple 实现实际上没有提供这里描述的二元对语法，因为元组已经是更一般的形式。

⁵花括号容易让人联想到标准的集合记法，因此用于二元对和元组并不十分理想。无论在 ML 等流行语言还是研究文献中，用圆括号包围二元对和元组都更为常见；方括号、尖括号等其他记法也有人使用。

×

扩展 $\lambda_{\rightarrow}$ (图 9-1)

$t ::= \dots$	项
$\{t, t\}$	二元对
$t.1$	第一投影
$t.2$	第二投影
$v ::= \dots$	值
$\{v, v\}$	二元对值
$T ::= \dots$	类型
$T \times T$	积类型

$\frac{}{\{v_1, v_2\}.1 \rightarrow v_1}$	E-PAIRBETA1	$\frac{}{\{v_1, v_2\}.2 \rightarrow v_2}$	E-PAIRBETA2
$\frac{t_1 \rightarrow t'_1}{t_1.1 \rightarrow t'_1.1}$	E-PROJ1	$\frac{t_1 \rightarrow t'_1}{t_1.2 \rightarrow t'_1.2}$	E-PROJ2
$\frac{t_1 \rightarrow t'_1}{\{t_1, t_2\} \rightarrow \{t'_1, t_2\}}$	E-PAIR1	$\frac{t_2 \rightarrow t'_2}{\{v_1, t_2\} \rightarrow \{v_1, t'_2\}}$	E-PAIR2
$\frac{\Gamma \vdash t_1 : T_1 \quad \Gamma \vdash t_2 : T_2}{\Gamma \vdash \{t_1, t_2\} : T_1 \times T_2}$	T-PAIR	$\frac{\Gamma \vdash t_1 : T_{11} \times T_{12}}{\Gamma \vdash t_1.1 : T_{11}}$	T-PROJ1
		$\frac{\Gamma \vdash t_1 : T_{11} \times T_{12}}{\Gamma \vdash t_1.2 : T_{12}}$	T-PROJ2

图 11-5: 二元对

规则中元变量 v 与 t 的使用次序，强制二元对采用从左到右的求值策略。例如：

$$\begin{aligned}
 & \{\text{pred } 4, \text{if true then false else false}\}.1 \rightarrow \{3, \text{if true then false else false}\}.1 \\
 & \rightarrow \{3, \text{false}\}.1 \\
 & \rightarrow 3
 \end{aligned}$$

还需扩充值的语法：只有当 v_1 和 v_2 都是值时，二元对 $\{v_1, v_2\}$ 才算作值。这一要求保证，二元对作为函数参数传入时，会先把两个分量都求值为值，再开始执行函数体。例如：

$$\begin{aligned}
 & (\lambda x : \text{Nat} \times \text{Nat}. x.2) \{\text{pred } 4, \text{pred } 5\} \rightarrow (\lambda x : \text{Nat} \times \text{Nat}. x.2) \{3, \text{pred } 5\} \\
 & \rightarrow (\lambda x : \text{Nat} \times \text{Nat}. x.2) \{3, 4\} \\
 & \rightarrow \{3, 4\}.2 \\
 & \rightarrow 4
 \end{aligned}$$

二元对与投影的类型规则都很直接。引入规则 T-PAIR 说明：若 $t_1 : T_1$ 、 $t_2 : T_2$ ，则 $\{t_1, t_2\} : T_1 \times T_2$ 。反过来，消去规则 T-PROJ1 与 T-PROJ2 说明：若 t_1 具有积类型 $T_{11} \times T_{12}$ ，即它会求值为一个二元对，那么它的两个投影分别具有类型 T_{11} 与 T_{12} 。

11.7 元组

上一节的二元积很容易推广为 n 元积，通常称为 元组 (*tuple*)。例如， $\{1, 2, \text{true}\}$ 是含两个数字和一个布尔值的三元组，类型写作 $\{\text{Nat}, \text{Nat}, \text{Bool}\}$ 。

这种推广唯一的代价，是在形式化系统时必须设计一套能够统一描述任意元数结构的记法。这类记法总有些麻烦：若把索引范围和边界情况写得十分严密，公式会显得繁琐；若追求简洁易读，又难免省略一些细节。我们用 $\{t_i\}^{i \in 1..n}$ 表示由 t_1 至 t_n 组成的 n 元组，用 $\{T_i\}^{i \in 1..n}$ 表示其类型。这里允许 $n = 0$ ；此时范围 $1..n$ 为空， $\{t_i\}^{i \in 1..n} = \{\}$ ，即空元组。还要注意，裸值 5 与单元组元组 $\{5\}$ 不同：对后者唯一合法的操作是投影它的第一分量。

$\{\}$

扩展 $\lambda_{\rightarrow}$ (图 9-1)

$t ::= \dots$	项
$\quad \quad \{t_i\}^{i \in 1..n}$	构造元组
$\quad \quad t.j$	投影
$v ::= \dots \{v_i\}^{i \in 1..n}$	元组值
$T ::= \dots \{T_i\}^{i \in 1..n}$	元组类型

$\frac{}{\{v_i\}^{i \in 1..n}.j \rightarrow v_j} \text{E-PROJTUPLE}$	$\frac{t_j \rightarrow t'_j}{\{\{v_i\}^{i < j}, t_j, \{t_i\}^{i > j}\} \rightarrow \{\{v_i\}^{i < j}, t'_j, \{t_i\}^{i > j}\}} \text{E-TUPLE}$
$\frac{\Gamma \vdash t_i : T_i \quad (\text{对每个 } i)}{\Gamma \vdash \{t_i\}^{i \in 1..n} : \{T_i\}^{i \in 1..n}} \text{T-TUPLE}$	$\frac{\Gamma \vdash t_1 : \{T_i\}^{i \in 1..n}}{\Gamma \vdash t_1.j : T_j} \text{T-PROJ}$

图 11-6: 元组

图11-6 给出了元组的形式化定义。它与 图11-5中的二元积定义相似，只是构造二元对各条规则都推广到 n 元情形，第一、第二投影的每一对规则也合并成从元组任意位置投影的一条规则。只有 E-TUPLE 值得特别说明：它合并并推广了 图11-5 的 E-PAIR1 与 E-PAIR2。用文字说，若第 j 个字段左边的所有字段都已经是值，并且 t_j 可以单步求值为 t'_j ，那么整个元组也可以进行一次单步求值：将第 j 个字段从 t_j 替换为 t'_j ，其余字段保持不变。元变量的使用仍然强制从左到右求值。

11.8 记录

从 n 元组推广到带标签的记录 (*record*) 同样直接：只需为每个字段 t_j 标注一个取自预定集合 $\mathcal{L}$ 的标签 l_j 。例如， $\{x = 5\}$ 与 $\{\text{partno} = 5524, \text{cost} = 30.27\}$ 都是记录值，类型分别是 $\{x : \text{Nat}\}$ 与 $\{\text{partno} : \text{Nat}, \text{cost} : \text{Float}\}$ 。要求同一记录项或记录类型内的所有标签互不相同。

{}

扩展 $\lambda \rightarrow$ (图 9-1)

$$\begin{array}{ll}
t ::= \dots \mid \{l_i = t_i\}^{i \in 1..n} & \text{记录} \\
& \mid t.l & \text{投影} \\
v ::= \dots \mid \{l_i = v_i\}^{i \in 1..n} & \text{记录值} \\
T ::= \dots \mid \{l_i : T_i\}^{i \in 1..n} & \text{记录类型} \\
\mathcal{R}_j(s) \triangleq \{\{l_i = v_i\}^{i < j}, l_j = s, \{l_i = t_i\}^{i > j}\} \\
\\
\frac{}{\{l_i = v_i\}^{i \in 1..n}.l_j \rightarrow v_j} \text{E-PROJRCd} & \frac{t_j \rightarrow t'_j}{\mathcal{R}_j(t_j) \rightarrow \mathcal{R}_j(t'_j)} \text{E-RCD} \\
\\
\frac{\Gamma \vdash t_i : T_i \quad (\text{对每个 } i)}{\Gamma \vdash \{l_i = t_i\}^{i \in 1..n} : \{l_i : T_i\}^{i \in 1..n}} \text{T-RCD} & \frac{\Gamma \vdash t_1 : \{l_i : T_i\}^{i \in 1..n}}{\Gamma \vdash t_1.l_j : T_j} \text{T-PROJ}
\end{array}$$

图 11-7: 记录

记录的规则见图11-7。唯一值得注意的是 E-PROJRCd。这条规则没有在前提中明确写出“所投影的标签正是记录中第 j 个字段的标签”，而是直接用下标 j 表达这一对应关系。具体说，若 $\{l_i = v_i\}^{i \in 1..n}$ 是记录，且 l_j 是其第 j 个字段的标签，则 $\{l_i = v_i\}^{i \in 1..n}.l_j$ 单步求值为第 j 个值 v_j 。也可以把这一对应关系改写为显式前提，从而不再依赖上述简写；E-PROJTUPLE 中的类似写法也可同样展开，但规则会冗长许多。

练习 11.8.1 ($\star, \rightarrow$)。把 E-PROJRCd 写成更显式的形式，以作比较。

查看练习 11.8.1 的译者参考解答

注意，图11-6和11-7左上角都用同一个符号 {} 标识本图引入的语言构造。事实上，只需让标签集合同时包含字母标识符和自然数，就能把元组作为记录的特例。记录的第 i 个字段若以 i 为标签，就省略标签。例如， $\{\text{Bool}, \text{Nat}, \text{Bool}\}$ 是 $\{1 : \text{Bool}, 2 : \text{Nat}, 3 : \text{Bool}\}$ 的缩写。这项约定其实还允许混合命名字段与位置字段：把 $\{a : \text{Bool}, \text{Nat}, c : \text{Bool}\}$ 视为 $\{a : \text{Bool}, 2 : \text{Nat}, c : \text{Bool}\}$ 的缩写，尽管实践中可能用处不大。许多语言仍让元组和记录在记法上保持区别，理由更实际：编译器对二者采用不同实现。

各程序语言对记录字段次序的处理并不相同。在许多语言中，记录值和记录类型里的字段次序都不影响含义；也就是说， $\{\text{partno} = 5524, \text{cost} = 30.27\}$ 和 $\{\text{cost} = 30.27, \text{partno} = 5524\}$ 有相同含义和相同类型，类型可以用两种次序书写。这里选择另一种方案：二者是不同的记录值，分别具有字段次序相应的不同类型。第十五章会采用更宽松的次序观念，引入一种子类型关系，使这两种记录类型相互等价——彼此都是对方的子类型——从而一种类型的项可以用在预期另一种类型的任意上下文中。在有子类型化时，采用有序还是无序记录会对性能产生重要影响，§15.6 会继续讨论。不过，一旦决定采用无序记录，是从一开始就把记录当作无序的，还是原始地把字段当作有序的，再给出允许忽略次序的规则，就纯粹是偏好问题。这里采用后一方案，因为它让我们能够讨论两种变体。

练习 11.8.2 (***). 这里的记录使用投影操作，每次提取一个字段。许多高级程序语言还提供模式匹配语法，一次提取全部字段，使一些程序可以写得简洁得多。模式通常还能嵌套，从而容易地从复杂嵌套数据结构中提取部分内容。

可以为带记录的无类型 lambda 演算加入一种简单模式匹配：添加模式这一新的句法范畴，并在项语法中增加模式匹配构造，见图11-8。请为该系统添加类型：

1. 给出新构造的类型规则；过程中可按需要修改语法。
2. 概述整个演算的类型保持性与进展性证明。无需写出完整证明，只需按正确顺序陈述所需引理。

查看原书附录 A 中的 11.8.2 解答

$\{\}, \text{let } p$ (无类型)

扩展 图11-7 与图11-4

$$\begin{array}{ll}
 p ::= x & \text{模式：变量模式} \\
 \quad | \quad \{l_i = p_i\}^{i \in 1..n} & \text{记录模式} \\
 t ::= \dots & \text{项} \\
 \quad | \quad \text{let } p = t \text{ in } t & \text{模式匹配}
 \end{array}$$

$$\begin{array}{c}
 \frac{\text{match}(p, v_1) = \sigma}{\text{let } p = v_1 \text{ in } t_2 \longrightarrow \sigma(t_2)} \text{E-LETV} \qquad \frac{t_1 \longrightarrow t'_1}{\text{let } p = t_1 \text{ in } t_2 \longrightarrow \text{let } p = t'_1 \text{ in } t_2} \text{E-LET} \\
 \\
 \frac{}{\text{match}(x, v) = [x \mapsto v]} \text{M-VAR} \qquad \frac{\text{match}(p_i, v_i) = \sigma_i \quad (\text{对每个 } i)}{\text{match}(\{l_i = p_i\}^{i \in 1..n}, \{l_i = v_i\}^{i \in 1..n}) = \sigma_1 \circ \dots \circ \sigma_n} \text{M-RCD}
 \end{array}$$

图 11-8: 无类型记录模式

模式匹配的计算规则推广了图11-4 的 let 绑定规则。它依赖辅助匹配函数：给定模式 p 与值 v ，该函数要么失败，表示 v 不匹配 p ；要么产生一个替换，把 p 中的变量映射到 v 中相应的部分。例如，

$$\text{match}(\{x, y\}, \{5, \text{true}\}) = [x \mapsto 5, y \mapsto \text{true}],$$

$$\text{match}(x, \{5, \text{true}\}) = [x \mapsto \{5, \text{true}\}],$$

而 $\text{match}(\{x\}, \{5, \text{true}\})$ 失败。E-LETV 用 match 计算 p 中变量的适当替换。

匹配函数本身由另一组推导规则定义。M-VAR 说明：若模式只是变量 x ，它便能匹配任意值 v ，并产生替换 $[x \mapsto v]$ ；这个替换表示，随后用完整的值 v 替换 let 体中自由出现的 x 。M-RCD 说明：要把记录模式 $\{l_i = p_i\}^{i \in 1..n}$ 与长度相同、标签相同的记录值 $\{l_i = v_i\}^{i \in 1..n}$ 匹配，逐一把子模式 p_i 与对应值 v_i 匹配，得到替换 σ_i ，再把这些替换合并为最终结果。由于规定同一变量在整个模式中至多出现一次，各个 σ_i 所绑定的变量彼此不同，不会发生冲突。因此，这里无须真正按次序复合替换；只要汇总各个 σ_i 中的变量绑定，也就是取它们的并集即可。

译者注：M-Var 与 M-Rcd 要按模式的结构递归配合使用。例如，匹配 $\{a = x, b = y\}$ 与 $\{a = 5, b = \text{true}\}$ 时，外层使用 M-Rcd，把问题分解为 $\text{match}(x, 5)$ 和 $\text{match}(y, \text{true})$ ，这两个子匹配再分别使用 M-Var。因此，M-Var 所说的“完整值”是当前这一次匹配所面对的完整值：变量模式位于顶层时，它接收整个记录；位于记录模式的字段内部时，它只接收对应字段的完整值。若子模式仍是记录模式，就继续递归使用 M-Rcd。

11.9 和类型

许多程序需要处理异质的值集合。例如，二叉树结点可以是叶子，也可以是带两个子结点的内部结点；列表单元可以是 `nil`，也可以是携带表头和表尾的 `cons`；编译器中的抽象语法树结点可以表示变量、抽象、应用等。⁶支持这种编程方式的类型论机制是变体类型。

在 §11.10 完整引入变体之前，先考察较简单的二元和类型 (*sum type*)。和类型描述的值集合恰好取自两个给定类型。例如，设不同种类的地址簿记录由以下类型表示：

$$\text{PhysicalAddr} = \{\text{firstlast} : \text{String}, \text{addr} : \text{String}\},$$

$$\text{VirtualAddr} = \{\text{name} : \text{String}, \text{email} : \text{String}\}$$

若想统一处理两种记录，例如构造同时包含两类记录的列表，可以引入和类型：

$$\text{Addr} = \text{PhysicalAddr} + \text{VirtualAddr}$$

它的每个元素要么是 `PhysicalAddr`，要么是 `VirtualAddr`。⁷

`Addr` 的值由带标签的地址记录构成；这些记录的类型是 `PhysicalAddr` 或 `VirtualAddr`。例如，若 $pa : \text{PhysicalAddr}$ ，则 $\text{inl } pa : \text{Addr}$ 。标签名 `inl` 与 `inr` 来自把它们看作下面两个注入函数：

$$\text{inl} : \text{PhysicalAddr} \rightarrow \text{PhysicalAddr} + \text{VirtualAddr}$$

$$\text{inr} : \text{VirtualAddr} \rightarrow \text{PhysicalAddr} + \text{VirtualAddr}$$

它们分别把 `PhysicalAddr` 与 `VirtualAddr` 的值注入和类型 `Addr` 的左分量与右分量；这里的“左”和“右”对应 `PhysicalAddr + VirtualAddr` 中加号两侧的类型。不过，在这里的形式系统中，二者是专门的语言构造，而不是普通函数。

一般地， $T_1 + T_2$ 的值由带 `inl` 标签的 T_1 值和带 `inr` 标签的 T_2 值共同构成。

为使用和类型的元素，引入 `case` 构造，以区分给定值来自和的左分支还是右分支。例如，可以这样从 `Addr` 提取姓名：

$$\text{getName} = \lambda a : \text{Addr}.$$

$$\text{case } a \text{ of}$$

$$\text{inl } x \Rightarrow x.\text{firstlast}$$

$$| \text{inr } y \Rightarrow y.\text{name}$$

参数 a 是以 `inl` 标注的 `PhysicalAddr` 时，`case` 取第一分支，把 x 绑定到该地址值；第一分支的体再提取 x 的 `firstlast` 字段并返回。类似地，若 a 是以 `inr` 标注的 `VirtualAddr`，就选择第二分支，返回其 `name` 字段。因此，`getName` 的类型是 `Addr → String`。

⁶这些例子与现实中的大多数变体类型用法一样，还涉及递归类型：列表去掉第一个元素后剩余的部分，本身仍然是列表，等等。第二十章会回到递归类型。

⁷`fullsimple` 没有实际支持这里描述的二元和构造，只支持下一节更一般的变体。

+

扩展 $\lambda_{\rightarrow}$ (图 9-1)

$$\begin{array}{ll}
t ::= \dots \mid \text{inl } t \mid \text{inr } t & \text{注入} \\
\mid \text{case } t \text{ of } \text{inl } x \Rightarrow t \mid \text{inr } x \Rightarrow t & \text{分支分析} \\
v ::= \dots \mid \text{inl } v \mid \text{inr } v & \text{和类型值} \\
T ::= \dots \mid T + T & \text{和类型} \\
\\
\frac{}{\text{case (inl } v_0) \text{ of } \text{inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2 \longrightarrow [x_1 \mapsto v_0]t_1} & \text{E-CASEINL} \\
\\
\frac{}{\text{case (inr } v_0) \text{ of } \text{inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2 \longrightarrow [x_2 \mapsto v_0]t_2} & \text{E-CASEINR} \\
\\
\frac{t_0 \longrightarrow t'_0}{\text{case } t_0 \text{ of } \dots \longrightarrow \text{case } t'_0 \text{ of } \dots} & \text{E-CASE} \quad \frac{t_1 \longrightarrow t'_1}{\text{inl } t_1 \longrightarrow \text{inl } t'_1} & \text{E-INL} \quad \frac{t_1 \longrightarrow t'_1}{\text{inr } t_1 \longrightarrow \text{inr } t'_1} & \text{E-INR} \\
\\
\frac{\Gamma \vdash t_1 : T_1}{\Gamma \vdash \text{inl } t_1 : T_1 + T_2} & \text{T-INL} \quad \frac{\Gamma \vdash t_2 : T_2}{\Gamma \vdash \text{inr } t_2 : T_1 + T_2} & \text{T-INR} \\
\\
\frac{\Gamma \vdash t_0 : T_1 + T_2 \quad \Gamma, x_1 : T_1 \vdash t_1 : T \quad \Gamma, x_2 : T_2 \vdash t_2 : T}{\Gamma \vdash \text{case } t_0 \text{ of } \text{inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2 : T} & \text{T-CASE}
\end{array}$$

图 11-9: 和类型

图11-9 形式化了上述直觉。项语法加入左右注入与 case 构造，类型语法加入和构造子。求值加入 case 的两条“beta 归约”规则：若第一个子项归约为以 inl 标注的值 v_0 ，选择第一分支并以 v_0 替换受约束变量；若是以 inr 标注的值，则选择第二分支。其他规则在 case 的第一个子项内，以及 inl 与 inr 标签下执行求值。

加标签的类型规则很直接：要证明 $\text{inl } t_1$ 具有和类型 $T_1 + T_2$ ，只需证明 t_1 具有类型 T_1 ，也就是该和类型的左分量类型；inr 类似。对 case，先检查第一个子项具有和类型 $T_1 + T_2$ ，再分别假定两分支受约束变量 x_1, x_2 的类型为 T_1, T_2 ，检查两分支体 t_1, t_2 具有同一结果类型 T ；整个 case 的结果类型就是 T 。按此前定义的惯例，图中没有显式说明 x_1, x_2 的作用域分别是分支体 t_1, t_2 ，但可以从 T-CASE 扩展上下文的方式读出这一事实。

练习 11.9.1 (★★). 注意 case 的类型规则与图8-1 中 if 的规则十分相似：可以把 if 看作一种退化的 case，不向分支传递任何信息。请用和类型与 Unit 把 true、false 和 if 定义为派生形式，从而形式化这一直觉。

查看原书附录 A 中的 11.9.1 解答

和类型与类型唯一性

纯 $\lambda_{\rightarrow}$ 类型关系的大部分性质（见 §9.3）都能扩展到带和类型的系统，但有一项重要性质失效：[类型唯一性定理9.3.3](#)。困难来自 inl 与 inr 两个加标签构造。例如 T-INL 说明：一旦证

明 t_1 是 T_1 的元素, 就能对任意类型 T_2 推导 $\text{inl } t_1 : T_1 + T_2$ 。因此既能推导 $\text{inl } 5 : \text{Nat} + \text{Nat}$, 也能推导 $\text{inl } 5 : \text{Nat} + \text{Bool}$, 以及无穷多个其他类型。

类型不再唯一, 意味着不能再像此前所有特性那样, 只靠“从下往上读规则”构造类型检查算法。此时有几种选择:

1. 让类型检查算法更复杂, 以某种方式“猜测” T_2 。具体地说, 暂时让 T_2 保持未定, 稍后尝试发现它应当取什么值。第二十二章讨论类型重构时会详细研究这类技术。
2. 细化类型语言, 使 T_2 的所有可能值能以某种方式得到统一表示。第十五章讨论子类型化时会考察这项方案。
3. 要求程序员显式标注预期的 T_2 。这项方案最简单, 而且不像初看那么不实用: 在完整语言设计中, 这类显式标注往往可以“附着”在其他语言构造上, 几乎完全隐藏; 下一节还会回到这一点。眼下采用这个方案。

图11-10 给出相对于图11-9 所需的扩展。不再只写 $\text{inl } t$ 或 $\text{inr } t$, 而写 $\text{inl } t \text{ as } T$ 或 $\text{inr } t \text{ as } T$, 其中 T 指定注入元素要属于的完整和类型。T-INL 与 T-INR 先检查被注入的项确实属于相应分支, 再以声明的和类型作为注入的类型。为了不在规则中反复书写 $T_1 + T_2$, 语法允许任意类型 T 作为注入标注; 若注入是良类型的, 类型规则会保证注解总是和类型。这种类型注解语法意在让人联想到 §11.4 的类型标注; 实际上, 可以把它看作语法上强制要求的类型标注。

+

扩展 $\lambda \rightarrow$ (图 11-9)

$$\begin{array}{c}
 t ::= \dots \mid \text{inl } t \text{ as } T \mid \text{inr } t \text{ as } T \\
 \\
 \frac{\Gamma \vdash t_1 : T_1}{\Gamma \vdash \text{inl } t_1 \text{ as } (T_1 + T_2) : T_1 + T_2} \text{ T-INL} \quad \frac{\Gamma \vdash t_2 : T_2}{\Gamma \vdash \text{inr } t_2 \text{ as } (T_1 + T_2) : T_1 + T_2} \text{ T-INR}
 \end{array}$$

图 11-10: 和类型 (具有唯一类型)

11.10 变体

正如二元积类型可以推广为带标签的记录类型, 二元和类型也可以推广为带标签的变体 (variant)。不再写 $T_1 + T_2$, 而写 $\langle l_1 : T_1, l_2 : T_2 \rangle$, 其中 l_1, l_2 为变体标签; 不再写 $\text{inl } t \text{ as } T_1 + T_2$, 而写 $\langle l_1 = t \rangle \text{ as } \langle l_1 : T_1, l_2 : T_2 \rangle$; case 分支也不用 inl、inr 标注, 而使用与和类型相同的标签。这样, 上一节的地址例子变为:

```

Addr = ⟨physical : PhysicalAddr, virtual : VirtualAddr⟩,
a = ⟨physical = pa⟩ as Addr : Addr,
getName = λa : Addr. case a of
  ⟨physical = x⟩ ⇒ x.firstlast
  | ⟨virtual = y⟩ ⇒ y.name : Addr → String

```

$\langle \rangle$ 扩展 $\lambda_{\rightarrow}$ (图 9-1)

$$\begin{array}{ll}
t ::= \dots \mid \langle l = t \rangle \text{ as } T & \text{变体} \\
\mid \text{ case } t \text{ of } \langle l_i = x_i \rangle \Rightarrow t_i \ (i \in 1..n) & \text{分支分析} \\
v ::= \dots \mid \langle l = v \rangle \text{ as } T & \text{变体值} \\
T ::= \dots \mid \langle l_i : T_i \rangle^{i \in 1..n} & \text{变体类型}
\end{array}$$

$$\frac{t_1 \longrightarrow t'_1}{\langle l = t_1 \rangle \text{ as } T \longrightarrow \langle l = t'_1 \rangle \text{ as } T} \text{E-VARIANT}$$

$$\frac{}{\text{case } (\langle l_j = v \rangle \text{ as } T) \text{ of } \langle l_i = x_i \rangle \Rightarrow t_i \ (i \in 1..n) \longrightarrow [x_j \mapsto v]t_j} \text{E-CASEVARIANT}$$

$$\frac{t_0 \longrightarrow t'_0}{\text{case } t_0 \text{ of } \dots \longrightarrow \text{case } t'_0 \text{ of } \dots} \text{E-CASE}$$

$$\frac{\Gamma \vdash t_j : T_j}{\Gamma \vdash \langle l_j = t_j \rangle \text{ as } \langle l_i : T_i \rangle^{i \in 1..n} : \langle l_i : T_i \rangle^{i \in 1..n}} \text{T-VARIANT}$$

$$\frac{\Gamma \vdash t_0 : \langle l_i : T_i \rangle^{i \in 1..n} \quad \Gamma, x_i : T_i \vdash t_i : T \text{ (对每个 } i)}{\Gamma \vdash \text{case } t_0 \text{ of } \langle l_i = x_i \rangle \Rightarrow t_i \ (i \in 1..n) : T} \text{T-CASE}$$

图 11-11: 变体

变体的形式定义见图11-11。注意，与 §11.8 的记录一样，这里变体类型中的标签次序具有意义。

译者注：原书图的值语法漏列了 $\langle l = v \rangle \text{ as } T$ ；本图已经按作者勘误补入。

可选值

变体的一种非常有用的惯用法是可选值 (*optional value*)。例如，

$$\begin{aligned}
\text{OptionalNat} = \langle & \text{none} : \text{Unit}, \\
& \text{some} : \text{Nat} \rangle
\end{aligned}$$

这个类型的值有两种形式：一种以 *none* 为标签，内部是唯一的 *unit* 值；另一种以 *some* 为标签，内部携带一个自然数。换言之，*OptionalNat* 的值与“所有自然数再加一个特殊值 *none*”一一对应；在这个意义上，二者同构。例如，类型名 *Table* 在这里表示一种映射表：

$$\text{Table} = \text{Nat} \rightarrow \text{OptionalNat}$$

它用于表示从数字到数字的 *有限部分映射*；这种映射可以只在部分输入上有结果。对 $t : \text{Table}$ 和输入 m ，若 $tm = \langle \text{some} = n \rangle$ ，就表示映射在 m 处有定义，并把 m 映射到 n ；若结果带有 *none* 标签，就表示映射在 m 处没有定义。因此，映射的定义域就是所有能得到 *some* 结果的输入。空映射表定义为

$$\text{emptyTable} = \lambda n : \text{Nat}. \langle \text{none} = \text{unit} \rangle \text{ as OptionalNat} \quad : \quad \text{Table}$$

它是对每个输入都返回 *none* 的常函数，所以所表示的部分映射处处没有定义。下面的构造函数

```
extendTable = λt : Table.λm : Nat.λv : Nat.λn : Nat.
  if equal n m then ⟨some = v⟩ as OptionalNat
  else t n
```

具有类型 $\text{Table} \rightarrow \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Table}$ ，它接受旧映射表 t 、输入 m 和输出 v ，并返回一张新映射表。查询新表的输入 n 时，若 $n = m$ ，就返回 $\langle \text{some} = v \rangle$ ；否则仍返回旧表的查询结果 $t n$ 。因此，它会增加映射 $m \mapsto v$ ，或者覆盖 m 原有的映射。函数 *equal* 在 [练习11.11.1](#) 的解答中定义。

要使用映射表的查找结果，可以在外面包一层 *case*。例如，若 t 是映射表，要查找输入 5，可以写：

```
x = case t 5 of  ⟨none = u⟩ ⇒ 999
                | ⟨some = v⟩ ⇒ v,
```

若 t 在 5 上没有定义，第一分支就把 999 作为 x 的默认值；若查找结果为 $\langle \text{some} = v \rangle$ ，第二分支就把实际结果 v 赋给 x 。

许多语言内建了对可选值的支持。例如 OCaml 预定义类型构造子 *option*，典型 OCaml 程序中的许多函数都会产生 *option*。C、C++ 与 Java 等语言的 *null* 值本质上也相当于一种没有显式标签的 *option*。这些语言中类型为 T 的变量——这里 T 是“引用类型”，即在堆上分配的东西——实际可以保存特殊值 *null*，也可以保存指向某个 T 型值的指针。因此，这种变量的真实类型是 $\text{Ref}(\text{Option}(T))$ ，其中 $\text{Option}(T) = \langle \text{none} : \text{Unit}, \text{some} : T \rangle$ 。[第十三章](#)会详细讨论 *Ref* 构造子。

枚举

变体类型有两种“退化情形”非常有用，值得特别说明：枚举类型和单字段变体。

枚举类型 (*enumerated type*) (或 枚举 (*enumeration*)) 是每个标签关联的字段类型均为 *Unit* 的变体类型。例如，工作日可以定义为：

```
Weekday = ⟨monday : Unit, tuesday : Unit, wednesday : Unit,
           thursday : Unit, friday : Unit⟩
```

它的元素形如 $\langle \text{monday} = \text{unit} \rangle$ as *Weekday*。由于 *Unit* 只有 *unit* 一个元素，*Weekday* 恰好有五个值，与五个工作日一一对应。可以用 *case* 定义枚举上的计算，例如：

```
nextBusinessDay = λw : Weekday. case w of
  ⟨monday = x⟩ ⇒ ⟨tuesday = unit⟩ as Weekday
| ⟨tuesday = x⟩ ⇒ ⟨wednesday = unit⟩ as Weekday
| ⟨wednesday = x⟩ ⇒ ⟨thursday = unit⟩ as Weekday
| ⟨thursday = x⟩ ⇒ ⟨friday = unit⟩ as Weekday
| ⟨friday = x⟩ ⇒ ⟨monday = unit⟩ as Weekday
```

显然，这里的具体语法并不适合让此类程序易写易读。一些语言从 Pascal 开始，提供声明和使用枚举的专门语法；另一些语言，如 ML，则把枚举作为变体的特例。

单字段变体

另一个有意思的特例，是只含单个标签 l 的变体类型：

$$V = \langle l : T \rangle$$

它乍看之下似乎没有多大用处： V 的元素与字段类型 T 的元素一一对应，因为 V 的每个成员都恰好形如 $\langle l = t \rangle$ ，其中 $t : T$ 。但关键在于，不先拆包就不能把 T 上的通常操作应用于 V 的元素； V 不会意外地被误当成 T 。

例如，设想编写一个对多种货币作金融计算的程序，其中包含美元与欧元之间的换算函数。若两者都表示成 `Float`，可以写：

```
dollars2euros = λd : Float. timesfloat d 1.1325 : Float → Float,
```

```
euros2dollars = λe : Float. timesfloat e 0.883 : Float → Float
```

这里 `timesfloat : Float → Float → Float` 做浮点乘法。若 `mybankbalance = 39.50`，将其换算为欧元再换回美元，得到：

```
euros2dollars (dollars2euros mybankbalance) ► 39.49990125 : Float
```

这完全合理。但也很容易作毫无意义的操作，例如把银行余额连续两次从美元“换算”为欧元：

```
dollars2euros (dollars2euros mybankbalance) ► 50.660971875 : Float
```

所有金额都只是浮点数，类型系统无从阻止这种荒谬操作。

若把美元和欧元定义为底层表示都是浮点数、彼此却不同的变体类型：

```
DollarAmount = ⟨dollars : Float⟩,    EuroAmount = ⟨euros : Float⟩,
```

便能定义只接受正确货币金额的安全换算函数：

```
dollars2euros = λd : DollarAmount. case d of ⟨dollars = x⟩ ⇒
```

```
    ⟨euros = timesfloat x 1.1325⟩ as EuroAmount,
```

```
    ► ⟨fun⟩ : DollarAmount → EuroAmount,
```

```
euros2dollars = λe : EuroAmount. case e of ⟨euros = x⟩ ⇒
```

```
    ⟨dollars = timesfloat x 0.883⟩ as DollarAmount,
```

```
    ► ⟨fun⟩ : EuroAmount → DollarAmount
```

类型检查器现在能追踪计算使用的货币，并提醒我们怎样解释最终结果：

```
mybankbalance = ⟨dollars = 39.50⟩ as DollarAmount,
```

```
euros2dollars (dollars2euros mybankbalance)
```

```
    ► ⟨dollars = 39.49990125⟩ as DollarAmount
```

```
: DollarAmount
```

若写下毫无意义的二次换算，类型无法匹配，程序会被正确拒绝：

```
dollars2euros (dollars2euros mybankbalance)
```

► Error: parameter type mismatch

变体与数据类型

形如 $\langle l_i : T_i \rangle_{i \in 1..n}$ 的变体类型 T ，大致对应以下 ML 数据类型：⁸

```
type T = l1 of T1
      | l2 of T2
      | ...
      | ln of Tn
```

Rust 对照实现（译者增补）

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Variant<T1, T2> {
    L1(T1),
    L2(T2),
}
```

但有几项区别值得注意：

1. 一项细小却可能令人困惑的区别，是这里假定的标识符大小写约定不同于 OCaml。OCaml 中类型必须以小写字母开头，数据类型构造子——按我们的术语就是标签——则以大写字母开头。因此严格地说，上面的声明应写成：

```
type t = L1 of t1 | ... | Ln of tn
```

为避免项 t 与类型 T 混淆，余下讨论忽略 OCaml 的约定，继续使用本书的约定。

2. 最有意思的区别是，使用构造子 l_i 把 T_i 的元素注入数据类型 T 时，OCaml 不要求类型注解，只写 $l_i(t)$ 。OCaml 能在仍然保持类型唯一的同时省去标注，是因为使用数据类型 T 前必须先声明它，而且 T 的标签不能被同一作用域中声明的其他数据类型使用。因此，类型检查器看到 $l_i(t)$ 时，知道标注只可能是 T 。实际上，标注“隐藏”在标签本身之中。

这个技巧消除了许多多余标注，但也会招致用户抱怨，因为不同数据类型不能共享标签——至少同一模块内不能。第十五章会看到另一种省略标注且没有这一缺点的方法。

3. OCaml 的另一个便捷技巧是：数据类型定义中若某标签关联的类型只是 `Unit`，就可以把类型完全省略。例如枚举可以写成：

⁸这里为与其他实现章节保持一致，使用 OCaml 的数据类型具体语法；这类数据类型源自早期 ML 方言，Standard ML 与 Haskell 等 ML 近亲中也有本质相同的形式。数据类型和模式匹配可以说是这些语言对日常编程最有用的优势之一。

```
type Weekday = monday | tuesday | wednesday | thursday | friday
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Weekday {
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
}
```

而不必写成每个构造子都带 `of Unit` 的形式。类似地，单独的标签 `monday`（而不是把 `monday` 应用于平凡值 `unit`）就被视为 `Weekday` 型的值。

4. 最后，OCaml 数据类型实际上把变体类型与若干其他特性捆绑在一起；本书后面会逐一考察。

数据类型定义可以递归，也就是说，被定义的类型可以出现在定义体内。例如，标准的自然数列表定义中，标签 `cons` 所携带的对，其第二个元素仍然是 `NatList`：

```
type NatList = nil
              | cons of Nat * NatList
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum NatList {
    Nil,
    Cons(Nat, Box<NatList>),
}
```

OCaml 数据类型还可以用类型变量参数化，例如一般的列表定义：

```
type 'a List = nil
          | cons of 'a * 'a List
```

Rust 对照实现 (译者增补)

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum List<T> {
    Nil,
    Cons(T, Box<List<T>>),
}
```

从类型论角度看, `List` 可以视为一种函数, 称为 类型算子 (*type operator*): 它把每一种 `'a` 的选择映射为具体数据类型, 例如把 `Nat` 映射为 `NatList`。类型算子是第二十九章的主题。

从不交并的角度理解变体

和类型与变体类型有时称为不交并 (*disjoint union*)。 $T_1 + T_2$ 是 T_1, T_2 的“并”, 因为其元素包括来自两个类型的全部元素。之所以称为“不交并”, 是因为在合并之前, 来自 T_1 与 T_2 的元素会分别带上 `inl` 与 `inr` 标签, 使两类元素在合并后仍然可以区分。因此, 看到其中任意一个元素时, 总能判断它来自哪一个类型。“并类型”一词也用于没有标签、因而并非不交的并类型; §15.7 会加以介绍。

Dynamic 类型

即使在静态类型语言中, 程序有时也必须处理编译时尚不知道具体类型的数据。当数据来自另一台机器, 或者保存在外部文件系统或数据库中、跨越编译器的多次运行而长期存在时, 这种情况尤其常见。为安全处理这类数据, 许多语言允许程序在运行时检查值的类型。

一种很有吸引力的做法, 是引入 `Dynamic` 类型。它的每个值都把某个值 v 和类型标签 T 包装在一起; 标签记录了 $v : T$, 即 v 具有类型 T 。构造 `Dynamic` 类型的值时, 必须显式附上这个标签; 使用其中的值时, 则通过类型安全的 `typecase` 构造检查标签, 再按相应类型处理 v 。从这个角度看, `Dynamic` 就像一种具有无限多个分支的变体类型: 每一种类型都可以充当一个分支标签, 该分支携带的正是这种类型的值。也就是说, 它是一个以类型为标签的无限不交并。参见 Gordon (约 1980)、Mycroft (1983)、Abadi、Cardelli、Pierce 与 Plotkin (1991b)、Leroy 与 Mauny (1991)、Abadi、Cardelli、Pierce 与 Rémy (1995), 以及 Henglein (1994)。

11.11 一般递归

大多数程序语言的另一项机制是定义递归函数, 即 一般递归 (*general recursion*)。第五章已经看到, 在无类型 `lambda` 演算中, 可以借助 `fix` 组合子定义这类函数。

有类型系统中也能以相似方式定义递归函数。例如，下面的 `iseven` 在参数为偶数时返回 `true`，否则返回 `false`：

```
ff = λie : Nat → Bool. λx : Nat.
  if iszero x then true
  else if iszero (pred x) then false
  else ie (pred (pred x))
  : (Nat → Bool) → Nat → Bool,
iseven = fix ff : Nat → Bool,
iseven 7 ► false : Bool
```

高阶函数 `ff` 可以看作 `iseven` 的生成器。它接收函数 `ie`，并在输入大于 1 时调用 `ie (x-2)`。令 `iseven = fix ff` 后，求值规则把 `iseven` 展开为 `ff iseven`。因此，`ff` 中的参数 `ie` 指向最终的 `iseven` 本身：每次需要递归时，函数都会再次调用 `iseven`。这也说明了“不动点”一词的含义；`iseven` 经过 `ff` 后仍具有相同的行为。

还可以从正确输入范围的扩展来理解这个生成过程。假设 `ie` 在 0 到 n 上与 `iseven` 一致。对于 2 到 $n+2$ 之间的输入 x ，`ff ie` 会调用 `ie (x-2)`，而 $x-2$ 正好落在 0 到 n 之间。因此，`ie` 是 `iseven` 在 0 到 n 上的有限近似，`ff ie` 则是 `iseven` 在 0 到 $n+2$ 上的有限近似。每应用一次 `ff`，能够正确处理的输入范围就扩大两个数；不断继续这一过程，任何给定的自然数最终都会落入这个范围。由此可以理解，`fix ff` 得到的不动点能够正确判断任意自然数的奇偶性。

不过，与无类型系统有一项重要区别：`fix` 本身无法在简单类型 `lambda` 演算中定义。第十二章会看到，只用简单类型，任何可能导致非终止计算的表达式都无法赋型。⁹因此，我们无法只用简单类型 `lambda` 演算已有的构造写出 `fix` 组合子，只能扩展语言的抽象语法，把 `fix t` 加作一种新的基本项形式；其求值规则模拟无类型 `fix` 组合子的行为，类型规则则刻画预期用法。

⁹后面的第十三章与 第二十章会看到，简单类型的一些扩展重新获得了在系统内部定义 `fix` 的能力。

Fix

扩展 $\lambda_{\rightarrow}$ (图 9-1)

$$\begin{array}{c}
\frac{}{\text{fix } (\lambda x : T_1. t_2) \longrightarrow [x \mapsto \text{fix } (\lambda x : T_1. t_2)] t_2} \text{E-FIXBETA} \\
\\
t ::= \dots \mid \text{fix } t \text{ 不动点} \\
\\
\frac{t_1 \longrightarrow t'_1}{\text{fix } t_1 \longrightarrow \text{fix } t'_1} \text{E-FIX} \\
\\
\frac{\Gamma \vdash t_1 : T_1 \rightarrow T_1}{\Gamma \vdash \text{fix } t_1 : T_1} \text{T-FIX} \\
\\
\text{letrec } x : T_1 = t_1 \text{ in } t_2 \triangleq \text{let } x = \text{fix } (\lambda x : T_1. t_1) \text{ in } t_2
\end{array}$$

图 11-12: 一般递归

译者注: T-FIX 的形式来自不动点等式。设 $h = \text{fix } t_1$ 且 $h : T_1$ 。不动点满足 $t_1 h = h$ ，所以 t_1 必须接收一个 T_1 类型的参数，并返回同为 T_1 类型的结果。这就得到前提 $\Gamma \vdash t_1 : T_1 \rightarrow T_1$ 和结论 $\Gamma \vdash \text{fix } t_1 : T_1$ 。在上面的例子中， $T_1 = \text{Nat} \rightarrow \text{Bool}$ ，而 $ff : (\text{Nat} \rightarrow \text{Bool}) \rightarrow (\text{Nat} \rightarrow \text{Bool})$ ，因此 $\text{fix } ff : \text{Nat} \rightarrow \text{Bool}$ 。同一类型条件也保证了 E-FIXBETA 中的替换不会改变归约前后的类型。

带自然数与 `fix` 的简单类型 lambda 演算长期以来一直是程序语言研究者偏爱的实验对象，因为它是最简单却能呈现完全抽象 (*full abstraction*) (即语义模型认为两个程序等价，当且仅当任何程序上下文也无法从运行行为上区分它们) 等多种微妙语义现象的语言 (Plotkin, 1977; Hyland 与 Ong, 2000; Abramsky, Jagadeesan 与 Malacaria, 2000)。它通常称为可计算函数程序设计语言 (*Programming Language for Computable Functions, PCF*)。

练习 11.11.1 (★★). 用 `fix` 定义 *equal*、*plus*、*times* 和 *factorial*。

查看原书附录 A 中的 11.11.1 解答

通常，我们把 `fix` 应用于一个以函数为输入、仍以函数为输出的生成器，由此得到一个递归函数。这个递归函数就是该生成器的不动点。不过，T-FIX 中的 T 并不限于函数类型。这项额外能力有时很方便。例如，可以先定义一个生成器：它接收一条各字段的值都是函数的记录，并返回一条同类型的记录。对这个生成器应用 `fix`，就会得到一条记录，其中包含一组相互递归调用的函数。下面的实现把 `iseven` 和辅助函数 `isodd` 放在同一条记录中，并借助 `fix` 使这两个函数能够在递归过程中调用对方。首先定义生成器 *ff*。它以一条名为 *ieio* 的记录为参数；这条记录包含 `iseven` 和 `isodd` 两个函数字段。在新记录的两个函数体中，需要递归调用时，分别调用 *ieio.isodd* 和 *ieio.iseven*：

$$\begin{aligned}
ff &= \lambda ieio : \{iseven : \text{Nat} \rightarrow \text{Bool}, isodd : \text{Nat} \rightarrow \text{Bool}\}. \\
&\quad \{iseven = \lambda x : \text{Nat}. \text{if iszero } x \text{ then true else } ieio.isodd(\text{pred } x), \\
&\quad \quad isodd = \lambda x : \text{Nat}. \text{if iszero } x \text{ then false else } ieio.iseven(\text{pred } x)\} \\
ff &\blacktriangleright \langle \text{fun} \rangle : \{iseven : \text{Nat} \rightarrow \text{Bool}, isodd : \text{Nat} \rightarrow \text{Bool}\} \\
&\quad \rightarrow \{iseven : \text{Nat} \rightarrow \text{Bool}, isodd : \text{Nat} \rightarrow \text{Bool}\}
\end{aligned}$$

ff 的输入与输出是同一种记录。对 ff 取不动点，得到记录 r 。由于 $r = ff\ r$ ，生成记录时作为参数传入的 $ieio$ 正是 r 本身，因此两个字段可以通过 $ieio$ 相互调用：

$$r = \text{fix } ff : \{iseven : \text{Nat} \rightarrow \text{Bool}, isodd : \text{Nat} \rightarrow \text{Bool}\}$$

投影第一个函数即得：

$$iseven = r.iseven : \text{Nat} \rightarrow \text{Bool}, \quad iseven\ 7 \blacktriangleright \text{false}$$

规则 T-FIX 对任意类型 T 都适用：只要 $t : T \rightarrow T$ ，就有 $\text{fix } t : T$ 。这会带来一个意外的结果：每个类型都有居留元。这里的“居留元”是指：无论给定什么类型 T ，都能找到一个不含自由变量的闭项 t ，使 $\vdash t : T$ 成立。不过，这个闭项可能永远不会结束求值。具体地，对每个 T 定义发散函数 $diverge_T$ ：

$$diverge_T = \lambda_ : \text{Unit}. \text{fix } (\lambda x : T. x) : \text{Unit} \rightarrow T$$

把 $diverge_T$ 应用于 unit 后，得到 $\text{fix } (\lambda x : T. x)$ 。这个项每次应用 E-FIXBETA 后仍是自身，因此求值永不终止。尽管如此， $\vdash diverge_T\ \text{unit} : T$ 仍然成立；所以它正是前面所说的 T 型闭项，只是永远不会产生结果。

还可以再作一项改进：为常见的“把变量绑定到递归定义结果”提供更方便的具体语法。在多数高级语言中，前面的第一个 `iseven` 定义大致写成：

```
letrec iseven : Nat → Bool = λx : Nat.
    if iszero x then true
    else if iszero (pred x) then false
    else iseven (pred (pred x))
in iseven 7,
```

结果为 `false : Bool`。递归绑定构造 `letrec` 很容易按图11-12 定义为派生形式。

练习 11.11.2 (*). 不用 `fix`，改用 `letrec` 重写 练习11.11.1 中 *plus*、*times* 与 *factorial* 的定义。

查看原书附录 A 中的 11.11.2 解答

关于不动点算子的更多资料，参见 Klop (1980) 与 Winskel (1993)。

11.12 列表

目前见过的类型特性可以分为两类：`Bool`、`Unit` 一类基本类型，以及 $\rightarrow$ 、 $\times$ 一类从旧类型构造新类型的类型构造子。另一个有用的类型构造子是 `List`。对每个类型 T ，类型 `List T` 描述元素取自 T 的有限长度列表。

图11-13 汇总列表的语法、语义和类型规则。除句法差异 (`List T` 而非 `T list` 等) 和所有句法形式都带显式类型注解之外，¹⁰ 这些列表与 ML 等函数式语言中的列表本质相同。

¹⁰ 多数显式注解实际上可以省略 (见练习11.12.2: 哪一个不能省?); 这里保留它们，是为了便于与 §23.4 中列表的编码比较。

B, List

扩展 $\lambda \rightarrow$ (图 9-1), 并带布尔值 (图 8-1)

$t ::= \dots \mid \text{nil}[T]$	空表
$\mid \text{cons}[T] \ t \ t$	列表构造
$\mid \text{isnil}[T] \ t$	空表测试
$\mid \text{head}[T] \ t$	表头
$\mid \text{tail}[T] \ t$	表尾
$v ::= \dots \mid \text{nil}[T] \mid \text{cons}[T] \ v \ v$	列表值
$T ::= \dots \mid \text{List } T$	列表类型

$\frac{t_1 \rightarrow t'_1}{\text{cons}[T] \ t_1 \ t_2 \rightarrow \text{cons}[T] \ t'_1 \ t_2} \text{E-CONS1}$	$\frac{t_2 \rightarrow t'_2}{\text{cons}[T] \ v_1 \ t_2 \rightarrow \text{cons}[T] \ v_1 \ t'_2} \text{E-CONS2}$
$\frac{}{\text{isnil}[S] \ (\text{nil}[T]) \rightarrow \text{true}} \text{E-ISNILNIL}$	$\frac{}{\text{isnil}[S] \ (\text{cons}[T] \ v_1 \ v_2) \rightarrow \text{false}} \text{E-ISNILCONS}$
$\frac{t_1 \rightarrow t'_1}{\text{isnil}[T] \ t_1 \rightarrow \text{isnil}[T] \ t'_1} \text{E-ISNIL}$	$\frac{}{\text{head}[S] \ (\text{cons}[T] \ v_1 \ v_2) \rightarrow v_1} \text{E-HEADCONS}$
$\frac{t_1 \rightarrow t'_1}{\text{head}[T] \ t_1 \rightarrow \text{head}[T] \ t'_1} \text{E-HEAD}$	$\frac{}{\text{tail}[S] \ (\text{cons}[T] \ v_1 \ v_2) \rightarrow v_2} \text{E-TAILCONS}$
$\frac{t_1 \rightarrow t'_1}{\text{tail}[T] \ t_1 \rightarrow \text{tail}[T] \ t'_1} \text{E-TAIL}$	$\frac{}{\Gamma \vdash \text{nil}[T_1] : \text{List } T_1} \text{T-NIL}$
$\frac{\Gamma \vdash t_1 : T_1 \quad \Gamma \vdash t_2 : \text{List } T_1}{\Gamma \vdash \text{cons}[T_1] \ t_1 \ t_2 : \text{List } T_1} \text{T-CONS}$	$\frac{\Gamma \vdash t_1 : \text{List } T_{11}}{\Gamma \vdash \text{isnil}[T_{11}] \ t_1 : \text{Bool}} \text{T-ISNIL}$
$\frac{\Gamma \vdash t_1 : \text{List } T_{11}}{\Gamma \vdash \text{head}[T_{11}] \ t_1 : T_{11}} \text{T-HEAD}$	$\frac{\Gamma \vdash t_1 : \text{List } T_{11}}{\Gamma \vdash \text{tail}[T_{11}] \ t_1 : \text{List } T_{11}} \text{T-TAIL}$

图 11-13: 列表

译者注：作者勘误指出，`fullsimple` 检查器实际上没有实现列表；这不改变这里给出的形式化定义。

元素类型为 T 的空表写作 $\text{nil}[T]$ 。在列表 t_2 前加入类型为 T 的新元素 t_1 得到的列表，写作 $\text{cons}[T] \ t_1 \ t_2$ 。列表 t 的表头和表尾分别写作 $\text{head}[T] \ t$ 与 $\text{tail}[T] \ t$ 。布尔谓词 $\text{isnil}[T] \ t$ 当且仅当 t 为空表时产生 `true`。¹¹

练习 11.12.1 (★★). 验证带布尔值与列表的简单类型 lambda 演算满足进展性和保持性定理。

查看原书附录 A 中的 11.12.1 解答

练习 11.12.2 (★★). 这里的列表呈现包含许多并非真正必要的类型注解，因为类型规则很容易从上下文推导出这些注解。能否删除全部类型注解？

查看原书附录 A 中的 11.12.2 解答

¹¹为求简单，这里采用列表的“head / tail / isnil 呈现”。从语言设计角度看，把列表作为数据类型，再用 case 表达式解构，或许更好，因为这样能把更多程序错误作为类型错误捕获。

第十二章 规范化

概述

本章考察纯简单类型 lambda 演算的另一项基本理论性质：良类型程序的求值保证在有限步内终止，也就是说，每个良类型项都是可规范化的 (*normalizable*)。

不同于此前考察的类型安全性质，规范化性质不能推广到完整程序语言，因为这类语言几乎总会用一般递归 (§11.11) 或递归类型 (第二十章) 等构造扩展简单类型 lambda 演算，而这些构造可以写出不终止程序。不过，§30.3 讨论 System F_ω 的元理论时，规范化问题会在类型层面再次出现：该系统的类型语言实际上包含一份简单类型 lambda 演算；类型检查算法能否终止，将取决于类型表达式上的“规范化”操作保证终止这一事实。

研究规范化证明还有一个理由：它们是类型论文献中最优美、也最令人震撼的数学成果之一，并且经常像本章这样，使用逻辑关系这一基本证明技术。

有些读者第一次阅读时也许愿意跳过本章；这样做不会妨碍后续章节。完整的章节依赖关系见 [图P-1](#)。

12.1 简单类型的规范化

这里考察以单一基本类型 A 为基础的简单类型 lambda 演算。该演算的规范化证明并非完全平凡，因为项的每次单步归约都可能复制子项中的可归约式。¹

练习 12.1.1 (★). 若试图直接对良类型项的大小作归纳来证明规范化，会在哪一步失败？

[查看原书附录 A 中的 12.1.1 解答](#)

这里的关键问题——许多归纳证明也一样——是找到足够强的归纳假设。为此，对每个类型 T ，我们在类型为 T 的闭项中定义一个集合 $\mathcal{R}_T$ 。下面按照类型 T 的结构，递归规定哪些闭项属于这个集合。我们把这些集合视为谓词，用 $\mathcal{R}_T(t)$ 表示 $t \in \mathcal{R}_T$ 。²

定义 12.1.2.

$$\begin{aligned}\mathcal{R}_A(t) &\iff t \text{ 的求值会在有限步内终止,} \\ \mathcal{R}_{T_1 \rightarrow T_2}(t) &\iff t \text{ 的求值会在有限步内终止, 并且对每个 } s : T_1, \\ &\quad \mathcal{R}_{T_1}(s) \text{ 都蕴含 } \mathcal{R}_{T_2}(ts)\end{aligned}$$

¹本章研究带一个基本类型 A (图11-1) 的简单类型 lambda 演算 (图9-1)。

²集合 $\mathcal{R}_T$ 有时称为 饱和集 (*saturated set*) 或 可归约性候选集 (*reducibility candidate*)。

这个定义给出了所需的加强归纳假设。我们的主要目标是证明全部程序——即全部基本类型的闭项——求值都会终止。然而，基本类型的闭项可能含有函数类型的子项，所以还需要知道这些子项的情况。

译者注：假设语言中有一个闭的基本值 $a : A$ ，并令

$$I = \lambda x : A. x, \quad F = \lambda f : A \rightarrow A. f a$$

那么

$$I : A \rightarrow A, \quad F : (A \rightarrow A) \rightarrow A, \quad FI : A$$

因此， FI 虽然是基本类型 A 的闭项，内部却含有两个函数类型的子项 F 和 I 。二者都是 lambda 抽象，本身已经是值；只知道它们自身的求值终止，并不能说明调用它们以后仍会终止。

对 I ，关系 $\mathcal{R}_{A \rightarrow A}(I)$ 要求：任取满足 $\mathcal{R}_A$ 的参数 u ，应用 Iu 后所得结果仍满足 $\mathcal{R}_A$ 。对 F ，关系 $\mathcal{R}_{(A \rightarrow A) \rightarrow A}(F)$ 要求：任取满足 $\mathcal{R}_{A \rightarrow A}$ 的函数 g ，应用 Fg 后所得结果满足 $\mathcal{R}_A$ 。在这个例子中可以取 $g = I$ ，于是

$$FI \longrightarrow Ia \longrightarrow a$$

这说明，要证明基本类型闭项的求值终止，还必须知道其中的函数类型子项在接受满足相应条件的参数后，所得结果的求值仍会终止。函数类型条件正是为表达这项更强的性质而设置的。

此外，只知道这些子项的求值会终止仍不够，因为把已经规范化的函数应用于已经规范化的参数会发生替换，从而可能产生更多求值步骤。因此，对函数类型的项需要更强的条件：它们自身的求值不仅应当终止，应用于求值会终止的参数后，所得结果的求值也应当终止。

假设我们想证明：所有类型为 A 的闭项都具有某项性质 P 。为了能够对类型作归纳，不能只规定 A 型项应当满足什么条件，还必须为每一种函数类型规定相应的条件。对于基本类型 A ，所要求的条件就是 P ；对于任意函数类型 $T_1 \rightarrow T_2$ ，则要求函数把满足 T_1 型条件的参数映射为满足 T_2 型条件的结果。因此，类型为 $A \rightarrow A$ 的函数必须保持性质 P ；类型为 $(A \rightarrow A) \rightarrow (A \rightarrow A)$ 的函数必须把“保持 P 的函数”映射为另一个“保持 P 的函数”；其余类型依此定义。

换言之，每一种类型 T 都对应一个谓词 $\mathcal{R}_T$ ，用于判断类型为 T 的项是否满足相应条件。这些随类型而变化、彼此配套的谓词，合称一个按类型索引的谓词族。像定义12.1.2 这样按照类型结构递归定义谓词，是逻辑关系 (*logical relation*) 证明法的典型做法。由于这里的 $\mathcal{R}_T(t)$ 每次只判断一个项 t ，更准确地说， $\mathcal{R}_T$ 是一个逻辑谓词 (*logical predicate*)。

我们用这个定义分两步证明规范化。第一步，观察每个集合 $\mathcal{R}_T$ 的每个元素都可规范化。第二步，证明每个类型为 T 的良类型闭项都是 $\mathcal{R}_T$ 的元素。

第一步直接来自 $\mathcal{R}_T$ 的定义。

引理 12.1.3. 若 $\mathcal{R}_T(t)$ ，则 t 的求值会在有限步内终止。

第二步拆成两个引理。首先， $\mathcal{R}_T$ 的成员关系在求值下不变。

引理 12.1.4. 若 $t : T$ 且 $t \longrightarrow t'$ ，则 $\mathcal{R}_T(t)$ 当且仅当 $\mathcal{R}_T(t')$ 。

证明. 对类型 T 的结构作归纳。首先，显然 t 的求值终止当且仅当 t' 的求值终止。若 $T = A$ ，无需再证。另一方面，设 $T = T_1 \rightarrow T_2$ 。

对“仅当”方向 ($\Rightarrow$), 假设 $\mathcal{R}_T(t)$, 并任取 $s : T_1$ 使 $\mathcal{R}_{T_1}(s)$ 。由定义, $\mathcal{R}_{T_2}(ts)$ 。而 $ts \rightarrow t's$, 于是把关于类型 T_2 的归纳假设应用于这一步, 得到 $\mathcal{R}_{T_2}(t's)$ 。这对任意 s 都成立, 所以由 $\mathcal{R}_T$ 的定义得到 $\mathcal{R}_T(t')$ 。“当”方向 ($\Leftarrow$) 类似。 $\square$

接下来要证明, 每个类型为 T 的良类型闭项都属于 $\mathcal{R}_T$ 。这里对类型推导作归纳——关于良类型项的证明若完全不在某处对类型推导作归纳, 反倒令人意外。唯一的技术困难是 lambda 抽象情形。既然是在作归纳, 要证明 $\lambda x : T_1. t_2$ 属于 $\mathcal{R}_{T_1 \rightarrow T_2}$, 自然应当使用归纳假设证明 $t_2 \in \mathcal{R}_{T_2}$ 。但 $\mathcal{R}_{T_2}$ 按定义是闭项集合, 而 t_2 可能自由出现 x , 所以这种说法没有意义。

解决办法是采用一个标准技巧, 适当推广归纳假设: 不再只证明涉及闭项的陈述, 而是覆盖开项 t 的所有闭实例。

引理 12.1.5. 若

$$x_1 : T_1, \dots, x_n : T_n \vdash t : T,$$

且 $v_1, \dots, v_n$ 是类型分别为 $T_1, \dots, T_n$ 的闭值, 并对每个 i 都有 $\mathcal{R}_{T_i}(v_i)$, 则

$$\mathcal{R}_T([x_1 \mapsto v_1] \cdots [x_n \mapsto v_n]t)$$

证明. 对 $x_1 : T_1, \dots, x_n : T_n \vdash t : T$ 的推导作归纳。最有意思的是抽象情形。

情形 T-VAR:

$$t = x_i, \quad T = T_i$$

立即成立。

情形 T-ABS:

$$\begin{aligned} t &= \lambda x : S_1. s_2, & x_1 : T_1, \dots, x_n : T_n, x : S_1 \vdash s_2 : S_2, \\ T &= S_1 \rightarrow S_2 \end{aligned}$$

令 w 表示对 t 完成引理所要求的替换:

$$w = [x_1 \mapsto v_1] \cdots [x_n \mapsto v_n]t$$

由于 $t = \lambda x : S_1. s_2$, 这些替换只改变函数体中自由出现的 $x_1, \dots, x_n$, 不会消去最外层的 lambda 抽象。因此

$$w = \lambda x : S_1. [x_1 \mapsto v_1] \cdots [x_n \mapsto v_n]s_2$$

右侧是 lambda 抽象, 所以 w 已经是值, 其求值会立即终止。这验证了 $\mathcal{R}_{S_1 \rightarrow S_2}(w)$ 定义中的第一个条件。

还需验证第二个条件: 任取 $s : S_1$, 若 $\mathcal{R}_{S_1}(s)$, 则必须证明

$$\mathcal{R}_{S_2}(ws)$$

现在任取一个满足上述条件的 s 。归纳假设要求用闭值替换函数体子推导上下文中的变量, 而 s 本身未必是值。由引理 12.1.3, s 的求值会终止, 因而存在某个闭值 v , 使

$$s \rightarrow^* v$$

沿这条求值序列反复应用引理12.1.4, 得到 $\mathcal{R}_{S_1}(v)$ 。

现在对函数体的子推导

$$x_1 : T_1, \dots, x_n : T_n, x : S_1 \vdash s_2 : S_2$$

使用归纳假设: 分别以闭值 $v_1, \dots, v_n, v$ 替换 $x_1, \dots, x_n, x$ 。归纳假设给出

$$\mathcal{R}_{S_2}([x_1 \mapsto v_1] \cdots [x_n \mapsto v_n][x \mapsto v]s_2)$$

另一方面, ws 的求值先把参数 s 求值为 v , 再进行 β 归约, 所以

$$ws \longrightarrow^* [x_1 \mapsto v_1] \cdots [x_n \mapsto v_n][x \mapsto v]s_2$$

再沿这条求值序列反复应用引理12.1.4, 便得到 $\mathcal{R}_{S_2}(ws)$ 。由于 s 是任取的, 第二个条件成立。结合前面已经验证的第一个条件, 得到 $\mathcal{R}_{S_1 \rightarrow S_2}(w)$, 即所需结论。

情形 T-APP:

$$t = t_1 t_2,$$

$$x_1 : T_1, \dots, x_n : T_n \vdash t_1 : T_{11} \rightarrow T_{12},$$

$$x_1 : T_1, \dots, x_n : T_n \vdash t_2 : T_{11}, \quad T = T_{12}$$

归纳假设分别给出

$$\mathcal{R}_{T_{11} \rightarrow T_{12}}([x_1 \mapsto v_1] \cdots [x_n \mapsto v_n]t_1)$$

与

$$\mathcal{R}_{T_{11}}([x_1 \mapsto v_1] \cdots [x_n \mapsto v_n]t_2)$$

由 $\mathcal{R}_{T_{11} \rightarrow T_{12}}$ 的定义可得

$$\mathcal{R}_{T_{12}}\left(\left([x_1 \mapsto v_1] \cdots [x_n \mapsto v_n]t_1\right) \left([x_1 \mapsto v_1] \cdots [x_n \mapsto v_n]t_2\right)\right),$$

也就是

$$\mathcal{R}_{T_{12}}\left([x_1 \mapsto v_1] \cdots [x_n \mapsto v_n](t_1 t_2)\right)$$

□

由上述两个引理可以直接推出规范化性质。考虑任意满足 $\vdash t : T$ 的闭项 t 。它的类型上下文为空, 因此在引理12.1.5 中取 $n = 0$, 无需替换任何自由变量, 便得到 $\mathcal{R}_T(t)$ 。再由引理12.1.3 可知 t 可规范化。

定理 12.1.6 (规范化). 若 $\vdash t : T$, 则 t 可规范化。

证明. 由引理12.1.5 得 $\mathcal{R}_T(t)$, 再由引理12.1.3 得 t 可规范化。□

练习 12.1.7 (推荐, ★★). 扩展本章的证明技术, 说明简单类型 lambda 演算加入布尔值 (图8-1) 与积 (图11-5) 之后仍然可规范化。

查看原书附录 A 中的 12.1.7 解答

译者注：定理12.1.6 正好回应了 §11.11 中关于 `fix` 的说明。本章考察的是纯简单类型 lambda 演算，其语法不含 `fix`。无类型 lambda 演算虽然能够定义不动点组合子，但这种定义需要自应用 xx 。若要给 xx 赋予简单类型，同一个 x 必须既具有函数类型 $U \rightarrow V$ ，又具有参数类型 U ，从而要求 $U = U \rightarrow V$ ；有限的简单类型无法满足这个等式。因此，无类型演算中的不动点组合子不能写成简单类型演算中的良类型项。

§11.11 把 `fix` 作为新的基本构造加入语言，并配套加入类型规则 T-FIX。加入这项扩展后，本章的规范化定理便不再适用。例如，对任意类型 T ，都有

$$\vdash \text{fix}(\lambda x : T. x) : T$$

但规则 E-FIXBETA 使它反复求值为自身：

$$\text{fix}(\lambda x : T. x) \longrightarrow \text{fix}(\lambda x : T. x) \longrightarrow \dots$$

所以它虽然是良类型闭项，求值却不会终止。简单类型保证终止的结论，适用于不含 `fix` 等一般递归机制的纯演算。

12.2 注记

理论文献最常把规范化性质表述为：采用完全（非确定性） β 归约的演算具有强规范化（*strong normalization*）。标准证明方法由 Tait (1967) 发明，经 Girard (1972、1989) 推广到 System F （见第二十三章），后来又由 Tait (1975) 简化。本章采用的呈现方式由 Martin Hofmann 在与作者的私下交流中提出，是把 Tait 的方法改写到按值调用系统的版本。逻辑关系证明技术的经典参考文献包括 Howard (1973)、Tait (1967)、Friedman (1975)、Plotkin (1973、1980) 和 Statman (1982、1985a、1985b)。Mitchell (1996) 与 Gunter (1992) 等语义学教材也讨论了这种技术。

这种规范化证明还可以转化为一种用于求值简单类型项的算法，称为以求值实现规范化（*normalization by evaluation*），或类型导向的部分求值（*type-directed partial evaluation*）(Berger, 1993; Danvy, 1998; 另见 Berger 与 Schwichtenberg, 1991; Filinski, 1999、2001; Reynolds, 1998a)。

译者注：原书在介绍“以求值实现规范化”时称“Tait 的强规范化证明与这种算法精确对应”；作者勘误指出，以求值实现规范化对应的是弱规范化（*weak normalization*），不能直接与强规范化证明等同。本译文据此将相应句子改为“这种规范化证明还可以转化为一种用于求值简单类型项的算法”，没有保留“精确对应”的说法。

第十三章 引用

概述

到目前为止，我们考察了多种纯语言特性，包括函数抽象、数字和布尔值一类基本类型，以及记录和变体一类结构类型。这些特性构成了多数程序语言的骨架：既包括 Haskell 一类纯函数式语言、ML 一类“以函数式为主”的语言，也包括 C 一类命令式语言和 Java 一类面向对象语言。

多数实用程序语言还包含多种非纯特性，无法用此前的简单语义框架描述。特别是，这些语言中的项求值除了产生结果，还可能给可变变量赋值（引用单元、数组、可变记录字段等），对文件、显示设备或网络连接执行输入输出，通过异常、跳转或续延作非局部控制转移，参与进程间同步与通信，等等。程序语言文献把计算的这类“副作用”更一般地称为计算效应（*computational effect*）。

本章将看到，怎样把一种计算效应——可变引用——加入此前的演算。主要扩展是显式处理存储（*store*）（或堆）。这项扩展很容易定义，最有意思的部分，是必须细化 [类型保持性定理13.5.3](#)的陈述。[第十四章](#)会考察另一类效应：异常与非局部控制转移。

13.1 引言

几乎每种程序语言¹都提供某种赋值操作，用来改变先前分配的一块存储的内容。² 在一些语言中，尤其是 ML 及其近亲，名字绑定机制与赋值机制彼此分开。可以有一个值为数字 5 的变量 x ，也可以有一个值为引用（或指针）的变量 y ，后者指向当前内容为 5 的可变单元；程序员能看见二者的区别。我们可以把 x 加到另一个数字上，却不能给它赋值。可以直接用 y 把新值写入它所指的单元，例如 $y := 84$ ，却不能直接把 y 作为 *plus* 的参数；必须显式解引用，写作 $!y$ ，才能取得当前内容。而在多数其他语言，尤其是包括 Java 在内的整个 C 家族中，每个变量名都指向可变单元；为取得当前内容而对变量解引用的操作是隐式的。³

为形式研究，把这两种机制分开很有用；本章紧随 ML 的模型。把这里的经验用于类 C 语言很直接：合并一些区别，并让解引用等特定操作由显式变为隐式。⁴

¹即使 Haskell 一类“纯函数式”语言，也可通过单子（*monad*）等扩展做到。

²本章研究带 Unit 与引用的简单类型 lambda 演算（图13-1），相应的 OCaml 实现为 `fullref`。

³严格地说，C 或 Java 中多数类型为 T 的变量，实际应看作指向 $\text{Option}(T)$ 型值之单元的指针，因为变量内容既可以是正常值，也可以是特殊值 `null`。

⁴从语言设计角度看，这种分离也有充分理由：把可变单元设为显式选择而非默认为行为，可鼓励以函数式为主、少量使用引用的风格，使程序更易编写、维护和推理，存在并发时尤其如此。

基本操作

引用的基本操作是分配、解引用和赋值。要分配引用，使用 `ref` 运算符，并给出新单元的初值：

$$r = \text{ref } 5 \quad \blacktriangleright \quad r : \text{Ref Nat}$$

类型检查器的响应说明， r 的值是一个引用，它指向的单元始终存放数字。用解引用运算符 `!` 读取这个单元的当前值：

$$!r \quad \blacktriangleright \quad 5 : \text{Nat}$$

用赋值运算符改变单元内保存的值：

$$r := 7 \quad \blacktriangleright \quad \text{unit} : \text{Unit}$$

赋值结果是平凡值 `unit`，见 §11.2。再次解引用 r ，便看到更新后的值：

$$!r \quad \blacktriangleright \quad 7 : \text{Nat}$$

副作用与顺序执行

赋值表达式的结果是平凡值 `unit`，这正好与 §11.3 定义的顺序执行记法配合。为了依次求值两个表达式并返回第二个表达式的值，可以写：

$$(r := \text{succ } (!r); !r) \quad \blacktriangleright \quad 8 : \text{Nat},$$

而不必写等价却更繁琐的：

$$(\lambda_ : \text{Unit}. !r) (r := \text{succ } (!r)) \quad \blacktriangleright \quad 9 : \text{Nat}$$

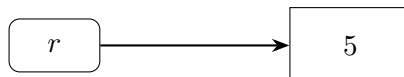
把第一个表达式的类型限制为 `Unit`，使类型检查器能捕获一些愚蠢错误：只有当第一个值保证无关紧要时，才允许丢弃它。

若第二个表达式也是赋值，整个序列的类型仍是 `Unit`，因此可以合法地把它放在另一个分号左边，构造更长的赋值序列：

$$(r := \text{succ } (!r); r := \text{succ } (!r); r := \text{succ } (!r); r := \text{succ } (!r); !r) \quad \blacktriangleright \quad 13 : \text{Nat}$$

引用与别名

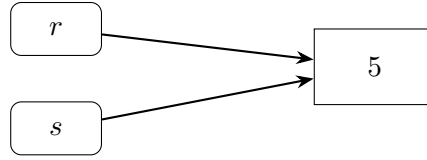
必须牢记：绑定到 r 的引用，与该引用所指向的存储单元是不同的。



例如把 r 的值绑定到另一变量 s ，从而复制 r ：

$$s = r \quad \blacktriangleright \quad s : \text{Ref Nat},$$

得到复制的只是引用，即图中的箭头，而不是单元：



给 s 写入新值，再通过 r 读出即可验证：

$$s := 82 \blacktriangleright \text{unit}, \quad !r \blacktriangleright 82 : \text{Nat}$$

称引用 r 与 s 是同一单元的 别名 (*alias*)。

练习 13.1.1 (★). 画出类似的图，说明求值

$$a = \{\text{ref } 0, \text{ref } 0\}$$

和

$$b = (\lambda x : \text{Ref Nat}. \{x, x\})(\text{ref } 0)$$

各自产生的效果。

[查看原书附录 A 中的 13.1.1 解答](#)

共享状态

存在别名会使含引用的程序很难推理。例如表达式 $(r := 1; r := !s)$ 先把 1 赋给 r ，紧接着又用 s 的当前值覆盖它。除非把它放在 r 与 s 是同一单元之别名的上下文中，否则它与单次赋值 $r := !s$ 效果完全相同。

当然，别名也是引用十分有用的一项重要原因。特别是，它让程序的不同部分之间可以建立“隐式通信通道”——共享状态 (*shared state*)。例如，定义一个引用单元和两个操纵其内容的函数：

$$c = \text{ref } 0 : \text{Ref Nat},$$

$$\text{incc} = \lambda x : \text{Unit}. (c := \text{succ}(!c); !c) : \text{Unit} \rightarrow \text{Nat},$$

$$\text{decc} = \lambda x : \text{Unit}. (c := \text{pred}(!c); !c) : \text{Unit} \rightarrow \text{Nat}$$

调用 incc unit 得到 1，它对 c 的修改可以由随后调用 decc unit 观察到，后者得到 0。

若把 incc 与 decc 打包进记录：

$$o = \{i = \text{incc}, d = \text{decc}\} : \{i : \text{Unit} \rightarrow \text{Nat}, d : \text{Unit} \rightarrow \text{Nat}\},$$

就可以把 o 作为一个值整体传递。它的两个函数字段 i 与 d 都捕获了同一个引用 c ：调用 $o.i \text{ unit}$ 会递增 c 中的值，调用 $o.d \text{ unit}$ 则会递减它。因此， o 已经具备了简单对象的基本结构：一份由多个操作共同访问的可变状态。[第十八章](#)会详细发展这一思路。

复合类型的引用

引用单元不必只存放数字：上述基本操作允许创建指向任意类型之值的引用，包括函数。例如，可以用函数引用低效地实现数字数组。令

$$\text{NatArray} = \text{Ref}(\text{Nat} \rightarrow \text{Nat})$$

新数组分配一个引用单元，填入一个接受任意索引都返回 0 的函数：

$$\text{newarray} = \lambda_ : \text{Unit}. \text{ref}(\lambda n : \text{Nat}. 0) : \text{Unit} \rightarrow \text{NatArray}$$

查找数组元素时，把保存的函数应用于所需索引：

$$\text{lookup} = \lambda a : \text{NatArray}. \lambda n : \text{Nat}. (!a) n : \text{NatArray} \rightarrow \text{Nat} \rightarrow \text{Nat}$$

编码中有意思的是更新函数。它接受数组、索引和要保存在该索引的新值；先创建一个新函数，再把它存入引用。新函数在被问及这个索引的值时，返回传给更新操作的新值；对所有其他索引，则把查找交给引用先前保存的函数。

$$\begin{aligned} \text{update} &= \lambda a : \text{NatArray}. \lambda m : \text{Nat}. \lambda v : \text{Nat}. \\ &\quad \text{let } \text{oldf} = !a \text{ in} \\ &\quad a := (\lambda n : \text{Nat}. \text{if } \text{equal } m \ n \text{ then } v \text{ else } \text{oldf } n) \\ &\quad : \text{NatArray} \rightarrow \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Unit} \end{aligned}$$

练习 13.1.2 (★). 若把更新更紧凑地定义成：

$$\begin{aligned} \text{update} &= \lambda a : \text{NatArray}. \lambda m : \text{Nat}. \lambda v : \text{Nat}. \\ &\quad a := (\lambda n : \text{Nat}. \text{if } \text{equal } m \ n \text{ then } v \text{ else } (!a) n), \end{aligned}$$

其行为相同吗？

[查看原书附录 A 中的 13.1.2 解答](#)

指向含有其他引用之值的引用也很有用，可以用来定义可变列表、可变树等数据结构。这类结构通常还涉及[第二十章](#)将引入的递归类型。

垃圾回收

正式处理引用之前，最后要提到存储释放。我们没有提供显式释放引用单元的基本操作。引用单元不再需要时，我们像许多现代语言（包括 ML 与 Java）一样，依靠运行时系统执行垃圾回收（*garbage collection*），收集并重用程序已无法到达的单元。这不只是语言设计的偏好问题：存在显式释放操作时，要实现类型安全极其困难。原因是熟悉的悬垂引用（*dangling reference*）问题：先分配一个保存数字的单元，把它的引用存入某数据结构并使用一段时间，然后释放它，再分配一个保存布尔值的新单元；新单元可能重用同一块存储。现在，同一存储单元可能有两个名字，一个类型是 `Ref Nat`，另一个类型是 `Ref Bool`。

练习 13.1.3 (★). 说明上述情形怎样导致类型安全性遭到破坏。

[查看原书附录 A 中的 13.1.3 解答](#)

13.2 类型

`ref`、`:=` 与 `!` 的类型规则直接来自我们为它们规定的行为：

$$\frac{\Gamma \vdash t_1 : T_1}{\Gamma \vdash \text{ref } t_1 : \text{Ref } T_1} \text{ T-REF}$$

$$\frac{\Gamma \vdash t_1 : \text{Ref } T_{11}}{\Gamma \vdash !t_1 : T_{11}} \text{ T-DEREF} \quad \frac{\Gamma \vdash t_1 : \text{Ref } T_{11} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 := t_2 : \text{Unit}} \text{ T-ASSIGN}$$

13.3 求值

形式化引用的操作行为时会出现更微妙的问题。只要问一句“类型 `Ref T` 的值应是什么”，就能看出原因。必须考虑的关键观察是：求值 `ref` 运算符应当真正做一件事——分配一些存储——并产生指向这块存储的引用。

那么，引用究竟是什么？

多数程序语言实现中的运行时存储，本质上就是巨大的字节数组。运行时系统追踪数组中哪些部分正在使用；需要分配新引用单元时，从空闲区域分出足够大的片段（整数单元可能为 4 字节，浮点单元可能为 8 字节，等等），标记为在用，再返回新分配区域开头的索引，通常是 32 位或 64 位整数。这些索引就是引用。

目前没有必要具体到这种程度。可以把存储看作值的数组，而不是字节的数组，从而抽象掉不同值的运行时表示大小不同这一事实。还可以进一步抽象掉引用（即数组索引）是数字这一事实。把引用取作某个未解释的存储位置（*store location*）集合 $\mathcal{L}$ 的元素；存储则只是从位置 l 到值的部分函数。用元变量 μ 表示存储。于是，引用就是一个位置，即存储的抽象索引。从现在开始使用“位置”而不用“引用”或“指针”，以强调这种抽象性质。⁵

接着，要扩展操作语义以纳入存储。表达式的求值结果通常取决于求值开始时的存储内容，所以求值规则的参数不能只有项，还必须包含存储。项的求值又可能对存储产生副作用，影响以后其他项的求值，因此规则还要返回新存储。于是，单步求值关系由 $t \longrightarrow t'$ 变成：

$$t \mid \mu \longrightarrow t' \mid \mu',$$

其中 μ, μ' 分别是存储的初始与最终状态。实际上，我们丰富了抽象机的概念：机器状态不再只有程序计数器（用项表示），而是程序计数器加上存储的当前内容。

为贯彻这一变化，先给所有已有求值规则加入存储。例如：

$$\frac{}{(\lambda x : T_{11}. t_{12}) v_2 \mid \mu \longrightarrow [x \mapsto v_2] t_{12} \mid \mu} \text{ E-APPABS}$$

$$\frac{t_1 \mid \mu \longrightarrow t'_1 \mid \mu'}{t_1 t_2 \mid \mu \longrightarrow t'_1 t_2 \mid \mu'} \text{ E-APP1} \quad \frac{t_2 \mid \mu \longrightarrow t'_2 \mid \mu'}{v_1 t_2 \mid \mu \longrightarrow v_1 t'_2 \mid \mu'} \text{ E-APP2}$$

⁵这样抽象地处理位置，使我们无法模拟 C 等底层语言中的指针算术；这是有意的限制。指针算术偶尔很有用，尤其适合实现垃圾回收器等运行时系统底层组件，却无法由多数类型系统追踪：知道存储位置 n 含有 `Float`，并不能让我们推断位置 $n+4$ 的类型。在 C 中，指针算术是造成类型安全破坏的著名根源。

第一条规则原样返回存储 μ ，因为函数应用本身没有副作用；另外两条只是把前提中的副作用传播到结论。

接着稍微扩展项语法。求值 `ref` 表达式会得到一个新分配的存储位置，所以必须把位置纳入求值可能得到的结果，即值：

$v ::=$	$\lambda x : T. t$	值：抽象值
	unit	unit 值
	l	存储位置

所有值也都是项，所以项集合也必须包括位置：

$t ::=$	x	项：变量
	$\lambda x : T. t$	抽象
	$t t$	应用
	unit	常量 unit
	$\text{ref } t$	创建引用
	$!t$	解引用
	$t := t$	赋值
	l	存储位置

当然，把具体位置加入项语法，不表示允许程序员直接书写含显式具体位置的项；这类项只会作为求值中间结果出现。实际上，本章的项语言应当看成一种中间语言，其中有些特性不直接开放给程序员。

在扩展后的语法上，可以陈述操纵位置与存储的新构造的求值规则。求值解引用表达式 $!t_1$ 时，先归约 t_1 直到它成为值：

$$\frac{t_1 \mid \mu \longrightarrow t'_1 \mid \mu'}{!t_1 \mid \mu \longrightarrow !t'_1 \mid \mu'} \text{ E-DEREF}$$

完成后应得到 $!l$ ，其中 l 是某个位置。试图解引用函数或 `unit` 等其他值的项是错误的，求值规则会卡住；§13.5 的类型安全性质保证良类型项不会这样出错。

$$\frac{\mu(l) = v}{!l \mid \mu \longrightarrow v \mid \mu} \text{ E-DEREFLOC}$$

求值赋值表达式 $t_1 := t_2$ 时，先求值 t_1 直到成为位置，再求值 t_2 直到成为任意种类的值：

$$\frac{t_1 \mid \mu \longrightarrow t'_1 \mid \mu'}{t_1 := t_2 \mid \mu \longrightarrow t'_1 := t_2 \mid \mu'} \text{ E-ASSIGN1}$$

$$\frac{t_2 \mid \mu \longrightarrow t'_2 \mid \mu'}{l := t_2 \mid \mu \longrightarrow l := t'_2 \mid \mu'} \text{ E-ASSIGN2}$$

两边都求值完后，表达式形如 $l := v_2$ ，执行它就是更新存储，使 l 包含 v_2 ：

$$\frac{l \in \text{dom}(\mu)}{l := v_2 \mid \mu \longrightarrow \text{unit} \mid [l \mapsto v_2]\mu} \text{E-ASSIGN}$$

记法 $[l \mapsto v_2]\mu$ 表示“把 l 映射到 v_2 ，其他位置的映射与 μ 相同的存储”。这一步所得项只是 unit ；有意思的结果是更新后的存储。

最后，求值 $\text{ref } t_1$ 时，先把 t_1 求值为值：

$$\frac{t_1 \mid \mu \longrightarrow t'_1 \mid \mu'}{\text{ref } t_1 \mid \mu \longrightarrow \text{ref } t'_1 \mid \mu'} \text{E-REF}$$

然后选择一个尚未分配的位置 l ，即 $l \notin \text{dom}(\mu)$ ，产生以新绑定 $l \mapsto v_1$ 扩展 μ 的存储：

$$\frac{l \notin \text{dom}(\mu)}{\text{ref } v_1 \mid \mu \longrightarrow l \mid (\mu, l \mapsto v_1)} \text{E-REFV}$$

这一步所得项就是新分配位置的名字 l 。

这些规则没有执行任何垃圾回收：求值过程中，存储可以无界增长。这不影响求值结果的正确性——“垃圾”按定义正是存储中已无法到达、不会再对求值发挥作用的部分——但意味着朴素求值器有时会耗尽内存，而更复杂的求值器本可重用内容已经成为垃圾的位置并继续运行。

练习 13.3.1 (★★). 可以怎样细化求值规则，以模拟垃圾回收？还需要证明什么定理，才能论证这种细化是正确的？

[查看原书附录 A 中的 13.3.1 解答](#)

13.4 存储类型

语法和求值规则已经扩展为容纳引用，最后要写下新构造的类型规则，并检查这些规则是否可靠。关键问题自然是：“位置具有什么类型？”

求值含具体位置的项时，结果类型取决于初始存储的内容。例如，在存储 $(l_1 \mapsto \text{unit}, l_2 \mapsto \text{unit})$ 中求值 l_2 ，结果是 unit ；在存储 $(l_1 \mapsto \text{unit}, l_2 \mapsto \lambda x : \text{Unit}. x)$ 中求值同一个项，结果是 $\lambda x : \text{Unit}. x$ 。相对于前一存储， l_2 的内容类型为 Unit ；相对于后一存储，它的内容类型为 $\text{Unit} \rightarrow \text{Unit}$ 。由此立即得到第一版位置类型规则：

$$\frac{\mu(l) = v_1 \quad \Gamma \mid \mu \vdash v_1 : T_1}{\Gamma \mid \mu \vdash l : \text{Ref } T_1}$$

也就是说，要找位置 l 的类型，就在存储中查找 l 的当前内容，再计算内容的类型 T_1 ；位置的类型就是 $\text{Ref } T_1$ 。

这样开始之后，还要再前进一步才能得到一致的系统。让项的类型依赖存储，实际已经把类型关系从上下文、项、类型之间的三元关系，改成了上下文、存储、项、类型之间的四元关系。

直观地说, 存储是计算项类型时所用上下文的一部分, 因此把这个四元关系写成 $\Gamma \mid \mu \vdash t : T$, 将存储写在判断符号 $\vdash$ 的左侧。所有其他类型规则也类似地加入存储; 它们不需要对存储作任何特殊处理, 只需将其从前提原样传递到结论。

但上述规则有两个问题。第一, 类型检查效率很低, 因为计算位置 l 的类型需要计算其当前内容 v 的类型。若 l 在项 t 中出现多次, 构造 t 的类型推导时就会重复计算 v 的类型。若 v 自己还含位置, 每次出现又要重算它们的类型。例如, 存储若含有:

$$\begin{aligned} (l_1 \mapsto \lambda x : \text{Nat}. 999, \\ l_2 \mapsto \lambda x : \text{Nat}. (!l_1) x, \\ l_3 \mapsto \lambda x : \text{Nat}. (!l_2) x, \\ l_4 \mapsto \lambda x : \text{Nat}. (!l_3) x, \\ l_5 \mapsto \lambda x : \text{Nat}. (!l_4) x), \end{aligned}$$

计算 l_5 的类型就涉及依次计算 l_4, l_3, l_2, l_1 的类型。

第二, 若存储含有环, 按照上述位置类型规则, 我们可能根本无法为某个位置构造出有限的类型推导。例如, 考虑下面这个存储:

$$(l_1 \mapsto \lambda x : \text{Nat}. (!l_2) x, \quad l_2 \mapsto \lambda x : \text{Nat}. (!l_1) x),$$

此时无法为 l_2 构造有限的类型推导, 因为计算 l_2 的类型需要先找 l_1 的类型, 而后者又涉及 l_2 , 如此往复。环形引用结构在实践中确实会出现, 例如可以构造双向链表; 我们希望类型系统能处理它们。

练习 13.4.1 (★). 能否找出一个项, 其求值会生成上面这个特定的环形存储?

[查看原书附录 A 中的 13.4.1 解答](#)

两个问题都源于上述位置类型规则要求每次在项中提及位置时重新计算其类型。但直观地说, 没有这个必要。位置刚创建时, 我们知道存入其中的初值类型; 以后虽然可能向该位置存入其他值, 它们却总会与初值具有相同类型。换言之, 存储中的每个位置都对应一个确定的类型, 而且该类型在分配位置时便已固定。可以把这些预期类型收集成存储类型 (*store typing*), 即从位置到类型的有限函数。用元变量 Σ 表示这类函数。

假设项 t 将在具体存储 μ 中求值, 而我们已经有了描述该存储的存储类型 Σ 。那么, 只需查阅 Σ , 就能判断 t 的求值结果具有何种类型, 无须直接查看 μ 中实际存放的值。例如, 若 $\Sigma = (l_1 \mapsto \text{Unit}, l_2 \mapsto \text{Unit} \rightarrow \text{Unit})$, 便能立即推断 $!l_2 : \text{Unit} \rightarrow \text{Unit}$ 。一般地, 位置的类型规则改写为:

$$\frac{\Sigma(l) = T_1}{\Gamma \mid \Sigma \vdash l : \text{Ref } T_1} \text{ T-LOC}$$

类型仍是四元关系, 但现在以存储类型而不是具体存储为参数。其余类型规则也类似地加入存储类型。

当然, 只有求值实际使用的具体存储确实符合类型检查时假定的存储类型, 这些规则才能准确预测求值结果。这里遵循的原则与自由变量的类型检查相同: 类型检查所依据的假设,

必须由求值时使用的实际值满足。判断 $\Gamma \vdash t : T$ 是在 Γ 给出的类型假设下成立的； Γ 规定了 t 中每个自由变量可以由何种类型的值替换。[替换引理9.3.8](#)保证，按照这些规定完成替换后，得到的闭项仍具有类型 T ；再由 [类型保持性定理9.3.9](#)，若该闭项产生求值结果，结果也仍具有类型 T 。存储的情形与此相同： Σ 规定每个位置应当存放什么类型的值，而实际存储 μ 必须符合这些规定。[§13.5](#)将正式定义这种对应关系，并证明相应性质。

最后，检查程序员实际书写的项时，不需要猜测该用什么存储类型。具体位置常量只会出现于求值的中间结果中，源程序本身不含这些位置，因此开始时只需使用空存储类型。求值过程中，执行 `ref v` 会创建新位置 l ；若初值 v 的类型为 T ，就在存储类型中加入 $l \mapsto T$ 。下面的 [保持性定理13.5.3](#)将证明，每一步求值之后总能以这种方式扩展存储类型，使求值后的项保持原来的类型，新的具体存储也符合扩展后的存储类型。

处理完位置后，其他新句法形式的类型规则都很直接。对 T_1 型值创建的引用具有类型 $\text{Ref } T_1$ ：

$$\frac{\Gamma \mid \Sigma \vdash t_1 : T_1}{\Gamma \mid \Sigma \vdash \text{ref } t_1 : \text{Ref } T_1} \text{ T-REF}$$

这里不需要扩展存储类型，因为新位置的名字到运行时才会确定，而 Σ 只记录已经分配的存储单元与其类型的联系。

反过来，若 t_1 求值为 $\text{Ref } T_{11}$ 型的位置，对它解引用保证产生 T_{11} 型的值：

$$\frac{\Gamma \mid \Sigma \vdash t_1 : \text{Ref } T_{11}}{\Gamma \mid \Sigma \vdash !t_1 : T_{11}} \text{ T-DEREF}$$

最后，若 t_1 表示 T_{11} 型单元，只要 t_2 的类型也是 T_{11} ，就能把它存入该单元：

$$\frac{\Gamma \mid \Sigma \vdash t_1 : \text{Ref } T_{11} \quad \Gamma \mid \Sigma \vdash t_2 : T_{11}}{\Gamma \mid \Sigma \vdash t_1 := t_2 : \text{Unit}} \text{ T-ASSIGN}$$

Ref

扩展 $\lambda \rightarrow$ (图 9-1), 并带 Unit (图 11-2)

$t ::= x$	项: 变量
$ \lambda x : T. t$	抽象
$ t t$	应用
$ \text{unit}$	常量 unit
$ \text{ref } t$	创建引用
$!t$	解引用
$ t := t$	赋值
$ l$	存储位置
$v ::= \lambda x : T. t$	值: 抽象值
$ \text{unit}$	unit 值
$ l$	存储位置
$T ::= T \rightarrow T$	类型: 函数类型
$ \text{Unit}$	unit 类型
$ \text{Ref } T$	引用类型
$\Gamma ::= \emptyset \mid \Gamma, x : T$	类型上下文
$\mu ::= \emptyset \mid \mu, l \mapsto v$	存储
$\Sigma ::= \emptyset \mid \Sigma, l : T$	存储类型

 $t \mid \mu \longrightarrow t' \mid \mu'$ 判断形式: 项与存储单步求值为新项与新存储

$\frac{}{(\lambda x : T_{11}. t_{12}) v_2 \mid \mu \longrightarrow [x \mapsto v_2] t_{12} \mid \mu} \text{E-APPABS}$	$\frac{t_1 \mid \mu \longrightarrow t'_1 \mid \mu'}{t_1 t_2 \mid \mu \longrightarrow t'_1 t_2 \mid \mu'} \text{E-APP1}$
$\frac{t_2 \mid \mu \longrightarrow t'_2 \mid \mu'}{v_1 t_2 \mid \mu \longrightarrow v_1 t'_2 \mid \mu'} \text{E-APP2}$	$\frac{t_1 \mid \mu \longrightarrow t'_1 \mid \mu'}{\text{ref } t_1 \mid \mu \longrightarrow \text{ref } t'_1 \mid \mu'} \text{E-REF}$
$\frac{l \notin \text{dom}(\mu)}{\text{ref } v_1 \mid \mu \longrightarrow l \mid (\mu, l \mapsto v_1)} \text{E-REFV}$	$\frac{t_1 \mid \mu \longrightarrow t'_1 \mid \mu'}{!t_1 \mid \mu \longrightarrow !t'_1 \mid \mu'} \text{E-DEREF}$
$\frac{\mu(l) = v}{!l \mid \mu \longrightarrow v \mid \mu} \text{E-DEREFLOC}$	$\frac{t_1 \mid \mu \longrightarrow t'_1 \mid \mu'}{t_1 := t_2 \mid \mu \longrightarrow t'_1 := t_2 \mid \mu'} \text{E-ASSIGN1}$
$\frac{t_2 \mid \mu \longrightarrow t'_2 \mid \mu'}{l := t_2 \mid \mu \longrightarrow l := t'_2 \mid \mu'} \text{E-ASSIGN2}$	$\frac{l \in \text{dom}(\mu)}{l := v_2 \mid \mu \longrightarrow \text{unit} \mid [l \mapsto v_2] \mu} \text{E-ASSIGN}$

 $\Gamma \mid \Sigma \vdash t : T$ 判断形式: 在上下文与存储类型下, 项具有类型

$\frac{x : T \in \Gamma}{\Gamma \mid \Sigma \vdash x : T} \text{T-VAR}$	$\frac{\Gamma, x : T_1 \mid \Sigma \vdash t_2 : T_2}{\Gamma \mid \Sigma \vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2} \text{T-ABS}$
$\frac{\Gamma \mid \Sigma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \mid \Sigma \vdash t_2 : T_{11}}{\Gamma \mid \Sigma \vdash t_1 t_2 : T_{12}} \text{T-APP}$	$\frac{}{\Gamma \mid \Sigma \vdash \text{unit} : \text{Unit}} \text{T-UNIT}$
$\frac{\Sigma(l) = T_1}{\Gamma \mid \Sigma \vdash l : \text{Ref } T_1} \text{T-LOC}$	$\frac{\Gamma \mid \Sigma \vdash t_1 : T_1}{\Gamma \mid \Sigma \vdash \text{ref } t_1 : \text{Ref } T_1} \text{T-REF}$
$\frac{\Gamma \mid \Sigma \vdash t_1 : \text{Ref } T_{11}}{\Gamma \mid \Sigma \vdash !t_1 : T_{11}} \text{T-DEREF}$	$\frac{\Gamma \mid \Sigma \vdash t_1 : \text{Ref } T_{11} \quad \Gamma \mid \Sigma \vdash t_2 : T_{11}}{\Gamma \mid \Sigma \vdash t_1 := t_2 : \text{Unit}} \text{T-ASSIGN}$

浅灰区域 表示相较于 $\lambda \rightarrow$ 与 Unit 新增或修改的部分。

图 13-1: 引用

图13-1 汇总带引用的简单类型 lambda 演算的类型规则，也列出语法与求值规则，便于查阅。

13.5 安全性

最后要检查，引用演算仍然满足标准类型安全性质。进展性定理——良类型项不会卡住——几乎可以照旧陈述和证明，见定理13.5.7；只需在证明中为新构造增加几个直接的情形。保持性定理则需要处理更多细节，所以先考察它。

引入引用后，单步求值关系不仅记录项从 t 变为 t' ，还记录存储从求值前的 μ 变为求值后的 μ' ；类型关系也增加了存储类型 Σ 这一参数。因此，保持性的陈述必须包含这些参数。不过，仅将它们加入定理还不够，还必须说明具体存储与存储类型之间具有怎样的关系：

$$\text{若 } \Gamma \mid \Sigma \vdash t : T \text{ 且 } t \mid \mu \longrightarrow t' \mid \mu', \text{ 则 } \Gamma \mid \Sigma \vdash t' : T. \quad (\text{错误!})$$

若类型检查依据的是一组关于存储值类型的假设，求值却使用违反这些假设的存储，结果必然是灾难。下面的要求表达所需约束。

定义 13.5.1. 若 $\text{dom}(\mu) = \text{dom}(\Sigma)$ ，并且对每个 $l \in \text{dom}(\mu)$ 都有 $\Gamma \mid \Sigma \vdash \mu(l) : \Sigma(l)$ ，则称存储 μ 相对于类型上下文 Γ 与存储类型 Σ 是良类型的，记作 $\Gamma \mid \Sigma \vdash \mu$ 。

直觉上，若存储中的每个值都具有存储类型所预测的类型，存储 μ 就与存储类型 Σ 一致。

译者注：作者勘误指出，由于本章实际只考察不含自由变量的完整程序，这里的类型上下文也可以固定为空，而不必允许任意 Γ 。原书采用的较一般陈述仍是正确的，本译文保留它，以便与随后各引理和定理的统一记法对应。

练习 13.5.2 (★★). 能否找出上下文 Γ 、存储 μ 以及两个不同的存储类型 Σ_1, Σ_2 ，使 $\Gamma \mid \Sigma_1 \vdash \mu$ 与 $\Gamma \mid \Sigma_2 \vdash \mu$ 都成立？

查看原书附录 A 中的 13.5.2 解答

现在可以陈述一项更接近目标的保持性质：

$$\left. \begin{array}{l} \Gamma \mid \Sigma \vdash t : T \\ t \mid \mu \longrightarrow t' \mid \mu' \\ \Gamma \mid \Sigma \vdash \mu \end{array} \right\} \implies \Gamma \mid \Sigma \vdash t' : T. \quad (\text{仍不够正确。})$$

上述陈述对除分配规则 E-REFV 外的其他求值规则都没有问题，因为这些规则不会改变存储的定义域。E-REFV 则会创建新位置 l ，使求值后的存储 μ' 比求值前的存储 μ 多出绑定 $l \mapsto v_1$ 。原来的存储类型 Σ 只记录了 μ 中已有的位置，因此 $l \notin \text{dom}(\Sigma)$ ；若结果项 t' 明确提到 l ，就无法在原来的 Σ 下为它赋型。

既然存储在求值过程中可能增长，保持性定理也必须允许存储类型相应增长。于是得到保持性的最终正确陈述。

定理 13.5.3 (保持性). 若

$$\Gamma \mid \Sigma \vdash t : T, \quad \Gamma \mid \Sigma \vdash \mu, \quad t \mid \mu \longrightarrow t' \mid \mu',$$

则存在某个 $\Sigma' \supseteq \Sigma$, 使

$$\Gamma \mid \Sigma' \vdash t' : T \quad \text{且} \quad \Gamma \mid \Sigma' \vdash \mu'$$

这里的 $\Sigma' \supseteq \Sigma$ 表示 Σ' 对所有旧位置的类型都与 Σ 一致。定理只断言存在某个合适的 Σ' , 没有进一步指定它。对一次求值而言, 实际只有两种情况: 若没有分配新位置, 就取 $\Sigma' = \Sigma$; 若 E-REFV 创建新位置 l , 并将类型为 T_1 的初值 v_1 存入其中, 就取 $\Sigma' = \Sigma, l : T_1$ 。把这两种情况直接写进定理只会使陈述更复杂, 不会给后续证明带来额外帮助。

上述存在性陈述已经足以处理多步求值。对一步求值 $t \mid \mu \longrightarrow t' \mid \mu'$ 应用定理后, 得到的结论说明 t' 与 μ' 在某个 Σ' 下仍是良类型的; 这正好又满足下一步应用定理所需的前提。因此可以反复应用保持性定理, 推出任意多步求值都保持良类型性。再与进展性质结合, 就得到通常的类型安全保证: 良类型程序不会在求值过程中因类型错误而卡住。

证明保持性需要几个辅助引理。第一个是 [标准替换引理9.3.8](#)的简单扩展。

引理 13.5.4 (替换). 若 $\Gamma, x : S \mid \Sigma \vdash t : T$, 且 $\Gamma \mid \Sigma \vdash s : S$, 则 $\Gamma \mid \Sigma \vdash [x \mapsto s]t : T$ 。

证明. 与[引理9.3.8](#)相同。 □

下一个引理说明, 以类型适当的新值替换存储中一个单元的内容, 不会改变存储整体的类型。

引理 13.5.5. 若

$$\Gamma \mid \Sigma \vdash \mu, \quad \Sigma(l) = T, \quad \Gamma \mid \Sigma \vdash v : T,$$

则 $\Gamma \mid \Sigma \vdash [l \mapsto v]\mu$ 。

证明. 由 $\Gamma \mid \Sigma \vdash \mu$ 的定义立即得到。 □

最后还需要存储的一种弱化引理: 若以新位置扩展存储, 扩展后的存储仍允许为原存储能赋型的所有项指派类型。

引理 13.5.6. 若 $\Gamma \mid \Sigma \vdash t : T$, 且 $\Sigma' \supseteq \Sigma$, 则 $\Gamma \mid \Sigma' \vdash t : T$ 。

证明. 作简单归纳。 □

[定理13.5.3](#) 的证明. 对求值推导作直接归纳, 使用上述引理以及类型规则的反演性质 ([引理9.3.1](#) 的直接扩展)。 □

[进展性定理9.3.5](#)的陈述也必须扩展, 以纳入存储与存储类型。

定理 13.5.7 (进展性). 设 t 是闭的良类型项, 即对某个 T, Σ 有 $\emptyset \mid \Sigma \vdash t : T$ 。那么, t 要么是值, 要么对每个满足 $\emptyset \mid \Sigma \vdash \mu$ 的存储 μ , 都存在项 t' 和存储 μ' , 使

$$t \mid \mu \longrightarrow t' \mid \mu'$$

证明. 沿定理9.3.5 的模式, 对类型推导作直接归纳。典范形式引理9.3.4需要再增加两种情形: 类型 $\text{Ref } T$ 的所有值都是位置, Unit 的所有值都是 unit 。□

练习 13.5.8 (推荐, ★★★). 本章的求值关系在良类型项上可规范化吗? 若是, 请证明; 若不是, 请在当前演算加入自然数和布尔值后, 写出一个良类型的阶乘函数。

[查看原书附录 A 中的 13.5.8 解答](#)

13.6 注记

本章呈现改编自 Harper (1994、1996) 的处理; Wright 与 Felleisen (1994) 也给出了风格相似的论述。

引用 (或其他计算效应) 与 ML 风格多态类型推断结合时, 会产生颇为微妙的问题 (见 §22.7), 在研究文献中受到广泛关注。参见 Tofte (1990)、Hoang 等 (1993)、Jouvelot 与 Gifford (1991)、Talpin 与 Jouvelot (1992)、Leroy 与 Weis (1991)、Wright (1992)、Harper (1994、1996) 及其中所引资料。

静态预测可能出现的别名, 是编译器实现与程序语言理论中由来已久的问题; 在编译器中称为别名分析。Reynolds (1978、1989) 的一项早期研究提出, 可以通过句法和类型规则限制程序片段经由共享状态相互影响, 并将这种方法称为 用句法控制相互干扰 (*syntactic control of interference*)。这些思想近年来再次活跃, 参见 O'Hearn 等 (1995) 与 Smith 等 (2000)。更一般的别名推理技术见 Reynolds (1981)、Ishtiaq 与 O'Hearn (2001) 及其中引用的其他资料。

Jones 与 Lins (1996) 全面讨论了垃圾回收; Morrisett 等 (1995) 给出更偏语义的处理。

求出这一个结果的原因,
或者不如说, 这一种病态的原因,
因为这个病态的结果不是无因而至的。

*Find out the cause of this effect,
Or rather say, the cause of this defect,
For this effect defective comes by cause.*

《哈姆雷特》第二幕第二场, 第 101 行起 (朱生豪译)

指向月亮的手指并不是月亮。
The finger pointing at the moon is not the moon.

佛教谚语

第十四章 异常

概述

第十三章说明了怎样用可变引用扩展纯简单类型 lambda 演算的简单操作语义，并考察这项扩展对类型规则和类型安全性证明的影响。本章处理对原计算模型的另一项扩展：抛出和处理异常。

现实编程中，函数经常需要通知调用者：由于某种原因，它无法完成任务。可能是某项计算涉及除零或算术溢出，字典中缺少查找键，数组索引越界，文件找不到或无法打开，系统内存耗尽、用户终止进程一类灾难性事件发生，等等。

§11.10 已经看到，有些异常条件可以通过让函数返回变体（或 option）来表示。但若这些条件确实属于异常情形，我们也许不愿强迫函数的每个调用者都处理它们发生的可能。更合意的做法可能是：异常条件把控制直接转移到程序更高层定义的异常处理器；甚至在异常足够罕见、或调用者无论如何也无法恢复时，直接终止程序。本章先考察后一情形 (§14.1)，把异常视为整个程序的终止；再加入捕获异常并从中恢复的机制 (§14.2)；最后细化两种机制，让程序员指定的额外数据可以从异常抛出点传给处理器 (§14.3)。¹

error

扩展 $\lambda \rightarrow$ (图 9-1)

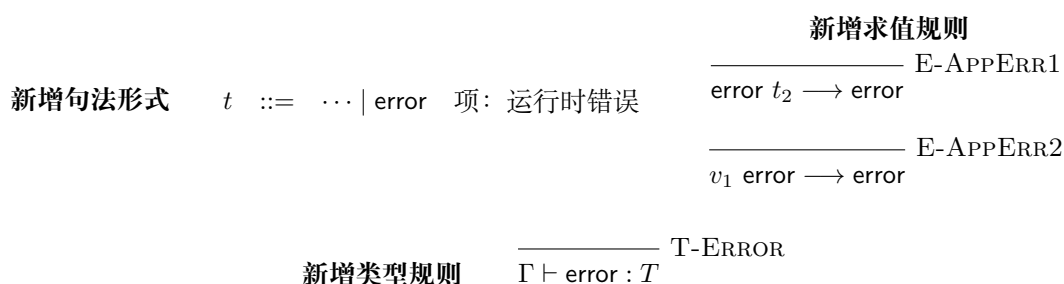


图 14-1: 错误

¹本章研究在简单类型 lambda 演算 (图9-1) 上加入多种异常抛出与处理基本构造的系统 (图14-1 与图14-2)。第一种扩展的 OCaml 实现是 `fullerror`；携带值的异常语言 (图14-3) 没有官方实现。

error, try

扩展 $\lambda \rightarrow$ 并带错误 (图14-1)

新增句法形式

 $t ::= \dots \mid \text{try } t \text{ with } t$ 捕获错误

新增求值规则

$$\begin{array}{c}
\frac{}{\text{try } v_1 \text{ with } t_2 \rightarrow v_1} \text{E-TRYV} \qquad \frac{}{\text{try error with } t_2 \rightarrow t_2} \text{E-TRYERROR} \\
\\
\frac{t_1 \rightarrow t'_1}{\text{try } t_1 \text{ with } t_2 \rightarrow \text{try } t'_1 \text{ with } t_2} \text{E-TRY}
\end{array}$$

新增类型规则

$$\frac{\Gamma \vdash t_1 : T \quad \Gamma \vdash t_2 : T}{\Gamma \vdash \text{try } t_1 \text{ with } t_2 : T} \text{T-TRY}$$

图 14-2: 错误处理

译者注: 图14-1 中的 `error` 表示一种不携带额外信息的运行时错误, 也是异常机制的最简模型。图14-2 概括了 §14.2 为它加入的处理机制; 图14-3 概括了 §14.3 用 `raise t` 对它所作的推广, 使异常可以携带数据。

exceptions

扩展 $\lambda \rightarrow$ (图9-1)

新增句法形式

 $t ::= \dots \mid \text{raise } t \mid \text{try } t \text{ with } t$

raise t : 抛出异常
try t with t : 处理异常

新增求值规则

$$\begin{array}{c}
\frac{}{(\text{raise } v_{11}) t_2 \rightarrow \text{raise } v_{11}} \text{E-APPRAISE1} \qquad \frac{}{v_1 (\text{raise } v_{21}) \rightarrow \text{raise } v_{21}} \text{E-APPRAISE2} \\
\\
\frac{t_1 \rightarrow t'_1}{\text{raise } t_1 \rightarrow \text{raise } t'_1} \text{E-RAISE} \qquad \frac{}{\text{raise } (\text{raise } v_{11}) \rightarrow \text{raise } v_{11}} \text{E-RAISERAISE} \\
\\
\frac{}{\text{try } v_1 \text{ with } t_2 \rightarrow v_1} \text{E-TRYV} \qquad \frac{}{\text{try } (\text{raise } v_{11}) \text{ with } t_2 \rightarrow t_2 v_{11}} \text{E-TRYRAISE} \\
\\
\frac{t_1 \rightarrow t'_1}{\text{try } t_1 \text{ with } t_2 \rightarrow \text{try } t'_1 \text{ with } t_2} \text{E-TRY}
\end{array}$$

新增类型规则

$$\frac{\Gamma \vdash t_1 : T_{\text{exn}}}{\Gamma \vdash \text{raise } t_1 : T} \text{T-EXN} \qquad \frac{\Gamma \vdash t_1 : T \quad \Gamma \vdash t_2 : T_{\text{exn}} \rightarrow T}{\Gamma \vdash \text{try } t_1 \text{ with } t_2 : T} \text{T-TRY}$$

图 14-3: 携带值的异常

14.1 抛出异常

先为简单类型 lambda 演算加入表示异常的最简单机制: 项 `error` 一经求值, 就完全终止包含它的项的求值。所需扩展见 图14-1。

为 `error` 编写规则时，主要设计决定是怎样在操作语义中形式化“异常终止”。这里采用一个简单办法：让 `error` 本身作为程序终止的结果。规则 E-APPERR1 与 E-APPERR2 刻画这种行为。E-APPERR1 说明，在试图把应用左端归约为值的过程中遇到 `error` 时，立即让整个应用以 `error` 为结果。类似地，E-APPERR2 说明，在把应用参数归约为值的过程中遇到错误时，放弃这个应用并立即产生 `error`。

注意，`error` 只加入项语法，没有加入值语法。这保证 E-APPABS 与 E-APPERR2 的左端永不重叠。也就是说，对项

$$(\lambda x : \text{Nat}. 0) \text{ error}$$

不会产生“究竟执行应用得到 0，还是终止”的歧义；只有终止这一种可能。类似地，规则 E-APPERR2 的左端写作 $v_1 \text{ error}$ ，而不是 $t_1 \text{ error}$ 。这表示只有当应用左端已经求值为值时，右端的 `error` 才会向外传播。因此，在

$$(\text{fix } (\lambda x : \text{Nat}. x)) \text{ error}$$

中，求值器必须先处理应用左端。左端会不断回到自身，始终不能成为值，所以整个项将一直求值下去，不会传播右端的 `error`。

这项限制与前面“不把 `error` 视为值”的规定共同消除了求值规则之间的重叠：对于任意项，每一步至多只有一条规则能够决定其下一步结果。因此，加入 `error` 后，求值关系仍然是确定的。

类型规则 T-ERROR 也很有意思。我们可能要在任何上下文抛出异常，所以允许项 `error` 具有任意类型。在

$$(\lambda x : \text{Bool}. x) \text{ error}$$

中，它具有类型 `Bool`；在

$$(\lambda x : \text{Bool}. x) (\text{error true})$$

中，它具有类型 `Bool → Bool`。

译者注：本章只在类型中记录计算正常完成时的结果。一个正常结果为 T 的计算，即使可能发生 `error`，类型仍然写作 T ，不会改成 Rust 中的 `Result<T, E>`、`Option<T>` 或和类型 $T + \text{Error}$ 。在这种设计下，错误向外传播并取代一个类型为 T 的表达式后，传播得到的 `error` 也必须能够具有类型 T ，这样良类型项经过求值后才能继续保持良类型。规则 T-ERROR 因而允许 `error` 具有任意类型。发生错误时，计算会异常终止，不会产生正常结果；这一点与 Rust 中表示永不返回的 `!` 类型相近。

`error` 的类型如此灵活，会给类型检查算法的实现带来困难，因为语言中每个可赋型项具有唯一类型这一性质（定理9.3.3）不再成立。可以用多种办法处理。在带子类型化的语言中，可以为 `error` 指派最小类型 `Bot`（见 §15.4），再按需提升为其他类型。在带参数多态的语言中（见第二十三章），可以为它赋予多态类型 $\forall X. X$ ，该类型可以实例化为任何其他类型。两种技巧都能用一个类型紧凑表示 `error` 无穷多个可能类型。

练习 14.1.1 (★). 要求程序员在每处使用 `error` 时标注预期类型，不是更简单吗？

[查看原书附录 A 中的 14.1.1 解答](#)

带异常语言的类型保持性质仍和以往一样：若项具有类型 T ，则它经过一次单步求值后仍具有类型 T 。进展性质则要稍加细化。它原本说明良类型程序必须求值为值（或发散）；现在新增了非值范式 `error`，它完全可能成为良类型程序的求值结果，所以必须在进展性陈述中允许它。

定理 14.1.2 (进展性). 设 t 是闭的良类型范式。那么， t 要么是值，要么 $t = \text{error}$ 。

14.2 处理异常

`error` 的求值规则可以理解为 栈展开 (*stack unwinding*)：不断丢弃尚未完成的函数调用，直到错误一路传播到最外层。真实异常语言的实现正是这样工作：调用栈由一组活动记录 (*activation record*) 组成，每个活动函数调用对应一条；抛出异常会从调用栈中依次弹出活动记录，直到栈空。

多数异常语言还允许在调用栈中安装异常处理器。抛出异常时，活动记录依次弹出，直到遇到异常处理器；随后从该处理器继续求值。换言之，异常充当一次非局部控制转移，其目标是最近安装的异常处理器，也就是调用栈中离它最近的处理器。

图14-2 汇总的异常处理器形式与 ML、Java 都很相似。表达式 `try  $t_1$  with  $t_2$` 意为：“返回 t_1 的求值结果；除非它异常终止，此时改为求值处理器 t_2 。”E-TRYV 说明， t_1 归约为值 v_1 后，可以丢掉 `try`，因为已经知道用不到它。E-TRYERROR 则说明，若 t_1 的求值结果是 `error`，就用 t_2 替换整个 `try`，从那里继续求值。E-TRY 说明，在 t_1 归约为值或 `error` 之前，应当只继续处理 t_1 ，而不动 t_2 。

`try` 的类型规则直接来自操作语义。整个 `try` 的结果可能是主表达式 t_1 的结果，也可能是处理器 t_2 的结果；只要求二者具有同一类型 T ，这也是整个 `try` 的类型。

类型安全性质及其证明与上一节相比，本质上没有变化。

14.3 携带值的异常

§14.1 与 §14.2 的机制让函数可以通知调用者“发生了某种不寻常的事情”。通常还需要传回额外信息，说明具体发生了哪种事情，因为处理器应该采取的行动——恢复后重试，或向用户呈现易懂的错误消息——可能取决于这些信息。

图14-3 说明怎样扩充基本异常处理构造，让每个异常携带一个值。这个值的类型写作 T_{exn} 。暂且不确定它的具体性质，稍后讨论几种选择。

原子项 `error` 被项构造子 `raise  $t$` 取代， t 是要传给异常处理器的额外信息。`try` 的语法不变，但 `try  $t_1$  with  $t_2$` 中的处理器 t_2 现在被解释为以额外信息为参数的函数。

求值规则 E-TRYRAISE 实现这种行为：取得 t_1 中 `raise` 携带的额外信息，再传给处理器 t_2 。E-APPRAISE1 与 E-APPRAISE2 像 图14-1 的 E-APPERR1 与 E-APPERR2 一样，使异常穿过应用向外传播。不过，这些规则只允许传播额外信息已经是值的异常；若试图求值的 `raise` 所携信息本身还要求值，规则会阻塞，迫使我们先用 E-RAISE 求值额外信息。E-RAISERAISE 传播在求值某个异常所要携带的信息时发生的异常。E-TRYV 与 §14.2 一样，

说明主表达式归约为值后可以丢弃 `try`。E-TRY 则让求值器继续处理 `try` 体，直到它变为值或 `raise`。

类型规则反映这些行为变化。在 T-EXN 中，要求额外信息具有类型 T_{exn} ；整个 `raise` 则可以具有上下文所需的任意类型 T 。在 T-TRY 中，检查处理器 t_2 是一个函数：给它 T_{exn} 型的额外信息，它产生与 t_1 相同类型的结果。

最后，考虑 T_{exn} 的几种选择。

1. 可以令 $T_{exn} = \text{Nat}$ 。这对应 Unix 操作系统函数等使用的 `errno` 约定：每个系统调用都返回数字“错误码”，其中 0 表示成功，其他值报告不同的异常条件。
2. 可以令 $T_{exn} = \text{String}$ 。这样不必查错误码表，异常抛出点也可以按需构造更具描述性的消息。额外灵活性的代价是，处理器可能需要解析这些字符串，才能知道发生了什么。
3. 若把 T_{exn} 定义成变体类型，就能既传递更丰富的异常，又避免解析字符串：

$$T_{exn} = \langle \text{divideByZero} : \text{Unit}, \text{overflow} : \text{Unit}, \\ \text{fileNotFound} : \text{String}, \text{fileNotReadable} : \text{String}, \dots \rangle$$

处理器可以用简单的 `case` 表达式区分异常种类；不同异常也能携带不同类型的附加信息。除零等异常无需额外负担；找不到文件则可携带字符串，指出打开哪个文件时发生错误。

这种方案的问题是很不灵活：要求预先固定任意程序可能抛出的全部异常，即变体类型 T_{exn} 的标签集合，程序员无法声明应用特有的异常。

4. 可以进一步把 T_{exn} 取作可扩展变体类型 (*extensible variant type*)，为用户定义异常留出空间。ML 采用这种思路，提供名为 `exn` 的单一可扩展变体类型。²在当前体系中，ML 声明 `exception l of T` 可理解成如下操作：保证 l 不同于 T_{exn} 中已有的全部标签；从现在开始，若声明前的可能变体为 $l_1 : T_1, \dots, l_n : T_n$ ，就令³

$$T_{exn} = \langle l_1 : T_1, \dots, l_n : T_n, l : T \rangle$$

ML 抛出异常的语法是 `raise l(t)`，其中 l 是当前作用域定义的异常标签。它可以理解为加标签运算符与简单 `raise` 的组合：

$$\text{raise } l(t) \triangleq \text{raise } (\langle l = t \rangle \text{ as } T_{exn})$$

类似地，ML 的 `try` 可以用简单 `try` 加 `case` 去糖：

$$\text{try } t \text{ with } l(x) \Rightarrow h \\ \triangleq \text{try } t \text{ with } (\lambda e : T_{exn}. \text{case } e \text{ of } \langle l = x \rangle \Rightarrow h \mid _ \Rightarrow \text{raise } e)$$

`case` 检查抛出的异常是否带标签 l 。若是，就把异常携带的值绑定到 x ，求值处理器 h ；若不是，就落入通配分支，重新抛出异常。异常会继续传播——途中也许被捕获后又抛出——直到到达愿意处理它的处理器，或一路到达最外层并终止整个程序。

²也可以更进一步，把可扩展变体类型作为一般语言特性；ML 设计者选择只把 `exn` 当作特例。

³异常形式是绑定子，因此需要时总能对它作 α 变换，保证 l 不同于 T_{exn} 中已经使用的标签。

5. Java 使用类而不是可扩展变体来支持用户定义异常。语言提供内建类 `Throwable`；它或其任意子类的实例都可以用于 `throw`（相当于这里的 `raise`）或 `try...catch`（相当于这里的 `try...with`）语句。只需定义 `Throwable` 的新子类，就能声明新异常。

这种异常机制与 ML 的机制其实密切对应。粗略地说，Java 异常对象的运行时表示由一个表示其类的标签——直接对应 ML 的可扩展变体标签——和一条实例变量记录——对应标签携带的附加信息——组成。

Java 异常在几个方面比 ML 更进一步。一是异常标签上存在由子类次序生成的自然偏序；异常 l 的处理器实际会捕获携带 l 类或其任意子类对象的所有异常。二是 Java 区分异常与错误：前者是内建类 `Exception` 的子类，而 `Exception` 又是 `Throwable` 的子类，应用程序可能希望捕获并尝试从中恢复；后者是同样继承自 `Throwable` 的 `Error` 子类，表示通常应直接终止执行的严重条件。两者的关键区别在类型检查规则：方法必须显式声明可能抛出哪些异常，但不必声明可能抛出哪些错误。

练习 14.3.1 (★★★). 上面对第 4 种方案的说明还只是一个非正式草图。请补全其语法、类型规则和求值规则，给出一套完整、严格的形式定义。

[查看原书附录 A 中的 14.3.1 解答](#)

练习 14.3.2 (★★★★). 上文指出，Java 异常——即 `Exception` 的子类——比 ML 中的异常以及图14-3 所定义的异常系统中的异常受到更严格控制：方法可能抛出的每个异常都必须在方法类型中声明。扩展练习14.3.1 的解答，让函数类型除了表示参数与结果类型，也表示它可能抛出的异常集合。证明你的系统类型安全。

[查看原书附录 A 中的 14.3.2 解答](#)

练习 14.3.3 (★★★). 许多其他控制构造也能用类似本章的技术形式化。熟悉 Scheme 的 `call-with-current-continuation`（通常缩写为 `call/cc`）运算符的读者（参见 Clinger、Friedman 与 Wand, 1985; Kelsey、Clinger 与 Rees, 1998; Dybvig, 1996; Friedman、Wand 与 Haynes, 2001），可以尝试以类型 `Cont T` 为基础构造类型规则；该类型表示期待 T 型参数的 T -续延。

[查看原书附录 A 中的 14.3.3 解答](#)

第三部分

子类型化

第十五章 子类型化

过去几章一直在简单类型 lambda 演算的框架中研究各种语言特性的类型行为。本章加入一项更基础的扩展：子类型化 (*subtyping*)，有时也称 子类型多态 (*subtype polymorphism*)。此前的许多特性大体可以彼此独立地加入语言，子类型化却会横贯整个类型系统，并以并不简单的方式与多数其他语言特性相互作用。

子类型化常见于面向对象语言，也常被视为面向对象风格的核心特性。[第十八章](#) 将详细讨论这层联系。本章先在一个只含函数和记录的精简语言中介绍子类型化；这样的框架已经足以呈现大多数关键问题。¹[§15.5](#)讨论它与前几章若干特性的组合，[§15.6](#)则考察一种更精细的语义：把子类型化的使用翻译成运行时强制转换。

15.1 包容规则

没有子类型化时，简单类型 lambda 演算的规则有时显得过于僵硬。类型系统要求实参类型与函数的参数类型完全相同，因而会拒绝不少程序员看来显然不会出错的程序。回顾函数应用的类型规则：

$$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}} \text{ T-APP}$$

按照这条规则，行为良好的项

$$(\lambda r : \{x : \text{Nat}\}. r.x) \{x = 0, y = 1\}$$

无法赋型：实参的类型是 $\{x : \text{Nat}, y : \text{Nat}\}$ ，函数却要求 $\{x : \text{Nat}\}$ 。然而，函数只需要实参含有一个类型为 Nat 的字段 x ，并不关心是否还含有其他字段。从函数的类型就能看出这一点，不必检查函数体。因此，把 $\{x : \text{Nat}, y : \text{Nat}\}$ 类型的实参传给要求 $\{x : \text{Nat}\}$ 的函数总是安全的。

子类型化的目标，就是让类型规则接受这样的项。若类型 S 比 T 提供更多信息，我们称 S 是 T 的子类型，写作 $S <: T$ 。它的含义是：任何类型为 S 的项，都能安全地用于本来需要 T 型项的上下文。这种理解常称为 安全替换原则 (*principle of safe substitution*)。

还可以把 $S <: T$ 读作“ S 所描述的每个值也都由 T 描述”，也就是“ S 的元素集合是 T 的元素集合的子集”。[§15.6](#)会看到，子类型化有时还适合采用更精细的解释；在多数场合，这种子集语义 (*subset semantics*) 已经足够。

¹本章研究带子类型化 ([图15-1](#)) 和记录 ([图15-3](#)) 的简单类型 lambda 演算 $\lambda_{<:}$ 。对应的 OCaml 实现是 `rcdsub`；含自然数的例子需要用 `fullsub` 检查。

类型关系与子类型关系由一条新的类型规则连接起来，这就是 包容规则 (*rule of subsumption*):

$$\frac{\Gamma \vdash t : S \quad S <: T}{\Gamma \vdash t : T} \text{ T-SUB}$$

若 $S <: T$ ，这条规则允许把每个 S 型项也当作 T 型项。例如，一旦规定 $\{x : \text{Nat}, y : \text{Nat}\} <: \{x : \text{Nat}\}$ ，就能由 T-SUB 得到 $\vdash \{x = 0, y = 1\} : \{x : \text{Nat}\}$ ，从而为开头的例子赋值。

$\lambda_{<}$:

扩展 $\lambda_{\rightarrow}$ (图 9-1)

新增句法形式

$T ::= \dots \mid \text{Top}$ 最大类型

求值 仍采用图9-1的 E-APP1、E-APP2 与 E-APPABS；子类型化不增加新的求值规则

子类型化 $S <: T$

$$\frac{}{S <: S} \text{ S-REFL}$$

$$\frac{}{S <: \text{Top}} \text{ S-TOP}$$

$$\frac{S <: U \quad U <: T}{S <: T} \text{ S-TRANS}$$

$$\frac{T_1 <: S_1 \quad S_2 <: T_2}{S_1 \rightarrow S_2 <: T_1 \rightarrow T_2} \text{ S-ARROW}$$

新增类型规则

$$\frac{\Gamma \vdash t : S \quad S <: T}{\Gamma \vdash t : T} \text{ T-SUB}$$

图 15-1: 带子类型化的简单类型 lambda 演算 ($\lambda_{<}$)

15.2 子类型关系

子类型关系由一组推导 $S <: T$ 的推理规则形式化。这个判断读作“ S 是 T 的子类型”，也可以说“ T 是 S 的超类型”。我们分别处理函数类型、记录类型等各种类型形式，为每种形式规定：何时可以把一种类型的元素安全地用于需要另一种类型的地方。

在讨论具体的类型构造子之前，先作两项一般规定。子类型关系具有自反性和传递性：

$$\frac{}{S <: S} \text{ S-REFL} \quad \frac{S <: U \quad U <: T}{S <: T} \text{ S-TRANS}$$

这两条规则都直接来自安全替换原则。

先看记录。设 $S = \{k_j : S_j\}^{j \in 1..m}$ ， $T = \{l_i : T_i\}^{i \in 1..n}$ 。如果 T 的字段少于 S ，我们希望把 S 看作 T 的子类型。特别地，从记录类型末尾“忘掉”若干字段是安全的。这由宽度子类型规则 (*width subtyping rule*) 表达：

$$\frac{}{\{l_i : T_i\}^{i \in 1..n+m} <: \{l_i : T_i\}^{i \in 1..n}} \text{ S-RCDWIDTH}$$

字段更多的类型反而是“更小”的子类型，乍看似乎反常。可以沿用 §11.8 的记录类型，并对它采取更宽松的解释： $\{x : \text{Nat}\}$ 描述所有“至少含有一个 Nat 型字段 x ”的记录。因此，

$\{x = 3\}$ 、 $\{x = 5\}$ 、 $\{x = 3, y = 100\}$ 和 $\{x = 3, a = \text{true}, b = \text{true}\}$ 都属于该类型。相比之下， $\{x : \text{Nat}, y : \text{Nat}\}$ 还要求字段 y ，所描述的值更少。例如， $\{x = 3, y = 100\}$ 和 $\{x = 3, y = 100, z = \text{true}\}$ 属于这个类型， $\{x = 3\}$ 和 $\{x = 3, a = \text{true}, b = \text{true}\}$ 则不属于。字段更多的记录类型提出了更严格、信息量更大的要求，因而描述更小的值集合。

宽度规则要求两边共有字段的类型完全相同。还可以进一步放宽这一要求：只要两个记录中各对应字段的类型满足子类型关系，共有字段的类型就不必完全相同。深度子类型规则 (*depth subtyping rule*) 表达这一点：

$$\frac{S_i <: T_i \quad (\text{对每个 } i \in 1..n)}{\{l_i : S_i\}^{i \in 1..n} <: \{l_i : T_i\}^{i \in 1..n}} \text{S-RCDDEPTH}$$

例如，结合宽度与深度规则可以推得：

$$\frac{\frac{\frac{}{\{a : \text{Nat}, b : \text{Nat}\} <: \{a : \text{Nat}\}} \text{S-RCDWIDTH}}{\{m : \text{Nat}\} <: \{\}} \text{S-RCDWIDTH}}{\{x : \{a : \text{Nat}, b : \text{Nat}\}, y : \{m : \text{Nat}\}\} <: \{x : \{a : \text{Nat}\}, y : \{\}\}} \text{S-RCDDEPTH}$$

若只想改变一个字段，可对其他字段用 S-REFL 构造平凡的子类型推导。还可以用 S-TRANS 拼接宽度与深度推导，在提升一个字段类型的同时丢掉另一个字段。

最后，记录一旦构造完成，安全使用它的基本操作只有字段投影，而字段投影不受字段排列次序影响。因此得到 排列子类型规则 (*permutation subtyping rule*)：

$$\frac{\{k_j : S_j\}^{j \in 1..n} \text{ 是 } \{l_i : T_i\}^{i \in 1..n} \text{ 的一个排列}}{\{k_j : S_j\}^{j \in 1..n} <: \{l_i : T_i\}^{i \in 1..n}} \text{S-RCDPERM}$$

例如，S-RCDPERM 同时给出 $\{c : \text{Bool}, b : \text{Bool}, a : \text{Nat}\} <: \{a : \text{Nat}, b : \text{Bool}, c : \text{Bool}\}$ 及其反向关系。这也说明子类型关系不具有反对称性。把排列、宽度和传递规则结合起来，就能丢掉记录类型中任意位置的字段。

练习 15.2.1 (★). 画出推导，证明 $\{x : \text{Nat}, y : \text{Nat}, z : \text{Nat}\} <: \{y : \text{Nat}\}$ 。

[查看原书附录 A 中的 15.2.1 解答](#)

宽度、深度和排列规则分别表达记录使用中的三种灵活性。为了说明概念，把它们分成三条规则很有帮助；有些语言确实只允许其中一部分。例如，Abadi 与 Cardelli (1996) 提出的对象演算有多个变体，其中大多数不采用深度子类型化。实际实现时，把三者合并成一条同时处理宽度、深度和排列的宏规则更为方便；[第十六章](#)将采用这种做法。

因为当前语言是高阶语言，函数本身也可以作为参数传递，所以还必须规定函数类型之间何时存在子类型关系：

$$\frac{T_1 <: S_1 \quad S_2 <: T_2}{S_1 \rightarrow S_2 <: T_1 \rightarrow T_2} \text{S-ARROW}$$

其中，参数类型的前提 $T_1 <: S_1$ 与结论 $S_1 \rightarrow S_2 <: T_1 \rightarrow T_2$ 中的子类型方向相反，称为参数类型上的 逆变 (*contravariance*)；结果类型的前提 $S_2 <: T_2$ 与结论中的方向相同，

称为协变 (*covariance*)。若函数 $f : S_1 \rightarrow S_2$ ，它能接收每个 S_1 型值，自然也能接收 S_1 的任意子类型 T_1 的值；它返回 S_2 型值，这些结果也可以被看作任意超类型 T_2 的值。因此， f 也能具有类型 $T_1 \rightarrow T_2$ 。换一种说法，传给函数的任何实参都不能超出它的接收能力 ($T_1 <: S_1$)，函数返回的任何结果也不能超出上下文的接收能力 ($S_2 <: T_2$)。

译者注：把类型理解为它所包含的值的集合。若 $T_1 <: S_1$ ，就有 $\llbracket T_1 \rrbracket \subseteq \llbracket S_1 \rrbracket$ 。固定结果类型 U 后， $S_1 \rightarrow U$ 型函数必须能处理集合 $\llbracket S_1 \rrbracket$ 中的每个输入； $T_1 \rightarrow U$ 型函数只须处理较小集合 $\llbracket T_1 \rrbracket$ 中的输入。前一条件更严格，因此满足它的函数也一定满足后一条件，即 $\llbracket S_1 \rightarrow U \rrbracket \subseteq \llbracket T_1 \rightarrow U \rrbracket$ ，从而 $S_1 \rightarrow U <: T_1 \rightarrow U$ 。换言之，能处理 S_1 中所有输入的函数，一定也能处理其子集 T_1 中的所有输入；反过来则未必。因此， $S_1 \rightarrow U$ 型函数都属于 $T_1 \rightarrow U$ ，参数位置上的子类型方向由此发生反转。

最后，引入最大类型 **Top**，并规定每个类型都是它的子类型：

$$\frac{}{S <: \text{Top}} \text{S-Top}$$

§15.4还会讨论这个类型。

形式上，子类型关系是由上述规则闭合生成的最小关系。自反性与传递性使它成为预序；记录排列规则又使不同类型可能互为子类型，所以它不是偏序。

{}

扩展 $\lambda \rightarrow$ (图9-1)

$$\begin{array}{lcl} t & ::= & \dots \mid \{l_i = t_i\}^{i \in 1..n} \quad \text{记录} \\ & & \mid t.l \quad \text{投影} \\ v & ::= & \dots \mid \{l_i = v_i\}^{i \in 1..n} \quad \text{记录值} \\ T & ::= & \dots \mid \{l_i : T_i\}^{i \in 1..n} \quad \text{记录类型} \\ \mathcal{R}_j(s) & \triangleq & \{\{l_i = v_i\}^{i < j}, l_j = s, \{l_i = t_i\}^{i > j}\} \\ \frac{}{\{l_i = v_i\}^{i \in 1..n}.l_j \rightarrow v_j} & \text{E-PROJ} & \frac{t_j \rightarrow t'_j}{\mathcal{R}_j(t_j) \rightarrow \mathcal{R}_j(t'_j)} \text{E-RCD} \\ \frac{\Gamma \vdash t_i : T_i \quad (\text{对每个 } i)}{\Gamma \vdash \{l_i = t_i\}^{i \in 1..n} : \{l_i : T_i\}^{i \in 1..n}} & \text{T-RCD} & \frac{\Gamma \vdash t_1 : \{l_i : T_i\}^{i \in 1..n}}{\Gamma \vdash t_1.l_j : T_j} \text{T-PROJ} \end{array}$$

图 15-2: 记录 (与图11-7相同)

<:

扩展 $\lambda_{<}$: (图15-1) 及记录 (图15-2)

$$\begin{array}{l} \frac{}{\{l_i : T_i\}^{i \in 1..n+m} <: \{l_i : T_i\}^{i \in 1..n}} \text{S-RCDWIDTH} \\ \frac{\{k_j : S_j\}^{j \in 1..n} \text{ 是 } \{l_i : T_i\}^{i \in 1..n} \text{ 的一个排列}}{\{k_j : S_j\}^{j \in 1..n} <: \{l_i : T_i\}^{i \in 1..n}} \text{S-RCDPERM} \\ \frac{S_i <: T_i \quad (\text{对每个 } i)}{\{l_i : S_i\}^{i \in 1..n} <: \{l_i : T_i\}^{i \in 1..n}} \text{S-RCDDEPTH} \end{array}$$

图 15-3: 记录与子类型化

为验证本章开头的例子，作缩写

$$\begin{aligned} f &\triangleq \lambda r : \{x : \text{Nat}\}. r.x & R_x &\triangleq \{x : \text{Nat}\} \\ xy &\triangleq \{x = 0, y = 1\} & R_{xy} &\triangleq \{x : \text{Nat}, y : \text{Nat}\} \end{aligned}$$

在通常的自然数类型规则下，可构造 $\vdash f xy : \text{Nat}$ 的推导：

$$\frac{\frac{\frac{\vdash 0 : \text{Nat} \quad \vdash 1 : \text{Nat}}{\vdash xy : R_{xy}} \text{ T-RCD} \quad \frac{}{R_{xy} <: R_x} \text{ S-RCDWIDTH}}{\vdash xy : R_x} \text{ T-SUB} \quad \frac{}{\vdash f xy : \text{Nat}} \text{ T-APP}$$

练习 15.2.2 (★). 这是 $\vdash f xy : \text{Nat}$ 的唯一推导吗？

[查看原书附录 A 中的 15.2.2 解答](#)

练习 15.2.3 (★). 1. $\{a : \text{Top}, b : \text{Top}\}$ 有多少个不同的超类型？

2. 能否在子类型关系中找到一条无限下降链，即类型序列 $S_0, S_1, \dots$ ，其中每个 $S_{i+1} <: S_i$ ？
3. 无限上升链呢？

[查看原书附录 A 中的 15.2.3 解答](#)

练习 15.2.4 (★). 是否存在一个类型，它是所有其他类型的子类型？是否存在一个箭头类型，它是所有其他箭头类型的超类型？

[查看原书附录 A 中的 15.2.4 解答](#)

练习 15.2.5 (★★). 设演算加入§11.6的积类型构造子 $T_1 \times T_2$ 。仿照记录的 S-RCDDEPTH 规则，可以为积类型加入下面的子类型规则：

$$\frac{S_1 <: T_1 \quad S_2 <: T_2}{S_1 \times S_2 <: T_1 \times T_2} \text{ S-PRODDEPTH}$$

再加入下面的积类型宽度规则是否合适？

$$\frac{}{T_1 \times T_2 <: T_1} \text{ S-PRODWIDTH}$$

[查看原书附录 A 中的 15.2.5 解答](#)

15.3 子类型关系与类型关系的性质

确定带子类型化的 lambda 演算后，还要验证定义确实合理，特别是简单类型 lambda 演算的保持性和进展性在加入子类型化后仍然成立。

练习 15.3.1 (推荐, ★★). 继续阅读前，先预测可能在哪里遇到困难。假设定义子类型关系时误加了一条不正确的规则，系统的哪些性质可能失效？反过来，如果删去一条子类型规则，是否也会破坏某些性质？

查看原书附录 A 中的 15.3.1 解答

下面先给出子类型关系的一项关键性质；它与简单类型 lambda 演算中类型关系的反演引理（引理9.3.1）相对应。若 S 是某个箭头类型的子类型，子类型反演引理说明 S 本身也必须是箭头类型，并给出两个参数类型间的逆变关系与两个结果类型间的协变关系。对于记录类型，若 $S <: \{l_i : T_i\}^{i \in 1..n}$ ，类似结论说明 S 至少含有 $\{l_i : T_i\}^{i \in 1..n}$ 列出的所有字段；字段可以采用不同次序，共有字段的类型满足子类型关系。

引理 15.3.2 (子类型关系的反演).

1. 若 $S <: T_1 \rightarrow T_2$ ，则 $S = S_1 \rightarrow S_2$ ，且 $T_1 <: S_1$ 、 $S_2 <: T_2$ 。
2. 若 $S <: \{l_i : T_i\}^{i \in 1..n}$ ，则 $S = \{k_j : S_j\}^{j \in 1..m}$ ，并且 $\{l_i \mid i \in 1..n\} \subseteq \{k_j \mid j \in 1..m\}$ ；对每对相同标签 $l_i = k_j$ ，都有 $S_j <: T_i$ 。

证明. 练习 (推荐, ★★)。

□

查看原书附录 A 中的 15.3.2 解答

为证明求值保持类型，先给出类型关系的反演引理。这里只列保持性证明实际需要的两个情形；更一般的形式可以从下一章的算法类型关系（定义16.2.2）读出。

引理 15.3.3.

1. 若 $\Gamma \vdash \lambda x : S_1. s_2 : T_1 \rightarrow T_2$ ，则 $T_1 <: S_1$ ，且 $\Gamma, x : S_1 \vdash s_2 : T_2$ 。
2. 若 $\Gamma \vdash \{k_a = s_a\}^{a \in 1..m} : \{l_i : T_i\}^{i \in 1..n}$ ，则类型中列出的每个标签 l_i 都出现在记录项的标签 k_a 中，即 $\{l_i\}^{i \in 1..n} \subseteq \{k_a\}^{a \in 1..m}$ ；对每对相同标签 $k_a = l_i$ ，都有 $\Gamma \vdash s_a : T_i$ 。

证明. 对类型推导作直接归纳；末条规则为 T-SUB 时使用 引理15.3.2。

□

接着需要类型关系的替换引理。它的陈述与简单类型 lambda 演算中的 引理9.3.8相同，证明也几乎一致。

引理 15.3.4 (替换). 若 $\Gamma, x : S \vdash t : T$ 且 $\Gamma \vdash s : S$ ，则 $\Gamma \vdash [x \mapsto s]t : T$ 。

证明. 对给定类型推导的深度作归纳。新增的 T-SUB、T-RCD 和 T-PROJ 情形直接使用归纳假设，其余与引理9.3.8相同。

□

保持性定理的陈述仍与以前相同，不过证明中的若干地方需要处理子类型化。

定理 15.3.5 (保持性). 若 $\Gamma \vdash t : T$ 且 $t \longrightarrow t'$, 则 $\Gamma \vdash t' : T$ 。

证明. 对给定类型推导的深度作归纳，并按最后一条类型规则分类。多数情形与 [定理9.3.9](#) 相同；这里只写新增或需要子类型化的情形。

情形 *T-VAR*. 变量没有求值规则，不可能发生。

情形 *T-ABS*. lambda 抽象已经是值，不可能发生。

情形 *T-APP*. 已知 $t = t_1 t_2$ 、 $\Gamma \vdash t_1 : T_{11} \rightarrow T_{12}$ 、 $\Gamma \vdash t_2 : T_{11}$ ，且 $T = T_{12}$ 。求值末条规则可能是 E-APP1、E-APP2 或 E-APPABS。前两种由归纳假设和 T-APP 立即得到。若为 E-APPABS，则 $t_1 = \lambda x : S_{11}. t_{12}$ 、 $t_2 = v_2$ ，且 $t' = [x \mapsto v_2]t_{12}$ 。由[引理15.3.3](#)第 1 项得到 $T_{11} <: S_{11}$ 和 $\Gamma, x : S_{11} \vdash t_{12} : T_{12}$ 。由 T-SUB 得 $\Gamma \vdash v_2 : S_{11}$ ，再由替换引理得到 $\Gamma \vdash t' : T_{12}$ 。

情形 *T-RCD*. 记录只能由 E-RCD 求值。设第 j 个字段从 t_j 变为 t'_j 。对相应前提 $\Gamma \vdash t_j : T_j$ 使用归纳假设，再用 T-RCD 重建整条记录的类型推导。

情形 *T-PROJ*. 若末条求值规则为 E-PROJ，由归纳假设和 T-PROJ 得证。若为 E-PROJRCD，则 $t_1 = \{k_a = v_a\}^{a \in 1..m}$ 、 $l_j = k_b$ ，且 $t' = v_b$ 。由[引理15.3.3](#)第 2 项，类型 $\{l_i : T_i\}^{i \in 1..n}$ 要求的标签都出现在 t_1 中，并且相同标签对应的值具有所需字段类型；特别地， $\Gamma \vdash v_b : T_j$ 。

情形 *T-SUB*. 由前提得 $\Gamma \vdash t : S$ 和 $S <: T$ 。归纳假设给出 $\Gamma \vdash t' : S$ ，再由 T-SUB 得 $\Gamma \vdash t' : T$ 。□

证明进展性之前，仍先给出典范形式引理，说明箭头类型和记录类型的闭值只能采用哪些形式。

引理 15.3.6 (典范形式).

1. 若闭值 v 具有类型 $T_1 \rightarrow T_2$ ，则 $v = \lambda x : S_1. t_2$ 。
2. 若闭值 v 具有类型 $\{l_i : T_i\}^{i \in 1..n}$ ，则 $v = \{k_a = v_a\}^{a \in 1..m}$ ，且类型 $\{l_i : T_i\}^{i \in 1..n}$ 中列出的每个标签都出现在该记录值中。

证明. 练习 (推荐, ★★)。

□

[查看原书附录 A 中的 15.3.6 解答](#)

定理 15.3.7 (进展性). 若 t 是闭的良类型项，则 t 要么是值，要么存在 t' 使 $t \longrightarrow t'$ 。

证明. 对类型推导作直接归纳。变量情形因 t 是闭项而不可能发生；lambda 抽象本身就是值。

情形 *T-APP*. 分别对 t_1 和 t_2 使用归纳假设。若 t_1 能求值，使用 E-APP1；若 t_1 已是值而 t_2 能求值，使用 E-APP2。若二者都是值，[引理15.3.6](#)说明 t_1 是 lambda 抽象，于是 E-APPABS 适用。

情形 *T-RCD*. 对每个字段使用归纳假设。若全部是值，整条记录就是值；否则，E-RCD 可以让最先尚未成为值的字段求值一步。

情形 $T\text{-}PROJ$ 。若被投影项能求值，使用 $E\text{-}PROJ$ 。若它已经是值，[引理15.3.6](#)说明它是一条记录，而且包含所请求的标签，因此 $E\text{-}PROJ\text{RCD}$ 适用。

情形 $T\text{-}SUB$ 。直接使用归纳假设。 $\square$

15.4 Top 与 Bot 类型

最大类型 Top 并不是带子类型化的简单类型 λ 演算所必需的；删去它不会破坏系统的性质。不过，多数论述仍会保留它。首先，它对应多数面向对象语言中的 `Object` 类型。其次，在把子类型化与参数多态结合的更复杂系统中， Top 是很方便的技术工具。例如，在 $F_{\leq}$ ([第二十六章](#)和[第二十八章](#)) 中，它使有界量化能够表达普通的无界量化，从而简化系统。甚至记录也能在 $F_{\leq}$ 中编码，而该编码关键依赖 Top 。最后，它的行为简单，例子里又常有用，所以没有多少理由把它删掉。

既然已经有了最大类型 Top ，一个相应的问题是：能否再加入最小类型 Bot ，使它成为所有类型的子类型？答案是肯定的，[图15-4](#)给出了相应规则。

译者注： [练习15.2.4](#)考察的是加入 Bot 之前的类型语法，结论是其中没有任何现成类型能够成为所有类型的共同子类型。本节随后扩展类型语法，新增 Bot ，并用 $S\text{-}BOT$ 规定 $Bot <: T$ 对任意 T 成立。最小类型来自这次扩展，因此与该练习的结论并不矛盾。

Bot

扩展 $\lambda_{\leq}$: ([图 15-1](#))

$$\begin{array}{c} T ::= \dots \mid Bot \quad \text{最小类型} \\ \hline S\text{-}BOT \\ Bot <: T \end{array}$$

图 15-4: 最小类型

首先， Bot 是空的：不存在类型为 Bot 的闭值。反设存在这样的值 v 。由 $S\text{-}BOT$ 和 $T\text{-}SUB$ ，可得 $\vdash v : Top \rightarrow Top$ ；典范形式引理于是要求 v 是 λ 抽象。另一方面，同样可得 $\vdash v : \{\}$ ，典范形式引理又要求 v 是记录。句法上一个值不可能同时是函数和记录，因此假设导致矛盾。

空类型并非没有用处。它可以方便地表达某些操作——尤其是抛出异常或调用续延——不会正常返回。给这类表达式类型 Bot 有两个好处：一方面，类型向程序员表明不应期待结果；另一方面，类型检查器知道该表达式可以安全地放入需要任意结果类型的上下文。例如，把[第十四章](#)的 `error` 赋予 Bot 后，下面的项可以赋型：

$\lambda x : T. \text{if } \langle \text{检查 } x \text{ 是否合理} \rangle \text{ then } \langle \text{计算结果} \rangle \text{ else error}$

正常分支无论具有什么类型，都能由包容规则把 `error` 提升到同一类型，使两个分支满足 $T\text{-}IF$ 。²

²在 ML 一类带多态的语言中，也可以把 $\forall X. X$ 用作 `error` 等构造的结果类型。它通过实例化为任意类型达到与 Bot 相似的效果；两者虽然原理不同，都是空类型。

译者注：续延 (*continuation*) 首先是描述普通程序求值过程的概念。考虑下面的程序：

```
x = f();
print(x + 1)
```

执行 f 时，运行时系统必须记住： f 返回以后，还要把返回值赋给 x ，计算 $x + 1$ ，再打印结果。这段“当前计算得到一个值以后，接下来要完成的全部工作”就是当前续延；机器通常用调用栈和返回地址保存它。普通的 `return 5` 可以理解为把 5 交给这段续延，让程序恢复调用者的计算并执行 `print(5 + 1)`。因此，即使一门语言没有把续延直接暴露给程序员，它的运行时仍然需要维护类似的信息。

同一概念也可以用带洞的求值上下文表示。例如，程序正在计算

$$1 + (2 \times \square)$$

时， $\square$ 表示当前等待结果的子表达式位置。若这里得到 x ，余下的计算便是 $1 + (2 \times x)$ 。所以带洞的上下文 $1 + (2 \times \square)$ 描述了此处的续延；当子表达式求得 5 时，普通求值会把 5 填入洞中，继续得到 11。

通常，程序只能沿着当前续延继续执行。Scheme 的 `call-with-current-continuation` (`call/cc`，也见[练习14.3.3](#)) 进一步允许程序把当前续延捕获为一个可调用的值，保存起来并在以后恢复。设 k 保存了上面的续延 $1 + (2 \times \square)$ 。调用 $k(5)$ 的含义是：放弃调用时正在使用的续延，恢复 k 保存的续延，并让保存位置像是刚刚得到了值 5。例如，程序后来正在计算

$$100 + k(5)$$

时，当前续延是 $100 + \square$ 。调用 $k(5)$ 会丢弃这个当前续延，恢复 $1 + (2 \times \square)$ ，于是程序继续得到 11，外围的加 100 不会执行。相比之下，若 $h(x) = 1 + (2 \times x)$ 只是普通函数， $100 + h(5)$ 会正常返回并得到 111。这个差别说明，捕获的续延代表一条可以恢复的控制路径；调用它是在替换当前控制路径。

把续延开放给程序后，非局部返回、异常、回溯、协程、生成器和任务调度等控制结构都可以在它的基础上表达。完整续延的能力很强，许多语言只提供用途更明确的受限形式，例如异常、生成器和 `async/await`。本节只需要其中一个性质：调用这种已捕获的续延不会返回当前调用位置。若续延等待一个 T 型值，就可以把它理解成具有类似 $T \rightarrow \text{Bot}$ 的类型；它接收一个值并把控制转移到保存的计算，当前调用点不会产生正常结果。这正是此处把调用续延与抛出异常并列的原因。

但 Bot 会显著增加类型检查的复杂度。没有它时，若应用 $t_1 t_2$ 良类型，就能推断 t_1 具有箭头类型；有了它，只能推断 t_1 具有箭头类型或 Bot。[§16.4](#)会展开这一点；在有界量化系统中，困难还会被放大，见[§28.8](#)。因此，加入 Bot 比加入 Top 更重。本书后续章节的主体系统不再把它作为默认组成部分；上述两节只在局部扩展中考察加入它所带来的问题。

15.5 子类型化与其他特性

把当前演算逐步扩展成完整程序语言时，每加入一种特性，都必须检查它与子类型化如何相互作用。本节讨论此前见过的若干特性。³子类型化与参数多态、递归类型和类型算子的组合要复杂得多，分别见[第二十六章](#)和[第二十八章](#)、[第二十章](#)和[第二十一章](#)、以及[第三十一章](#)。

类型断言与类型转换

[§11.4](#)引入的类型断言 $t \text{ as } T$ 是一种经过检查的说明文字，让程序员在程序中记录某个子项应具有的类型。本书例子也用它控制类型的打印形式，要求类型检查器采用更易读的缩写。

³本节讨论的大多数扩展没有在 `fullsub` 检查器中实现。

在 Java、C++ 等带子类型化的语言中，这个构造更为重要，常称为 类型转换 (*casting*)，写作 $(T)t$ 。类型转换分成两种性质很不相同的形式：向上类型转换 (*up-cast*) 和 向下类型转换 (*down-cast*)。前者直接沿用类型断言，后者需要运行时类型测试。

向上类型转换把项指定为其自然类型的某个超类型。类型检查器先推导 $\Gamma \vdash t : S$ ，再由 $S <: T$ 和 T-SUB 得到 $\Gamma \vdash t : T$ ，最后应用类型断言规则。完整推导为：

$$\frac{\frac{\Gamma \vdash t : S \quad S <: T}{\Gamma \vdash t : T} \text{ T-SUB}}{\Gamma \vdash t \text{ as } T : T} \text{ T-ASCRIIBE}$$

上图最下面的一步使用的 T-ASCRIIBE，就是§11.4 定义的类型断言规则：

$$\frac{\Gamma \vdash t : T}{\Gamma \vdash t \text{ as } T : T} \text{ T-ASCRIIBE}$$

它可以看作一种抽象手段：把值的某些组成部分隐藏起来，使周围上下文无法使用。例如，可以通过向上类型转换隐藏记录或对象的一些字段或方法。

向下类型转换允许给项指定静态类型规则无法推导出的类型。为此，把类型断言规则改为：

$$\frac{\Gamma \vdash t : S}{\Gamma \vdash t \text{ as } T : T} \text{ T-DOWNCAST}$$

这里只检查 t 本身良类型，不要求 S 与 T 有任何静态关系。例如：

$$f = \lambda x : \text{Top}. (x \text{ as } \{a : \text{Nat}\}).a$$

程序员等于告诉类型检查器：根据类型规则表达不了的外部理由，自己确信 f 只会接收到含有自然数字段 a 的记录。

若编译器无条件相信这种断言，语言会失去安全性。实际策略是“信任，但验证”：编译期接受断言的类型，运行时再检查实际值是否确实属于所声称的类型。因此，原本直接丢掉断言的规则

$$\frac{}{v \text{ as } T \longrightarrow v} \text{ E-ASCRIIBE}$$

改成带运行时类型前提的规则：

$$\frac{\vdash v : T}{v \text{ as } T \longrightarrow v} \text{ E-DOWNCAST}$$

把 f 应用于 $\{a = 5, b = \text{true}\}$ 时，检查成功；应用于 $\{b = \text{true}\}$ 时，规则无法使用，求值卡住。运行时检查恢复了保持性。

练习 15.5.1 ($\star\star, \rightarrow$). 证明加入上述向下类型转换规则后，类型保持性仍然成立。

查看练习 15.5.1 的译者参考解答

进展性则会失效，因为错误的向下类型转换可以让良类型程序卡住。真实语言通常把失败的转换变成可捕获的动态异常，或改用动态类型测试：

$$\frac{\Gamma \vdash t_1 : S \quad \Gamma, x : T \vdash t_2 : U \quad \Gamma \vdash t_3 : U}{\Gamma \vdash \text{if } t_1 \text{ in } T \text{ then } x \mapsto t_2 \text{ else } t_3 : U} \text{ T-TEST}$$

成功与失败两种求值情形分别是：

$$\frac{\vdash v_1 : T}{\text{if } v_1 \text{ in } T \text{ then } x \mapsto t_2 \text{ else } t_3 \longrightarrow [x \mapsto v_1]t_2} \text{ E-TEST1}$$

$$\frac{\not\vdash v_1 : T}{\text{if } v_1 \text{ in } T \text{ then } x \mapsto t_2 \text{ else } t_3 \longrightarrow t_3} \text{ E-TEST2}$$

译者注：这里的 x 是程序员在类型测试表达式中写下的变量名，用来给通过运行时检查的值命名。可以把

$$\text{if } t_1 \text{ in } T \text{ then } x \mapsto t_2 \text{ else } t_3$$

读作：“先求出 t_1 的值；若该值具有类型 T ，就把它命名为 x ，然后执行 t_2 ；否则执行 t_3 。”箭头 $\mapsto$ 在这里标记变量绑定， x 的作用域仅限于成功分支 t_2 。

静态检查时，只知道 t_1 具有某个类型 S 。运行时若得到值 v_1 ，并确认 $v_1 : T$ ，成功分支便可以在 $x : T$ 的假设下使用这个值；E-TEST1 通过替换 $[x \mapsto v_1]t_2$ 落实这一绑定。这个过程不会复制该值，也不会再次求值 t_1 。若检查失败，E-TEST2 直接选择 t_3 ，所以失败分支中没有变量 x 。

例如，

$$\text{if } obj \text{ in } \{a : \text{Nat}\} \text{ then } x \mapsto x.a \text{ else } 0$$

会在 obj 的实际值含有自然数字段 a 时，用 x 表示这个值并读取 $x.a$ ；若检查失败，则返回 0。

Java 一类语言经常使用向下类型转换，其中一个重要用途是模拟一种简陋的多态机制。旧式 Java 集合类（如 `Set` 和 `List`）是单态的：Java 没有为每个类型 T 提供“元素类型为 T 的列表” `List T`，只有一个 `List` 类型，其元素统一属于最大类型 `Object`。由于每个对象类型都是 `Object` 的子类型，任意对象都可以先通过包容规则提升为 `Object`，再放入列表。可是从列表取出元素时，类型检查器只知道它具有 `Object` 类型。这个类型只列出打印等少数所有 Java 对象共有的通用方法，不能保证对象拥有更具体的方法；要实际使用它，程序必须先把它向下转换成所预期的类型 T 。

Pizza (Odersky 与 Wadler, 1997)、Generic Java (GJ; Bracha、Odersky、Stoutamire 与 Wadler, 1998)、PolyJ (Myers、Bank 与 Liskov, 1997) 以及 NextGen (Cartwright 与 Steele, 1998) 的设计者主张，为 Java 类型系统加入真正的参数多态（见第二十三章）。参数多态比上述向下转换惯用法更安全、更高效，也不需要运行时测试。另一方面，这种扩展会使本已庞大的语言明显复杂化，并与语言和类型系统的许多其他特性发生作用（例如见 Igarashi、Pierce 与 Wadler, 1999、2001）。因此，也可以把向下转换看作安全性与复杂度之间一种务实的折中。

向下类型转换也是 Java 反射机制的关键。程序员可以要求 Java 运行时系统动态加载某个字节码文件，并创建其中某个类的实例。静态类型检查器不可能预知此时会加载怎样的类

——字节码甚至可以在需要时从网络取得——所以最多只能把新实例赋予 `Object` 类型。要实际使用它，程序仍须把它向下转换为所预期的类型 T ；若字节码提供的类与 T 不匹配，还要处理运行时抛出的异常，成功后才能把对象当作 T 使用。

从前面的规则看，加入向下转换似乎意味着把整套类型检查机制都搬进运行时系统。问题还不止于此：值在运行时的表示通常不同于编译器内部的表示，函数尤其会被编译成字节码或本机机器指令；若直接检查这些运行时表示，似乎还得另写一套用于动态检查的类型检查器。真实语言通常用类型标签 (*type tag*) 避免这样做：每个值携带一个机器字大小的标签，保存编译期类型在运行时留下的必要信息。这种标签在某些方面类似于 ML 数据类型的构造子和§11.10 的变体标签，已经足以完成动态子类型测试。第十九章将详细说明这种机制的一种具体形式。

变体

变体的子类型规则与记录几乎相同，差别在于宽度规则的方向：从子类型移向超类型时，可以增加变体标签。带标签的值 $\langle l = t \rangle$ 属于列出标签 l 的变体类型；类型列出的可能标签越多，对值提供的信息越少。单标签类型 $\langle l_1 : T_1 \rangle$ 精确说明其中的值都带标签 l_1 ，双标签类型 $\langle l_1 : T_1, l_2 : T_2 \rangle$ 只能说明值带 l_1 或 l_2 ，其余情形依此类推。反过来，使用变体值时总要写 `case`，并为类型列出的每个标签提供分支；类型中增加标签，只会迫使 `case` 多写一些可能永远用不到的分支。

$\langle \rangle$ 与 $<$:

扩展变体 (图 11-11) 及 $\lambda<$: (图 15-1)

$$\begin{array}{c}
 \frac{}{\langle l_i : T_i \rangle^{i \in 1..n} <: \langle l_i : T_i \rangle^{i \in 1..n+m}} \text{S-VARIANTWIDTH} \\
 \\
 \frac{S_i <: T_i \quad (\text{对每个 } i)}{\langle l_i : S_i \rangle^{i \in 1..n} <: \langle l_i : T_i \rangle^{i \in 1..n}} \text{S-VARIANTDEPTH} \\
 \\
 \frac{\langle k_j : S_j \rangle^{j \in 1..n} \text{ 是 } \langle l_i : T_i \rangle^{i \in 1..n} \text{ 的一个排列}}{\langle k_j : S_j \rangle^{j \in 1..n} <: \langle l_i : T_i \rangle^{i \in 1..n}} \text{S-VARIANTPERM} \\
 \\
 \frac{\Gamma \vdash t_1 : T_1}{\Gamma \vdash \langle l_1 = t_1 \rangle : \langle l_1 : T_1 \rangle} \text{T-TAG}
 \end{array}$$

图 15-5: 变体与子类型化

结合包容规则后，可以删去标签构造上的类型标注：直接把 $\langle l = t \rangle$ 赋予最精确的单标签变体类型，再由 S-VARIANTWIDTH 提升到任意更大的变体类型。

列表

前面已经见过多种协变关系：记录字段和变体分支所携带的类型都是协变的，函数的返回类型也是协变的；函数的参数类型则是逆变的。列表的元素类型同样是协变的：若 $S_1 <: T_1$ ，则元素类型为 S_1 的列表可以安全地看作元素类型为 T_1 的列表。

$$\frac{S_1 <: T_1}{\text{List } S_1 <: \text{List } T_1} \text{ S-LIST}$$

引用

并非所有类型构造子都协变或逆变。为了保持类型安全，Ref 必须是不变的：

$$\frac{S_1 <: T_1 \quad T_1 <: S_1}{\text{Ref } S_1 <: \text{Ref } T_1} \text{ S-REF}$$

只有当引用中存放的类型 S_1 与 T_1 互为子类型时，才能建立 $\text{Ref } S_1$ 与 $\text{Ref } T_1$ 之间的子类型关系。这允许记录字段的排列顺序不同，却不允许改变引用实际能够保存的值。

限制来自引用的读写两种用途。读取 $\text{Ref } T_1$ 时，上下文期望得到 T_1 ；若引用实际存放 S_1 ，必须有 $S_1 <: T_1$ 。写入时，上下文提供 T_1 ；若引用实际类型是 $\text{Ref } S_1$ ，以后别处可能把新值当作 S_1 读出，所以还必须有 $T_1 <: S_1$ 。

练习 15.5.2 ($\star, \rightarrow$)。 1. 若删去 S-REF 的第一个前提，写一个会在运行时发生类型错误（即求值卡住）的短程序。

2. 若删去第二个前提，再写一个这样的程序。

[查看练习 15.5.2 的译者参考解答](#)

数组

数组同样既能读取又能写入，所以也应采用不变规则。Java 却允许数组协变：若 $S_1 <: T_1$ ，则 $S_1[] <: T_1[]$ 。这项设计最初用于弥补若干基本数组操作缺少参数多态，如今普遍被视为缺陷。为了补偿不安全的规则，运行时必须检查每次数组写入，确认待写值属于数组实际元素类型的子类型，影响了数组程序的性能。

$$\frac{S_1 <: T_1 \quad T_1 <: S_1}{\text{Array } S_1 <: \text{Array } T_1} \text{ S-ARRAY} \qquad \frac{S_1 <: T_1}{\text{Array } S_1 <: \text{Array } T_1} \text{ S-ARRAYJAVA}$$

练习 15.5.3 ($\star\star\star, \rightarrow$)。写一个涉及数组的短 Java 程序：它能通过类型检查，却会在运行时抛出 `ArrayStoreException`。

[查看练习 15.5.3 的译者参考解答](#)

再次考察引用

Reynolds 在 Forsythe (1988) 中率先提出更精细的引用分析：引入 **Source** 和 **Sink** 两个构造子，分别表示对引用单元的读取权限和写入权限。具体而言，**Source** T 只允许从单元读取 T 型值，**Sink** T 只允许向单元写入 T 型值；原有的 **Ref** T 则同时保留这两种权限，既可读取，也可写入。相应类型规则只要求所需能力：

$$\frac{\Gamma \vdash t_1 : \text{Source } T_1}{\Gamma \vdash !t_1 : T_1} \text{ T-DEREF} \quad \frac{\Gamma \vdash t_1 : \text{Sink } T_1 \quad \Gamma \vdash t_2 : T_1}{\Gamma \vdash t_1 := t_2 : \text{Unit}} \text{ T-ASSIGN}$$

只读能力协变，只写能力逆变：

$$\frac{S_1 <: T_1}{\text{Source } S_1 <: \text{Source } T_1} \text{ S-SOURCE} \quad \frac{T_1 <: S_1}{\text{Sink } S_1 <: \text{Sink } T_1} \text{ S-SINK}$$

译者注：可以把 **Source** T 看作一个返回 T 型值的函数 $\text{Unit} \rightarrow T$ ，把 **Sink** T 看作一个接收 T 型值的函数 $T \rightarrow \text{Unit}$ 。因此，前者与函数的返回类型一样协变，后者与函数的参数类型一样逆变。

例如，若 $\text{Dog} <: \text{Animal}$ ，那么

$$\text{Sink Animal} <: \text{Sink Dog}$$

一个能够接收任意动物的写入口，可以安全地交给只会写入狗的程序使用。反方向并不安全：若把只能接收狗的入口当作能够接收任意动物的入口，使用者便可能向其中写入一只猫。**Sink** 中的类型参数因而反转了原来的子类型方向。

引用则可以降为其中任一种能力：

$$\frac{}{\text{Ref } T_1 <: \text{Source } T_1} \text{ S-REFSOURCE} \quad \frac{}{\text{Ref } T_1 <: \text{Sink } T_1} \text{ S-REFSINK}$$

信道

Pict 等并发语言中的信道类型也区分读写权限，并采用与 **Source**、**Sink** 相同的子类型规则。从类型角度看，通信信道与引用单元类似：程序既可以向信道写入（发送）值，也可以从信道读取（接收）值。静态分析通常很难确定某次读取会接收到哪次写入的值，因此，保证安全的一种简单办法是要求同一信道上传递的所有值具有相同的类型。

两种信道权限的变型方向也与引用相同。若某个信道允许写入 S 型值，而 $T <: S$ ，那么它可以安全地交给只会写入 T 型值的程序使用，因此输出信道类型构造子是逆变的。若从某个信道读出的值保证具有类型 S ，而 $S <: T$ ，那么这些值也可以作为 T 型值读取，因此输入信道类型构造子是协变的。关于这种信道类型的进一步讨论，见 Pierce 与 Turner (2000) 以及 Pierce 与 Sangiorgi (1993)。

基本类型

在基本类型较多的语言中，常会在它们之间加入原始子类型关系。例如，若语言用数字 1 和 0 表示布尔值，可以公开实现细节并加入公理 $\text{Bool} <: \text{Nat}$ 。此后可写紧凑的 $5 * b$ ，而不必写 $\text{if } b \text{ then } 5 \text{ else } 0$ 。

15.6 子类型化的强制转换语义

到目前为止，子类型化在语义上没有改变程序的求值方式，只让项的赋型更灵活。这种解释简单自然，但可能给数值计算和记录字段访问带来高性能实现无法接受的开销。本节概述另一种强制转换语义（*coercion semantics*），并讨论它引出的新问题；读者可以跳过本节。

子集语义的性能问题

设加入 $\text{Int} <: \text{Float}$ ，允许把整数直接用于浮点运算。例如，原本需要写成 $4.5 + \text{intToFloat}(6)$ 的表达式，现在可以简写为 $4.5 + 6$ 。子集语义要求整数值集合确实是浮点值集合的子集。但真实机器通常用补码表示整数，用尾数、指数和符号位表示浮点数，还为 NaN（非数）等特殊值保留编码；两者的具体表示完全不同。

一种兼容子集语义的办法，是让所有数都采用带标签（或装箱）的共同表示：机器整数附带整数标签，机器浮点数附带另一种标签。标签既可单独占用一个头部机器字，也可放在保存实际整数那个机器字的高位。此时 Float 描述所有带标签的数，包括浮点数和整数； Int 只描述带整数标签的数。

这种方案并不荒唐，许多现代语言实现确实采用类似策略，标签位或标签字也能为垃圾回收提供信息。代价是每个基本数值操作都要先检查实参标签，用若干指令拆箱出机器数，执行真正的运算，再用若干指令把结果重新装箱。编译器优化可以消除部分开销，但即使采用目前最好的技术，在图形和科学计算等数值密集程序中仍会显著降低性能。

记录排列规则会造成另一种问题。简单的字段投影规则

$$\frac{}{\{l_i = v_i\}^{i \in 1..n}.l_j \longrightarrow v_j} \text{E-PROJRCd}$$

可以读作“在记录的标签中搜索 l_j ，并返回与它关联的值 v_j ”。真实实现当然不愿在运行时线性扫描整条记录。若语言没有子类型化，或者有子类型化却没有排列规则，而标签 l_j 在记录类型中列为第三个字段，编译器就能静态确定：该类型的每个运行时值也把 l_j 放在第三个字段。因此运行时完全不必查看标签；甚至可以删掉标签，把记录编译成元组。访问 l_j 时，只需通过指向记录起始处的寄存器间接读取，再加上固定的三个机器字偏移。

排列规则破坏了这种技术。即使某个记录值属于一个把 l_j 列在第三位的类型，我们也无法据此判断它在实际记录中存放在哪里。优化和运行时技巧可以缓解问题，但一般而言，字段投影仍可能需要某种运行时搜索。⁴

强制转换语义

另一种语义把子类型化“编译掉”，以运行时强制转换取而代之。把 Int 提升到 Float 时，运行时真正把机器整数改为机器浮点表示；使用记录排列规则时，则生成实际重排字段的代码。这样，基本数值操作和字段访问无需承担拆箱或搜索开销。

⁴类似情况也出现在允许对象子类型排列的语言中访问对象字段和方法时。因此，Java 类之间的子类型化只允许在末尾添加字段。接口之间、类与接口之间则允许排列——否则接口几乎失去用途——Java 手册也明确提醒，接口方法查找通常比类方法查找慢。

形式上, 强制转换语义把带子类型化的源语言项翻译到不带子类型化的低层语言。这里以带记录的 $\lambda_{<}$ 为源语言, 以带记录和 **Unit** 的纯简单类型 **lambda** 演算为目标语言; **Unit** 用来解释 **Top**。编译包含类型、子类型推导和类型推导三种翻译, 都写作 $\llbracket - \rrbracket$, 上下文会说明正在翻译哪一种对象。

类型翻译为:

$$\begin{aligned}\llbracket \text{Top} \rrbracket &= \text{Unit}, \\ \llbracket T_1 \rightarrow T_2 \rrbracket &= \llbracket T_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket, \\ \llbracket \{l_i : T_i\}^{i \in 1..n} \rrbracket &= \{l_i : \llbracket T_i \rrbracket\}^{i \in 1..n}\end{aligned}$$

例如, $\llbracket \text{Top} \rightarrow \{a : \text{Top}, b : \text{Top}\} \rrbracket = \text{Unit} \rightarrow \{a : \text{Unit}, b : \text{Unit}\}$ 。

翻译项时必须知道类型推导在哪里使用了包容规则, 因为这些位置要插入运行时强制转换。同样, 要生成从 S 到 T 的强制转换函数, 仅知道 $S <: T$ 还不够, 还必须知道这个判断怎样推导出来。因此, 翻译以推导本身为输入。写 $\mathcal{C} :: S <: T$, 表示 $\mathcal{C}$ 是结论为该判断的子类型推导树; 写 $\mathcal{D} :: \Gamma \vdash t : T$, 表示 $\mathcal{D}$ 是相应类型推导树。

设 $\mathcal{C}$ 是一棵证明 $S <: T$ 的子类型推导树。翻译时, 我们把这棵推导树编译成目标语言中的一个显式转换函数 $\llbracket \mathcal{C} \rrbracket$ 。该函数接收一个 $\llbracket S \rrbracket$ 型值, 返回一个 $\llbracket T \rrbracket$ 型值。具体生成什么函数, 取决于推导树 $\mathcal{C}$ 最后一步所用的子类型规则:

$$\begin{aligned}\llbracket \text{S-REFL}_T \rrbracket &= \lambda x : \llbracket T \rrbracket. x \\ \llbracket \text{S-TOP}_S \rrbracket &= \lambda x : \llbracket S \rrbracket. \text{unit} \\ \llbracket \text{S-TRANS}(\mathcal{C}_1, \mathcal{C}_2) \rrbracket &= \lambda x : \llbracket S \rrbracket. \llbracket \mathcal{C}_2 \rrbracket(\llbracket \mathcal{C}_1 \rrbracket x) \\ \llbracket \text{S-ARROW}(\mathcal{C}_1, \mathcal{C}_2) \rrbracket &= \lambda f : \llbracket S_1 \rightarrow S_2 \rrbracket. \lambda x : \llbracket T_1 \rrbracket. \llbracket \mathcal{C}_2 \rrbracket(f(\llbracket \mathcal{C}_1 \rrbracket x)) \\ \llbracket \text{S-RCDWIDTH} \rrbracket &= \lambda r : \{l_i : \llbracket T_i \rrbracket\}^{i \in 1..n+m}. \{l_i = r.l_i\}^{i \in 1..n} \\ \llbracket \text{S-RCDDEPTH}(\mathcal{C}_i) \rrbracket &= \lambda r : \{l_i : \llbracket S_i \rrbracket\}^{i \in 1..n}. \{l_i = \llbracket \mathcal{C}_i \rrbracket(r.l_i)\}^{i \in 1..n} \\ \llbracket \text{S-RCDPERM} \rrbracket &= \lambda r : \{k_j : \llbracket S_j \rrbracket\}^{j \in 1..n}. \{l_i = r.l_i\}^{i \in 1..n}\end{aligned}$$

引理 15.6.1. 若 $\mathcal{C}$ 是 $S <: T$ 的一棵合法推导树, 那么由它生成的强制转换 $\llbracket \mathcal{C} \rrbracket$ 也是目标语言中的良类型函数: 它接收一个 $\llbracket S \rrbracket$ 型的值, 返回一个 $\llbracket T \rrbracket$ 型的值。即

$$\vdash \llbracket \mathcal{C} \rrbracket : \llbracket S \rrbracket \rightarrow \llbracket T \rrbracket$$

证明. 对 $\mathcal{C}$ 作直接归纳。 □

接下来定义如何根据类型推导树生成目标语言程序。若 $\mathcal{D} :: \Gamma \vdash t : T$, 则 $\llbracket \mathcal{D} \rrbracket$ 表示根据整棵推导树 $\mathcal{D}$ 生成的目标语言项。普通类型规则只需递归翻译程序的各个子项: 变量仍是自身, 抽象、应用、记录和投影仍保留原有结构。关键是 **T-SUB**: 源程序中没有写出的子类型

转换会记录在这个推导节点中，因此翻译到不带子类型化的目标语言时，必须在这里插入相应的显式强制转换：

$$\begin{aligned}
 \llbracket \text{T-VAR}_x \rrbracket &= x \\
 \llbracket \text{T-ABS}(\mathcal{D}_2) \rrbracket &= \lambda x : \llbracket T_1 \rrbracket. \llbracket \mathcal{D}_2 \rrbracket \\
 \llbracket \text{T-APP}(\mathcal{D}_1, \mathcal{D}_2) \rrbracket &= \llbracket \mathcal{D}_1 \rrbracket \llbracket \mathcal{D}_2 \rrbracket \\
 \llbracket \text{T-RCD}(\mathcal{D}_i) \rrbracket &= \{l_i = \llbracket \mathcal{D}_i \rrbracket\}^{i \in 1..n} \\
 \llbracket \text{T-PROJ}(\mathcal{D}_1) \rrbracket &= \llbracket \mathcal{D}_1 \rrbracket.l_j \\
 \llbracket \text{T-SUB}(\mathcal{D}, \mathcal{C}) \rrbracket &= \llbracket \mathcal{C} \rrbracket \llbracket \mathcal{D} \rrbracket
 \end{aligned}$$

这项翻译常称为 Penn 翻译 (*Penn translation*)，名称来自最早研究它的宾夕法尼亚大学团队 (Breazu-Tannen、Coquand、Gunter 与 Scedrov, 1991)。

定理 15.6.2. 若推导树 $\mathcal{D}$ 证明源语言中的项 t 在上下文 Γ 下具有类型 T ，那么根据 $\mathcal{D}$ 生成的目标语言项 $\llbracket \mathcal{D} \rrbracket$ ，在翻译后的上下文 $\llbracket \Gamma \rrbracket$ 下也具有翻译后的类型 $\llbracket T \rrbracket$ 。即

$$\mathcal{D} :: \Gamma \vdash t : T \implies \llbracket \Gamma \rrbracket \vdash \llbracket \mathcal{D} \rrbracket : \llbracket T \rrbracket$$

换言之，这项翻译会保持程序的类型正确性。其中，上下文中的绑定分别翻译： $\llbracket \emptyset \rrbracket = \emptyset$ ，且 $\llbracket \Gamma, x : T \rrbracket = \llbracket \Gamma \rrbracket, x : \llbracket T \rrbracket$ 。

证明. 对 $\mathcal{D}$ 作直接归纳；T-SUB 情形使用 [引理15.6.1](#)。□

有了这些翻译，可以不再给高级语言直接定义求值规则。求值一个项时，先用高级语言的类型规则和子类型规则检查它，再把类型推导翻译到低层目标语言，最后使用目标语言的求值关系。某些高性能子类型语言实现确实采用这种策略 (League、Shao 与 Trifonov, 1999; League、Trifonov 与 Shao, 2001)。

练习 15.6.3 (★★★, $\rightarrow$). 修改上述翻译，以带元组而非记录的简单类型 lambda 演算作为目标语言，并检查 [定理15.6.2](#) 仍然成立。

[查看练习 15.6.3 的译者参考解答](#)

一致性

强制转换语义还要避免一个陷阱。设语言含有 Int、Bool、Float 和 String，并提供如下基本转换：

$$\begin{aligned}
 \llbracket \text{Bool} <: \text{Int} \rrbracket &= \lambda b : \text{Bool}. \text{if } b \text{ then } 1 \text{ else } 0 \\
 \llbracket \text{Int} <: \text{String} \rrbracket &= \text{intToString} \\
 \llbracket \text{Bool} <: \text{Float} \rrbracket &= \lambda b : \text{Bool}. \text{if } b \text{ then } 1.0 \text{ else } 0.0 \\
 \llbracket \text{Float} <: \text{String} \rrbracket &= \text{floatToString}
 \end{aligned}$$

设 $\text{intToString}(1) = "1"$ ，而 $\text{floatToString}(1.0) = "1.000"$ 。项 $(\lambda x : \text{String}. x) \text{ true}$ 可以沿两条路径赋型：把 Bool 先提升到 Int 再到 String，或先提升到 Float 再到 String。两棵类型推导树翻译出的程序分别得到 "1" 和 "1.000"。程序员只写项，不写推导；若编译器内部选择的推导能改变程序行为，语言的行为便无法由程序员预测。

为避免这一问题，翻译必须满足一致性 (*coherence*)。

定义 15.6.4. 从一种语言的类型推导到另一种语言项的翻译 $\llbracket - \rrbracket$ 称为一致的, 如果对任意两棵具有同一结论 $\Gamma \vdash t : T$ 的推导 $\mathcal{D}_1$ 和 $\mathcal{D}_2$, 目标语言项 $\llbracket \mathcal{D}_1 \rrbracket$ 与 $\llbracket \mathcal{D}_2 \rrbracket$ 在行为上等价。

没有基本类型时, 上面的翻译是一致的。加入前述基本类型公理后, 只需规定

$$\text{floatToString}(0.0) = "0", \quad \text{floatToString}(1.0) = "1"$$

即可恢复这个例子中的一致性。对复杂语言证明一致性可能相当困难; 相关研究见 Reynolds (1980、1991)、Breazu-Tannen 等 (1991) 以及 Curien 与 Ghelli (1992)。

15.7 交类型与并类型

在类型语言中加入交运算, 可以显著增强子类型关系。交类型由 Coppo、Dezani、Sallé 和 Pottinger 提出 (Coppo 与 Dezani-Ciancaglini, 1978; Coppo、Dezani-Ciancaglini 与 Sallé, 1979; Pottinger, 1980)。Reynolds (1988、1998b)、Hindley (1992) 和 Pierce (1991b) 给出了较易入门的介绍。 $T_1 \wedge T_2$ 的居留元同时属于 T_1 与 T_2 , 所以它是序理论中两者的交, 即最大下界。这由三条规则表达:

$$\begin{array}{c} T_1 \wedge T_2 <: T_1 \quad (\text{S-INTER1}) \quad T_1 \wedge T_2 <: T_2 \quad (\text{S-INTER2}) \\[1em] \frac{S <: T_1 \quad S <: T_2}{S <: T_1 \wedge T_2} \text{S-INTER3} \end{array}$$

交类型与箭头类型之间还满足下面的子类型关系:

$$(S \rightarrow T_1) \wedge (S \rightarrow T_2) <: S \rightarrow (T_1 \wedge T_2) \quad (\text{S-INTER4})$$

如果一项同时具有函数类型 $S \rightarrow T_1$ 和 $S \rightarrow T_2$, 那么给它一个 S 型实参后, 结果同时具有 T_1 和 T_2 。

下面这个事实说明了交类型的能力: 在按名调用的、带子类型化和交类型的简单类型 lambda 演算中, 可以获得类型的无类型 lambda 项恰好就是可规范化项——也就是说, 一个无类型 lambda 项能够获得某种类型, 当且仅当其求值会终止。由此立即可知, 带交类型演算的类型重构问题 (第二十二章) 不可判定。

译者注: 这里的关键是, 交类型把类型重构与终止性连成了同一个判定问题。类型重构接收一个没有类型标注的 lambda 项, 并尝试为它找到一种类型; 若不存在这样的类型, 也应给出否定答案。普通简单类型系统只能保证“能够找到类型的项一定会终止”, 反方向并不成立。例如, $\lambda x.xx$ 已经是正规形式, 因为 xx 左侧的 x 只是变量, 不会触发 β 规约; 但同一个 x 在这里既要作为函数又要作为参数, 普通简单类型系统无法为它找到一种同时满足两种用法的类型。交类型允许令 $x : (A \rightarrow B) \wedge A$, 于是整个抽象可获得类型 $((A \rightarrow B) \wedge A) \rightarrow B$ 。更一般地, 书中引用的结果表明:

$$\text{一个无类型 lambda 项能够获得交类型} \iff \text{它的求值会终止}$$

因此, 若存在一个总能完成的交类型重构算法, 只需观察它能否找到类型, 就能判断任意无类型 lambda 项的求值是否会终止。终止性不可判定, 所以一般交类型的类型重构也不可判定。一般交类型强大的描述能力与这里的不可判定性正是同一件事的两个方面。

在实践中，交类型支持有限重载。例如，可以把

$$(\text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}) \wedge (\text{Float} \rightarrow \text{Float} \rightarrow \text{Float})$$

赋给既能做自然数加法又能做浮点加法的运算符；运行时可按参数标签选择指令。交类型的能力也带来棘手的语言设计问题。迄今只有一种完整规模语言全面采用一般交类型，即 Forsythe (Reynolds, 1988)；受限的精细化类型 (*refinement type*) 可能更易管理 (Freeman 与 Pfenning, 1991; Pfenning, 1993b; Davies, 1997)。

译者注：“精细化类型” (*refinement type*，也常译作“细化类型”) 在已有类型之上描述更精确的静态性质。例如，可以把一般列表类型进一步区分为非空列表或单元素列表，使类型检查器能够排除只会在空列表上发生的错误。不同系统表达这些性质的方式并不相同；现代系统常见 $\{x : T \mid P(x)\}$ 这样的写法，表示满足条件 P 的 T 型值。本段所引 Freeman-Pfenning 系统只为原本已经通过 ML 类型检查的程序增加由程序员声明的精细化信息，从而保留类型推断的可判定性；这正是它比一般交类型更易于实现和使用之处。

对偶的并类型 (*union type*) $T_1 \vee T_2$ 也很有用。它与和类型和带标签变体不同： $T_1 \vee T_2$ 只是 T_1 与 T_2 的普通值集合之并，不增加标明来源的标签，所以 $\text{Nat} \vee \text{Nat}$ 只是 Nat 的另一个名字。无标签并类型长期用于程序分析 (Palsberg 与 Pavlopoulou, 1998)，却只见于少数程序语言，其中包括 Algol 68 (参见 van Wijngaarden 等, 1975)。近来，它还被用于 XML 等半结构化数据库格式的类型系统 (Buneman 与 Pierce, 1998; Hosoya、Vouillon 与 Pierce, 2001)。

不交并类型 (即和类型与带标签变体) 同这里的无标签并类型相比，形式上的主要区别是：后者没有 `case` 构造。若只知道 $v : T_1 \vee T_2$ ，唯一安全的操作是对 T_1 和 T_2 都适用的操作。例如，若二者都是记录，只能投影共有字段。C 的无标签 `union` 允许执行只适用于其中一种类型的操作，即使其中实际存放的值来自另一种类型；这种做法忽略了上述限制，因而会破坏类型安全。

15.8 注记

程序语言中的子类型化思想可追溯到 20 世纪 60 年代的 Simula 及其相关语言。相关介绍见 Birtwistle、Dahl、Myhrhaug 与 Nygaard (1979)。最早的形式化处理来自 Reynolds (1980) 和 Cardelli (1984)。

记录的类型规则，尤其是子类型规则，比此前多数规则都重：它们或含有数目可变的前提——每个字段一个——或要额外处理字段下标的排列。其他写法也有类似复杂度，或者依靠省略号等非正式约定回避它。Cardelli 与 Mitchell (1991) 为此发展了“记录上的运算”演算，把构造多字段记录的宏操作拆成空记录值与逐次加入一个字段的 basic 操作；在同一框架中还可加入原地字段更新和记录拼接 (Harper 与 Pierce, 1991)。带参数多态时，这些运算的类型规则会相当微妙，多数语言设计者因此仍采用普通记录。不过，这套系统在概念上仍是重要里程碑。Wand (1987、1988、1989b)、Rémy (1989、1990、1992) 等人发展的另一条路线以行变量多态为基础，后来成为 OCaml 面向对象特性的基础 (Rémy 与 Vouillon, 1998; Vouillon, 2000)。

类型理论要解决的根本问题，是保证程序有意义。类型理论带来的根本问题，是有意义的程序未必能被赋予类型。对更丰富类型系统的追求，正源于这层张力。

The fundamental problem addressed by a type theory is to insure that programs have meaning.

The fundamental problem caused by a type theory is that meaningful programs may not have meanings ascribed to them.

The quest for richer type systems results from this tension.

——Mark Manasse

第十六章 子类型化的元理论

概述

上一章定义的带子类型化简单类型 lambda 演算还不适合直接实现。与此前的演算不同，这套规则不是语法导向的 (*syntax directed*)：给定一个待检查的结论时，无法总是唯一确定应采用哪条规则，也无法总是由结论直接确定其前提。因此，不能直接把这些推导规则倒着执行，将其作为类型检查算法。主要问题来自类型关系中的包容规则 T-SUB 和子类型关系中的传递规则 S-TRANS。

T-SUB 的结论只把项写成不带任何结构限制的元变量 t ：

$$\frac{\Gamma \vdash t : S \quad S <: T}{\Gamma \vdash t : T} \text{ T-SUB}$$

其他类型规则都要求项具有某种具体形式：T-ABS 只适用于 lambda 抽象，T-VAR 只适用于变量，等等。T-SUB 却能用于任何项。因此，给定一个待计算类型的项 t 时，既可能选择 T-SUB，也可能选择结论与 t 的句法形式相符的那条规则。

S-TRANS 也有同样的问题：它的结论与所有其他子类型规则的结论重叠。

$$\frac{S <: U \quad U <: T}{S <: T} \text{ S-TRANS}$$

S 与 T 都是不带结构限制的元变量，所以任何子类型判断都可能以 S-TRANS 收尾。若把规则机械地从结论向前提执行，算法无法知道应该尝试传递规则，还是应该尝试另一条与输入类型结构相符的规则。

S-TRANS 还有第二个问题：两个前提都出现了结论中没有的 U 。从结论反推前提时，必须先猜一个 U ，再检查 $S <: U$ 和 $U <: T$ 。可供猜测的类型有无穷多个，这种搜索方式无法成为实用算法。

S-REFL 的结论也与其他子类型规则重叠，不过问题较轻：它没有前提；输入两端同时可以立即成功。即便如此，它仍使规则不是语法导向的。

解决方法是用两套语法导向的关系替换普通的、也称 声明式 (*declarative*) 的子类型关系和类型关系：新关系分别称为 算法子类型关系 (*algorithmic subtyping relation*) 和 算法类型关系 (*algorithmic typing relation*)。随后证明两种表述实际上等价：算法规则可导出的子类型判断恰好也是声明式规则可导出的判断；一个项能由算法规则赋型，当且仅当它能由声明式规则赋型。

§16.1构造算法子类型关系, §16.2构造算法类型关系。§16.3加入图8-1中的布尔值与条件式。为条件式赋型时, 还需要计算两个分支类型的最小公共超类型; 这个类型称为二者的并。§16.4讨论最小类型 Bot。¹

16.1 算法子类型关系

带子类型化语言的实现必须能够检查一个类型是否为另一个类型的子类型。例如, 类型检查器遇到应用 $t_1 t_2$, 并算出 $t_1 : T \rightarrow U$ 、 $t_2 : S$ 时, 需要判断 $S <: T$ 能否由图15-1和15-3中的规则导出。

子类型检查器实际判断的是另一个关系

$$\vdash_A S <: T$$

读作“ S 在算法关系下是 T 的子类型”。算法可以根据 S 与 T 的类型结构选择规则, 再递归检查它们的组成部分。虽然算法关系使用的规则与声明式关系不同, 但二者给出的判断结果完全相同: 声明式关系能够证明 $S <: T$, 当且仅当算法关系也能判定 $S <: T$ 。两者最明显的区别是, 算法关系没有 S-TRANS 和 S-REFL。

不过, 在删去传递规则之前, 先要重组记录子类型规则。如 §15.2 所见, 宽度、深度和排列推导必须借助传递性拼接。将三条规则合并为一条宏规则:

$$\frac{\{l_i \mid i \in 1..n\} \subseteq \{k_j \mid j \in 1..m\} \quad k_j = l_i \implies S_j <: T_i}{\{k_j : S_j\}^{j \in 1..m} <: \{l_i : T_i\}^{i \in 1..n}} \text{ S-RCD}$$

第二个前提表示: 对目标类型中的每个标签 l_i , 都在源类型中找到同名字段 k_j , 并检查对应字段类型 $S_j <: T_i$ 。

引理 16.1.1. 若 $S <: T$ 能由包含 S-RCDDEPTH、S-RCDWIDTH 和 S-RCDPERM、但不含 S-RCD 的规则导出, 那么也能由使用 S-RCD、但不使用前三条规则的系统导出; 反之亦然。

证明. 对推导作直接归纳。□

引理16.1.1允许用 S-RCD 取代三条分散的记录规则。图16-1汇总重组后的系统。

¹本章研究图15-1和15-3中的演算。原书相应 OCaml 实现为 `rcdsub`; §16.3 加入布尔值与条件式, 相应实现为 `joinsub`; §16.4加入 Bot, 相应实现为 `bot`。

<:

扩展 图15-1和15-3

$$\begin{array}{c}
\frac{}{S <: S} \text{S-REFL} \\
\\
\frac{S <: U \quad U <: T}{S <: T} \text{S-TRANS} \\
\\
\frac{}{S <: \text{Top}} \text{S-TOP} \\
\\
\frac{T_1 <: S_1 \quad S_2 <: T_2}{S_1 \rightarrow S_2 <: T_1 \rightarrow T_2} \text{S-ARROW} \\
\\
\frac{\{l_i \mid i \in 1..n\} \subseteq \{k_j \mid j \in 1..m\} \quad k_j = l_i \implies S_j <: T_i}{\{k_j : S_j\}^{j \in 1..m} <: \{l_i : T_i\}^{i \in 1..n}} \text{S-RCD}
\end{array}$$

图 16-1: 带记录的子类型关系（紧凑版本）

下面证明，在图16-1的系统中，自反规则和传递规则并非必需。

引理 16.1.2.

1. 对每个类型 S ，不用 $S\text{-REFL}$ 也能导出 $S <: S$ 。
2. 若判断 $S <: T$ 能由图16-1中的规则导出，那么还存在一份不使用 $S\text{-TRANS}$ 的推导，其结论仍为 $S <: T$ 。

证明. 练习（推荐，***）。

□

[查看原书附录 A 中的 16.1.2 解答](#)

练习 16.1.3 (★). 若加入类型 `Bool`，这两项性质需要怎样修改？

[查看原书附录 A 中的 16.1.3 解答](#)

定义 16.1.4. 算法子类型关系是由图16-2中的规则闭合生成的最小类型关系。

$\vdash_A S <: T$

算法子类型关系

$$\begin{array}{c}
\frac{}{\vdash_A S <: \text{Top}} \text{SA-Top} \\
\\
\frac{\{l_i \mid i \in 1..n\} \subseteq \{k_j \mid j \in 1..m\} \quad k_j = l_i \implies \vdash_A S_j <: T_i}{\vdash_A \{k_j : S_j\}^{j \in 1..m} <: \{l_i : T_i\}^{i \in 1..n}} \text{SA-RCD} \\
\\
\frac{\vdash_A T_1 <: S_1 \quad \vdash_A S_2 <: T_2}{\vdash_A S_1 \rightarrow S_2 <: T_1 \rightarrow T_2} \text{SA-ARROW}
\end{array}$$

图 16-2: 算法子类型关系

算法规则的可靠性是指：它导出的每个判断也能由声明式规则导出；换言之，算法不会错误接受声明式规则无法证明的判断。算法规则的完备性是指：声明式规则能够导出的每个判断，算法规则也能导出；换言之，声明式系统认可的判断不会被算法遗漏。

命题 16.1.5 (可靠性与完备性).

$$S <: T \iff \vdash_A S <: T$$

证明. 两个方向都对推导作归纳，分别使用[引理16.1.1](#)和[16.1.2](#)。

□

现在可以把这些算法规则逐条转写为子类型检查算法：

```

subtype(S, T) =
  if T = Top then true
  else if S = S1 -> S2 and T = T1 -> T2 then
    subtype(T1, S1) and subtype(S2, T2)
  else if S = {kj:Sj} and T = {li:Ti} then
    every target label li occurs as some kj
    and subtype(Sj, Ti) for every matching field
  else false

```

具体的 OCaml 实现见[第十七章](#)。

最后还要证明算法对每一对输入都在有限时间内返回 `true` 或 `false`，即它是全函数。

命题 16.1.6 (终止性). 若 $\vdash_A S <: T$ 可导出，则 `subtype(S, T)` 返回 `true`；否则返回 `false`。

证明. 首先，若算法子类型判断 $\vdash_A S <: T$ 可导出，则对这份推导作归纳，可以验证 `subtype(S, T)` 会沿着相应的算法分支执行并返回 `true`。反过来，若 `subtype(S, T)` 返回 `true`，根据它所经过的分支递归构造推导，也能得到 $\vdash_A S <: T$ 。

现在考虑该判断不可导出的情况。根据上面的结论，算法不可能返回 `true`；要证明它一定返回 `false`，还需排除无限递归。把类型的大小理解为其语法树中构造节点的数量。算法每次递归调用时，传入的都是原类型的组成部分，因此两个输入类型的大小之和都会严格减小。这个和始终是正整数，所以不可能存在无限的递归调用序列。算法必定终止；既然不能返回 `true`，就只能返回 `false`。

□

命题16.1.5和16.1.6共同说明 `subtype` 是声明式子类型关系的判定过程 (*decision procedure*)。

也可以一开始就把算法关系当作子类型关系的正式定义, 完全不提声明式版本。不过, 这并不能省去多少工作: 为了证明依赖子类型关系的类型系统具有良好性质, 仍需证明算法子类型关系具有自反性和传递性, 工作量与本节相近。实际语言定义有时确实直接采用算法式类型规则; 第十九章就是一个例子。

16.2 算法类型关系

处理好子类型关系后, 还要以同样方式处理类型关系。如本章概述所见, 唯一不语法导向的类型规则是 T-SUB。它不能简单删掉; 必须先找出包容规则在类型推导中不可替代的用途, 再把这种用途嵌入其他类型规则。

在函数应用中, 包容规则负责连接函数期望的参数类型与实参的实际类型。例如

$$(\lambda r : \{x : \text{Nat}\}. r.x) \{x = 0, y = 1\}$$

离开包容规则便无法赋型。

稍微出人意料的是, 这也是包容规则唯一不可替代的用途。在其他位置, 可以重排类型推导, 把 T-SUB 的使用逐步推迟到更靠近最终结论的位置。

译者注: 这里重排 T-SUB, 是为了从声明式类型规则中提取可直接执行的类型检查算法。T-SUB 可以在推导的任意位置把已经得到的类型提升为任意超类型; 如果它散落在推导各处, 类型检查器就无法只根据项的语法确定下一步。把它尽量移到推导末端后, 末端的 T-SUB 只会把已经算出的类型换成信息更少的超类型, 算法可以省略这一步, 保留更小、更精确的类型。唯一无法完全消除的情形是函数应用中实参类型与参数类型的匹配; 这项必要的子类型检查随后会被吸收到增强后的应用规则 TA-APP 中。下面的推导重排正是在说明, 经过这样的整理, 原来能够赋型的项仍然能够通过类型检查。

先看以 T-ABS 结束、而其直接子推导以 T-SUB 结束的推导:

$$\frac{\frac{\Gamma, x : S_1 \vdash s_2 : S_2 \quad S_2 <: T_2}{\Gamma, x : S_1 \vdash s_2 : T_2} \text{ T-SUB}}{\Gamma \vdash \lambda x : S_1. s_2 : S_1 \rightarrow T_2} \text{ T-ABS}$$

可以把包容规则移到抽象规则之后:

$$\frac{\frac{\Gamma, x : S_1 \vdash s_2 : S_2}{\Gamma \vdash \lambda x : S_1. s_2 : S_1 \rightarrow S_2} \text{ T-ABS} \quad \frac{\frac{\text{S-REFL}}{S_1 <: S_1} \quad S_2 <: T_2}{S_1 \rightarrow S_2 <: S_1 \rightarrow T_2} \text{ S-ARROW}}{\Gamma \vdash \lambda x : S_1. s_2 : S_1 \rightarrow T_2} \text{ T-SUB}$$

应用规则有两个直接子推导，其中任意一个都可能以 T-SUB 结束。先看包容规则出现在左侧函数子推导末尾的情形。重排前的推导为：

$$\begin{array}{c}
 \frac{\Gamma \vdash s_1 : S_{11} \rightarrow S_{12} \quad \frac{T_{11} <: S_{11} \quad S_{12} <: T_{12}}{S_{11} \rightarrow S_{12} <: T_{11} \rightarrow T_{12}} \text{S-ARROW}}{\Gamma \vdash s_1 : T_{11} \rightarrow T_{12}} \text{T-SUB} \quad \Gamma \vdash s_2 : T_{11} \\
 \hline
 \Gamma \vdash s_1 s_2 : T_{12} \quad \text{T-APP}
 \end{array}$$

这里先用 T-SUB 把函数类型从 $S_{11} \rightarrow S_{12}$ 提升为 $T_{11} \rightarrow T_{12}$ ，再应用于 T_{11} 型实参。子类型反演给出 $T_{11} <: S_{11}$ 和 $S_{12} <: T_{12}$ 。重排后，前一项用来把实参类型提升到 S_{11} ，后一项则在应用完成后把结果从 S_{12} 提升到 T_{12} ：

$$\begin{array}{c}
 \frac{\Gamma \vdash s_2 : T_{11} \quad T_{11} <: S_{11}}{\Gamma \vdash s_2 : S_{11}} \text{T-SUB} \\
 \frac{\Gamma \vdash s_1 : S_{11} \rightarrow S_{12} \quad \Gamma \vdash s_2 : S_{11}}{\Gamma \vdash s_1 s_2 : S_{12}} \text{T-APP} \quad S_{12} <: T_{12} \\
 \hline
 \Gamma \vdash s_1 s_2 : T_{12} \quad \text{T-SUB}
 \end{array}$$

这里发生了两种移动。原来 S-ARROW 右侧的前提 $S_{12} <: T_{12}$ 被下移到整个应用推导的末端，用来提升最终结果类型；原来 T-APP 右侧、用于推导实参 $\Gamma \vdash s_2 : T_{11}$ 的子推导，则被嵌入新的实参推导中，再由 $T_{11} <: S_{11}$ 把实参提升为函数真正接受的类型。

再看包容规则出现在 T-APP 的右侧实参子推导末尾的情形。重排前的推导为：

$$\begin{array}{c}
 \frac{\Gamma \vdash s_2 : T_2 \quad T_2 <: T_{11}}{\Gamma \vdash s_2 : T_{11}} \text{T-SUB} \\
 \frac{\Gamma \vdash s_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash s_2 : T_{11}}{\Gamma \vdash s_1 s_2 : T_{12}} \text{T-APP}
 \end{array}$$

若仍要移动右侧实参子推导末尾的这次 T-SUB，唯一可行的做法是把它转移到左侧函数子推导中。它与前一种重排的方向恰好相反：前面把“使函数与实参的类型相匹配”这项调整从函数一侧移到了实参一侧，这里则将它移回函数一侧。具体做法是利用参数类型的逆变性，把函数的参数类型从 T_{11} 改为实参原有的 T_2 ：

$$\begin{array}{c}
 \frac{\Gamma \vdash s_1 : T_{11} \rightarrow T_{12} \quad \frac{T_2 <: T_{11} \quad T_{12} <: T_{12}}{T_{11} \rightarrow T_{12} <: T_2 \rightarrow T_{12}} \text{S-REFL}}{\Gamma \vdash s_1 : T_2 \rightarrow T_{12}} \text{S-ARROW} \quad \Gamma \vdash s_2 : T_2 \\
 \hline
 \Gamma \vdash s_1 s_2 : T_{12} \quad \text{T-APP}
 \end{array}$$

由此可见，这次包容可以在应用的两个前提之间移动，却无法彻底消除：既可以把实参提升到函数要求的参数类型，也可以把函数调整为接受实参的实际类型。为了得到统一的标准形式，下面固定采用“提升实参类型”的写法，把这次包容保留在应用的右侧实参子推导末尾。

若一条 T-SUB 的直接子推导也以 T-SUB 结束，则两次使用可以合并。重排前的推导为：

$$\begin{array}{c}
 \frac{\Gamma \vdash s : S \quad S <: U}{\Gamma \vdash s : U} \text{T-SUB} \quad U <: T \\
 \hline
 \Gamma \vdash s : T \quad \text{T-SUB}
 \end{array}$$

由 $S <: U$ 和 $U <: T$ 先以 S-TRANS 得到 $S <: T$, 便可把相邻的两次 T-SUB 合并为一次:

$$\frac{\Gamma \vdash s : S \quad \frac{S <: U \quad U <: T}{S <: T} \text{S-TRANS}}{\Gamma \vdash s : T} \text{T-SUB}$$

练习 16.2.1 ($\star\star, \rightarrow$). 完成上述实验: 说明当 T-SUB 出现在 T-RCD 或 T-PROJ 之前时, 怎样作类似重排。

查看练习 16.2.1 的译者参考解答

固定上述重排方向后, 反复应用这些变换, 可以把任意类型推导整理成一种特殊形式: T-SUB 只会出现在应用的右侧实参子推导末尾, 以及整个推导的最后一步。删掉最末端那次包容仍会为同一项留下一个类型, 只是这个类型可能更小, 也就是携带更多信息。现在只有应用中的包容无法消除。把它吸收到应用规则中:

$$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_2 \quad T_2 <: T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}}$$

便能替换每个“应用之前先包容”的子推导, 并完全删去 T-SUB。这条增强后的应用规则仍然语法导向, 因为结论中的项明确是一项应用。

定义 16.2.2. 算法类型关系是由 图16-3 中的规则闭合生成的最小关系。写作 $\Gamma \vdash_A t : T$, 以区别于声明式判断 $\Gamma \vdash t : T$ 。

$\Gamma \vdash_A t : T$

算法类型关系

$$\begin{array}{c} \frac{x : T \in \Gamma}{\Gamma \vdash_A x : T} \text{TA-VAR} \\[10pt] \frac{\Gamma, x : T_1 \vdash_A t_2 : T_2}{\Gamma \vdash_A \lambda x : T_1. t_2 : T_1 \rightarrow T_2} \text{TA-ABS} \\[10pt] \frac{\Gamma \vdash_A t_1 : T_1 \quad T_1 = T_{11} \rightarrow T_{12} \quad \Gamma \vdash_A t_2 : T_2 \quad \vdash_A T_2 <: T_{11}}{\Gamma \vdash_A t_1 t_2 : T_{12}} \text{TA-APP} \\[10pt] \frac{\Gamma \vdash_A t_i : T_i \quad (\text{对每个 } i)}{\Gamma \vdash_A \{l_i = t_i\}^{i \in 1..n} : \{l_i : T_i\}^{i \in 1..n}} \text{TA-RCD} \\[10pt] \frac{\Gamma \vdash_A t_1 : R_1 \quad R_1 = \{l_i : T_i\}^{i \in 1..n}}{\Gamma \vdash_A t_1.l_j : T_j} \text{TA-PROJ} \end{array}$$

图 16-3: 算法类型关系

TA-APP 中的等式 $T_1 = T_{11} \rightarrow T_{12}$ 只是显式记录类型检查的执行顺序: 先计算 t_1 的类型 T_1 , 再检查它是否为箭头类型。也可以删掉该等式, 直接把第一项前提写成 $\Gamma \vdash_A t_1 : T_{11} \rightarrow T_{12}$ 。TA-PROJ 中的记录类型等式作用相同。应用规则的子类型前提使用算法关系; 由 命题16.1.5, 改写为声明式关系也没有区别。

练习 16.2.3 ($\star\star$, $\Rightarrow$). 找出项 s, t 和类型 S, T , 使 $\vdash_A s : S, \vdash_A t : T, s \longrightarrow^* t, T <: S$, 但 $S \not<: T$. 这说明项在求值后由算法规则算出的类型可能变得更小。

查看练习 16.2.3 的译者参考解答

译者注: 这里的 S 与 T 分别是两个不同项 s 与 t 的最小类型。求值后得到的项 t 可能暴露出更多类型信息, 所以它的最小类型 T 可以严格小于 s 的最小类型 S 。这并不破坏声明式关系的保持性: 由 $T <: S$ 和 T-SUB, 声明式关系仍可给 t 赋予原来的类型 S 。发生变化的只是算法为求值结果算出的最精确类型。

下面正式证明算法规则与声明式规则对应。上面的推导重排只用于说明思路; 把它发展成完整证明会很冗长, 直接对推导作归纳更简单。

定理 16.2.4 (可靠性). 若 $\Gamma \vdash_A t : T$, 则 $\Gamma \vdash t : T$ 。

证明. 对算法类型推导作直接归纳。 □

完备性的陈述稍有不同。声明式关系可为同一项赋予许多类型, 算法关系至多赋予一个类型, 所以不能简单把定理16.2.4倒过来。可以证明: 算法规则为 t 算出的唯一类型 S , 是声明式规则能够赋予 t 的任意类型 T 的子类型, 即 $S <: T$ 。再由可靠性定理, S 本身也是声明式关系认可的类型。因此, S 位于 t 的所有可赋类型之下, 正是 t 的最小类型 (*minimal type*)。

定理 16.2.5 (完备性, 也称最小类型定理). 若 $\Gamma \vdash t : T$, 则存在 $S <: T$, 使 $\Gamma \vdash_A t : S$ 。

证明. 练习 (推荐, $\star\star$)。 □

查看原书附录 A 中的 16.2.5 解答

练习 16.2.6 ($\star\star$). 若删去箭头子类型规则 S-ARROW, 但保留其余声明式子类型规则和类型规则, 系统是否仍具有最小类型性质? 若有, 请证明该性质; 若没有, 给出一个可赋型却没有最小类型的项。

查看原书附录 A 中的 16.2.6 解答

16.3 并与交

带子类型化的语言在检查条件式、case 表达式等多分支构造时, 还需要额外机制。声明式条件类型规则要求两个结果分支具有相同类型:

$$\frac{\Gamma \vdash t_1 : \text{Bool} \quad \Gamma \vdash t_2 : T \quad \Gamma \vdash t_3 : T}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \text{ T-IF}$$

有包容规则时, 让两个分支取得相同类型的方法可能很多。例如

if true then $\{x = \text{true}, y = \text{false}\}$ else $\{x = \text{false}, z = \text{true}\}$

具有类型 $\{x : \text{Bool}\}$ 。then 分支的最小类型为 $\{x : \text{Bool}, y : \text{Bool}\}$ ，else 分支的最小类型为 $\{x : \text{Bool}, z : \text{Bool}\}$ ，二者都能由包容规则提升到 $\{x : \text{Bool}\}$ 。整项也能具有 $\{x : \text{Top}\}$ 、 $\{\}$ 等任意公共超类型；其中最小的公共超类型是 $\{x : \text{Bool}\}$ 。

因此，计算条件式的最小类型时，应先计算两个分支的最小类型，再计算二者的最小公共超类型。偏序论通常把它称为两种类型的并。

定义 16.3.1. 若 $S <: J$ 、 $T <: J$ ，并且对每个 U ，从 $S <: U$ 和 $T <: U$ 都能推出 $J <: U$ ，则称 J 为 S, T 的并，写作 $S \sqcup T = J$ 。

对偶地，若 $M <: S$ 、 $M <: T$ ，并且对每个 L ，从 $L <: S$ 和 $L <: T$ 都能推出 $L <: M$ ，则称 M 为 S, T 的交，写作 $S \sqcap T = M$ 。

一种子类型关系不一定为每对类型都提供并或交。若每对 S, T 都有某个并，就说该关系具有并；若每对类型都有某个交，就说它具有交。

本节的子类型关系²具有并，却不具有交。例如， $\{\}$ 与 $\text{Top} \rightarrow \text{Top}$ 连公共子类型都没有，自然也没有最大公共子类型。不过，若只要求那些拥有共同下界的类型对存在交，这一较弱的性质仍然成立。若存在 L ，使 $L <: S$ 且 $L <: T$ ，就称 S, T 有共同下界 (*bounded below*)。若每对有共同下界的类型都有交，就说子类型关系具有有界交 (*bounded meets*)。

并和交不一定在语法上唯一。例如， $\{x : \text{Top}, y : \text{Top}\}$ 与 $\{y : \text{Top}, x : \text{Top}\}$ 都是 $\{x : \text{Top}, y : \text{Top}, z : \text{Top}\}$ 和 $\{x : \text{Top}, y : \text{Top}, w : \text{Top}\}$ 的并。不过，对于固定的一对类型，若 J_1 和 J_2 都是它们的并，则 $J_1 <: J_2$ 且 $J_2 <: J_1$ ；交也具有同样的性质。

命题 16.3.2 (并与有界交的存在性)。

1. 对每对类型 S, T ，都存在 J ，使 $S \sqcup T = J$ 。
2. 对每对具有公共子类型的 S, T ，都存在 M ，使 $S \sqcap T = M$ 。

证明. 练习 (推荐, ★★)。

□

查看原书附录 A 中的 16.3.2 解答

利用并运算，可以给条件式写出算法类型规则：

$$\frac{\Gamma \vdash_A t_1 : T_1 \quad T_1 = \text{Bool} \quad \Gamma \vdash_A t_2 : T_2 \quad \Gamma \vdash_A t_3 : T_3 \quad T_2 \sqcup T_3 = T}{\Gamma \vdash_A \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \text{TA-If}$$

练习 16.3.3 (★★). `if true then false else {}` 的最小类型是什么？这是我们想要的结果吗？

查看原书附录 A 中的 16.3.3 解答

练习 16.3.4 (★★★). 把计算并与交的算法扩展到§15.5所述那类带引用的命令式语言中，是否容易？若再把不变的 Ref 分解为协变的 Source 与逆变的 Sink，情况如何？

查看原书附录 A 中的 16.3.4 解答

²即图15-1和15-3定义的关系，再加入 `Bool`。没有为 `Bool` 增加声明式子类型规则，所以它唯一的严格超类型是 `Top`。

16.4 算法类型关系与 Bot 类型

若子类型关系加入最小类型 Bot，子类型算法和类型算法也要稍作扩展。算法子类型关系直接加入以下规则：

$$\frac{}{\vdash_A \text{Bot} <: T} \text{SA-BOT}$$

算法类型关系还要加入两条稍微特殊的规则：

$$\frac{\Gamma \vdash_A t_1 : T_1 \quad T_1 = \text{Bot} \quad \Gamma \vdash_A t_2 : T_2}{\Gamma \vdash_A t_1 t_2 : \text{Bot}} \text{TA-APPBOT}$$

$$\frac{\Gamma \vdash_A t_1 : R_1 \quad R_1 = \text{Bot}}{\Gamma \vdash_A t_1.l_i : \text{Bot}} \text{TA-PROJBOT}$$

在声明式系统中，包容规则可以把 Bot 提升为任意函数类型，所以一个 Bot 型项可以应用于任意类型的实参，结果又能被看作任意类型。投影的情形相同。上面两条算法规则把这种行为直接写了出来。

练习 16.4.1 (★). 若语言还包含条件式，是否需要再为 if 加一条算法类型规则？

查看原书附录 A 中的 16.4.1 解答

在当前语言中，为 Bot 增加这些支持并不复杂。不过，§28.8 会看到，Bot 与有界量化结合时会产生更严重的困难。

第十七章 子类型化的 ML 实现

本章扩展第十章实现的简单类型 lambda 演算，加入支持子类型化所需的机制，其中最重要的是检查子类型关系的函数。

17.1 语法

类型和项的数据类型定义遵循图15-1和15-3中的抽象语法。

```
type ty =  
  TyRecord of (string * ty) list  
| TyTop  
| TyArr of ty * ty  
  
type term =  
  TmRecord of info * (string * term) list  
| TmProj of info * term * string  
| TmVar of info * int * int  
| TmAbs of info * string * ty * term  
| TmApp of info * term * term
```

Rust 对照实现（译者增补）

```
#[derive(Clone, Debug, PartialEq, Eq)]  
pub enum Type {  
  Top,  
  Arrow(Box<Type>, Box<Type>),  
  Record(Vec<(String, Type)>),  
}  
  
#[derive(Clone, Debug, PartialEq, Eq)]  
pub enum Term {  
  Record(Vec<(String, Term)>),  
  Projection(Box<Term>, String),  
  Variable(usize),  
  Abstraction(String, Type, Box<Term>),  
}
```

```

    Application(Box<Term>, Box<Term>),
}

impl std::fmt::Display for Type {
    fn fmt(&self, formatter: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        match self {
            Type::Top => write!(formatter, "Top"),
            Type::Arrow(input, output) => write!(formatter, "{input} ->
↪ {output}"),
            Type::Record(fields) => {
                write!(formatter, "{{")?;
                for (index, (label, ty)) in fields.iter().enumerate() {
                    if index > 0 {
                        write!(formatter, ", ")?;
                    }
                    write!(formatter, "{label}: {ty}")?;
                }
                write!(formatter, "}}")
            }
        }
    }
}

pub type Context = Vec<Type>;

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum TypeError {
    UnboundVariable(usize),
    MissingField(String),
    ExpectedRecord(Type),
    ExpectedFunction(Type),
    ParameterMismatch { expected: Type, actual: Type },
}

```

与纯粹的简单类型 lambda 演算相比，这里新增了类型 `TyTop`、类型构造子 `TyRecord`，以及项构造子 `TmRecord` 和 `TmProj`。记录及其类型采用最直接的表示：字段名与对应的项或类型所组成的列表。

17.2 子类型关系

§16.1给出的算法子类型关系伪代码可以直接写成下面的 OCaml 函数。

```

let rec subtype tyS tyT =
  (=) tyS tyT ||
  match (tyS,tyT) with
  (TyRecord(fS), TyRecord(fT)) ->
    List.for_all
      (fun (li,tyTi) ->
        try let tySi = List.assoc li fS in
          subtype tySi tyTi
        with Not_found -> false)
      fT
  | (_,TyTop) ->
    true
  | (TyArr(tyS1,tyS2),TyArr(tyT1,tyT2)) ->
    (subtype tyT1 tyS1) && (subtype tyS2 tyT2)
  | (_,_) ->
    false

```

Rust 对照实现 (译者增补)

```

pub fn is_subtype(source: &Type, target: &Type) -> bool {
  if source == target {
    return true;
  }

  match (source, target) {
    (_, Type::Top) => true,
    (Type::Arrow(source_in, source_out), Type::Arrow(target_in, target_out))
    => {
      is_subtype(target_in, source_in) && is_subtype(source_out, target_out)
    }
    (Type::Record(source_fields), Type::Record(target_fields)) => {
      target_fields.iter().all(|(label, target_type)| {
        source_fields
          .iter()
          .find(|(source_label, _)| source_label == label)
          .is_some_and(|(_, source_type)| is_subtype(source_type,
            target_type))
      })
    }
    _ => false,
  }
}

```


这里对伪代码稍作修改，在函数开头增加了自反性检查。运算符(=)是普通的相等比较；这里把它写在前缀位置，是因为在另外一些子类型实现中，它会被替换成对其他比较函数的调用。|| 是短路布尔“或”：若第一个分支得到 `true`，第二个分支便不会执行。严格说来，这项检查并非必需，但在真实编译器中却是一项重要优化。现实程序很少实际利用子类型化；换言之，子类型检查器收到的大多数类型对本来就相等。进一步说，若类型表示能保证结构同构的类型也具有相同的物理表示——例如在构造类型时使用哈希驻留 (*hash consing*) (Goto, 1974; Appel 与 Gonçalves, 1993) ——这项检查只需一条指令。

译者注：哈希驻留是一种复用不可变结构的实现技术。构造类型节点时，实现先根据节点的结构内容计算哈希，再查询已有节点；若已存在结构相同的节点，就直接复用它，否则创建并登记新节点。例如，两次构造 `Nat → Bool` 都会得到指向同一个类型节点的引用。由此，比较两个类型是否结构相同时，只需比较它们是否指向同一物理对象，无须递归遍历整个类型结构。

记录子类型规则自然需要对列表作一些处理。`List.for_all` 把谓词（第一个参数）应用于列表的每个元素，只有所有应用都返回 `true` 时才返回 `true`。`List.assoc li fS` 在字段列表 `fS` 中查找标签 `li`，并返回相应的字段类型 `tySi`；若 `fS` 中不存在该标签，它就抛出 `Not_found`，这里捕获该异常并把结果改为 `false`。

17.3 类型检查

类型检查函数是在先前实现的 `typeof` 函数上直接扩展而成的。主要变化出现在应用项的分支：这里要检查实参类型是否为函数所要求参数类型的子类型。此外，还增加了构造记录和投影字段的两个分支。

```
let rec typeof ctx t =
  match t with
  | TmRecord(fi, fields) ->
    let fieldtys =
      List.map (fun (li,ti) -> (li, typeof ctx ti)) fields in
    TyRecord(fieldtys)
  | TmProj(fi, t1, l) ->
    (match (typeof ctx t1) with
     | TyRecord(fieldtys) ->
        (try List.assoc l fieldtys
         with Not_found -> error fi ("label " ^ l ^ " not found")))
     | _ -> error fi "Expected record type")
  | TmVar(fi,i,_) -> getTypeFromContext fi ctx i
  | TmAbs(fi,x,tyT1,t2) ->
    let ctx' = addbinding ctx x (VarBind(tyT1)) in
    let tyT2 = typeof ctx' t2 in
    TyArr(tyT1, tyT2)
  | TmApp(fi,t1,t2) ->
    let tyT1 = typeof ctx t1 in
```

```

let tyT2 = typeof ctx t2 in
(match tyT1 with
  TyArr(tyT11,tyT12) ->
    if subtype tyT2 tyT11 then tyT12
    else error fi "parameter type mismatch"
  | _ -> error fi "arrow type expected")

```

Rust 对照实现 (译者增补)

```

pub fn type_of(context: &Context, term: &Term) -> Result<Type, TypeError> {
  match term {
    Term::Record(fields) => fields
      .iter()
      .map(|(label, field)| Ok((label.clone(), type_of(context, field)?)))
      .collect::<Result<Vec<_, _>>()>()
      .map(Type::Record),
    Term::Projection(record, label) => match type_of(context, record)? {
      Type::Record(fields) => fields
        .iter()
        .find(|(field_label, _)| field_label == label)
        .map(|(_, field_type)| field_type.clone())
        .ok_or_else(|| TypeError::MissingField(label.clone())),
      other => Err(TypeError::ExpectedRecord(other)),
    },
    Term::Variable(index) => context
      .get(*index)
      .cloned()
      .ok_or(TypeError::UnboundVariable(*index)),
    Term::Abstraction(_, parameter_type, body) => {
      let mut body_context = context.clone();
      body_context.insert(0, parameter_type.clone());
      Ok(Type::Arrow(
        Box::new(parameter_type.clone()),
        Box::new(type_of(&body_context, body)?),
      ))
    },
    Term::Application(function, argument) => {
      let function_type = type_of(context, function)?;
      let argument_type = type_of(context, argument)?;
      match function_type {
        Type::Arrow(parameter_type, result_type) => {
          if is_subtype(&argument_type, &parameter_type) {
            Ok(*result_type)
          } else {

```


时，很难一眼看出后者缺少字段 `d`。

可以把 `subtype` 从“返回 `true` 或 `false`”改成“成功时返回 `unit` 值 `()`，失败时抛出异常”。异常会在两个类型真正无法匹配的位置抛出，因而能够更准确地说明原因。这不会改变类型检查器最终是接受还是拒绝程序：原来的子类型检查只要返回 `false`，类型检查器随后本来就会调用 `error` 抛出异常。

请重新实现 `typeof` 和 `subtype`，使所有错误消息尽可能提供有用信息。

[查看练习 17.3.3 的译者参考解答](#)

练习 17.3.4 ($\star\star\star$, $\rightarrow$). §15.6 用以下方法定义带记录和子类型化语言的强制转换语义：每当子类型推导证明 $S <: T$ 时，就把它翻译成一个显式的强制转换函数；再把类型推导翻译成已经在适当位置插入这些函数的纯简单类型 `lambda` 项。源程序的行为由翻译后项的求值结果来解释。

请实现上述两种翻译。修改上面的 `subtype`，让它构造并返回以项表示的强制转换函数；相应地修改 `typeof`，让它同时返回类型与翻译后的项。随后应当求值并打印翻译后的项，而不是原始输入项。

[查看练习 17.3.4 的译者参考解答](#)

第十八章 案例研究：命令式对象

本章进入第一个较为完整的程序设计示例。我们将综合运用此前定义的大部分机制——函数、记录、一般递归、可变引用与子类型化——构造一组支持对象和类（*class*）的常用编程模式。这些对象与类类似于 Smalltalk、Java 等面向对象语言中的相应机制。本章不会为对象或类加入新的具体语法；这里的目标，是用层次更低的构造近似实现这些较复杂的语言特性，从而理解它们的工作方式。

在本章大部分内容中，这种近似相当准确：把对象和类视为派生形式，并将其去糖为已有机制的简单组合，便能令人满意地实现它们的大多数特性。不过，§18.9 讨论虚方法（*virtual method*）与 `self` 时会遇到求值次序问题，使这种去糖不够贴近真实实现。更直接的做法是明确规定对象与类的语法、操作语义和类型规则；第十九章将采用这种方式。

18.1 什么是面向对象程序设计？

围绕“某某的本质是什么？”展开的争论，往往更能暴露参与者各自的偏好，而不是揭示讨论对象的某种客观真相。试图严格定义“面向对象”也不例外。尽管如此，大多数面向对象语言仍共享若干基本特性；这些特性共同支持一种独特的程序设计风格，其优点与缺点都已得到较充分的认识。¹

1. **多种内部表示。**面向对象风格的一项基本特点是：调用对象的某个操作时，实际执行哪段代码取决于该对象。两个对象即使具有相同的接口、支持同一组操作，也可以采用完全不同的内部表示；它们只需各自提供一套适合自身表示的操作实现。这些实现称为对象的方法（*method*）。调用对象上的操作称为方法调用（*method invocation*），也常形象地称为向对象“发送消息”。调用时需要在运行期根据操作名查找对象所关联的方法表；这个过程称为动态分派（*dynamic dispatch*）。

相比之下，传统的抽象数据类型（ADT）由一组值和这组值上的一套固定操作实现组成。操作实现由静态定义，既有优点也有缺点；§24.2 会进一步比较二者。

译者注：抽象数据类型（*abstract data type*）（ADT）隐藏数据的具体表示，只向使用者公开一组操作。例如，一个栈可以只公开 `empty`、`push`、`pop` 等操作；内部究竟使用数组还是链表，外部代码无须知道，也不能直接查看。传统 ADT 通常由模块等结构统一提供一套操作，同一抽象类型的值共享这套实现；对象则把适合自身内部表示的方法与对象放在一起，因此具有同一接口的不同对象可以采

¹本章示例使用带子类型化的简单类型 `lambda` 演算（图15-1），并加入记录（图15-3）与引用（图13-1）。相应的 OCaml 实现是 `fullref`。

用不同的表示和方法实现。这里的 *abstract data type* 与有时也缩写为 ADT 的 *algebraic data type* (代数数据类型) 是两个不同的概念。

2. 封装 (*encapsulation*)。对象的内部表示通常不会暴露在对象定义之外；只有对象自身的方法可以直接查看或修改其字段。² 这意味着修改对象的内部表示时，影响通常局限在程序中一小块容易定位的区域，从而显著改善大型系统的可读性和可维护性。

抽象数据类型也提供相似的封装：值的具体表示只在某个作用域——例如模块或 ADT 定义——内可见，作用域之外的代码只能调用该作用域内定义的操作来处理这些值。

3. 子类型化。对象的类型，也就是它的接口，只列出操作的名称与类型。内部表示不会出现在对象类型中，因为它不影响外部可以直接对对象执行哪些操作。

对象接口很自然地适合子类型关系。若对象满足接口 *I*，它也必然满足只列出 *I* 中一部分操作的接口 *J*：期待 *J* 型对象的上下文只能调用 *J* 中的操作，所以传入 *I* 型对象总是安全的。因此，对象子类型化与记录子类型化相似；在本章的对象模型中，二者实际上完全相同。忽略接口中一部分操作的能力，使同一段客户代码能够统一处理多种对象，只要求它们提供某组共同操作。

4. 继承 (*inheritance*)。接口有部分重合的对象通常也会共享部分行为，我们希望这些共同行为只实现一次。大多数面向对象语言通过类实现这种复用：类是创建对象的模板；子类化 (*subclassing*) 允许在旧类上增加新方法，并在必要时选择地覆盖旧方法，从而派生新类。有些语言不用类，而采用委托 (*delegation*)，把对象与类的特性结合起来。
5. 开放递归 (*open recursion*)。大多数具有对象和类的语言还允许一个方法体通过特殊变量 `self` (有些语言写作 `this`) 调用同一对象的其他方法。`self` 的特殊之处在于它采用后期绑定 (*late binding*)：在某个类中定义的方法，可以调用直到该类的某个子类中才定义的方法。

本章余下各节依次发展这些特性：先构造很简单的独立对象，再逐步加入能力更强的类机制。后续章节还会从其他角度研究对象与类。第十九章直接定义 Java 风格的对象与类，不再把它们编码成其他构造；第二十七章借助有界量化改进本章类构造的运行效率；第三十二章则在纯函数式环境中发展更强的对象编码。

²不同语言对封装的具体规定并不相同。在 Smalltalk 中，对象外部根本无法命名其内部字段；Java 和 C++ 则允许把字段标为 `public` 或 `private`。反过来，Smalltalk 的所有方法都公开可用，而 Java 与 C++ 允许把方法标为私有，只能从同一对象的其他方法中调用。本章忽略这些细化，但相关问题在研究文献中已有深入讨论 (Pierce 与 Turner, 1993; Fisher 与 Mitchell, 1998; Fisher, 1996a; Fisher 与 Mitchell, 1996; Fisher, 1996b; Fisher 与 Reppy, 1999)。

虽然大多数面向对象语言都把封装视为核心概念，也有一些语言并不如此。CLOS (Bobrow、DeMichiel、Gabriel、Keene、Kiczales 与 Moon, 1988; Kiczales、des Rivières 与 Bobrow, 1991)、Cecil (Chambers, 1992, 1993)、Dylan (Feinberg、Keene、Mathews 与 Withington, 1997; Shalit)、KEA (Mugridge、Hamer 与 Hosking, 1991)、以及 Castagna、Ghelli 与 Longo (1995; 另见 Castagna, 1997) 的 lambda- $\&$ 演算采用多方法，把对象状态与方法分开，并在方法调用时借助特殊类型标签从重载的方法体中选择合适分支。这些语言在对象创建、方法调用和类定义方面的底层机制与本章差别很大，不过最终形成的高层编程模式颇为相似。

18.2 对象

最简单的对象就是一种数据结构：它封装内部状态，并通过一组方法允许客户代码访问这些状态。内部状态通常由若干可变的实例变量（*instance variable*）（或字段）组成；这些变量由各方法共享，程序的其他部分无法访问。

本章始终以简单计数器对象为例。每个计数器保存一个数，并提供两个方法：`get` 返回当前值，`inc` 把该值加一。

利用前几章的机制，可以用一个引用单元保存内部状态，用一条函数字段记录保存各方法。当前状态为 1 的计数器如下：

```
c = let x = ref 1 in
  {get = lambda _:Unit. !x,
   inc = lambda _:Unit. x := succ(!x)};

► c : {get:Unit->Nat, inc:Unit->Unit}
```

两个方法都写成 `lambda` 抽象，并接受一个不会在函数体中使用的参数；该参数写作 `_`。由于 `lambda` 抽象本身是值，创建对象时不会执行其中的代码；以后每次以 `unit` 为参数调用方法时，方法体才会执行。还要注意，对象状态由两个方法共享，却无法从程序其余部分访问；这种封装直接来自变量 `x` 的词法作用域。

调用对象 `c` 的方法，只需从记录中投影相应字段，再把它应用于合适参数：

```
c.inc unit;           ► unit : Unit
c.get unit;           ► 2 : Nat
(c.inc unit; c.inc unit; c.get unit); ► 4 : Nat
```

`inc` 返回 `unit`，所以可以用§11.3 的分号记法连续执行多次递增。最后一行也可以写成：

```
let _ = c.inc unit in let _ = c.inc unit in c.get unit;
```

我们可能要创建并操作许多计数器，因此为其类型引入缩写：

```
Counter = {get:Unit->Nat, inc:Unit->Unit};
```

本章关注对象的构造方式，而不是如何用对象组织大型程序。不过，为了说明使用对象的代码只依赖其公开接口、无须知道内部表示，下面先定义一个操作计数器的简单函数。即使以后换用内部表示不同的计数器，这个函数仍可原样使用。该函数接受一个计数器，并连续调用三次 `inc`：

```
inc3 = lambda c:Counter.
  (c.inc unit; c.inc unit; c.inc unit);

► inc3 : Counter -> Unit

(inc3 c; c.get unit); ► 7 : Nat
```

18.3 对象生成器

前面一次构造了一个计数器。编写对象生成器 (*object generator*) 同样简单：这是一个每次调用都会创建并返回新计数器的函数。

```
newCounter =
  lambda _:Unit. let x = ref 1 in
    {get = lambda _:Unit. !x,
     inc = lambda _:Unit. x := succ(!x)};

► newCounter : Unit -> Counter
```

18.4 子类型化

面向对象风格广受欢迎的一个原因，是同一段客户代码能够处理结构不同的对象。例如，除了前面的 `Counter`，还可以创建带 `reset` 方法的对象，使其随时恢复初始状态 1。

```
ResetCounter =
  {get:Unit->Nat, inc:Unit->Unit, reset:Unit->Unit};

newResetCounter =
  lambda _:Unit. let x = ref 1 in
    {get = lambda _:Unit. !x,
     inc = lambda _:Unit. x := succ(!x),
     reset = lambda _:Unit. x := 1};

► newResetCounter : Unit -> ResetCounter
```

`ResetCounter` 包含 `Counter` 的全部字段，并多出一个字段，所以记录子类型规则给出 `ResetCounter <: Counter`。因此，接受普通计数器的 `inc3` 也能安全地用于可重置计数器：

```
rc = newResetCounter unit; ► rc : ResetCounter
(inc3 rc; rc.reset unit; inc3 rc; rc.get unit); ► 4 : Nat
```

18.5 组合实例变量

迄今为止，对象状态只包含一个引用单元。更实际的对象通常有多个实例变量。后文需要把这些变量作为整体传递，因此把计数器的内部表示改成“引用单元组成的记录”，方法体通过投影字段访问实例变量：

```
c = let r = {x=ref 1} in
  {get = lambda _:Unit. !(r.x),
   inc = lambda _:Unit. r.x := succ(!(r.x))};
```



```
► c : Counter
```

这条实例变量记录的类型称为对象的表示类型 (*representation type*):

```
CounterRep = {x:Ref Nat};
```

18.6 简单的类

`newCounter` 与 `newResetCounter` 除了后者多出 `reset` 方法外，定义完全相同。这两个例子很短，重复并不严重；但真实程序中的定义可能延续许多页，最好把共同功能只写一次。大多数面向对象语言用类来实现这种复用。

现实语言的类机制往往十分复杂，包含 `self`、`super`、可见性注解、静态字段与方法、内部类、友元类、`final`、`Serializable` 等许多特性。³本章忽略其中大部分，只讨论类最基本的两方面：通过继承复用代码，以及 `self` 的后期绑定。先从继承开始。

最原始的类，就是保存一组方法的数据结构；它既可以实例化成新对象，也可以扩展成另一个类。

要把普通计数器的 `get` 和 `inc` 复用于可重置计数器，最直接的做法似乎是从某个现成的计数器对象复制这两个方法。然而，这些方法已经引用了原对象的实例变量记录；复制后仍会读写原对象的状态，而不会操作新对象的状态。因此，要让同一套方法代码配合不同对象的实例变量使用，必须把实例变量记录变成方法代码的参数。于是，前面的 `newCounter` 可以拆成两部分。第一部分以任意实例变量记录为参数，定义方法体：

```
counterClass =
  lambda r:CounterRep.
    {get = lambda _:Unit. !(r.x),
     inc = lambda _:Unit. r.x := succ(!(r.x))};

► counterClass : CounterRep -> Counter
```

第二部分分配实例变量记录，并把它交给方法体，构造对象：

```
newCounter =
  lambda _:Unit. let r = {x=ref 1} in counterClass r;

► newCounter : Unit -> Counter
```

`counterClass` 中的方法体可以复用于新类，也就是子类。例如，可重置计数器类定义为：

```
resetCounterClass =
  lambda r:CounterRep.
```

³多数语言之所以把大量机制都塞进类，是因为类往往是唯一的大规模程序结构组织手段。广泛使用的语言中，只有 OCaml 同时提供类和成熟的模块系统。因此，在许多语言中，凡是与大型程序结构有关的特性，最终都容易集中到类机制里。

```

let super = counterClass r in
  {get = super.get,
   inc = super.inc,
   reset = lambda _:Unit. r.x := 1};

► resetCounterClass : CounterRep -> ResetCounter

```

与 `counterClass` 相同，该函数接受实例变量记录并返回对象。它先用同一条实例变量记录 `r` 调用 `counterClass`，得到的“父对象”绑定为 `super`；随后复制 `super` 的 `get` 与 `inc` 字段，并为 `reset` 提供新函数，构造新的对象。由于 `super` 也是基于 `r` 构造的，三个方法共享同一组实例变量。

创建可重置计数器时，再次分配实例变量，并调用下面这个实际完成构造工作的类生成器：

```

newResetCounter =
  lambda _:Unit. let r = {x=ref 1} in resetCounterClass r;

► newResetCounter : Unit -> ResetCounter

```

练习 18.6.1 (推荐, ★★). 编写 `resetCounterClass` 的子类，新增方法 `dec`，把计数器当前保存的值减一。使用 `fullref` 检查器测试这个类。

[查看原书附录 A 中的 18.6.1 解答](#)

练习 18.6.2 (★★, ⇨). 在子类方法记录中显式复制超类的大多数字段，记法显得笨重。它虽不必重新写出超类方法的代码，却仍要重复许多字段名。若用这种风格开发更大的面向对象程序，很快就会希望采用更紧凑的写法，例如

$$super \text{ with } \{reset = \lambda_ : Unit. r.x := 1\}$$

表示“一条与 `super` 相同、但新增或替换了 `reset` 字段的记录”。请为这种构造写出语法、操作语义和类型规则。

[查看练习 18.6.2 的译者参考解答](#)

这里要强调：这些类是用于构造对象的值，本身并不充当类型。可以定义多个不同的类；只要它们生成的对象具有相同的公开接口，这些对象在本章的结构类型系统中就具有相同的类型。在 C++、Java 等主流面向对象语言中，类的地位更复杂：既充当编译期类型，也充当运行期数据结构。§19.3 会进一步讨论。

18.7 增加实例变量

普通计数器与可重置计数器恰好使用相同的内部表示；一般而言，子类不仅要扩展超类的方法，也可能要扩展其实例变量。例如，可以定义“备份计数器”：它的 `reset` 不恢复某个固定常量，而恢复到最近一次调用 `backup` 时保存的状态。

```
BackupCounter =
  {get:Unit->Nat, inc:Unit->Unit,
   reset:Unit->Unit, backup:Unit->Unit};

BackupCounterRep = {x:Ref Nat, b:Ref Nat};
```

实现这种计数器需要额外的实例变量 `b` 保存备份值。与 `resetCounterClass` 复制 `counterClass` 的 `get`、`inc` 并添加 `reset` 相似，`backupCounterClass` 从 `resetCounterClass` 复制 `get`、`inc`，并提供自己的 `reset` 与 `backup`：

```
backupCounterClass =
  lambda r:BackupCounterRep.
    let super = resetCounterClass r in
      {get = super.get,
       inc = super.inc,
       reset = lambda _:Unit. r.x := !(r.b),
       backup = lambda _:Unit. r.b := !(r.x)};

► backupCounterClass : BackupCounterRep -> BackupCounter
```

这个定义有两点值得注意。第一，父对象 `super` 已经有 `reset` 方法，但我们希望它表现不同，所以另写了一份新实现。新类覆盖（*override*）了超类的 `reset` 方法。第二，在构造 `super` 时，子类型化不可或缺：`resetCounterClass` 需要 `CounterRep` 型参数，而实际参数 `r` 的类型是其子类型 `BackupCounterRep`。也就是说，传给 `resetCounterClass` 的实际记录除了包含其方法所需的字段 `x`，还包含额外字段 `b`；记录宽度子类型化允许它忽略这个多余字段。

练习 18.7.1 (推荐, ★★). 为 `backupCounterClass` 定义一个子类，增加方法 `reset2` 与 `backup2`，控制第二个备份槽。它应与 `backupCounterClass` 引入的备份槽完全独立：`reset` 恢复最近一次 `backup` 保存的值，`reset2` 恢复最近一次 `backup2` 保存的值。使用 `fullref` 测试这个类。

[查看原书附录 A 中的 18.7.1 解答](#)

18.8 调用超类方法

此前，变量 `super` 用于把超类的功能复制进子类。也可以在新方法体中调用 `super` 的方法，在超类行为上增加额外操作。例如，定义一种备份计数器，使每次调用 `inc` 前都自动调用 `backup`。这个类究竟有什么实际用途并不重要；它只是用于说明写法。

```
funnyBackupCounterClass =
  lambda r:BackupCounterRep.
    let super = backupCounterClass r in
```

```

{get = super.get,
 inc = lambda _:Unit.
   (super.backup unit; super.inc unit),
 reset = super.reset,
 backup = super.backup};

```

```
► funnyBackupCounterClass : BackupCounterRep -> BackupCounter
```

新的 `inc` 定义调用 `super.inc` 与 `super.backup`，不必在这里重写超类两项方法的代码。示例较大时，这种方式可以显著减少重复实现。

18.9 带 `self` 的类

最后一项扩展允许类的方法通过 `self` 引用同一对象的其他方法。考虑一种带 `set` 方法的计数器，外部可以用它把当前计数设为任意自然数：

```

SetCounter =
  {get:Unit->Nat, set:Nat->Unit, inc:Unit->Unit};

```

假设还希望 `inc` 通过 `set` 和 `get` 实现，而不直接读取或修改实例变量 `x`。在较大的例子中，`set` 与 `get` 的定义可能各占许多页。此时，`inc` 必须能够访问与自己一同定义的 `get` 和 `set`。为此，可以先用 `self` 代表包含这三个方法的记录。本例的调用方向只有从 `inc` 到 `get` 和 `set`；一般而言，同一机制也允许多个方法经由这条记录相互调用。

§11.11 已经用 `fix` 构造过递归函数记录。这里把方法记录写成一个以 `self` 为参数的函数，再用 `fix` “打结”，使最终构造出的记录本身成为传给这个函数的 `self`。

```

setCounterClass =
  lambda r:CounterRep.
    fix
      (lambda self:SetCounter.
        {get = lambda _:Unit. !(r.x),
         set = lambda i:Nat. r.x := i,
         inc = lambda _:Unit.
           self.set (succ (self.get unit))});

```

```
► setCounterClass : CounterRep -> SetCounter
```

这个类没有父类，所以不需要 `super`。`inc` 方法体从 `self` 方法记录中调用 `get`，再调用 `set`。`fix` 完全封装在 `setCounterClass` 内部；创建计数器时仍使用一贯的方式：

```

newSetCounter =
  lambda _:Unit. let r = {x=ref 1} in setCounterClass r;

```

```
► newSetCounter : Unit -> SetCounter
```

译者注：本节把 `fix` 放在 `setCounterClass` 内部。执行 `setCounterClass r` 时，`self` 就会连接到该类自身的方法记录，从而使 `get`、`set` 和 `inc` 能够互相引用。不过，这条连接在子类覆盖方法之前就已经确定：若子类随后复用 `inc` 并重新定义 `set`，`inc` 中的 `self.set` 仍会调用原方法记录中的 `set`。下一节将把这条连接推迟到最终对象创建时完成。

18.10 通过 `self` 实现开放递归

上一节在执行 `setCounterClass r` 时就已经确定了 `self`。多数面向对象语言支持一种更一般的机制：`self` 到最终对象创建时才绑定到该对象的方法记录。因此，定义在超类中的方法也能通过 `self` 调用子类后来覆盖的方法。这称为开放递归，也称 `self` 的后期绑定。要实现这种行为，先从类定义中移走 `fix`：

```
setCounterClass =
  lambda r:CounterRep.
  lambda self:SetCounter.
    {get = lambda _:Unit. !(r.x),
     set = lambda i:Nat. r.x := i,
     inc = lambda _:Unit.
       self.set (succ(self.get unit))};

► setCounterClass : CounterRep -> SetCounter -> SetCounter
```

再把 `fix` 放到对象创建函数中：

```
newSetCounter =
  lambda _:Unit. let r = {x=ref 1} in
    fix (setCounterClass r);

► newSetCounter : Unit -> SetCounter
```

移动 `fix` 会改变 `setCounterClass` 的类型：它不仅对实例变量记录作抽象，还对一个 `self` 对象作抽象；二者都在实例化时提供。

开放递归的关键作用是：超类中的方法能够调用子类的方法，即使定义超类时子类还不存在。现在，`self` 指向创建当前对象时实际使用的那个类的方法，其范围不再限于当前类；创建对象时使用的类可能是当前类的某个子类。

例如，构造一种计数器，记录 `set` 被调用的次数。接口多出一项读取调用次数的操作，表示中也增加一个实例变量：

```
InstrCounter =
  {get:Unit->Nat, set:Nat->Unit,
   inc:Unit->Unit, accesses:Unit->Nat};

InstrCounterRep = {x:Ref Nat, a:Ref Nat};
```

下面的插桩计数器类从 `setCounterClass` 复制 `inc` 和 `get`；`accesses` 按通常方式定义；`set` 先把调用计数加一，再通过 `super` 调用超类的 `set`：

```

instrCounterClass =
  lambda r:InstrCounterRep.
  lambda self:InstrCounter.
    let super = setCounterClass r self in
    {get = super.get,
     set = lambda i:Nat.
              (r.a := succ(!r.a)); super.set i),
     inc = super.inc,
     accesses = lambda _:Unit. !(r.a)};

► instrCounterClass :
  InstrCounterRep -> InstrCounter -> InstrCounter

```

由于 `self` 采用开放递归，`inc` 方法体中对 `set` 的调用最终会执行子类重新定义的 `set`，从而递增实例变量 `a`。虽然 `inc` 定义在超类中，而“递增调用次数”的行为定义在子类中，这种动态联系仍然成立。

18.11 开放递归与求值次序

上述 `instrCounterClass` 有一个问题：无法用它创建实例。若按通常方式定义

```

newInstrCounter =
  lambda _:Unit. let r = {x=ref 1, a=ref 0} in
    fix (instrCounterClass r);

► newInstrCounter : Unit -> InstrCounter

ic = newInstrCounter unit;

```

再计算 `newInstrCounter unit`，求值器会发散。下面逐步说明原因。用 $\langle ivars \rangle$ 表示分配完成的实例变量记录，用 $\langle f \rangle$ 表示等待 `self` 的函数，用 $\langle imethods \rangle$ 表示插桩计数器的方法记录，用 $\langle smethods \rangle$ 表示其超类——可设置计数器——的方法记录。这些记号只是下列求值步骤中对相应完整表达式的缩写。

1. 把 `newInstrCounter` 应用于 `unit`，得到

$$\text{let } r = \{x = \text{ref } 1, a = \text{ref } 0\} \text{ in fix } (\text{instrCounterClass } r)$$

2. 分配两个引用单元，组成 $\langle ivars \rangle$ ，并在余下项中用它替换 `r`：

$$\text{fix } (\text{instrCounterClass } \langle ivars \rangle)$$

3. 把 $\langle ivars \rangle$ 传给 `instrCounterClass`。由于该类以两个 `lambda` 抽象开头，立即得到一个等待 `self` 的函数：

$$\text{fix } (\lambda \text{self} : \text{InstrCounter}. \text{let } \text{super} = \text{setCounterClass } \langle ivars \rangle \text{self in } \langle imethods \rangle)$$

把括号中的函数记作 $\langle f \rangle$ ，当前项便是 $\text{fix } \langle f \rangle$ 。

4. 按照图11-12中的规则 E-FIXBETA 展开 $\text{fix } \langle f \rangle$ ，也就是在 $\langle f \rangle$ 的函数体中，用 $\text{fix } \langle f \rangle$ 替换 `self`：

$$\text{let } super = \text{setCounterClass } \langle ivars \rangle (\text{fix } \langle f \rangle) \text{ in } \langle imethods \rangle$$

5. 先把 `setCounterClass` 应用于实例变量记录，得到

$$\text{let } super = (\lambda self : \text{SetCounter}. \langle smethods \rangle)(\text{fix } \langle f \rangle) \text{ in } \langle imethods \rangle$$

6. 按值调用要求先把实参 $\text{fix } \langle f \rangle$ 求值为值，才能执行外层 beta 归约。于是下一步只能再次展开 `fix`，得到

$$\begin{aligned} \text{let } super = (\lambda self : \text{SetCounter}. \langle smethods \rangle) \\ (\text{let } super = \text{setCounterClass } \langle ivars \rangle (\text{fix } \langle f \rangle) \\ \text{in } \langle imethods \rangle) \\ \text{in } \langle imethods \rangle \end{aligned}$$

7. 外层 lambda 抽象的实参仍不是值，因此继续求值其中的内层项。把 `setCounterClass` 应用于 $\langle ivars \rangle$ ，得到

$$\begin{aligned} \text{let } super = (\lambda self : \text{SetCounter}. \langle smethods \rangle) \\ (\text{let } super = (\lambda self : \text{SetCounter}. \langle smethods \rangle)(\text{fix } \langle f \rangle) \\ \text{in } \langle imethods \rangle) \\ \text{in } \langle imethods \rangle \end{aligned}$$

8. 新出现的应用仍须先把实参 $\text{fix } \langle f \rangle$ 求值，于是再次展开 `fix`。相同结构会不断加深，外层应用永远无法执行。

问题在于，传给 `fix` 的函数过早使用了自己的参数 `self`。`fix` 的求值规则适合这样的函数：对 $\lambda x. t$ 取不动点时， t 只在暂时不会求值的位置引用 x ，例如内层 lambda 抽象的函数体。第11.11节的 `iseven` 示例11.11把 `fix` 用于形如 $\lambda ie. \lambda x. \dots$ 的函数；其中，对 `ie` 的递归引用位于内层抽象 λx 的函数体中，要等所得函数真正应用于参数时才会求值。这里的 `instrCounterClass` 却在计算 `super` 时立刻使用 `self`。

可以采用三种办法。

1. 在 `instrCounterClass` 中加入一层不使用参数的 lambda 抽象，推迟 `self` 的求值。下面先说明这种办法。它并不完全理想，但只需使用已有机制，因而容易描述和理解；第三十二章讨论纯函数式对象编码时还会用到它。
2. 改用其他底层机制模拟类。例如，不用 `fix` 构造方法表，而用引用单元显式地把递归联系“接起来”。§18.12将具体实现这种办法，第二十七章还会进一步改进。

3. 不再把对象和类编码成 lambda 抽象、记录与 **fix**，而把它们当作语言的基本构造，直接规定求值规则与类型规则。这样便可以选择符合预期的对象和类语义，而不必绕过既有应用与 **fix** 规则的限制。[第十九章](#) 将采用这种方法。

增加一层不使用参数的 lambda 抽象来控制求值次序，是函数式程序设计中常用的技巧。可以把任意表达式 t 包装成函数 $\lambda_ : \text{Unit}. t$ ，也就是 延迟计算封装 (*thunk*)。按照本书的求值规则，lambda 抽象本身就是值，因此其中的 t 暂时不会求值；需要执行这段计算时，再把该函数应用于 **unit**。这样便可以先传递尚未求值的 t ，以后再触发它的计算。

当前需要推迟的是 **self** 的求值。以 **SetCounter** 为例，修改后 **self** 是一个返回该对象的延迟计算封装，类型为

$$\text{Unit} \rightarrow \text{SetCounter}$$

这个封装应用于 **unit** 后，才会返回真正的对象。具体需要三处修改：改变类的 **self** 参数类型；在构造结果对象前增加一层不使用参数的抽象；把方法体中每个 **self** 改成 **self unit**。

```
setCounterClass =
  lambda r:CounterRep.
  lambda self:Unit->SetCounter.
  lambda _:Unit.
    {get = lambda _:Unit. !(r.x),
     set = lambda i:Nat. r.x := i,
     inc = lambda _:Unit.
       (self unit).set
       (succ((self unit).get unit))};

► setCounterClass :
  CounterRep -> (Unit->SetCounter) -> Unit -> SetCounter
```

我们不希望 **newSetCounter** 的类型改变，它仍应返回对象。因此，取 **setCounterClass** 的不动点后，还要把得到的延迟计算封装应用于 **unit**，取得真正的对象：

```
newSetCounter =
  lambda _:Unit. let r = {x=ref 1} in
    fix (setCounterClass r) unit;

► newSetCounter : Unit -> SetCounter
```

instrCounterClass 也作相同修改。改变 **self** 的类型后，其他位置相应调整即可：

```
instrCounterClass =
  lambda r:InstrCounterRep.
  lambda self:Unit->InstrCounter.
  lambda _:Unit.
    let super = setCounterClass r self unit in
    {get = super.get,
     set = lambda i:Nat.
```



```

        (r.a := succ(!r.a)); super.set i),
    inc = super.inc,
    accesses = lambda _:Unit. !(r.a)};

► instrCounterClass : InstrCounterRep ->
  (Unit->InstrCounter) -> Unit -> InstrCounter

```

最后，`newInstrCounter` 同样把 `fix` 得到的延迟计算封装应用于 `unit`：

```

newInstrCounter =
  lambda _:Unit. let r = {x=ref 1, a=ref 0} in
    fix (instrCounterClass r) unit;

► newInstrCounter : Unit -> InstrCounter

ic = newInstrCounter unit; ► ic : InstrCounter

```

加入延迟计算封装前，创建 `ic` 的这一步会发散。现在可以实际使用对象，以下测试表明 `accesses` 会按预期统计 `set` 与 `inc` 的调用：

```

(ic.set 5; ic.accesses unit); ► 1 : Nat
(ic.inc unit; ic.get unit); ► 6 : Nat
ic.accesses unit; ► 2 : Nat

```

练习 18.11.1 (推荐, ★★). 使用 `fullref` 检查器实现下列扩展：

1. 重写 `instrCounterClass`，使它也统计 `get` 的调用；
2. 在修改后的 `instrCounterClass` 上定义子类，加入 §18.4 中的 `reset` 方法；
3. 再定义一个子类，加入 §18.7 中的备份功能。

[查看原书附录 A 中的 18.11.1 解答](#)

18.12 更高效的实现

前面的测试表明，这种类实现确实具有 Smalltalk、C++、Java 等语言中通过 `self` 进行开放递归的方法调用行为。不过，从效率看，这个实现并不理想。为使 `fix` 收敛而加入的延迟计算封装，会推迟类方法表的构造。特别是，方法体中每次调用 `self` 都变成 `self unit`；也就是说，每次通过 `self` 调用另一个方法时，都会重新计算 `self` 所指的方法记录！

可以不用不动点，而在创建对象时用引用单元把 `self` 与最终的方法记录连接起来，从而避免重复计算。⁴ 类不再对一条以后用 `fix` 构造的方法记录 `self` 作抽象，而是对“指向方法记录的引用”作抽象，并先分配这条记录。具体说，实例化类时先为方法分配一个堆单

⁴这种做法与练习13.5.8的解答本质相同。James Riely 指出，可以利用 `Source` 类型的协变性，把它用于类构造。

元，以占位值初始化；然后构造真正的方法，并把指向该堆单元的指针交给它们，使它们可以通过 `self` 调用同一对象的其他方法；最后回填堆单元，使它保存真正的方法。下面再次给出 `setCounterClass`：

```
setCounterClass =
  lambda r:CounterRep. lambda self:Ref SetCounter.
    {get = lambda _:Unit. !(r.x),
     set = lambda i:Nat. r.x := i,
     inc = lambda _:Unit.
       (!self).set (succ ((!self).get unit))};

► setCounterClass : CounterRep -> Ref SetCounter -> SetCounter
```

`self` 指向保存当前对象方法的单元。调用 `setCounterClass` 时，该单元先装入占位值：

```
dummySetCounter =
  {get = lambda _:Unit. 0,
   set = lambda i:Nat. unit,
   inc = lambda _:Unit. unit};

► dummySetCounter : SetCounter

newSetCounter =
  lambda _:Unit.
    let r = {x=ref 1} in
    let cAux = ref dummySetCounter in
    (cAux := (setCounterClass r cAux); !cAux);

► newSetCounter : Unit -> SetCounter
```

方法体中的所有 `!self` 都受 `lambda` 抽象保护，因此在 `newSetCounter` 回填 `cAux` 之前，程序不会实际解引用它。

为了支持 `setCounterClass` 的子类，还需细化 `self` 的类型。每个类原先要求 `self` 与它构造的方法记录同型。因此，插桩计数器子类的 `self` 是指向 `InstrCounter` 方法记录的指针。然而，§15.5 已经看到，`Ref SetCounter` 与 `Ref InstrCounter` 不可比较（可变引用同时允许读取和写入，因此其元素类型不能协变）；把后者提升为前者并不安全。于是，在 `instrCounterClass` 中构造 `super` 时会发生参数类型不匹配。

```
instrCounterClass =
  lambda r:InstrCounterRep.
  lambda self:Ref InstrCounter.
    let super = setCounterClass r self in
    {get = super.get,
     set = lambda i:Nat.
       (r.a := succ(!(r.a)); super.set i),
```

```
inc = super.inc,
accesses = lambda _:Unit. !(r.a)};
```

► Error: parameter type mismatch

解决办法是把 `self` 类型中的 `Ref` 换成 `Source`：只把读取方法指针的能力交给类，不交出它并不需要的写入能力。`Source` 允许协变子类型化，所以

Source InstrCounter <: Source SetCounter

构造 `super` 便能通过类型检查。

```
setCounterClass =
  lambda r:CounterRep. lambda self:Source SetCounter.
    {get = lambda _:Unit. !(r.x),
     set = lambda i:Nat. r.x := i,
     inc = lambda _:Unit.
       (!self).set (succ ((!self).get unit))};
```

```
instrCounterClass =
  lambda r:InstrCounterRep.
  lambda self:Source InstrCounter.
    let super = setCounterClass r self in
    {get = super.get,
     set = lambda i:Nat.
       (r.a := succ(!(r.a)); super.set i),
     inc = super.inc,
     accesses = lambda _:Unit. !(r.a)};
```

► `instrCounterClass : InstrCounterRep ->`
`Source InstrCounter -> InstrCounter`

创建插桩计数器时，先定义一组占位方法作为 `self` 指针的初值，再分配实例变量和方法的堆空间，调用类构造真正的方法，最后回填引用单元：

```
dummyInstrCounter =
  {get = lambda _:Unit. 0,
   set = lambda i:Nat. unit,
   inc = lambda _:Unit. unit,
   accesses = lambda _:Unit. 0};

newInstrCounter =
  lambda _:Unit.
    let r = {x=ref 1, a=ref 0} in
    let cAux = ref dummyInstrCounter in
    (cAux := (instrCounterClass r cAux); !cAux);
```

```
► newInstrCounter : Unit -> InstrCounter
```

现在，`instrCounterClass` 与 `setCounterClass` 中构造方法表的代码，每创建一个对象只执行一次，而不再每次调用方法都执行。不过，同一类产生的每个对象所构造的方法表完全相同，理想情况下似乎只应在定义类时计算一次。[第二十七章](#)将借助[第二十六章](#)引入的有界量化实现这一优化。

18.13 小结

回到本章开头列出的面向对象程序设计特性，看看它们如何体现在本章示例中。

1. **多种表示**。本章所有对象都是计数器，也就是都能作为 `Counter` 使用；它们的内部表示却差别很大，从 [§18.2](#) 中的单个引用单元，到 [§18.9](#) 中包含多个引用的记录。每个对象都表示为一条由函数组成的记录，其中包含适合自身内部表示的 `Counter` 方法实现；有些对象还提供额外的方法。
2. **封装**。构造对象时，把保存实例变量的记录交给负责构造方法的函数。实例变量的名称因而只在方法定义的作用域内可见，程序其余部分无法直接访问它们。
3. **子类型化**。在本章模型中，对象类型之间的子类型化，就是普通的函数字段记录类型之间的子类型化。
4. **继承**。继承通过把已有超类的方法实现复制到新子类来模拟。严格说来，超类和子类都是“从实例变量到方法记录”的函数。子类先等待自己的实例变量，再用这些变量实例化超类，得到一条操作同一组变量的超类方法记录。
5. **开放递归**。现实语言中由 `self` 或 `this` 提供的开放递归，在这里通过另一项类参数 `self` 模拟。方法可以借它引用同一对象的其他方法。创建对象时再用 `fix` 把这项参数与最终的方法记录连接起来。

练习 18.13.1 (★★★). 对象还有一项在某些场合很有用的特性：对象身份 (*object identity*)。设想一种 `sameObject` 操作：若两个参数求值为同一个对象，返回 `true`；若它们由不同时间的两次对象生成器调用创建，则返回 `false`。应怎样扩展本章的对象模型来支持对象身份？

[查看原书附录 A 中的 18.13.1 解答](#)

18.14 注记

对象编码一直是程序语言研究中产生示例与问题的重要来源。Reynolds (1978) 很早就给出过一种编码；Cardelli (1984) 的文章引发了该领域的广泛兴趣。Cook (1989)、Cook 与 Palsberg (1989)、Kamin (1988) 和 Reddy (1988) 发展了用不动点理解 `self` 的方法；Kamin 与 Reddy (1994) 以及 Bruce (1991) 研究了这些模型之间的关系。

Gunter 与 Mitchell (1994) 收录了该领域若干重要的早期论文。Bruce、Cardelli 与 Pierce (1999) 以及 Abadi 与 Cardelli (1996) 综述了后续发展；Bruce (2002) 给出了当时最新的进展。Palsberg 与 Schwartzbach (1994) 以及 Castagna (1997) 提出了对象及其类型系统的其他基础方法。第三十二章末尾还会给出更多历史注记。

继承的重要性被大大高估了。

Inheritance is highly overrated.

——Grady Booch

译者注：Booch 并未否定继承本身。他所批评的，是早期面向对象方法把继承看得过于重要，尤其是把它当作代码复用的主要手段：发现几处相似实现，便抽出一个父类，再不断加深继承层次。这样建立的层次往往反映代码碰巧可以共享，却未必表示子类在概念和行为上确实是父类的一种。子类还会依赖父类的实现细节；父类发生变化，影响可能沿整条继承链传播，方法重写与动态分派也会使实际调用关系越来越难以判断。组合与委托通常能以较松散的关系表达同样的复用需求。

Booch 后来回顾自己早期的面向对象方法时说，以类组织数据与操作、用类表达抽象是正确的方向，错误在于过分强调继承，并期待通过泛化层次节省代码。他在自己的著作中也指出，继承不足以表达对象之间全部丰富的关系，还需要聚合、使用等其他关系。结合本章来看，继承是复用和扩展对象行为的一种有力机制，却不是面向对象的全部；封装、动态分派以及对象之间的协作同样重要。

第十九章 案例研究：Featherweight Java

第十八章展示了如何用带子类型化、记录和引用的 lambda 演算模拟面向对象程序设计的若干核心特性；其目的，是把这些特性编码成更基本的机制，以加深理解。本章改用另一种方式：以前几章的方法为基础，直接定义一门以 Java 为原型、只保留关键机制的面向对象语言。这里假定读者已经熟悉 Java。

19.1 引言

形式模型可以显著帮助程序语言这类复杂系统的设计：它能精确描述设计的某个方面，陈述并证明相关性质，还能把注意力引向原本容易忽略的问题。不过，建立模型时必须在完整性与紧凑性之间作出取舍；同时覆盖的方面越多，模型就越臃肿。很多时候，选用不那么完整却更紧凑的模型更为明智，因为它能以尽可能小的投入，帮助我们获得尽可能充分的理解。近期不少研究 Java 形式性质的论文都采用这种策略：省略并发、反射等高级特性，把注意力集中在理论已经较成熟的语言片段上。

Igarashi、Pierce 与 Wadler (1999) 提出 Featherweight Java (*Featherweight Java*) (FJ)，把它作为模拟 Java 类型系统的最小核心演算。FJ 几乎偏执地追求紧凑，只有五种项：创建对象、调用方法、访问字段、类型转换和变量。它的语法、类型规则与操作语义用一张 A4 纸便能完整排下。设计目标是尽可能删去特性——甚至删去赋值——同时保留 Java 类型系统的核心。FJ 与 Java 的纯函数式核心直接对应：每个 FJ 程序从字面上看也是可以执行的 Java 程序。¹

FJ 只比 lambda 演算或 Abadi 与 Cardelli (1996) 的对象演算大一点，却明显小于其他基于类的 Java 形式模型，包括 Drossopoulou、Eisenbach 与 Khurshid (1999)、Syme (1997)、Nipkow 与 Oheimb (1998)，以及 Flatt、Krishnamurthi 与 Felleisen (1998a, 1998b) 的系统。由于规模小，FJ 可以集中研究少数关键问题。例如，我们会看到，要用小步操作语义准确刻画 Java 的类型转换，比预想中更棘手。

FJ 主要用于研究 Java 的扩展。由于 FJ 本身十分紧凑，扩展中的本质部分会更醒目；同时，纯 FJ 的类型安全性证明很短，因此即使加入相当重要的扩展，严格证明其安全性通常也不会复杂到难以处理。FJ 原始论文借鉴 GJ 的设计，为 FJ 加入泛型类和泛型方法，以具体说明怎样用 FJ 为 Java 的扩展建立形式模型。GJ (Generic Java) 是为 Java 引入这两项机制的早期扩展方案 (Bracha、Odersky、Stoutamire 与 Wadler, 1998)。后续论文 (Igarashi、

¹本章示例都是 Featherweight Java 的项，语法与规则见图19-1至19-4。本章没有配套 OCaml 实现；FJ 被设计为 Java 的严格子集，所以任何 Java 实现都可以执行这些示例。

Pierce 与 Wadler, 2001) 还用 FJ 形式化了原始类型；这种类型是 GJ 为方便既有 Java 程序逐步迁移到 GJ 而引入的。Igarashi 与 Pierce (2000) 以 FJ 为基础研究 Java 内部类；FJ 也被用于研究 Java 的类型保持编译 (*type-preserving compilation*)，也就是保证良类型的源程序编译后仍得到良类型的目标程序 (League、Trifonov 与 Shao, 2001)，以及 Java 的语义基础 (Studer, 2001)。

设计 FJ 时，希望在保留完整版 Java 核心特性之安全性论证的同时，让类型安全性证明尽量简短。若某项语言特性只会把证明拉长，却不会使论证发生实质变化，就可以考虑省略。因此，FJ 不含并发、反射、赋值、接口、重载、发送给 `super` 的消息、`null` 指针、`int` 与 `bool` 等基本类型、抽象方法声明、内部类、子类字段遮蔽超类字段、访问控制以及异常。它保留的 Java 特性包括：相互递归的类定义、对象创建、字段访问、方法调用、方法覆盖、通过 `this` 进行方法递归、子类型化与类型转换。

省略赋值是关键的简化。对象字段由构造函数初始化，此后不再改变。FJ 因而只覆盖 Java 的“函数式”片段，许多常见 Java 惯用法——例如枚举的用法——无法表示。尽管如此，这个片段仍然计算完备——把 lambda 演算编码进去并不困难——也足以容纳有实际意义的程序；例如 Felleisen 与 Friedman (1998) 的 Java 教材中，许多程序采用纯函数式风格。

译者注：原书把“枚举的用法”列为 FJ 无法表示的 Java 惯用法，但作者勘误说明，这里的“enumerations”具体指什么已经无法确定。本章后文不依赖这个例子；读者只需把握本段的主旨：FJ 没有赋值，因此不能表达某些依赖可变状态的 Java 写法。

19.2 概览

FJ 程序由一组类定义和一个待求值项组成；后者对应完整 Java 程序中 `main` 方法的函数体。下面是几个典型类定义。

```
class A extends Object { A() { super(); } }

class B extends Object { B() { super(); } }

class Pair extends Object {
  Object fst;
  Object snd;
  // Constructor:
  Pair(Object fst, Object snd) {
    super(); this.fst=fst; this.snd=snd;
  }
  // Method definition:
  Pair setfst(Object newfst) {
    return new Pair(newfst, this.snd);
  }
}
```

为了使语法形式始终统一，即使超类是 `Object`，也总会明确写出 `extends Object`；即使 `A`、`B` 的构造函数没有实际内容，也总会完整写出；访问字段或调用方法时也总会写明接收者，例如 `this.snd`，即使接收者就是 `this`。构造函数始终采用固定形式：每个字段对应一个同名形参；先调用超类构造函数初始化继承字段，再用其余形参初始化本类字段。上例三个类的超类都是没有字段的 `Object`，所以 `super` 不带参数。`super` 和等号只会出现在 FJ 构造函数中。由于 FJ 没有产生副作用的操作，方法体始终写成 `return t`；的形式，其中 `t` 是表示返回结果的 FJ 项。

FJ 有五种项。`new A()`、`new B()` 与 `new Pair(...)` 是对象创建；`.setfst(...)` 是方法调用；方法体中的 `this.snd` 是字段访问；`newfst` 与 `this` 是变量。²在上述类定义下，

```
new Pair(new A(), new B()).setfst(new B())
```

求值为 `new Pair(new B(), new B())`。

剩下的一种项是类型转换（见§15.5）。表达式

```
((Pair)new Pair(new Pair(new A(), new B()), new A()).fst).snd
```

求值为 `new B()`。其中 `(Pair)t` 是类型转换，`t` 为 `new Pair(...).fst`。字段 `fst` 的声明类型是 `Object`，而继续访问 `snd` 只对 `Pair` 合法，所以转换不可省略。运行期规则会检查 `fst` 中保存的对象是否确实属于 `Pair` 类。本例中，`fst` 保存的是 `new Pair(new A(), new B())`，所以类型转换成功，随后可以继续访问字段 `snd`。

删去副作用还有一个好处：FJ 的求值可以完全在其句法内部形式化，无需像第十三章那样另设堆模型。它只有三条基本计算规则，分别处理字段访问、方法调用与类型转换。在 `lambda` 演算中，对函数应用 $t_1 t_2$ 求值时，必须先把函数位置上的 t_1 求值成 `lambda` 抽象；类似地，FJ 的计算规则要求先把字段访问或方法调用的接收者求值成 `new` 项。`lambda` 演算常说“一切皆函数”，这里则是“一切皆对象”。

字段访问规则 E-PROJNEW 的一个实例是

$$\text{new Pair}(\text{new A}(), \text{new B}()).\text{snd} \longrightarrow \text{new B}()$$

对象构造函数的固定语法保证：每个字段都有一个形参，次序与字段声明相同。`Pair` 的字段依次为 `fst`、`snd`，所以访问 `snd` 会选出第二个实参。

方法调用规则 E-INVKNEW 在本章第一个示例中的作用如下。接收者是 `new Pair(new A(), new B())`，因此在 `Pair` 类中查找 `setfst`，得到形参 `newfst` 和下面的方法体：

```
new Pair(newfst, this.snd)
```

²FJ 与 Java 略有不同，把 `this` 视为变量，而不是关键字。

调用方法时，需要同时进行两项替换：把方法体中的形参 `newfst` 换成实参 `new B()`，并把其中的 `this` 换成实际接收这次方法调用的对象。下面方括号中的两个映射就是这份替换说明。完整的求值过程为

$$\begin{aligned}
 & \text{new Pair}(\text{new A}(), \text{new B}()).\text{setfst}(\text{new B}()) \\
 \longrightarrow & [\text{newfst} \mapsto \text{new B}(), \text{ this} \mapsto \text{new Pair}(\text{new A}(), \text{new B}())] \text{new Pair}(\text{newfst}, \text{this.snd}) \\
 = & \text{new Pair}(\text{new B}(), \text{new Pair}(\text{new A}(), \text{new B}()).\text{snd}) \\
 \longrightarrow & \text{new Pair}(\text{new B}(), \text{new B}())
 \end{aligned}$$

第一步按照方法调用规则代入实参和接收者；最后一步再按照前述字段访问规则，从接收者对象中取出 `snd` 字段。这类似 lambda 演算的 beta 归约规则 E-APPABS，但有两个重要区别：接收者的类决定到哪里查找方法体，从而支持方法覆盖；接收者替换 `this`，从而支持通过 `self` 进行开放递归。³

若形参在方法体中出现多次，替换会复制实参值；不过 FJ 没有副作用，所以无法观察到这与标准 Java 语义的差异。

类型转换规则 E-CASTNEW 的一个实例是

$$(\text{Pair}) \text{new Pair}(\text{new A}(), \text{new B}()) \longrightarrow \text{new Pair}(\text{new A}(), \text{new B}())$$

转换对象简化为 `new` 项后，只需检查构造函数的类是否为目标类型的子类。若是，规约会删去转换；若不是，例如 `(A)new B()`，则没有规则可用，计算卡住，表示运行时错误。

计算可能以三种方式卡住：访问该类没有声明的字段；调用该类没有声明的方法，即“无法理解消息”；或者试图把对象转换为其运行时类的非超类。后文会证明：良类型程序绝不会出现前两种情况；不含向下类型转换和下文所谓“荒谬转换”的良类型程序也不会出现第三种情况。

FJ 采用标准的按值调用求值策略。前面第二个示例按以下四步求值；每一行的下划线标出下一步要规约的子项：

$$\begin{aligned}
 & ((\text{Pair}) \text{new Pair}(\text{new Pair}(\text{new A}(), \text{new B}()), \text{new A}()).\text{fst}).\text{snd} \\
 \longrightarrow & (\text{Pair}) \text{new Pair}(\text{new A}(), \text{new B}()).\text{snd} \\
 \longrightarrow & \text{new Pair}(\text{new A}(), \text{new B}()).\text{snd} \\
 \longrightarrow & \text{new B}()
 \end{aligned}$$

19.3 名义类型系统与结构类型系统

在正式定义 FJ 前，先讨论 FJ（以及 Java）同本书主要研究的带类型 lambda 演算之间一项根本风格差异：类型名称的地位。

前几章经常为较长的复合类型定义短名，以提高示例可读性：

$$\text{NatPair} = \{fst : \text{Nat}, snd : \text{Nat}\}$$

³熟悉 Abadi 与 Cardelli (1996) 对象演算的读者会发现，这与其 ζ 归约规则很相似。

这种定义只是排印上的便利；`NatPair` 纯粹是右侧记录类型的缩写，二者在任何上下文中都可以互换。演算的形式定义一直忽略这些缩写。

Java 与许多常用语言不同：类型定义具有更重要的地位。Java 程序中的每种复合类型都有名称；声明局部变量、字段或方法参数的类型时，总是写出名称。`{fst : Nat, snd : Nat}` 这种裸结构类型不能出现在这些位置。

类型名称在 Java 子类型关系中也至关重要。每当通过类或接口声明引入一个新的类型名时，程序员都必须显式说明它继承哪些已有的类或接口；如果新声明的是类，而相应的已有类型是接口，则使用 `implements`（实现），而不是 `extends`（继承）。编译器检查新类或接口提供的操作是否确实扩展各个超类或超接口，这对应带类型 lambda 演算中的记录子类型检查。随后，类型名称之间的子类型关系被定义为“显式声明的直接子类型关系”的自反传递闭包：每个类型都是自身的子类型；如果沿着直接子类型关系经过一步或多步能从类型 S 到达类型 T ，那么 S 也是 T 的子类型。若一个名称没有被声明为另一个名称的子类型，也不能通过这样的关系链到达后者，它就不是后者的子类型。

像 Java 这样名称具有语义作用、子类型关系需要显式声明的系统，称为 名义类型系统 (*nominal type system*)。本书大多数系统不在意名称，而直接按类型结构定义子类型关系，称为结构类型系统 (*structural type system*)。

两种风格各有利弊。名义系统最重要的优点也许是：类型名称不仅用于类型检查，还可在运行期使用。多数名义语言会在运行期对象上附加一个含类型名的头字段；具体表示通常是指向运行期类型描述结构的指针，结构中还包含指向直接超类型的指针。这类类型标签可用于运行期类型测试（例如 Java 的 `instanceof` 与向下转换）、打印、将数据结构序列化为二进制形式以便保存或传输，以及反射。结构系统也能支持运行期类型标签，但它们是额外的独立机制；名义系统则把运行期标签与编译期类型统一起来。结构系统中的相关实现可参见 Glew (1999)、League、Shao 与 Trifonov (1999) 以及 League、Trifonov 与 Shao (2001) 及这些论文所引文献。

名义系统还自然地处理递归类型，也就是定义中提到自身的类型。第二十章将详细讨论递归类型。列表、树等常见结构都需要这种类型；在名义系统中，在 `List` 的定义体里引用 `List` 与引用其他类型名没有区别。相互递归类型也很直接：从一开始便把全部类型名视为已给定，所以即使 A 的定义提到 B ，而 B 的定义又提到 A ，也不必争论“先定义哪一个”。

结构类型系统同样能够处理递归类型。ML 等采用结构类型的高级程序语言通常会把递归类型与其他特性一起封装，使其对程序员而言同样自然。但在用于类型安全性证明的基础演算中，严谨处理递归类型所需的机制可能相当繁重，尤其允许相互递归时更是如此。递归类型在名义系统中几乎无需额外代价，这是一项明显优势。

名义系统中，判断一种类型是否为另一种类型的子类型也几乎不费力。编译器仍须验证程序员声明的子类型关系是否安全；这项验证所做的工作，本质上与结构子类型检查相同。不过，每种类型只需在定义时检查一次，不必在以后每次判断子类型关系时重新检查。名义类型检查器因此较容易获得良好性能。不过在成熟编译器中，两种风格的实际性能差异未必很大，因为工程良好的结构类型检查器会采用哈希驻留等内部表示技术，使结构相同的类型共享同一个节点。因此，大多数子类型检查只需比较一次节点引用即可结束；参见§17.2。

显式子类型声明还常被认为可以防止“偶然包容”：一个值虽与预期类型用途完全不同，却因结构相容而被类型检查器接受。这个优点更有争议，因为单构造子数据类型（见§11.10）或抽象数据类型（见第二十四章）等办法也能防止偶然包容，而且可能更合适。

鉴于运行期类型标签十分有用，递归类型处理又很简单，主流程序语言普遍采用名义类型系统并不意外。程序语言研究文献却几乎完全关注结构类型系统。

一个直接原因是：至少在不考虑递归类型的情况下，结构类型系统更为简洁、优雅。结构类型表达式是封闭实体，自身携带理解其含义所需的全部信息；名义系统则始终依赖一组全局类型名及其定义，形式定义与证明因而更冗长。

更重要的原因是，研究文献常关注更高级的特性，尤其是强大的类型抽象机制：参数多态、抽象数据类型、用户定义的类型算子、函子等，以及实现这些特性的 ML、Haskell 等语言。这些机制很难自然纳入纯名义系统。例如 `List(T)` 无法看作原子名称：程序某处只有一份 `List` 构造子的定义，必须查看它才能理解 `List(T)` 的行为。若干名义语言已经扩展了“泛型”机制（Myers、Bank 与 Liskov, 1997；Agesen、Freund 与 Mitchell, 1997；Bracha、Odersky、Stoutamire 与 Wadler, 1998；Cartwright 与 Steele, 1998；Stroustrup, 1997），但结果不再是纯名义系统，而是两种方法的复杂混合。因此，带高级类型特性的语言设计者往往偏好结构方法。

名义类型系统与结构类型系统的完整关系，至今仍是研究课题。

19.4 定义

下面正式定义 FJ。

语法

FJ 的语法见图19-1。元变量 A, B, C, D, E 表示类名； f, g 表示字段名； m 表示方法名； x 表示参数名； s, t 表示项； u, v 表示值； CL 表示类声明； K 表示构造函数声明； M 表示方法声明。变量集合包含特殊变量 `this`，但方法形参不得命名为 `this`。它在每项方法声明中隐式受约束。方法调用求值规则 E-INVKNEW 除了用实参值替换形参，还会用合适的接收者对象替换 `this`。

FJ

语法与子类型关系

语法	子类型关系
$CL ::= \text{class } C \text{ extends } D \{ \overline{C} \ \overline{f}; \ K \ \overline{M} \}$ 类声明 $K ::= C(\overline{C} \ \overline{f})\{\text{super}(\overline{f}); \text{this}.\overline{f} = \overline{f}; \}$ 构造函数声明 $M ::= C \ m(\overline{C} \ \overline{x})\{\text{return } t; \}$ 方法声明 $t ::= x$ 变量 $\quad \quad t.f$ 字段访问 $\quad \quad t.m(\overline{t})$ 方法调用 $\quad \quad \text{new } C(\overline{t})$ 对象创建 $\quad \quad (C)t$ 类型转换 $v ::= \text{new } C(\overline{v})$ 值	$C <: D$ $\frac{}{C <: C} \text{ S-REFL}$ $\frac{C <: D \quad D <: E}{C <: E} \text{ S-TRANS}$ $\frac{CT(C) = \text{class } C \text{ extends } D \{ \dots \}}{C <: D} \text{ S-CLASS}$

图 19-1: Featherweight Java: 语法与子类型关系

用 $\overline{f}$ 缩写 $f_1, \dots, f_n$; $\overline{v}$ 、 $\overline{x}$ 、 $\overline{t}$ 等同理。 $\overline{M}$ 表示不带逗号的方法声明序列。成对序列也采用同样的缩写, 例如 $\overline{C} \ \overline{f}$ 表示 $C_1 f_1, \dots, C_n f_n$ 。字段声明序列、形参名序列和方法声明序列都不含重名项。

带分号的字段声明序列和构造函数中的赋值序列分别展开为

$$\begin{aligned} \overline{C} \ \overline{f}; &= C_1 f_1; \dots C_n f_n; \\ \text{this}.\overline{f} = \overline{f}; &= \text{this}.f_1 = f_1; \dots \text{this}.f_n = f_n; \end{aligned}$$

这里每一项之后都保留分号, 包括序列的最后一项。

类声明

$$\text{class } C \text{ extends } D \{ \overline{C} \ \overline{f}; \ K \ \overline{M} \}$$

引入名为 C 、超类为 D 的类。新类包含类型为 $\overline{C}$ 的字段 $\overline{f}$ 、一个构造函数 K 和一组方法 $\overline{M}$ 。 C 声明的实例变量会追加到 D 及其各超类声明的变量之后, 名称不得重复。⁴方法则可以覆盖 D 中已有同名方法, 也可以增加 C 特有的新功能。

对于上面的类声明, 构造函数的参数、**super** 调用和字段赋值必须按如下方式对应:

$$C(\overline{D} \ \overline{g}, \overline{C} \ \overline{f})\{\text{super}(\overline{g}); \text{this}.\overline{f} = \overline{f}; \}$$

其中 $\overline{D} \ \overline{g}$ 是继承字段, $\overline{C} \ \overline{f}$ 是当前类新声明的字段。构造函数的形式完全由 C 及其超类的字段声明决定: 形参数量必须恰好等于全部实例变量的数量; 函数体先把对应形参传给超类构造函数以初始化继承字段, 再把其余形参赋给本类声明的同名字段。图19-4的类类型规则负责检查这些约束。完整 Java 要求子类构造函数在超类构造函数有参数时调用它。FJ 保留这项要求, 是为了让符合 FJ 语法的构造函数同时也是合法的 Java 构造函数。若不追求这一点, 可以删去 **super** 调用, 让每个构造函数直接初始化全部实例变量。

方法声明

$$D \ m(\overline{C} \ \overline{x})\{\text{return } t; \}$$

⁴完整 Java 允许子类重新声明超类字段, 并让新声明在当前类及其子类中遮蔽旧声明; FJ 省略这项特性。

引入结果类型为 D 、形参类型为 $\bar{C}$ 的方法 m 。方法体只包含一条 `return  $t$` ；语句，其中 t 可以是任意 FJ 项； $\bar{x}$ 和特殊变量 `this` 都在 t 中受约束。

类表 CT 是从类名 C 到类声明 CL 的映射。一个 FJ 程序由类表 CT 和一个顶层待求值项 t 组成，记为 (CT, t) 。FJ 的求值过程不会新增或修改类声明，因此同一程序始终使用开始时给定的类表。为简化后文记号，各项定义和判断不再重复写出 CT ，但类与方法的查找仍以它为依据。

每个类都用 `extends` 声明超类；那么 `Object` 的超类是什么？本书采用最简单的处理：把 `Object` 作为特殊类名，它的定义不出现在类表中。查找字段的辅助函数为 `Object` 设有特例，返回空字段序列 $\bullet$ ；同时假定它没有方法。⁵

从类表即可读出类之间的子类型关系。 $C <: D$ 表示 C 是 D 的子类型；这个关系是 CT 中 `extends` 子句所给直接子类关系的自反传递闭包，形式定义见图19-1。

类表须满足几项基本条件：

1. 对每个 $C \in \text{dom}(CT)$ ， $CT(C)$ 必须形如 `class  $C$  extends  $D\{\dots\}$` ；
2. $\text{Object} \notin \text{dom}(CT)$ ；
3. 除 `Object` 外，类表中出现的每个类名都属于 $\text{dom}(CT)$ ；
4. `extends` 关系没有环，并且任何声明 `class  $C$  extends  $D\{\dots\}$` 都满足 $C \neq D$ 。因此，由类表中的 `extends` 声明确定的 $<:$ 关系是反对称的：如果 $C <: D$ 且 $D <: C$ ，那么必有 $C = D$ 。

译者注：作者勘误为第四项补充了 $C \neq D$ 条件。反对称性只能排除不同类之间的继承环，不能排除 `class  $C$  extends  $C\{\dots\}$` 这样的自环，因此还须单独禁止类直接继承自身。

类表允许定义递归类型：类 A 的方法或实例变量类型可以使用名称 A 。类定义之间也可以相互递归。

练习 19.4.1 (★). 类比带子类型化 lambda 演算中的 S-Top，似乎还应有一条规则声明 `Object` 是每个类的超类型。这里为什么不需要？

[查看原书附录 A 中的 19.4.1 解答](#)

辅助定义

类型规则与求值规则需要图19-2中的几个辅助定义。 $\text{fields}(C)$ 返回 C 及其所有超类声明的字段，把每个字段类型与字段名配成序列 $\bar{C} \bar{f}$ 。 $\text{mtype}(m, C) = \bar{B} \rightarrow B$ 返回 C 中方法 m 的形参类型序列与结果类型。 $\text{mbody}(m, C) = (\bar{x}, t)$ 返回形参名序列与方法体。谓词 $\text{override}(m, D, \bar{C} \rightarrow C_0)$ 判断是否可以在 D 的子类中声明这个类型的方法 m ：若超类已有同名方法，其类型必须完全相同。

⁵完整 Java 的 `Object` 实际包含若干方法，FJ 将其忽略。

FJ

辅助定义

字段查找

$$\boxed{fields(C) = \overline{C} \ \overline{f}}$$

$$fields(Object) = \bullet$$

$$\frac{CT(C) = \text{class } C \text{ extends } D \{ \overline{C} \ \overline{f}; K \overline{M} \} \quad fields(D) = \overline{D} \ \overline{g}}{fields(C) = \overline{D} \ \overline{g}, \overline{C} \ \overline{f}}$$

方法类型查找

$$\boxed{mtype(m, C) = \overline{B} \rightarrow B}$$

$$\frac{CT(C) = \text{class } C \text{ extends } D \{ \overline{C} \ \overline{f}; K \overline{M} \} \quad B \ m(\overline{B} \ \overline{x}) \{ \text{return } t; \} \in \overline{M}}{mtype(m, C) = \overline{B} \rightarrow B}$$

$$\frac{CT(C) = \text{class } C \text{ extends } D \{ \dots \} \quad m \text{ 未在 } C \text{ 中声明}}{mtype(m, C) = mtype(m, D)}$$

方法体查找

$$\boxed{mbody(m, C) = (\overline{x}, t)}$$

$$\frac{CT(C) = \text{class } C \text{ extends } D \{ \overline{C} \ \overline{f}; K \overline{M} \} \quad B \ m(\overline{B} \ \overline{x}) \{ \text{return } t; \} \in \overline{M}}{mbody(m, C) = (\overline{x}, t)}$$

$$\frac{CT(C) = \text{class } C \text{ extends } D \{ \dots \} \quad m \text{ 未在 } C \text{ 中声明}}{mbody(m, C) = mbody(m, D)}$$

有效的方法覆盖

$$\boxed{override(m, D, \overline{C} \rightarrow C_0)}$$

$$\frac{mtype(m, D) = \overline{D} \rightarrow D_0 \implies \overline{C} = \overline{D} \text{ 且 } C_0 = D_0}{override(m, D, \overline{C} \rightarrow C_0)}$$

图 19-2: Featherweight Java: 辅助定义

求值

FJ 使用标准的按值调用操作语义，见图19-3。字段访问、方法调用和类型转换三条计算规则已在§19.2解释；其余规则规定按值调用次序。正常终止时得到的值，都是参数已完全求值的对象创建项 $\text{new } C(\overline{v})$ 。

FJ

求值

$$\boxed{t \longrightarrow t'}$$

$$\frac{fields(C) = \overline{C} \ \overline{f}}{(\text{new } C(\overline{v})).f_i \longrightarrow v_i} \text{ E-PROJNEW}$$

$$\frac{mbody(m, C) = (\overline{x}, t_0)}{(\text{new } C(\overline{v})).m(\overline{u}) \longrightarrow [\overline{x} \mapsto \overline{u}, \text{this} \mapsto \text{new } C(\overline{v})]t_0} \text{ E-INVKNW}$$

$$\frac{C <: D}{(D)(\text{new } C(\overline{v})) \longrightarrow \text{new } C(\overline{v})} \text{ E-CASTNEW}$$

$$\frac{t_0 \longrightarrow t'_0}{t_0.f \longrightarrow t'_0.f} \text{ E-FIELD}$$

$$\frac{t_0 \longrightarrow t'_0}{t_0.m(\overline{t}) \longrightarrow t'_0.m(\overline{t})} \text{ E-INVK-RECV}$$

$$\frac{t_i \longrightarrow t'_i}{v_0.m(\overline{v}, t_i, \overline{t}) \longrightarrow v_0.m(\overline{v}, t'_i, \overline{t})} \text{ E-INVK-ARG}$$

$$\frac{t_i \longrightarrow t'_i}{\text{new } C(\overline{v}, t_i, \overline{t}) \longrightarrow \text{new } C(\overline{v}, t'_i, \overline{t})} \text{ E-NEW-ARG}$$

$$\frac{t_0 \longrightarrow t'_0}{(C)t_0 \longrightarrow (C)t'_0} \text{ E-CAST}$$

图 19-3: Featherweight Java: 求值

运行期类型转换会检查对象的实际类是否为转换目标的子类型。若是，转换被删去，结果仍是对象本身；若不是，转换失败。Java 会为失败转换抛出异常；FJ 没有异常机制，所以失败转换只是卡住。

类型规则

项、方法声明与类声明的类型规则见图19-4。环境 Γ 是从变量到类型的有限映射。 $\Gamma \vdash t : C$ 读作“在环境 Γ 中，项 t 具有类型 C ”。除类型转换有三条规则外，类型规则按句法导向，每种项对应一条。⁶构造函数与方法调用都检查每个实参类型是否为相应形参声明类型的子类型。为简化记号，对一组项或类型的判断按对应位置逐项理解。例如， $\Gamma \vdash \bar{t} : \bar{C}$ 表示每个 t_i 都具有对应类型 C_i ，而 $\bar{C} <: \bar{D}$ 表示每个 $C_i <: D_i$ 。

FJ

类型规则

项的类型规则	方法声明
$\frac{x : C \in \Gamma}{\Gamma \vdash x : C} \text{ T-VAR}$ $\frac{\Gamma \vdash t_0 : C_0 \quad \text{fields}(C_0) = \bar{C} \bar{f}}{\Gamma \vdash t_0.f_i : C_i} \text{ T-FIELD}$ $\frac{\Gamma \vdash t_0 : C_0 \quad \text{mtype}(m, C_0) = \bar{D} \rightarrow C \quad \Gamma \vdash \bar{t} : \bar{C} \quad \bar{C} <: \bar{D}}{\Gamma \vdash t_0.m(\bar{t}) : C} \text{ T-INVK}$ $\frac{\text{fields}(C) = \bar{D} \bar{f} \quad \Gamma \vdash \bar{t} : \bar{C} \quad \bar{C} <: \bar{D}}{\Gamma \vdash \text{new } C(\bar{t}) : C} \text{ T-NEW}$ $\frac{\Gamma \vdash t_0 : D \quad D <: C}{\Gamma \vdash (C)t_0 : C} \text{ T-UCAST}$ $\frac{\Gamma \vdash t_0 : D \quad C <: D \quad C \neq D}{\Gamma \vdash (C)t_0 : C} \text{ T-DCAST}$ $\frac{\Gamma \vdash t_0 : D \quad D \not<: C \quad \text{stupid warning}}{\Gamma \vdash (C)t_0 : C} \text{ T-SCAST}$	$\frac{\bar{x} : \bar{C}, \text{this} : C \vdash t_0 : E_0 \quad E_0 <: C_0 \quad \text{CT}(C) = \text{class } C \text{ extends } D \{ \dots \} \quad \text{override}(m, D, \bar{C} \rightarrow C_0)}{C_0 \text{ m}(\bar{C} \bar{x}) \{ \text{return } t_0; \} \text{ OK in } C}$ 类声明 $\frac{K = C(\bar{D} \bar{g}, \bar{C} \bar{f}) \{ \text{super}(\bar{g}); \text{this}.\bar{f} = \bar{f}; \} \quad \text{fields}(D) = \bar{D} \bar{g} \quad \bar{M} \text{ OK in } C}{\text{class } C \text{ extends } D \{ \bar{C} \bar{f}; K \bar{M} \} \text{ OK}}$

图 19-4: Featherweight Java：类型规则

译者注：图中的三条类型转换规则可以通过比较源类型 D 与目标类型 C 来区分。T-UCAST 要求 $D <: C$ ，表示从子类型转换到超类型；这种转换在运行时必定成功。T-DCAST 要求 $C <: D$ 且 $C \neq D$ ，表示从超类型转换到真子类型；通过类型检查只说明转换可能成功，最终仍须检查对象的运行时类。T-SCAST 处理 C 与 D 互不为子类型的荒谬转换。*stupid warning* 虽然写在前提的位置，却不是需要由其他规则证明的程序性质，而是标明这份类型推导使用了仅为元理论证明保留的规则；含有该标记的推导不对应合法的 Java 类型检查结果。

FJ 的一项小技术处理是引入“荒谬转换”。类型转换有三类：向上转换时，源项的类型是目标类型的子类；向下转换时，目标类型是源项类型的子类；荒谬转换中，目标与源项类型互不相关。Java 编译器会拒绝包含荒谬转换的项，但若要用小步语义下的类型保持性定理

⁶这里沿用完整 Java 的做法，以算法形式给出类型关系。对完整 Java 而言，这项选择实际上是必需的；见练习19.4.6。

表述 FJ 的安全性，就必须在 FJ 的内部类型系统中允许它，因为一个合理的项可能单步求值成含荒谬转换的项。例如

$$(A)(\text{Object}) \text{ new } B() \longrightarrow (A) \text{ new } B()$$

其中 A, B 沿用§19.2的定义，二者都继承 `Object`，但彼此没有子类型关系。左式中，`new B()` 先向上转换为 `Object`，所以内层表达式的静态类型是 `Object`；由于 $A <: \text{Object}$ ，外层再从 `Object` 向下转换为 A ，符合规则 T-DCAST，因而整个左式能够通过类型检查。求值时，内层的向上转换首先成功并被消去，整个表达式随即变成右式。此时转换直接发生在互不相关的 B 与 A 之间，因而属于荒谬转换。这个例子说明，一个能够通过类型检查的项经过一步求值，确实可能产生含荒谬转换的项。

规则 T-SCAST 用前提 *stupid warning* 标出这种特殊性。只有不使用该规则的 FJ 类型推导，才对应合法的 Java 类型检查结果。

方法声明的判断写作 $M \text{ OK in } C$ ，表示“方法声明 M 出现在类 C 中时是良构的”。检查方法体时，环境包含各形参及其声明类型，并加入 $\text{this} : C$ 。类声明的判断写作 $CL \text{ OK}$ ，检查构造函数是否把超类字段传给 `super`、是否初始化本类字段，以及类中的每项方法声明是否良构。

项的类型可能依赖它调用的方法类型，而方法类型检查又依赖方法体这项的类型，看起来像是循环。不过每项方法的类型都显式声明，循环由此断开。因此，类型检查可以分两步进行：先读取整张类表，使所有字段和方法的声明类型都可供查询；再依据这些声明，逐一检查各个类的构造函数和方法体。各类按什么次序检查并不重要，只要最终检查完所有类声明即可。

练习 19.4.2 (★★). FJ 的若干设计选择，是为了保证它成为 Java 的子集：FJ 中良类型或非良类型的程序，在 Java 中也相应地良类型或非良类型；良类型程序在两门语言中的行为相同。若不再要求这一点，只想设计一门类似 Java 的核心演算，可以怎样简化或改进 FJ？

[查看原书附录 A 中的 19.4.2 解答](#)

练习 19.4.3 (推荐, ★★★, ⇔). 为简化表述，FJ 省略了修改对象字段的操作。以第十三章对引用的处理为模型，为 FJ 加入字段赋值。

[查看练习 19.4.3 的译者参考解答](#)

练习 19.4.4 (★★★, ⇔). 以第十四章对异常的处理为模型，为 FJ 加入 Java 的 `throw` 与 `try` 所对应的构造。

[查看练习 19.4.4 的译者参考解答](#)

练习 19.4.5 (★★, ⇔). 与完整 Java 一样，FJ 的类型规则采用算法式表述：系统没有统一的包容规则，而把所需的子类型检查直接写进其他规则的前提。能否把这个系统改写成更偏声明式的形式，删去多数或全部这类前提，改用一条包容规则？

[查看练习 19.4.5 的译者参考解答](#)

练习 19.4.6 (★★★). 完整 Java 同时提供类与接口；接口只规定方法类型，不给出实现。接口允许更丰富、不再呈树状的子类型关系：每个类只有一个超类，从中继承实例变量与方法体，却可以实现任意多个接口。接口与 Java 条件表达式 $t_1 ? t_2 : t_3$ 的相互作用，迫使类型关系采用算法表述，使每个可赋型项获得唯一的最小类型。

1. 按 Java 的风格为 FJ 加入接口；
2. 证明加入接口后，子类型关系未必对并封闭；回顾§16.3，并的存在对条件式的最小类型性质至关重要；
3. Java 的条件表达式采用什么类型规则？这条规则合理吗？

[查看原书附录 A 中的 19.4.6 解答](#)

练习 19.4.7 (★★★). FJ 包含 Java 的 `this`，却省略了 `super`。说明怎样加入 `super`。

[查看原书附录 A 中的 19.4.7 解答](#)

19.5 性质

FJ 满足标准的类型保持性结论；由于算法类型在求值后可能变得更精确，结论允许新类型是原类型的子类型。

定理 19.5.1 (保持性). 若 $\Gamma \vdash t : C$ 且 $t \longrightarrow t'$ ，则存在 $C' <: C$ ，使 $\Gamma \vdash t' : C'$ 。

证明. 练习 (★★★)。 □

[查看原书附录 A 中的 19.5.1 解答](#)

还可以证明标准进展性定理的一种变体：良类型程序只有在某次类型转换无法执行时才可能卡住。为了准确描述类型转换可能在程序的哪个求值位置失败，下面引入求值上下文。

引理 19.5.2. 设 t 是良类型项。

1. 若 $t = (\text{new } C_0(\bar{t})).f$ ，则存在字段序列 $\bar{C} \bar{f}$ ，使 $\text{fields}(C_0) = \bar{C} \bar{f}$ ，且 $f \in \bar{f}$ 。
2. 若 $t = (\text{new } C_0(\bar{t})).m(\bar{s})$ ，则存在 $\bar{x}, t_0$ ，使 $\text{mbdy}(m, C_0) = (\bar{x}, t_0)$ ，且 $|\bar{x}| = |\bar{s}|$ 。

证明. 由相应类型规则与辅助定义直接得到。 □

定义 19.5.3. FJ 的求值上下文由以下文法给出：

$E ::= []$	孔
$E.f$	字段访问
$E.m(\bar{t})$	方法调用：接收者
$v.m(\bar{v}, E, \bar{t})$	方法调用：实参
$\text{new } C(\bar{v}, E, \bar{t})$	对象创建：实参
$(C)E$	类型转换

文法第一行的 $[]$ 称为“孔”：它是描述程序结构时使用的占位符，并不是 FJ 程序自身的语法。把一项中下一步要计算的子项暂时取出，留下的位置就是孔；带有恰好一个孔的外层项结构便是求值上下文。对于求值上下文 E ， $E[t]$ 表示把它的孔填入 t 后得到的普通项。求值上下文由此标出下一步应当规约的子项：若 $t \longrightarrow t'$ ，则存在唯一的 E, r, r' ，使 $t = E[r]$ 、 $t' = E[r']$ ，并且 $r \longrightarrow r'$ 是 E-PROJNEW、E-INVKNEW 或 E-CASTNEW 的实例。

定理 19.5.4 (进展性). 设 t 是闭的良类型范式。则以下两种情况之一成立：

1. t 是值；
2. 存在求值上下文 E 、类 C, D 与值序列 $\bar{v}$ ，使

$$t = E[(C)(\text{new } D(\bar{v}))] \quad \text{且} \quad D \not\prec C$$

也就是说，项卡在一次运行时检查失败的类型转换上。

证明. 对类型推导作直接归纳。 □

还可以稍微加强进展性：若 t 只包含向上转换，它就不会卡住；若原始程序只包含向上转换，求值也不会产生非向上转换。不过，一般程序需要用类型转换降低对象的静态类型，所以必须接受转换可能在运行期失败。完整 Java 不会因转换失败而让整个程序停止，而是抛出可由外围处理器捕获的异常。

练习 19.5.5 ($\star\star\star, \rightarrow$). 以本书某个 lambda 演算类型检查器为起点，实现 Featherweight Java 的类型检查器与解释器。

[查看练习 19.5.5 的译者参考解答](#)

练习 19.5.6 ($\star\star\star\star, \rightarrow$). FJ 原始论文还形式化了 GJ 风格的多态类型。请以[练习19.5.5](#)的类型检查器与解释器为基础，加入这些特性。(尝试本题前，需要先阅读[第二十三章](#)、[第二十五章](#)、[第二十六章](#)与[第二十八章](#)。)

[查看练习 19.5.6 的译者参考解答](#)

19.6 对象编码与作为基本构造的对象

至此，我们看到了两种不同的简单面向对象语言语义与类型表述方式。[第十八章](#) 组合简单类型 lambda 演算中的记录、引用和子类型化，给出了对象、类与继承的编码；本章则直接定义一门以对象和类为基本机制的语言。

两种方法各有用途。研究对象编码能揭示封装与复用背后的基本机制，并把它们与目标相近的其他机制比较；还可帮助理解编译器如何把对象翻译到更低层的语言，以及对象同其他语言特性的相互作用。把对象视为基本构造，则可以直接讨论其操作语义与类型行为，更适合高级语言设计和文档说明。

最终，我们当然希望同时拥有这两种视角。首先，定义一门以对象、类等为基本构造，并有自身类型规则和操作语义的高级语言。其次，给出从这门语言到低层语言的翻译；目标语言可以只提供记录和函数，甚至只提供寄存器、指针与指令序列。最后，证明这项翻译正确实现了高级语言，即它保留原语言的求值与类型性质。许多工作已经完成过不同版本的这一任务：对 FJ 本身可参见 League、Trifonov 与 Shao (2001)；对其他面向对象核心演算可参见 Hofmann 与 Pierce (1995b)、Bruce (1994, 2002)、Abadi、Cardelli 与 Viswanathan (1996) 等。

19.7 注记

本章改编自 Igarashi、Pierce 与 Wadler (1999) 的 FJ 原始论文。表述上的主要区别是：为与全书一致，本章采用按值调用操作语义，而原论文采用不确定的归约关系。

已有多项工作证明 Java 子集的类型安全性。Drossopoulou、Eisenbach 与 Khurshid (1999) 使用一种后来由 Syme (1997) 借助形式化工具检验的技术，最早为不含并发机制的 Java 的一个较大子集证明了安全性。他们也使用小步操作语义，但完全省略类型转换，避开了荒谬转换问题。Nipkow 与 Oheimb (1998) 则为一个更大的核心语言给出了经过形式化工具检验的安全性证明；其语言包含类型转换，却使用大步操作语义，同样绕开该问题。Flatt、Krishnamurthi 与 Felleisen (1998a, 1998b) 使用小步语义，形式化同时含赋值与类型转换的语言，并像 FJ 一样处理荒谬转换。其系统约为 FJ 的三倍大，安全性证明相应更长，但复杂度相近。

这三项研究中，Flatt、Krishnamurthi 与 Felleisen 的工作在设计目标上最接近 FJ：它们都针对特定任务选择尽可能小的核心演算，只保留与任务相关的 Java 特性。他们的任务是分析带 Common Lisp 风格 mixin 的 Java 扩展。另两套系统则希望覆盖尽可能大的 Java 子集，因为主要目标是证明 Java 本身的安全性。

面向对象语言基础研究中，许多论文形式化了基于类的语言：有的把类作为基本构造 (Wand, 1989a; Bruce, 1994; Bono、Patel、Shmatikov 与 Mitchell, 1999b; Bono、Patel 与 Shmatikov, 1999a)，有的把类翻译为低层机制 (Fisher 与 Mitchell, 1998; Bono 与 Fisher, 1998; Abadi 与 Cardelli, 1996; Pierce 与 Turner, 1994)。

另一条路线以方法覆盖或委托 (Ungar 与 Smith, 1987) 取代类，让单个对象可以从其他对象继承行为；由于概念更少，所得演算通常比基于类的演算简单。Abadi 与 Cardelli (1996) 的对象演算是其中发展最成熟、最著名的一种；Fisher、Honsell 与 Mitchell (1994) 还发展了另一种常用演算。Castagna、Ghelli 与 Longo (1995) 则形式化了称为多方法的另一套对象、类与继承方法；更多相关引文见 第18.1节“封装”条目后的脚注²。

每一门大型语言内部，都有一门小语言在奋力挣脱。⁷

Inside every large language is a small language struggling to get out....

——Igarashi、Pierce 与 Wadler (1999)

⁷译者注：第一则引文改写自紧随其后的 Hoare 名言，将其中的“程序”换成了“语言”，借此点出 FJ 从 Java 中抽取小型核心语言的设计思路。

每一个大型程序内部，都有一个小程序在奋力挣脱。

Inside every large program is a small program struggling to get out....

——*Tony Hoare, Efficient Production of Large Programs (1970)*

我很胖，但内在的我很瘦。

你是否想过，每个胖子身体里都住着一个瘦子？

I'm fat, but I'm thin inside.

Has it ever struck you that there's a thin man inside every fat man?

——*George Orwell, Coming Up for Air (1939)*

第四部分

递归类型

第二十章 递归类型

概览

§11.12曾把简单类型系统扩展为带有类型构造子 $\text{List}(T)$ 的系统；该类型的元素，是由 T 型元素组成的列表。列表只是众多常见数据结构中的一例。队列、二叉树、带标签树、抽象语法树等结构都可以任意增长，却具有简单而规则的形态。例如， $\text{List}(\text{Nat})$ 中的元素要么是 nil ，要么是一个数与另一个 $\text{List}(\text{Nat})$ 组成的二元对，后者也称 cons 单元 (*cons cell*)。为每一种结构分别加入一种原始语言特性显然不合适；我们需要一种通用机制，在需要时用较简单的成分定义它们。这个机制就是 递归类型 (*recursive type*)。¹

仍以自然数列表为例。借助§11.10的变体和 §11.7的元组，可以先写出：

```
NatList = <nil:Unit, cons:{...,...}>;
```

空列表标签 nil 已经包含识别空列表所需的全部信息，因此它携带的数据只需是 unit 。 cons 标签携带的值则是一个数与另一条列表组成的二元对。其第一分量的类型为 Nat ：

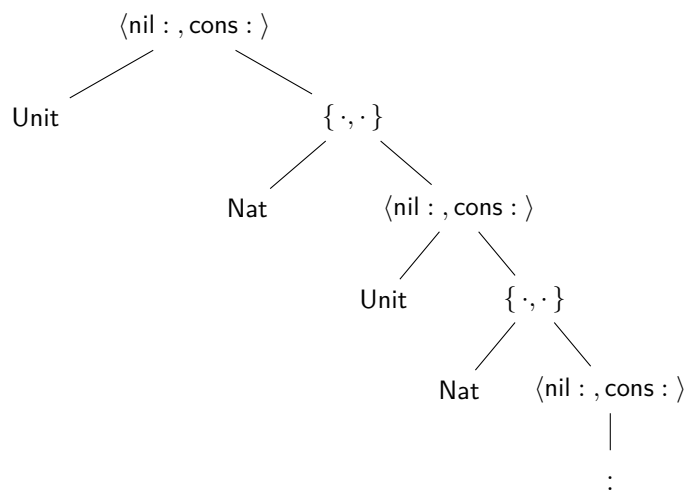
```
NatList = <nil:Unit, cons:{Nat,...}>;
```

第二分量仍是自然数列表，也就是我们正在定义的 NatList 本身：

```
NatList = <nil:Unit, cons:{Nat,NatList}>;
```

这个等式并非普通的缩写定义：等号右侧又用到了尚在定义中的名称 NatList ，因而不能把它看成给一个含义已知的类型表达式另取名字。更恰当的理解，是它规定了下面所示的无限树。

¹本章不讨论怎样为元素类型任意的列表给出一份通用定义。为此还需要类型算子；第二十九章会引入这一机制。



规定这棵无限类型树的递归等式，与§5.2定义递归阶乘函数时使用的等式相似。这里也可以把等式中的“回路”移到右侧，引入显式的类型递归算子 μ ，把等式写成真正的定义：²

```
NatList =  $\mu$ X. <nil:Unit, cons:{Nat,X}>;
```

直观地说，令 T 表示上面定义的整个递归类型。这个类型由下面的不动点方程刻画：

$$T = \langle \text{nil} : \text{Unit}, \text{cons} : \{\text{Nat}, T\} \rangle$$

也就是说， T 展开一层后仍会在 **cons** 分支中包含自身；**NatList** 正是这个无限类型的名称。

§20.2将看到，递归类型有两种略有差异的形式化方式，即等递归表示和同构递归表示。二者的差别在于，程序员需要用类型标注给类型检查器提供多少帮助。下一节的程序示例采用较为轻便的等递归表示。³

20.1 示例

列表

先完成前面的自然数列表示例。要操作列表，需要空列表常量 **nil**，把元素加入已有列表开头的构造函数 **cons**，判断列表是否为空的 **isnil**，以及分别取出非空列表首元素和表尾的 **hd** 与 **tl**。图11-13曾把它们作为内建操作；这里要用更简单的构造定义它们。

nil 与 **cons** 的定义直接来自 **NatList** 的二标签变体结构：

```
nil = <nil=unit> as NatList;
► nil : NatList

cons = lambda n:Nat. lambda l:NatList.
```

²这样写使我们不必先给递归类型命名。不过，显式命名递归类型也有优点；参见§19.3关于名义类型系统与结构类型系统的讨论。

³本章研究带递归类型的简单类型演算。示例还会使用前面章节的多种特性，原书配套检查器为 **fullequirec**；§20.2对应的检查器为 **fullisorec**。

```

    <cons={n,l}> as NatList;
► cons : Nat -> NatList -> NatList

```

回顾§11.10, `<l=t> as T` 给值 `t` 加上标签 `l`, 再把它注入变体类型 `T`。这里的类型检查器会把递归类型 `NatList` 自动展开为 `<nil:Unit, cons:{Nat,NatList}>`。

其余基本操作都要检查列表的结构, 再取出相应分量, 因此都可以用 `case` 实现:

```

isnil = lambda l:NatList. case l of
    <nil=u> ==> true
  | <cons=p> ==> false;
► isnil : NatList -> Bool

hd = lambda l:NatList. case l of
    <nil=u> ==> 0
  | <cons=p> ==> p.1;
► hd : NatList -> Nat

tl = lambda l:NatList. case l of
    <nil=u> ==> l
  | <cons=p> ==> p.2;
► tl : NatList -> NatList

```

这里约定空列表的 `hd` 为 0, `tl` 仍为空列表; 也可以在这两种情况下抛出异常。

把这些定义组合起来, 可以写出递归函数 `sumlist`, 计算列表中全部元素的和:

```

sumlist = fix (lambda s:NatList->Nat. lambda l:NatList.
    if isnil l then 0
    else plus (hd l) (s (tl l)));
► sumlist : NatList -> Nat

mylist = cons 2 (cons 3 (cons 5 nil));
sumlist mylist;
► 10 : Nat

```

虽然 `NatList` 自身表示一棵无限类型树, 它的每个值却都是有限列表: 二元对、标签和按值调用的 `fix` 都无法构造真正无限大的数据结构。

练习 20.1.1 (★★). 一种带标签二叉树的定义规定: 树要么是没有标签的叶子, 要么是带有自然数标签和两棵子树的内部节点。请定义类型 `NatTree`, 并给出构造、分解和检查树的适当操作。再写一个函数, 对树进行深度优先遍历, 返回沿途遇到的标签列表。请用 `fullequirec` 检查这些代码。

[查看原书附录 A 中的 20.1.1 解答](#)

永远“吃不饱”的函数

下面这个例子展示递归类型一种稍微微妙的用法。先定义：

```
Hungry =  $\mu A$ . Nat  $\rightarrow$  A;
```

将这个递归类型展开一层，可得

$$\text{Hungry} = \text{Nat} \rightarrow \text{Hungry}$$

因此，Hungry 类型的值可以看作一个函数：它每接受一个自然数，返回的仍是 Hungry 类型的函数，可以继续接受更多自然数。这样的值可以用 fix 构造：

```
f = fix (lambda f:Nat->Hungry. lambda n:Nat. f);  
► f : Hungry
```

连续传入任意多个自然数，得到的仍是一个 Hungry 类型的函数：

```
f 0 1 2 3 4 5;  
► <fun> : Hungry
```

译者注：把 fix 中作为递归参数的 f 临时改名为 self，并令

$$G = \lambda \text{self} : \text{Nat} \rightarrow \text{Hungry}. \lambda n : \text{Nat}. \text{self}$$

则上面的 f 就是 fix G。根据 fix 的求值规则 E-FixBeta，应当把 fix G 直接替换到函数体中的 self：

$$\text{fix } G \longrightarrow [\text{self} \mapsto \text{fix } G] (\lambda n : \text{Nat}. \text{self}) = \lambda n : \text{Nat}. \text{fix } G$$

因此，fix G 求值为一个函数：这个函数接收自然数后，函数体又返回 fix G，随后再次得到同样形式的函数值。函数应用按左结合解析，因而 f 0 1 2 会依次接收三个参数，每次都返回行为相同的函数。传入有限多个参数后，求值会结束并得到函数值；这里并没有发生无限求值。这个简单例子展示了递归类型也能形成无限展开的函数类型，并为下一小节的流类型作铺垫。

流

Hungry 的一个实用变体是流 (stream)：它可以接受任意多个 unit；每次都返回一个数，以及可以继续使用的流。

```
Stream =  $\mu A$ . Unit  $\rightarrow$  {Nat, A};
```

可以为流定义两个“分解操作”。若 s 是流，hd s 是把 unit 传给它时返回的第一个数，tl s 则是同一次调用返回的新流：

```
hd = lambda s:Stream. (s unit).1;  
► hd : Stream -> Nat  
  
tl = lambda s:Stream. (s unit).2;  
► tl : Stream -> Stream
```

构造流时仍使用 `fix`:

```
upfrom0 =
  fix (lambda f:Nat->Stream. lambda n:Nat. lambda _:Unit.
      {n,f (succ n)}) 0;
► upfrom0 : Stream

hd upfrom0;
► 0 : Nat

hd (tl (tl (tl upfrom0)));
► 3 : Nat
```

译者注: `lambda _:Unit.` 把一个流节点的计算封装成等待调用的函数。这里的 `unit` 不携带数据；它只有一个可能的值，作用是通知流“现在展开一层”。例如，向 `upfrom0` 传入 `unit`，才会得到当前数字 0 和从 1 开始的后续流。

这种封装对于本章采用的按值调用求值策略很重要。若直接把流写成递归二元组 $\mu A.\{\text{Nat}, A\}$ ，为了构造当前二元组，求值器必须先求出第二分量；第二分量又会立即要求构造再下一个节点，计算将无限展开，连第一个节点也无法返回。加入 `Unit` 参数后，后续节点先成为一个 `lambda` 抽象，也就是一个已经完成求值的函数值；只有 `hd` 或 `tl` 以 `s unit` 调用它时，下一层计算才会发生。

练习 20.1.2 (推荐, ★★). 定义一个流，依次产生 Fibonacci 数列的元素 1, 1, 2, 3, 5, 8, 13, ...。

[查看原书附录 A 中的 20.1.2 解答](#)

还可以把流推广成一种简单的进程 (*process*): 它接受一个数，再返回一个数和一个新进程。这里的“进程”泛指能够反复接收输入、产生输出并返回后继状态的计算对象，不特指操作系统管理的进程。

```
Process =  $\mu A.$  Nat->{Nat,A};
```

例如，下面的进程在每一步都返回迄今收到的全部数字之和：

```
p = fix (lambda f:Nat->Process. lambda acc:Nat. lambda n:Nat.
    let newacc = plus acc n in
    {newacc, f newacc}) 0;
► p : Process
```

与流类似，可以再定义两个辅助函数来与进程交互：

```
curr = lambda s:Process. (s 0).1;
► curr : Process -> Nat

send = lambda n:Nat. lambda s:Process. (s n).2;
► send : Nat -> Process -> Process

curr (send 20 (send 3 (send 5 p)));
► 28 : Nat
```

最后一行依次把 5、3 和 20 送给进程 `p`，最后一次交互得到 28。

对象

稍微调整上一个例子，便得到另一种熟悉的数据交互方式：对象。例如，下面的计数器对象保存一个数，并允许查询或递增这个数：

```
Counter =  $\mu$ C. {get:Nat, inc:Unit->C};
```

这里的对象是纯函数式的，类似第十九章而不同于第十八章：向计数器发送 `inc` 消息不会在原对象内部修改状态，而会返回一个内部状态已经加一的新对象。递归类型保证返回对象与原对象具有完全相同的类型。

这种对象与前述进程的差别只在观察方式：对象是递归定义的记录，其中包含函数；进程则是递归定义的函数，返回一个二元组。记录可以很自然地增加多个操作，例如再加入递减：

```
Counter =  $\mu$ C. {get:Nat, inc:Unit->C, dec:Unit->C};
```

和前面的例子一样，可以用不动点组合子创建计数器对象：

```
c = let create =
  fix (lambda f:{x:Nat}->Counter. lambda s:{x:Nat}.
    {get = s.x,
     inc = lambda _:Unit. f {x=succ(s.x)},
     dec = lambda _:Unit. f {x=pred(s.x)}})
  in create {x=0};
► c : Counter

c1 = c.inc unit;
c2 = c1.inc unit;
c2.get;
► 2 : Nat
```

调用对象的操作，只需投影相应字段；上例连续取得两次 `inc` 字段并调用，再读取 `get`。

练习 20.1.3 (★★). 扩展上面的 `Counter` 类型和计数器 `c`，加入备份和重置操作，行为与§18.7相同：调用 `backup` 时，把计数器当前值保存到独立的内部寄存器；调用 `reset` 时，把计数器恢复成寄存器中保存的值。

[查看原书附录 A 中的 20.1.3 解答](#)

由递归类型构造递归值

递归类型还有一种更令人意外的用途，也最能显示它的表达能力：为不动点组合子给出良类型实现。对于任意类型 T ，可以为输入类型和输出类型都是 T 的函数定义如下不动点组合子：

```
fixT = lambda f:T->T.
      (lambda x:(μA.A->T). f (x x))
      (lambda x:(μA.A->T). f (x x));
► fixT : (T->T) -> T
```

关键技巧是用递归类型为两处 $x\ x$ 赋型。[练习9.3.2](#) 已经指出，要给这一项赋型， x 必须具有箭头类型，而且其参数类型恰好就是 x 自己的类型。令 $R = \mu A. A \rightarrow T$ ，展开一层可得 $R = R \rightarrow T$ 。因此，当 $x : R$ 时， $x\ x$ 左边的 x 可以作为 $R \rightarrow T$ 型函数使用，右边的 x 则恰好可以作为它所需的 R 型参数，于是 $x\ x$ 具有类型 T 。普通有限类型无法满足 $R = R \rightarrow T$ ；递归类型 $\mu A. A \rightarrow T$ 则用有限记法表示了这个无限展开的类型。

译者注：作者勘误澄清：上面的项擦除类型后得到的是按名调用不动点组合子，而不是第 5 章给出的按值调用版本；按值版本还需要在自应用处加入额外的 lambda 抽象。这个差别不影响下文用它构造发散项，也不影响本节展示递归类型可以为自应用赋型的主旨。

因此，加入递归类型会破坏强规范化性质：可以用 `fixT` 写出良类型却不会结束求值的项，例如

```
divergeT = lambda _:Unit. fixT (lambda x:T. x);
► divergeT : Unit -> T
```

对每个类型都能构造这样的项，所以，与简单类型 lambda 演算 $\lambda \rightarrow$ 不同，这个系统中的每个类型都有居留元。⁴

再次考察无类型 lambda 演算

递归类型表达能力最鲜明的例证，或许是它能把整个无类型 lambda 演算以良类型方式嵌入带递归类型的静态类型语言。令下面的类型为 (D)：⁵

```
D = μX. X->X;
```

并定义“注入函数”`lam`，把从 D 到 D 的函数变成 D 的元素：

```
lam = lambda f:D->D. f as D;
► lam : (D->D) -> D
```

⁴这一事实使带递归类型的系统无法用作有意义的逻辑系统。按照 Curry-Howard 对应（见§9.4），若把“类型 T 有居留元”理解为“命题 T 可证”，那么每个类型都有居留元，就意味着每个命题都可证，因而这样的逻辑是不一致的。

⁵熟悉指称语义的读者会注意到，(D) 的定义正是纯 lambda 演算语义模型所用通用域 (*universal domain*) 的定义性质。

要把一个 D 的元素应用于另一个，只需把前者的类型展开成函数类型，再应用于后者：

```
ap = lambda f:D. lambda a:D. f a;
► ap : D -> D -> D
```

译者注：下面将构造一种从纯无类型 lambda 演算到带递归类型语言的统一嵌入。直接目标是让任意无类型 lambda 项 M 经过翻译后都能通过目标语言的静态类型检查，即得到 $\vdash M^* : D$ 。无类型演算允许同一个项出现在函数位置和参数位置，甚至写成 xx ；若原样放进普通简单类型系统，类型检查器无法为这类项赋型。为了统一处理所有情况，翻译规定每个源项及其子项都取得类型 D 。这样一来，应用左边的 D 型项还必须能作为 $D \rightarrow D$ 型函数使用，而应用右边和应用结果仍须具有类型 D ，所需的类型方程正是 $D = D \rightarrow D$ 。递归类型 $D = \mu X. X \rightarrow X$ 满足这个方程；**lam** 负责把 $D \rightarrow D$ 型抽象封装成 D 型项，**ap** 则把 D 型项展开成函数后完成应用。因此，下述三条翻译规则产生的项都能通过类型检查。这个构造也表明递归类型足以表达无类型 lambda 演算的全部计算；由于所有源项最终都归入 D ，它不会保留更精细的类型区别。

设 M 是只含变量、抽象和应用的闭 lambda 项。可以按照下面的规则，把 M 翻译成一个类型为 D 的项，记作 M^* ：

$$\begin{aligned} x^* &= x \\ (\lambda x. M)^* &= \text{lam}(\lambda x : D. M^*) \\ (M N)^* &= \text{ap } M^* N^* \end{aligned}$$

例如，无类型不动点组合子可翻译成如下 D 型项：

```
fixD = lam (lambda f:D.
    ap (lam (lambda x:D. ap f (ap x x)))
    (lam (lambda x:D. ap f (ap x x))));
► fixD : D
```

译者注：作者勘误指出，**fixD** 编码的是按名调用的 (Y) 组合子。若要编码按值调用版本，需要像§5.2那样在自应用处再加入一层 lambda 抽象；递归类型同样可以为该版本赋型。

还可以扩展这项嵌入，加入自然数等特性。把 D 改成一个变体类型，以两个标签分别表示数和函数：

```
D =  $\mu X$ . <nat:Nat, fn:X->X>;
```

也就是说， D 的元素要么是带 **nat** 标签的数，要么是带 **fn** 标签的 $D \rightarrow D$ 函数。**lam** 的实现与此前基本相同：

```
lam = lambda f:D->D. <fn=f> as D;
► lam : (D->D) -> D
```

ap 的实现则多了一项重要工作：

```
ap = lambda f:D. lambda a:D. case f of
    <nat=n> ==> divergeD unit
  | <fn=g> ==> g a;
► ap : D -> D -> D
```

应用 f 之前，必须先用 `case` 从中取出函数；这也迫使我们规定 f 不是函数时应用应怎样处理。这里选择发散，也可以抛出异常。这种标签检查与 Scheme 等动态类型语言实现中的运行时标签检查十分相似。在这个意义上，带类型计算可以把无类型或动态类型计算包容在自身之内。

在 D 的元素上定义后继函数时也需要同样的标签检查：

```
suc = lambda f:D. case f of
  <nat=n> ==> (<nat=succ n> as D)
  | <fn=g>   ==> divergeD unit;
► suc : D -> D

zro = <nat=0> as D;
► zro : D
```

练习 20.1.4 (★). 扩展这项编码，加入布尔值与条件表达式；再把 `if false then 1 else 0` 和 `if false then 1 else false` 编码为 D 的元素。对这些项求值会得到什么？

[查看原书附录 A 中的 20.1.4 解答](#)

练习 20.1.5 (推荐, ★★). 扩展数据类型 D ，加入记录：

```
D =  $\mu X$ . <nat:Nat, fn:X->X, rcd:Nat->X>;
```

并实现记录构造和字段投影。为简单起见，用自然数作为字段标签，也就是把记录表示成从自然数到 D 的函数。请用 `fullequirec` 检查扩展。

[查看原书附录 A 中的 20.1.5 解答](#)

20.2 形式定义

类型系统文献对递归类型主要采用两种处理方式。它们对一个简单问题给出不同答案： $\mu X.T$ 与其展开一次所得的类型是什么关系？例如，`NatList` 与下面这个类型是什么关系？

$$\langle \text{nil} : \text{Unit}, \text{cons} : \{\text{Nat}, \text{NatList}\} \rangle$$

1. 等递归方法把两个类型表达式视为定义上相等，因而可以在所有上下文中互换；它们表示的是同一棵无限树。类型检查器负责保证：某函数需要其中一个类型的参数时，另一个类型的项也可以传入，其他场景同理。

这种方法的优点是，声明式系统只需允许类型表达式表示无限的正则结构。只要已有定义、安全性定理和证明不依赖对类型表达式作归纳，它们都可以保持不变（这里特指对类型表达式本身作结构归纳；对项、求值推导或类型推导等有限对象的归纳仍可照常使用）。实现却需要额外工作，因为类型检查算法不能直接操作真正的无限结构；[第二十一章](#)将说明怎样用有限表示完成检查。

2. 同构递归方法把递归类型与其展开视为不同但同构的类型。

递归类型 $\mu X.T$ 的展开，是在类型体 T 中把 X 的全部出现替换成整个递归类型：

$$[X \mapsto \mu X.T]T$$

例如，

$$\mu X.\langle \text{nil} : \text{Unit}, \text{cons} : \{\text{Nat}, X\} \rangle$$

展开为

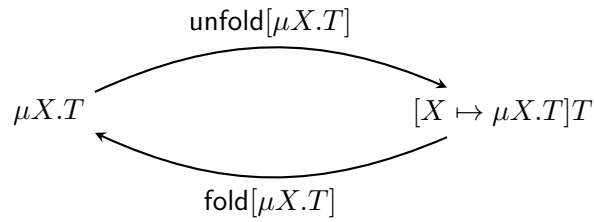
$$\langle \text{nil} : \text{Unit}, \text{cons} : \{\text{Nat}, \mu X.\langle \text{nil} : \text{Unit}, \text{cons} : \{\text{Nat}, X\} \rangle\} \rangle$$

同构递归系统为每个 $\mu X.T$ 提供一对函数：

$$\text{unfold}[\mu X.T] : \mu X.T \rightarrow [X \mapsto \mu X.T]T$$

$$\text{fold}[\mu X.T] : [X \mapsto \mu X.T]T \rightarrow \mu X.T$$

它们分别完成两个方向的转换，具体给出了这两个类型之间的同构关系。



λ_μ

扩展 $\lambda \rightarrow$ (图 9-1)

新增句法形式

$t ::= \dots$	项
$\text{fold}[T] t$	折叠
$\text{unfold}[T] t$	展开
$v ::= \dots$	值
$\text{fold}[T] v$	折叠值
$T ::= \dots$	类型
X	类型变量
$\mu X.T$	递归类型

新增求值规则

$$\frac{}{\text{unfold}[S](\text{fold}[T]v_1) \rightarrow v_1} \text{E-UNFLD} \text{FLD}$$

同余规则

$$\frac{t_1 \rightarrow t'_1}{\text{fold}[T]t_1 \rightarrow \text{fold}[T]t'_1} \text{E-FOLD}$$

$$\frac{t_1 \rightarrow t'_1}{\text{unfold}[T]t_1 \rightarrow \text{unfold}[T]t'_1} \text{E-UNFOLD}$$

类型规则

$$\frac{U = \mu X.T_1 \quad \Gamma \vdash t_1 : [X \mapsto U]T_1}{\Gamma \vdash \text{fold}[U]t_1 : U} \text{T-FOLD}$$

$$\frac{U = \mu X.T_1 \quad \Gamma \vdash t_1 : U}{\Gamma \vdash \text{unfold}[U]t_1 : [X \mapsto U]T_1} \text{T-UNFOLD}$$

图 20-1: 同构递归类型 (λ_μ)

语言把 `fold` 与 `unfold` 作为基本构造。E-UNFLDFLD 规定，对刚按某递归类型折叠起来的值再作展开，会消去这对操作。这里先由 T-FOLD 与 T-UNFOLD 共同保证两个类型标注相同：内层 `fold[T]` 的结果具有类型 T ，而外层 `unfold[S]` 又要求参数具有类型 S ，所以在任何良类型程序中必有 $S = T$ 。求值规则因而无须重复检查这一条件；否则求值器还要额外执行类型比较。

译者注：同构递归系统把递归类型 T 与 T 展开一层后得到的类型视为两个不同类型。`fold[T]` 完成从“展开一层后的类型”到 T 的显式转换，`unfold[T]` 则完成反方向的转换。图20-1的值语法也直接体现了这种区别：只有内部项已经是值的 `fold[T] v` 被列为值，`unfold[T] t` 没有被列为值。前者所表示的转换已经完成，没有后续计算；后者的目的则是取出当前这一层的内容，所以仍是一项需要执行的计算。

以 `NatList` 为例，假设 `nil` 已经具有 `NatList` 类型，那么 `<cons={0,nil}>` 给出的是列表展开一层后的结构，`fold[NatList] <cons={0,nil}>` 再把它转为 `NatList` 类型。这里每次只处理当前一层，表尾 `nil` 不会被继续展开。反过来，`unfold[NatList]` 会取出这一层结构；因此 `unfold[T] (fold[T] v)` 会继续求值为 v ，而不会成为值。

两种方法都广泛用于理论研究和程序语言设计。等递归写法较为直观，却对类型检查器要求更高：检查器实际上要推断应在何处加入折叠和展开。等递归类型与有界量化、类型算子等高级类型特性的交互也可能非常复杂，带来显著的理论困难（Ghelli, 1993；Colazzo 与 Ghelli, 1999），甚至使类型检查不可判定（Solomon, 1978）。

同构递归写法在记号上较重，程序每次使用递归类型时都要写出折叠或展开。不过，这些标注在实际语言中常能与其他构造合并。例如，ML 家族中的数据类型定义可能隐式引入递归类型；用构造子建立该类型的值时，构造子隐含一次折叠；在模式匹配中出现构造子时，则隐含一次展开。类似地，Java 的类定义隐式引入递归类型，调用对象方法也隐含展开。这些语言原有的构造子、模式匹配和方法调用正好可以承担 `fold` 与 `unfold` 的作用，程序员通常无须再显式写出这些操作。因此，同构递归类型在实际语言中不会带来太重的书写负担。

下面把 `NatList` 改写成同构递归形式。先为其展开形式定义缩写：

```
NLBody = <nil:Unit, cons:{Nat,NatList}>;
```

`nil` 先构造 `NLBody` 型变体，再把它折叠成 `NatList`；`cons` 同理：

```
nil = fold [NatList] (<nil=unit> as NLBody);
cons = lambda n:Nat. lambda l:NatList.
    fold [NatList] (<cons={n,l}> as NLBody);
```

反过来，`isnil`、`hd` 和 `tl` 必须把 `NatList` 看成变体，才能根据标签作分类讨论；为此先展开参数 `l`：

```
isnil = lambda l:NatList.
    case unfold [NatList] l of
        <nil=u> ==> true
    | <cons=p> ==> false;

hd = lambda l:NatList.
    case unfold [NatList] l of
        <nil=u> ==> 0
```



```

| <cons=p> ==> p.1;

t1 = lambda l:NatList.
  case unfold [NatList] l of
    <nil=u> ==> l
  | <cons=p> ==> p.2;

```

练习 20.2.1 (推荐, ★★). 用显式 fold 与 unfold 标注, 重写 §20.1 的其他一些示例, 尤其是 fixT。请用 fullisorec 检查结果。

[查看原书附录 A 中的 20.2.1 解答](#)

练习 20.2.2 (★★, ⇨). 概述同构递归系统的进展性与保持性定理的证明。

[查看练习 20.2.2 的译者参考解答](#)

严格说来, 等递归方法允许的不是任意无限类型表达式, 而是正则的无限结构; §21.7 将给出精确定义。 μ 类型到无限树展开的映射则在 §21.8 定义。

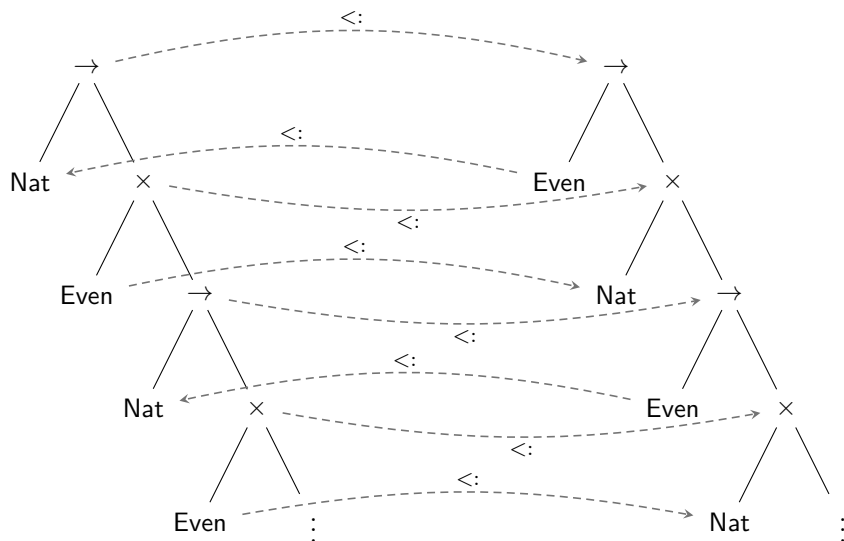
20.3 子类型化

最后还要考察递归类型与简单类型 lambda 演算的另一项重要扩展——子类型化——怎样结合。假设 $\text{Even} <: \text{Nat}$, 那么

$$\mu X. \text{Nat} \rightarrow (\text{Even} \times X) \quad \text{与} \quad \mu X. \text{Even} \rightarrow (\text{Nat} \times X)$$

之间应是什么关系?

最直接的分析方法, 是按等递归方式把两者不断展开。两种类型的值都可以看成 [本章前面的反应式进程](#): 给它一个数, 它会返回另一个数和一个准备继续接收数字的新进程。第一种进程能接受任意自然数, 却总是返回偶数; 第二种进程只保证接受偶数, 但可能返回任意自然数。第一种类型对参数接收范围和结果范围都作出了更强保证, 因此应当是第二种类型的子类型。把这几项比较画在无限展开的类型树上, 可得:



译者注：图中的黑色实线表示类型展开后形成的树，灰色虚线表示对应节点之间需要成立的子类型关系。令

$$A = \mu X. \text{Nat} \rightarrow (\text{Even} \times X), \quad B = \mu X. \text{Even} \rightarrow (\text{Nat} \times X)$$

图最上方表示要说明 $A <: B$ 。根据箭头类型的子类型规则，这一目标分解为

$$\text{Even} <: \text{Nat} \quad \text{和} \quad \text{Even} \times A <: \text{Nat} \times B$$

第一项的比较方向与整体相反，因为函数参数位置是逆变的。第二项再由二元积的子类型规则分解为

$$\text{Even} <: \text{Nat} \quad \text{和} \quad A <: B$$

后一项恰好回到最初的目标，因此同一比较会沿递归类型无限重复。

这种直观论证能否得到严格表述？可以。[第二十一章](#)将用余归纳给出严格论证。

20.4 注记

计算机科学中的递归类型至少可以追溯到 Morris (1968)。不带子类型化的递归类型，其基本句法和语义性质由 Huet (1976) 以及 MacQueen、Plotkin 与 Sethi (1986) 的早期论文建立，并汇集于 Cardone 与 Coppo (1991)。Pair 在 1970 年发表的一篇论文似乎还给出了等递归类型相等性可判定性的第一个证明。无限树与正则树的性质可参见 Courcelle (1983) 的综述；同构递归系统与等递归系统之间的关系由 Abadi 与 Fiore (1996) 研究。递归类型与子类型化的更多文献见[§21.12](#)。

Morris (1968, pp. 122–124) 最早注意到，可以用递归类型为项构造良类型的 **fix** 算子 ([§20.1](#))。

这两种递归类型形式自该领域最早期的研究便已存在；便于记忆的名称 *iso-recursive* 与 *equi-recursive* 则是 Crary、Harper 与 Puri (1999) 较晚提出的。

第二十一章 递归类型的元理论

概览

第 20 章介绍了递归类型的两种表示。等递归类型在定义上就等同于自身的展开；同构递归类型则使用 `fold` 和 `unfold` 项，显式表示递归类型与其展开之间的等价关系。本章为等递归类型的类型检查器建立理论基础；同构递归类型的实现相对直接。实际语言经常把递归类型与子类型化结合起来，因此本章同时讨论二者。即使去掉子类型化，系统也只会略微简单一些，因为类型检查器仍须判断两个递归类型是否等价。

第 20 章曾用无限类型上的无限子类型推导，直观说明等递归类型的子类型关系。本章将借助余归纳把这一理解精确化，并严格联系无限树、无限推导与实际子类型算法所处理的有限表示。

§21.1 回顾归纳定义、余归纳定义及其证明原则；§21.2 与 §21.3 把一般理论用于子类型关系，分别给出有限类型上的归纳定义和无限类型上的余归纳定义。§21.4 短暂离开主线，讨论传递规则带来的一些问题（正如前文所见，这条规则在子类型系统中一向是出了名的麻烦来源）；§21.5 给出检查归纳集合和余归纳集合成员关系的简单算法，§21.6 进一步改进这些算法。§21.7 把算法用于正则无限类型；§21.8 用有限的 μ -类型表示无限类型，并证明 μ -类型的子类型关系对应于无限类型上的普通余归纳定义。§21.9 证明算法终止；§21.10 把它与 Amadio 和 Cardelli 的算法比较；§21.11 简要讨论同构递归类型的子类型化。¹

21.1 归纳与余归纳

先固定一个全集 U ，作为归纳定义和余归纳定义的论域。 U 包含当前讨论中所有可能的对象；归纳定义或余归纳定义的作用，是从中选出某个子集。稍后将取 U 为全部类型对的集合，使 U 的子集成为类型上的关系；眼下任意集合均可。

定义 21.1.1. 函数 $F : \mathcal{P}(U) \rightarrow \mathcal{P}(U)$ 是单调的 (*monotone*)，若 $X \subseteq Y$ 蕴含 $F(X) \subseteq F(Y)$ 。这里 $\mathcal{P}(U)$ 表示 U 的所有子集组成的集合。

本节以下假定 F 是 $\mathcal{P}(U)$ 上的单调函数，并常称它为生成函数 (*generating function*)。

定义 21.1.2. 设 $X \subseteq U$ 。

¹本章研究的系统，是带子类型化的简单类型演算（图15-1），再加上二元积（图11-5）和等递归类型。原书配套检查器名为 `equirec`。

1. 若 $F(X) \subseteq X$, 则称 X 为 F -闭的;
2. 若 $X \subseteq F(X)$, 则称 X 为 F -一致的;
3. 若 $F(X) = X$, 则称 X 为 F 的不动点。

可以把 U 的元素看作陈述, 把 F 看作一种“论证关系”: 给定一组作为前提的陈述, F 告诉我们能够推出哪些结论。一个 F -闭集合已经包含其成员所能推出的全部结论; 一个 F -一致集合中的每条陈述, 都能由集合内的其他陈述提供根据。 F 的不动点同时满足这两个条件: 成员需要的根据和成员能够推出的结论都在集合中, 除此以外没有别的元素。

例 21.1.3. 在三元素全集 $U = \{a, b, c\}$ 上考虑生成函数 E_1 :

$$\begin{array}{llll} E_1(\emptyset) & = & \{c\} & E_1(\{a, b\}) & = & \{c\} \\ E_1(\{a\}) & = & \{c\} & E_1(\{a, c\}) & = & \{b, c\} \\ E_1(\{b\}) & = & \{c\} & E_1(\{b, c\}) & = & \{a, b, c\} \\ E_1(\{c\}) & = & \{b, c\} & E_1(\{a, b, c\}) & = & \{a, b, c\} \end{array}$$

唯一的 E_1 -闭集合是 $\{a, b, c\}$; 四个 E_1 -一致集合是 $\emptyset$ 、 $\{c\}$ 、 $\{b, c\}$ 和 $\{a, b, c\}$ 。

也可以用以下推导规则紧凑地表示 E_1 :

$$\frac{}{c} \quad \frac{c}{b} \quad \frac{b \quad c}{a}$$

每条规则的含义是: 若横线上方的元素都属于输入集合, 横线下方的元素就属于输出集合。

定理 21.1.4 (Knaster–Tarski (Tarski, 1955)).

1. 所有 F -闭集合的交, 是 F 的最小不动点;
2. 所有 F -一致集合的并, 是 F 的最大不动点。

证明. 只证第 2 项; 第 1 项对称。令 $\mathcal{C} = \{X \mid X \subseteq F(X)\}$, 即全部 F -一致集合组成的集合, 并令 $P = \bigcup_{X \in \mathcal{C}} X$ 。对任意 $X \in \mathcal{C}$, 有

$$X \subseteq F(X) \subseteq F(P),$$

第二个包含关系来自 F 的单调性。对所有这样的 X 取并, 得到 $P \subseteq F(P)$, 所以 P 是 F -一致的; 按定义, 它也是最大的 F -一致集合。

再由单调性和 $P \subseteq F(P)$, 得到 $F(P) \subseteq F(F(P))$, 所以 $F(P) \in \mathcal{C}$ 。既然 P 是 $\mathcal{C}$ 全部成员的并, 便有 $F(P) \subseteq P$ 。合并两边的包含关系可知 $F(P) = P$ 。因此 P 既是最大 F -一致集合, 也是最大不动点。□

定义 21.1.5. F 的最小不动点写作 μF , 最大不动点写作 νF 。

例 21.1.6. 对例21.1.3的生成函数 E_1 , 有 $\mu E_1 = \nu E_1 = \{a, b, c\}$ 。

练习 21.1.7 (★). 设全集 $\{a, b, c\}$ 上的生成函数 E_2 由以下规则定义:

$$\frac{}{a} \quad \frac{c}{b} \quad \frac{a \quad b}{c}$$

像例21.1.3那样, 显式列出关系 E_2 中的各个数对; 列出所有 E_2 -闭集合和 E_2 -一致集合, 并求 μE_2 与 νE_2 。

查看原书附录 A 中的 21.1.7 解答

注意, μF 本身是 F -闭的, 因而是最小的 F -闭集合; νF 本身是 F -一致的, 因而是最大的 F -一致集合。这给出两条基本推理原则。

推论 21.1.8.

1. **归纳原理:** 若 X 是 F -闭的, 则 $\mu F \subseteq X$;
2. **余归纳原理:** 若 X 是 F -一致的, 则 $X \subseteq \nu F$ 。

把集合 X 看成谓词的特征集合, 便容易理解这两条原则: 证明性质 X 对元素 x 成立, 等价于证明 $x \in X$ 。归纳原理说, 只要某个性质在 F 的作用下保持成立, 也就是其特征集合对 F 闭合, 那么它就对归纳定义集合 μF 的全部元素成立。

余归纳原理则给出证明 $x \in \nu F$ 的办法: 只需找到一个包含 x 的 F -一致集合 X 。余归纳虽然不如归纳熟悉, 却是许多计算机科学领域的核心工具, 例如并发理论中的互模拟 (*bisimulation*) 和许多 模型检查 (*model checking*) 算法。

译者注: 互模拟用于比较两个状态转移系统。若某个关系中的一对状态满足: 一方的每一步转移都能由另一方匹配, 并且转移后得到的两个状态仍处于这个关系中; 反方向也满足同样的条件, 那么这个关系就是互模拟。所有互模拟关系的并仍是互模拟, 因此可以用最大不动点和余归纳来刻画。模型检查则是在状态转换图上自动判断系统是否满足给定性质; 其中许多算法同样可以表述为计算最小或最大不动点。

本章会反复使用归纳与余归纳。为简洁起见, 归纳论证常写作结构归纳等熟悉形式, 不再每次都展开为生成函数和谓词; 余归纳论证则会写得更明确。

练习 21.1.9 (推荐, ★★★). 证明: 自然数上的普通归纳原理 (公理2.4.1) 以及自然数对上的字典序归纳原理 (公理2.4.4), 都可以从推论21.1.8的归纳原理推出。

查看原书附录 A 中的 21.1.9 解答

21.2 有限类型与无限类型

接下来要把最大不动点和余归纳证明法用于子类型关系。为此, 先精确定义怎样把类型视为有限树或无限树。本章只使用三种类型构造子: $\rightarrow$ 、 $\times$ 和 **Top**。树的每个节点都用其中一个符号标记。下面的定义只针对本章所需的树; 无限带标签树的一般理论可参阅 Courcelle (1983)。

令 $\{1, 2\}^*$ 表示由 1 和 2 组成的全部有限序列, 空序列记为 ϵ , i^k 表示 k 个 i 连成的序列; 若 π 和 σ 都是序列, 则 π, σ 表示二者的连接。

定义 21.2.1. 树类型 (*tree type*) (简称树) 是满足以下条件的偏函数²

$$T : \{1, 2\}^* \rightarrow \{\rightarrow, \times, \text{Top}\}.$$

1. $T(\epsilon)$ 有定义;
2. 若 $T(\pi, \sigma)$ 有定义, 则 $T(\pi)$ 有定义;
3. 若 $T(\pi) = \rightarrow$ 或 $T(\pi) = \times$, 则 $T(\pi, 1)$ 与 $T(\pi, 2)$ 都有定义;
4. 若 $T(\pi) = \text{Top}$, 则 $T(\pi, 1)$ 与 $T(\pi, 2)$ 都无定义。

若 $\text{dom}(T)$ 有限, 则称 T 为有限树类型。所有树类型的集合记作 $\mathcal{T}$, 其中全部有限树类型组成的子集记作 $\mathcal{T}_f$ 。

为方便记号, Top 也表示唯一节点标为 Top 的树。若 T_1, T_2 是树, 则 $T_1 \times T_2$ 和 $T_1 \rightarrow T_2$ 分别表示由以下函数定义的树, 其中 $i = 1, 2$:

$$\begin{aligned} (T_1 \times T_2)(\epsilon) &= \times, & (T_1 \times T_2)(i, \pi) &= T_i(\pi), \\ (T_1 \rightarrow T_2)(\epsilon) &= \rightarrow, & (T_1 \rightarrow T_2)(i, \pi) &= T_i(\pi) \end{aligned}$$

例如, $(\text{Top} \times \text{Top}) \rightarrow \text{Top}$ 表示有限树

$$T(\epsilon) = \rightarrow, \quad T(1) = \times, \quad T(2) = T(1, 1) = T(1, 2) = \text{Top}.$$

省略号可用于非有限树。例如, $\text{Top} \rightarrow (\text{Top} \rightarrow (\text{Top} \rightarrow \cdots))$ 对应的树满足: 对每个 $k \geq 0$, $T(2^k) = \rightarrow$ 且 $T(2^k, 1) = \text{Top}$ 。

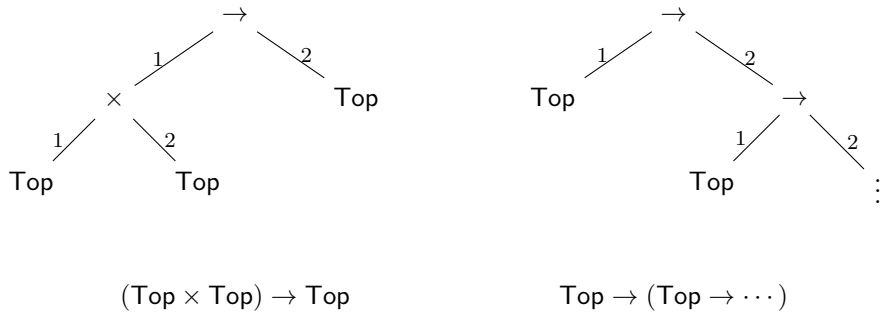


图 21-1: 树类型示例

有限树类型还可以用文法紧凑定义:

$$T ::= \text{Top} \mid T \times T \mid T \rightarrow T.$$

形式上, $\mathcal{T}_f$ 是该文法所描述生成函数的最小不动点。该生成函数以所有由 Top 、 $\rightarrow$ 和 $\times$ 标记的有限树和无限树为全集; 换言之, 在定义21.2.1中删去最后两项条件, 就得到这个全集。同一生成函数取最大不动点, 便得到全部树类型 $\mathcal{T}$ 。

² “树类型”这个说法略显别扭, 但有助于和§21.8将要讨论的另一种有限表示相区分; 后者称为“ μ -类型”。

译者注：把上述文法理解成作用于树集合的函数，可以看清它与不动点的关系。对任意树集合 X ，定义

$$F(X) = \{\text{Top}\} \cup \{T_1 \times T_2 \mid T_1, T_2 \in X\} \cup \{T_1 \rightarrow T_2 \mid T_1, T_2 \in X\}$$

也就是说， F 总能生成单节点树 Top ，还可以用 X 中的两棵树作为左右子树，生成一棵新的积类型树或箭头类型树。

从空集开始反复应用 F ，第一个阶段得到 Top ，以后每个阶段都在已有树的外面增加一层。任意一个阶段只含深度有界的有限树；把所有阶段的结果取并，便得到 $\mathcal{T}_f$ 。 $\mathcal{T}_f$ 本身是无穷集合，但其中每一棵树都是有限的。用两棵有限树再构造一层，结果仍是有限树；反过来，任意有限树或者就是 Top ，或者可以在根节点处拆成两棵有限子树。因此 $F(\mathcal{T}_f) = \mathcal{T}_f$ ，所以 $\mathcal{T}_f$ 是 F 的一个不动点。它还是最小的不动点：任取另一个不动点 Y ，由 $F(Y) = Y$ 可知 Y 必须包含 Top ，并且只要包含 T_1, T_2 ，就必须同时包含 $T_1 \times T_2$ 和 $T_1 \rightarrow T_2$ 。按照有限树的构造逐层应用这条性质，可以得到 $\mathcal{T}_f \subseteq Y$ 。因此，每个不动点都至少包含全部有限树，不可能比 $\mathcal{T}_f$ 更小。

同一生成函数的最大不动点还会收入无限向下展开的树。这类树不会在上述任何有限阶段出现，但它的根节点和两棵直接子树仍然满足相同的生成条件。因此， $\mu F = \mathcal{T}_f$ 给出全部有限树类型， $\nu F = \mathcal{T}$ 则给出全部有限或无限树类型。

练习 21.2.2 (推荐, $\star\star$). 沿用上一段的思路，给出全集 U 和生成函数 $F : \mathcal{P}(U) \rightarrow \mathcal{P}(U)$ ，使有限树类型集合 $\mathcal{T}_f$ 等于 μF ，全部树类型集合 $\mathcal{T}$ 等于 νF 。

[查看原书附录 A 中的 21.2.2 解答](#)

21.3 子类型化

有限树类型上的子类型关系和任意树类型上的子类型关系，将分别定义为某个单调函数的最小不动点和最大不动点。有限情形的全集是 $\mathcal{T}_f \times \mathcal{T}_f$ 。这个全集的子集就是有限树类型上的关系；生成函数把这样的关系映射到同一全集中的另一个关系，所以该生成函数的各个不动点也都是有限树类型上的关系。无限情形的全集则是 $\mathcal{T} \times \mathcal{T}$ 。

定义 21.3.1 (有限子类型关系). 若 $(S, T) \in \mu \mathcal{S}_f$ ，则称有限树类型 S 是 T 的子类型。单调函数 $\mathcal{S}_f : \mathcal{P}(\mathcal{T}_f \times \mathcal{T}_f) \rightarrow \mathcal{P}(\mathcal{T}_f \times \mathcal{T}_f)$ 定义为

$$\begin{aligned} \mathcal{S}_f(R) = & \{(T, \text{Top}) \mid T \in \mathcal{T}_f\} \\ & \cup \{(S_1 \times S_2, T_1 \times T_2) \mid (S_1, T_1), (S_2, T_2) \in R\} \\ & \cup \{(S_1 \rightarrow S_2, T_1 \rightarrow T_2) \mid (T_1, S_1), (S_2, T_2) \in R\}. \end{aligned}$$

该函数与熟悉的三条子类型规则完全对应：

$$\frac{}{T <: \text{Top}} \quad \frac{S_1 <: T_1 \quad S_2 <: T_2}{S_1 \times S_2 <: T_1 \times T_2} \quad \frac{T_1 <: S_1 \quad S_2 <: T_2}{S_1 \rightarrow S_2 <: T_1 \rightarrow T_2}.$$

横线上方的 $S <: T$ 表示“类型对 (S, T) 属于 $\mathcal{S}_f$ 的输入关系”，横线下方向则表示该类型对属于输出关系。

定义 21.3.2 (无限子类型关系). 若 $(S, T) \in \nu\mathcal{S}$, 则称树类型 S 是 T 的子类型, 其中 $\mathcal{S} : \mathcal{P}(\mathcal{T} \times \mathcal{T}) \rightarrow \mathcal{P}(\mathcal{T} \times \mathcal{T})$ 定义为

$$\begin{aligned} \mathcal{S}(R) = & \{(T, \text{Top}) \mid T \in \mathcal{T}\} \\ & \cup \{(S_1 \times S_2, T_1 \times T_2) \mid (S_1, T_1), (S_2, T_2) \in R\} \\ & \cup \{(S_1 \rightarrow S_2, T_1 \rightarrow T_2) \mid (T_1, S_1), (S_2, T_2) \in R\}. \end{aligned}$$

它的推导规则与有限情形完全相同; 改变的只有全集和不动点方向: 类型可以是无限树, 并取最大不动点而不是最小不动点。

练习 21.3.3 ($\star$). 举出一个不属于 $\nu\mathcal{S}$ 的类型对 (S, T) , 由此验证 $\nu\mathcal{S} \neq \mathcal{T} \times \mathcal{T}$ 。

查看原书附录 A 中的 21.3.3 解答

练习 21.3.4 ($\star$). 是否存在属于 $\nu\mathcal{S}$ 却不属于 $\mu\mathcal{S}$ 的类型对 (S, T) ? 是否存在属于 $\nu\mathcal{S}_f$ 却不属于 $\mu\mathcal{S}_f$ 的有限类型对?

查看原书附录 A 中的 21.3.4 解答

现在应当立即验证无限树类型上的子类型关系的一项基本性质: 传递性。(有限类型上的子类型关系已经在§16.1中证明为传递。) 若存在 $S <: T$ 、 $T <: U$ 但 $S \not<: U$, 令 s 是 S 型值、 f 是 $U \rightarrow \text{Top}$ 型函数, 则 $(\lambda x : T. f x) s$ 可以借助两次 T-Sub 通过类型检查, 却会一步求值为不能赋型的 $f s$, 从而破坏保持性。

定义 21.3.5. 关系 $R \subseteq U \times U$ 是传递的, 若它在单调函数

$$\text{TR}(R) = \{(x, y) \mid \exists z \in U. (x, z), (z, y) \in R\}$$

下闭合, 即 $\text{TR}(R) \subseteq R$ 。

引理 21.3.6. 设 $F : \mathcal{P}(U \times U) \rightarrow \mathcal{P}(U \times U)$ 单调。若对每个 $R \subseteq U \times U$ 都有

$$\text{TR}(F(R)) \subseteq F(\text{TR}(R)),$$

则 νF 是传递关系。

证明. 因为 $\nu F = F(\nu F)$, 有

$$\text{TR}(\nu F) = \text{TR}(F(\nu F)) \subseteq F(\text{TR}(\nu F)).$$

所以 $\text{TR}(\nu F)$ 是 F -一致的。余归纳原理给出 $\text{TR}(\nu F) \subseteq \nu F$, 这正是传递性。□

该引理与消去传递规则的传统方法相似, 通常称为割消去证明 (见§16.1)。条件

$$\text{TR}(F(R)) \subseteq F(\text{TR}(R))$$

表明: 先用 F 中的规则、再用传递规则得到的结论, 也能调换顺序, 先用传递规则、再用 F 中的规则得到。

译者注：§9.4曾从 Curry-Howard 对应的角度介绍割消去：通过重排证明，消除其中不必要的中间命题。对子类型关系，最直接的前例是引理16.1.2及其作者解答；那里按照两份子推导的末条规则分类，把 S-TRANS 推入规模更小的子推导，最终消去传递规则。这里的包含关系以生成函数的记法表达同一种重排思想。

定理 21.3.7. νS 是传递关系。

证明. 依引理21.3.6, 只需对任意 $R \subseteq \mathcal{T} \times \mathcal{T}$ 证明 $\text{TR}(\mathcal{S}(R)) \subseteq \mathcal{S}(\text{TR}(R))$ 。取 $(S, T) \in \text{TR}(\mathcal{S}(R))$ 。按 TR 的定义, 存在 U 使 $(S, U), (U, T) \in \mathcal{S}(R)$ 。对 U 的根节点分类。

若 $U = \text{Top}$, 由 $(U, T) \in \mathcal{S}(R)$ 可知 $T = \text{Top}$ 。对任意 $A \in \mathcal{T}$ 和 $Q \subseteq \mathcal{T} \times \mathcal{T}$, 都有 $(A, \text{Top}) \in \mathcal{S}(Q)$ ；取 $A = S$, $Q = \text{TR}(R)$, 便得到 $(S, T) = (S, \text{Top}) \in \mathcal{S}(\text{TR}(R))$ 。

若 $U = U_1 \times U_2$, 当 $T = \text{Top}$ 时仍由上一条得到结论；否则 $T = T_1 \times T_2$, 并有 $(U_1, T_1), (U_2, T_2) \in R$ 。同理, $S = S_1 \times S_2$, 且 $(S_1, U_1), (S_2, U_2) \in R$ 。因此 $(S_1, T_1), (S_2, T_2) \in \text{TR}(R)$, 再按 $\mathcal{S}$ 的积分支即可得到结论。若 $U = U_1 \rightarrow U_2$, 论证相同, 并在参数位置使用逆变方向。□

练习 21.3.8 (推荐, ★★). 证明无限树类型上的子类型关系也是自反的。

[查看原书附录 A 中的 21.3.8 解答](#)

下一节继续比较有限类型与无限树类型中的传递性；初次阅读可以略读或跳过。

21.4 关于传递性的题外讨论

第 16 章已经看到, 归纳定义的子类型关系通常有两种表示：声明式系统便于阅读, 算法式系统则较为直接地对应实现。简单系统中的二者很相近；复杂系统中的差别可能很大, 证明二者定义同一关系也会成为重要工作。第二十八章将以 $F_{<}$ 为例, 证明声明式子类型规则和算法式子类型规则定义了相同的关系；研究者也在许多其他类型系统中做过同类工作。

声明式系统与算法式系统最显眼的区别之一, 是前者含有显式传递规则——若 $S <: U$ 且 $U <: T$, 则 $S <: T$ ——而后者没有。传递规则无法直接用于目标导向算法, 因为应用它就必须猜测中间类型 U 。它在声明式系统中却有两项作用：一是让传递性一目了然；二是让其他规则保持简单。例如, §15.2把记录的深度、宽度和字段置换规则分别表述, 因而每条规则都更容易理解；去掉传递规则后, 三者必须合并成一条同时处理所有情况的宏规则, §16.1采用的正是这种方式。

声明式系统能够加入传递规则, 来自归纳定义的一项性质。传递性要求子类型关系对传递规则闭合；有限类型的子类型关系本来就是一组规则的闭包, 因此只需把传递规则加入生成规则。抽象地说, 有以下命题。

命题 21.4.1. 设 F, G 单调, 并令 $H(X) = F(X) \cup G(X)$ 。则 μH 是同时对 F 和 G 闭合的最小集合。

证明. 由 $\mu H = H(\mu H) = F(\mu H) \cup G(\mu H)$, 可知 μH 同时 F -闭和 G -闭。反过来, 若 $F(X) \subseteq X$ 且 $G(X) \subseteq X$, 则 $H(X) \subseteq X$, 所以 X 是 H -闭的。Knaster-Tarski 定理说明 μH 是最小的 H -闭集合, 故 $\mu H \subseteq X$ 。□

这个办法不能直接用于以余归纳方式定义的关系（即取生成函数的最大不动点所得的关系）。下一题说明，若把传递规则加入这类关系的生成规则，所得最大不动点将退化为全关系 $U \times U$ ：任意两个元素都会被判定为具有该关系。

练习 21.4.2 (★). 设 F 是全集 U 上的生成函数。证明生成函数

$$F^{\text{TR}}(R) = F(R) \cup \text{TR}(R)$$

的最大不动点 νF^{TR} 等于 $U \times U$ 。

查看原书附录 A 中的 21.4.2 解答

因此，在余归纳情形中，我们舍弃声明式表示，只使用算法式表示。

21.5 成员关系检查

现在讨论本章的核心问题：给定全集 U 上的生成函数 F 和元素 $x \in U$ ，怎样判定 x 是否属于 νF 。最小不动点的成员关系检查留到练习 21.5.13 简要处理。

译者注：这个抽象问题直接对应递归类型的子类型检查。应用到子类型化时，全集 U 是类型对的全集，生成函数 F 是子类型生成函数 $\mathcal{S}$ ，待检查的元素 x 则是类型对 (S, T) 。因此，判断 $x \in \nu F$ ，就是判断 $S <: T$ 。后文先为一般的生成函数建立成员关系检查算法，再把它用于正则无限树类型和以有限语法表示递归类型的 μ -类型。

一个元素 x 可能由 F 以多种方式生成。凡满足 $x \in F(X)$ 的集合 $X \subseteq U$ ，都称为 x 的一个生成集。由单调性，生成集的任意超集仍是生成集，所以只需考虑极小生成集。下面进一步限制到可逆（invertible）的生成函数：每个元素至多有一个最小生成集。

定义 21.5.1. 对 $x \in U$ ，令

$$\mathcal{G}_x = \{X \subseteq U \mid x \in F(X)\}.$$

若每个 $\mathcal{G}_x$ 要么为空，要么含有唯一的最小集合 $X_{\min}$ ，即对任意 $X' \in \mathcal{G}_x$ 都有 $X_{\min} \subseteq X'$ ，则称生成函数 F 可逆。此时定义偏函数 $\text{support}_F: U \rightarrow \mathcal{P}(U)$ ：

$$\text{support}_F(x) = \begin{cases} X, & X \in \mathcal{G}_x \text{ 且对每个 } X' \in \mathcal{G}_x \text{ 都有 } X \subseteq X', \\ \uparrow, & \mathcal{G}_x = \emptyset. \end{cases}$$

其中 $\uparrow$ 表示无定义。把该函数提升到集合上：

$$\text{support}_F(X) = \begin{cases} \bigcup_{x \in X} \text{support}_F(x), & \text{若每个 } x \in X \text{ 的支持集均有定义,} \\ \uparrow, & \text{否则.} \end{cases}$$

上下文能够确定 F 时，常省略下标。

练习 21.5.2 (★★). 验证定义 21.3.1 和 21.3.2 的 $\mathcal{S}_f$ 和 $\mathcal{S}$ 都可逆，并写出相应的支持函数。

查看原书附录 A 中的 21.5.2 解答

成员关系算法的基本步骤，是反向追问“ F 怎样才能生成 x ”。可逆性保证答案至多有一个；若同一元素有多个彼此不可比较的生成方式，算法就可能需要探索组合数量的分支。下文只讨论可逆生成函数。

定义 21.5.3. 若 $\text{support}_F(x) \downarrow$ ，即支持集有定义，则称 x 得到 F 的支持；否则称 x 不受 F 支持。若 $\text{support}_F(x) = \emptyset$ ，则称 x 为基元素 (*ground element*)。

不受支持的元素不会出现在任何 $F(X)$ 中；基元素则出现在每个 $F(X)$ 中。可逆函数可以画成支持图。图21-2在全集 $\{a, b, c, d, e, f, g, h, i\}$ 上定义了函数 E ：从 x 指向 y 的箭头表示 $y \in \text{support}_E(x)$ 。带斜线的圆表示不受支持的元素。图中只有 i 不受支持，只有 g 是基元素。按定义， h 仍算得到支持，尽管它的支持集中含有不受支持的 i 。

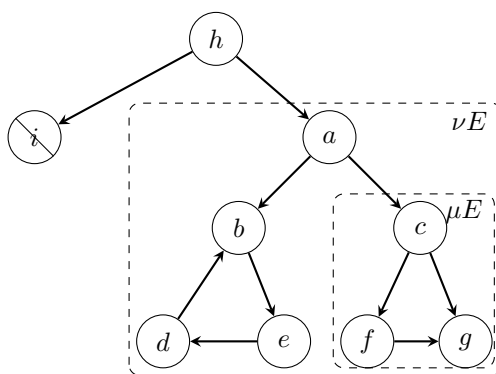


图 21-2: 支持函数示例

练习 21.5.4 (*). 像例21.1.3那样，给出与图21-2对应的推导规则。验证 $E(\{b, c\}) = \{g, a, d\}$ 、 $E(\{a, i\}) = \{g, h\}$ ，并验证图中标出的 μE 与 νE 确实分别为最小、最大不动点。

查看原书附录 A 中的 21.5.4 解答

从支持图可以看出： $x \in \nu F$ 当且仅当从 x 出发无法到达任何不受支持的元素。因此可以枚举从 x 出发经支持关系可达的全部元素；遇到不受支持的元素便失败，否则成功。支持图可能含环，枚举过程必须避免陷入无限循环。

定义 21.5.5. 设 F 可逆。定义布尔函数 gfp_F (简称 gfp):

$$\text{gfp}(X) = \begin{cases} \text{false}, & \text{support}(X) \uparrow, \\ \text{true}, & \text{support}(X) \subseteq X, \\ \text{gfp}(\text{support}(X) \cup X), & \text{否则}. \end{cases}$$

它从 X 开始不断加入支持集，直到得到 F -一致集合，或发现不受支持的元素。对单个元素约定 $\text{gfp}(x) = \text{gfp}(\{x\})$ 。

练习 21.5.6 (*). 若 $x \in \nu F$ 位于支持图中的环上，或可到达环上的元素，则 $x \notin \mu F$ 。逆命题是否成立？也就是说，若 $x \in \nu F \setminus \mu F$ ， x 是否必能到达某个环？

查看原书附录 A 中的 21.5.6 解答

本节余下部分证明 gfp 的正确性与终止性；初次阅读可以直接跳到下一节。先说明支持函数的两项性质。

引理 21.5.7.

$$X \subseteq F(Y) \iff \text{support}_F(X) \downarrow \text{ 且 } \text{support}_F(X) \subseteq Y.$$

证明. 只需对单个 x 证明。若 $x \in F(Y)$, 则 $Y \in \mathcal{G}_x$, 故 $\mathcal{G}_x \neq \emptyset$ 。可逆性保证最小生成集 $\text{support}(x)$ 存在, 且包含于 Y 。反过来, 若 $\text{support}(x) \subseteq Y$, 单调性给出 $F(\text{support}(x)) \subseteq F(Y)$; 而支持集的定义保证 $x \in F(\text{support}(x))$, 所以 $x \in F(Y)$ 。□

引理 21.5.8. 若 P 是 F 的不动点, 则

$$X \subseteq P \iff \text{support}_F(X) \downarrow \text{ 且 } \text{support}_F(X) \subseteq P.$$

证明. 由 $P = F(P)$ 和引理 21.5.7 立即得到。□

先证明 gfp 的部分正确性；此时尚不保证算法总会终止。

定理 21.5.9.

1. 若 $\text{gfp}_F(X) = \text{true}$, 则 $X \subseteq \nu F$;
2. 若 $\text{gfp}_F(X) = \text{false}$, 则 $X \not\subseteq \nu F$ 。

证明. 两项都对一次算法运行的递归结构作归纳。

若算法因 $\text{support}(X) \subseteq X$ 返回真, 引理 21.5.7 给出 $X \subseteq F(X)$, 所以 X 是 F -一致的, 余归纳原理给出 $X \subseteq \nu F$ 。若递归调用 $\text{gfp}(\text{support}(X) \cup X)$ 返回真, 归纳假设给出 $\text{support}(X) \cup X \subseteq \nu F$, 结论同样成立。

若算法因 $\text{support}(X) \uparrow$ 返回假, 引理 21.5.8 给出 $X \not\subseteq \nu F$ 。若递归调用返回假, 归纳假设说明 $\text{support}(X) \cup X \not\subseteq \nu F$, 即 X 或其支持集不包含于 νF ; 后一种情况再由引理 21.5.8 推出 $X \not\subseteq \nu F$ 。□

下面给出保证算法终止的一项充分条件。

定义 21.5.10. 对可逆生成函数 F 和 $x \in U$, 定义 x 的直接前驱集合

$$\text{pred}_F(x) = \begin{cases} \emptyset, & \text{support}_F(x) \uparrow, \\ \text{support}_F(x), & \text{support}_F(x) \downarrow, \end{cases} \quad \text{pred}_F(X) = \bigcup_{x \in X} \text{pred}_F(x).$$

从 X 沿支持关系可达的全部元素组成

$$\text{reachable}_F(X) = \bigcup_{n \geq 0} \text{pred}_F^n(X), \quad \text{reachable}_F(x) = \text{reachable}_F(\{x\}).$$

若 $y \in \text{reachable}_F(x)$, 便称 y 从 x 可达。

定义 21.5.11. 若每个 $x \in U$ 的 $\text{reachable}_F(x)$ 都有限, 则称可逆生成函数 F 为有限状态的 (*finite state*)。

定理 21.5.12. 若 $\text{reachable}_F(X)$ 有限, 则 $\text{gfp}_F(X)$ 有定义。因而, 若 F 是有限状态的, 则算法对任意有限 $X \subseteq U$ 都终止。

证明. 从初始调用 $\text{gfp}(X)$ 产生的每次递归调用 $\text{gfp}(Y)$, 都满足 $Y \subseteq \text{reachable}(X)$ 。每次继续递归时, 都会向 Y 加入至少一个此前尚未包含的可达元素, 使下一次调用的参数集合严格包含 Y 。由于可达集有限,

$$m(Y) = |\text{reachable}(X)| - |Y|$$

是严格递减且非负的终止度量。 □

译者注: 对递归调用次数归纳可知, 每次调用的参数集合 Y 都包含于 $\text{reachable}(X)$: 初始参数就是 X , 而从可达元素再沿一条支持边得到的元素仍然可达。算法只有在 $\text{support}(Y) \not\subseteq Y$ 时才会继续递归。此时存在 $z \in \text{support}(Y) \setminus Y$, 所以下一次调用的参数

$$Y' = Y \cup \text{support}(Y)$$

满足 $Y \subsetneq Y'$ 。这样, 全部递归参数都被限制在有限集合内, 每次递归却又至少加入一个新元素, 因而不可能无限递归。

练习 21.5.13 (*).** 设 F 可逆, 定义

$$\text{lfp}(X) = \begin{cases} \text{false}, & \text{support}(X) \uparrow, \\ \text{true}, & X = \emptyset, \\ \text{lfp}(\text{support}(X)), & \text{否则}. \end{cases}$$

它从 X 开始沿支持关系缩减集合, 直至集合为空。证明该算法部分正确:

1. 若 $\text{lfp}_F(X) = \text{true}$, 则 $X \subseteq \mu F$;
2. 若 $\text{lfp}_F(X) = \text{false}$, 则 $X \not\subseteq \mu F$ 。

再找出一类生成函数, 使 lfp_F 对所有有限输入都保证终止。

[查看原书附录 A 中的 21.5.13 解答](#)

21.6 更高效的算法

上一节的 gfp 每次递归都会重新计算整个集合的支持集。例如, 对图21-2的 E 有

$$\begin{aligned} \text{gfp}(\{a\}) &= \text{gfp}(\{a, b, c\}) \\ &= \text{gfp}(\{a, b, c, e, f, g\}) \\ &= \text{gfp}(\{a, b, c, e, f, g, d\}) \\ &= \text{true}, \end{aligned}$$

其中 $\text{support}(a)$ 被重复计算四次。可以用假设集 A 记录已经检查过支持集的元素, 用目标集 X 记录尚待检查的元素。

定义 21.6.1. 设 F 可逆, 定义函数 gfp_F^a , 其中上标 a 表示 assumptions:

$$\text{gfp}^a(A, X) = \begin{cases} \text{false}, & \text{support}(X) \uparrow, \\ \text{true}, & X = \emptyset, \\ \text{gfp}^a(A \cup X, \text{support}(X) \setminus (A \cup X)), & \text{否则}. \end{cases}$$

检查 $x \in \nu F$ 时计算 $\text{gfp}^a(\emptyset, \{x\})$ 。

该算法以及本节后面的两个算法, 都至多计算每个元素的支持集一次。上例的运行轨迹为

$$\begin{aligned} \text{gfp}^a(\emptyset, \{a\}) &= \text{gfp}^a(\{a\}, \{b, c\}) \\ &= \text{gfp}^a(\{a, b, c\}, \{e, f, g\}) \\ &= \text{gfp}^a(\{a, b, c, e, f, g\}, \{d\}) \\ &= \text{gfp}^a(\{a, b, c, e, f, g, d\}, \emptyset) \\ &= \text{true}. \end{aligned}$$

定理 21.6.2.

1. 若 $\text{support}_F(A) \subseteq A \cup X$ 且 $\text{gfp}_F^a(A, X) = \text{true}$, 则 $A \cup X \subseteq \nu F$;
2. 若 $\text{gfp}_F^a(A, X) = \text{false}$, 则 $X \not\subseteq \nu F$ 。

证明. 与定理21.5.9相似。 □

译者注: 条件 $\text{support}_F(A) \subseteq A \cup X$ 表示: 已检查元素的每项依赖, 要么已经检查过而属于 A , 要么已经列入待检查集合 X 。例如, 若 $A = \{a, b\}$ 、 $X = \{c\}$, 而 $\text{support}_F(a) = \{b, c\}$ 、 $\text{support}_F(b) = \{a\}$, 则 $\text{support}_F(A) = \{a, b, c\} \subseteq A \cup X$: a 依赖的 b 已经检查过, c 尚未检查但已经列入 X , b 依赖的 a 也已经检查过。

标准初始调用 $\text{gfp}^a(\emptyset, \{x\})$ 满足该条件。若某次调用满足它, 下一次调用的参数为

$$A' = A \cup X, \quad X' = \text{support}_F(X) \setminus (A \cup X).$$

这一定义把当前待检查元素 X 全部移入已检查集合 A' , 再把它们尚未处理的依赖放入新的待检查集合 X' 。原有条件给出 $\text{support}_F(A) \subseteq A' = A \cup X$; 而按照 X' 的定义, $\text{support}_F(X)$ 中的每个元素或者已经属于 A' , 或者属于 X' 。因此

$$\text{support}_F(A') = \text{support}_F(A) \cup \text{support}_F(X) \subseteq A' \cup X'$$

其中等号来自 $A' = A \cup X$ 。这就说明递归进入下一轮后, 同一条件仍然成立。定理允许用一般的 A, X 调用算法, 所以必须显式写出这一条件; 否则, 若 $X = \emptyset$, 算法会立即返回真, 即使 A 的某项依赖既不在 A 中, 也未列入 X 。

本节余下部分给出两个更接近常见递归子类型算法的变体; 初次阅读可以跳到下一节。

定义 21.6.3. 算法 gfp^s 每次只从 X 中取一个元素并展开其支持集，其中上标 s 表示 single:

$$\text{gfp}^s(A, X) = \begin{cases} \text{true}, & X = \emptyset, \\ \text{gfp}^s(A, X \setminus \{x\}), & x \in A, \\ \text{false}, & x \notin A \text{ 且 } \text{support}(x) \uparrow, \\ \text{gfp}^s(A \cup \{x\}, (X \cup \text{support}(x)) \setminus (A \cup \{x\})), & \text{否则}, \end{cases}$$

其中在 $X \neq \emptyset$ 的分支任取 $x \in X$ 。其正确性不变量与 [定理21.6.2](#)完全相同。

许多现有递归子类型算法一次只接收一个候选元素，而不是候选集合。再作一次小改动即可得到这种形式。新算法不再尾递归，因为调用栈负责记住尚未检查的子目标；它同时把假设集 A 作为参数，并把更新后的假设集作为结果，从而复用已经完成的递归调用所发现的假设。假设集就这样贯穿整个调用图，因此算法记作 gfp^t 。³

定义 21.6.4. 设 F 可逆，定义

$$\text{gfp}^t(A, x) = \begin{cases} A, & x \in A, \\ \text{fail}, & \text{support}(x) \uparrow, \\ A_n, & \text{否则}, \end{cases}$$

最后一项中的 A_n 依次计算如下:

$$\begin{aligned} \{x_1, \dots, x_n\} &= \text{support}(x), & A_0 &= A \cup \{x\}, \\ A_1 &= \text{gfp}^t(A_0, x_1), & \dots, & A_n = \text{gfp}^t(A_{n-1}, x_n). \end{aligned}$$

检查 $x \in \nu F$ 时计算 $\text{gfp}^t(\emptyset, x)$: 调用成功表示属于，失败表示不属于。上述顺序计算还约定失败自动向外传播: 若某一步递归调用失败，后续步骤便不再执行，整个调用也返回失败。采用这一约定后，就不必在每次递归调用旁重复写出失败处理。

译者注: 判断递归调用是否处于尾位置，关键不在于它是否写在最后一行，而在于调用返回后，当前函数是否还有工作要做。粗略地说，`return f(y)` 中的调用是尾调用，因为 `f(y)` 的结果会直接成为当前函数的结果；`return 1 + f(y)` 中的调用则不是，因为 `f(y)` 返回后还要执行加法。从上式可以看出本算法为何不是尾递归：计算出 $A_1 = \text{gfp}^t(A_0, x_1)$ 后，当前调用还要用返回的 A_1 继续检查 x_2 ；得到 A_2 后还要继续检查 x_3 ，依此类推。因此，对 x_i 的递归调用返回后，调用者仍有后续工作，不能把它的结果直接作为自身的结果返回；尚未检查的 $x_{i+1}, \dots, x_n$ 便由调用栈暂时记住。

由于该算法不是尾递归，正确性陈述还要加入一个“栈”集合 X ，表示支持集尚待检查的元素。

³尾递归调用（或尾调用）是调用者函数执行的最后一个动作；也就是说，递归调用返回的结果会直接成为调用者的结果。尾调用之所以值得注意，是因为多数函数式语言编译器会把它实现为一次简单跳转：复用调用者的栈空间，而不为递归调用分配新的栈帧。因此，以尾递归函数实现的循环可以编译成与等价的 `while` 循环相同的机器代码。

引理 21.6.5.

1. 若 $\text{gfp}_F^t(A, x) = A'$, 则 $A \cup \{x\} \subseteq A'$;
2. 对任意 X , 若 $\text{support}_F(A) \subseteq A \cup X \cup \{x\}$ 且 $\text{gfp}_F^t(A, x) = A'$, 则 $\text{support}_F(A') \subseteq A' \cup X$.

译者注: 第 1 项说明, 算法成功运行时只会扩充假设集: 原有的 A 不会丢失, 当前检查的 x 也会被加入结果 A' 。第 2 项说明, 算法完成后仍保持同一个关键条件: 结果集中各元素的依赖, 要么已经收入结果集合 A' , 要么仍由调用栈中的待检查集合 X 负责。

证明. 两项都对算法成功运行时形成的递归调用结构作归纳。

先证第 1 项。若 $x \in A$, 算法直接返回 $A' = A$, 而 $A \cup \{x\} = A$ 。否则, 由算法成功返回 A' 可知 $\text{support}(x)$ 有定义。设 $\text{support}(x) = \{x_1, \dots, x_n\}$ 。算法先令 $A_0 = A \cup \{x\}$, 再依次计算 $A_i = \text{gfp}^t(A_{i-1}, x_i)$, 最后返回 $A' = A_n$ 。对每个较小的递归调用应用第 1 项的归纳假设, 可知 $A_{i-1} \subseteq A_i$ 。因此 $A \cup \{x\} = A_0 \subseteq A_n = A'$ 。

再证第 2 项。任取 X_0 , 并假设

$$\text{support}(A) \subseteq A \cup X_0 \cup \{x\}$$

若 $x \in A$, 算法返回 $A' = A$; 此时 $\{x\} \subseteq A$, 所以上述前提立即给出 $\text{support}(A') \subseteq A' \cup X_0$ 。

先讨论 $\text{support}(x) = \{x_1, x_2\}$ 的情形。算法依次计算

$$A_0 = A \cup \{x\}, \quad A_1 = \text{gfp}^t(A_0, x_1), \quad A_2 = \text{gfp}^t(A_1, x_2)$$

并返回 $A' = A_2$ 。证明的关键是依次对这两次递归调用使用归纳假设: 处理 x_1 时, 尚未处理的 x_2 仍须列入待检查集合; 处理 x_2 时, 它本身成为当前检查的元素, 待检查集合便恢复为 X_0 。

具体而言, 令 $X_1 = X_0 \cup \{x_2\}$ 。为了把归纳假设应用于 $A_1 = \text{gfp}^t(A_0, x_1)$, 需要先验证 $\text{support}(A_0) \subseteq A_0 \cup X_1 \cup \{x_1\}$ 。这由

$$\begin{aligned} \text{support}(A_0) &= \text{support}(A) \cup \text{support}(x) \\ &= \text{support}(A) \cup \{x_1, x_2\} \\ &\subseteq A \cup \{x\} \cup X_0 \cup \{x_1, x_2\} \\ &= A_0 \cup X_1 \cup \{x_1\} \end{aligned}$$

得到。于是, 第 2 项的归纳假设给出

$$\text{support}(A_1) \subseteq A_1 \cup X_1 = A_1 \cup X_0 \cup \{x_2\}$$

这正是把归纳假设应用于 $A_2 = \text{gfp}^t(A_1, x_2)$ 时所需的前提, 其中引理中的辅助集合 X 取为 X_0 。因此

$$\text{support}(A_2) \subseteq A_2 \cup X_0$$

又因 $A' = A_2$, 结论成立。

若 $\text{support}(x)$ 含有任意有限个元素, 论证完全相同: 依次处理 x_i 时, 把 $x_{i+1}, \dots, x_n$ 与 X_0 一同视为尚待检查的元素。形式上对支持集大小作内层归纳, 即得一般结论。 $\square$

定理 21.6.6.

1. 若 $\text{gfp}_F^t(\emptyset, x) = A'$, 则 $x \in \nu F$;
2. 若 $\text{gfp}_F^t(\emptyset, x) = \text{fail}$, 则 $x \notin \nu F$ 。

证明. 第 1 项中, 引理21.6.5第 1 项给出 $x \in A'$; 再把第 2 项的 X 取为空集, 得到 $\text{support}(A') \subseteq A'$, 所以 A' 是 F -一致的, 余归纳原理给出 $A' \subseteq \nu F$ 。第 2 项对失败运行的深度作归纳, 并使用引理21.5.8。□

译者注: 第 2 项的归纳论证可以展开如下。先把结论加强为: 对任意假设集 A , 若 $\text{gfp}_F^t(A, x) = \text{fail}$, 则 $x \notin \nu F$; 再对这次失败运行的递归深度作归纳。由于 νF 是 F 的不动点, 引理21.5.8对单个元素给出

$$x \in \nu F \iff \text{support}_F(x) \downarrow \text{ 且 } \text{support}_F(x) \subseteq \nu F$$

若当前调用因 $\text{support}_F(x) \uparrow$ 直接失败, 上式立即说明 $x \notin \nu F$ 。否则, 失败来自某次较小的递归调用 $\text{gfp}_F^t(A_{i-1}, x_i) = \text{fail}$, 其中 $x_i \in \text{support}_F(x)$ 。归纳假设给出 $x_i \notin \nu F$, 因而 $\text{support}_F(x) \not\subseteq \nu F$; 再次使用上述等价关系, 即得 $x \notin \nu F$ 。也就是说, 最深处的不受支持元素先被判定为不属于 νF , 此结论再沿失败的调用栈逐层向外传递。最后取 $A = \emptyset$, 便得到定理第 2 项。

本节所有算法都会检查可达集, 因此, 只要 F 是有限状态的, 它们便会对所有输入终止。

21.7 正则树

至此, 我们已经为如下问题给出了通用算法: 若一个集合被定义为生成函数 F 的最大不动点, 怎样判断给定元素是否属于这个集合; 这里假定 F 可逆且具有有限状态。另一方面, 我们已经把无限树之间的子类型关系定义为特定生成函数 S 的最大不动点。接下来要做的很明确: 把 S 代入前述算法, 得到具体的子类型检查算法。当然, 这一算法不会对所有输入都终止, 因为从一对无限类型出发能够到达的状态一般可能有无穷多个。不过本节将会看到, 若把讨论限制在某种性质良好的无限类型, 即所谓的正则类型 (*regular type*) 上, 可达状态集合便一定有限, 子类型检查算法也必然终止。

定义 21.7.1. 若存在路径 π , 使

$$S = \lambda\sigma. T(\pi, \sigma),$$

则称树类型 S 是树类型 T 的子树。换句话说, 把树看作从路径到符号的函数时, S 是在传给 T 的路径参数前加上固定前缀 π 得到的。这个前缀是从 T 的根到 S 的根的路径。 T 的全部子树组成的集合记作 $\text{subtrees}(T)$ 。

定义 21.7.2. 若 $\text{subtrees}(T)$ 有限, 则称树类型 T 正则。全部正则树类型的集合记作 $\mathcal{T}_r$ 。

例 21.7.3.

1. 每个有限树类型都正则；不同子树的数量不超过节点数。例如 $T = \text{Top} \rightarrow (\text{Top} \times \text{Top})$ 有五个节点，却只有 T 自身、 $\text{Top} \times \text{Top}$ 和 Top 三棵不同子树。
2. 有些无限树也是正则的。例如 $T = \text{Top} \times (\text{Top} \times (\text{Top} \times \cdots))$ 只有 T 自身和 Top 两棵不同子树。
3. 树

$$T = B \times (A \times (B \times (A \times (A \times (B \times (A \times (A \times (A \times (B \times \cdots))))))))))$$

中，相邻的 B 之间依次夹着越来越多个 A ，因此它不正则。证明 $T <: T$ 所需的可达子类型对也会有无限多个。

命题 21.7.4. 把生成函数 $\mathcal{S}$ 的作用范围限定为正则树类型后，所得的生成函数 $\mathcal{S}_r$ 是有限状态的。

证明. 对任意正则树类型对 (S, T) ,

$$\text{reachable}_{\mathcal{S}_r}(S, T) \subseteq (\text{subtrees}(S) \times \text{subtrees}(T)) \cup (\text{subtrees}(T) \times \text{subtrees}(S)).$$

函数参数位置的逆变可能交换有序对的两个分量，因此需要同时包含这两个方向。两个子树集合都有限，右侧的并集也有限，故左侧有限。 $\square$

译者注：证明右侧的两个笛卡尔积列出了子类型检查过程中所有可能遇到的类型对。以检查 $S_1 \rightarrow S_2 <: T_1 \rightarrow T_2$ 为例，箭头规则接下来要求检查 $T_1 <: S_1$ 和 $S_2 <: T_2$ ：参数位置因逆变而交换方向，结果位置则保持原方向。因此，后续遇到的类型对只可能落在证明所列的两个笛卡尔积中。正则树虽然可以无限展开，却只有有限多棵互不相同的子树，所以这样的类型对只有有限多个；实现只要记住已经检查过的类型对，便不会因循环而无限递归。

因此，把成员关系算法用于 $\mathcal{S}_r$ ，就得到正则树类型子类型关系的判定过程。实际实现还需要有限结构来表示正则树；下一节介绍一种这样的结构，即 μ -记法。

21.8 μ -类型

本节给出有限的 μ -记法，定义 μ -表达式上的子类型关系，并证明它与树类型上的子类型关系相对应。

定义 21.8.1. 预先固定一系列类型变量名 $X_1, X_2, \dots$ ，并用 X 表示其中任意一个。原始 μ -类型 (*raw μ -type*) 由以下文法生成：

$$T ::= X \mid \text{Top} \mid T \times T \mid T \rightarrow T \mid \mu X. T$$

全部原始 μ -类型的集合记作 $\mathcal{T}_m^{\text{raw}}$ 。在 $\mu X. T$ 中， μX 绑定后面类型表达式 T 中出现的 X ，其作用与 λ 抽象中的变量绑定相同。因此，我们可以区分类型中的自由变量与受约束变量，并把不含自由变量的原始 μ -类型称为闭类型。两个原始 μ -类型如果只在受约束变量的名称上不同，就视为相同。 $\text{FV}(T)$ 表示 T 中自由变量的集合； $[X \mapsto S]T$ 表示把 T 中 X 的所有自由出现替换为 S 。替换时若可能发生变量捕获，就先对相应的受约束变量重命名。

译者注：“左”“右”指 μ -类型出现在子类型判断符号 $<$ 的哪一侧。沿推导树从结论向前提读，两条规则分别展开右侧或左侧的递归类型；反向从前提向结论读，则是把展开形式折叠回 μ -类型。

定义 21.8.4. 若 $(S, T) \in \nu\mathcal{S}_\mu$ ，则称 μ -类型 S 是 T 的子类型，其中 $\mathcal{S}_\mu : \mathcal{P}(\mathcal{T}_m \times \mathcal{T}_m) \rightarrow \mathcal{P}(\mathcal{T}_m \times \mathcal{T}_m)$ 定义为

$$\begin{aligned} \mathcal{S}_\mu(R) = & \{(S, \text{Top}) \mid S \in \mathcal{T}_m\} \\ & \cup \{(S_1 \times S_2, T_1 \times T_2) \mid (S_1, T_1), (S_2, T_2) \in R\} \\ & \cup \{(S_1 \rightarrow S_2, T_1 \rightarrow T_2) \mid (T_1, S_1), (S_2, T_2) \in R\} \\ & \cup \{(S, \mu X.T) \mid (S, [X \mapsto \mu X.T]T) \in R\} \\ & \cup \{(\mu X.S, T) \mid ([X \mapsto \mu X.S]S, T) \in R, \quad T \neq \text{Top}, T \neq \mu Y.T_1\}. \end{aligned}$$

最后两项故意不对称：若两边都以 μ 开头，或右边是 Top ，最后两项原本会重叠；加入条件后，生成函数才可逆。更自然的声明式生成函数 $\mathcal{S}_d$ 与上面的折叠规则逐项对应，下一题说明它虽不可逆，却生成同一个最大不动点。⁴

练习 21.8.5 (★★). 写出 $\mathcal{S}_d$ ，证明它不可逆，并证明 $\nu\mathcal{S}_d = \nu\mathcal{S}_\mu$ 。

查看原书附录 A 中的 21.8.5 解答

$\mathcal{S}_\mu$ 的支持函数为

$$\text{support}_{\mathcal{S}_\mu}(S, T) = \begin{cases} \emptyset, & T = \text{Top}, \\ \{(S_1, T_1), (S_2, T_2)\}, & S = S_1 \times S_2, T = T_1 \times T_2, \\ \{(T_1, S_1), (S_2, T_2)\}, & S = S_1 \rightarrow S_2, T = T_1 \rightarrow T_2, \\ \{(S, [X \mapsto \mu X.T_1]T_1)\}, & T = \mu X.T_1, \\ \{([X \mapsto \mu X.S_1]S_1, T)\}, & S = \mu X.S_1, T \neq \mu Y.T_1, T \neq \text{Top}, \\ \uparrow, & \text{其他情形}. \end{cases}$$

还需确认：有限的 μ -表示与无限树表示给出的子类型关系确实一致。

引理 21.8.6. 设 $R \subseteq \mathcal{T}_m \times \mathcal{T}_m$ 是 $\mathcal{S}_\mu$ -一致的。对任意 $(S, T) \in R$ ，存在 $(S', T') \in R$ ，使 $\text{treeof}(S', T') = \text{treeof}(S, T)$ ，并且或者 $T' = \text{Top}$ ，或者 S', T' 都不以 μ 开头。

证明. 以 S 与 T 最外层连续出现的 μ 绑定总数为归纳对象。若 $T = \text{Top}$ ，取 $(S', T') = (S, T)$ ；若两边都不以 μ 开头，也取 $(S', T') = (S, T)$ 。

若 $(S, T) = (S, \mu X.T_1)$ ，由于 R 是 $\mathcal{S}_\mu$ -一致的，有 $R \subseteq \mathcal{S}_\mu(R)$ ，故

$$(S, \mu X.T_1) \in \mathcal{S}_\mu(R).$$

根据定义 21.8.4，右侧以 μ 开头的类型对只能由右 μ -折叠分支生成。因此

$$(S, [X \mapsto \mu X.T_1]T_1) \in R.$$

⁴下标 d 表示该函数来自声明式 (declarative) 的 μ -折叠规则； $\mathcal{S}_\mu$ 则采用算法式分支。

收缩性保证把右侧的 $\mu X.T_1$ 展开一层后, 所得类型最外层连续出现的 μ 绑定恰好少一个, 故可应用归纳假设。按照 treeof 对递归类型的定义, 类型与其展开一层所得的类型表示同一棵无限树, 因此这一步展开不会改变 treeof 的结果。

最后考虑左侧以 μ 开头的情形。前两种情形已经排除了 $T = \text{Top}$ 和右侧以 μ 开头的情况; 此时 $\mathcal{S}_\mu$ 的左 μ -折叠分支要求展开左侧, 并在 R 中给出相应的类型对。展开使归纳量减一, 余下论证与右侧情形相同。 $\square$

译者注: 原书引理的结论没有列出 $T' = \text{Top}$ 这一例外。例如 $R = \{(\mu X.\text{Top}, \text{Top})\}$ 是 $\mathcal{S}_\mu$ -一致的, 因为 $\mathcal{S}_\mu$ 的第一项支持任意 (S, Top) ; 但 R 中没有左侧不以 μ 开头的类型对。译文把引理修正为后续定理实际需要的形式: 右侧为 Top 时直接处理, 其余情形才把两侧展开到都不以 μ 开头。

定理 21.8.7. 对 $(S, T) \in \mathcal{T}_m \times \mathcal{T}_m$,

$$(S, T) \in \nu\mathcal{S}_\mu \iff \text{treeof}(S, T) \in \nu\mathcal{S}$$

证明. 先证从左到右, 即由 $(S, T) \in \nu\mathcal{S}_\mu$ 推出 $\text{treeof}(S, T) \in \nu\mathcal{S}$ 。令 $(A, B) = \text{treeof}(S, T) \in \mathcal{T} \times \mathcal{T}$ 。按照余归纳原理, 只要能找到一个包含 (A, B) 的 $\mathcal{S}$ -一致关系 $Q \subseteq \mathcal{T} \times \mathcal{T}$, 结论便成立。取

$$Q = \text{treeof}(\nu\mathcal{S}_\mu) = \{\text{treeof}(S', T') \mid (S', T') \in \nu\mathcal{S}_\mu\}$$

这里把 treeof 逐元素作用于关系 $\nu\mathcal{S}_\mu$ 中的每个类型对。要证明 Q 是 $\mathcal{S}$ -一致的, 必须证明: 对每个 $(A', B') \in Q$, 都有 $(A', B') \in \mathcal{S}(Q)$ 。

任取 $(A', B') \in Q$, 并选取 $(S', T') \in \nu\mathcal{S}_\mu$, 使得 $\text{treeof}(S', T') = (A', B')$ 。由引理21.8.6, 可以选择这样的 (S', T') : 或者 $T' = \text{Top}$, 或者 S' 和 T' 都不以 μ 开头。由于 $\nu\mathcal{S}_\mu$ 是 $\mathcal{S}_\mu$ -一致的, (S', T') 必须由 $\mathcal{S}_\mu$ 定义中的某一项支持; 在上述选择下, 只需讨论以下三种形式。

情形: $(S', T') = (S', \text{Top})$ 。此时 $B' = \text{Top}$, 由 $\mathcal{S}$ 的定义立即得到 $(A', B') \in \mathcal{S}(Q)$ 。

情形:

$$(S', T') = (S_1 \times S_2, T_1 \times T_2), \quad (S_1, T_1), (S_2, T_2) \in \nu\mathcal{S}_\mu$$

由 treeof 的定义, $B' = \text{treeof}(T') = B_1 \times B_2$, 其中 $B_i = \text{treeof}(T_i)$; 类似地, $A' = A_1 \times A_2$, 其中 $A_i = \text{treeof}(S_i)$ 。分别对上面的两个类型对应用 treeof , 得到

$$(A_1, B_1), (A_2, B_2) \in Q$$

再由 $\mathcal{S}$ 的定义可知

$$(A', B') = (A_1 \times A_2, B_1 \times B_2) \in \mathcal{S}(Q)$$

情形:

$$(S', T') = (S_1 \rightarrow S_2, T_1 \rightarrow T_2), \quad (T_1, S_1), (S_2, T_2) \in \nu\mathcal{S}_\mu$$

论证与积类型情形类似; 函数参数位置的类型对按逆变方向排列。由此得到 $(A', B') \in \mathcal{S}(Q)$ 。

三种情形都满足要求, 故 $Q \subseteq \mathcal{S}(Q)$, 即 Q 是 $\mathcal{S}$ -一致的。余归纳原理于是给出 $(A, B) \in \nu\mathcal{S}$, 从左到右的方向得证。

再证从右到左, 即由 $\text{treeof}(S, T) \in \nu\mathcal{S}$ 推出 $(S, T) \in \nu\mathcal{S}_\mu$ 。按照余归纳原理, 只需找到一个包含 (S, T) 的 $\mathcal{S}_\mu$ -一致关系 $R \subseteq \mathcal{T}_m \times \mathcal{T}_m$ 。取

$$R = \{(S', T') \in \mathcal{T}_m \times \mathcal{T}_m \mid \text{treeof}(S', T') \in \nu\mathcal{S}\}$$

显然 $(S, T) \in R$ 。为了完成证明, 还需证明: 若 $(S', T') \in R$, 则 $(S', T') \in \mathcal{S}_\mu(R)$ 。

由于 $\nu\mathcal{S}$ 是 $\mathcal{S}$ -一致的, 其中任意类型对 (A', B') 只能具有以下三种形式之一: (A', Top) 、 $(A_1 \times A_2, B_1 \times B_2)$ 或 $(A_1 \rightarrow A_2, B_1 \rightarrow B_2)$ 。结合 treeof 的定义可知, R 中的任意类型对 (S', T') 必须具有以下五种形式之一: (S', Top) 、 $(S_1 \times S_2, T_1 \times T_2)$ 、 $(S_1 \rightarrow S_2, T_1 \rightarrow T_2)$ 、 $(S', \mu X.T_1)$ 或 $(\mu X.S_1, T')$ 。下面逐一讨论。

情形: $(S', T') = (S', \text{Top})$ 。由 $\mathcal{S}_\mu$ 的定义, 立即有 $(S', T') \in \mathcal{S}_\mu(R)$ 。

情形: $(S', T') = (S_1 \times S_2, T_1 \times T_2)$ 。令 $(A', B') = \text{treeof}(S', T')$, 则

$$(A', B') = (A_1 \times A_2, B_1 \times B_2)$$

其中 $A_i = \text{treeof}(S_i)$, $B_i = \text{treeof}(T_i)$ 。由 $(S', T') \in R$ 的定义可知 $(A', B') \in \nu\mathcal{S}$ 。又因为 $\nu\mathcal{S}$ 是 $\mathcal{S}$ -一致的, 所以

$$(A_1, B_1), (A_2, B_2) \in \nu\mathcal{S}$$

再由 R 的定义得到 $(S_1, T_1), (S_2, T_2) \in R$ 。因此, $\mathcal{S}_\mu$ 定义中的积类型分支给出

$$(S', T') = (S_1 \times S_2, T_1 \times T_2) \in \mathcal{S}_\mu(R)$$

情形: $(S', T') = (S_1 \rightarrow S_2, T_1 \rightarrow T_2)$ 。论证类似; 函数参数位置使用反向的类型对 (T_1, S_1) , 结果位置使用 (S_2, T_2) 。

情形: $(S', T') = (S', \mu X.T_1)$ 。令

$$T'' = [X \mapsto \mu X.T_1]T_1$$

根据定义, $\text{treeof}(T'') = \text{treeof}(T')$ 。因此, 由 R 的定义可知 $(S', T'') \in R$, 再由 $\mathcal{S}_\mu$ 的定义得到 $(S', T') \in \mathcal{S}_\mu(R)$ 。

情形: $(S', T') = (\mu X.S_1, T')$ 。若 $T' = \text{Top}$, 则归入第一种情形; 若 T' 也以 μ 开头, 则归入“右侧为 μ -类型”的情形。余下只需考虑 T' 既不是 Top 、也不以 μ 开头的情况; 此时展开左侧的递归类型, 论证与展开右侧时对称。

以上五种情形都给出 $(S', T') \in \mathcal{S}_\mu(R)$, 故 $R \subseteq \mathcal{S}_\mu(R)$, 即 R 是 $\mathcal{S}_\mu$ -一致的。余归纳原理于是给出 $(S, T) \in \nu\mathcal{S}_\mu$, 从右到左的方向得证。□

该定理表明, μ -类型的子类型关系相对于其所表示的无限树子类型关系既可靠又完备。

21.9 子表达式计数

把定义21.6.4的一般算法 gfp^t 用于 定义21.8.4的支持函数, 就得到图21-4的具体子类型算法。§21.6已经说明, 只要 $\text{reachable}_{\mathcal{S}_\mu}(S, T)$ 对每对 μ -类型都有限, 算法便会终止; 本节证明这一点。

$$\text{subtype}(A, S, T) = \begin{cases} A, & (S, T) \in A, \\ \text{subtype}'(A \cup \{(S, T)\}, S, T), & \text{否则}, \end{cases}$$

$$\text{subtype}'(A_0, S, T) = \begin{cases} A_0, & T = \text{Top}, \\ \text{subtype}(A_1, S_2, T_2), & \begin{matrix} S = S_1 \times S_2, T = T_1 \times T_2, \\ A_1 = \text{subtype}(A_0, S_1, T_1), \end{matrix} \\ \text{subtype}(A_1, S_2, T_2), & \begin{matrix} S = S_1 \rightarrow S_2, T = T_1 \rightarrow T_2, \\ A_1 = \text{subtype}(A_0, T_1, S_1), \end{matrix} \\ \text{subtype}(A_0, S, [X \mapsto \mu X.T_1]T_1), & T = \mu X.T_1, \\ \text{subtype}(A_0, [X \mapsto \mu X.S_1]S_1, T), & S = \mu X.S_1, \\ \text{fail}, & \text{其他情形}. \end{cases}$$

图 21-4: μ -类型的具体子类型算法

看起来这几乎显然, 严格证明却需要不少工作。困难在于, “闭子表达式” 有两种定义。自顶向下子表达式 (*top-down subexpression*) 正好对应支持函数在展开时产生的表达式; 自底向上子表达式 (*bottom-up subexpression*) 则便于证明任意闭 μ -类型只有有限多个闭子表达式。终止性证明会同时定义二者, 再证明前者包含于后者。以下论证依据 Brandt 与 Henglein (1997)。

定义 21.9.1. 若 $(S, T) \in \mu\text{TD}$, 则称 S 是 T 的自顶向下子表达式, 记作 $S \sqsubseteq T$, 其中

$$\begin{aligned} \text{TD}(R) = & \{(T, T) \mid T \in \mathcal{T}_m\} \\ & \cup \{(S, T_1 \times T_2) \mid (S, T_1) \in R\} \\ & \cup \{(S, T_1 \times T_2) \mid (S, T_2) \in R\} \\ & \cup \{(S, T_1 \rightarrow T_2) \mid (S, T_1) \in R\} \\ & \cup \{(S, T_1 \rightarrow T_2) \mid (S, T_2) \in R\} \\ & \cup \{(S, \mu X.T) \mid (S, [X \mapsto \mu X.T]T) \in R\}. \end{aligned}$$

练习 21.9.2 ($\star$). 用一组推导规则给出关系 $S \sqsubseteq T$ 的等价定义。

[查看原书附录 A 中的 21.9.2 解答](#)

引理 21.9.3. 若 $(S', T') \in \text{support}_{\mathcal{S}_\mu}(S, T)$, 则 $S' \sqsubseteq S$ 或 $S' \sqsubseteq T$, 且 $T' \sqsubseteq S$ 或 $T' \sqsubseteq T$ 。

证明. 逐项检查 $\text{support}_{\mathcal{S}_\mu}$ 的定义即可。 □

译者注: 本引理说明, 支持函数从 (S, T) 产生下一批待检查类型对时, 其中的类型都来自原输入 S 或 T 的自顶向下子表达式, 不会凭空出现新的类型。积类型分支产生的 (S_i, T_i) 分别来自 S 和 T ; 箭头类型的参数位置逆变, 产生的依赖项为 (T_1, S_1) , 左右来源会交换, 因此结论必须写成“是 S 或 T 的子表达式”, 不能固定对应哪一侧。递归类型分支只是把一侧展开一层, 而展开所得类型也属于原递归类型的自顶向下子表达式。这个结论随后用于证明: 算法从初始类型对出发能够到达的所有类型, 都受原输入子表达式集合的限制。

引理 21.9.4. 若 $S \sqsubseteq U$ 且 $U \sqsubseteq T$, 则 $S \sqsubseteq T$ 。

证明. 令

$$R = \{(U, T) \mid \forall S. S \sqsubseteq U \Rightarrow S \sqsubseteq T\}$$

要证 $\mu\text{TD} \subseteq R$ 。依归纳原理, 只需证明 R 对 TD 闭合, 即 $\text{TD}(R) \subseteq R$ 。任取 $(U, T) \in \text{TD}(R)$, 按照它由 TD 定义中的哪一项生成, 分类讨论。若当前类型对由“每个类型都是自身的子表达式”这一分支生成, 即 $U = T$, 结论直接成立。若当前类型对由积类型的左分支生成, 则 $T = T_1 \times T_2$ 且 $(U, T_1) \in R$ 。要证明当前类型对 $(U, T_1 \times T_2)$ 也属于 R , 任取 $S \sqsubseteq U$; 由 $(U, T_1) \in R$ 的定义先有 $S \sqsubseteq T_1$, 再由 TD 的积类型分支得到 $S \sqsubseteq T_1 \times T_2$ 。其他分支相同。 $\square$

译者注: 本引理要证明: 只要 $S \sqsubseteq U$ 且 $U \sqsubseteq T$, 就有 $S \sqsubseteq T$ 。作者把待证结论编码进关系

$$R = \{(U, T) \mid \forall S. S \sqsubseteq U \Rightarrow S \sqsubseteq T\}$$

按照这个定义, $(U, T) \in R$ 表示 U 的每个子表达式也都是 T 的子表达式。因此, 只要证明每个满足 $U \sqsubseteq T$ 的类型对都属于 R , 也就是证明

$$\mu\text{TD} \subseteq R$$

便能立即得到传递性: 已知 $U \sqsubseteq T$, 即 $(U, T) \in \mu\text{TD}$, 上述包含关系给出 $(U, T) \in R$; 再按 R 的定义, 已知 $S \sqsubseteq U$ 时便有 $S \sqsubseteq T$ 。

还需说明为什么证明 R 对 TD 闭合就足够。 μTD 是对 TD 的生成规则闭合的最小关系; 最小不动点归纳原理因而给出

$$\text{TD}(R) \subseteq R \implies \mu\text{TD} \subseteq R$$

作者证明中的分类讨论, 正是在验证左边的闭合条件: 自身情形直接成立; 若 U 位于积类型或箭头类型的某个分量中, 就先使用归纳假设, 再套上最外层类型构造; 递归类型情形则在展开一层后作同样处理。直观地说, 整个证明只是把从 T 到 U 与从 U 到 S 的两段向下路径接成一段。

命题 21.9.5. 若 $(S', T') \in \text{reachable}_{S_\mu}(S, T)$, 则 $S' \sqsubseteq S$ 或 $S' \sqsubseteq T$, 且 $T' \sqsubseteq S$ 或 $T' \sqsubseteq T$ 。

证明. 对可达关系的定义作归纳, 使用引理21.9.3和21.9.4。 $\square$

由上一命题可知, 只要证明任意 μ -类型 U 的自顶向下子表达式集合有限, 就能推出从初始类型对出发的可达集有限。最自然的尝试是对 U 的语法结构作归纳, 但在 $U = \mu X.T$ 的情形下无法继续: 自顶向下定义需要考察可能更大的展开式 $[X \mapsto \mu X.T]T$, 无法直接对它应用结构归纳假设。自底向上定义把替换推迟到先收集完函数体的子表达式之后, 从而避开这个问题。

定义 21.9.6. 若 $(S, T) \in \mu\text{BU}$, 则称 S 是 T 的自底向上子表达式, 记作 $S \preceq T$, 其中

$$\begin{aligned} \text{BU}(R) = & \{(T, T) \mid T \in \mathcal{T}_m\} \\ & \cup \{(S, T_1 \times T_2) \mid (S, T_1) \in R\} \\ & \cup \{(S, T_1 \times T_2) \mid (S, T_2) \in R\} \\ & \cup \{(S, T_1 \rightarrow T_2) \mid (S, T_1) \in R\} \\ & \cup \{(S, T_1 \rightarrow T_2) \mid (S, T_2) \in R\} \\ & \cup \{([X \mapsto \mu X.T]S, \mu X.T) \mid (S, T) \in R\}. \end{aligned}$$

两个定义只在 μ 分支不同。自顶向下定义遇到 $\mu X.T$ 时，先将它展开为 $[X \mapsto \mu X.T]T$ ，再从展开后的类型中继续寻找子表达式。自底向上定义则先在有限的类型体 T 中寻找子表达式 S ，再将其中的 X 替换为完整的递归类型 $\mu X.T$ ，得到 $[X \mapsto \mu X.T]S$ 。

译者注：有序对 (A, B) 表示 A 是 B 的子表达式。令 $U = \mu X.T$ ，两种定义的 μ 分支可以分别写成

$$\frac{S \sqsubseteq [X \mapsto U]T}{S \sqsubseteq U} \quad (\text{自顶向下})$$

$$\frac{S_0 \preceq T}{[X \mapsto U]S_0 \preceq U} \quad (\text{自底向上})$$

自顶向下定义先展开完整的递归类型 U ，再到展开式 $[X \mapsto U]T$ 中寻找子表达式。自底向上定义则先在有限的类型体 T 中寻找子表达式 S_0 ，然后把 S_0 中原本由外层 μX 约束的 X 替换为 U ，使其重新成为完整递归类型中的子表达式。

例如，令

$$U = \mu X.(\text{Top} \times X)$$

自顶向下定义先将 U 展开为

$$\text{Top} \times U$$

再从这个展开式中找到 Top ，因而得到 $\text{Top} \sqsubseteq U$ 。自底向上定义先在类型体 $\text{Top} \times X$ 中找到 Top 。由于

$$[X \mapsto U]\text{Top} = \text{Top}$$

替换不改变它，因而得到 $\text{Top} \preceq U$ 。如果取整个类型体 $S_0 = \text{Top} \times X$ ，则

$$[X \mapsto U](\text{Top} \times X) = \text{Top} \times U$$

所以 $\text{Top} \times U \preceq U$ 。

两种定义的区别在于替换发生的时机：自顶向下定义先替换整个类型体，再寻找子表达式；自底向上定义先寻找类型体的子表达式，再分别对它们作替换。这里的“先”和“再”描述的是规则前提与结论之间的依赖关系，并不表示这些规则必须按照固定轮次执行。自顶向下定义与子类型算法实际展开递归类型的方式一致；自底向上定义始终先分析有限类型体的语法结构，因而更适合用结构归纳证明子表达式集合有限。后面的命题21.9.10将进一步证明，每个自顶向下子表达式也都是自底向上子表达式。

练习 21.9.7 (★★). 用一组推导规则给出关系 $S \preceq T$ 的等价定义。

[查看原书附录 A 中的 21.9.7 解答](#)

引理 21.9.8. 对每个 T ，集合 $\{S \mid S \preceq T\}$ 都有限。

证明. 对 T 作结构归纳。定义直接给出：

- 若 $T = \text{Top}$ 或 $T = X$ ，子表达式集合为 $\{T\}$ ；
- 若 $T = T_1 \times T_2$ 或 $T = T_1 \rightarrow T_2$ ，子表达式集合等于 $\{T\}$ 与两个分量子表达式集合之并；
- 若 $T = \mu X.T'$ ，子表达式集合为 $\{T\} \cup \{[X \mapsto T]S \mid S \preceq T'\}$ 。

归纳假设保证每个右侧集合有限。 □

引理 21.9.9. 若 $S \preceq [X \mapsto Q]T$ ，则或者 $S \preceq Q$ ，或者存在 $S' \preceq T$ 使 $S = [X \mapsto Q]S'$ 。

译者注：直观地说，替换不会凭空产生第三类子表达式。对 $[X \mapsto Q]T$ 而言，其子表达式要么来自代入的 Q ，要么由 T 原有的子表达式经过同一替换得到。

证明. 对 T 作结构归纳。

情形： $T = \text{Top}$ 。若以 Top 为右侧类型，自底向上子表达式关系只能由自反分支得到，所以 $S = \text{Top}$ 。取 $S' = \text{Top}$ ，便有 $S' \preceq T$ 且 $S = [X \mapsto Q]S'$ 。

情形： $T = Y$ ，其中 Y 是类型变量。这里的 X 是替换指定的变量， Y 则是当前考察的类型变量。若 $Y = X$ ，则 $[X \mapsto Q]T = Q$ ，而前提正是 $S \preceq Q$ ，结论的第一种情况成立。若 $Y \neq X$ ，则 $[X \mapsto Q]T = Y$ 。以变量 Y 为右侧类型时，仍只能使用自反分支，所以 $S = Y$ 。取 $S' = Y$ ，便得到结论的第二种情况。

情形： $T = T_1 \times T_2$ 。替换后的类型为

$$[X \mapsto Q]T = [X \mapsto Q]T_1 \times [X \mapsto Q]T_2$$

按照自底向上子表达式关系的定义， S 有三种来源。若 S 就是上式中的整个积类型，取 $S' = T$ 即可。若 $S \preceq [X \mapsto Q]T_1$ ，对 T_1 应用归纳假设：要么 $S \preceq Q$ ，结论的第一种情况成立；要么存在 $S' \preceq T_1$ ，使得 $S = [X \mapsto Q]S'$ 。由积类型的生成分支， $S' \preceq T_1$ 又推出 $S' \preceq T_1 \times T_2 = T$ ，因而得到结论的第二种情况。 S 来自第二个分量时，论证相同。

情形： $T = T_1 \rightarrow T_2$ 。箭头类型同样只有三种可能： S 是替换后的整个箭头类型，或者来自其参数类型，或者来自其结果类型。分别取 $S' = T$ ，或对相应分量应用归纳假设，再由箭头类型的生成分支把该分量的自底向上子表达式提升为 T 的自底向上子表达式，即得结论。

情形： $T = \mu Y.T'$ 。按照受约束变量的命名约定，可假定 $Y \neq X$ 且 $Y \notin \text{FV}(Q)$ ，于是

$$[X \mapsto Q]T = \mu Y.[X \mapsto Q]T'$$

若 S 就是这个完整的递归类型，则取 $S' = T$ 。否则，根据递归类型的生成分支，存在 $S_1 \preceq [X \mapsto Q]T'$ ，使得

$$S = [Y \mapsto \mu Y.[X \mapsto Q]T']S_1$$

现在对 T' 应用归纳假设。

第一种可能是 $S_1 \preceq Q$ 。因为 $Y \notin \text{FV}(Q)$ ， Q 的自底向上子表达式 S_1 也不含自由的 Y 。上式中的替换因而不起作用，所以 $S = S_1 \preceq Q$ 。

第二种可能是存在 $S_2 \preceq T'$ ，使 $S_1 = [X \mapsto Q]S_2$ 。将它代入上式，并交换这两个互不捕获的替换，得到

$$S = [X \mapsto Q]([Y \mapsto \mu Y.T']S_2)$$

令

$$S' = [Y \mapsto \mu Y.T']S_2$$

由 $S_2 \preceq T'$ 和递归类型的生成分支可知 $S' \preceq \mu Y.T' = T$ ，所以这正是结论所需的第二种情况。□

命题 21.9.10. 若 $S \sqsubseteq T$ ，则 $S \preceq T$ 。

证明. 要证 $\mu\text{TD} \subseteq \mu\text{BU}$. 依归纳原理, 只需证明 μBU 对 TD 闭合. 两种生成函数除最后一项外完全相同. 最后一项中, 若 $S \preceq [X \mapsto \mu X.T]T$, 由引理21.9.9, 或者 $S \preceq \mu X.T$, 结论已得; 或者 $S = [X \mapsto \mu X.T]S'$ 且 $S' \preceq T$, 此时 BU 的最后一项同样给出 $S \preceq \mu X.T$. $\square$

命题 21.9.11. 对任意 μ -类型 S, T , 集合 $\text{reachable}_{S_\mu}(S, T)$ 有限。

证明. 令 T_d 为 S, T 的全部自顶向下子表达式, B_u 为二者的全部自底向上子表达式. 由命题21.9.5, 可达集包含于 $T_d \times T_d$; 由命题21.9.10, $T_d \times T_d \subseteq B_u \times B_u$; 由引理21.9.8, 最后这个集合有限. $\square$

21.10 题外讨论：指数时间算法

图21-4的 `subtype` 返回更新后的假设集. 若把它简化为只返回布尔值, 就得到 Amadio 和 Cardelli (1993) 的算法 `subtypeac`, 见图21-5. 二者判定同一关系, 后者却不在箭头和积的两次递归调用之间保留已经发现的类型对, 因而效率低得多: `subtype` 的递归调用次数与两个输入类型的子表达式总数平方成正比 (由引理21.9.8和命题21.9.11的证明可以看出), `subtypeac` 则可能达到指数级。

```

subtypeac(A, S, T) =
  if (S, T) ∈ A then true
  else let A0 = A ∪ {(S, T)} in
    if T = Top then true
    else if S = S1 × S2 and T = T1 × T2 then
      subtypeac(A0, S1, T1) and subtypeac(A0, S2, T2)
    else if S = S1 → S2 and T = T1 → T2 then
      subtypeac(A0, T1, S1) and subtypeac(A0, S2, T2)
    else if S = μX.S1 then
      subtypeac(A0, [X ↦ μX.S1]S1, T)
    else if T = μX.T1 then
      subtypeac(A0, S, [X ↦ μX.T1]T1)
    else false

```

图 21-5: Amadio–Cardelli 子类型算法

译者注: 两种算法的关键差别体现在积类型和箭头类型的两个递归分支. 以积类型为例, 图21-4先让第一个分支返回更新后的假设集 A_1 , 再用 A_1 检查第二个分支:

$$A_1 = \text{subtype}(A_0, S_1, T_1)$$

然后计算 $\text{subtype}(A_1, S_2, T_2)$

因此, 第一个分支发现的类型对会传给第二个分支. AC 算法则写成

$$\text{subtype}^{\text{ac}}(A_0, S_1, T_1) \text{ and } \text{subtype}^{\text{ac}}(A_0, S_2, T_2)$$

两个分支都使用原来的 A_0 ，第一个分支发现的新类型对不会传给第二个分支。后者因而可能在不同分支中反复检查相同的类型对，产生指数级的重复计算。

指数行为可由以下类型族看出：

$$\begin{aligned} S_0 &= \mu X. \mathbf{Top} \times X, & S_{n+1} &= \mu X. X \rightarrow S_n, \\ T_0 &= \mu X. \mathbf{Top} \times (\mathbf{Top} \times X), & T_{n+1} &= \mu X. X \rightarrow T_n. \end{aligned}$$

展开缩写后， S_n, T_n 的大小都与 n 成线性关系。然而检查 $S_n <: T_n$ 会生成指数规模的推导。具体的递归调用依次为

$$\begin{aligned} &\text{subtype}^{ac}(\emptyset, S_n, T_n) \\ &= \text{subtype}^{ac}(A_1, S_n \rightarrow S_{n-1}, T_n) \\ &= \text{subtype}^{ac}(A_2, S_n \rightarrow S_{n-1}, T_n \rightarrow T_{n-1}) \\ &= \text{subtype}^{ac}(A_3, T_n, S_n) \text{ and } \underline{\text{subtype}^{ac}(A_3, S_{n-1}, T_{n-1})} \\ &= \text{subtype}^{ac}(A_4, T_n \rightarrow T_{n-1}, S_n) \text{ and } \dots \\ &= \text{subtype}^{ac}(A_5, T_n \rightarrow T_{n-1}, S_n \rightarrow S_{n-1}) \text{ and } \dots \\ &= \text{subtype}^{ac}(A_6, S_n, T_n) \text{ and } \underline{\text{subtype}^{ac}(A_6, T_{n-1}, S_{n-1})} \text{ and } \dots \end{aligned}$$

其中每一步使用的假设集为

$$\begin{aligned} A_1 &= \{(S_n, T_n)\}, \\ A_2 &= A_1 \cup \{(S_n \rightarrow S_{n-1}, T_n)\}, \\ A_3 &= A_2 \cup \{(S_n \rightarrow S_{n-1}, T_n \rightarrow T_{n-1})\}, \\ A_4 &= A_3 \cup \{(T_n, S_n)\}, \\ A_5 &= A_4 \cup \{(T_n \rightarrow T_{n-1}, S_n)\}, \\ A_6 &= A_5 \cup \{(T_n \rightarrow T_{n-1}, S_n \rightarrow S_{n-1})\}. \end{aligned}$$

初始调用由此产生了上面画线的两个递归调用：二者都比较 S_{n-1} 与 T_{n-1} ，只是方向相反。每个调用又会各自产生两个涉及 S_{n-2}, T_{n-2} 的调用，逐层重复。因此递归调用总数与 2^n 成正比。

21.11 同构递归类型的子类型化

§20.2曾提到，有些递归类型系统采用同构递归表示，用项构造子 `fold` 和 `unfold` 显式表示递归类型的折叠和展开。在这种语言中， μ 类型构造子是“刚性的”：它在类型中的位置会影响该类型的项能够怎样使用。

为同构递归类型加入子类型化时， μ 构造子的刚性也会影响子类型关系。本章大部分内容把递归类型直观地“展开到极限，再判断子类型关系”；同构递归情形则必须直接定义涉及递归类型的子类型规则。

最常见的定义是 Amber 规则 (*Amber rule*), 因 Cardelli 的 Amber 语言 (1986) 而广为人知:

$$\frac{\Sigma, X <: Y \vdash S <: T}{\Sigma \vdash \mu X.S <: \mu Y.T} \text{ S-AMBER} \quad \frac{(X, Y) \in \Sigma}{\Sigma \vdash X <: Y} \text{ S-ASSUMPTION.}$$

第一条规则说: 要在假设集 Σ 下证明 $\mu X.S <: \mu Y.T$, 只需在新增假设 $X <: Y$ 后证明 $S <: T$ 。 Σ 是递归变量对的集合, 记录已经比较过的递归类型对; 第二条规则允许直接使用当前假设。⁵

把这两条规则加入第 16 章的算法式子类型系统, 并让其他规则把 Σ 从前提传到结论, 就会得到与图 21-5 相似的算法。区别有二: 只有当 μ 同时出现在子类型判断两侧时才“展开”; 展开后也不把递归类型替换回函数体, 只保留递归变量, 因此终止性十分明显。FJ 等名义类型系统中的子类型规则与 Amber 规则关系密切。

练习 21.11.1 (推荐, ★★). 找出递归类型 S, T , 使等递归定义能够证明 $S <: T$, Amber 规则却不能证明。

[查看原书附录 A 中的 21.11.1 解答](#)

21.12 注记

本章依据 Gapeyev、Levin 与 Pierce (2000) 的教程论文。余归纳的背景材料包括 Barwise 与 Moss 的著作 *Vicious Circles* (1996)、Gordon 关于余归纳与函数式编程的教程 (1995), 以及 Milner 与 Tofte 关于程序语言语义中余归纳的综述论文 (1991a)。单调函数和不动点的基础材料见 Aczel (1977) 和 Davey 与 Priestley (1990)。

计算机科学中的余归纳证明方法可追溯到 20 世纪 70 年代, 例如 Milner (1980) 和 Park (1981) 关于并发的的工作; 另见 Arbib 与 Manes (1975) 从范畴论角度对自动机理论中对偶性的讨论。数学家更早便熟悉归纳的对偶形式, 通用代数与范畴论中都有明确发展。Aczel (1988) 关于非良基集合的著作附有简短历史回顾。

Amadio 与 Cardelli (1993) 给出了第一个递归类型子类型算法。他们的论文定义了三种关系: 无限树之间的包含关系、检查 μ -类型子类型关系的算法, 以及以一组声明式推导规则的最小不动点定义的 μ -类型参考子类型关系。论文证明三者等价, 并把它们联系到一个以偏等价关系 (*partial equivalence relation*) 为基础的模式构造。其证明没有使用余归纳; 为了论证无限树, 作者引入了无限树的有限近似, 这一概念在多项证明中起关键作用。

Brandt 与 Henglein (1997) 揭示了 Amadio–Cardelli 系统的余归纳本质, 并给出一种新的归纳公理化; 该公理化相对于原系统既可靠又完备。其中的 Arrow/Fix 规则集中体现了系统的余归纳性质。论文还提出一套一般方法, 把天然以余归纳定义的关系转写为归纳公理系统, 并详细证明了其子类型算法的终止性。§21.9 紧随这一证明; 该算法的复杂度为 $O(n^2)$ 。

Kozen、Palsberg 与 Schwartzbach (1993) 观察到, 正则递归类型可以对应为带标签状态的自动机。他们构造两个自动机的积, 得到一个普通的字自动机: 它接受某个字, 当且仅当原

⁵与图 26-1 中的 S-All 等同时涉及两侧绑定子的规则不同, 应用 Amber 规则前必须把受约束变量 X, Y 改名为彼此不同的名称。

来两个自动机所对应的类型不满足子类型关系。对子类型问题作判定，因而可归结为一次线性时间的空语言检查。再加上二次时间的积自动机构造和线性时间的类型到自动机转换，整个算法的复杂度为二次时间。

Hosoya、Vouillon 与 Pierce (2001) 采用相关的自动机方法，把带并类型的递归类型对应到树自动机，从而得到一项针对 XML 处理应用优化的子类型算法。

Jim 与 Palsberg (1999) 研究带子类型化和递归类型的类型重构（见[第二十二章](#)）。与本章相同，他们把无限树上的子类型关系看成余归纳关系，并把子类型检查算法解释为：从给定类型对出发，逐步构造包含它的最小模拟关系，也就是本章所说的一致集合。他们定义了类型关系的一致性和 P_1 -闭包；这两个概念分别对应本章的一致集合与可达集合。

想得足够久，你就会发现它显而易见。

If you think about it long enough, you'll see that it's obvious.

——Saul Gorn

第五部分

多态

第二十二章 类型重构

此前各章的类型检查算法都依赖显式类型标注，尤其要求 lambda 抽象注明参数类型。本章将建立一种能力更强的类型重构 (*type reconstruction*) 算法：即使项中的部分乃至全部类型标注都被省略，它仍能计算出该项的主类型 (*principal type*)。ML 和 Haskell 等语言的类型系统以相关算法为核心。

类型重构与其他语言特性的组合往往需要格外谨慎；记录和子类型化都会带来显著困难。为突出基本思想，本章只研究简单类型；§22.8 列出了进一步阅读其他组合的起点。

22.1 类型变量与替换

前面一些演算假定类型集合包含无限多个没有预先解释的基本类型 (§11.1)。与 Bool、Nat 不同，这些类型没有引入形式和消去形式；可以把它们看成确切身份并不重要的类型占位符。本章会问：“若把项 t 中的占位符 X 换成具体类型 Bool，得到的项能否赋型？”因此，我们把这些基本类型当作可以由其他类型实例化的类型变量 (*type variable*)。

在技术上，把类型替换分成两步比较方便：先用类型替换 (*type substitution*) σ 描述从类型变量到类型的有限映射，再把它作用于类型 T ，得到实例 σT 。¹例如，若 $\sigma = [X \mapsto \text{Bool}]$ ，则 $\sigma(X \rightarrow X) = \text{Bool} \rightarrow \text{Bool}$ 。

定义 22.1.1. 类型替换（上下文明确时也简称替换）是从类型变量到类型的有限映射。记 $[X \mapsto T, Y \mapsto U]$ 为把 X 映射到 T 、把 Y 映射到 U 的替换； $\text{dom}(\sigma)$ 是等式左侧出现的变量集合， $\text{range}(\sigma)$ 是右侧出现的类型集合。

同一个变量可以同时出现在替换的定义域和值域中；与项替换一样，此时各项映射同时生效。例如，替换

$$[X \mapsto \text{Bool}, Y \mapsto X \rightarrow X]$$

把 Y 映射到 $X \rightarrow X$ ，而非 $\text{Bool} \rightarrow \text{Bool}$ 。替换作用于类型的方式很直接，具体定义如下：

$$\begin{aligned}\sigma X &= \begin{cases} T & \text{若 } X \mapsto T \in \sigma, \\ X & \text{若 } X \notin \text{dom}(\sigma), \end{cases} \\ \sigma \text{Nat} &= \text{Nat}, \\ \sigma \text{Bool} &= \text{Bool}, \\ \sigma(T_1 \rightarrow T_2) &= \sigma T_1 \rightarrow \sigma T_2.\end{aligned}$$

¹本章研究的系统是带布尔值 (图8-1)、自然数 (图8-2) 和无限多个基本类型 (图11-1) 的简单类型 lambda 演算 (图9-1)。原书相应的 OCaml 实现为 `recon` 和 `fullrecon`。

这里不必考虑变量捕获，因为本章类型表达式中的变量都是自由的；还没有任何类型构造会在内部声明并约束一个类型变量。第二十三章才会引入这类构造。

替换逐点作用于上下文：

$$\sigma(x_1 : T_1, \dots, x_n : T_n) = (x_1 : \sigma T_1, \dots, x_n : \sigma T_n)$$

它作用于项时，则替换项中全部类型标注。

若 σ 与 γ 都是替换，复合替换写作 $\sigma \circ \gamma$ ，并定义为

$$(\sigma \circ \gamma)(X) = \begin{cases} \sigma(\gamma X) & X \in \text{dom}(\gamma), \\ \sigma X & X \notin \text{dom}(\gamma) \end{cases}$$

由此可得，对任意类型 S ，复合替换的作用等同于先应用 γ ，再应用 σ ：

$$(\sigma \circ \gamma)S = \sigma(\gamma S)$$

类型替换有一项关键性质：它保持赋型判断的有效性。也就是说，若一个含有类型变量的项能够赋型，那么它的每个替换实例也都能够赋型。

定理 22.1.2 (类型替换保持赋型). 若 σ 是任意类型替换且 $\Gamma \vdash t : T$ ，则 $\sigma\Gamma \vdash \sigma t : \sigma T$ 。

证明. 对给定的类型推导作直接归纳。 □

22.2 理解类型变量的两种方式

设项 t 和上下文 Γ 可能含有类型变量。我们可以提出两个很不相同的问题：

1. t 的所有替换实例是否都能赋型？也就是，对每个 σ ，是否都存在某个 T ，使 $\sigma\Gamma \vdash \sigma t : T$ ？
2. t 的某个替换实例能否赋型？也就是，能否找到 σ 和 T ，使 $\sigma\Gamma \vdash \sigma t : T$ ？

采用第一种理解时，类型检查过程中要始终把类型变量保持为抽象对象，从而保证以后无论代入何种具体类型，项的行为都仍然正确。例如

$$\lambda f : X \rightarrow X. \lambda a : X. f(f a)$$

具有类型 $(X \rightarrow X) \rightarrow X \rightarrow X$ ；把 X 换成任意具体类型 T ，得到

$$\lambda f : T \rightarrow T. \lambda a : T. f(f a)$$

这个实例仍然良类型。这引出了参数多态 (*parametric polymorphism*)：类型变量表明同一个项可以用于多种具体类型的上下文。本章 §22.7 会先讨论一种受限形式，第二十三章再作完整研究。

采用第二种理解时，原项本身甚至可能无法赋型；我们的目标是选择类型变量的值，使某个实例成为良类型项。例如

$$\lambda f : Y. \lambda a : X. f(f a)$$

按普通赋型规则，它目前不能赋型： f 的标注 Y 还不是箭头类型，而应用 $f a$ 要求 f 具有函数类型。但令 $Y = \text{Nat} \rightarrow \text{Nat}$ 、 $X = \text{Nat}$ ，即可得到

$$\lambda f : \text{Nat} \rightarrow \text{Nat}. \lambda a : \text{Nat}. f(f a)$$

它具有类型 $(\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat}$ 。也可以只令 $Y = X \rightarrow X$ ，得到

$$\lambda f : X \rightarrow X. \lambda a : X. f(f a)$$

这个实例虽仍含类型变量，却已经可以赋型。事实上，它是原项的最一般实例：在所有能使原项赋型的选择中，它对类型变量的取值施加的限制最少。

寻找使项可赋型的实例，就是类型重构（也常称类型推断（*type inference*））的出发点。像 ML 那样，语言甚至可以允许省略所有 lambda 参数的类型；解析器先给每个省略处放入互不相同的类型变量，随后由重构算法求出使整个项通过类型检查的最一般实例。遇到 ML 的 let 多态时，这一过程还要稍加修改；§22.6和§22.7会回到这个问题。

为了形式化类型重构，还需要一种简洁记法，用来描述：应当怎样在项及其上下文中用类型替换类型变量，才能得到有效的赋型判断。²

定义 22.2.1. 给定上下文 Γ 和项 t ，若 $\sigma\Gamma \vdash \sigma t : T$ ，则称 (σ, T) 是 (Γ, t) 的一个解（*solution*）。

例 22.2.2. 令 $\Gamma = f : X, a : Y$ 且 $t = f a$ 。以下各对都是 (Γ, t) 的解：

$$\begin{array}{ll} ([X \mapsto Y \rightarrow \text{Nat}], \text{Nat}) & ([X \mapsto Y \rightarrow Z], Z) \\ ([X \mapsto Y \rightarrow Z, Z \mapsto \text{Nat}], Z) & ([X \mapsto Y \rightarrow \text{Nat} \rightarrow \text{Nat}], \text{Nat} \rightarrow \text{Nat}) \\ ([X \mapsto \text{Nat} \rightarrow \text{Nat}, Y \mapsto \text{Nat}], \text{Nat}) & \end{array}$$

练习 22.2.3 ($\star, \Rightarrow$)。在空上下文中，为下列项找出三个不同的解：

$$\lambda x : X. \lambda y : Y. \lambda z : Z. (x z)(y z)$$

[查看练习 22.2.3 的译者参考解答](#)

22.3 基于约束的赋型

给定项 t 和上下文 Γ ，下面的算法会生成一组类型表达式之间的等式；任何解都必须满足这些约束。它与普通类型检查器的思路相同，区别在于先记录约束，稍后再集中求解。若应用 $t_1 t_2$ 的两个子项分别得到类型 T_1, T_2 ，算法不立即检查 T_1 是否为 $T_2 \rightarrow T_{12}$ ，而是选择新鲜类型变量 X ，记录 $T_1 = T_2 \rightarrow X$ ，并把 X 作为应用的结果类型。

²还有其他建立基础定义的方式。Kirchner 与 Jouannaud (1990) 提出的存在合一项（*existential unificand*），可以取代图22-1约束生成规则中一条条新鲜性条件。Rémy (1992a, 1992b 的长版；1998，第 5 章) 的另一种改进是把赋型判断本身当作合一项：从三元组 (Γ, t, T) 出发，允许三个分量都含类型变量，再寻找使 $\sigma\Gamma \vdash \sigma t : \sigma T$ 成立的替换 σ ；换言之，寻找能够合一这个示意性赋型判断 $\Gamma \vdash t : T$ 的替换。

定义 22.3.1. 约束集 (*constraint set*) 是类型等式组成的有限集合 $C = \{S_i = T_i \mid i \in 1..n\}$ 。若 $\sigma S = \sigma T$, 则称 σ 合一 (*unify*) 等式 $S = T$ 。若它合一 C 中的每个等式, 则称 σ 合一或满足 C 。

定义 22.3.2. 约束赋型关系写作 $\Gamma \vdash t : T \mid_X C$, 由图22-1的规则定义。它可以读作: “只要约束 C 得到满足, 项 t 在假设 Γ 下就具有类型 T 。” 下标 X 记录这份推导中引入的新类型变量; $\text{FV}(T)$ 表示 T 中出现的类型变量集合。

$$\begin{array}{c}
\frac{x : T \in \Gamma}{\Gamma \vdash x : T \mid_{\emptyset} \emptyset} \text{CT-VAR} \qquad \frac{\Gamma, x : T_1 \vdash t_2 : T_2 \mid_X C}{\Gamma \vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2 \mid_X C} \text{CT-ABS} \\
\\
\frac{\Gamma \vdash t_1 : T_1 \mid_{X_1} C_1 \quad \Gamma \vdash t_2 : T_2 \mid_{X_2} C_2 \quad X_1 \cap X_2 = \emptyset \quad X_1 \cap \text{FV}(T_2) = X_2 \cap \text{FV}(T_1) = \emptyset \quad X \notin X_1 \cup X_2 \cup \text{FV}(T_1) \cup \text{FV}(T_2) \cup \text{FV}(C_1) \cup \text{FV}(C_2)}{\Gamma \vdash t_1 t_2 : X \mid_{X_1 \cup X_2 \cup \{X\}} C_1 \cup C_2 \cup \{T_1 = T_2 \rightarrow X\}} \text{CT-APP} \\
\\
\frac{}{\Gamma \vdash \text{true} : \text{Bool} \mid_{\emptyset} \emptyset} \text{CT-TRUE} \qquad \frac{}{\Gamma \vdash \text{false} : \text{Bool} \mid_{\emptyset} \emptyset} \text{CT-FALSE} \\
\\
\frac{\Gamma \vdash t_1 : T_1 \mid_{X_1} C_1 \quad \Gamma \vdash t_2 : T_2 \mid_{X_2} C_2 \quad \Gamma \vdash t_3 : T_3 \mid_{X_3} C_3 \quad X_1, X_2, X_3 \text{ 两两不交}}{\Gamma \vdash \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T_2 \mid_{X_1 \cup X_2 \cup X_3} C_1 \cup C_2 \cup C_3 \cup \{T_1 = \text{Bool}, T_2 = T_3\}} \text{CT-IF} \\
\\
\frac{}{\Gamma \vdash 0 : \text{Nat} \mid_{\emptyset} \emptyset} \text{CT-ZERO} \qquad \frac{\Gamma \vdash t_1 : T_1 \mid_X C}{\Gamma \vdash \text{succ } t_1 : \text{Nat} \mid_X C \cup \{T_1 = \text{Nat}\}} \text{CT-SUCC} \\
\\
\frac{\Gamma \vdash t_1 : T_1 \mid_X C}{\Gamma \vdash \text{pred } t_1 : \text{Nat} \mid_X C \cup \{T_1 = \text{Nat}\}} \text{CT-PRED} \\
\\
\frac{\Gamma \vdash t_1 : T_1 \mid_X C}{\Gamma \vdash \text{iszero } t_1 : \text{Bool} \mid_X C \cup \{T_1 = \text{Nat}\}} \text{CT-ISZERO}
\end{array}$$

图 22-1: 约束赋型规则

这些下标和新鲜性条件只用于避免不同子推导把同一个变量名同时当作新鲜变量使用。第一次阅读规则时可以暂时忽略它们; 第二次再检查两项保证: 某条规则新选的变量不会与子推导已选变量冲突; 一条规则有多个子推导时, 各子推导所选变量彼此不交。由于变量名取之不尽, 这些条件不会阻止我们为某个项构造推导。

从结论向前提读取这些规则, 可以得到由 Γ, t 计算 T, C, X 的过程。它不会像普通简单类型检查器那样中途失败: 每个 Γ, t 都能生成一组 T, C ; 无法赋型的问题会表现为 C 无解。若不计新鲜变量名称的具体选择, 结果是唯一的。下文有时省略下标 X , 只写 $\Gamma \vdash t : T \mid C$ 。

练习 22.3.3 ($\star, \rightarrow$). 为下列结论构造约束赋型推导, 并求出适当的 S, X, C :

$$\vdash \lambda x : X. \lambda y : Y. \lambda z : Z. (x z)(y z) : S \mid_X C$$

查看练习 22.3.3 的译者参考解答

约束赋型先收集使项可赋型所需的 C ，同时给出与 C 共享变量的结果类型 S 。每个满足 C 的替换 σ 都让 σS 成为项的一种可能类型；若 C 没有满足者，项便不存在可赋型实例。例如，项 $\lambda x : X \rightarrow Y. x 0$ 生成约束 $\{X \rightarrow Y = \text{Nat} \rightarrow Z\}$ ，结果类型为 $(X \rightarrow Y) \rightarrow Z$ 。替换 $[X \mapsto \text{Nat}, Z \mapsto \text{Bool}, Y \mapsto \text{Bool}]$ 满足约束，所以 $(\text{Nat} \rightarrow \text{Bool}) \rightarrow \text{Bool}$ 是一种可能类型。

定义 22.3.4. 设 $\Gamma \vdash t : S \mid C$ 。若 σ 满足 C 且 $\sigma S = T$ ，则称 (σ, T) 是 (Γ, t, S, C) 的一个解。

寻找合一给定约束集 C 的替换，是下一节要解决的算法问题。在此之前，先检验约束赋型算法是否与原来的声明式赋型关系一致。

声明式赋型与约束赋型从两个角度刻画同一批实例：前者直接取 定义22.2.1 意义下 (Γ, t) 的全部解；后者先生成 S, C ，再取 (Γ, t, S, C) 的全部解。下面分别证明后者不会产生错误解，并且不会漏掉前者的解。

定理 22.3.5 (约束赋型的可靠性). 若 $\Gamma \vdash t : S \mid C$ ，且 (σ, T) 是 (Γ, t, S, C) 的解，则它也是 (Γ, t) 的解。

在这一方向的证明中，新鲜变量集合 X 不起关键作用，可以省略。

证明. 对约束赋型推导作归纳，并按最后使用的规则分类。

CT-Var: 此时 $t = x, x : S \in \Gamma, C = \emptyset$ 。由解的定义， $\sigma S = T$ ；T-Var 立即给出 $\sigma \Gamma \vdash x : T$ 。

CT-Abs: 此时 $t = \lambda x : T_1. t_2, S = T_1 \rightarrow S_2$ ，且 $\Gamma, x : T_1 \vdash t_2 : S_2 \mid C$ 。已知 σ 满足 C ，且 $T = \sigma S = \sigma T_1 \rightarrow \sigma S_2$ 。因此 $(\sigma, \sigma S_2)$ 是 $((\Gamma, x : T_1), t_2, S_2, C)$ 的解。归纳假设表明它也是 $((\Gamma, x : T_1), t_2)$ 的解，即 $\sigma \Gamma, x : \sigma T_1 \vdash \sigma t_2 : \sigma S_2$ 。再用 T-Abs 得到

$$\sigma \Gamma \vdash \lambda x : \sigma T_1. \sigma t_2 : \sigma T_1 \rightarrow \sigma S_2 = T$$

CT-App: 设两个子项生成 S_1, C_1 与 S_2, C_2 ，结果变量为 X 。因为 σ 满足合并后的约束，它分别满足 C_1, C_2 ，并有 $\sigma S_1 = \sigma S_2 \rightarrow \sigma X$ 。所以 $(\sigma, \sigma S_1)$ 与 $(\sigma, \sigma S_2)$ 分别是两个子问题的解。归纳假设给出

$$\sigma \Gamma \vdash \sigma t_1 : \sigma S_1 \quad \sigma \Gamma \vdash \sigma t_2 : \sigma S_2$$

把第一个类型改写为 $\sigma S_2 \rightarrow \sigma X$ ，再应用 T-App，得到 $\sigma \Gamma \vdash \sigma(t_1 t_2) : \sigma X = T$ 。其余情形类似。 $\square$

反方向的完备性论证更为细致，因为必须谨慎处理约束赋型规则引入的新鲜类型变量名。

定义 22.3.6. 记 $\sigma \setminus X$ 为从 σ 中删去定义域属于 X 的映射后得到的替换。

定理 22.3.7 (约束赋型的完备性). 设 $\Gamma \vdash t : S \mid C$ 。若 (σ, T) 是 (Γ, t) 的解且 $\text{dom}(\sigma) \cap X = \emptyset$ ，则存在 (Γ, t, S, C) 的解 (σ', T) ，并且 $\sigma' \setminus X = \sigma$ 。

证明. 对给定的约束赋值推导归纳。

CT-Var: 由普通赋值关系的反演引理可知 $T = \sigma S$, 故 (σ, T) 已是 $(\Gamma, x, S, \emptyset)$ 的解。

CT-Abs: 普通赋值的反演引理给出某个 T_2 , 使 $\sigma\Gamma, x : \sigma T_1 \vdash \sigma t_2 : T_2$, 且 $T = \sigma T_1 \rightarrow T_2$ 。对子项使用归纳假设, 得到延伸替换 σ' 。新鲜变量集合不含 T_1 中的变量, 所以 $\sigma' T_1 = \sigma T_1$, 于是 $\sigma'(T_1 \rightarrow S_2) = T$ 。

CT-App: 普通赋值的反演引理给出某个 T_1 , 使 $\sigma\Gamma \vdash \sigma t_1 : T_1 \rightarrow T$ 且 $\sigma\Gamma \vdash \sigma t_2 : T_1$ 。对两个子推导使用归纳假设, 得到解 $(\sigma_1, T_1 \rightarrow T)$ 和 (σ_2, T_1) , 并且 $\sigma_1 \setminus X_1 = \sigma = \sigma_2 \setminus X_2$ 。我们需要构造替换 σ' , 使它满足:

1. $\sigma' \setminus (X_1 \cup X_2 \cup \{X\}) = \sigma$;
2. $\sigma' X = T$;
3. σ' 同时满足 C_1 和 C_2 ;
4. $\sigma' S_1 = \sigma' S_2 \rightarrow \sigma' X$ 。

按下式定义 σ' :

$$\sigma' Y = \begin{cases} \sigma_1 Y & Y \in X_1, \\ \sigma_2 Y & Y \in X_2, \\ T & Y = X, \\ \sigma Y & \text{其他情形.} \end{cases}$$

由 $X_1 \cap X_2 = \emptyset$ 及 $X \notin X_1 \cup X_2$, 这一定义无歧义。第 1、2 项直接成立; 第 3 项由归纳假设和两个新鲜变量集合不相交得到。对第 4 项, 由新鲜性条件可知, $\text{FV}(S_1) \cap (X_2 \cup \{X\}) = \emptyset$, 以及 $\text{FV}(S_2) \cap (X_1 \cup \{X\}) = \emptyset$ 。因此

$$\sigma' S_1 = \sigma_1 S_1 = T_1 \rightarrow T = \sigma_2 S_2 \rightarrow T = \sigma' S_2 \rightarrow \sigma' X$$

所以 σ' 也满足 CT-App 新增的约束。其余情形类似。 $\square$

推论 22.3.8. 设 $\Gamma \vdash t : S \mid_X C$, 且 $X \cap \text{FV}(\Gamma, t) = \emptyset$ 。 (Γ, t) 有解, 当且仅当 (Γ, t, S, C) 有解。

证明. 由定理22.3.5和 定理22.3.7。 $\square$

练习 22.3.9 (推荐, ***). CT-App 只要求任意选取一个新鲜类型变量名, 并未规定具体选取哪一个。实际编译器通常调用名称生成函数完成这一步: 它每次返回的名称既不同于此前生成的所有名称, 也不同于当前上下文或待检查项中明确出现的类型变量。全局名称生成器依赖隐藏的可变状态, 不便于形式推理。可将一系列互不相同、尚未使用的变量名 F 显式传过规则, 把判断改写为 $\Gamma \vdash_F t : T \mid_{F'} C$: Γ, F, t 是输入, T, F', C 是输出; 需要新鲜类型变量时取 F 的首项, 把余下序列作为 F' 。写出规则, 并证明它与原约束生成规则在适当意义下等价。

[查看原书附录 A 中的 22.3.9 解答](#)

练习 22.3.10 (推荐, **). 用 ML 实现[练习22.3.9](#)的算法。类型与约束集可以使用下面的数据类型；还需要表示一系列无限的新鲜变量名，实现方式很多，代码中给出一种直接的递归表示：

```
type ty =
  TyBool
| TyArr of ty * ty
| TyId of string
| TyNat

type constr = (ty * ty) list

type nextuvar = NextUVar of string * uvargenerator
and uvargenerator = unit -> nextuvar

let uvargen =
  let rec f n () = NextUVar("?X_" ^ string_of_int n, f (n+1))
  in f 0
```

这里的 `uvargen` 是一个函数；以 `()` 调用后，它返回 `NextUVar(x,f)`，其中 `x` 是新鲜类型变量名，`f` 又是同类生成函数。

Rust 对照实现 (译者增补)

```
use std::collections::{BTreeMap, BTreeSet};

#[derive(Clone, Debug, PartialEq, Eq, PartialOrd, Ord)]
pub enum Type {
  Bool,
  Nat,
  Unit,
  Variable(String),
  Arrow(Box<Type>, Box<Type>),
  Reference(Box<Type>),
}

pub type Constraints = Vec<(Type, Type)>;

#[derive(Clone, Debug, Default)]
pub struct FreshVariables {
  next: usize,
  reserved: BTreeSet<String>,
}
```

```
impl FreshVariables {
    pub fn fresh(&mut self) -> Type {
        loop {
            let name = format!("?X{}", self.next);
            self.next += 1;
            if self.reserved.insert(name.clone()) {
                return Type::Variable(name);
            }
        }
    }
}
```

Rust 实现中的 `reserved` 保存已经使用的类型变量名，`fresh` 会跳过这些名称。扫描上下文和类型标注、填充该集合的通用辅助代码仍保留在完整工程中，此处不再刊载。

[查看原书附录 A 中的 22.3.10 解答](#)

练习 22.3.11 (★). 说明如何扩展约束生成算法，以处理一般递归函数定义 (§11.11)。

[查看原书附录 A 中的 22.3.11 解答](#)

22.4 合一

为了求解约束集，我们采用 Hindley (1969) 和 Milner (1978) 使用的合一方法 (Robinson, 1971)。它先判断解集是否为空；若不为空，再找出一个“最佳”解，使所有其他解都能由它直接生成。

定义 22.4.1. 两个类型替换的相等按它们的作用结果定义：若对每个类型 T 都有 $\sigma T = \sigma' T$ ，则记 $\sigma = \sigma'$ 。这里不要求两个有限映射具有完全相同的写法或定义域。

若存在替换 γ ，使 $\sigma' = \gamma \circ \sigma$ ，则称 σ 比 σ' 更不具体 (*less specific*)，也称更一般，记作 $\sigma \preceq \sigma'$ 。

定义 22.4.2. 约束集 C 的主合一子 (*principal unifier*) (也称 最一般合一子 (*most general unifier*)) 是满足 C 的替换 σ ，并且对每个满足 C 的 σ' 都有 $\sigma \preceq \sigma'$ 。

练习 22.4.3 (★). 为以下约束集写出主合一子 (若存在)：

$$\begin{array}{lll} \{X = \text{Nat}, Y = X \rightarrow X\} & \{\text{Nat} \rightarrow \text{Nat} = X \rightarrow Y\} & \{X \rightarrow Y = Y \rightarrow Z, Z = U \rightarrow W\} \\ \{\text{Nat} = \text{Nat} \rightarrow Y\} & \{Y = \text{Nat} \rightarrow Y\} & \emptyset \end{array}$$

[查看原书附录 A 中的 22.4.3 解答](#)

定义 22.4.4. 类型合一算法见图22-2。³ 第二行的“令 $\{S = T\} \cup C' = C$ ”表示从 C 中任选一个约束 $S = T$ ，其余约束组成 C' 。若 σ 是替换，则 $\sigma C = \{\sigma S = \sigma T \mid S = T \in C\}$ ，即把替换 σ 分别应用于每个约束两侧的类型。满足约束集 C 的替换称为 C 的合一子 (*unifier*)。

³这个算法并不依赖待合一的表达式恰好是类型；同一算法可以求解任意一阶表达式之间的等式约束。

$$\text{unify}(C) = \begin{cases} [] & C = \emptyset, \\ \text{unify}(C') & C = \{S = T\} \cup C', S = T, \\ \text{unify}([X \mapsto T]C') \circ [X \mapsto T] & S = X, X \notin \text{FV}(T), \\ \text{unify}([X \mapsto S]C') \circ [X \mapsto S] & T = X, X \notin \text{FV}(S), \\ \text{unify}(C' \cup \{S_1 = T_1, S_2 = T_2\}) & S = S_1 \rightarrow S_2, T = T_1 \rightarrow T_2, \\ \text{失败} & \text{其他情形} \end{cases}$$

图 22-2: 合一算法

译者注：可以把约束集 C 看作一张尚待处理的类型等式清单。算法每次取出一个等式 $S = T$ ，再按其形式化简：两侧相同便直接删除；两侧都是箭头类型，便把它拆成参数类型和结果类型的两个等式；一侧是类型变量 X 时，便暂定 X 等于另一侧的类型，把这一替换应用于其余约束，再继续求解；无法归入这些情形时，合一失败。递归调用返回后，沿途得到的替换通过复合合并起来。

例如，从

$$C = \{X = \text{Nat}, Y = X \rightarrow X\}$$

开始，先由第一个等式得到 $[X \mapsto \text{Nat}]$ ，并把它应用于剩余约束，得到 $Y = \text{Nat} \rightarrow \text{Nat}$ 。继续求解便得到 $[Y \mapsto \text{Nat} \rightarrow \text{Nat}]$ 。两项替换复合后，结果为

$$[X \mapsto \text{Nat}, Y \mapsto \text{Nat} \rightarrow \text{Nat}]$$

这就是该约束集的主合一子。

变量分支中的旁条件称为出现检查 (*occurs check*)。它禁止产生 $[X \mapsto X \rightarrow X]$ 之类的循环替换，因为有限类型表达式无法表示这种解。若语言允许递归类型，出现检查便可以去掉；第二十章和第二十一章讨论的正是这类类型。

定理 22.4.5. unify 总会终止；输入不可合一的约束集时失败，否则返回主合一子。更明确地说：

1. 对每个 C ， $\text{unify}(C)$ 都会失败或返回替换；
2. 若 $\text{unify}(C) = \sigma$ ，则 σ 合一 C ；
3. 若 δ 合一 C ，则 $\text{unify}(C) = \sigma$ ，且 $\sigma \preceq \delta$ 。

证明. 对第 1 项，把 C 的度量定义为数对 (m, n) ： m 是 C 中不同类型变量的数量， n 是 C 中所有等式左右两侧的类型表达式大小之和。每个分支要么立即结束，要么以字典序下更小的度量递归调用算法，故算法终止。

第 2 项对递归调用次数归纳。除变量分支外都直接成立；变量分支使用如下事实：若 σ 合一 $[X \mapsto T]D$ ，则 $\sigma \circ [X \mapsto T]$ 合一 $X = T \cup D$ 。

第 3 项仍对计算 $\text{unify}(C)$ 时的递归调用次数归纳。若 $C = \emptyset$ ，算法返回空替换；因为 $\delta = \delta \circ []$ ，所以 $[] \preceq \delta$ 。下面设 $C \neq \emptyset$ ，算法选取约束 $S = T$ ，并按 S, T 的形式分类。

$S = T$ ：因为 δ 合一 C ，它也合一剩余的 C' 。直接应用归纳假设。

$S = X$ 且 $X \notin \text{FV}(T)$: 由 δ 合一 $X = T$ 可知 $\delta X = \delta T$ 。因此对任意类型 U , 都有 $\delta U = \delta([X \mapsto T]U)$; 再利用 δ 合一 C' , 可得 δ 也合一 $[X \mapsto T]C'$ 。归纳假设给出

$$\text{unify}([X \mapsto T]C') = \sigma' \quad \text{且} \quad \delta = \gamma \circ \sigma'$$

算法在当前分支返回 $\sigma' \circ [X \mapsto T]$ 。要证明它比 δ 更一般, 只需验证 $\delta = \gamma \circ (\sigma' \circ [X \mapsto T])$ 。对任意类型变量 Y 分两种情形: 若 $Y \neq X$, 则

$$(\gamma \circ (\sigma' \circ [X \mapsto T]))Y = (\gamma \circ \sigma')Y = \delta Y$$

若 $Y = X$, 则

$$(\gamma \circ (\sigma' \circ [X \mapsto T]))X = (\gamma \circ \sigma')T = \delta T = \delta X$$

因此两个替换对所有变量的作用都相同。

$T = X$ 且 $X \notin \text{FV}(S)$: 对称。

$S = S_1 \rightarrow S_2$ 且 $T = T_1 \rightarrow T_2$: 一个替换合一 $\{S_1 \rightarrow S_2 = T_1 \rightarrow T_2\} \cup C'$, 当且仅当它合一 $C' \cup \{S_1 = T_1, S_2 = T_2\}$, 所以直接使用归纳假设。

若没有分支适用, 只可能是 **Nat** 与箭头类型冲突, 或 $S = X$ 但 $X \in \text{FV}(T)$ (及其对称情形)。前者不可能被任何替换合一。对后者, 若假定 $\delta X = \delta T$, 由于 X 出现在 T 中, δT 将严格大于 δX , 矛盾。所以算法只会在 C 确实不可合一时失败。□

练习 22.4.6 (推荐, ★★). 实现合一算法。

[查看原书附录 A 中的 22.4.6 解答](#)

22.5 主类型

如果某种类型变量实例化方式能让项获得类型, 就存在一种最一般的方式。现在把这一点形式化。

定义 22.5.1. (Γ, t, S, C) 的主解 (*principal solution*) 是一个解 (σ, T) , 并且对它的每个其他解 (σ', T') 都有 $\sigma \preceq \sigma'$ 。此时称 T 为 t 在 Γ 下的一个主类型。⁴

练习 22.5.2 ($\star, \rightarrow$). 在空上下文 (事实上, 任意上下文均可) 中, 求 $\lambda x : X. \lambda y : Y. \lambda z : Z. (xz)(yz)$ 的主类型。

[查看练习 22.5.2 的译者参考解答](#)

定理 22.5.3 (主类型). 若 (Γ, t, S, C) 有解, 则它有主解。图 22-2 的合一算法可以判断它是否有解; 若有, 还能计算一个主解。

证明. 由定义 22.3.4 中解的定义和定理 22.4.5 的合一性质直接得到。□

推论 22.5.4. 判断 (Γ, t) 是否有解是可判定的。

⁴主类型不要与相近的“主赋型”混淆; §22.8 会说明两者的区别。

证明. 由[推论22.3.8](#)和 [定理22.5.3](#)。 □

练习 22.5.5 (推荐, ★★★, $\rightarrow$). 把[练习22.3.10](#)的约束生成与 [练习22.4.6](#)的合一算法结合起来。从 `reconbase` 检查器出发, 构造一个计算主类型的类型检查器。典型交互如下:

```
lambda x:X. x;
==> <fun> : X -> X

lambda z:ZZ. lambda y:YY. z (y true);
==> <fun> : (?X0 -> ?X1) -> (Bool -> ?X0) -> ?X1

lambda w:W. if true then false else w false;
==> <fun> : (Bool -> Bool) -> Bool
```

?X0 这类类型变量名由检查器自动生成。

[查看练习 22.5.5 的译者参考解答](#)

练习 22.5.6 (★★★). 把上述定义扩展到记录类型时会遇到哪些困难? 可以怎样处理?

[查看原书附录 A 中的 22.5.6 解答](#)

主类型的概念还可用于构造一种更具增量性的类型重构算法。它不再先生成全部约束、再统一求解, 而是把约束生成与求解交错进行, 使重构算法在每一步都返回主类型。由于中间结果始终保持最一般性, 算法无需重新分析已经处理过的子项; 每一步只作获得可赋型性所必需的最少承诺。这种增量形式的一项重要优点, 是能够更准确地定位源程序中的类型错误。

练习 22.5.7 (★★★, $\rightarrow$). 修改[练习22.5.5](#)的实现, 使它增量执行合并并返回主类型。

[查看练习 22.5.7 的译者参考解答](#)

22.6 隐式类型标注

支持类型重构的语言通常允许程序员完全省略 `lambda` 抽象的类型标注。解析器可以在省略处自动填入新鲜类型变量; 更直接的做法是把无标注抽象加入项的语法, 并增加规则:

$$\frac{X \notin X_1 \quad \Gamma, x : X \vdash t_2 : T_2 \mid_{X_1} C}{\Gamma \vdash \lambda x. t_2 : X \rightarrow T_2 \mid_{X_1 \cup \{X\}} C} \text{CT-AbsInf}$$

把无标注抽象作为原始形式还有一项虽小却有用的区别: 复制一个无标注抽象时, `CT-AbsInf` 可以为每份副本选择不同的参数类型变量。若把它当成已经带有某个不可见类型变量的语法糖, 复制后各份表达式仍会共享同一参数类型。下一节的 `let` 多态正需要前一种行为。

22.7 let 多态

多态 (*polymorphism*) 泛指让同一段程序在不同上下文中以不同类型使用的一系列语言机制。§23.2 会更详细地区分其中几种。

上面的类型重构可以推广为 let 多态 (*let-polymorphism*), 也称 ML 风格多态或 Damas–Milner 多态。它最早出现在原始 ML 方言中 (Milner, 1978), 后来被多种成功的语言设计采用, 并成为列表、数组、树、哈希表、流和界面组件等通用程序库的基础。

考虑把第一个参数连续应用两次于第二个参数的函数 `double`。若要将它用于类型为 $\text{Nat} \rightarrow \text{Nat}$ 的函数, 可以写作:

```
let double = lambda f:Nat->Nat. lambda a:Nat. f (f a) in
double (lambda x:Nat. succ (succ x)) 2
```

也可以用布尔类型的标注定义另一版:

```
let double = lambda f:Bool->Bool. lambda a:Bool. f (f a) in
double (lambda x:Bool. x) false
```

问题在于, 同一个 `double` 定义无法同时服务于这两种类型。要在一个程序中同时使用两者, 只能把除类型标注外完全相同的定义写两遍:

```
let doubleNat = lambda f:Nat->Nat. lambda a:Nat. f (f a) in
let doubleBool = lambda f:Bool->Bool. lambda a:Bool. f (f a) in
let a = doubleNat (lambda x:Nat. succ (succ x)) 1 in
let b = doubleBool (lambda x:Bool. x) false in
...
```

即使把 `double` 中的标注改成类型变量, 也不能解决问题:

```
let double = lambda f:X->X. lambda a:X. f (f a) in
let a = double (lambda x:Nat. succ (succ x)) 1 in
let b = double (lambda x:Bool. x) false in
...
```

第一次使用产生约束 $X \rightarrow X = \text{Nat} \rightarrow \text{Nat}$, 第二次则产生 $X \rightarrow X = \text{Bool} \rightarrow \text{Bool}$, 整组约束无解。

这里的 X 本应承担两项彼此独立的职责。在计算 a 时, 它用来表示 `double` 的函数参数与另一个参数必须具有匹配的定义域和值域; 在计算 b 时, 它又用来表示同样的局部关系。把两个职责都压在同一个 X 上, 就额外引入了不应存在的约束: 两次使用的第二个参数必须具有同一类型。我们真正需要的是打破这条联系, 让每次使用 `double` 都得到一份独立的新类型变量。

第一步是改变 `let` 的类型规则。普通规则先给右侧 t_1 求类型, 再以该类型检查体 t_2 :

$$\frac{\Gamma \vdash t_1 : T_1 \quad \Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : T_2} \text{ T-LET}$$

新规则则先把 t_1 代入 t_2 ，再检查展开后的表达式：

$$\frac{\Gamma \vdash [x \mapsto t_1]t_2 : T_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : T_2} \text{ T-LET POLY}$$

约束版本同样写成

$$\frac{\Gamma \vdash [x \mapsto t_1]t_2 : T_2 \mid_X C}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : T_2 \mid_X C} \text{ CT-LET POLY}$$

也就是说，赋型前先执行一次下列 let 求值步骤：

$$\frac{}{\text{let } x = v_1 \text{ in } t_2 \longrightarrow [x \mapsto v_1]t_2} \text{ E-LET V}$$

第二步是用上一节的无标注抽象定义 double：

```
let double = lambda f. lambda a. f (f a) in
let a = double (lambda x:Nat. succ (succ x)) 1 in
let b = double (lambda x:Bool. x) false in
...
```

CT-LetPoly 为两次使用复制两份定义，CT-AbsInf 再为每份副本中的抽象选择不同类型变量，普通约束求解即可完成其余工作。

这个朴素方案还不能直接投入实际使用。若 x 在 t_2 中根本没有出现，右侧 t_1 就不会被检查，例如

```
let x = <utter garbage> in 5
```

也会通过类型检查。修正方法是在规则中增加对 t_1 的检查：

$$\frac{\Gamma \vdash t_1 : T_1 \quad \Gamma \vdash [x \mapsto t_1]t_2 : T_2}{\Gamma \vdash \text{let } x = t_1 \text{ in } t_2 : T_2} \text{ T-LET POLY}$$

CT-LetPoly 也需增加生成 t_1 的类型和约束的相应前提。

另一个问题是，这条规则先计算 $[x \mapsto t_1]t_2$ ，因而会把 t_2 中每个自由出现的 x 都替换成一份 t_1 。若 x 出现很多次，赋型过程就会重复检查多份相同的 t_1 ，无论 t_1 是否含有隐式标注的 lambda 抽象。若 t_1 本身又包含 let 绑定，这种复制还会逐层累积，使类型检查器的工作量相对于原项大小呈指数增长。

实际实现使用一个形式上等价而更高效的过程。检查 $\text{let } x = t_1 \text{ in } t_2$ 时，需要用到类型方案 (type scheme) $\forall X_1 \dots X_n. T$ ： $\forall$ 绑定其中列出的类型变量；每次使用该方案时，都以一组全新的类型变量替换它们，再使用所得类型。具体过程如下：

1. 为 t_1 生成类型 S_1 和约束 C_1 ；
2. 求 C_1 的最一般解 σ ，把它作用于 S_1 和 Γ ，得到 t_1 的主类型 T_1 ；
3. 泛化 T_1 中仍然自由、但不在 Γ 中自由的变量。若它们是 $X_1, \dots, X_n$ ，则 $\forall X_1 \dots X_n. T_1$ 是 t_1 的主类型方案；

4. 在上下文中把 x 绑定到这个类型方案，继续检查 t_2 ；
5. 每遇到一次 x ，都生成新鲜变量 $Y_1, \dots, Y_n$ ，以 $[X_1 \mapsto Y_1, \dots, X_n \mapsto Y_n]T_1$ 实例化该方案。⁵

不能泛化上下文中也出现的变量，因为它们表达 t_1 与周围环境之间的真实约束。例如在 $\lambda f : X \rightarrow X. \lambda x : X. \text{let } g = f \text{ in } g x$ 中， g 的 X 不能泛化。否则就会把与周围绑定相连的 X 误当成每次可以独立实例化的变量，从而让以下错误程序通过检查：

```
(lambda f:X->X. lambda x:X. let g = f in g 0)
  (lambda x:Bool. if x then true else false)
    true
```

该算法实践中接近线性，但 Kfoury、Tiuryn 与 Urzyczyn (1990) 以及 Mairson (1990) 分别证明其最坏情况仍为指数级。他们构造的表达式通过嵌套 let，使推断出的类型比表达式本身大得多。下面这个由 Jacques Garrigue 给出的 OCaml 程序类型正确，却会使类型检查花费很长时间：

```
let l0 = ([], [], [], []) in
let l1 = (l0, l0, l0, l0) in
let l2 = (l1, l1, l1, l1) in
let l3 = (l2, l2, l2, l2) in
let l4 = (l3, l3, l3, l3) in
let l5 = (l4, l4, l4, l4) in
let l6 = (l5, l5, l5, l5) in
let l7 = (l6, l6, l6, l6) in
let l8 = (l7, l7, l7, l7) in
let l9 = (l8, l8, l8, l8) in
ignore (l9)
```

从 l0、l1 逐步输入 OCaml 顶层，可以观察到类型的快速增长。可参见 Kfoury、Tiuryn 与 Urzyczyn (1994) 的进一步讨论。

译者注：每个 `[]` 都可以具有独立的元素类型，因此 l0 的类型含有四个列表分量，其元素类型变量彼此独立。定义 l1 时，右侧四次出现 l0；每次出现都会独立实例化 l0 的整个类型方案，所以 l1 的类型含有四份 l0 的类型结构，共有 $4 \times 4 = 16$ 个列表分量。一般地，定义 l_{k+1} 时，其类型包含四份 l_k 的类型结构。若 N_k 表示 l_k 类型中的列表分量数，则 $N_0 = 4$ ， $N_{k+1} = 4N_k$ ，因而 $N_k = 4^{k+1}$ 。源程序每层只增加一行，推断出的类型大小却按指数增长。末尾使用 `ignore` 可避免打印这个巨大类型，因此观察到的耗时主要来自类型检查本身。

最后还要处理 let 多态与可变引用等副作用的相互作用。考虑

⁵引入泛化和实例化后，“显式标注了类型变量的 lambda 抽象”与“由约束生成算法为之创建类型变量的无标注抽象”之间的区别不再重要：两种情形下，let 右侧都会先得到含变量的类型，该变量在加入上下文前被泛化，以后每次实例化时又换成新鲜变量。

```
let r = ref (lambda x. x) in
(r := (lambda x:Nat. succ x);
 (!r) true)
```

上述算法会先把右侧赋予主类型 $\text{Ref}(X \rightarrow X)$ ，再把 X 泛化。赋值时可将它实例化为 $\text{Ref}(\text{Nat} \rightarrow \text{Nat})$ ，读取时又实例化为 $\text{Ref}(\text{Bool} \rightarrow \text{Bool})$ 。程序因此通过检查，运行时却会把 `succ` 用于 `true`。

问题在于，新引入的 `let` 赋型规则与按值求值规则并不一致。赋型规则立即计算 $[x \mapsto t_1]t_2$ ，所以在上例中，两次出现的 r 会被分别替换成两份 `ref (lambda x. x)`，类型检查器也就能以不同类型分析它们。实际求值时却必须先计算 `let` 右侧，分配出一个引用单元，再把这个引用值代入体中；因此两次 r 访问的其实是同一个引用单元。这就造成了程序能够通过类型检查、运行时却出错的问题。

可以从求值规则或赋型规则两方面消除这种不一致。先看修改求值规则的办法：不再先计算 `let` 右侧，而是立即把尚未求值的 t_1 代入 t_2 。这样，复制出来的每一份 `ref (lambda x. x)` 都会在自己的使用位置单独求值，因而分配不同的引用单元。此时 `let` 的求值规则改为⁶

$$\frac{}{\text{let } x = t_1 \text{ in } t_2 \longrightarrow [x \mapsto t_1]t_2} \text{ E-LET}$$

上述不安全程序的第一步会变成

```
(ref (lambda x. x)) := (lambda x:Nat. succ x);
(! (ref (lambda x. x))) true
```

这一版确实安全：第一行分配一个初值为恒等函数的引用，再写入自然数后继函数；第二行另外分配一个引用，从中取出恒等函数并用于 `true`。但这也显示了该做法的代价：它不再符合标准按值求值对副作用顺序的理解。非按值求值的命令式语言并非从未存在（Augustsson, 1984），但由于运行时副作用顺序难以理解和控制，它们始终没有广泛流行。

更好的做法是让类型规则服从求值规则，施加载限制 (*value restriction*)：仅当 `let` 右侧在句法上已经是值时，才泛化其自由类型变量。上述不安全程序的右侧是分配操作，不是值，所以把 r 加入上下文时，它获得的是单态类型 $\text{Ref}(X \rightarrow X)$ ，而非泛化后的类型方案 $\forall X. \text{Ref}(X \rightarrow X)$ 。赋值迫使 X 成为 Nat 后，下一次以 Bool 使用便会被拒绝。值限制会损失一部分表达能力：let 右侧若要执行实际计算，就不能同时获得多态类型方案。令人意外的是，这项限制几乎不影响实际程序。Wright (1995) 分析了大量以 1990 年版 Standard ML (Milner、Tofte 与 Harper, 1990) 写成的程序。那一版语言使用基于弱类型变量的更宽松 `let` 赋型规则，但除了极少数例外，实际程序的 `let` 右侧本来就是句法值。这项观察基本结束了当时的争论；现代采用 ML 风格 `let` 多态的主要语言都使用值限制。

练习 22.7.1 ($\star\star, \rightarrow$)。实现本节概述的算法。

⁶严格说来，这里讨论的语言含有引用，所以规则应像第十三章一样显式携带存储：

$$\text{let } x = t_1 \text{ in } t_2 \mid \mu \longrightarrow [x \mapsto t_1]t_2 \mid \mu \quad \text{E-LET}$$

[查看练习 22.7.1 的译者参考解答](#)

22.8 注记

lambda 演算主类型的思想至少可追溯到 Curry 在 20 世纪 50 年代的工作 (Curry 与 Feys, 1958)。Hindley (1969) 给出了基于这些思想的主类型算法; Morris (1968) 和 Milner (1978) 也独立发现了类似算法。在命题逻辑中, 相关思想出现得更早: 可能可以追溯到 20 世纪 20 年代的 Tarski, 至少可以确定在 20 世纪 50 年代的 Meredith 堂兄弟工作中已经出现 (Lemmon、Meredith、Meredith、Prior 与 Thomas, 1957); David Meredith 在 1957 年首次用计算机实现了这些思想。关于主类型的更多历史说明可参见 Hindley (1997)。

合一 (Robinson, 1971) 是计算机科学许多领域的基础工具。Baader 与 Nipkow (1998)、Baader 与 Siekmann (1994) 以及 Lassez 与 Plotkin (1991) 都给出了系统介绍。

ML 风格 let 多态由 Milner (1978) 首先描述。经典的 Damas–Milner 算法 W (*Algorithm W*) 见 Damas 与 Milner (1982; 又见 Lee 与 Yi, 1998)。算法 W 专门处理“纯类型重构”, 即给完全无类型的 lambda 项赋主类型; 本章允许显式标注存在, 也允许标注含变量, 因此把类型检查与重构混合起来。这使技术叙述更繁琐, 尤其是定理 22.3.7 的完备性证明: 需要把程序员写出的类型变量与约束生成规则新引入的变量仔细分开。不过, 这种组织更符合本书其余章节的风格。

Cardelli (1987) 的经典论文系统阐述了实现类型重构时需要处理的若干问题。Appel (1998)、Aho 等 (1986) 与 Reade (1989) 也介绍了类型重构算法。名为 mini-ML 的核心系统有一种格外优雅的叙述 (Clément、Despeyroux、Despeyroux 与 Kahn, 1986), 理论讨论常以它为基础。Tiuryn (1990) 综述了一系列类型重构问题。

与主类型相近的另一个概念是主赋型 (*principal typing*)。计算主类型时, Γ, t 是输入, T 是输出; 计算主赋型时, 只有 t 是输入, Γ, T 都是输出, 即同时求出自由变量类型所需的最小假设。主赋型有助于独立编译: 它能给出模块对外部名称的最小类型要求, 从而尽量缩小程序修改后的重编译范围。它还可用于增量类型推断和精确定位类型错误。许多语言 (尤其 ML) 有主类型而没有主赋型。可参见 Jim (1996)。

ML 风格多态以难得的简洁性提供了很强的能力, 但与其他高级类型特性组合时常常十分微妙。记录类型重构的一条成功路线是引入行变量 (*row variable*): 它表示整行字段标签及其类型, 再用等式合一求解含行变量的约束 (Wand, 1987, 1988, 1989b; Rémy, 1989, 1990, 1992a, 1992b, 1998)。见练习 22.5.6。Garrigue (1994) 等人又将相关方法用于变体类型。这些技术还扩展到了一般的类型类 (Kaes, 1988; Wadler 与 Blott, 1989)、约束类型 (Odersky、Sulzmann 与 Wehr, 1999) 和限定类型 (Jones, 1994a, 1994b)。限定类型构成了 Haskell 类型类系统的基础 (Hall 等, 1996; Hudak 等, 1992; Thompson, 1999); 类似思路也出现在 Mercury (Somogyi、Henderson 与 Conway, 1996) 和 Clean (Plasmeijer, 1998) 中。

正如第二十三章将要讨论的, 能力更强的非直谓多态其类型重构不可判定 (Wells, 1994), 甚至该系统的若干部分类型重构形式也不可判定。§23.6 还会进一步讨论这些结果; §23.8 则会介绍秩 2 多态 (*rank-2 polymorphism*), 并说明如何把 ML 风格类型重构与这种更强的多态形式结合。秩 2 多态允许函数以多态函数为参数, 但限制全称量词在函数类型中的嵌套位置。

将子类型化与 ML 风格类型重构结合,已有一些很有希望的初步结果 (Aiken 与 Wimmers, 1993; Eifrig、Smith 与 Trifonov, 1995; Jagannathan 与 Wright, 1995; Trifonov 与 Smith, 1996; Odersky、Sulzmann 与 Wehr, 1999; Flanagan 与 Felleisen, 1997; Pottier, 1997), 但实用检查器尚未广泛应用。

把 ML 风格类型重构扩展到递归类型 (第二十章) 并不会带来太大困难 (Huet, 1975, 1976)。与本章算法最显著的区别在合一的定义: 要删去出现检查, 因为原本的检查正是用来保证结果替换无环的。删去后要保证终止, 还需改造合一算法的表示方式, 让它保持共享, 例如对可能有环的指针结构作破坏性更新。这类表示在高性能实现中很常见。

类型重构与递归定义的项结合时, 却会出现另一个棘手问题: 多态递归 (*polymorphic recursion*)。ML 通常采用一条简单、易于处理的递归函数赋型规则: 在函数自身的定义体内, 只能以单态方式使用该函数, 即所有递归调用的参数和结果类型必须相同; 在程序其余位置却可以多态使用, 即以不同类型的参数和结果实例化。Mycroft (1984) 与 Meertens (1983) 提出了一条多态的递归定义规则, 允许函数在自身定义的体中以不同类型作递归调用。这项扩展常称 Milner–Mycroft 演算。Henglein (1993) 与 Kfoury、Tiuryn、Urzyczyn (1993a) 独立证明了它的重构问题不可判定; 两个证明都依赖无限制半合一问题的不可判定性 (Kfoury、Tiuryn 与 Urzyczyn, 1993b)。

第二十三章 全称类型

上一章考察了 ML 中简单形式的 let 多态。本章转向一种更一般的参数多态系统：System F (*System F*)，也称多态 lambda 演算 (*polymorphic lambda-calculus*) 或二阶 lambda 演算。本章主要研究纯 System F (图23-1)；示例会自由使用此前介绍的扩展。¹

23.1 动机

正如§22.7所述，在简单类型 lambda 演算中，可以写出无穷多个“加倍”函数：

```
doubleNat = λf : Nat → Nat. λx : Nat. f(f x),  
doubleRcd = λf : {l : Bool} → {l : Bool}. λx : {l : Bool}. f(f x),  
doubleFun = λf : (Nat → Nat) → (Nat → Nat). λx : Nat → Nat. f(f x)
```

三者接受不同类型的参数，行为却完全相同；除类型标注外，程序文本也相同。若同一程序需要在不同类型上使用这一操作，简单类型系统迫使我们复制定义。

抽象原则：程序中每项重要功能都应只在源码的一处实现。若多段代码执行相似功能，通常应抽取其中变化的部分，把它们合并为一个定义。

这里变化的正是类型。因此，我们需要从项中抽象出类型，并在使用该项时再提供具体类型。

23.2 多态的几种形式

允许同一段代码以多种类型使用的系统统称为多态系统 (*polymorphic system*)；其中 *poly* 意为“多”，*morph* 意为“形态”。现代语言中常见的多态形式包括以下几种。下面的分类源自 Strachey (1967) 以及 Cardelli 与 Wegner (1985)。

参数多态 (parametric polymorphism) 以类型变量代替具体类型编写通用代码，并在需要时以具体类型实例化。参数多态定义具有一致的行为：不同类型实例执行同一种算法。

特设多态 (ad-hoc polymorphism) 同一个名称在不同类型下可以采用不同实现。最常见的例子是重载 (*overloading*)：编译器或运行时系统根据参数类型选择具体实现。多方法分派、受限的运行时类型分析等机制也属于这一大类。

¹原书相应的 OCaml 实现为 `fullpoly`；涉及序对和列表的部分示例需要 `fullomega`。

子类型多态 (subtype polymorphism) 借助 T-Sub, 同一个项可以获得多个类型; 把它看作某个超类型时, 会有选择地忘掉关于该项行为的部分信息。

本章研究参数多态中能力最强的一种形式: 非直谓多态, 也称一等多态。实践中更常见的是 ML 风格的 let 多态: 它把多态限制在 let 绑定处, 不允许函数接收多态值作为参数, 换来较自然的自动类型重构 (第二十二章)。一等参数多态也日益受到编程语言的重视, 并构成 ML 等语言中强大模块系统的技术基础 (Harper 与 Stone, 2000)。

重载还可以推广为多方法分派, 这是 CLOS (Bobrow 等, 1988; Kiczales 等, 1991) 和 Cecil (Chambers, 1992; Chambers 与 Leavens, 1994) 等语言的基础机制。Castagna、Ghelli 与 Longo (1995; 又见 Castagna, 1997) 的 lambda- $\&$ 演算对这项机制作了形式化。

能力更强的内涵多态 (*intensional polymorphism*) (Harper 与 Morrisett, 1995; Crary、Weirich 与 Morrisett, 1998) 允许在运行时对类型作受限计算。它支撑了多态语言的若干高级实现技术, 包括无标签垃圾回收、不装箱的函数参数、多态编组, 以及更节省空间的扁平数据结构。

还可以从 `typecase` 原语构造更强的特设多态: 该原语允许在运行时对类型信息作任意模式匹配 (Abadi、Cardelli、Pierce 与 Rémy, 1995; Abadi、Cardelli、Pierce 与 Plotkin, 1991b; Henglein, 1994; Leroy 与 Mauny, 1991; Thatte, 1990)。Java 的 `instanceof` 可看作 `typecase` 的受限形式。

上述类别并不互斥, 一门语言可以同时具有多种多态。例如, Standard ML 兼有参数多态和简单的内建算术运算重载, 但没有子类型; 本书写作时的 Java 具有子类型、重载和受限的特设多态 (`instanceof`), 却还没有参数多态。当时已有若干为 Java 加入参数多态的方案, 其中最著名的是 GJ (Bracha、Odersky、Stoutamire 与 Wadler, 1998)。

不同社群单说“多态”时, 含义也可能不同。函数式编程社群通常用它指参数多态; 面向对象社群则通常用它指子类型多态, 而用“泛型”指参数多态。

23.3 System F

System F 最早由 Jean-Yves Girard (1972) 在证明论研究中提出; John Reynolds (1974) 随后独立提出了能力基本相同的多态 lambda 演算。借助 Curry-Howard 对应, 它对应于二阶直觉主义逻辑: 量化对象不再限于个体 (项), 也可以包括谓词 (类型)。System F 长期作为多态基础研究的核心工具, 也成为众多编程语言设计的基础。

System F 是对简单类型 lambda 演算的直接扩展。在简单类型 lambda 演算中, lambda 抽象从项中抽象出一个项参数, 应用则为这个参数提供一个值。现在我们还希望从项中抽象出类型, 并在以后填入具体类型, 因此增加两种项:

$$\lambda X. t \quad \text{和} \quad t[T]$$

前者是类型抽象 (*type abstraction*), 其参数 X 是类型变量; 后者是类型应用 (*type application*) 或实例化, 其实参 T 是类型表达式。两者在类型层面的关系, 正好对应普通 lambda 抽象和普通应用在项层面的关系。

求值过程中, 类型抽象遇到类型应用时也会形成可归约式: 把类型实参替换进抽象体, 就像普通 beta 归约把项实参替换进函数体一样:

$$(\lambda X. t_{11})[T_2] \longrightarrow [X \mapsto T_2]t_{11}$$

例如, 多态恒等函数

$$\text{id} = \lambda X. \lambda x : X. x$$

写成 $\text{id}[\text{Nat}]$ 后, 先得到 $[X \mapsto \text{Nat}](\lambda x : X. x)$, 也就是 $\lambda x : \text{Nat}. x$ 。

最后还要说明类型抽象本身具有什么类型。普通箭头类型用来刻画接受项参数的函数; 类型抽象接受的却是类型参数, 因此需要另一种“箭头类型”, 即全称类型

$$\forall X. T$$

来刻画它。恒等函数的类型是 $\forall X. X \rightarrow X$: 无论给它什么类型 T , 实例 $\text{id}[T]$ 都具有类型 $T \rightarrow T$ 。关键在于, 结果类型依赖于实际传入的类型参数: 给 id 传入 T , 就得到 $T \rightarrow T$ 型函数。全称类型正是用来表达这种依赖关系的。

System F

扩展简单类型 lambda 演算 (图 9-1)

语法

$$\begin{aligned} t &::= x \mid \lambda x : T. t \mid t t \mid \lambda X. t \mid t[T] && \text{项} \\ v &::= \lambda x : T. t \mid \lambda X. t && \text{值} \\ T &::= X \mid T \rightarrow T \mid \forall X. T && \text{类型} \\ \Gamma &::= \emptyset \mid \Gamma, x : T \mid \Gamma, X && \text{上下文} \end{aligned}$$

求值

$$\begin{aligned} &\frac{v_2 \text{ 为值}}{(\lambda x : T_{11}. t_{12}) v_2 \longrightarrow [x \mapsto v_2]t_{12}} \text{E-APPABS} \\ &\frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2} \text{E-APP1} \quad \frac{t_2 \longrightarrow t'_2}{v_1 t_2 \longrightarrow v_1 t'_2} \text{E-APP2} \\ &\frac{t_1 \longrightarrow t'_1}{t_1[T_2] \longrightarrow t'_1[T_2]} \text{E-TAPP} \\ &\frac{}{(\lambda X. t_{11})[T_2] \longrightarrow [X \mapsto T_2]t_{11}} \text{E-TAPPTABS} \end{aligned}$$

赋型

$$\begin{aligned} &\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \text{T-VAR} \\ &\frac{\Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2} \text{T-ABS} \\ &\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}} \text{T-APP} \\ &\frac{\Gamma, X \vdash t_2 : T_2}{\Gamma \vdash \lambda X. t_2 : \forall X. T_2} \text{T-TABS} \\ &\frac{\Gamma \vdash t_1 : \forall X. T_{11}}{\Gamma \vdash t_1[T_2] : [X \mapsto T_2]T_{11}} \text{T-TAPP} \end{aligned}$$

图 23-1: 多态 lambda 演算 (System F)

T-TAbs 的前提在上下文中加入 X , 以记录它在子推导中的作用域。沿用 第5.3节的约定, 新绑定的项变量或类型变量不得与 Γ 中已经绑定的名称重复; 必要时可以一致地重命名 lambda 所绑定的类型变量。有些 System F 的表述会把“名称必须未在上下文中使用”这项新鲜性条件作为 T-TAbs 的显式旁条件; 本书则把它纳入构造上下文的规则。另外, 只有当 T 中的每个自由类型变量都已在 Γ 中绑定时, $\Gamma, x : T$ 才是良构上下文。

目前，写在上下文中的类型变量只负责记录作用域，并保证同一个类型变量不会重复加入上下文。到[第二十六章](#)和[第二十九章](#)，还会分别为类型变量记录上界和种类等信息。[图23-1](#)给出了完整的纯 System F；灰底部分是相对于简单类型 lambda 演算新增的内容。纯系统没有记录、自然数和布尔值等类型构造子，也没有 let 和 fix 等项构造；这些扩展都可以直接加入，后面的示例会自由使用它们。

23.4 示例

下面给出若干多态编程示例。我们先用几个短小、但难度逐渐增加的例子熟悉 System F，再回顾列表、树等常见数据结构上的多态程序。最后两个小节将[第五章](#)在无类型 lambda 演算中给出的简单代数数据类型编码改写为带类型版本。这些编码的实际用途不大：布尔值、数和列表等重要特性若作为高级程序语言的基本构造，通常更容易编译出高效代码；不过，这些编码是理解 System F 的细节和能力的绝佳练习。[第二十四章](#)还会展示多态在模块化程序设计和抽象数据类型中的用途。

热身

多态恒等函数为

$$\text{id} = \lambda X. \lambda x : X. x$$

它的类型和几个实例为

$$\begin{aligned} \text{id} & : \quad \forall X. X \rightarrow X, \\ \text{id}[\text{Nat}] & : \quad \text{Nat} \rightarrow \text{Nat}, \\ \text{id}[\text{Nat}] \ 0 & \longrightarrow^* 0 : \text{Nat} \end{aligned}$$

更实用一些的例子是多态加倍函数：

$$\text{double} = \lambda X. \lambda f : X \rightarrow X. \lambda a : X. f(f \ a)$$

它具有类型 $\forall X. (X \rightarrow X) \rightarrow X \rightarrow X$ 。对不同的类型实参作实例化，可以得到

$$\begin{aligned} \text{doubleNat} = \text{double}[\text{Nat}] & : \quad (\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat}, \\ \text{doubleNatArrowNat} & = \text{double}[\text{Nat} \rightarrow \text{Nat}] \end{aligned}$$

后者的类型为

$$\begin{aligned} & ((\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat}) \\ & \rightarrow (\text{Nat} \rightarrow \text{Nat}) \rightarrow \text{Nat} \rightarrow \text{Nat} \end{aligned}$$

完成类型实例化后，还可以继续提供普通函数和值：

$$\text{double}[\text{Nat}](\lambda x : \text{Nat}. \text{succ}(\text{succ} \ x)) \ 3 \longrightarrow^* 7$$

回想[练习9.3.2](#)：简单类型 lambda 演算无法为无类型项 $\lambda x. x \ x$ 赋型。System F 则可以让 x 具有全称类型，并先在合适的类型处将它实例化，从而允许多态自应用。令

$$\text{selfApp} = \lambda x : \forall X. X \rightarrow X. x[\forall X. X \rightarrow X] \ x$$

则

$$\text{selfApp} : (\forall X. X \rightarrow X) \rightarrow (\forall X. X \rightarrow X)$$

这里先把 x 的全称类型实例化为它自身的类型，所得函数便可以接收原来的 x 。同理，

$$\text{quadruple} = \lambda X. \text{double}[X \rightarrow X](\text{double}[X])$$

? quadruple : All X. (X->X) -> X -> X

把加倍操作应用于自身，得到四次应用。

练习 23.4.1 (★★). 使用图23-1的规则，验证上述各项确实具有所标出的类型。

[查看练习 23.4.1 的译者参考解答](#)

多态列表

若语言提供列表构造子，常用操作可具有如下全称类型：

$$\begin{aligned} \text{nil} &: \forall X. \text{List } X, \\ \text{cons} &: \forall X. X \rightarrow \text{List } X \rightarrow \text{List } X, \\ \text{isnil} &: \forall X. \text{List } X \rightarrow \text{Bool}, \\ \text{head} &: \forall X. \text{List } X \rightarrow X, \\ \text{tail} &: \forall X. \text{List } X \rightarrow \text{List } X \end{aligned}$$

这与第11.12节为列表专门编写的规则表达同一约束，但列表现在可以被看作提供若干多态常量的库，而不必烘焙进核心语言。同样的处理也适用于第十三章的 Ref 类型和引用单元基本操作，以及许多其他常见的数据结构和控制结构。

$$\begin{aligned} \text{map} = & \lambda X. \lambda Y. \lambda f : X \rightarrow Y. \text{fix}(\lambda m : \text{List } X \rightarrow \text{List } Y. \lambda l : \text{List } X. \\ & \text{if isnil}[X] \ l \ \text{then} \ \text{nil}[Y] \\ & \text{else} \ \text{cons}[Y] \ (f(\text{head}[X] \ l)) \ (m(\text{tail}[X] \ l))) \end{aligned}$$

它具有类型

$$\forall X. \forall Y. (X \rightarrow Y) \rightarrow \text{List } X \rightarrow \text{List } Y$$

例如，令

$$l = \text{cons}[\text{Nat}] \ 4 \ (\text{cons}[\text{Nat}] \ 3 \ (\text{cons}[\text{Nat}] \ 2 \ (\text{nil}[\text{Nat}])))$$

则 $l : \text{List Nat}$ ，而

$$\text{head}[\text{Nat}](\text{map}[\text{Nat}][\text{Nat}](\lambda x : \text{Nat}. \text{succ } x) \ l) \longrightarrow^* 5$$

练习 23.4.2 (★★). 验证 map 具有上述类型。

[查看练习 23.4.2 的译者参考解答](#)

练习 23.4.3 (推荐, ★★). 仿照 `map`, 写出多态列表反转函数

$$\text{reverse} : \forall X. \text{List } X \rightarrow \text{List } X$$

本题最好直接使用检查器完成。请使用 `fullomega`, 并把该目录下 `test.f` 的内容复制到自己的输入文件开头。其中 `List` 及相关操作使用了第二十九章才会介绍的 `System Fω`; 完成本题并不需要先理解这些定义的具体写法。

[查看原书附录 A 中的 23.4.3 解答](#)

练习 23.4.4 (★★). 写出简单的多态排序函数

$$\text{sort} : \forall X. (X \rightarrow X \rightarrow \text{Bool}) \rightarrow \text{List } X \rightarrow \text{List } X$$

第一个参数是 X 元素的比较函数。

[查看练习 23.4.4 的译者参考解答](#)

Church 编码

第5.2节曾在无类型 lambda 演算中用函数编码布尔值、自然数和列表。这里说明怎样在 `System F` 中为这些 Church 编码加上类型; 如对编码本身已经陌生, 可以先回看第5.2节。

研究这些编码有两个理由。第一, 它们是练习类型抽象和类型应用的好材料。第二, 它们说明纯 `System F` 的计算表达能力十分丰富: 许多数据结构和控制结构无需加入新的基本构造便能表达。因此, 以 `System F` 为核心设计完整语言时, 可以为了效率和更方便的具体语法而把这些结构加入为基本构造, 而不改变核心语言的基本性质。当然, 并非所有高级语言特性都能如此加入; 例如, 像第十三章那样加入引用, 就会实质改变系统的计算性质。

先看 Church 布尔值。在无类型 lambda 演算中, 它们定义为

$$\text{tru} = \lambda t. \lambda f. t \quad \text{fls} = \lambda t. \lambda f. f$$

二者都接收两个参数并返回其中一个。若要给二者相同的类型, 两个参数必须具有同一类型: 调用者并不知道自己面对的是 `tru` 还是 `fls`。但这个共同类型可以任意选择, 因为二者除了返回一个参数外, 不会对参数做任何操作。于是得到 Church 布尔值的类型

$$\text{CBool} = \forall X. X \rightarrow X \rightarrow X$$

在无类型版本上加入类型抽象和参数类型, 就得到

$$\begin{aligned} \text{tru} &= \lambda X. \lambda t : X. \lambda f : X. t, \\ \text{fls} &= \lambda X. \lambda t : X. \lambda f : X. f \end{aligned}$$

```
? tru : CBool
? fls : CBool
```

例如

$$\text{not} = \lambda b : \text{CBool}. \lambda X. \lambda t : X. \lambda f : X. b[X] f t$$

```
? not : CBool -> CBool
```

练习 23.4.5 (推荐, ★). 写出接受两个 CBool 参数并计算合取的 and。

[查看原书附录 A 中的 23.4.5 解答](#)

Church 自然数也可以用同样方法赋型。第5.2节给出的无类型编码是

$$\begin{aligned} \text{c0} &= \lambda s. \lambda z. z, \\ \text{c1} &= \lambda s. \lambda z. s z, \\ \text{c2} &= \lambda s. \lambda z. s(s z), \\ \text{c3} &= \lambda s. \lambda z. s(s(s z)) \end{aligned}$$

表示数字 n 的函数接收 s 和 z ，把 s 对 z 作用 n 次。因而 z 的类型必须等于 s 的参数类型，而 s 的结果还必须具有同一类型，以便反复作用。于是 Church 自然数的类型为

$$\text{CNat} = \forall X. (X \rightarrow X) \rightarrow X \rightarrow X$$

为无类型编码补上相应标注，便得到

$$\begin{aligned} \text{c0} &= \lambda X. \lambda s : X \rightarrow X. \lambda z : X. z, \\ \text{c1} &= \lambda X. \lambda s : X \rightarrow X. \lambda z : X. s z, \\ \text{c2} &= \lambda X. \lambda s : X \rightarrow X. \lambda z : X. s(s z) \end{aligned}$$

```
? c0 : CNat
? c1 : CNat
? c2 : CNat
```

带类型的后继函数定义如下：

$$\text{csucc} = \lambda n : \text{CNat}. \lambda X. \lambda s : X \rightarrow X. \lambda z : X. s(n[X] s z)$$

```
? csucc : CNat -> CNat
```

也就是说， $n[X] s z$ 已经把 s 对 z 作用了 n 次， csucc 再作用一次 s 。加法则既可以借助后继定义，

$$\text{cplus} = \lambda m : \text{CNat}. \lambda n : \text{CNat}. m[\text{CNat}] \text{csucc } n$$

```
? cplus : CNat -> CNat -> CNat
```

也可以直接写成

$$\text{cplus} = \lambda m : \text{CNat}. \lambda n : \text{CNat}. \lambda X. \lambda s : X \rightarrow X. \lambda z : X. m[X] s (n[X] s z)$$


```
? cplus : CNat -> CNat -> CNat
```

若语言还包含图8-2中的基本自然数，可以把 Church 数转换为普通自然数：

$$\text{cnat2nat} = \lambda m : \text{CNat}. m[\text{Nat}](\lambda x : \text{Nat}. \text{succ } x) 0$$

```
? cnat2nat : CNat -> Nat
```

于是可以直接检验前面的运算：

$$\text{cnat2nat}(\text{cplus}(\text{csucc } c0)(\text{csucc}(\text{csucc } c0))) \longrightarrow^* 3$$

```
? 3 : Nat
```

练习 23.4.6 (推荐, ★★). 写出函数 `iszro`，使其应用于 `c0` 时返回 `tru`，应用于其他 Church 数时返回 `fls`。

[查看原书附录 A 中的 23.4.6 解答](#)

练习 23.4.7 (★★). 验证

$$\text{ctimes} = \lambda m : \text{CNat}. \lambda n : \text{CNat}. \lambda X. \lambda s : X \rightarrow X. n[X](m[X]s),$$

$$\text{cexp} = \lambda m : \text{CNat}. \lambda n : \text{CNat}. \lambda X. n[X \rightarrow X](m[X])$$

均具有类型 $\text{CNat} \rightarrow \text{CNat} \rightarrow \text{CNat}$ ，并说明它们为何分别表示乘法和幂运算。

[查看练习 23.4.7 的译者参考解答](#)

练习 23.4.8 (推荐, ★★). 证明

$$\text{PairNat} = \forall X. (\text{CNat} \rightarrow \text{CNat} \rightarrow X) \rightarrow X$$

可以表示自然数对，并定义和验证下列三个函数：

$$\text{pairNat} : \text{CNat} \rightarrow \text{CNat} \rightarrow \text{PairNat},$$

$$\text{fstNat} : \text{PairNat} \rightarrow \text{CNat},$$

$$\text{sndNat} : \text{PairNat} \rightarrow \text{CNat}$$

[查看原书附录 A 中的 23.4.8 解答](#)

练习 23.4.9 (推荐, ★★★). 利用练习23.4.8的数对写出 Church 数的前驱函数 `prd`；输入为 0 时也应返回 0。提示：定义 $f(i, j) = (i + 1, i)$ ，把它从 $(0, 0)$ 开始迭代 n 次，再取第二分量。

[查看原书附录 A 中的 23.4.9 解答](#)

练习 23.4.10 (★★★). 设 $k = \lambda x. \lambda y. x$ 、 $i = \lambda x. x$ 。为无类型 Church 数的前驱函数 `vpred`

$$\lambda n. \lambda s. \lambda z. n(\lambda p. \lambda q. q(p s))(k z) i$$

补上类型抽象、类型应用和参数类型，使其成为 System F 项。Barendregt (1992) 将这一项归功于 J. Velmans。额外加分：说明它为何能够计算前驱。

[查看原书附录 A 中的 23.4.10 解答](#)

列表编码

这个例子进一步展示了纯 System F 的表达能力：上一小节利用列表基本操作写出的所有程序，实际上也能在纯系统中表达。下面为了方便仍会在一个定义中使用 `fix`；[练习23.4.11](#)和[23.4.12](#)会说明，实质相同的构造也可以避开它。

[练习5.2.8](#)曾把无类型列表编码成类似 Church 自然数的函数：一元记法中的自然数可以看作由若干虚设元素组成的列表。推广到任意元素类型后，列表 $[x, y, z]$ 被表示成一个函数：给它函数 f 和初值 v ，它会计算 $f x (f y (f z v))$ 。用 OCaml 的术语说，列表被表示成了它自己的 `fold_right` 函数。

含 X 元素的 Church 列表可以看作它自己的右折叠函数：

$$\text{List } X = \forall R. (X \rightarrow R \rightarrow R) \rightarrow R \rightarrow R$$

于是可以定义如下操作；其中 `nil` 后的 `as` 标注² 只影响类型的打印形式：

```
nil = λX. (λR. λc : X → R → R. λn : R. n) as List X,
cons = λX. λhd : X. λtl : List X. (λR. λc : X → R → R. λn : R. c hd (tl[R] c n)) as List X,
isnil = λX. λl : List X. l[Bool](λhd : X. λtl : Bool.false) true
```

```
? nil : All X. List X
? cons : All X. X -> List X -> List X
? isnil : All X. List X -> Bool
```

为处理空列表的 `head`，若语言带 `fix`，可先定义

$$\text{diverge} = \lambda X. \lambda_ : \text{Unit}. \text{fix}(\lambda x : X. x) : \forall X. \text{Unit} \rightarrow X$$

最直接的写法是把 `diverge[X] unit` 作为折叠基值：

$$\text{head} = \lambda X. \lambda l : \text{List } X. l[X](\lambda hd : X. \lambda tl : X. hd)(\text{diverge}[X] \text{ unit})$$

```
? head : All X. List X -> X
```

遗憾的是，按值求值会在把基值传给 l 之前先求它；所以这一定义即使作用于非空列表也会发散。应把 `unit` 参数留到折叠完成后再传入：

$$\text{head} = \lambda X. \lambda l : \text{List } X.$$

$$(l[\text{Unit} \rightarrow X] (\lambda hd : X. \lambda tl : \text{Unit} \rightarrow X. \lambda_ : \text{Unit}. hd) (\text{diverge}[X])) \text{ unit}$$

```
? head : All X. List X -> X
```

²这项标注使类型检查器能以易读的形式输出 `nil` 的类型。[第11.4节](#)提过，本书各类型检查器在输出类型前都会作一次简单的缩写折叠，但该过程还不能自动处理 `List` 这种“带参数的缩写”。

其中, l 的折叠函数参数具有类型 $X \rightarrow (\text{Unit} \rightarrow X) \rightarrow (\text{Unit} \rightarrow X)$, 基值具有类型 $\text{Unit} \rightarrow X$; 二者共同构造出一个 $\text{Unit} \rightarrow X$ 型函数。若 l 是空列表, 这个结果就是 `diverge[X]`; 若 l 非空, 它忽略 `unit` 并返回首元素。最后再把所得函数应用于 `unit`, 才得到真正的列表首元素 (或在列表为空时发散), 因此 `head` 具有预期类型。

尾部函数还需要通用的 Church 数对编码。令

$$\text{Pair } X Y = \forall R. (X \rightarrow Y \rightarrow R) \rightarrow R$$

并使用以下三个多态操作:

$$\begin{aligned} \text{pair} &: \forall X. \forall Y. X \rightarrow Y \rightarrow \text{Pair } X Y, \\ \text{fst} &: \forall X. \forall Y. \text{Pair } X Y \rightarrow X, \\ \text{snd} &: \forall X. \forall Y. \text{Pair } X Y \rightarrow Y \end{aligned}$$

便可以写出

```
tail = λX. λl : List X.
  fst[List X][List X](
    l[Pair(List X)(List X)]
    (λhd : X. λtl : Pair(List X)(List X).
      pair[List X][List X](snd[List X][List X] tl)(cons[X] hd (snd[List X][List X] tl)))
    (pair[List X][List X](nil[X])(nil[X])))
```

其类型为 $\forall X. \text{List } X \rightarrow \text{List } X$ 。

练习 23.4.11 (★★). 纯 System F 没有 `fix`。写出另一版 `head`, 额外接收一个参数, 在列表为空时返回该参数。

[查看原书附录 A 中的 23.4.11 解答](#)

练习 23.4.12 (推荐, ★★★). 在不含 `fix` 的纯 System F 中, 定义

$$\text{insert} : \forall X. (X \rightarrow X \rightarrow \text{Bool}) \rightarrow \text{List } X \rightarrow X \rightarrow \text{List } X,$$

并用它构造列表排序函数。这个函数的三个实参依次是: 比较两个 X 型元素的函数、一个已排序的 X 型列表, 以及要插入的新元素。结果应把新元素插入到恰当位置, 也就是所有比它小的元素之后。

[查看原书附录 A 中的 23.4.12 解答](#)

23.5 基本性质

System F 的类型安全性质与简单类型 lambda 演算基本相同。其中, 保持性和进展性的证明都是第九章中相应证明的直接扩展。

定理 23.5.1 (保持性; 证明为推荐练习, $\star\star\star$). 若 $\Gamma \vdash t : T$ 且 $t \longrightarrow t'$, 则 $\Gamma \vdash t' : T$ 。

查看原书附录 A 中的 23.5.1 解答

定理 23.5.2 (进展性; 证明为推荐练习, $\star\star\star$). 若 t 是闭的良类型项, 则 t 或为值, 或存在 t' 使 $t \longrightarrow t'$ 。

查看原书附录 A 中的 23.5.2 解答

System F 还和简单类型 lambda 演算一样具有规范化性质, 即每个良类型程序的求值都会终止。与上面的类型安全定理不同, 这一性质的证明相当困难。它甚至有些令人惊讶: 正如练习 23.4.12 所示, 纯 System F 不借助 fix 也能表达排序函数。Girard 在博士论文中把第十二章的方法推广到 System F, 证明了规范化; 这是其论文的一项重大成果 (Girard, 1972; 另见 Girard、Lafont 与 Taylor, 1989)。此后, 许多研究者又对这项证明技术作了分析和改写; 可参见 Gallier (1990)。³

定理 23.5.3 (规范化). 每个良类型 System F 项都可规范化。

23.6 擦除、可赋型性与类型重构

System F 项可以通过删去类型标注、类型抽象和类型应用而映射到无类型项:

$$\begin{aligned} \text{erase}(x) &= x, \\ \text{erase}(\lambda x : T_1. t_2) &= \lambda x. \text{erase}(t_2), \\ \text{erase}(t_1 t_2) &= \text{erase}(t_1) \text{erase}(t_2), \\ \text{erase}(\lambda X. t_2) &= \text{erase}(t_2), \\ \text{erase}(t_1 [T_2]) &= \text{erase}(t_1) \end{aligned}$$

若存在良类型 System F 项 t 使 $\text{erase}(t) = m$, 则称无类型项 m 在 System F 中可赋型。类型重构问题要求由 m 找出这样的 t , 或判断不存在。

System F 的类型重构曾是程序语言研究中悬而未决时间最长的问题之一: 从 20 世纪 70 年代初一直没有答案, 直到 Wells 在 90 年代初证明其不可判定。

定理 23.6.1 (Wells, 1994). 给定闭的无类型 lambda 项 m , 判断是否存在良类型 System F 项 t 满足 $\text{erase}(t) = m$, 是不可判定的。

完整类型重构以外, System F 的若干部分类型重构问题也已知不可判定。这里必须先准确说明“部分”的含义。Pfenning (1993a) 把输入写成带有可选类型信息的预项 (*preterm*) P , 并以关系 $P \preceq M$ 表示: P 是 System F 项 M 的一种部分擦除。变量、应用和类型抽象

³若采用允许任意位置作完全 beta 归约的操作语义, System F 还具有强规范化性质: 从良类型项出发的每一条归约路径都必定终止。这里的确定性求值关系每项至多只有一条求值路径, 因而“规范化”和“强规范化”的区别不会显现出来。

保留结构；普通 lambda 抽象既可以保留参数类型，也可以省去它；类型应用既可以保留类型实参，也可以只留下空方括号：

$$\begin{array}{c}
 x \preceq x \\
 \hline
 P_1 \preceq M_1 \quad P_2 \preceq M_2 \\
 \hline
 P_1 P_2 \preceq M_1 M_2 \\
 \hline
 P \preceq M \\
 \hline
 P[T] \preceq M[T]
 \end{array}
 \qquad
 \begin{array}{c}
 P \preceq M \\
 \hline
 \lambda x : T. P \preceq \lambda x : T. M
 \end{array}
 \qquad
 \begin{array}{c}
 P \preceq M \\
 \hline
 \lambda x. P \preceq \lambda x : T. M
 \end{array}
 \qquad
 \begin{array}{c}
 P \preceq M \\
 \hline
 \lambda X. P \preceq \lambda X. M
 \end{array}
 \qquad
 \begin{array}{c}
 P \preceq M \\
 \hline
 P[] \preceq M[T]
 \end{array}$$

部分类型重构要判断：给定良构上下文 Γ 和预项 P ，是否存在在 Γ 下良类型的 System F 项 M ，使 $P \preceq M$ 。

定理 23.6.2 (Pfenning, 1993a). 上述部分类型重构问题不可判定。

这一结果沿着 Boehm (1985, 1989) 把部分类型重构与高阶合一联系起来的工作发展而来。Boehm 证明的是稍有不同的变体；Pfenning (1988, 1993a) 借助 Huet (1975) 的高阶合一算法得到了一种实用的部分类型重构技术。后来的高阶约束求解算法又消除了不终止或产生多个不唯一解的麻烦 (Dowek、Hardin、Kirchner 与 Pfenning, 1996)。LEAP (Pfenning 与 Lee, 1991)、Elf (Pfenning, 1989) 和 FX (O’Toole 与 Gifford, 1989) 等语言的实践表明，这类算法通常表现良好。

译者注：原书把“保留所有普通参数类型和类型抽象、只挖去类型应用实参”写成一个函数 erase_p ，并把相应判定问题列为不可判定定理。作者勘误指出，该定义并不正确，Boehm 的论文证明的也是略有差别的问题；恰好按原书文字所陈述的那个判定问题，目前仍属开放问题。上文因此采用勘误所指向的 Pfenning 正式定义，并只陈述这一变体已经证明的结论。

另一条路线来自 Perry (1990) 的观察：一等存在类型（第二十四章）可以同 ML 的数据类型机制结合。数据构造子和析构子可看作显式类型标注，指出何处需要把值注入或投影出不交并类型，何处需要折叠或展开递归类型；加入存在类型后，它们也指出何处要进行打包和开包。Läufer (1992) 以及 Läufer 与 Odersky (1994) 进一步发展了这一想法，Rémy (1994) 又把它扩展到一等、非直谓的全称量词。Odersky 与 Läufer (1996) 以及 Garrigue 与 Rémy (1997) 提出的方案保守扩展 ML：程序员可以显式标注函数参数，这些标注可以包含自动推断无法产生的嵌套全称量词，从而部分弥合 ML 与更强非直谓系统之间的差距。这一路线的优点是相对简单，并能自然地融入 ML 多态。

对于同时具有子类型和非直谓多态的系统，Pierce 与 Turner (1998；另见 Pierce 与 Turner, 1997；Hosoya 与 Pierce, 1999) 提出了局部类型推断 (*local type inference*)，亦称局部类型重构。GJ 和 Funnell 等语言设计采用了这一办法，其中 GJ 见 Bracha、Odersky、Stoutamire 与 Wadler (1998)，Funnell 见 Odersky 与 Zenger (2001)；Funnell 还引入了能力更强的着色局部类型推断 (Odersky、Zenger 与 Zenger, 2001)。

Cardelli (1993) 提出了更简单、但较难预测的贪心算法；依赖类型证明检查器 NuPrl (Howe, 1988) 和 Lego (Pollack, 1990) 也使用过类似算法。程序员可以省略任意类型标注，

解析器为每处省略生成新鲜的合一变量。子类型检查遇到 $X <: T$ 或 $T <: X$ 时，立即令 $X = T$ ，以最直接的选择满足当前约束。这个选择未必对后续约束最有利：其他实例化本可使整个程序通过，而贪心选择却可能令很远处的检查失败。Cardelli 的实现和早期 Pict 的经验 (Pierce 与 Turner, 2000) 表明，它几乎总能作出正确选择；一旦选错，错误信息却往往出现在远离错误实例化的位置，令程序员难以理解。

练习 23.6.3 (**).** 规范化性质已经说明，无类型项 $\Omega = (\lambda x.x x)(\lambda y.y y)$ 不可能在 System F 中赋值：它永远无法归约到范式。下面只使用赋值关系的规则，给出一个更直接的组合性证明。

1. 若 System F 项是变量、普通抽象 $\lambda x:T.t$ 或普通应用 ts ，就称它为 **暴露项**；也就是说，它不是类型抽象或类型应用。证明：若 t 良类型且 $\text{erase}(t) = m$ ，则存在某个同样擦除为 m 的良类型暴露项 s (上下文可以不同)。
2. 使用下列缩写表示嵌套的类型抽象、类型应用和全称类型：

$$\begin{aligned}\lambda \overline{X}.t &= \lambda X_1. \dots \lambda X_n. t, \\ t[\overline{A}] &= ((t[A_1]) \dots [A_{n-1}])[A_n], \\ \forall \overline{X}.T &= \forall X_1. \dots \forall X_n. T\end{aligned}$$

这些序列均可为空。证明：若 $\text{erase}(t) = m$ 且 $\Gamma \vdash t : T$ ，则存在 $s = \lambda \overline{X}.(u[\overline{A}])$ ，其中 u 是暴露项，并且 $\text{erase}(s) = m$ 、 $\Gamma \vdash s : T$ 。

3. 证明：若暴露项 t 在 Γ 下具有类型 T ，且 $\text{erase}(t) = mn$ ，则 $t = su$ ，其中 $\text{erase}(s) = m$ 、 $\text{erase}(u) = n$ ，并且对某个 U 有 $\Gamma \vdash s : U \rightarrow T$ 和 $\Gamma \vdash u : U$ 。
4. 设 $x:T \in \Gamma$ 。证明：若 $\Gamma \vdash u : U$ 且 $\text{erase}(u) = xx$ ，则以下两种情况之一成立：
 - (a) $T = \forall \overline{X}.X_i$ ，其中 $X_i \in \overline{X}$ ；
 - (b) $T = \forall \overline{X}_1 \overline{X}_2. T_1 \rightarrow T_2$ ，并且对某些类型序列 $\overline{A}, \overline{B}$,

$$[\overline{X}_1 \overline{X}_2 \mapsto \overline{A}]T_1 = [\overline{X}_1 \mapsto \overline{B}](\forall Z. T_1 \rightarrow T_2),$$

$$\text{其中 } |\overline{A}| = |\overline{X}_1 \overline{X}_2|, |\overline{B}| = |\overline{X}_1|.$$

5. 证明：若 $\text{erase}(s) = \lambda x.m$ 且 $\Gamma \vdash s : S$ ，则对某些 $\overline{X}, S_1, S_2$ ，有 $S = \forall \overline{X}. S_1 \rightarrow S_2$ 。
6. 定义类型的最左叶：

$$\begin{aligned}\text{leftmost-leaf}(X) &= X, \\ \text{leftmost-leaf}(S \rightarrow T) &= \text{leftmost-leaf}(S), \\ \text{leftmost-leaf}(\forall X.S) &= \text{leftmost-leaf}(S)\end{aligned}$$

证明：若

$$[\overline{X}_1 \overline{X}_2 \mapsto \overline{A}](\forall \overline{Y}. T_1) = [\overline{X}_1 \mapsto \overline{B}](\forall Z. (\forall \overline{Y}. T_1) \rightarrow T_2),$$

则 $\text{leftmost-leaf}(T_1) = X_i$ ，其中某个 $X_i \in \overline{X}_1 \overline{X}_2$ 。

7. 证明 Ω 在 System F 中不可赋型。

[查看原书附录 A 中的 23.6.3 解答](#)

23.7 擦除与求值次序

图23-1采用类型传递语义 (*type-passing semantics*): 运行时会把类型实参替换进类型抽象体。第二十五章的 System F OCaml 实现采用的正是这种办法。实际编译器通常希望类型检查后擦除类型。在真正以 System F 为基础的解释器或编译器中, 运行时操作类型会带来显著开销。而且, 类型标注通常不参与运行时选择: 任意改写一个良类型程序中的类型标注, 得到的程序仍会有相同的运行行为。因此, 许多多态语言在类型检查后擦除全部类型, 再解释无类型项或把它们编译成机器码。⁴ 不过, 完整语言可能还有可变引用或异常等副作用; 这时, 按值调用下的擦除必须更谨慎, 因为简单擦除可能改变求值次序。例如

$$\text{let } f = (\lambda X.\text{error}) \text{ in } 0$$

求值为 0, 因为类型抽象是值, 其内部错误不会立即执行; 若直接擦除类型抽象, 就得到 $\text{let } f = \text{error} \text{ in } 0$, 会立刻抛出错误。⁵

适合按值调用的擦除应把类型抽象保留为无类型的零信息函数, 把类型应用变成向它传递任意哑值:

$$\begin{aligned} \text{erase}_v(x) &= x, \\ \text{erase}_v(\lambda x : T_1. t_2) &= \lambda x. \text{erase}_v(t_2), \\ \text{erase}_v(t_1 t_2) &= \text{erase}_v(t_1) \text{erase}_v(t_2), \\ \text{erase}_v(\lambda X. t_2) &= \lambda _. \text{erase}_v(t_2), \\ \text{erase}_v(t_1 [T_2]) &= \text{erase}_v(t_1) \text{dummyv} \end{aligned}$$

其中 dummyv 可以取任意无类型值, 如 unit 。这样, 类型抽象原来阻止函数体立即求值的作用仍然保留。⁶ 这个新的擦除函数与无类型求值“可交换”: 先擦除再求值, 或先求值再擦除, 都会得到相应的结果。下面的定理精确表述了这一点。

练习 23.7.1 (★★). 把§22.7关于引用与不安全 let 多态的反例, 写成带引用的 System F 项 (引用的定义见图13-1)。

[查看原书附录 A 中的 23.7.1 解答](#)

定理 23.7.2. 若 $\text{erase}_v(t) = u$, 则有且仅有以下两种情况之一:

⁴有些语言包含类型转换 (第15.5节) 等特性, 因而必须采用某种类型传递实现。高性能实现通常只在运行时保留最低限度的类型信息, 例如只把类型传给确实会使用它的多态函数。

⁵这和§22.7中引用与 ML 风格 let 多态组合后出现的不安全问题有关: 那个例子对 let 右侧作泛化, 对应于这里显式引入类型抽象。

⁶与此不同, §22.7为带副作用的 ML 类型重构引入的值限制会擦除类型抽象; 在那里, 对类型变量作泛化相当于插入类型抽象, 但只有当它紧邻普通项抽象或其他句法值构造子时才允许泛化。这些构造本身也会阻止按值求值进入其内部, 因而仍能保证安全。

1. t 与 u 都是各自求值关系下的范式;
2. $t \longrightarrow t'$ 、 $u \longrightarrow u'$, 并且 $\text{erase}_v(t') = u'$ 。

23.8 System F 的受限片段

System F 的优雅和表达能力使它在多态理论研究中占据核心地位。对实际语言设计而言,完整类型重构不可判定却常被视为过高的代价,尤其是完整 System F 的能力在许多程序中并不常用。因此,人们提出了多种能力较弱、重构问题也更易处理的片段。ML 的 let 多态也称前束多态 (*prenex polymorphism*): 类型变量只实例化为不含量词的单型,而且量化类型不出现在箭头左侧。由于 ML 的 let 在类型系统中承担特殊作用,这一对应关系的精确表述并不简单;详见 Jim (1995)。

另一种常见限制是 Leivant (1983) 提出的秩 2 多态,此后又有许多研究;可参见 Jim (1995, 1996)。把类型画成树;若从根到任一 $\forall$ 的路径,在两个或更多箭头处进入参数一侧,该类型就超过秩 2。秩 2 系统把所有类型都限制为秩 2。例如

$$(\forall X. X \rightarrow X) \rightarrow \text{Nat}$$

属于秩 2; $\text{Nat} \rightarrow \text{Nat}$ 和 $\text{Nat} \rightarrow (\forall X. X \rightarrow X) \rightarrow \text{Nat} \rightarrow \text{Nat}$ 也属于秩 2。相反,

$$((\forall X. X \rightarrow X) \rightarrow \text{Nat}) \rightarrow \text{Nat}$$

不属于。秩 2 系统能给比 ML 片段更多的无类型项赋型;其重构复杂度与 ML 相同,即 DEXPTIME 完备 (Kfoury 与 Tiuryn, 1990)。Kfoury 与 Wells (1994) 给出了第一个直接的秩 2 类型重构算法;秩 3 及以上的 System F 重构则不可判定。

秩 2 的限制也可用于量词以外的强类型构造子。例如,可以把交类型 (第15.7节) 限制为:从类型树根到任一交类型的路径,不得在两个或更多箭头处进入参数一侧 (Kfoury、Mairson、Turbak 与 Wells, 1999)。System F 的秩 2 片段与一阶交类型系统的秩 2 片段关系密切;Jim (1995) 证明,它们恰好能为同一批无类型 lambda 项赋型。

23.9 参数性

回想第23.4节中的定义:

$$\text{CBool} = \forall X. X \rightarrow X \rightarrow X$$

以及布尔常量

$$\begin{aligned} \text{tru} &= \lambda X. \lambda t : X. \lambda f : X. t, & ? \text{tru} : \text{CBool}, \\ \text{fls} &= \lambda X. \lambda t : X. \lambda f : X. f, & ? \text{fls} : \text{CBool} \end{aligned}$$

只看这个类型的结构,几乎就能机械地写出 tru 和 fls。最外层是 $\forall X$, 所以值必须先写成类型抽象 $\lambda X. \dots$ 。余下类型是 $X \rightarrow X \rightarrow X$, 所以抽象体必须再接收两个 X 型参数,例如 $\lambda t : X. \lambda f : X. \dots$ 。最后必须返回一个 X 型结果;但 X 是未知的类型,程序无法自行

构造这种值，只能返回已经收到的 t 或 f 。于是便得到 tru 和 fls 。严格说来， CBool 还有 $(\lambda b : \text{CBool}. b) \text{tru}$ 之类的其他项，但它们在运行行为上仍必定等同于 tru 或 fls 。

这一现象来自参数性 (*parametricity*)：参数多态程序不能根据未知类型的内部结构改变行为，其类型因而蕴含很强的行为约束。

参数性由 Reynolds (1974, 1983) 提出；此后又有大量相关研究，包括 Reynolds (1984)、Reynolds 与 Plotkin (1993)、Bainbridge 等 (1990)、Ma (1992)、Mitchell (1986)、Mitchell 与 Meyer (1985)、Hasegawa (1991)、Pitts (1987, 1989, 2000)、Abadi、Cardelli 与 Curien (1993)、Plotkin 与 Abadi (1993)、Plotkin、Abadi 与 Cardelli (1994)，以及 Wadler (1989, 2001) 等。Wadler (1989) 给出了一篇适合入门的综述。

23.10 非直谓性

若一个定义中的量词范围包含正在定义的对象本身，就称该定义是 非直谓的 (*impredicative*)。例如

$$T = \forall X. X \rightarrow X$$

中的 X 遍历所有类型，其中也包括 T 本身；所以 T 型项可以在 T 处实例化。ML 多态通常称为直谓的 (*predicative*) 或分层的，因为类型变量只遍历不含量词的单型。

“直谓”与“非直谓”来自逻辑。Quine (1987) 对其历史作了如下清晰概括：

在与 *Henri Poincaré* 的讨论中……*Russell* 曾把 *Russell* 悖论暂时归因于他所谓的“恶性循环谬误”。这种“谬误”是：用一个成员资格条件规定一个类，而该条件又直接或间接引用一批类，其中包括正在规定的这个类本身。例如，*Russell* 悖论背后的成员资格条件是“不属于自身”： $x \notin x$ 。若让这个条件中的 x 也可以取作正由该条件定义的类本身，就会产生悖论。*Russell* 与 *Poincaré* 后来把这样的成员资格条件称为“非直谓的”，并认为它不能用来规定一个类。于是，*Russell* 悖论及其他集合论悖论便被拆解了……

In exchanges with Henri Poincaré...Russell attributed [Russell's] paradox tentatively to what he called a vicious-circle fallacy. The "fallacy" consisted in specifying a class by a membership condition that makes reference directly or indirectly to a range of classes one of which is the very class that is being specified. For instance the membership condition behind Russell's Paradox is non-self-membership: x not a member of x . The paradox comes of letting the x of the membership condition be, among other things, the very class that is being defined by the membership condition. Russell and Poincaré came to call such a membership condition impredicative, and disqualified it as a means of specifying a class. The paradoxes of set theory, Russell's and others, were thus dismantled...

——*W. V. Quine, 1987*

再说术语：“直谓”和“非直谓”从何而来？关于类和成员资格条件，我们那条已经破旧不堪的陈词是 *Russell* 所说的“每个谓词都决定一个类”；后来，为了应付这条陈词的破损，他不再把那些不应视为决定某个类的成员资格条件称为“谓词”。因此，“直谓”一词本来并不特指层级方法，也不特指逐步构造的比喻；那只不过是 *Russell* 与 *Poincaré* 对于哪些成员资格条件可以产生类，也就是哪些条件属于“直谓”，所提的具体方案。不过，后来尾巴摇起了狗。今天，直谓集合论就是构造性集合论；而“非直谓定义”则严格按上一段所述来理解，无论人们选择把哪些成员资格条件看作能够决定一个类。

Speaking of terminology, whence “predicative” and “impredicative”? Our tattered platitude about classes and membership conditions was, in Russell’s phrase, that every predicate determines a class; and then he accommodates the tattering of the platitude by withdrawing the title of predicate from such membership conditions as were no longer to be seen as determining classes. “Predicative” thus did not connote the hierarchical approach in particular, or the metaphor of progressive construction; that was just Russell and Poincaré’s particular proposal of what membership conditions to accept as class-productive, or “predicative.” But the tail soon came to wag the dog. Today predicative set theory is constructive set theory, and impredicative definition is strictly as explained in the foregoing paragraph, regardless of what membership conditions one may choose to regard as determining classes.

——*W. V. Quine, 1987*

23.11 注记

关于 System F 的进一步入门材料，可参见 Reynolds (1990) 及其 *Theories of Programming Languages* (1998b)。

第二十四章 存在类型

第二十三章讨论了类型系统中的全称量词。本章考察与之对偶的存在量词；存在类型 (*existential type*) 为数据抽象和信息隐藏提供了简洁的类型论基础。

24.1 动机

存在类型在根本上并不比全称类型复杂；事实上，§24.3 将用全称类型直接编码存在类型。不过，存在类型的引入式和消去式在句法上比类型抽象和类型应用略显繁复，初看时可能令人困惑。下面先从两种互补的理解入手。

全称类型 $\forall X.T$ 可以从两个角度理解。按逻辑的理解，它的值对每种 S 都具有类型 $[X \mapsto S]T$ 。这与类型擦除语义相吻合：例如，多态恒等函数 $\lambda X.\lambda x : X.x$ 擦除类型信息后就是无类型恒等函数 $\lambda x.x$ ，能把任意类型 S 的参数原样返回。按操作语义的理解，全称类型的值则像一个接收类型 S 、返回相应特化项的函数；这正是第二十三章所用的解释，其中类型应用的归约本身就是一步计算。

存在类型

$$\{\exists X, T\}$$

也有两种对应理解。按逻辑的理解，它表示“对某个类型 S ，存在一个 $[X \mapsto S]T$ 型的值”¹；按操作语义的理解，它是一个由类型 S 和项 t 组成的包，其中 t 具有类型 $[X \mapsto S]T$ 。后者与程序语言中的模块和抽象数据类型联系得更直接，因此本章主要采用这一理解。我们用花括号写存在类型，以突出它的值是一种元组，而不用更常见的记法 $\exists X.T$ 。

要掌握存在类型，需要分别说明怎样构造其中的值，以及怎样在计算中使用这些值；用§9.4的术语说，就是给出它的引入式与消去式。

存在类型的值把一个类型和一个项配成一对。本章采用

$$\{*S, t\} \text{ as } \{\exists X, T\}$$

¹本章大部分内容研究的是第二十三章的 System F 加上图24-1中的存在类型；示例还使用记录（图11-7）和自然数（图8-2）。原书对应的 OCaml 实现是 `fullpoly`。

作为打包形式。² S 是 隐藏表示类型 (*hidden representation type*), 在逻辑语境中也称见证类型 (参见§9.4); t 是值分量。可以把这样的值看作一个只有一个隐藏类型分量和一个项分量的小包或小模块。³

显式类型标注不可省略, 因为同一个包可能属于不止一个存在类型。例如, 不带标注的

$$p = \{*\text{Nat}, \{a = 5, f = \lambda x : \text{Nat}. \text{succ } x\}\}$$

既可按 $\{\exists X, \{a : X, f : X \rightarrow X\}\}$ 打包, 也可按 $\{\exists X, \{a : X, f : X \rightarrow \text{Nat}\}\}$ 打包。类型检查器无法自行决定程序员想要哪一个接口, 因此包的完整写法必须明确给出预期类型:

```
p = {*Nat, {a=5, f=lambda x:Nat. succ(x)}}
  as {Some X, {a:X, f:X->X}};
? p : {Some X, {a:X, f:X->X}}

p1 = {*Nat, {a=5, f=lambda x:Nat. succ(x)}}
  as {Some X, {a:X, f:X->Nat}};
? p1 : {Some X, {a:X, f:X->Nat}}
```

第一个包的类型分量是 Nat , 项分量则是包含字段 a 和 f 的记录; 检查这个包时, 规则 T-Pack 取 $X = \text{Nat}$ 。这里的 **as** 与§11.4的类型归属形式相似; 我们只是把一次类型归属直接纳入打包句法。

存在类型的引入规则为

$$\frac{\Gamma \vdash t_2 : [X \mapsto T_1]T_2}{\Gamma \vdash \{*T_1, t_2\} \text{ as } \{\exists X, T_2\} : \{\exists X, T_2\}} \text{ T-PACK}$$

不同的隐藏表示类型也可以产生相同的存在类型:

```
p2 = {*Nat, 0} as {Some X, X};
? p2 : {Some X, X}

p3 = {*Bool, true} as {Some X, X};
? p3 : {Some X, X}
```

更有用的例子是

```
p4 = {*Nat, {a=0, f=lambda x:Nat. succ(x)}}
  as {Some X, {a:X, f:X->Nat}};
? p4 : {Some X, {a:X, f:X->Nat}}
```

²这里用星号标出类型分量, 以免与 §11.7 的普通项元组混淆。存在类型的引入式也常写作 **pack** $X = S$ **with** t 。

³为了让记号便于处理, 这里只采用一个类型分量和一个项分量。多个类型分量可以用单类型存在包嵌套表示; 多个项分量可以放进元组或记录。按此约定, 多分量包可展开为

$$\{*S_1, *S_2, t_1, t_2\} \stackrel{\text{def}}{=} \{*S_1, \{*S_2, \{t_1, t_2\}\}\}$$

```
p5 = {*Bool, {a=true, f=lambda x:Bool. 0}}
      as {Some X, {a:X, f:X->Nat}};
? p5 : {Some X, {a:X,f:X->Nat}}
```

练习 24.1.1 (*). 下面是同一主题的三种变体:

```
p6 = {*Nat, {a=0, f=lambda x:Nat. succ(x)}}
      as {Some X, {a:X, f:X->X}};
? p6 : {Some X, {a:X,f:X->X}}

p7 = {*Nat, {a=0, f=lambda x:Nat. succ(x)}}
      as {Some X, {a:X, f:Nat->X}};
? p7 : {Some X, {a:X,f:Nat->X}}

p8 = {*Nat, {a=0, f=lambda x:Nat. succ(x)}}
      as {Some X, {a:Nat, f:Nat->Nat}};
? p8 : {Some X, {a:Nat,f:Nat->Nat}}
```

它们在哪些方面不如 p_4 和 p_5 有用?

[查看原书附录 A 中的 24.1.1 解答](#)

消去存在类型可以类比为打开或导入模块: 它允许后续程序使用模块分量, 同时仍把类型分量的具体身份保持为抽象。我们用带模式的绑定

$$\mathbf{let} \{X, x\} = t_1 \mathbf{in} t_2$$

打开它: 若 t_1 求值得到一个存在包, 就把它的类型分量和项分量分别绑定到模式变量 X 和 x , 供 t_2 使用。另一种常见写法是 $\mathbf{open} t_1 \mathbf{as} \{X, x\} \mathbf{in} t_2$ 。

相应的消去规则为

$$\frac{\Gamma \vdash t_1 : \{\exists X, T_{11}\} \quad \Gamma, X, x : T_{11} \vdash t_2 : T_2}{\Gamma \vdash \mathbf{let} \{X, x\} = t_1 \mathbf{in} t_2 : T_2} \text{ T-UNPACK}$$

例如, 打开上面的 p_4 , 可以使用其记录字段计算自然数:

```
let {X,x}=p4 in (x.f x.a);
? 1 : Nat

let {X,x}=p4 in (lambda y:X. x.f y) x.a;
? 1 : Nat
```

第二个例子的开包体也使用了类型变量 X 。检查开包体时, 包的表示类型仍保持抽象, 因此只允许接口 $\{a : X, f : X \rightarrow \text{Nat}\}$ 所保证的操作。特别是, 不能擅自把 $x.a$ 当作自然数:

```
let {X,x}=p4 in succ(x.a);
? Error: argument of succ is not a number
```

这是必要的限制，因为 p_4 和 p_5 的接口相同，实际表示却分别是 Nat 和 Bool 。

开包还可能以一种更微妙的方式无法通过类型检查。T-Unpack 的前提在扩展了 X 的上下文中计算 t_2 的类型，但规则结论的上下文已经没有 X ，所以结果类型 T_2 不能自由出现 X ：

```
let {X,x}=p in x.a;
? Error: Scoping error!
```

否则隐藏类型会越过开包作用域而逃逸。§25.5将进一步讨论这一点。

存在包的计算规则也很直接：

$$\frac{}{\text{let } \{X, x\} = \{*T_{11}, v_{12}\} \text{ as } \{\exists X, T_{12}\} \text{ in } t_2 \longrightarrow [X \mapsto T_{11}][x \mapsto v_{12}]t_2} \text{E-UNPACKPACK}$$

当 **let** 的第一项已经求值为具体的存在包时，规则把包的类型分量和值分量分别代入开包体。

图24-1汇总了 System F 加入存在类型后的全部新增规则。

存在类型

扩展 System F (图23-1)

新增句法形式

$t ::= \dots$	项
$\{*T, t\} \text{ as } T$	打包
$\text{let } \{X, x\} = t \text{ in } t$	开包
$v ::= \dots$	值
$\{*T, v\} \text{ as } T$	包值
$T ::= \dots$	类型
$\{\exists X, T\}$	存在类型

新增求值规则 $t \longrightarrow t'$

E-UNPACKPACK

$$\frac{}{\text{let } \{X, x\} = \{*T_{11}, v_{12}\} \text{ as } \{\exists X, T_{12}\} \text{ in } t_2 \longrightarrow [X \mapsto T_{11}][x \mapsto v_{12}]t_2}$$

$$\frac{t_2 \longrightarrow t'_2}{\{*T_1, t_2\} \text{ as } T \longrightarrow \{*T_1, t'_2\} \text{ as } T} \text{E-PACK}$$

$$\frac{t_1 \longrightarrow t'_1}{\text{let } \{X, x\} = t_1 \text{ in } t_2 \longrightarrow \text{let } \{X, x\} = t'_1 \text{ in } t_2} \text{E-UNPACK}$$

新增赋型规则 $\Gamma \vdash t : T$

$$\frac{\Gamma \vdash t_2 : [X \mapsto T_1]T_2}{\Gamma \vdash \{*T_1, t_2\} \text{ as } \{\exists X, T_2\} : \{\exists X, T_2\}} \text{T-PACK}$$

$$\Gamma \vdash t_1 : \{\exists X, T_{11}\}$$

$$\frac{\Gamma, X, x : T_{11} \vdash t_2 : T_2}{\Gamma \vdash \text{let } \{X, x\} = t_1 \text{ in } t_2 : T_2} \text{T-UNPACK}$$

图 24-1: 存在类型

T-Unpack 没有把 $X \notin \text{FV}(T_2)$ 另列为前提；这项限制来自上下文的良构性。结论回到上下文 Γ 后，结果类型 T_2 仍须在 Γ 中良构，所以其中不能自由出现只在第二个前提中有效的 X 。按照 E-UnpackPack，包求值完成后，实际表示类型和实际值会同时代入开包体。用模块作类比，这相当于一次链接：用模块的实际内容替换表示其分量的符号名 X 与 x 。由于这一步会消去 X ，归约后的程序可以具体访问包的内部。这再次说明，项在计算过程中可能变得“更有类型”；甚至非良类型项也可能归约成良类型项。

24.2 用存在类型实现数据抽象

§1.2曾指出，类型系统不只用于发现 `2 + true` 一类局部错误，也为大型程序的组织提供关键支持。类型既能维护语言内建的抽象，也能维护程序员自己定义的抽象；换句话说，它们既能保护机器免受程序破坏，也能把程序的不同部分彼此隔离。⁴本节以存在类型为共同框架，比较两种数据抽象风格：经典的抽象数据类型和对象。

与第十八章的对象编码不同，本节示例全部采用纯函数式程序。这只是为了便于讲解：模块化和抽象机制在很大程度上独立于抽象内部是否含有状态；练习24.2.2与练习24.2.4会把正文示例改成有状态的版本。纯函数式示例所需的形式框架更精简，而且有时会让类型问题更突出。命令式程序可以借助可变变量，让相距很远的程序片段通过一条“旁路”直接通信；纯函数式程序中的信息只能经由函数的参数和结果传递，因而都暴露给类型系统。这一点对对象尤其重要，也迫使我们把子类型与继承留到第三十二章，等具备更强的类型论工具后再处理。

抽象数据类型

传统抽象数据类型（ADT）包含四部分：抽象类型名、具体表示类型、创建和操作该表示的若干操作，以及把实现同客户代码隔开的抽象边界。在边界内，抽象类型的元素按具体表示类型处理；边界外，它们只以抽象类型名出现，可以传递或存入数据结构，却不能被直接检查或修改，只能通过 ADT 公开的操作使用。

例如，下面用一种类似 Ada (U.S. Dept. of Defense, 1980) 或 CLU (Liskov 等, 1981) 的伪代码声明纯函数式计数器：

```
ADT counter =
  type Counter
  representation Nat
  signature
    new : Counter,
    get : Counter->Nat,
    inc : Counter->Counter;
  operations
    new = 1,
    get = lambda i:Nat. i,
    inc = lambda i:Nat. succ(i);

counter.get (counter.inc counter.new);
```

抽象类型名是 `Counter`，具体表示是 `Nat`。操作的实现具体地把计数器当作自然数：`new` 是常量 1，`inc` 是后继函数，`get` 是恒等函数。**signature** 部分规定外部如何使用这些操作，把其具体类型中的一些 `Nat` 换成抽象的 `Counter`。抽象边界从 **ADT** 延伸到结束声明的分号；最后一行已无法知道 `Counter` 实际就是 `Nat`，因此 `counter.new` 只能交给公开的 `get` 或 `inc`。

⁴公平地说，类型并不是保护程序员自定义抽象的唯一办法；在无类型语言中，函数闭包、对象以及 `MzScheme` 的 `unit` 等专门构造也能达到相似效果 (Flatt 与 Felleisen, 1998)。

这个伪代码几乎可以逐符号翻译为存在类型。首先把 ADT 内部打成包：

```
counterADT =
  {*Nat,
   {new = 1,
    get = lambda i:Nat. i,
    inc = lambda i:Nat. succ(i)}}
  as {Some Counter,
     {new: Counter,
      get: Counter->Nat,
      inc: Counter->Counter}}};
? counterADT :
  {Some Counter,
   {new:Counter,get:Counter->Nat,inc:Counter->Counter}}
```

然后打开包，引入代表隐藏表示类型的 Counter，并用变量 counter 访问操作：

```
let {Counter,counter} = counterADT in
counter.get (counter.inc counter.new);
? 2 : Nat
```

存在类型版本在视觉上比带句法糖的伪代码稍复杂，但二者结构完全相同。一般来说，打开包的 let 体会包含程序余下的全部内容：

```
let {Counter,counter} = <counter package> in
<rest of program>
```

在余下程序中，Counter 可以像内建基本类型一样使用。例如：

```
let {Counter,counter}=counterADT in
let add3 = lambda c:Counter.
    counter.inc (counter.inc (counter.inc c)) in
counter.get (add3 counter.new);
? 4 : Nat
```

甚至还能用已有的计数器表示另一个 ADT。下面用计数器作为内部表示，定义一个并不高效的触发器：

```
let {Counter,counter} = counterADT in

let {FlipFlop,flipflop} =
  {*Counter,
   {new = counter.new,
    read = lambda c:Counter. iseven (counter.get c),
    toggle = lambda c:Counter. counter.inc c,
    reset = lambda c:Counter. counter.new}}}
  as {Some FlipFlop,
     {new: FlipFlop, read: FlipFlop->Bool,
```

```

toggle: FlipFlop->FlipFlop,
reset: FlipFlop->FlipFlop}} in

flipflop.read (flipflop.toggle (flipflop.toggle flipflop.new));
? false : Bool

```

大型程序可以由一串这样的 ADT 声明组成；后一个 ADT 用前面的类型和操作实现自身，再把一个界面明确的抽象交给之后的程序。

这种信息隐藏的关键性质是表示独立性 (*representation independence*)。例如，可以把计数器的内部表示从一个自然数换成含自然数字段的记录，而接口完全不变：

```

counterADT =
  {*{x:Nat},
   {new = {x=1},
    get = lambda i:{x:Nat}. i.x,
    inc = lambda i:{x:Nat}. {x=succ(i.x)}}}
  as {Some Counter,
     {new: Counter, get: Counter->Nat,
      inc: Counter->Counter}}};
? counterADT :
  {Some Counter,
   {new:Counter,get:Counter->Nat,inc:Counter->Counter}}

```

整个程序仍然类型安全，因为外部只能经由 `get` 和 `inc` 访问计数器实例，无法依赖内部表示。

实践表明，ADT 风格能显著提高大型系统的健壮性和可维护性。第一，存在包的赋型规则限制了改动的影响范围：替换表示类型和全部操作实现，不会影响无法看到内部表示的其余程序。第二，它鼓励程序员缩小各部分之间的依赖，使 ADT 接口尽量精简。最后也最重要的是，显式写出操作接口会迫使程序员认真设计抽象。

练习 24.2.1 (推荐, ★★★). 参照上面的示例，以§23.4介绍的 `List Nat` 为表示，定义自然数栈 ADT，提供 `new`、`push`、`top`、`pop` 和 `isempty`。再写一个简单主程序：创建一个栈，压入几个自然数，并取得栈顶。本题最好在检查器中完成；使用 `fullomega`，并把 `test.f` 中的列表构造子及相关操作复制到输入文件开头。

[查看原书附录 A 中的 24.2.1 解答](#)

练习 24.2.2 (推荐, ★★). 用第十三章的引用单元构造可变计数器 ADT。令 `new : Unit → Counter`，并令 `inc` 返回 `Unit`。`fullomega` 除存在类型外也提供引用单元。

[查看原书附录 A 中的 24.2.2 解答](#)

存在对象

上一小节的“先打包、再立即开包”是用存在包编写 ADT 的典型方式：包定义抽象类型及其操作，随即打开包，把抽象类型绑定到类型变量，并把操作以抽象接口暴露出来。存

在类型还支持另一种编程方式，可以表达简单的对象式数据抽象；第三十二章会继续发展这一对象模型。

下面的例子既沿用本章前面的存在类型 ADT，也沿用第十八章和第十九章中的对象示例。这里仍以计数器为例，并继续采用纯函数式风格：向计数器发送 `inc` 消息不会就地修改其状态，而会返回一个内部状态已递增的新对象。计数器对象包含两类基本分量：内部状态，以及操作该状态的 `get` 和 `inc` 方法。为保证状态只能经由这两个方法查询或更新，需要把状态和方法封装在存在包中，并隐藏状态类型：

$$\text{Counter} = \{\exists X, \{\text{state} : X, \text{methods} : \{\text{get} : X \rightarrow \text{Nat}, \text{inc} : X \rightarrow X\}\}\}$$

例如，状态为 5 的计数器对象可以写成

```
c = {*Nat,
  {state = 5,
   methods = {get = lambda x:Nat. x,
              inc = lambda x:Nat. succ(x)}}}
as Counter;
```

使用对象方法时，先打开存在包，再把相应方法应用于状态。读取 `c` 得到：

```
let {X,body} = c in body.methods.get(body.state);
? 5 : Nat
```

一般地，可以定义一个向任意计数器“发送 `get` 消息”的函数：

$$\text{sendget} = \lambda c : \text{Counter}. \text{let } \{X, \text{body}\} = c \text{ in } \text{body.methods.get}(\text{body.state})$$

检查器给出 $\text{sendget} : \text{Counter} \rightarrow \text{Nat}$ 。

调用 `inc` 稍微复杂。如果照搬 `get` 的写法，

```
let {X,body} = c in body.methods.inc(body.state);
? Error: Scoping error!
```

类型检查器会拒绝它，因为 `let` 体的结果类型中自由出现了 `X`。从程序含义看，这个结果也不完整：它只是一份裸的内部状态，并不是对象。正确做法是把新状态重新打包成计数器对象，沿用原对象的表示类型和方法记录：

```
c1 = let {X,body} = c in
  {*X,
   {state = body.methods.inc(body.state),
    methods = body.methods}}
as Counter;
```

一般的 `sendinc` 函数正是这一操作：

$$\text{sendinc} = \lambda c : \text{Counter}. \text{let } \{X, \text{body}\} = c \text{ in } \{*X, \{\text{state} = \text{body.methods.inc}(\text{body.state}), \\ \text{methods} = \text{body.methods}\}\} \text{ as Counter}$$

检查器给出 $\text{sendinc} : \text{Counter} \rightarrow \text{Counter}$ 。以这两个基本操作为基础，还能定义更复杂的计数器操作：

```
add3 = lambda c:Counter. sendinc (sendinc (sendinc c));
? add3 : Counter -> Counter
```

练习 24.2.3 (推荐, ★★★). 以上面正文中的 FlipFlop ADT 为模型，以计数器对象作为内部表示，实现 FlipFlop 对象。

[查看原书附录 A 中的 24.2.3 解答](#)

练习 24.2.4 (推荐, ★★). 使用 fullomega，参照[练习24.2.2](#)实现带可变状态的计数器对象。

[查看原书附录 A 中的 24.2.4 解答](#)

对象与 ADT

上一小节的示例还不能算完整的面向对象模型：它缺少[第十八章](#)和[第十九章](#)中的子类型、类、继承以及经由 self 和 super 实现的递归。第三十二章会在类型系统具备必要的扩展后重新处理这些特性。不过，现在已经可以比较这种简单对象与前面的 ADT。

从最粗略的层面看，两种编程方式位于一条光谱的两端：ADT 的存在包一经构造便立即打开；表示对象的包则尽量保持封闭，直到必须把方法应用于内部状态时才打开。

由此，“计数器的抽象类型”在两种风格中所指不同。ADT 风格的客户代码（例如 add3）操作的是底层表示类型的元素；运行时的计数器值只是裸的自然数。对象风格中的每个计数器则是完整的包，既包含状态，也包含 get 和 inc 的实现。这一区别也反映在类型上：ADT 风格的 Counter 是开包时由 let 绑定的类型变量；对象风格的 Counter 则是完整的存在类型

$$\{\exists X, \{\text{state} : X, \text{methods} : \{\text{get} : X \rightarrow \text{Nat}, \text{inc} : X \rightarrow X\}\}\}$$

因此，一个 ADT 产生的所有计数器值共享同一种表示和同一套操作实现；每个对象却各自携带表示类型及适用于该表示的方法。

这给两种风格带来不同的实践优势。对象自行选择表示并携带操作，所以同一程序可以混用同一对象类型的多种实现；结合子类型和继承后，这一点尤其方便。可以先定义一般的对象类，再派生许多表示各异的细化类，而这些实例仍共享一般类型，能交给相同的通用代码处理或放入同一列表。

例如，用户界面库可以定义通用的 Window 类，再派生 TextWindow、ContainerWindow、ScrollableWindow、TitledWindow 和 DialogBox 等类。每个子类都可加入自己的实例变量：例如，TextWindow 可以用一个 String 型变量保存当前文本，ContainerWindow 则可以保存一系列 Window 对象。各子类还会为 repaint 和 handleMouseEvent 等操作提供专门实现。若把 Window 做成 ADT，则其统一表示通常要包含一个[第 11.10 节介绍的变体类型](#)，每种窗口对应一个分支并携带自己的数据；repaint 等操作再对变体作分支分析。窗口种类很多时，这个单体式 ADT 会变得庞大而难以维护。

另一个主要区别涉及二元操作，即接收两个或更多同一抽象类型参数的操作。这里需要区分两类。

弱二元操作 (*weak binary operation*) 只用两个抽象值的公开操作就能实现。例如, 比较两个计数器是否相等, 只需分别调用 `get`, 再比较得到的自然数; 这个相等操作完全可以放在保护具体表示的抽象边界之外。

强二元操作 (*strong binary operation*) 必须以受信任的方式具体访问两个抽象值的内部表示。例如, 若自然数集合用满足复杂不变量的带标签树表示, 高效的并集算法就需要把两个集合都当作这种树来查看。但公共接口不应暴露这项表示, 因此并集必须位于抽象边界之内, 对两个参数都拥有普通客户代码没有的访问权限。

弱二元操作对两种抽象风格都容易处理。放在边界外时, 它就是一个接收对象或 ADT 值的普通函数; 放进 ADT 时, 它虽然获得了并不需要的具体表示访问权, 也不会造成问题。把弱二元操作放进对象只稍微复杂一些, 因为对象类型会成为递归类型:⁵

$$\begin{aligned} \text{EqCounter} = \{ & \exists X, \{ \text{state} : X, \\ & \text{methods} : \{ \text{get} : X \rightarrow \text{Nat}, \text{inc} : X \rightarrow X, \\ & \text{eq} : X \rightarrow \text{EqCounter} \rightarrow \text{Bool} \} \} \} \end{aligned}$$

在本章的对象模型中, 强二元操作则无法作为对象方法表达。可以照样写出类型

$$\begin{aligned} \text{NatSet} = \{ & \exists X, \{ \text{state} : X, \\ & \text{methods} : \{ \text{empty} : X, \text{singleton} : \text{Nat} \rightarrow X, \\ & \text{member} : X \rightarrow \text{Nat} \rightarrow \text{Bool}, \\ & \text{union} : X \rightarrow \text{NatSet} \rightarrow X \} \} \}, \end{aligned}$$

但无法给出令人满意的实现: `union` 的第二个参数只公开了 `NatSet` 接口, 其中没有任何操作能列出集合元素, 算法也就无法计算并集。

练习 24.2.5 (★). 为什么不能改用下面的类型?

$$\begin{aligned} \text{NatSet} = \{ & \exists X, \{ \text{state} : X, \\ & \text{methods} : \{ \text{empty} : X, \text{singleton} : \text{Nat} \rightarrow X, \\ & \text{member} : X \rightarrow \text{Nat} \rightarrow \text{Bool}, \\ & \text{union} : X \rightarrow X \rightarrow X \} \} \} \end{aligned}$$

[查看原书附录 A 中的 24.2.5 解答](#)

总的说来, ADT 只有一种表示, 直接支持强二元操作; 对象允许多种表示, 以放弃强二元方法为代价换来实用的灵活性。两者优势互补, 没有哪一种全面优于另一种。

这里还需加一项限定: 上述比较针对的是本章前面所用的简单“纯对象”模型。C++ 和 Java 等主流面向对象语言的类允许某些强二元方法, 更适合看作纯对象与纯 ADT 的折中。在这些语言中, 对象类型就是实例所属类的名字, 即使两个类提供完全相同的操作, 类名不同, 类型也不同 (参见§19.3); 因而一个对象类型只有相应类声明给出的一种实现。此外, 子

⁵对象类型中的这种递归与继承之间存在一些微妙的相互作用, 参见 Bruce 等 (1996)。

类只能在继承来的实例变量基础上增加新变量。于是，类型为 C 的每个对象都必定包含类 C 唯一声明所规定的全部实例变量，并且可能还有更多变量。类 C 的方法便可以接收另一个 C 型对象，并只具体访问 C 所定义的实例变量，从而实现集合并集等强二元操作。Pierce 与 Turner (1993) 以及 Katiyar 等 (1994) 形式化了这种混合对象模型；Fisher 与 Mitchell (1996, 1998) 和 Fisher (1996a, 1996b) 作了更详细的发展。

24.3 用全称类型编码存在类型

§23.4 对序对的多态编码启发了一种类似做法：把存在包看成“一个隐藏类型和一个值的对”，并让这个数据主动执行自己的消去操作：

$$\llbracket \{\exists X, T\} \rrbracket \stackrel{\text{def}}{=} \forall Y. (\forall X. \llbracket T \rrbracket \rightarrow Y) \rightarrow Y$$

存在包在这种编码下接收一个结果类型和一个续延，再调用续延产生最终结果。续延接收两个参数：一个类型 X ，以及一个 T 型的值；然后用它们计算最终结果。

打包

$$\{\ast S, t\} \text{ as } \{\exists X, T\}$$

要编码这个包，必须用 S 和 t 构造一个类型为 $\forall Y. (\forall X. \llbracket T \rrbracket \rightarrow Y) \rightarrow Y$ 的值。这个类型先是全称量词，随后是箭头类型，所以编码应当从两层抽象开始：先抽象结果类型 Y ，再接收 $f : \forall X. \llbracket T \rrbracket \rightarrow Y$ ：

$$\llbracket \{\ast S, t\} \text{ as } \{\exists X, T\} \rrbracket \stackrel{\text{def}}{=} \lambda Y. \lambda f : (\forall X. \llbracket T \rrbracket \rightarrow Y). \dots$$

此时需要构造 Y 型结果，手中唯一能产生 Y 的是 f 。先把 f 实例化到 $\llbracket S \rrbracket$ ：

$$\llbracket \{\ast S, t\} \text{ as } \{\exists X, T\} \rrbracket \stackrel{\text{def}}{=} \lambda Y. \lambda f : (\forall X. \llbracket T \rrbracket \rightarrow Y). f[\llbracket S \rrbracket] \dots$$

此时 $f[\llbracket S \rrbracket]$ 的类型为 $(\llbracket X \mapsto \llbracket S \rrbracket \rrbracket \llbracket T \rrbracket) \rightarrow Y$ 型函数；再把类型恰好匹配的 $\llbracket t \rrbracket$ 交给它，就得到所需的 Y ：

$$\llbracket \{\ast S, t\} \text{ as } \{\exists X, T\} \rrbracket \stackrel{\text{def}}{=} \lambda Y. \lambda f : (\forall X. \llbracket T \rrbracket \rightarrow Y). f[\llbracket S \rrbracket][\llbracket t \rrbracket]$$

整个构造由目标类型逐步确定。

开包

$$\text{let } \{X, x\} = t_1 \text{ in } t_2$$

的翻译也由目标类型逐步确定。T-Unpack 告诉我们： t_1 具有某个存在类型 $\{\exists X, T_{11}\}$ ，在加入 X 和 $x : T_{11}$ 的上下文中， t_2 具有类型 T_2 ，而 T_2 也是整个开包表达式的预期类型。编码后的 t_1 会“自行完成开包”，因此只需向它提供足够的参数，使结果成为 $\llbracket T_2 \rrbracket$ 。编码先从 $\llbracket t_1 \rrbracket$ 开始：

$$\llbracket \text{let } \{X, x\} = t_1 \text{ in } t_2 \rrbracket \stackrel{\text{def}}{=} \llbracket t_1 \rrbracket \dots$$

先把整个表达式所需的结果类型 $\llbracket T_2 \rrbracket$ 作为第一个参数：

$$\llbracket \text{let } \{X, x\} = t_1 \text{ in } t_2 \rrbracket \stackrel{\text{def}}{=} \llbracket t_1 \rrbracket[\llbracket T_2 \rrbracket] \dots$$

此时 $\llbracket t_1 \rrbracket \llbracket T_2 \rrbracket$ 的类型为 $(\forall X. \llbracket T_{11} \rrbracket \rightarrow \llbracket T_2 \rrbracket) \rightarrow \llbracket T_2 \rrbracket$ 。再把开包体对 X 和 x 抽象，便得到所需的第二个参数：

$$\llbracket \text{let } \{X, x\} = t_1 \text{ in } t_2 \rrbracket \stackrel{\text{def}}{=} \llbracket t_1 \rrbracket \llbracket T_2 \rrbracket (\lambda X. \lambda x : \llbracket T_{11} \rrbracket. \llbracket t_2 \rrbracket)$$

第二个参数恰好具有类型 $\forall X. \llbracket T_{11} \rrbracket \rightarrow \llbracket T_2 \rrbracket$ ，于是整个应用得到 $\llbracket T_2 \rrbracket$ 。这就完成了编码。⁶

练习 24.3.1 (推荐, ★★★). 不查看正文，依据目标类型自行重新推导上述编码。

[查看练习 24.3.1 的译者参考解答](#)

练习 24.3.2 (★★★). 要确信这一编码正确，需要证明哪些性质？

[查看原书附录 A 中的 24.3.2 解答](#)

练习 24.3.3 (★★★). 能否反过来只用存在类型编码全称类型？

[查看原书附录 A 中的 24.3.3 解答](#)

24.4 注记

Mitchell 与 Plotkin (1988) 首先系统阐明了 ADT 与存在类型的对应，并注意到它同对象的联系。Pierce 与 Turner (1994) 详细发展了这一联系；更多内容 and 文献见第三十二章。Reynolds (1975)、Cook (1991)、Bruce 等 (1996) 以及许多其他研究都讨论过对象与 ADT 之间的取舍，其中 Bruce 等 (1996) 专门深入讨论了二元方法。

本章说明了存在类型如何为一种简单的抽象数据类型提供自然的类型论基础。ML 等语言的模块系统与此相关，但能力强得多，需要更复杂的机制。Cardelli 与 Leroy (1990)、Leroy (1994)、Harper 与 Lillibridge (1994)、Lillibridge (1997)、Harper 与 Stone (2000) 以及 Crary 等 (2002) 是这一领域较好的入门材料。

类型结构是一种句法纪律，用来强制维持不同的抽象层次。

Type structure is a syntactic discipline for enforcing levels of abstraction.

——John Reynolds, 1983

⁶严格说来，翻译需要从赋型推导取得项句法中没有的 T_{11} 和 T_2 ，所以它翻译的是赋型推导，而不只是项。我们在 §15.6 定义子类型的强制转换语义时见过相似情形。

第二十五章 System F 的 ML 实现

本章以简单类型 lambda 演算的实现为基础（见第十章），加入全称类型与存在类型；这两种系统分别见第二十三章与第二十四章。这个系统的规则与简单类型 lambda 演算一样是语法导向的，不像含子类型或等式递归类型的演算，因此 OCaml 实现相当直接。主要的新问题是如何表示带有变量绑定的类型；这里再次采用第六章介绍的 de Bruijn 索引。

25.1 类型的无名表示

类型语法加入变量、全称量词和存在量词：

```
type ty =  
  TyVar of int * int  
| TyArr of ty * ty  
| TyAll of string * ty  
| TySome of string * ty
```

Rust 对照实现（译者增补）

```
#[derive(Clone, Debug, PartialEq, Eq)]  
pub enum Type {  
  Var(usize, usize),  
  Arrow(Box<Type>, Box<Type>),  
  All(String, Box<Type>),  
  Some(String, Box<Type>),  
}
```

TyVar(x, n) 的两个整数与§7.1中项变量的表示相同： x 是该变量到绑定子的距离， n 是变量出现处预期的上下文总长度。量词保存一个名称提示，只供解析器和打印器恢复可读名称。

上下文绑定增加 TyVarBind：

```

type binding =
    NameBind
  | VarBind of ty
  | TyVarBind

```

Rust 对照实现 (译者增补)

```

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Binding {
    Name,
    Variable(Type),
    TypeVariable,
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Error {
    InvalidShift,
    UnboundVariable(usize),
    ExpectedArrow(Type),
    ExpectedUniversal(Type),
    ExpectedExistential(Type),
    ParameterMismatch { expected: Type, actual: Type },
    PackageMismatch { expected: Type, actual: Type },
    NoRuleApplies,
}

fn shifted_index(index: usize, distance: isize) -> Result<usize, Error> {
    index
        .checked_add_signed(distance)
        .ok_or(Error::InvalidShift)
}

```

NameBind 仍只供解析和打印使用；VarBind 携带项变量的类型；TyVarBind 不附加数据，因为纯 System F 的类型变量没有界或种类等额外假设。第二十六章的有界量化和第二十九章的高阶种类会改变这一点。

25.2 类型移位与替换

类型现在含有变量，因此需要定义类型移位 (*type shifting*) 和替换。

练习 25.2.1 (★). 仿照定义 6.2.1 的项移位，给出类型变量移位的数学定义。

查看原书附录 A 中的 25.2.1 解答

§7.2把项的移位与替换分别定义成两个函数，同时提到作者网站上的实现其实用一个通用“映射”函数完成两项工作。类型也可以用同样方法。移位与替换只在变量叶子处行为不同，其余类型构造都只是递归遍历。先把练习 25.2.1 的定义机械翻译成专用移位函数：

```
let typeShiftAbove d c tyT =
  let rec walk c tyT = match tyT with
    | TyVar(x,n) ->
      if x>=c then TyVar(x+d,n+d) else TyVar(x,n+d)
    | TyArr(tyT1,tyT2) ->
      TyArr(walk c tyT1,walk c tyT2)
    | TyAll(tyX,tyT2) ->
      TyAll(tyX,walk (c+1) tyT2)
    | TySome(tyX,tyT2) ->
      TySome(tyX,walk (c+1) tyT2)
  in walk c tyT
```

译者注：相应的 Rust 移位实现与替换操作合并展示。

d 是自由变量的移动距离， c 是截止位置；进入量词体时把 c 加一，从而不移动该量词绑定的变量。把变量分支抽成参数后，得到通用映射器：

```
let tymap onvar c tyT =
  let rec walk c tyT = match tyT with
    | TyArr(tyT1,tyT2) ->
      TyArr(walk c tyT1,walk c tyT2)
    | TyVar(x,n) -> onvar c x n
    | TyAll(tyX,tyT2) ->
      TyAll(tyX,walk (c+1) tyT2)
    | TySome(tyX,tyT2) ->
      TySome(tyX,walk (c+1) tyT2)
  in walk c tyT
```

译者注：Rust 对照直接以递归辅助函数表达同一遍历；两种写法共享同一不变量：经过量词时截止位置加一，只在变量分支决定移位或替换。

移位、替换和顶层替换由这个映射器实例化：

```
let typeShiftAbove d c tyT =
  tymap
    (fun c x n ->
      if x>=c then TyVar(x+d,n+d) else TyVar(x,n+d))
    c tyT

let typeShift d tyT = typeShiftAbove d 0 tyT
```

Rust 对照实现 (译者增补)

```
/// 将每个自由类型变量移动 `distance`，但不改变索引小于 `cutoff` 的变量；
/// 每个变量节点中记录的上下文长度也要相应移动。
pub fn type_shift_above(distance: isize, cutoff: usize, ty: &Type) -> Result<Type,
↳ Error> {
  fn walk(distance: isize, cutoff: usize, ty: &Type) -> Result<Type, Error> {
    Ok(match ty {
      Type::Var(index, context_len) => Type::Var(
        if *index >= cutoff {
          shifted_index(*index, distance)?
        } else {
          *index
        },
        shifted_index(*context_len, distance)?,
      ),
      Type::Arrow(parameter, result) => Type::Arrow(
        Box::new(walk(distance, cutoff, parameter)?),
        Box::new(walk(distance, cutoff, result)?),
      ),
      Type::All(name, body) => {
        Type::All(name.clone(), Box::new(walk(distance, cutoff + 1,
↳ body)?))
      },
      Type::Some(name, body) => {
        Type::Some(name.clone(), Box::new(walk(distance, cutoff + 1,
↳ body)?))
      },
    })
  }
  walk(distance, cutoff, ty)
}

pub fn type_shift(distance: isize, ty: &Type) -> Result<Type, Error> {
```

```

    type_shift_above(distance, 0, ty)
}

```

```

let typeSubst tyS j tyT =
  tmap
    (fun j x n ->
      if x=j then typeShift j tyS else TyVar(x,n))
    j tyT

let typeSubstTop tyS tyT =
  typeShift (-1) (typeSubst (typeShift 1 tyS) 0 tyT)

```

Rust 对照实现 (译者增补)

```

/// 用 `replacement` 替换类型变量 `variable`; 遍历进入量词时,
/// 目标变量与替换类型一同上移。
pub fn type_substitute(replacement: &Type, variable: usize, ty: &Type) ->
  ⇨ Result<Type, Error> {
  fn walk(replacement: &Type, variable: usize, cutoff: usize, ty: &Type) ->
  ⇨ Result<Type, Error> {
    Ok(match ty {
      Type::Var(index, _) if *index == variable + cutoff => type_shift(
        isize::try_from(variable + cutoff).map_err(|_|
⇨ Error::InvalidShift)?,
        replacement,
      )?,
      Type::Var(index, context_len) => Type::Var(*index, *context_len),
      Type::Arrow(parameter, result) => Type::Arrow(
        Box::new(walk(replacement, variable, cutoff, parameter)?),
        Box::new(walk(replacement, variable, cutoff, result)?),
      ),
      Type::All(name, body) => Type::All(
        name.clone(),
        Box::new(walk(replacement, variable, cutoff + 1, body)?),
      ),
      Type::Some(name, body) => Type::Some(
        name.clone(),
        Box::new(walk(replacement, variable, cutoff + 1, body)?),
      ),
    })
  }
  walk(replacement, variable, 0, ty)
}

```

```

/// 打开最外层的类型绑定：先上移替换类型，再替换变量 0，
/// 最后从其余索引中移除这层绑定。
pub fn type_substitute_top(replacement: &Type, body: &Type) -> Result<Type, Error>
↪ {
    type_shift(-1, &type_substitute(&type_shift(1, replacement)?, 0, body)?)
}

```

`typeSubstTop` 的三步与项替换相同：先把替换类型上移一位，使它进入绑定子时不会捕获变量；替换编号 0；最后整体下移一位，删除已经打开的绑定子。

25.3 项

在项这一层，要做的工作与类型部分相似。首先在第十章的项数据类型中加入全称类型和存在类型的引入、消去形式：

```

type term =
    TmVar of info * int * int
  | TmAbs of info * string * ty * term
  | TmApp of info * term * term
  | TmTAbs of info * string * term
  | TmTApp of info * term * ty
  | TmPack of info * ty * term * ty
  | TmUnpack of info * string * string * term * term

```

Rust 对照实现（译者增补）

```

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    Var(usize, usize),
    Abs(String, Type, Box<Term>),
    App(Box<Term>, Box<Term>),
    TypeAbs(String, Box<Term>),
    TypeApp(Box<Term>, Type),
    Pack(Type, Box<Term>, Type),
    Unpack(String, String, Box<Term>, Box<Term>),
}

```

项的移位与替换同第十章的定义相似。不过，与上一节处理类型时一样，这里也用了一个通用映射函数来统一表述这些操作：


```

let tmmmap onvar ontype c t =
  let rec walk c t = match t with
    | TmVar(fi,x,n) -> onvar fi c x n
    | TmAbs(fi,x,tyT1,t2) ->
      TmAbs(fi,x,ontype c tyT1,walk (c+1) t2)
    | TmApp(fi,t1,t2) ->
      TmApp(fi,walk c t1,walk c t2)
    | TmTAbs(fi,tyX,t2) ->
      TmTAbs(fi,tyX,walk (c+1) t2)
    | TmTApp(fi,t1,tyT2) ->
      TmTApp(fi,walk c t1,ontype c tyT2)
    | TmPack(fi,tyT1,t2,tyT3) ->
      TmPack(fi,ontype c tyT1,walk c t2,ontype c tyT3)
    | TmUnpack(fi,tyX,x,t1,t2) ->
      TmUnpack(fi,tyX,x,walk c t1,walk (c+2) t2)
  in walk c t

```

译者注：相应的 Rust 遍历实现与移位、替换操作合并展示。

`tmmmap` 有四个参数，比 `tymap` 多一个。这是因为项中可能出现两类变量：项变量，以及嵌在类型标注中的类型变量。因此，在执行移位等操作时，有两类“叶子”可能真正需要处理：项变量与类型。`onvar` 指定如何处理项变量；`ontype` 则指定遍历到带类型标注的项构造（例如 `TmAbs`）时，如何处理其中的类型。在规模更大的语言中，带类型标注的项构造还会更多。

`TmUnpack` 的开包表达式体同时位于类型变量 X 和项变量 x 两个新绑定之下，所以遍历进入其中时，截止位置增加二。给 `tmmmap` 传入适当的参数，就能定义项移位、项替换和把类型替换进项的操作：

```

let termShiftAbove d c t =
  tmmmap
    (fun fi c x n ->
      if x >= c then TmVar(fi,x+d,n+d) else TmVar(fi,x,n+d))
    (typeShiftAbove d)
    c t

let termShift d t = termShiftAbove d 0 t

let termSubst j s t =
  tmmmap
    (fun fi j x n ->
      if x=j then termShift j s else TmVar(fi,x,n))
    (fun j tyT -> tyT)
    j t

```

```

let rec tytermSubst tyS j t =
  tmmmap
    (fun fi c x n -> TmVar(fi,x,n))
    (fun j tyT -> typeSubst tyS j tyT)
  j t

let termSubstTop s t =
  termShift (-1) (termSubst 0 (termShift 1 s) t)

let tytermSubstTop tyS t =
  termShift (-1) (tytermSubst (typeShift 1 tyS) 0 t)

```

Rust 对照实现 (译者增补)

```

/// 同时移动项变量和类型标注中的类型变量，因为本实现把两类绑定存放在同一上下文中。
pub fn term_shift_above(distance: isize, cutoff: usize, term: &Term) ->
↳ Result<Term, Error> {
  fn walk(distance: isize, cutoff: usize, term: &Term) -> Result<Term, Error> {
    Ok(match term {
      Term::Var(index, context_len) => Term::Var(
        if *index >= cutoff {
          shifted_index(*index, distance)?
        } else {
          *index
        },
        shifted_index(*context_len, distance)?,
      ),
      Term::Abs(name, parameter, body) => Term::Abs(
        name.clone(),
        type_shift_above(distance, cutoff, parameter)?,
        Box::new(walk(distance, cutoff + 1, body)?),
      ),
      Term::App(function, argument) => Term::App(
        Box::new(walk(distance, cutoff, function)?),
        Box::new(walk(distance, cutoff, argument)?),
      ),
      Term::TypeAbs(name, body) => {
        Term::TypeAbs(name.clone(), Box::new(walk(distance, cutoff + 1,
↳ body)?))
      }
      Term::TypeApp(function, argument) => Term::TypeApp(
        Box::new(walk(distance, cutoff, function)?),
        type_shift_above(distance, cutoff, argument)?,

```

```

    ),
    Term::Pack(hidden, value, package) => Term::Pack(
        type_shift_above(distance, cutoff, hidden)?,
        Box::new(walk(distance, cutoff, value)?),
        type_shift_above(distance, cutoff, package)?,
    ),
    Term::Unpack(type_name, name, package, body) => Term::Unpack(
        type_name.clone(),
        name.clone(),
        Box::new(walk(distance, cutoff, package)?),
        Box::new(walk(distance, cutoff + 2, body)?),
    ),
    })
}
walk(distance, cutoff, term)
}

pub fn term_shift(distance: isize, term: &Term) -> Result<Term, Error> {
    term_shift_above(distance, 0, term)
}

/// 替换一个自由项变量；每当遍历进入项绑定或类型绑定时，先上移替换项，
/// 以免其中的自由变量被捕获。
pub fn term_substitute(replacement: &Term, variable: usize, term: &Term) ->
    Result<Term, Error> {
    fn walk(
        replacement: &Term,
        variable: usize,
        cutoff: usize,
        term: &Term,
    ) -> Result<Term, Error> {
        Ok(match term {
            Term::Var(index, _) if *index == variable + cutoff => term_shift(
                isize::try_from(variable + cutoff).map_err(|_|
                    Error::InvalidShift)?,
                replacement,
            )?,
            Term::Var(index, context_len) => Term::Var(*index, *context_len),
            Term::Abs(name, parameter, body) => Term::Abs(
                name.clone(),
                parameter.clone(),
                Box::new(walk(replacement, variable, cutoff + 1, body)?),
            ),
            Term::App(function, argument) => Term::App(

```

```

        Box::new(walk(replacement, variable, cutoff, function)?),
        Box::new(walk(replacement, variable, cutoff, argument)?),
    ),
    Term::TypeAbs(name, body) => Term::TypeAbs(
        name.clone(),
        Box::new(walk(replacement, variable, cutoff + 1, body)?),
    ),
    Term::TypeApp(function, argument) => Term::TypeApp(
        Box::new(walk(replacement, variable, cutoff, function)?),
        argument.clone(),
    ),
    Term::Pack(hidden, value, package) => Term::Pack(
        hidden.clone(),
        Box::new(walk(replacement, variable, cutoff, value)?),
        package.clone(),
    ),
    Term::Unpack(type_name, name, package, body) => Term::Unpack(
        type_name.clone(),
        name.clone(),
        Box::new(walk(replacement, variable, cutoff, package)?),
        Box::new(walk(replacement, variable, cutoff + 2, body)?),
    ),
    })
}
walk(replacement, variable, 0, term)
}

pub fn term_substitute_top(replacement: &Term, body: &Term) -> Result<Term, Error>
↳ {
    term_shift(-1, &term_substitute(&term_shift(1, replacement)?, 0, body)?)
}

/// 在项包含的所有类型标注中替换类型变量。项变量保持不变；
/// 进入类型绑定时增加 `cutoff`。
pub fn type_term_substitute(
    replacement: &Type,
    variable: usize,
    term: &Term,
) -> Result<Term, Error> {
    fn walk(
        replacement: &Type,
        variable: usize,
        cutoff: usize,
        term: &Term,

```

```

) -> Result<Term, Error> {
  Ok(match term {
    Term::Var(index, context_len) => Term::Var(*index, *context_len),
    Term::Abs(name, parameter, body) => Term::Abs(
      name.clone(),
      type_substitute(replacement, variable + cutoff, parameter)?,
      Box::new(walk(replacement, variable, cutoff + 1, body)?),
    ),
    Term::App(function, argument) => Term::App(
      Box::new(walk(replacement, variable, cutoff, function)?),
      Box::new(walk(replacement, variable, cutoff, argument)?),
    ),
    Term::TypeAbs(name, body) => Term::TypeAbs(
      name.clone(),
      Box::new(walk(replacement, variable, cutoff + 1, body)?),
    ),
    Term::TypeApp(function, argument) => Term::TypeApp(
      Box::new(walk(replacement, variable, cutoff, function)?),
      type_substitute(replacement, variable + cutoff, argument)?,
    ),
    Term::Pack(hidden, value, package) => Term::Pack(
      type_substitute(replacement, variable + cutoff, hidden)?,
      Box::new(walk(replacement, variable, cutoff, value)?),
      type_substitute(replacement, variable + cutoff, package)?,
    ),
    Term::Unpack(type_name, name, package, body) => Term::Unpack(
      type_name.clone(),
      name.clone(),
      Box::new(walk(replacement, variable, cutoff, package)?),
      Box::new(walk(replacement, variable, cutoff + 2, body)?),
    ),
  })
}

walk(replacement, variable, 0, term)
}

pub fn type_term_substitute_top(replacement: &Type, body: &Term) -> Result<Term,
↳ Error> {
  term_shift(
    -1,
    &type_term_substitute(&type_shift(1, replacement)?, 0, body)?,
  )
}

```

在 `termShiftAbove` 的项变量分支中，程序像 `typeShiftAbove` 一样检查截止位置，再构造新的变量；在类型标注处，则调用上一节定义的类型移位函数。

`termSubst` 用于把一个项代入另一个项。它不改变类型标注，因为类型中不可能含有项变量，项替换自然不会影响类型。另一方面，求值类型应用还需要把类型代入项：

这里第二种替换正是类型应用归约所需的操作：

$$(\lambda X.t_{12})[T_2] \longrightarrow [X \mapsto T_2]t_{12} \quad \text{E-TAPP} \text{ TABS}$$

`tytermSubst` 仍由项映射函数定义。这一次，处理项变量的回调是恒等操作，只重建原来的项变量；遍历到类型标注时，才对它执行类型替换。最后，与类型部分一样，`termSubstTop` 和 `tytermSubstTop` 把基本替换操作封装成两个供求值器使用的便捷函数；类型检查器使用的顶层类型替换则是上一节的 `typeSubstTop`。

25.4 求值

对 `eval` 函数的扩展，就是把图23-1和24-1中的求值规则逐条写成代码；其中最费力的移位与替换工作，已经由上一节的函数完成：

```
let rec eval1 ctx t = match t with
  (* ... *)
  | TmTApp(fi, TmTAbs(_, x, t11), tyT2) ->
    tytermSubstTop tyT2 t11
  | TmTApp(fi, t1, tyT2) ->
    let t1' = eval1 ctx t1 in
    TmTApp(fi, t1', tyT2)
  | TmUnpack(fi, _, _, TmPack(_, tyT11, v12, _), t2)
    when isval ctx v12 ->
    tytermSubstTop tyT11
    (termSubstTop (termShift 1 v12) t2)
  | TmUnpack(fi, tyX, x, t1, t2) ->
    let t1' = eval1 ctx t1 in
    TmUnpack(fi, tyX, x, t1', t2)
  | TmPack(fi, tyT1, t2, tyT3) ->
    let t2' = eval1 ctx t2 in
    TmPack(fi, tyT1, t2', tyT3)
  (* ... *)
```

Rust 对照实现 (译者增补)

```
fn is_value(term: &Term) -> bool {
  match term {
    Term::Abs(..) | Term::TypeAbs(..) => true,
    Term::Pack(_, value, _) => is_value(value),
```

```

    _ => false,
  }
}

/// 按值调用, 对 System F 构造执行一次单步求值。
pub fn evaluate_one(term: &Term) -> Result<Term, Error> {
  Ok(match term {
    Term::App(function, argument) if is_value(function) && is_value(argument)
    ↪ => {
      if let Term::Abs(_, _, body) = function.as_ref() {
        term_substitute_top(argument, body)?
      } else {
        return Err(Error::NoRuleApplies);
      }
    }
    Term::App(function, argument) if is_value(function) => {
      Term::App(function.clone(), Box::new(evaluate_one(argument)?))
    }
    Term::App(function, argument) => {
      Term::App(Box::new(evaluate_one(function)?), argument.clone())
    }
    Term::TypeApp(function, argument) => {
      if let Term::TypeAbs(_, body) = function.as_ref() {
        type_term_substitute_top(argument, body)?
      } else {
        Term::TypeApp(Box::new(evaluate_one(function)?), argument.clone())
      }
    }
    Term::Unpack(type_name, name, package, body) => {
      if let Term::Pack(hidden, value, _) = package.as_ref() {
        if is_value(value) {
          // 这个值原本位于 X、x 两层绑定之外。替换 x 前先将它上移一层;
          // 随后打开 X 时, 再移除余下的类型绑定位置。
          let with_value = term_substitute_top(&term_shift(1, value)?,
          ↪ body)?;
          type_term_substitute_top(hidden, &with_value)?
        } else {
          Term::Unpack(
            type_name.clone(),
            name.clone(),
            Box::new(evaluate_one(package)?),
            body.clone(),
          )
        }
      }
    }
  })
}

```



```

    } else {
      Term::Unpack(
        type_name.clone(),
        name.clone(),
        Box::new(evaluate_one(package)?),
        body.clone(),
      )
    }
  }
  Term::Pack(hidden, value, package) => Term::Pack(
    hidden.clone(),
    Box::new(evaluate_one(value)?),
    package.clone(),
  ),
  _ => return Err(Error::NoRuleApplies),
})
}

```

练习 25.4.1 (*). 第一个 TmUnpack 分支中, 为什么在 termSubstTop 前还要执行 termShift 1 v12?

[查看原书附录 A 中的 25.4.1 解答](#)

25.5 赋型

类型检查器的新分支同样直接来自赋型规则。下面保留相关上下文, 展示普通抽象与类型抽象、普通应用与类型应用之间的对应。

```

let rec typeof ctx t = match t with
| TmVar(fi,i,_) -> getTypeFromContext fi ctx i
| TmAbs(fi,x,tyT1,t2) ->
  let ctx' = addbinding ctx x (VarBind tyT1) in
  let tyT2 = typeof ctx' t2 in
  TyArr(tyT1,typeShift (-1) tyT2)
| TmApp(fi,t1,t2) ->
  let tyT1 = typeof ctx t1 in
  let tyT2 = typeof ctx t2 in
  (match tyT1 with
  | TyArr(tyT11,tyT12) ->
    if (=) tyT2 tyT11 then tyT12
    else error fi "parameter type mismatch"
  | _ -> error fi "arrow type expected")

```

```

| TmTAbs(fi,tyX,t2) ->
    let ctx = addbinding ctx tyX TyVarBind in
    let tyT2 = typeof ctx t2 in
    TyAll(tyX,tyT2)
| TmTApp(fi,t1,tyT2) ->
    let tyT1 = typeof ctx t1 in
    (match tyT1 with
     TyAll(_,tyT12) -> typeSubstTop tyT2 tyT12
     | _ -> error fi "universal type expected")
| TmPack(fi,tyT1,t2,tyT) ->
    (match tyT with
     TySome(tyY,tyT2) ->
        let tyU = typeof ctx t2 in
        let tyU' = typeSubstTop tyT1 tyT2 in
        if (==) tyU tyU' then tyT
        else error fi "doesn't match declared type"
     | _ -> error fi "existential type expected")
| TmUnpack(fi,tyX,x,t1,t2) ->
    let tyT1 = typeof ctx t1 in
    (match tyT1 with
     TySome(tyY,tyT11) ->
        let ctx' = addbinding ctx tyX TyVarBind in
        let ctx'' = addbinding ctx' x (VarBind tyT11) in
        let tyT2 = typeof ctx'' t2 in
        typeShift (-2) tyT2
     | _ -> error fi "existential type expected")

```

Rust 对照实现 (译者增补)

```

fn binding_type(context: &[Binding], index: usize) -> Result<Type, Error> {
    let binding = context
        .iter()
        .rev()
        .nth(index)
        .ok_or(Error::UnboundVariable(index))?;
    match binding {
        Binding::Variable(ty) => type_shift(
            isize::try_from(index + 1).map_err(|_| Error::InvalidShift)?,
            ty,
        ),
        _ => Err(Error::UnboundVariable(index)),
    }
}

```

```

/// 实现本章 System F 构造的全部类型规则。
/// T-Unpack 最后的下移会拒绝仍含有隐藏类型 X 的结果类型。
pub fn type_of(context: &[Binding], term: &Term) -> Result<Type, Error> {
  match term {
    Term::Var(index, _) => binding_type(context, *index),
    Term::Abs(_, parameter, body) => {
      let mut extended = context.to_vec();
      extended.push(Binding::Variable(parameter.clone()));
      Ok(Type::Arrow(
        Box::new(parameter.clone()),
        Box::new(type_shift(-1, &type_of(&extended, body)?)),
      ))
    }
    Term::App(function, argument) => {
      let function_type = type_of(context, function)?;
      let argument_type = type_of(context, argument)?;
      match function_type {
        Type::Arrow(parameter, result) if *parameter == argument_type =>
        ↪ Ok(*result),
        Type::Arrow(parameter, _) => Err(Error::ParameterMismatch {
          expected: *parameter,
          actual: argument_type,
        }),
        other => Err(Error::ExpectedArrow(other)),
      }
    }
    Term::TypeAbs(name, body) => {
      let mut extended = context.to_vec();
      extended.push(Binding::TypeVariable);
      Ok(Type::All(name.clone(), Box::new(type_of(&extended, body)?)))
    }
    Term::TypeApp(function, argument) => match type_of(context, function)? {
      Type::All(_, body) => type_substitute_top(argument, &body),
      other => Err(Error::ExpectedUniversal(other)),
    },
    Term::Pack(hidden, value, package) => match package {
      Type::Some(_, body) => {
        let expected = type_substitute_top(hidden, body)?;
        let actual = type_of(context, value)?;
        if expected == actual {
          Ok(package.clone())
        } else {
          Err(Error::PackageMismatch { expected, actual })
        }
      }
    }
  }
}

```

```

    }
    other => Err(Error::ExpectedExistential(other.clone())),
  },
  Term::Unpack(_, _, package, body) => match type_of(context, package)? {
    Type::Some(_, packed_body) => {
      let mut extended = context.to_vec();
      extended.push(Binding::TypeVariable);
      extended.push(Binding::Variable(*packed_body));
      type_shift(-2, &type_of(&extended, body)?)
    }
    other => Err(Error::ExpectedExistential(other)),
  },
}
}
}

```

最值得注意的新分支是 `TmUnpack`。它依次完成以下步骤：

1. 检查子表达式 t_1 ，确认它具有存在类型 $\{\exists X, T_{11}\}$ ；
2. 用类型变量绑定 X 和项变量绑定 $x : T_{11}$ 扩展上下文 Γ ，并检查 t_2 具有某个类型 T_2 ；
3. 把 T_2 中自由变量的索引下移二位，使它重新相对于原上下文 Γ 有意义；
4. 返回所得类型，作为整个 `let...in...` 表达式的类型。

显然，如果 T_2 中仍自由出现隐藏变量 X ，第 3 步的下移就会产生含有负索引的无意义类型；实现应在此处报告作用域错误，而不能返回这样的类型。

```

let typeShiftAbove d c tyT =
  tormap
    (fun c x n ->
      if x >= c then
        if x + d < 0 then err "Scoping error!"
        else TyVar(x + d, n + d)
      else TyVar(x, n + d))
  c tyT

```

译者注：Rust 对照以无符号索引和 `checked_add_signed` 实现同一检查；任何会产生负索引的下移都会返回 `Error::InvalidShift`。

只要开包表达式体的类型中仍含有它所绑定的隐藏类型变量 X ，这项检查就会报告作用域错误。例如：

```

let {X,x} = ({*Nat,0} as {Some X,X}) in x;
? Error: Scoping error!

```

第二十六章 有界量化

程序语言中许多值得深入研究的问题，都来自一些单独考察时相对直接的特性之间的相互作用。本章把多态与子类型结合，得到 有界量化 (*bounded quantification*) 以及演算 $F_{<}$ (*System F-sub*)。它既显著增强表达能力，也使元理论更复杂。 $F_{<}$ 形成于 20 世纪 80 年代中期，此后一直在程序语言研究中占据核心地位，尤其是在面向对象编程的理论基础研究中。

26.1 动机

把子类型和多态组合起来，最简单的办法是把它们看作彼此正交的特性，直接合并第十五章和第二十三章的系统。这样的系统在理论上没有特别困难，也同时拥有两项特性各自的用途。但二者出现在同一语言中后，自然会想把它们结合得更紧密。下面先看一个很小的例子；§26.3 还有更多示例，第二十七章和第三十二章则会给出更大的实际案例。

设 f 是带自然数字段 a 的记录上的恒等函数：

```
f = lambda x:{a:Nat}. x;  
? f : {a:Nat} -> {a:Nat}
```

若 r_a 只有一个 a 字段，

```
ra = {a=0};  
f ra;  
? {a=0} : {a:Nat}
```

把它传给 f 会得到同类型的记录。再定义一个含 a 、 b 两个字段的更大记录：

```
rab = {a=0, b=true};  
f rab;  
? {a=0, b=true} : {a:Nat}
```

T-Sub 会把 r_{ab} 的类型提升为 $\{a : \text{Nat}\}$ ，以匹配 f 的参数类型；但应用结果的静态类型也只剩字段 a ，所以 $(f\ r_{ab}).b$ 无法通过类型检查。也就是说，仅仅让 r_{ab} 经过恒等函数，程序就失去了访问字段 b 的能力。

System F 的多态性可以避免这项损失：

```

fpoly = lambda X. lambda x:X. x;
? fpoly : All X. X -> X

fpoly [{a:Nat, b:Bool}] rab;
? {a=0, b=true} : {a:Nat, b:Bool}

```

然而，把 x 的类型改成变量，也会丢掉函数体可能需要的信息。假设另一个函数既返回原参数，又计算其 a 字段的后继：

```

f2 = lambda x:{a:Nat}. {orig=x, asucc=succ(x.a)};
? f2 : {a:Nat} -> {orig:{a:Nat}, asucc:Nat}

f2 ra;
? {orig={a=0}, asucc=1} : {orig:{a:Nat}, asucc:Nat}

f2 rab;
? {orig={a=0,b=true}, asucc=1} : {orig:{a:Nat}, asucc:Nat}

```

子类型让两个调用都合法，却再次让第二个结果丢掉字段 b 的静态信息。这次普通多态也无济于事：

```

f2poly = lambda X. lambda x:X. {orig=x, asucc=succ(x.a)};
? Error: Expected record type

```

若把 x 的类型只写成未知的 X ，检查器便不知道它必定含有 a 字段。

我们真正想在类型中表达的是：函数接收任意含自然数字段 a 的记录类型 R ，结果同时含一个 R 型字段和一个自然数字段。借助子类型关系，可以说 R 是 $\{a : \text{Nat}\}$ 的任意子类型。把这项约束写进类型变量的绑定，得到

$$\text{f2poly} = \lambda X <: \{a : \text{Nat}\}. \lambda x : X. \\ \{\text{orig} = x, \text{asucc} = \text{succ}(x.a)\}$$

其类型为

$$\forall X <: \{a : \text{Nat}\}. X \rightarrow \{\text{orig} : X, \text{asucc} : \text{Nat}\}$$

量词上的上界既保证函数体可以安全使用 a ，又保留实参的精确类型 X 。这种在量词上附加子类型约束的形式，就是 System $F_{<}$ 的标志性特征。¹

26.2 定义

$F_{<}$ 把 System F 的项和类型同第十五章的子类型关系结合，并给全称量词加上界。比较两个有界全称类型有两种规则：限制较强、理论较易处理的核规则 (*kernel rule*)，以及表

¹本章主要研究纯 $F_{<}$ (图26-1)；示例还使用记录 (图11-7) 和自然数 (图8-2)。原书对应的 OCaml 实现是 `fullfsub` 和 `fullfomsub`。多数示例用 `fullfsub` 即可；涉及带参数的类型缩写 (例如 `Pair`) 时，需要 `fullfomsub`。

达能力更强但算法性质更棘手的 完全规则 (*full rule*)。相应系统分别称核 $F_{<}$ 和完全 $F_{<}$ 。有界存在量词也可以用同样方式定义，§26.5 将给出相应规则。

有界与无界量化

语法只保留有界量词

$$\forall X <: T_1. T_2 \quad \text{和} \quad \lambda X <: T_1. t_2$$

上界为 **Top** 时，量词变量遍历 **Top** 的所有子类型，也就是所有类型。因此，无界量化可以作为以下缩写恢复：

$$\forall X. T \stackrel{\text{def}}{=} \forall X <: \text{Top}. T$$

后文会经常使用这个缩写。

作用域

图26-1本身没有显示一个重要的技术细节：类型变量的作用域。讨论 $\Gamma \vdash t : T$ 时，当然要求 t 和 T 的自由类型变量都在 Γ 中绑定；上下文内各项所含的自由类型变量也要遵守作用域。比较下面四个上下文：

$$\begin{aligned} \Gamma_1 &= X <: \text{Top}, y : X \rightarrow \text{Nat}, \quad \Gamma_2 = y : X \rightarrow \text{Nat}, X <: \text{Top}, \\ \Gamma_3 &= X <: \{a : \text{Nat}, b : X\}, \quad \Gamma_4 = X <: \{a : \text{Nat}, b : Y\}, Y <: \{c : \text{Bool}, d : X\} \end{aligned}$$

Γ_1 显然良构：先引入 X ，后面的项变量类型才使用它；它可来自 $\lambda X <: \text{Top}. \lambda y : X \rightarrow \text{Nat}. t$ 这样的项。 Γ_2 则把使用放在绑定之前，无法说明 y 的类型中 X 的作用域，所以不良构。

Γ_3 更微妙。若允许 X 的绑定作用域覆盖它自己的上界，便可以把它视为良构；这种系统称为 F-有界量化 (*F-bounded quantification*) (Canning、Cook、Hill、Olthoff 与 Mitchell, 1989b)，常见于面向对象语言的类型研究，也用于 GJ 的设计。不过，其理论比普通 $F_{<}$ 更复杂，而且只有结合递归类型时才真正有意义，因为不存在非递归类型 X 能满足 $X <: \{a : \text{Nat}, b : X\}$ 这样的约束。

像 Γ_4 这样让类型变量通过上界相互递归的上下文也曾有人研究；这类演算通常允许每个新变量绑定引入一组涉及新变量和已有变量的不等式。本书不再讨论 F-有界量化，并把 Γ_2 、 Γ_3 、 Γ_4 全部视为不良构。正式地说，上下文出现的每个类型，只能自由引用它左侧已经绑定的类型变量。

子类型

正如普通项变量关联着类型， $F_{<}$ 中的类型变量关联着上界；在子类型检查和类型检查期间都必须跟踪这些上界。为此，上下文中的类型变量绑定改写为 $X <: T$ 。子类型推导可以用这样的绑定证明“类型变量 X 是 T 的子类型，因为上下文中作了这一假设”：

$$\frac{X <: T \in \Gamma}{\Gamma \vdash X <: T} \text{S-TVAR}$$

加入这条规则后，子类型判断成为三元关系 $\Gamma \vdash S <: T$ ，读作“在假设 Γ 下， S 是 T 的子类型”。相应地，其余子类型规则也都要带上上下文。

除了新的变量规则，还需要一条比较有界全称类型的规则。核 S-All 要求两边量词使用完全相同的上界，只在这个共同上界下比较量词体。“核”这一名称来自 Cardelli 与 Wegner (1985) 的原始论文；其中相应的演算称为 Kernel Fun。

赋型

普通全称类型的两条赋型规则也要带上界。T-TAbs 在检查类型抽象体时，把类型变量及其上界加入上下文；T-TApp 则检查实际类型参数确实是声明上界的子类型。图26-1汇总了核 $F_{<}$ 的完整定义。

练习 26.2.1 (★). 在 $\Gamma = B <: \text{Top}, X <: B, Y <: X$ 下，画出

$$B \rightarrow Y <: X \rightarrow B$$

的子类型推导。

[查看练习 26.2.1 的译者参考解答](#)

完全 $F_{<}$

核 S-All 要求两边量词的上界完全相同。若把量词看作一种从类型实参到项实例的函数，核规则就类似于把通常的箭头子类型规则限制成只允许结果类型协变、参数类型 U 必须保持不变：

$$\frac{\Gamma \vdash S_2 <: T_2}{\Gamma \vdash U \rightarrow S_2 <: U \rightarrow T_2}$$

无论对箭头还是对量词，这种限制都显得不太自然。由此可以把核 S-All 放宽，让有界量词的“左侧”像函数参数一样逆变：右侧类型只要求接受 T_1 的子类型；左侧值能接受更宽的 S_1 范围，因而可在右侧所需的位置使用。换言之， $\forall X <: S_1. S_2$ 描述一组从 S_1 的各个子类型映射到 S_2 实例的多态值。若 $T_1 <: S_1$ ，那么右侧只会在更窄的 T_1 范围内使用这些值；左侧值当然也能接受这一范围。对于两边都接受的每个类型实参 $U <: T_1$ ，若 $[X \mapsto U]S_2$ 又是 $[X \mapsto U]T_2$ 的子类型，左侧类型便可安全用于右侧类型的位置。

练习 26.2.2 (★). 举出若干在完全 $F_{<}$ 中成立、在核 $F_{<}$ 中不成立的子类型对。

[查看练习 26.2.2 的译者参考解答](#)

练习 26.2.3 (★★★). 能否找到实际有用、确实需要完全 S-All 的例子？

[查看原书附录 A 中的 26.2.3 解答](#)

语法

$$\begin{aligned}
T &::= X \mid \text{Top} \mid T \rightarrow T \mid \forall X <: T. T, \\
t &::= x \mid \lambda x : T. t \mid t t \mid \lambda X <: T. t \mid t[T], \\
v &::= \lambda x : T. t \mid \lambda X <: T. t, \\
\Gamma &::= \emptyset \mid \Gamma, x : T \mid \Gamma, X <: T
\end{aligned}$$

求值

$$\begin{array}{c}
\frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2} \text{E-APP1} \qquad \frac{t_2 \longrightarrow t'_2}{v_1 t_2 \longrightarrow v_1 t'_2} \text{E-APP2} \\
\\
\frac{}{(\lambda x : T_{11}. t_{12}) v_2 \longrightarrow [x \mapsto v_2] t_{12}} \text{E-APPAbs} \\
\\
\frac{t_1 \longrightarrow t'_1}{t_1[T_2] \longrightarrow t'_1[T_2]} \text{E-TAPP} \qquad \frac{}{(\lambda X <: T_{11}. t_{12})[T_2] \longrightarrow [X \mapsto T_2] t_{12}} \text{E-TAPPTAbs}
\end{array}$$

核子类型规则

$$\begin{array}{c}
\frac{}{\Gamma \vdash T <: T} \text{S-REFL} \qquad \frac{\Gamma \vdash S <: U \quad \Gamma \vdash U <: T}{\Gamma \vdash S <: T} \text{S-TRANS} \qquad \frac{}{\Gamma \vdash S <: \text{Top}} \text{S-TOP} \\
\\
\frac{X <: T \in \Gamma}{\Gamma \vdash X <: T} \text{S-TVAR} \qquad \frac{\Gamma \vdash T_1 <: S_1 \quad \Gamma \vdash S_2 <: T_2}{\Gamma \vdash S_1 \rightarrow S_2 <: T_1 \rightarrow T_2} \text{S-ARROW} \\
\\
\frac{\Gamma, X <: S_1 \vdash S_2 <: T_2}{\Gamma \vdash \forall X <: S_1. S_2 <: \forall X <: S_1. T_2} \text{S-ALL}
\end{array}$$

赋型

$$\begin{array}{c}
\frac{x : T \in \Gamma}{\Gamma \vdash x : T} \text{T-VAR} \\
\\
\frac{\Gamma, x : T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2} \text{T-ABS} \qquad \frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}} \text{T-APP} \\
\\
\frac{\Gamma, X <: T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda X <: T_1. t_2 : \forall X <: T_1. T_2} \text{T-TABS} \\
\\
\frac{\Gamma \vdash t_1 : \forall X <: T_{11}. T_{12} \quad \Gamma \vdash T_2 <: T_{11}}{\Gamma \vdash t_1[T_2] : [X \mapsto T_2] T_{12}} \text{T-TAPP} \qquad \frac{\Gamma \vdash t : S \quad \Gamma \vdash S <: T}{\Gamma \vdash t : T} \text{T-SUB}
\end{array}$$

图 26-1: 有界量化 (核 $F_{<}$)

$$\frac{\Gamma \vdash T_1 <: S_1 \quad \Gamma, X <: T_1 \vdash S_2 <: T_2}{\Gamma \vdash \forall X <: S_1. S_2 <: \forall X <: T_1. T_2} \text{S-ALL (完全)}$$

图 26-2: “完全”有界量化

26.3 示例

本节用几个小程序说明 $F_{<}$ 的性质，而不是展示实际应用；更大、更完整的案例将在第二十七章和第三十二章出现。以下例子同时适用于核 $F_{<}$ 和完全 $F_{<}$ 。

序对编码

§23.4的多态序对编码

$$\text{Pair } T_1 T_2 = \forall X. (T_1 \rightarrow T_2 \rightarrow X) \rightarrow X$$

的值表示由 T_1 和 T_2 元素组成的序对。构造函数及两个投影定义为

```
pair = lambda X. lambda Y. lambda x:X. lambda y:Y.
      (lambda R. lambda p:X->Y->R. p x y) as Pair X Y;
? pair : All X. All Y. X -> Y -> Pair X Y

fst = lambda X. lambda Y. lambda p:Pair X Y.
      p [X] (lambda x:X. lambda y:Y. x);
? fst : All X. All Y. Pair X Y -> X

snd = lambda X. lambda Y. lambda p:Pair X Y.
      p [Y] (lambda x:X. lambda y:Y. y);
? snd : All X. All Y. Pair X Y -> Y
```

pair 定义中的类型标注帮助检查器以更易读的形式打印类型。同一编码当然也能用于 $F_{<}$ ，因为 $F_{<}$ 包含 System F 的全部特性。更有意思的是，它还自动具有预期的子类型性质：

$$S_1 <: T_1, \quad S_2 <: T_2 \quad \Longrightarrow \quad \text{Pair } S_1 S_2 <: \text{Pair } T_1 T_2$$

练习 26.3.1 (★). 从序对编码和 $F_{<}$ 的规则推导上述性质。

[查看练习 26.3.1 的译者参考解答](#)

记录编码

记录、记录类型及其子类型规律都可以在纯 $F_{<}$ 中编码。下面的编码由 Cardelli (1992) 提出。

定义 26.3.2. 对任意 $n \geq 0$ 及类型 $T_1, \dots, T_n$, 定义

$$\{T_i\}^{i \in 1..n} \stackrel{\text{def}}{=} \text{Pair } T_1 (\text{Pair } T_2 \cdots (\text{Pair } T_n \text{ Top}) \cdots)$$

特别地, $\{\}^{\text{def}} = \text{Top}$ 。对项 $t_1, \dots, t_n$, 相应定义

$$\{t_i\}^{i \in 1..n} \stackrel{\text{def}}{=} \text{pair } t_1 (\text{pair } t_2 \cdots (\text{pair } t_n \text{ top}) \cdots),$$

其中为简洁起见省略 **pair** 的类型实参; **top** 是任取的 **Top** 型闭项。第 n 个投影定义为先取 $n-1$ 次 **snd**, 再取一次 **fst**:

$$t.n \stackrel{\text{def}}{=} \text{fst}(\underbrace{\text{snd}(\cdots (\text{snd } t)) \cdots}_{n-1 \text{ 次}})$$

这些缩写定义的就是柔性元组 (*flexible tuple*); 与普通元组不同, 它们在子类型关系下可以从右端扩展。由定义直接得到

$$\frac{\Gamma \vdash S_i <: T_i \quad (i \in 1..n)}{\Gamma \vdash \{S_i\}^{i \in 1..n+k} <: \{T_i\}^{i \in 1..n}}$$

以及赋型规则

$$\frac{\Gamma \vdash t_i : T_i \quad (i \in 1..n)}{\Gamma \vdash \{t_i\}^{i \in 1..n} : \{T_i\}^{i \in 1..n}} \quad \frac{\Gamma \vdash t : \{T_i\}^{i \in 1..n}}{\Gamma \vdash t.i : T_i}$$

现在取一个可数标签集合 $\mathcal{L}$, 并固定一个双射 $\text{label-with-index} : \mathbb{N} \rightarrow \mathcal{L}$, 由此给标签规定全序。

定义 26.3.3. 设 $L \subseteq \mathcal{L}$ 为有限集, 并为每个 $l \in L$ 给定类型 S_l 。令 m 为 L 中标签编号的最大值, 并定义

$$\widehat{S}_i = \begin{cases} S_l, & \text{label-with-index}(i) = l \in L, \\ \text{Top}, & \text{label-with-index}(i) \notin L \end{cases}$$

记录类型 $\{l : S_l\}^{l \in L}$ 定义为柔性元组 $\{\widehat{S}_i\}^{i \in 1..m}$ 。类似地, 若为每个 $l \in L$ 给定项 t_l , 定义

$$\widehat{t}_i = \begin{cases} t_l, & \text{label-with-index}(i) = l \in L, \\ \text{top}, & \text{label-with-index}(i) \notin L \end{cases}$$

记录值 $\{l = t_l\}^{l \in L}$ 定义为 $\{\widehat{t}_i\}^{i \in 1..m}$ 。若 $\text{label-with-index}(i) = l$, 则字段投影 $t.l$ 就是元组投影 $t.i$ 。

该编码验证了记录预期应有的赋型与子类型规则, 即 图15-2和图15-3中的 S-RcdWidth、S-RcdDepth、S-RcdPerm、T-Rcd 与 T-Proj。不过, 它的意义主要在理论方面; 从实践角度看, 对所有字段标签采用全局次序是严重缺点。在支持独立编译的语言中, 不能逐个模块给标签分配编号, 而必须在链接时一次性完成。

带子类型的 Church 编码

作为 $F_{<}$ 表达能力的最后一个小例子，考察给 System F 的 Church 数编码加入有界量化会发生什么。普通 Church 数类型

$$\text{CNat} = \forall X. (X \rightarrow X) \rightarrow X \rightarrow X$$

可以拟人化地读成：“先告诉我结果类型 X ，再给我一个 X 上的函数和一个 X 型基点；我把函数在基点上迭代 n 次，再交回一个 X 。”加入两个有界量词，并细化参数 s 与 z 的类型，得到

$$\text{SNat} = \forall X <: \text{Top}. \forall S <: X. \forall Z <: X. (X \rightarrow S) \rightarrow Z \rightarrow X$$

进一步令

$$\text{SZero} = \forall X <: \text{Top}. \forall S <: X. \forall Z <: X. (X \rightarrow S) \rightarrow Z \rightarrow Z,$$

$$\text{SPos} = \forall X <: \text{Top}. \forall S <: X. \forall Z <: X. (X \rightarrow S) \rightarrow Z \rightarrow S$$

SNat 可以读成：“给我一般结果类型 X 及其两个子类型 S 、 Z ，再给我一个把任意 X 映到 S 的函数和一个特殊的 Z 型基点；我迭代函数 n 次，返回一个 X 。”

SZero 的形式几乎相同，却承诺最终结果属于 Z ，而不只是 X 。唯一可能的做法就是直接返回参数 z ：

```
szero = lambda X. lambda S<:X. lambda Z<:X.
      lambda s:X->S. lambda z:Z. z;
? szero : SZero
```

在“所有值的行为都与 szero 相同”的意义下，它是 SZero 的唯一值。又因为 $\text{SZero} <: \text{SNat}$ ，也有 $\text{szero} : \text{SNat}$ 。

SPos 则有更多值：

```
sone = lambda X. lambda S<:X. lambda Z<:X.
      lambda s:X->S. lambda z:Z. s z;
stwo = lambda X. lambda S<:X. lambda Z<:X.
      lambda s:X->S. lambda z:Z. s (s z);
sthree = lambda X. lambda S<:X. lambda Z<:X.
      lambda s:X->S. lambda z:Z. s (s (s z));
```

事实上， SPos 包含除 szero 外的全部 SNat 值。

同样可以细化 Church 数操作的类型。类型系统能检查后继函数必定返回正数：

```
ssucc = lambda n:SNat.
      lambda X. lambda S<:X. lambda Z<:X.
      lambda s:X->S. lambda z:Z.
      s (n [X] [S] [Z] s z);
? ssucc : SNat -> SPos

spluspp = lambda n:SPos. lambda m:SPos.
      lambda X. lambda S<:X. lambda Z<:X.
```

```

lambda s:X->S. lambda z:Z.
  n [X] [S] [S] s (m [X] [S] [Z] s z);
? spluspp : SPos -> SPos -> SPos

```

练习 26.3.4 (★★). 写出仅类型标注不同的两版加法，类型分别为

$$\text{SZero} \rightarrow \text{SZero} \rightarrow \text{SZero} \quad \text{和} \quad \text{SPos} \rightarrow \text{SNat} \rightarrow \text{SPos}$$

[查看原书附录 A 中的 26.3.4 解答](#)

上面的示例和练习提出一个值得注意的问题。我们显然不想为加法写许多不同名称的版本，再根据参数的预期类型挑选；更希望一个 `plus` 同时具有全部这些类型：

$$\begin{aligned}
&\text{plus} : \text{SZero} \rightarrow \text{SZero} \rightarrow \text{SZero} \\
&\quad \wedge \text{SNat} \rightarrow \text{SPos} \rightarrow \text{SPos} \\
&\quad \wedge \text{SPos} \rightarrow \text{SNat} \rightarrow \text{SPos} \\
&\quad \wedge \text{SNat} \rightarrow \text{SNat} \rightarrow \text{SNat}
\end{aligned}$$

其中 $t : S \wedge T$ 表示 t 同时具有 S 和 T 两种类型。支持这类重载的需求推动了交类型 (§15.7) 与有界量化的组合研究，参见 Pierce (1997b)。

练习 26.3.5 (推荐, ★★). 仿照 `SNat`，把 Church 布尔值精化为 `SBool` 及其子类型 `STrue`、`SFalse`，并写出函数 `notft`，使其类型为 `SFalse` $\rightarrow$ `STrue`；再写出类似的函数 `nottf`，使其类型为 `STrue` $\rightarrow$ `SFalse`。

[查看原书附录 A 中的 26.3.5 解答](#)

练习 26.3.6 (★). 本章开头指出，还可以用一种比 $F_{<}$ 更直接、更正交的方式组合子类型与多态：从 System F（也可加入记录等构造）出发，像带子类型的简单类型 lambda 演算那样加入子类型关系，但仍保留无界量化。对子类型关系只增加普通量词体的协变规则：

$$\frac{\Gamma, X \vdash S_2 <: T_2}{\Gamma \vdash \forall X. S_2 <: \forall X. T_2}$$

本章的哪些例子可以在这个更简单的系统中表达？

[查看练习 26.3.6 的译者参考解答](#)

26.4 安全性

核与完全 $F_{<}$ 都具有类型安全性。本节详细证明核系统；完全系统的基本论证相似，但第二十八章将看到，它不仅更难分析，还缺少核系统的一些重要性质，包括可判定的类型检查。

先列出若干关于赋型关系和子类型关系的技术性质。这里称上下文“良构”，专指其中各类型变量的作用域正确；以下各项均可按推导作常规归纳证明。

引理 26.4.1 (排列). 设 Γ 良构, Γ' 是 Γ 的一个排列: 二者包含相同的绑定, 而且 Γ' 保持 Γ 中类型变量的作用域依赖; 也就是说, 若一个绑定引入的类型变量出现在其右侧另一个绑定中, 那么这两个绑定在 Γ' 中的次序不变。于是:

1. 若 $\Gamma \vdash t : T$, 则 $\Gamma' \vdash t : T$;
2. 若 $\Gamma \vdash S <: T$, 则 $\Gamma' \vdash S <: T$ 。

引理 26.4.2 (弱化). 在下面各项中, 均假定扩展后的上下文良构:

1. 若 $\Gamma \vdash t : T$, 则 $\Gamma, x : U \vdash t : T$;
2. 若 $\Gamma \vdash t : T$, 则 $\Gamma, X <: U \vdash t : T$;
3. 若 $\Gamma \vdash S <: T$, 则 $\Gamma, x : U \vdash S <: T$;
4. 若 $\Gamma \vdash S <: T$, 则 $\Gamma, X <: U \vdash S <: T$ 。

练习 26.4.3 ($\star$). 弱化证明中的哪些情形需要使用排列引理?

[查看原书附录 A 中的 26.4.3 解答](#)

引理 26.4.4 (子类型推导中可删除无关项变量). 若 $\Gamma, x : U, \Delta \vdash S <: T$, 则 $\Gamma, \Delta \vdash S <: T$ 。

证明. 显然成立: 项变量的赋型假设不会参与子类型推导。 $\square$

与以往一样, 保持性证明需要把替换与赋型、子类型关系联系起来。这里由于求值还会把类型代入类型变量, 除通常的项替换引理外, 还要证明类型替换分别保持子类型和赋型。

引理 26.4.5 (收窄).

1. 若 $\Gamma, X <: Q, \Delta \vdash S <: T$ 且 $\Gamma \vdash P <: Q$, 则 $\Gamma, X <: P, \Delta \vdash S <: T$;
2. 若 $\Gamma, X <: Q, \Delta \vdash t : T$ 且 $\Gamma \vdash P <: Q$, 则 $\Gamma, X <: P, \Delta \vdash t : T$ 。

之所以称为“收窄”, 是因为它把变量 X 可取的类型范围从 Q 的所有子类型缩小到 P 的所有子类型。

证明练习 ($\star$)。

[查看原书附录 A 中的 26.4.5 解答](#)

引理 26.4.6 (项替换保持赋型). 若 $\Gamma, x : Q, \Delta \vdash t : T$ 且 $\Gamma \vdash q : Q$, 则

$$\Gamma, \Delta \vdash [x \mapsto q]t : T$$

证明. 对 $\Gamma, x : Q, \Delta \vdash t : T$ 的推导归纳, 并使用上面的排列、弱化与删除无关项变量等性质。 $\square$

定义 26.4.7. $[X \mapsto P]\Delta$ 表示把 P 代入 Δ 中所有绑定的右侧。只替换 X 绑定之后的上下文，因为其左侧按作用域约定不可能含 X 。

引理 26.4.8 (类型替换保持子类型). 若

$$\Gamma, X <: Q, \Delta \vdash S <: T \quad \text{且} \quad \Gamma \vdash P <: Q,$$

则

$$\Gamma, [X \mapsto P]\Delta \vdash [X \mapsto P]S <: [X \mapsto P]T$$

证明. 对 $\Gamma, X <: Q, \Delta \vdash S <: T$ 的推导归纳。只有最后使用 S-TVar 或 S-All 的情形需要展开说明。

若最后使用 S-TVar, 则 $S = Y$, 且绑定 $Y <: T$ 出现在上下文中。若 $Y \neq X$, 替换后仍可在 $\Gamma, [X \mapsto P]\Delta$ 中找到相应绑定, 直接再次使用 S-TVar。若 $Y = X$, 则 $T = Q$, 而 $[X \mapsto P]S = P$ 、 $[X \mapsto P]T = Q$; 所需结论正是前提 $\Gamma \vdash P <: Q$, 再由弱化把它放入替换后的完整上下文即可。

若最后使用 S-All, 则

$$S = \forall Z <: U_1. S_2, \quad T = \forall Z <: U_1. T_2$$

且

$$\Gamma, X <: Q, \Delta, Z <: U_1 \vdash S_2 <: T_2$$

归纳假设给出

$$\Gamma, [X \mapsto P]\Delta, Z <: [X \mapsto P]U_1 \vdash [X \mapsto P]S_2 <: [X \mapsto P]T_2$$

再次应用 S-All, 恰好得到对原来两个全称类型整体作替换后的子类型判断。其余规则直接应用归纳假设即可。 $\square$

引理 26.4.9 (类型替换保持赋型). 若

$$\Gamma, X <: Q, \Delta \vdash t : T \quad \text{且} \quad \Gamma \vdash P <: Q,$$

则

$$\Gamma, [X \mapsto P]\Delta \vdash [X \mapsto P]t : [X \mapsto P]T$$

证明. 对赋型推导归纳, 只展开 T-TApp 与 T-Sub 两个关键情形。

若最后使用 T-TApp, 则

$$t = t_1[T_2], \quad T = [Z \mapsto T_2]T_{12}, \quad \Gamma, X <: Q, \Delta \vdash t_1 : \forall Z <: T_{11}. T_{12}$$

归纳假设给出替换后的 t_1 及全称类型; 再次使用 T-TApp, 可得

$$\Gamma, [X \mapsto P]\Delta \vdash [X \mapsto P](t_1[T_2]) : [Z \mapsto [X \mapsto P]T_2]([X \mapsto P]T_{12})$$

右侧正是 $[X \mapsto P]([Z \mapsto T_2]T_{12})$ 。

若最后使用 T-Sub, 其两个前提分别是 $\Gamma, X <: Q, \Delta \vdash t : S$ 和 $\Gamma, X <: Q, \Delta \vdash S <: T$ 。第一个由归纳假设在替换后仍成立, 第二个由引理26.4.8在替换后仍成立; 最后再应用 T-Sub。 $\square$

引理 26.4.10 (从右向左反演子类型关系).

1. 若 $\Gamma \vdash S <: X$, 则 S 是类型变量。
2. 若 $\Gamma \vdash S <: T_1 \rightarrow T_2$, 则 S 是类型变量, 或 $S = S_1 \rightarrow S_2$, 且 $\Gamma \vdash T_1 <: S_1$, $\Gamma \vdash S_2 <: T_2$ 。
3. 若 $\Gamma \vdash S <: \forall X <: U_1. T_2$, 则 S 是类型变量, 或 $S = \forall X <: U_1. S_2$, 且 $\Gamma, X <: U_1 \vdash S_2 <: T_2$ 。

证明. 第 1 项对子类型推导归纳。只有 S-Trans 情形需要特别说明：先把归纳假设用于右侧前提，得知中间类型必须是类型变量；再用于左侧前提，得知 S 也必须是类型变量。第 2、3 项同理；处理传递性时先使用第 1 项排除不合外层构造的中间类型，再分别应用相应的归纳假设。 $\square$

练习 26.4.11 (推荐, $\star\star$). 证明下面这些“从左向右”的反演性质：

1. 若 $\Gamma \vdash S_1 \rightarrow S_2 <: T$, 则 $T = \text{Top}$, 或 $T = T_1 \rightarrow T_2$, 且 $\Gamma \vdash T_1 <: S_1$, $\Gamma \vdash S_2 <: T_2$ 。
2. 若 $\Gamma \vdash \forall X <: U. S_2 <: T$, 则 $T = \text{Top}$, 或 $T = \forall X <: U. T_2$, 且 $\Gamma, X <: U \vdash S_2 <: T_2$ 。
3. 若 $\Gamma \vdash X <: T$, 则 $T = \text{Top}$ 、 $T = X$, 或存在 $X <: S \in \Gamma$ 使 $\Gamma \vdash S <: T$ 。
4. 若 $\Gamma \vdash \text{Top} <: T$, 则 $T = \text{Top}$ 。

[查看原书附录 A 中的 26.4.11 解答](#)

引理 26.4.12.

1. 若 $\Gamma \vdash \lambda x : S_1. s_2 : T$ 且 $\Gamma \vdash T <: U_1 \rightarrow U_2$, 则 $\Gamma \vdash U_1 <: S_1$, 并存在 S_2 使 $\Gamma, x : S_1 \vdash s_2 : S_2$ 且 $\Gamma \vdash S_2 <: U_2$ 。
2. 若 $\Gamma \vdash \lambda X <: S_1. s_2 : T$ 且 $\Gamma \vdash T <: \forall X <: U_1. U_2$, 则 $U_1 = S_1$, 并存在 S_2 , 使 $\Gamma, X <: S_1 \vdash s_2 : S_2$ 且 $\Gamma, X <: S_1 \vdash S_2 <: U_2$ 。

证明. 对赋值推导归纳。若最后使用 T-Sub, 则用[引理26.4.10](#)反演新增的子类型前提，再应用归纳假设；其余情形直接由最后一条赋值规则得到。 $\square$

定理 26.4.13 (保持性). 若 $\Gamma \vdash t : T$ 且 $t \longrightarrow t'$, 则 $\Gamma \vdash t' : T$ 。

证明. 对 $\Gamma \vdash t : T$ 的推导归纳。T-Var、T-Abs 与 T-TAbs 不可能是最后一条规则，因为变量、项抽象和类型抽象都没有单步求值规则。

情形 T-App. 此时 $t = t_1 t_2$ 、 $T = T_{12}$, 并有 $\Gamma \vdash t_1 : T_{11} \rightarrow T_{12}$ 和 $\Gamma \vdash t_2 : T_{11}$ 。按实际使用的求值规则分三种情形：若 $t_1 \longrightarrow t'_1$, 对左侧前提使用归纳假设，再应用 T-App；若 t_1 已是值而 $t_2 \longrightarrow t'_2$, 对右侧前提使用归纳假设，再应用 T-App；最后，若 $t_1 = \lambda x : U_{11}. u_{12}$ 、 t_2 是值，且 $t' = [x \mapsto t_2]u_{12}$, 则由[引理26.4.12](#)可得某个 U_{12} , 满足

$$\Gamma, x : U_{11} \vdash u_{12} : U_{12}, \quad \Gamma \vdash T_{11} <: U_{11}, \quad \Gamma \vdash U_{12} <: T_{12}$$

先用 T-Sub 得到 $\Gamma \vdash t_2 : U_{11}$, 再用 [引理26.4.6](#)得到 $\Gamma \vdash [x \mapsto t_2]u_{12} : U_{12}$, 最后以 T-Sub 提升至 T_{12} 。

情形 T-TApp. 此时 $t = t_1[T_2]$ 、 $T = [X \mapsto T_2]T_{12}$, 且

$$\Gamma \vdash t_1 : \forall X <: T_{11}.T_{12}, \quad \Gamma \vdash T_2 <: T_{11}$$

若 $t_1 \longrightarrow t'_1$, 由归纳假设和 T-TApp 得到结论。若 $t_1 = \lambda X <: U_{11}.u_{12}$, 且 $t' = [X \mapsto T_2]u_{12}$, 则[引理26.4.12](#)给出 $U_{11} = T_{11}$ 及某个 U_{12} , 满足

$$\Gamma, X <: U_{11} \vdash u_{12} : U_{12}, \quad \Gamma, X <: U_{11} \vdash U_{12} <: T_{12}$$

由[引理26.4.9](#)把 T_2 代入第一式, 得到 $\Gamma \vdash [X \mapsto T_2]u_{12} : [X \mapsto T_2]U_{12}$; 再由 [引理26.4.8](#)把 T_2 代入第二式, 并应用 T-Sub, 得到所需类型 $[X \mapsto T_2]T_{12}$ 。这里按官方勘误使用的是类型替换保持赋值引理 [引理26.4.9](#)。

情形 T-Sub. 归纳假设先给出 $\Gamma \vdash t' : S$, 再沿用原有前提 $\Gamma \vdash S <: T$ 应用 T-Sub。□

引理 26.4.14 (典范形式).

1. 若 v 是类型 $T_1 \rightarrow T_2$ 的闭值, 则 v 具有形式 $\lambda x : S_1.t_2$;
2. 若 v 是类型 $\forall X <: T_1.T_2$ 的闭值, 则 v 具有形式 $\lambda X <: T_1.t_2$ 。

证明. 两项都对赋值推导归纳; 这里只写第 2 项。闭值 v 获得类型 $\forall X <: T_1.T_2$ 的推导, 最后一条规则只能是 T-TAbs 或 T-Sub。前者直接说明 v 是类型抽象。若最后使用 T-Sub, 则前提给出 $\emptyset \vdash v : S$ 和 $\emptyset \vdash S <: \forall X <: T_1.T_2$ 。由 [引理26.4.10](#), S 只能是相同上界的全称类型; 再对前一棵较小的赋值推导使用归纳假设, 便知 v 仍是类型抽象。第 1 项对箭头类型作同样论证。□

定理 26.4.15 (进展性). 闭的良类型 $F_{<}$ 项或为值, 或能作一次单步求值。

证明. 对赋值推导归纳。变量情形不会出现, 因为 t 是闭项; 项抽象和类型抽象本身就是值。

在 T-App 情形中, 先对函数部分使用归纳假设: 若它能继续求值, 就应用 E-App1; 若它已是值, 再对实参使用归纳假设。实参若能求值, 就应用 E-App2; 若二者都是值, [引理26.4.14](#)说明函数部分必为项抽象, 于是可以应用 E-AppAbs。

在 T-TApp 情形中, 若 t_1 能求值, 就应用 E-TApp; 若 t_1 是值, [引理26.4.14](#)说明它必为类型抽象, 于是应用 E-TAppTAbs。T-Sub 不改变项本身, 直接沿用其赋值前提的归纳结论。□

练习 26.4.16 (★★★). 把本节安全性论证推广到完全 $F_{<}$ 。

[查看练习 26.4.16 的译者参考解答](#)

26.5 有界存在类型

存在量词也可以像全称量词一样加入上界：

$$\{\exists X <: T_1, T_2\}$$

它隐藏实际表示类型，却公开保证该表示是 T_1 的子类型。与有界全称类型相同，S-Some 也有两个版本：核版本要求所比较的两个量词具有相同上界，完全版本则允许上界不同。图中给出的是核版本，量词体协变；相应的打包与开包规则也要记录这个上界。

新增语法

$$T ::= \dots \mid \{\exists X <: T_1, T_2\}$$

新增子类型规则

$$\frac{\Gamma, X <: U \vdash S <: T}{\Gamma \vdash \{\exists X <: U, S\} <: \{\exists X <: U, T\}} \text{S-SOME}$$

新增赋型规则

$$\frac{\Gamma \vdash t_2 : [X \mapsto U]T_2 \quad \Gamma \vdash U <: T_1}{\Gamma \vdash \{*U, t_2\} \text{ as } \{\exists X <: T_1, T_2\} : \{\exists X <: T_1, T_2\}} \text{T-PACK}$$

$$\frac{\Gamma \vdash t_1 : \{\exists X <: T_{11}, T_{12}\} \quad \Gamma, X <: T_{11}, x : T_{12} \vdash t_2 : T_2 \quad X \notin \text{FV}(T_2)}{\Gamma \vdash \text{let } \{X, x\} = t_1 \text{ in } t_2 : T_2} \text{T-UNPACK}$$

图 26-3: 有界存在量化（核版本）

练习 26.5.1 (★). 写出完全版本的 S-Some。

[查看原书附录 A 中的 26.5.1 解答](#)

练习 26.5.2 (★). 在带记录和存在类型、但没有子类型的纯 System F 中，有多少种存在类型能使

$$\{*\text{Nat}, \{a = 5, b = 7\}\} \text{ as } T$$

良类型？加入子类型与有界存在量词后是否会增加？

[查看原书附录 A 中的 26.5.2 解答](#)

§24.2 说明了怎样用普通存在类型实现 ADT。给存在量词加入上界后，便得到对应的细化形式；Cardelli 与 Wegner (1985) 称之为 部分抽象类型 (*partially abstract type*)。客户不知道表示的确切身份，却知道它至少具有某些结构。例如

```
counterADT =
  {*Nat,
   {new = 1,
```

```

    get = lambda i:Nat. i,
    inc = lambda i:Nat. succ(i)}}
as {Some Counter<:Nat,
    {new:Counter,
     get:Counter->Nat,
     inc:Counter->Counter}}};
? counterADT
: {Some Counter<:Nat,
   {new:Counter,get:Counter->Nat,inc:Counter->Counter}}

```

包的上界 `Counter <: Nat` 向客户公开了一部分表示信息。像§24.2一样开包后，可以通过接口操作计数器：

```

let {Counter,counter} = counterADT in
counter.get (counter.inc (counter.inc counter.new));
? 3 : Nat

```

现在还可以把 `Counter` 型值直接用作自然数：

```

let {Counter,counter} = counterADT in
succ (succ (counter.inc counter.new));
? 4 : Nat

```

反方向仍不成立；普通自然数不能冒充只能由实现创建的计数器：

```

let {Counter,counter} = counterADT in
counter.inc 3;
? Error: parameter type mismatch

```

因此，这个版本有意公开表示类型的一部分，让客户使用计数器更加方便，同时仍由实现控制哪些值可以作为计数器产生。

练习 26.5.3 (*)**. 用有界存在包定义 `Counter` 与 `ResetCounter`，满足以下要求：两种 ADT 都提供 `new`、`get` 和 `inc`；`ResetCounter` 还提供 `reset`，它接收一个计数器，返回一个重置到固定值（例如 1）的新计数器；客户可以把 `ResetCounter` 用作 `Counter`，但不能得知两者表示方式的更多细节。

[查看原书附录 A 中的 26.5.3 解答](#)

有界存在类型也能细化§24.2用存在类型编码对象的方法。那里把存在包的见证类型用作对象内部状态的类型；状态本身是一条实例变量记录。把无界存在量词换成有界存在量词，就能只向外公开一部分实例变量的名称和类型，而继续隐藏其余字段。下面的计数器公开状态中的 `x`，隐藏私有字段 `private`：

```

c = {*[x:Nat, private:Bool],
     {state = {x=5, private=false},
      methods =
        {get = lambda s:{x:Nat}. s.x,

```

```

    inc = lambda s:{x:Nat,private:Bool}.
      {x=succ(s.x), private=s.private}}}}
  as {Some X<:{x:Nat},
    {state:X,
      methods:{get:X->Nat, inc:X->X}}}};
? c
: {Some X<:{x:Nat},
  {state:X,methods:{get:X->Nat,inc:X->X}}}

```

客户既可以调用 `get` 方法取得计数器的数值，也可以通过已公开的 `state.x` 字段直接读取它；私有字段仍然不可见。

练习 26.5.4 (★★). 把§24.3的编码推广为：用有界全称类型编码有界存在类型。检查 `S-Some` 是否可由这一编码以及有界全称类型的子类型规则推出。

[查看原书附录 A 中的 26.5.4 解答](#)

26.6 注记

CLU (Liskov 等, 1977, 1981; Schaffert, 1978; Scheifler, 1978) 可能是最早提供类型安全有界量化的语言。CLU 的参数界大致相当于把 §26.2 的量词有界形式推广到多个类型参数。

Cardelli 与 Wegner (1985) 在 Fun 语言中提出了本章采用的有界量化形式；其 Kernel Fun 对应核 $F_{<}$ 。Fun 建立在 Cardelli 早期的非形式思想上，并用 Mitchell (1984b) 的技术加以形式化；它把 Girard-Reynolds 多态 (Girard, 1972; Reynolds, 1974) 与 Cardelli (1984) 的一阶子类型演算结合起来。Bruce 与 Longo (1990) 先对原来的 Fun 作了简化和小幅推广，Curien 与 Ghelli (1992) 随后又作了一次，得到本书所称的完全 $F_{<}$ 。Cardelli、Martini、Mitchell 与 Scedrov (1994) 的综述对有界量化作了最全面的讨论。

程序语言理论与设计领域对 $F_{<}$ 及其近亲作了大量研究。Cardelli 与 Wegner 的综述最早给出了使用有界量化的程序示例；Cardelli (1988a) 对幂种类的研究又发展了更多示例。Curien 与 Ghelli (1992; Ghelli, 1990) 研究了 $F_{<}$ 的若干句法性质。与它密切相关的系统，其语义方面分别由 Bruce 与 Longo (1990)、Martini (1988)、Breazu-Tannen、Coquand、Gunter 与 Scedrov (1991)、Cardone (1989)、Cardelli 与 Longo (1991)、Cardelli、Martini、Mitchell 与 Scedrov (1994)、Curien 与 Ghelli (1992, 1991)，以及 Bruce 与 Mitchell (1992) 研究。Cardelli 与 Mitchell (1991)、Bruce (1991)、Cardelli (1992)，以及 Canning、Cook、Hill、Olthoff 与 Mitchell (1989b)，还把 $F_{<}$ 扩展为包含记录类型和更丰富的继承形式。有界量化也是 Cardelli 的程序语言 Quest (1991; Cardelli 与 Longo, 1991)、HP 实验室开发的 Abel (Canning、Cook、Hill 与 Olthoff, 1989a; Canning、Cook、Hill、Olthoff 与 Mitchell, 1989b; Canning、Hill 与 Olthoff, 1988; Cook、Hill 与 Canning, 1990)，以及较新的 GJ (Bracha、Odersky、Stoutamire 与 Wadler, 1998)、Pict (Pierce 与 Turner, 2000) 和 Funnel (Odersky, 2000) 等语言设计中的关键机制。

Ghelli (1990) 以及 Cardelli、Martini、Mitchell 与 Scedrov (1994) 研究了有界量化对§26.3中代数数据类型 Church 编码的影响。Pierce (1991b, 1997b) 研究了加入交类型的 $F_{<}$; Compagnoni 与 Pierce (1996) 还用带高阶种类的变体刻画带多重继承的面向对象语言, 其元理论由 Compagnoni (1994) 分析。

第二十七章 案例研究：再论命令式对象

概述

第十八章在带记录、引用和子类型的简单类型演算中，建立了一组表达命令式对象的常用编码。类接收一个指向 self 方法表的引用，并在定义自己的方法时通过该引用调用其他方法；类返回完整的方法表后，再把 self 引用回填为指向这张表。[§18.12](#)试图把构造方法表的工作从每次方法调用移到对象创建时。本章借助有界量化进一步改造这一编码。¹

设可设值计数器的公开接口和内部表示为

$$\begin{aligned}\text{SetCounter} &= \{\text{get} : \text{Unit} \rightarrow \text{Nat}, \text{set} : \text{Nat} \rightarrow \text{Unit}, \text{inc} : \text{Unit} \rightarrow \text{Unit}\}, \\ \text{CounterRep} &= \{x : \text{Ref Nat}\}.\end{aligned}$$

[§18.12](#)把 self 方法表的引用传入类：

$$\begin{aligned}\text{setCounterClass} &= \lambda r : \text{CounterRep}. \lambda \text{self} : \text{Source SetCounter}. \\ &\quad \{\text{get} = \lambda _ : \text{Unit}. ! (r.x), \\ &\quad \text{set} = \lambda i : \text{Nat}. r.x := i, \\ &\quad \text{inc} = \lambda _ : \text{Unit}. (!\text{self}).\text{set}(\text{succ}((!\text{self}).\text{get unit}))\}.\end{aligned}$$

交互式类型检查器给出：

```
? setCounterClass
: CounterRep -> (Source SetCounter) -> SetCounter
```

使用 Source 而不是不变的 Ref，是为了让子类 self 能按协变关系提升为父类所需的接口。

例如，带访问计数的计数器接口及其表示为

```
InstrCounter =
  {get:Unit->Nat, set:Nat->Unit,
   inc:Unit->Unit, accesses:Unit->Nat};

InstrCounterRep = {x:Ref Nat, a:Ref Nat};

instrCounterClass =
```

¹本章示例使用带记录（[图15-3](#)）和引用（[图13-1](#)）的 $F_{<}$ 。原书为这些示例列出的 OCaml 实现名为 fullfsubref。

```

lambda r:InstrCounterRep. lambda self:Source InstrCounter.
let super = setCounterClass r self in
{get = super.get,
 set = lambda i:Nat.
      (r.a:=succ(!(r.a)); super.set i),
 inc = super.inc,
 accesses = lambda _:Unit. !(r.a)};
? instrCounterClass
: InstrCounterRep ->
  (Source InstrCounter) -> InstrCounter

```

构造 `super` 时，要把子类的 `Source InstrCounter` 型 `self` 传给只要求 `Source SetCounter` 的父类。由于 `Source` 协变而 `InstrCounter <: SetCounter`，这项转换成立；若 `self` 使用不变的 `Ref`，便无法通过类型检查。

但同一类生成的所有对象原则上可以共享方法代码。真实面向对象实现通常让对象只携带指向类数据结构的指针，方法保存在类中。²上面的参数顺序先接收实例变量 `r`、再接收 `self`，使方法表直到每个对象创建时才形成。若先接收 `self`，就可先对类作一次部分应用：

```

setCounterClass = λself : Source(CounterRep → SetCounter).λr : CounterRep.
  {get = λ_ : Unit.!(r.x),
   set = λi : Nat.r.x := i,
   inc = λ_ : Unit.(!self r).set(succ(!self r).get unit))}.

```

其推断类型为

```

? setCounterClass
: (Source (CounterRep->SetCounter)) ->
  CounterRep -> SetCounter

```

与旧版本相比，这里有三项实质变化。第一，`self` 移到了 `r` 之前。第二，`self` 的内容从 `SetCounter` 改成了 `CounterRep → SetCounter`：类返回的方法表现在本身是等待实例变量记录的函数，所以 `self` 方法表也必须有相同类型。第三，类体中每个 `!self` 都改成 `!self r`，因为解引用得到的是函数，还要把当前实例变量记录传给它。对象创建函数为

```

newSetCounter = let m = ref(λr : CounterRep.error as SetCounter) in
  let m' = setCounterClass m in
  (m := m'; λ_ : Unit.let r = {x = ref 1} in m' r).

```

```

? newSetCounter : Unit -> SetCounter

```

²原书进一步把实际语言概括为：类只保存相对于父类新增或覆写的方法，调用时沿类层级向上查找目标方法；这样就不会构造包含全部继承方法的完整方法表。不过，这种运行时查找会给静态类型分析带来棘手问题，本书不再讨论。作者勘误同时提醒，这只是过度简化的说明；真实实现通常采用更复杂且更高效的分派结构。

绑定建立时只执行到最后的 `lambda` 抽象；以后每次调用只分配新的实例变量记录并把 m' 应用于它。

参数重排后，问题重新出现在继承处。若直接把前面带访问计数的子类所用

$$self : \text{Source}(\text{InstrCounterRep} \rightarrow \text{InstrCounter})$$

传给父类，所需关系是

$$\text{Source}(\text{InstrCounterRep} \rightarrow \text{InstrCounter}) <: \text{Source}(\text{CounterRep} \rightarrow \text{SetCounter}),$$

但箭头参数逆变， $\text{InstrCounterRep} <: \text{CounterRep}$ 的方向恰好无法满足这一判断。

失败会直接出现在 `super` 的定义处。把参数重排后的子类照旧写成

```
instrCounterClass =
  lambda self:Source (InstrCounterRep->InstrCounter).
  let super = setCounterClass self in
  lambda r:InstrCounterRep.
    {get = (super r).get,
     set = lambda i:Nat.
       (r.a:=succ(!(r.a)); (super r).set i),
     inc = (super r).inc,
     accesses = lambda _:Unit. !(r.a)};
? Error: parameter type mismatch
```

当前 `self` 是如下函数类型的只读引用：

$$\text{InstrCounterRep} \rightarrow \text{InstrCounter}$$

父类要求的则是

$$\text{CounterRep} \rightarrow \text{SetCounter}$$

虽然结果类型方向正确，函数参数位置逆变却要求 $\text{CounterRep} <: \text{InstrCounterRep}$ ，而事实恰好相反。

有界量化把具体表示类型抽成 R ：

$$\begin{aligned} \text{setCounterClass} = & \lambda R <: \text{CounterRep}. \lambda self : \text{Source}(R \rightarrow \text{SetCounter}). \lambda r : R. \\ & \{ \text{get} = \lambda _ : \text{Unit}. !(r.x), \\ & \quad \text{set} = \lambda i : \text{Nat}. r.x := i, \\ & \quad \text{inc} = \lambda _ : \text{Unit}. (!self\ r). \text{set}(\text{succ}(!self\ r). \text{get}\ \text{unit}) \}. \end{aligned}$$

其类型为

$$\forall R <: \text{CounterRep}. \text{Source}(R \rightarrow \text{SetCounter}) \rightarrow R \rightarrow \text{SetCounter}.$$

交互式检查器相应输出：

```
? setCounterClass
: All R<:CounterRep.
  (Source (R->SetCounter)) -> R -> SetCounter
```


可以把这项变化理解为父类降低了对 *self* 的要求。旧版本要求：“给我一个恰好接收 `CounterRep` 的 *self*，我将据此构造同样接收 `CounterRep` 的方法表。”新版本先让调用者指定正在构造对象的表示类型 *R*；上界保证 *R* 至少含有 `x` 字段。父类随后只要求 *self* 接收这个同一个 *R*，并返回至少具有 `SetCounter` 接口的方法表，自己也返回同类方法表。子类便可在 $R <: \text{InstrCounterRep}$ 下定义：

```
instrCounterClass = λR <: InstrCounterRep. λself : Source(R → InstrCounter). λr : R.
  let super = setCounterClass[R]self in
  {get = (super r).get,
   set = λi : Nat. (r.a := succ(!(r.a)); (super r).set i),
   inc = (super r).inc,
   accesses = λ_ : Unit. !(r.a)}.
```

其类型为

```
? instrCounterClass
  : All R<:InstrCounterRep.
    (Source (R->InstrCounter)) -> R -> InstrCounter
```

现在传给父类只需

$$\text{Source}(R \rightarrow \text{InstrCounter}) <: \text{Source}(R \rightarrow \text{SetCounter}),$$

两边箭头参数同为 *R*，结果位置由 $\text{InstrCounter} <: \text{SetCounter}$ 协变即可。换言之，*super* 处仍然使用包容规则把 *self* 提升为父类所需类型；但有界量化先把父类实例化到子类选择的 *R*，已经消除了原来两个函数参数类型相反的问题。

创建子类对象时，把实际表示类型显式提供给类：

```
newInstrCounter = let m = ref(λr : InstrCounterRep.error as InstrCounter) in
  let m' = instrCounterClass[InstrCounterRep]m in
  (m := m'; λ_ : Unit. let r = {x = ref 1, a = ref 0} in m' r).
```

```
? newInstrCounter : Unit -> InstrCounter
```

唯一新增的步骤，是用实际的实例变量记录类型实例化 `instrCounterClass`；这里使用的记录类型是 `InstrCounterRep`。与前面一样，定义 `newInstrCounter` 时，创建并回填 *self* 引用的步骤只执行一次。

最后用几次调用检查对象行为：

```
ic = newInstrCounter unit;
ic.inc unit;
ic.get unit;
? 2 : Nat
```

```
ic.accesses unit;  
? 1 : Nat
```

译者注：作者的官方勘误指出，本章宣称的效率改进实际上没有实现。虽然类先接收 *self*，并且 m' 只在创建函数绑定时计算一次，但 m' 仍是从实例变量记录到方法记录的函数。开放递归调用中的 *!self r* 会再次运行这个函数，重新构造方法记录；最终效率甚至比§18.12 末尾的编码更差。本章推导在类型层面仍展示了有界量化如何解决表示类型逆变造成的继承障碍，但不应把它当作成功的方法表共享方案。勘误另提到 John Tang Boyland 建议用第 32 章风格的 $F_{\omega<}$ 给出另一种案例研究。

练习 27.1 (推荐, ★★★). 上述新编码依赖 *Source* 的协变性。若语言只有有界量化和不变的 *Ref*，能否得到操作行为相同且具有同等效率的良类型类编码？

[查看原书附录 A 中的 27.1 解答](#)

第二十八章 有界量化的元理论

本章为 $F_{<}$ 建立子类型检查和类型检查算法。核系统与完全系统的表现不同：有些性质二者都有，但完全系统更难证明；另一些性质在完全系统中彻底失效，这是增强表达能力付出的代价。

§28.1与§28.2先给出一套对核系统和完全系统都适用的类型检查算法。随后分别考察核系统 (§28.3) 和完全系统 (§28.4) 的子类型检查。§28.5继续讨论完全 $F_{<}$ ，重点是其子类型关系不可判定这一出人意料的结果；§28.6说明核系统有并和交，而完全系统没有；§28.7讨论有界存在类型带来的问题；最后，§28.8考察加入最小类型 Bot 的影响。

28.1 暴露

§16.2的算法由子项最小类型计算整个项的最小类型。 $F_{<}$ 多了类型变量，因而要处理一个新问题。设

```
f = lambda X<:Nat->Nat. lambda y:X. y 5;
? f : All X<:Nat->Nat. X -> Nat
```

该项显然良类型，因为在应用 $y\ 5$ 中，T-Sub 可把 y 的类型提升为 $\text{Nat} \rightarrow \text{Nat}$ 。但 y 的最小类型是变量 X ，¹而检查应用需要得到箭头类型。为算出整个应用的最小类型，必须找到 y 所具有的最小箭头类型，也就是变量 X 的最小箭头超类型。做法是沿着上界反复提升 X ，直到结果不再是类型变量。

形式上，判断 $\Gamma \vdash_A S \uparrow T$ 读作“在 Γ 下， S 暴露 (*exposure*) 为 T ”；它表示 T 是 S 的最小非变量超类型。暴露就是反复提升类型变量，如图28-1所示。

$\rightarrow, \forall, <: \text{Top}$

$\Gamma \vdash_A S \uparrow T$

暴露关系

$$\frac{X <: U \in \Gamma \quad \Gamma \vdash_A U \uparrow T}{\Gamma \vdash_A X \uparrow T} \text{XA-PROMOTE} \qquad \frac{S \text{ 不是类型变量}}{\Gamma \vdash_A S \uparrow S} \text{XA-OTHER}$$

图 28-1: $F_{<}$ 的暴露算法

¹本章研究纯 $F_{<}$ (图26-1)。作者相应的实现是 `purefsub`; `fullfsub` 还包括存在类型 (图24-1) 和第十一章的若干扩展。

这些规则定义了一个全函数：良构上下文中的变量上界只能引用更早的变量，因此沿上界反复查找必会遇到非变量类型。暴露结果也是具有某种非变量外形的最小超类型。例如，若

$$\Gamma = X <: \text{Top}, Y <: \text{Nat} \rightarrow \text{Nat}, Z <: Y, W <: Z,$$

则有五个判断：

$$\begin{aligned} &\Gamma \vdash_{\mathbf{A}} \text{Top} \uparrow \text{Top}, \quad \Gamma \vdash_{\mathbf{A}} Y \uparrow \text{Nat} \rightarrow \text{Nat}, \quad \Gamma \vdash_{\mathbf{A}} W \uparrow \text{Nat} \rightarrow \text{Nat}, \\ &\Gamma \vdash_{\mathbf{A}} X \uparrow \text{Top}, \quad \Gamma \vdash_{\mathbf{A}} Z \uparrow \text{Nat} \rightarrow \text{Nat} \end{aligned}$$

引理 28.1.1 (暴露). 若 $\Gamma \vdash_{\mathbf{A}} S \uparrow T$ ，则：

1. $\Gamma \vdash S <: T$;
2. 若 $\Gamma \vdash S <: U$ 且 U 不是变量，则 $\Gamma \vdash T <: U$ 。

证明. 第 1 项对 $\Gamma \vdash_{\mathbf{A}} S \uparrow T$ 的推导归纳；第 2 项对 $\Gamma \vdash S <: U$ 的推导归纳。 $\square$

28.2 最小赋型

这里所说的最小赋型 (*minimal typing*)，是指为一项计算其最小类型。算法沿用带子类型简单类型 lambda 演算的思路，只增加一项处理：检查项应用时，先计算左侧项的最小类型，再把它暴露成箭头类型。若暴露结果不是箭头，TA-App 无法使用，该应用便不良类型。类型应用同理，要把左侧项的最小类型暴露成全称类型。完整规则见图 28-2。

$\rightarrow, \forall, <: \text{Top}$ 扩展 $\lambda_{\rightarrow}$ (图 16-3)
 $\Gamma \vdash_{\mathbf{A}} t : T$ 算法赋型关系

$$\begin{array}{c} \frac{x : T \in \Gamma}{\Gamma \vdash_{\mathbf{A}} x : T} \text{TA-VAR} \qquad \frac{\Gamma, x : T_1 \vdash_{\mathbf{A}} t_2 : T_2}{\Gamma \vdash_{\mathbf{A}} \lambda x : T_1. t_2 : T_1 \rightarrow T_2} \text{TA-ABS} \\[10pt] \frac{\Gamma \vdash_{\mathbf{A}} t_1 : T_1 \quad \Gamma \vdash_{\mathbf{A}} T_1 \uparrow T_{11} \rightarrow T_{12} \quad \Gamma \vdash_{\mathbf{A}} t_2 : T_2 \quad \Gamma \vdash_{\mathbf{A}} T_2 <: T_{11}}{\Gamma \vdash_{\mathbf{A}} t_1 t_2 : T_{12}} \text{TA-APP} \\[10pt] \frac{\Gamma, X <: T_1 \vdash_{\mathbf{A}} t_2 : T_2}{\Gamma \vdash_{\mathbf{A}} \lambda X <: T_1. t_2 : \forall X <: T_1. T_2} \text{TA-TABS} \\[10pt] \frac{\Gamma \vdash_{\mathbf{A}} t_1 : T_1 \quad \Gamma \vdash_{\mathbf{A}} T_1 \uparrow \forall X <: T_{11}. T_{12} \quad \Gamma \vdash_{\mathbf{A}} T_2 <: T_{11}}{\Gamma \vdash_{\mathbf{A}} t_1 [T_2] : [X \mapsto T_2] T_{12}} \text{TA-TAPP} \end{array}$$

图 28-2: $F_{<}$ 的算法赋型规则

下面证明该算法相对于原始赋型规则可靠且完备。这里只写核 $F_{<}$ 的证明；完全系统的相应修改见练习 28.2.3。

定理 28.2.1 (最小赋型).

1. 若 $\Gamma \vdash_A t : T$, 则 $\Gamma \vdash t : T$.
2. 若 $\Gamma \vdash t : T$, 则存在 M 使 $\Gamma \vdash_A t : M$ 且 $\Gamma \vdash M <: T$.

证明. 第 1 项对算法赋型推导归纳. 在应用和类型应用情形, 用 [引理28.1.1](#)第 1 项把左侧项的最小类型提升为暴露结果; 其余前提直接使用归纳假设.

第 2 项对 $\Gamma \vdash t : T$ 的推导归纳, 并按最后一条规则分类.

情形 T-Var. 此时 $t = x$, 且 $x : T \in \Gamma$. TA-Var 给出 $\Gamma \vdash_A x : T$, S-Refl 给出 $\Gamma \vdash T <: T$.

情形 T-Abs. 此时 $t = \lambda x : T_1. t_2$, $T = T_1 \rightarrow T_2$, 且 $\Gamma, x : T_1 \vdash t_2 : T_2$. 归纳假设给出某个 M_2 , 满足

$$\Gamma, x : T_1 \vdash_A t_2 : M_2, \quad \Gamma, x : T_1 \vdash M_2 <: T_2.$$

子类型判断不使用项变量绑定 ([引理26.4.4](#)), 故后一判断等价于 $\Gamma \vdash M_2 <: T_2$. TA-Abs 得到 $\Gamma \vdash_A t : T_1 \rightarrow M_2$, 再由 S-Refl 与 S-Arrow 得 $\Gamma \vdash T_1 \rightarrow M_2 <: T_1 \rightarrow T_2$.

情形 T-App. 此时 $t = t_1 t_2$, $T = T_{12}$, 前提为

$$\Gamma \vdash t_1 : T_{11} \rightarrow T_{12}, \quad \Gamma \vdash t_2 : T_{11}.$$

归纳假设给出 $\Gamma \vdash_A t_1 : M_1$ 与 $\Gamma \vdash_A t_2 : M_2$, 并有

$$\Gamma \vdash M_1 <: T_{11} \rightarrow T_{12}, \quad \Gamma \vdash M_2 <: T_{11}.$$

令 N_1 为 M_1 的最小非变量超类型, 即 $\Gamma \vdash_A M_1 \uparrow N_1$. 由 [引理28.1.1](#)第 2 项, $\Gamma \vdash N_1 <: T_{11} \rightarrow T_{12}$. N_1 不是变量, 故 [引理26.4.10](#)说明 $N_1 = N_{11} \rightarrow N_{12}$, 且

$$\Gamma \vdash T_{11} <: N_{11}, \quad \Gamma \vdash N_{12} <: T_{12}.$$

由传递性有 $\Gamma \vdash M_2 <: N_{11}$, 所以 TA-App 得到 $\Gamma \vdash_A t_1 t_2 : N_{12}$, 而 N_{12} 正是所需的 T_{12} 的子类型.

情形 T-TAbs. 此时 $t = \lambda X <: T_1. t_2$, $T = \forall X <: T_1. T_2$, 且 $\Gamma, X <: T_1 \vdash t_2 : T_2$. 归纳假设给出某个 M_2 , 满足

$$\Gamma, X <: T_1 \vdash_A t_2 : M_2, \quad \Gamma, X <: T_1 \vdash M_2 <: T_2.$$

TA-TAbs 得到最小类型 $\forall X <: T_1. M_2$, S-All 则给出它是原类型的子类型.

情形 T-TApp. 此时 $t = t_1 [T_2]$, $T = [X \mapsto T_2] T_{12}$, 且

$$\Gamma \vdash t_1 : \forall X <: T_{11}. T_{12}, \quad \Gamma \vdash T_2 <: T_{11}.$$

归纳假设给出 $\Gamma \vdash_A t_1 : M_1$ 和 $\Gamma \vdash M_1 <: \forall X <: T_{11}. T_{12}$. 令 $\Gamma \vdash_A M_1 \uparrow N_1$. 暴露引理与 [引理26.4.10](#)说明

$$N_1 = \forall X <: T_{11}. N_{12}, \quad \Gamma, X <: T_{11} \vdash N_{12} <: T_{12}.$$

TA-TApp 得到 $\Gamma \vdash_A t_1[T_2] : [X \mapsto T_2]N_{12}$ ；再由 [引理26.4.8](#)可知该结果类型是 $[X \mapsto T_2]T_{12}$ 的子类型。

情形 T-Sub. 归纳假设给出某个 M ，满足 $\Gamma \vdash_A t : M$ 和 $\Gamma \vdash M <: S$ ；与原前提 $\Gamma \vdash S <: T$ 使用传递性，得到 $\Gamma \vdash M <: T$ 。 $\square$

推论 28.2.2 (赋型可判定性). 若子类型关系可判定，则核 $F_{<}$ 的赋型关系可判定。

证明. 对任意 Γ, t ，按算法规则尝试构造某个 $\Gamma \vdash_A t : T$ 。算法规则由项的外层语法唯一选择，而且递归调用总在真子项上。算法成功时由 [定理28.2.1](#)第 1 项得到声明式赋型；失败时第 2 项保证不存在声明式类型。最后，算法子类型关系若可判定，全部前提也都可以有效检查。 $\square$

练习 28.2.3 (★★). 上述证明推广到完全 $F_{<}$ 时，哪些位置需要改变？

[查看原书附录 A 中的 28.2.3 解答](#)

28.3 核 $F_{<}$ 的子类型算法

[§16.1](#)指出，带子类型简单类型 lambda 演算的声明式子类型关系不是语法定向的，有两个原因。第一，S-Refl 和 S-Trans 的结论与其他规则重叠；自下而上读取规则时，无法决定应试用哪一条。第二，S-Trans 的前提含有未出现在结论中的元变量，朴素算法必须猜测它。简单类型系统可以删去这两条规则；不过在此之前，还要先把三条分立的记录子类型规则合并成一条。

核 $F_{<}$ 的情况类似。有问题的仍是 S-Refl 和 S-Trans；删去它们之后，还需略微调整余下的规则，以保留这两条被删规则的必要用法。

在带子类型的简单类型 lambda 演算中，删除 S-Refl 不会损失可导判断；这里却不同， $\Gamma \vdash X <: X$ 只能由反身性得到。因此要保留一个只适用于类型变量的反身规则。

$$\frac{}{\Gamma \vdash_A X <: X} \text{SA-REFL-TVAR}$$

删除 S-Trans 前还必须处理它与 S-TVar 的必要交互。例如令 $\Gamma = W <: \text{Top}, X <: W, Y <: X, Z <: Y$ 。不借助传递性，就无法从逐级上界推出 $\Gamma \vdash Z <: W$ 。其中不能消除的基本片段是“先用 S-TVar 查到变量的直接上界，再用传递性继续向上”：

$$\frac{\frac{Z <: Y \in \Gamma}{\Gamma \vdash Z <: Y} \text{S-TVAR} \quad \Gamma \vdash Y <: W}{\Gamma \vdash Z <: W} \text{S-TRANS}$$

幸好，这种形式的推导正是传递性的全部必要用法。新的 SA-Trans-TVar 把“查找变量紧接着使用传递性”这一模式合成一条规则；稍后将证明，用它替换 S-TVar 和 S-Trans，不会改变可导子类型判断的集合。

$$\frac{X <: U \in \Gamma \quad \Gamma \vdash_A U <: T}{\Gamma \vdash_A X <: T} \text{SA-TRANS-TVAR}$$

这些改动得到图28-3中的核 $F_{<}$: 算法子类型关系。我们在转门上加一个箭头, 以便在同时讨论两种关系时, 区分算法子类型判断和原始的声明式子类型判断。

$\rightarrow, \forall, <: \text{Top}$ 扩展 $\lambda_{\rightarrow}$ (图16-2)
 $\Gamma \vdash_A S <: T$ 算法子类型关系

$$\begin{array}{c}
 \frac{}{\Gamma \vdash_A X <: X} \text{SA-REFL-TVAR} \qquad \frac{X <: U \in \Gamma \quad \Gamma \vdash_A U <: T}{\Gamma \vdash_A X <: T} \text{SA-TRANS-TVAR} \\
 \\
 \frac{}{\Gamma \vdash_A S <: \text{Top}} \text{SA-TOP} \qquad \frac{\Gamma \vdash_A T_1 <: S_1 \quad \Gamma \vdash_A S_2 <: T_2}{\Gamma \vdash_A S_1 \rightarrow S_2 <: T_1 \rightarrow T_2} \text{SA-ARROW} \\
 \\
 \frac{\Gamma, X <: U \vdash_A S <: T}{\Gamma \vdash_A \forall X <: U. S <: \forall X <: U. T} \text{SA-ALL}
 \end{array}$$

图 28-3: 核 $F_{<}$ 的算法子类型规则

引理 28.3.1 (算法子类型关系的反身性). 对每个良构 Γ, T , 都有 $\Gamma \vdash_A T <: T$ 。

证明. 对类型 T 的结构归纳。变量使用 SA-Refl-TVAr, Top 使用 SA-Top, 箭头与全称类型分别对组成部分使用归纳假设, 再应用 SA-Arrow 或 SA-All。□

引理 28.3.2 (算法子类型关系的传递性). 若 $\Gamma \vdash_A S <: Q$ 且 $\Gamma \vdash_A Q <: T$, 则 $\Gamma \vdash_A S <: T$ 。

证明. 对两份推导大小之和归纳, 并对二者的最后一条规则分类。

若右侧推导最后使用 SA-Top, 目标立即由 SA-Top 得到。若左侧最后使用 SA-Top, 则 $Q = \text{Top}$; 检查算法规则可知右侧也只能以 SA-Top 结束。若任一推导最后使用 SA-Refl-TVAr, 另一份推导本身就是所需结论。

若左侧最后使用 SA-Trans-TVAr, 则 $S = Y, Y <: U \in \Gamma$, 并有更小的子推导 $\Gamma \vdash_A U <: Q$ 。归纳假设把它与右侧推导组合成 $\Gamma \vdash_A U <: T$, 再用 SA-Trans-TVAr 得到 $\Gamma \vdash_A Y <: T$ 。

若左侧最后使用 SA-Arrow, 则 $S = S_1 \rightarrow S_2, Q = Q_1 \rightarrow Q_2$ 。已经处理右侧为 SA-Top 的情形, 所以右侧只能也以 SA-Arrow 结束, 且 $T = T_1 \rightarrow T_2$ 。四个子推导分别给出

$$\Gamma \vdash_A Q_1 <: S_1, \quad \Gamma \vdash_A S_2 <: Q_2, \quad \Gamma \vdash_A T_1 <: Q_1, \quad \Gamma \vdash_A Q_2 <: T_2.$$

两次应用归纳假设得到参数与结果所需的传递结论, 再应用 SA-Arrow。

若左侧最后使用 SA-All, 则 $S = \forall X <: U_1. S_2, Q = \forall X <: U_1. Q_2$; 右侧同样只能使用 SA-All, 故 $T = \forall X <: U_1. T_2$ 。在共同上下文 $\Gamma, X <: U_1$ 下, 把两份量词体子推导用归纳假设组合, 再应用 SA-All。□

定理 28.3.3 (算法子类型的可靠性与完备性).

$$\Gamma \vdash S <: T \iff \Gamma \vdash_A S <: T.$$

证明. 两个方向都对推导归纳。由右到左的可靠性直接把算法规则翻译为声明式规则；由左到右的完备性在声明式 S-Refl 与 S-Trans 情形分别使用 引理28.3.1和引理28.3.2，其余情形直接使用归纳假设。□

定义 28.3.4. 类型 T 在上下文 Γ 中的权重写作 $\text{weight}_\Gamma(T)$ ，递归定义为

$$\begin{aligned} \text{weight}_\Gamma(\text{Top}) &= 1, \\ \text{weight}_\Gamma(X) &= 1 + \text{weight}_{\Gamma_1}(U) && \text{若 } \Gamma = \Gamma_1, X <: U, \Gamma_2, \\ \text{weight}_\Gamma(S \rightarrow T) &= 1 + \text{weight}_\Gamma(S) + \text{weight}_\Gamma(T), \\ \text{weight}_\Gamma(\forall X <: S. T) &= 1 + \text{weight}_{\Gamma, X <: S}(T). \end{aligned}$$

子类型判断 $\Gamma \vdash_A S <: T$ 的权重是

$$\text{weight}_\Gamma(S) + \text{weight}_\Gamma(T).$$

这项递归定义良好：每次递归调用时，上下文中所有类型的总大小，加上当前正在定义权重的类型大小，总会严格减小。

定理 28.3.5. 核 $F_{<}$ 子类型算法对所有输入都会终止。

证明. 逐条检查算法规则：每个前提的权重都严格小于结论。尤其 SA-Trans-TVar 沿变量的上界移动，而良构上下文中的上界只引用更早的变量。□

推论 28.3.6. 核 $F_{<}$ 的子类型关系可判定。

28.4 完全 $F_{<}$ 的子类型

完全系统的算法只把 SA-All 换成允许上界逆变的版本。反身性仍与核系统相同，其规则见图28-4。算法关系相对于声明式关系的可靠性与完备性，仍可由反身性和传递性推出。

$\rightarrow, \forall, <: \text{Top}$ *full*

扩展 图28-3

$\Gamma \vdash_A S <: T$

算法子类型关系

$$\begin{array}{c} \frac{}{\Gamma \vdash_A X <: X} \text{SA-REFL-TVAR} \qquad \frac{X <: U \in \Gamma \quad \Gamma \vdash_A U <: T}{\Gamma \vdash_A X <: T} \text{SA-TRANS-TVAR} \\[10pt] \frac{}{\Gamma \vdash_A S <: \text{Top}} \text{SA-TOP} \qquad \frac{\Gamma \vdash_A T_1 <: S_1 \quad \Gamma \vdash_A S_2 <: T_2}{\Gamma \vdash_A S_1 \rightarrow S_2 <: T_1 \rightarrow T_2} \text{SA-ARROW} \\[10pt] \frac{\Gamma \vdash_A T_1 <: S_1 \quad \Gamma, X <: T_1 \vdash_A S_2 <: T_2}{\Gamma \vdash_A \forall X <: S_1. S_2 <: \forall X <: T_1. T_2} \text{SA-ALL} \end{array}$$

图 28-4: 完全 $F_{<}$ 的算法子类型规则

反身性的证明与核系统相同，传递性却更微妙。设有两份以 SA-All 结束的推导，其结论为

$$\Gamma \vdash_A \forall X <: S_1.S_2 <: \forall X <: Q_1.Q_2$$

和

$$\Gamma \vdash_A \forall X <: Q_1.Q_2 <: \forall X <: T_1.T_2.$$

它们分别包含四个子推导：

$$\Gamma \vdash_A Q_1 <: S_1, \quad \Gamma, X <: Q_1 \vdash_A S_2 <: Q_2,$$

$$\Gamma \vdash_A T_1 <: Q_1, \quad \Gamma, X <: T_1 \vdash_A Q_2 <: T_2.$$

对两个上界子推导使用传递性的归纳假设没有问题，可以得到 $\Gamma \vdash_A T_1 <: S_1$ 。但两个量词体子推导的上下文不同：一个把 X 的上界写成 Q_1 ，另一个写成 T_1 ，无法直接套用同一个传递性归纳假设。

第二十六章的收窄性质提供了对齐上下文的办法。由 $\Gamma \vdash_A T_1 <: Q_1$ ，可把第一份量词体推导收窄为 $\Gamma, X <: T_1 \vdash_A S_2 <: Q_2$ ，再与第二份组合。但这里不能直接引用声明式系统的收窄引理：它只保证构造出结论经收窄的新推导，不保证新推导与原推导一样大。实际上，每当 S-TVar 查找被收窄的变量时，证明都要接入一份可能任意大的推导，所以结果通常更大；这项拼接还会创建新的 S-Trans，而当前目的正是证明没有 S-Trans 的算法关系仍具有传递性。因此，必须把算法关系的传递性和收窄性放在同一个归纳中证明。

引理 28.4.1 (排列与弱化).

1. 设 Γ' 是 Γ 的良构排列。若 $\Gamma \vdash_A S <: T$ ，则 $\Gamma' \vdash_A S <: T$ 。
2. 若 $\Gamma \vdash_A S <: T$ 且 $\text{dom}(\Gamma) \cap \text{dom}(\Delta) = \emptyset$ ，则 $\Gamma, \Delta \vdash_A S <: T$ 。

证明. 均为常规归纳；第 2 项的 SA-All 情形使用第 1 项调整新绑定的次序。 $\square$

引理 28.4.2 (完全 $F_{<}$ 的传递性与收窄).

1. 若 $\Gamma \vdash_A S <: Q$ 且 $\Gamma \vdash_A Q <: T$ ，则 $\Gamma \vdash_A S <: T$ 。
2. 若 $\Gamma, X <: Q, \Delta \vdash_A M <: N$ 且 $\Gamma \vdash_A P <: Q$ ，则 $\Gamma, X <: P, \Delta \vdash_A M <: N$ 。

证明. 两项同时对 Q 的大小归纳。每一层先证明第 1 项，再证明第 2 项；证明当前 Q 的第 2 项时可以使用刚刚得到的传递性，而证明第 1 项只会调用严格小于 Q 的上界所对应的收窄结论。

第 1 项再对第一份推导的大小作内层归纳，并对两份推导的最后规则分类。SA-Top、SA-Refl-TVar 与 SA-Trans-TVar 情形和核系统相同。若两边都以 SA-Arrow 结束，中间类型 $Q = Q_1 \rightarrow Q_2$ ，分别对严格更小的 Q_1, Q_2 使用外层归纳假设，再应用 SA-Arrow。

关键是两边都以 SA-All 结束。此时 $Q = \forall X <: Q_1.Q_2$ ，并有上文列出的四个子推导。先对 Q_1 使用第 1 项，得到 $\Gamma \vdash_A T_1 <: S_1$ 。再对严格更小的 Q_1 使用第 2 项，把

$\Gamma, X <: Q_1 \vdash_A S_2 <: Q_2$ 收窄为 $\Gamma, X <: T_1 \vdash_A S_2 <: Q_2$ 。现在两个量词体判断具有相同上下文；对严格更小的 Q_2 使用第 1 项，得到 $\Gamma, X <: T_1 \vdash_A S_2 <: T_2$ 。最后应用 SA-All。

第 2 项对待收窄推导的大小作内层归纳。多数规则直接使用归纳假设。关键情形是最后使用 SA-Trans-TVar，且被查找的变量恰为 X 。原推导含子推导 $\Gamma, X <: Q, \Delta \vdash_A Q <: N$ 。内层归纳假设把它收窄为 $\Gamma, X <: P, \Delta \vdash_A Q <: N$ ；弱化又把前提 $\Gamma \vdash_A P <: Q$ 放入同一上下文。使用当前 Q 已经证明的第 1 项，得到 $P <: N$ ，再由 SA-Trans-TVar 推出 $X <: N$ 。□

练习 28.4.3 (★★★, $\Rightarrow$)。考虑一种介于核规则与完全规则之间的量词子类型规则：

$$\frac{\Gamma \vdash S_1 <: T_1 \quad \Gamma \vdash T_1 <: S_1 \quad \Gamma, X <: T_1 \vdash S_2 <: T_2}{\Gamma \vdash \forall X <: S_1.S_2 <: \forall X <: T_1.T_2} \text{S-ALL}$$

它不要求两个上界在语法上完全相同，只要求二者在子类型关系下等价。只有加入能产生非平凡等价类的构造后，它才和核规则表现不同。例如，带记录的纯核 $F_{<}$ 不能推出

$$\forall X <: \{a : \text{Top}, b : \text{Top}\}.X <: \forall X <: \{b : \text{Top}, a : \text{Top}\}.X,$$

而上述规则可以。采用这条规则的系统，其子类型关系是否可判定？

[查看练习 28.4.3 的译者参考解答](#)

可靠性与完备性仍可由反身性和传递性推出。然而，核系统的权重终止证明不能推广到完全系统。

28.5 完全 $F_{<}$ 的不可判定性

上一节已经证明完全 $F_{<}$ 的算法子类型规则可靠且完备，也就是它们闭合生成的最小关系与原声明式规则生成的关系相同。剩下的问题是：按这些规则实现的过程是否对所有输入都终止？答案是否定的；这一结论刚发现时颇出人意料。

练习 28.5.1 (★). 完全 $F_{<}$ 的算法规则不能保证总是终止；核系统的终止证明究竟在哪一步无法推广？

[查看原书附录 A 中的 28.5.1 解答](#)

Ghelli (1995) 给出了一个使子类型过程发散的例子。先定义缩写

$$\neg S \stackrel{\text{def}}{=} \forall X <: S.X.$$

这个“否定”操作的关键性质，是把子类型判断的左右两侧交换。

事实 28.5.2.

$$\Gamma \vdash \neg S <: \neg T \iff \Gamma \vdash T <: S.$$

证明练习 ($\star\star, \rightarrow$)。

查看练习 28.5.2 的译者参考解答

定义

$$T = \forall X <: \text{Top}. \neg(\forall Y <: X. \neg Y).$$

若按算法规则自下而上尝试构造

$$X_0 <: T \vdash_A X_0 <: \forall X_1 <: X_0. \neg X_1,$$

便会得到不断增大的子目标:

$$\begin{aligned} X_0 <: T &\vdash_A X_0 <: \forall X_1 <: X_0. \neg X_1 \\ X_0 <: T &\vdash_A \forall X_1 <: \text{Top}. \neg(\forall X_2 <: X_1. \neg X_2) <: \forall X_1 <: X_0. \neg X_1 \\ X_0 <: T, X_1 <: X_0 &\vdash_A \neg(\forall X_2 <: X_1. \neg X_2) <: \neg X_1 \\ X_0 <: T, X_1 <: X_0 &\vdash_A X_1 <: \forall X_2 <: X_1. \neg X_2 \\ X_0 <: T, X_1 <: X_0 &\vdash_A X_0 <: \forall X_2 <: X_1. \neg X_2 \\ &\vdots \end{aligned}$$

这里默默完成了加入新鲜类型变量时为保持上下文良构所需的重新命名，并特意选择名称来显示循环模式。关键步骤是第二行到第三行发生的“重新定界”：左侧 X_1 的上界从 Top 改成 X_0 。而第二行的整个左侧又恰好是 X_0 的上界，于是产生循环；每轮都要穿过上下文中更长的变量链。这个例子只利用 $\neg$ 的句法效果，不应把它理解为具有通常逻辑否定的语义。

问题还不止这一种算法会在某些输入上发散。Pierce (1994) 证明，不存在既对原完全 $F_{<}$ 子类型关系可靠、完备，又对所有输入终止的算法。完整证明篇幅过长，本书只用下面的编码概述其思路。

定义 28.5.3. 类型中位置的正负性递归定义如下：整个类型处于正位置；在箭头 $T_1 \rightarrow T_2$ 中，参数位置翻转正负，结果位置保持；在 $\forall X <: T_1. T_2$ 中，上界位置翻转，量词体保持。对于子类型判断 $\Gamma \vdash S <: T$ ，左侧 S 和上下文中的各上界处于负位置，右侧 T 处于正位置。

“正”“负”来自逻辑。按 Curry-Howard 对应 (§9.4)， $S \rightarrow T$ 对应蕴含 $S \Rightarrow T$ ；在经典逻辑中，它等价于 $\neg S \vee T$ 。因此，整个蕴含位于偶数层否定之内时，子命题 S 恰好位于奇数层否定之内，即负位置。还要注意， T 中的正出现，在 $\neg T$ 中对应负出现。

事实 28.5.4. 若 X 在 S 中只正出现、在 T 中只负出现，则

$$X <: U \vdash S <: T \iff \vdash [X \mapsto U]S <: [X \mapsto U]T.$$

证明练习 ($\star\star, \rightarrow$)。

查看练习 28.5.4 的译者参考解答

下面令

$$T = \forall X_0 <: \text{Top}. \forall X_1 <: \text{Top}. \forall X_2 <: \text{Top}. \neg(\forall Y_0 <: X_0. \forall Y_1 <: X_1. \forall Y_2 <: X_2. \neg X_0)$$

并考察判断

$$\begin{aligned} & \vdash T <: \forall X_0 <: T. \forall X_1 <: P. \forall X_2 <: Q. \\ & \neg(\forall Y_0 <: \text{Top}. \forall Y_1 <: \text{Top}. \forall Y_2 <: \text{Top}. \\ & \neg(\forall Z_0 <: Y_0. \forall Z_1 <: Y_2. \forall Z_2 <: Y_1. U)). \end{aligned}$$

可以把它看作一台简单计算机的状态。 X_1, X_2 是两个“寄存器”，当前内容分别为类型 P, Q 。第三行开始是“指令流”：第一条指令编码在 Z_1, Z_2 的上界 Y_2, Y_1 中，次序特意交换；未指定的 U 表示余下指令。类型 T 、嵌套否定以及变量 X_0, Y_0 的作用与前一个例子相同：它们使一次规则展开后重新得到与原目标同形的子目标；每展开一轮，就模拟机器的一步。

这里指令流开头编码的是“交换寄存器 1 和 2”。用前两个事实逐步变换，可得

$$\begin{aligned} & \vdash T <: \forall X_0 <: T. \forall X_1 <: P. \forall X_2 <: Q. \neg A \\ \iff & \vdash \neg(\forall Y_0 <: T. \forall Y_1 <: P. \forall Y_2 <: Q. \neg T) <: \neg A & \text{由事实 28.5.4} \\ \iff & \vdash A <: \forall Y_0 <: T. \forall Y_1 <: P. \forall Y_2 <: Q. \neg T & \text{由事实 28.5.2} \\ \iff & \vdash \neg(\forall Z_0 <: T. \forall Z_1 <: Q. \forall Z_2 <: P. U) <: \neg T & \text{由事实 28.5.4} \\ \iff & \vdash T <: \forall Z_0 <: T. \forall Z_1 <: Q. \forall Z_2 <: P. U & \text{由事实 28.5.2,} \end{aligned}$$

其中

$$A = \forall Y_0 <: \text{Top}. \forall Y_1 <: \text{Top}. \forall Y_2 <: \text{Top}. \neg(\forall Z_0 <: Y_0. \forall Z_1 <: Y_2. \forall Z_2 <: Y_1. U).$$

最后不仅 P, Q 已交换，完成交换的指令也已被消耗， U 来到指令流最前端，等待下一轮“执行”。若让 U 再以同一交换指令开头：

$$U = \neg(\forall Y_0 <: \text{Top}. \forall Y_1 <: \text{Top}. \forall Y_2 <: \text{Top}. \neg(\forall Z_0 <: Y_0. \forall Z_1 <: Y_2. \forall Z_2 <: Y_1. U')),$$

就能再次交换并恢复两个寄存器，随后继续执行 U' 。也可以选择不同的 U 。例如令

$$U = \neg(\forall Y_0 <: \text{Top}. \forall Y_1 <: \text{Top}. \forall Y_2 <: \text{Top}. \neg(\forall Z_0 <: Y_0. \forall Z_1 <: Y_1. \forall Z_2 <: Y_2. Y_1)),$$

下一轮会把寄存器 1 当前的值 Q 放到 U 的位置，相当于通过寄存器 1 间接跳转到 Q 表示的指令流。推广这项技巧还能编码条件、后继、前驱和零测试。

综合这些构造，可把两计数器机约化到子类型判断。两计数器机由有限控制器和两个保存自然数的计数器组成，是普通图灵机的一种简单变体。

定理 28.5.5 (Pierce, 1994). 对每台两计数器机 M ，都能构造子类型判断 $S(M)$ ，使 $S(M)$ 可导出当且仅当 M 的执行会停机。

因此，若完全 $F_{<}$ 子类型可判定，就能判定两计数器机停机问题，矛盾。两计数器机的停机问题不可判定（见 Hopcroft 与 Ullman, 1979），所以完全 $F_{<}$ 的子类型问题也不可判定。

这里的不可判定性并不意味着§28.4的子类型半算法不可靠或不完备。若声明式规则可以证明 $\Gamma \vdash S <: T$ ，该过程必定终止并返回真；若不能证明，它可能返回假，也可能发散。不

可导判断有两种失败方式：一是生成无穷子目标链，意味着不存在以该结论收尾的有限推导；二是到达 $\text{Top} <: S \rightarrow T$ 这类明显不可能的目标。过程能识别后一种失败，却不能识别前一种。因此它仍然可靠且完备，只是对否定实例不保证终止。

这是否使完全 $F_{<}$ 在实践中毫无用处？通常认为，单就不可判定性本身而言，问题没有那么严重。Ghelli (1995) 证明，要使检查器发散，目标必须同时具有三项相当特殊、程序员不容易偶然写出的性质。此外，一些常用语言的类型检查或类型重构在理论上也可能代价极高，例如§22.7 提到的 ML 与 Haskell；C++ 和 λProlog 的相应问题甚至不可判定 (Felty、Gunter、Hannan、Miller、Nadathur 与 Scedrov, 1988)。实际经验表明，下一节所述缺少并与交的问题（见[练习28.6.3](#)）比不可判定性更棘手。

练习 28.5.6 (***). 研究同时提供有界与无界量词、但没有 Top 的系统；也就是说，变量既可以有 $X <: T$ 形式的绑定，也可以有无上界的 X 绑定。这个变体称为 完全有界量化 (*completely bounded quantification*)。

1. 定义这个系统；
2. 证明其子类型关系可判定；
3. 这一限制是否真正解决了本节提出的根本问题？特别是，当语言还包含数字、记录、变体等构造时，结论是否仍然成立？

[查看原书附录 A 中的 28.5.6 解答](#)

28.6 并与交

§16.3指出，带子类型的语言最好能为每对类型 S, T 找到并，也就是二者所有公共超类型中的最小者。本节用算法证明：核 $F_{<}$ 的每对类型都有并；只要一对类型至少有一个公共子类型，它们也有交。完全 $F_{<}$ 不具备这两项性质（见[练习28.6.3](#)）。

用

$$\Gamma \vdash S \vee T = J \quad \text{和} \quad \Gamma \vdash S \wedge T = M.$$

分别表示“在上下文 Γ 中， J 是 S, T 的并”和“ M 是 S, T 的交”。[图28-5](#)同时递归定义这两个算法。同一输入有时会符合多项条件；把定义作为确定算法读取时，总是选最先适用的一项。

这两个定义都是全函数：并总返回一个类型；交要么返回类型，要么明确失败。终止性来自[定义28.3.4](#)的总权重：每次递归调用都会严格减小 S, T 相对于 Γ 的总权重。下面再验证算法算出的确实是并与交。证明分两步：[命题28.6.1](#)说明算法结果分别是公共上界和公共下界；[命题28.6.2](#)再说明并小于每个公共上界，而交大于每个公共下界，并且只要公共下界存在，交算法就会成功。

$\rightarrow, \forall, <: \text{Top}$

$$\Gamma \vdash S \vee T = \left\{ \begin{array}{ll} T & \Gamma \vdash_A S <: T \text{ 时,} \\ S & \Gamma \vdash_A T <: S \text{ 时,} \\ J & S = X, X <: U \in \Gamma, \\ & \Gamma \vdash U \vee T = J \text{ 时,} \\ J & T = X, X <: U \in \Gamma, \\ & \Gamma \vdash S \vee U = J \text{ 时,} \\ M_1 \rightarrow J_2 & S = S_1 \rightarrow S_2, T = T_1 \rightarrow T_2, \\ & \Gamma \vdash S_1 \wedge T_1 = M_1, \\ & \Gamma \vdash S_2 \vee T_2 = J_2 \text{ 时,} \\ \forall X <: U_1. J_2 & S = \forall X <: U_1. S_2, \\ & T = \forall X <: U_1. T_2, \\ & \Gamma, X <: U_1 \vdash S_2 \vee T_2 = J_2 \text{ 时,} \\ \text{Top} & \text{其他情况。} \end{array} \right.$$

$$\Gamma \vdash S \wedge T = \left\{ \begin{array}{ll} S & \Gamma \vdash_A S <: T \text{ 时,} \\ T & \Gamma \vdash_A T <: S \text{ 时,} \\ J_1 \rightarrow M_2 & S = S_1 \rightarrow S_2, T = T_1 \rightarrow T_2, \\ & \Gamma \vdash S_1 \vee T_1 = J_1, \\ & \Gamma \vdash S_2 \wedge T_2 = M_2 \text{ 时,} \\ \forall X <: U_1. M_2 & S = \forall X <: U_1. S_2, \\ & T = \forall X <: U_1. T_2, \\ & \Gamma, X <: U_1 \vdash S_2 \wedge T_2 = M_2 \text{ 时,} \\ \text{fail} & \text{其他情况。} \end{array} \right.$$

图 28-5: 核 $F_{<}$ 的并与交算法

命题 28.6.1.

1. 若 $\Gamma \vdash S \vee T = J$, 则 $\Gamma \vdash S <: J$ 且 $\Gamma \vdash T <: J$ 。
2. 若 $\Gamma \vdash S \wedge T = M$, 则 $\Gamma \vdash M <: S$ 且 $\Gamma \vdash M <: T$ 。

证明. 对计算 J 或 M 所需的递归调用次数作直接归纳。 □

命题 28.6.2.

1. 若 $\Gamma \vdash S <: V$ 且 $\Gamma \vdash T <: V$, 则存在 J , 使 $\Gamma \vdash S \vee T = J$ 且 $\Gamma \vdash J <: V$ 。
2. 若 $\Gamma \vdash L <: S$ 且 $\Gamma \vdash L <: T$, 则存在 M , 使 $\Gamma \vdash S \wedge T = M$ 且 $\Gamma \vdash L <: M$ 。

证明. 两项同时归纳。第 1 项的归纳对象是两份算法子类型推导 $\Gamma \vdash_A S <: V$ 与 $\Gamma \vdash_A T <: V$ 的大小之和；第 2 项同样对 $\Gamma \vdash_A L <: S$ 与 $\Gamma \vdash_A L <: T$ 的大小之和归纳。定理 28.3.3 保证声明式前提总有这样的算法推导。

先证第 1 项。若任一推导以 SA-Top 结束，则 $V = \text{Top}$ ，结论立即成立。若 $T <: V$ 的推导以 SA-Refl-TVar 结束，则 $T = V$ ；由另一前提已有 $S <: T$ ，并算法第一项返回 T 。 $S <: V$ 以 SA-Refl-TVar 结束的情形对称。

若 $S <: V$ 以 SA-Trans-TVar 结束，则 $S = X$ 、 $X <: U \in \Gamma$ ，并且有较小的子推导 $U <: V$ 。归纳假设给出 $\Gamma \vdash U \vee T = J$ 及 $J <: V$ ，并算法第三项于是给出 $\Gamma \vdash S \vee T = J$ 。另一侧以 SA-Trans-TVar 结束的情形对称。

其余可能是两边都以 SA-Arrow 或都以 SA-All 结束。箭头情形中，写 $S = S_1 \rightarrow S_2$ 、 $T = T_1 \rightarrow T_2$ 、 $V = V_1 \rightarrow V_2$ 。归纳假设第 2 项给出参数的交 M_1 ，且 $V_1 <: M_1$ ；第 1 项给出结果的并 J_2 ，且 $J_2 <: V_2$ 。并算法返回 $M_1 \rightarrow J_2$ ，再由 SA-Arrow 得到 $M_1 \rightarrow J_2 <: V_1 \rightarrow V_2$ 。全称类型情形的共同上界具有相同的上界 U_1 ；在 $\Gamma, X <: U_1$ 中对子类型体使用第 1 项，再应用 SA-All。

第 2 项类似。若 $L <: T$ 以 SA-Top 结束，则 $T = \text{Top}$ ，交算法第一项返回 S ，而另一前提已经给出 $L <: S$ ；另一侧对称。若任一推导以 SA-Refl-TVar 结束，则 L 就等于相应一侧，另一前提使交算法直接返回该侧。

若两份推导都以 SA-Trans-TVar 结束，则 $L = X$ 、 $X <: U \in \Gamma$ ，并有两个较小的子推导 $\Gamma \vdash_A U <: S$ 与 $\Gamma \vdash_A U <: T$ 。归纳假设第 2 项给出交 M 及 $\Gamma \vdash_A U <: M$ ；结合上下文中的绑定 $X <: U$ ，直接应用 SA-Trans-TVar，得到 $\Gamma \vdash_A L <: M$ 。箭头情形中，写 $L = L_1 \rightarrow L_2$ ；归纳假设第 1 项给出参数的并 J_1 以及 $J_1 <: L_1$ ，第 2 项给出结果的交 M_2 以及 $L_2 <: M_2$ 。交算法返回 $J_1 \rightarrow M_2$ ，SA-Arrow 给出 $L_1 \rightarrow L_2 <: J_1 \rightarrow M_2$ 。两份推导都以 SA-All 结束的情形相同：在共同上界扩展的上下文中对量词体使用第 2 项，再应用 SA-All。以上各情形先得到算法子类型判断，再由 定理 28.3.3 的可靠性得到命题所要求的声明式判断。□

练习 28.6.3 (推荐, ★★). 考虑 Ghelli (1990) 给出的类型

$$S = \forall X <: Y \rightarrow Z. Y \rightarrow Z, \quad T = \forall X <: Y' \rightarrow Z'. Y' \rightarrow Z'$$

以及上下文

$$\Gamma = Y <: \text{Top}, Z <: \text{Top}, Y' <: Y, Z' <: Z.$$

1. 在完全 $F_{<}$ 中， S, T 在 Γ 下共有多少个公共子类型？
2. 证明它们在 Γ 下没有交。
3. 找出一对在 Γ 下没有并的类型。

查看原书附录 A 中的 28.6.3 解答

28.7 有界存在类型

把核 $F_{<}$ 的类型检查算法扩展到有界存在类型时，还要处理一项细节。先回顾声明式的存在类型消去规则：

$$\frac{\Gamma \vdash t_1 : \{\exists X <: T_{11}, T_{12}\} \quad \Gamma, X <: T_{11}, x : T_{12} \vdash t_2 : T_2}{\Gamma \vdash \mathbf{let} \{X, x\} = t_1 \mathbf{in} t_2 : T_2} \text{T-UNPACK}$$

正如§24.1所述，第二个前提在包含 X 的上下文中计算 t_2 的类型，结论的上下文却没有 X 。因此 T_2 不能自由出现 X ，否则结论中的这些 X 会越出作用域。§25.5 从无名 de Bruijn 表示的角度说明过同一现象：上下文从前提变到结论，对应把 T_2 中的变量索引下移一位；若 T_2 自由含有 X ，这次下移会失败。

这对存在类型的最小赋型算法有什么影响？考虑 $t = \mathbf{let} \{X, x\} = p \mathbf{in} x$ ，其中

$$p : \{\exists X, \text{Nat} \rightarrow X\},$$

包体 x 最自然的类型是 $\text{Nat} \rightarrow X$ ，其中含有受约束的类型变量 X 。不过，在带 T-Sub 的声明式系统中， x 还可具有 $\text{Nat} \rightarrow \text{Top}$ 和 Top ；二者都不含 X ，所以整项 t 可以合法地取得这两种类型。

一般地，应用 T-Unpack 前，总可以把开包体的类型提升为任何不含受约束变量 X 的超类型。要使最小赋型算法完备，当它发现开包体的最小类型 T_2 自由含有 X 时，便不能直接失败，而应寻找一个不含 X 的超类型。关键事实是：在核 $F_{<}$ 中，一个类型所有不含 X 的超类型总有最小者。下面的练习给出相应算法；解答来自 Ghelli 与 Pierce (1998)。

练习 28.7.1 (★★★). 为核 $F_{<}$ 定义算法 $R_{X, \Gamma}(T)$ ，计算 T 的最小 X -自由超类型。

查看原书附录 A 中的 28.7.1 解答

存在类型消去的算法赋型规则现在可以写成：

$$\frac{\Gamma \vdash_A t_1 : T_1 \quad \Gamma \vdash_A T_1 \uparrow \{\exists X <: T_{11}, T_{12}\} \quad \Gamma, X <: T_{11}, x : T_{12} \vdash_A t_2 : T_2 \quad R_{X, (\Gamma, X <: T_{11}, x : T_{12})}(T_2) = T'_2}{\Gamma \vdash_A \mathbf{let} \{X, x\} = t_1 \mathbf{in} t_2 : T'_2} \text{TA-UNPACK}$$

完全 $F_{<}$ 加入有界存在类型后的情形更糟。Ghelli 与 Pierce (1998) 给出了类型 T 、上下文 Γ 和变量 X ，使 T 在 Γ 下所有不含 X 的超类型没有最小者。因此，该系统的赋型关系不具有最小类型性质。

练习 28.7.2 (★★★). 证明只含有界存在量词、不含全称量词的完全 $F_{<}$ 变体，其子类型关系同样不可判定。

查看原书附录 A 中的 28.7.2 解答

28.8 有界量化与底类型

加入最小类型 Bot (§15.4) 会使 $F_{<}$ 的元理论更复杂。在 $\forall X <: \text{Bot}. T$ 内, 假设给出 $X <: \text{Bot}$, S-Bot 又给出 $\text{Bot} <: X$, 所以 X 与 Bot 在子类型关系下等价。因此

$$\forall X <: \text{Bot}. X \rightarrow X \quad \text{和} \quad \forall X <: \text{Bot}. \text{Bot} \rightarrow \text{Bot}$$

互为子类型, 却并非语法相同。更进一步, 若周围的上下文含有 $X <: \text{Bot}$ 与 $Y <: \text{Bot}$, 那么 $X \rightarrow Y$ 和 $Y \rightarrow X$ 也互为子类型, 尽管二者都没有显式写出 Bot。即便存在这些困难, 加入底类型后仍能建立核 $F_{<}$ 的核心性质; 细节见 Pierce (1997a)。

第七部分

附录

附录 A 习题解答选编

本附录译自原书附录 A，收录作者为部分练习给出的解答。原书以 $\Rightarrow$ 标明、未由作者提供解答的练习，另见书末“译者附录：原书未解答练习参考解答”。

A.1 第 3 章：无类型算术表达式

3.2.4 解答

$$|S_{i+1}| = |S_i|^3 + 3|S_i| + 3, \quad |S_0| = 0$$

因此 $|S_3| = 59439$ 。

[返回练习 3.2.4](#)

3.2.5 解答

直接作归纳即可。 $i = 0$ 时没有什么需要证明。以下设 $i = j + 1$ ，其中 $j \geq 0$ ，并以 $S_j \subseteq S_i$ 为归纳假设；我们要证明 $S_i \subseteq S_{i+1}$ 。任取 $t \in S_i$ 。由 S_i 是三个集合之并的定义， t 必为以下三种形式之一：

1. $t \in \{\text{true}, \text{false}, 0\}$ 。由 S_{i+1} 的定义，显然 $t \in S_{i+1}$ 。
2. t 为 $\text{succ } t_1$ 、 $\text{pred } t_1$ 或 $\text{iszero } t_1$ ，其中 $t_1 \in S_j$ 。由归纳假设， $t_1 \in S_i$ ，故按定义 $t \in S_{i+1}$ 。
3. $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$ ，其中 $t_1, t_2, t_3 \in S_j$ 。再次由归纳假设，三个子项都属于 S_i ，故按定义 $t \in S_{i+1}$ 。

[返回练习 3.2.5](#)

3.3.4 解答

这里只证明对深度归纳的原理；另外两条与之类似。已知对每个项 s ，只要对所有满足 $\text{depth}(r) < \text{depth}(s)$ 的项 r 都有 $P(r)$ ，就能证明 $P(s)$ ；需要推出 $P(s)$ 对所有 s 成立。定义自然数上的新谓词

$$Q(n) \iff \text{对每个满足 } \text{depth}(s) = n \text{ 的项 } s, \text{ 都有 } P(s)。$$

现在对自然数 n 使用完全归纳原理 (2.4.2), 即可证明 $Q(n)$ 对所有 n 成立。

译者注: 原书只给出了上面的证明提示, 并略去了具体的归纳步骤。下面分别展开三条归纳原理。

(1) 对深度归纳。 题设假定: 对于任意项 s , 只要所有满足 $\text{depth}(r) < \text{depth}(s)$ 的项 r 都有 $P(r)$, 就能推出 $P(s)$ 。我们的目标是证明所有项 s 都满足 $P(s)$ 。

定义自然数上的命题

$$Q(n) \iff \text{每个深度为 } n \text{ 的项 } s \text{ 都满足 } P(s).$$

依照原书的提示, 我们对自然数 n 作完全归纳。任取 $n \in \mathbb{N}$, 假设 $Q(m)$ 对所有满足 $m < n$ 的自然数 m 都成立; 需要证明 $Q(n)$ 。

若 $n = 0$, 则不存在深度为 0 的项, 所以 $Q(0)$ 空真。这是完全归纳中的基础情形; 它本身不包含任何项。

以下设 $n \geq 1$ 。

任取一个满足 $\text{depth}(s) = n$ 的项 s 。为了利用题设假定, 需要证明: 每个满足 $\text{depth}(r) < \text{depth}(s)$ 的项 r 都满足 $P(r)$ 。于是任取满足

$$\text{depth}(r) < \text{depth}(s) = n$$

的项 r , 并令 $m = \text{depth}(r)$ 。于是 m 是满足 $m < n$ 的自然数, 归纳假设 $Q(m)$ 因而适用, 并给出 $P(r)$ 。由于 r 是任取的, 我们已经证明了上述条件中的每个项 r 都满足 $P(r)$ 。而题设假定正是: 这一条件成立时可以推出 $P(s)$ 。因此 $P(s)$ 成立。又因为 s 是任取的, 所以所有深度为 n 的项都满足 P , 即 $Q(n)$ 成立。特别地, 当 $n = 1$ 时, 不存在满足 $\text{depth}(r) < \text{depth}(s)$ 的项 r , 上述条件自动满足; 题设假定于是直接给出 $P(s)$ 。也就是说, $n = 0$ 是归纳原理的基础情形, 而 $n = 1$ 是第一个包含实际项的层次。

完全归纳原理最终说明 $Q(n)$ 对所有自然数 n 都成立。每个项都有某个有限的自然数深度, 因此 $P(s)$ 对所有项 s 都成立。

(2) 对大小归纳。 题设假定: 对于任意项 s , 只要所有满足 $\text{size}(r) < \text{size}(s)$ 的项 r 都有 $P(r)$, 就能推出 $P(s)$ 。定义

$$R(n) \iff \text{每个大小为 } n \text{ 的项 } s \text{ 都满足 } P(s).$$

对自然数 n 作完全归纳。任取 $n \in \mathbb{N}$, 假设 $R(m)$ 对所有满足 $m < n$ 的自然数 m 都成立, 并证明 $R(n)$ 。

若 $n = 0$, 不存在大小为 0 的项, 所以 $R(0)$ 空真。以下设 $n \geq 1$, 并任取大小为 n 的项 s 。

若 $\text{size}(r) < \text{size}(s)$, 令 $m = \text{size}(r)$, 则 $m < n$ 。由归纳假设 $R(m)$ 可得 $P(r)$ 。由于 r 是任取的, 每个满足 $\text{size}(r) < \text{size}(s)$ 的项 r 都满足 $P(r)$ 。根据题设假定, 这就推出 $P(s)$, 所以 $R(n)$ 成立。

当 $n = 1$ 时, 不存在大小更小的项, 题设假定直接给出 $P(s)$ 。完全归纳原理于是说明 $P(s)$ 对所有项 s 都成立。这里同样是 $n = 0$ 构成空的基础情形, $n = 1$ 才是第一个包含实际项的层次。

(3) 结构归纳。 题设假定: 对于任意项 s , 只要 s 的所有直接子项都满足 P , 就能推出 $P(s)$ 。定义

$$U(n) \iff \text{每个深度不大于 } n \text{ 的项都满足 } P.$$

下面对自然数 n 作普通归纳。

基础情形是 $n = 0$ 。不存在深度不大于 0 的项, 所以 $U(0)$ 空真。这并不是在证明某个实际项满足 P , 而是在为从 $U(0)$ 推出 $U(1)$ 准备归纳起点。

对于归纳步骤, 假设 $U(n)$ 成立, 需要证明 $U(n+1)$ 。任取深度不大于 $n+1$ 的项 s 。若 $\text{depth}(s) \leq n$, 则由归纳假设 $U(n)$ 直接得到 $P(s)$ 。否则 $\text{depth}(s) = n+1$ 。根据深度的定义, s 的每个直接子项的深度都不大于 n , 所以归纳假设说明这些直接子项都满足 P 。再由题设假定推出 $P(s)$ 。

因此, 每个深度不大于 $n+1$ 的项都满足 P , 即 $U(n+1)$ 成立。普通自然数归纳最终说明 $P(s)$ 对所有项 s 都成立。

3.5.5 解答

设 P 是求值判断的推导上的谓词。若对每个推导 $\mathcal{D}$ ，在已知 $P(\mathcal{C})$ 对 $\mathcal{D}$ 的所有直接子推导 $\mathcal{C}$ 成立时，都能证明 $P(\mathcal{D})$ ，那么 $P(\mathcal{D})$ 对所有推导 $\mathcal{D}$ 成立。

[返回练习 3.5.5](#)

3.5.10 解答

$$\frac{t \longrightarrow t'}{t \longrightarrow^* t'} \quad \frac{}{t \longrightarrow^* t} \quad \frac{t \longrightarrow^* t' \quad t' \longrightarrow^* t''}{t \longrightarrow^* t''}$$

[返回练习 3.5.10](#)

3.5.13 解答

- (1) 定理 3.5.4 与定理 3.5.11 不再成立；定理 3.5.7、3.5.8 和 3.5.12 仍然成立。
- (2) 现在只有定理 3.5.4 不成立，其余各条仍然成立。值得注意的是：尽管加入这条规则后，单步求值变得不确定，多步求值的最终结果仍然是确定的，也就是说“条条大路通罗马”。严格证明并不困难，只是没有原来那么直接。关键在于单步求值关系具有下面这项局部汇合性质。

引理 A.1 (弱菱形性质)。若 $r \longrightarrow s$ 、 $r \longrightarrow t$ 且 $s \neq t$ ，则以下三种情形至少有一种成立：

$$s \longrightarrow t, \quad t \longrightarrow s, \quad \text{存在 } u, \text{ 使 } s \longrightarrow u \text{ 且 } t \longrightarrow u。$$

证明。从求值规则可见，只有当 $r = \text{if } r_1 \text{ then } r_2 \text{ else } r_3$ 时才会出现这种情形。对 $r \longrightarrow s$ 与 $r \longrightarrow t$ 的两个推导同时作归纳，并对二者最后使用的规则分类讨论。

例如，设 $r \longrightarrow s$ 使用 E-IFTRUE， $r \longrightarrow t$ 使用 E-FUNNY2。由规则形式可知

$$s = r_2, \quad t = \text{if true then } r'_2 \text{ else } r_3, \quad r_2 \longrightarrow r'_2$$

取 $u = r'_2$ 即可，因为 $s \longrightarrow u$ ，而 $t \longrightarrow u$ 可由 E-IFTRUE 得到。

若第一个推导使用 E-IFFALSE、第二个使用 E-FUNNY2，则

$$s = r_3, \quad t = \text{if false then } r'_2 \text{ else } r_3,$$

此时直接有 $t \longrightarrow s$ 。

再如，若两个推导最后都使用 E-IF，则

$$\begin{aligned} s &= \text{if } r'_1 \text{ then } r_2 \text{ else } r_3, & t &= \text{if } r''_1 \text{ then } r_2 \text{ else } r_3, \\ r_1 &\longrightarrow r'_1, & r_1 &\longrightarrow r''_1 \end{aligned}$$

由归纳假设, r'_1 与 r''_1 或者能直接归约到彼此, 或者能各自归约到某个共同的 r'''_1 。在前两种情形中, 相应的两个条件式也能直接归约到彼此; 在后一种情形中, 取

$$u = \text{if } r'''_1 \text{ then } r_2 \text{ else } r_3$$

即可。其他情形类似。 $\square$

结果唯一性可由一次直接的“图追踪”得到。若 $r \rightarrow^* s$ 且 $r \rightarrow^* t$, 从两条序列的第一步开始, 反复应用引理 A.1: 若一边一步到达另一边, 就沿较长的一边继续; 否则, 把两个后继拉到一个共同后继。如此逐层把两条路径拉拢起来。由于求值终止 (定理 3.5.12 仍然成立), 这个过程最终得到两条路径的共同后继。

因此, 若 r 既能求值到值 v , 又能求值到值 w , 则 $v = w$: 二者都是范式, 而两个范式要继续求值到同一个项, 唯一的可能就是它们本来相同。

译者注: 原书附录 A 的 Lemma A.1 把这里的性质写成了更强的菱形性质, 但作者官方勘误指出该陈述为假; 本解答采用勘误给出的弱菱形版本, 并相应调整了后面的图追踪论证。

返回练习 3.5.13

3.5.14 解答

对 t 的结构作归纳。

情形: t 是值。每个值都处于范式, 所以这一情形不可能发生。

情形: $t = \text{succ } t_1$ 。检查求值规则可知, 只有 E-SUCC 能用来导出 $t \rightarrow t'$ 与 $t \rightarrow t''$: 其他规则左侧的最外层构造子都不是 succ 。因此, 必有两个子推导, 其结论分别为 $t_1 \rightarrow t'_1$ 与 $t_1 \rightarrow t''_1$ 。因为 t_1 是 t 的子项, 归纳假设给出 $t'_1 = t''_1$, 于是 $\text{succ } t'_1 = \text{succ } t''_1$ 。

情形: $t = \text{pred } t_1$ 。可以把 t 归约为 t' 或 t'' 的规则有三条: E-PRED、E-PREDZERO 与 E-PREDSUCC。不过, 这三条规则彼此不重叠: 若 t 匹配其中一条的左侧, 就不可能再匹配另外两条。例如, 若它匹配 E-PRED, 则 t_1 必定不是值, 特别不可能是 0 或 $\text{succ } v$ 。所以两个推导最后必定使用同一条规则。若该规则是 E-PRED, 就像上一情形一样使用归纳假设; 若是另两条规则之一, 结论立即得到。

其他情形。类似。

返回练习 3.5.14

3.5.16 解答

用元变量 t 表示加入 **wrong** 后的新项集中的项 (包括以 **wrong** 为子短语的项), 用 g 表示原来不含 **wrong** 的“良好”项。记 $\xrightarrow{w}$ 为加入 **wrong** 规则后的求值关系, $\xrightarrow{o}$ 为原求值关系。两种处理方式“彼此一致”可以精确表述为: 一个原始项在旧语义中求值到卡住项, 当且仅当它在新语义中求值到 **wrong**。

命题 A.2. 对每个原始项 g ,

$$(g \xrightarrow{o}^* g', \text{ 其中 } g' \text{ 卡住}) \iff g \xrightarrow{w}^* \text{wrong}$$

此外, 对每个原始项 g 和原始值 v ,

$$g \xrightarrow{o}^* v \iff g \xrightarrow{w}^* v$$

证明分几步进行。首先, 新增的归约规则不会破坏练习 3.5.14 中的确定性性质。

引理 A.3. 扩充后的求值关系是确定的。

这意味着: 只要 g 在原语义中能单步归约到 g' , 它在扩充语义中也能归约到 g' , 而且 g' 是扩充语义中 g 唯一能归约到的项。

其次证明, 在原语义中已经卡住的项, 在扩充语义中总会求值到 **wrong**。

引理 A.4. 若 g 卡住, 则 $g \xrightarrow{w}^* \text{wrong}$ 。

证明。对 g 的结构作归纳。

情形: $g = \text{true}, \text{false}$ 或 0 。不可能, 因为假定了 g 卡住。

情形: $g = \text{if } g_1 \text{ then } g_2 \text{ else } g_3$ 。由于 g 卡住, g_1 必须处于范式, 否则 E-IF 可以应用。 g_1 显然不能是 **true** 或 **false**, 否则 E-IFTRUE 或 E-IFFALSE 可以应用, g 就不会卡住。考虑其余情形。

若 $g_1 = \text{if } g_{11} \text{ then } g_{12} \text{ else } g_{13}$, 则 g_1 处于范式而又不是值, 所以它本身卡住。归纳假设给出 $g_1 \xrightarrow{w}^* \text{wrong}$ 。由此可以构造

$$g \xrightarrow{w}^* \text{if wrong then } g_2 \text{ else } g_3,$$

再使用 E-IF-WRONG 得到 $g \xrightarrow{w}^* \text{wrong}$ 。

若 $g_1 = \text{succ } g_{11}$, 且 g_{11} 是数值, 则 g_1 属于 *badbool*, 规则 E-IF-WRONG 立即给出 $g \xrightarrow{w}^* \text{wrong}$ 。否则, 按定义 g_1 卡住, 归纳假设给出 $g_1 \xrightarrow{w}^* \text{wrong}$ 。从这个推导可构造

$$g \xrightarrow{w}^* \text{if wrong then } g_2 \text{ else } g_3,$$

最后使用 E-IF-WRONG 得到所需结果。其他子情形类似。

情形: $g = \text{succ } g_1$ 。因为 g 卡住, 由值的定义可知, g_1 必须处于范式且不是数值。于是有两种可能: g_1 是 **true** 或 **false**; 或者 g_1 不是值, 因而自身卡住。第一种情况下, E-SUCC-WRONG 立即给出 $g \xrightarrow{w}^* \text{wrong}$; 第二种情况下, 归纳假设给出 $g_1 \xrightarrow{w}^* \text{wrong}$, 随后按上面同样的方法完成推导。

其他情形。 类似。 □

引理 A.3 与 A.4 一起给出命题 A.2 从左到右的方向。反方向需要说明: 在扩充语义中“即将出错”的原始项, 在原语义中必定卡住。

引理 A.5. 若 $g \xrightarrow{w}^* t$, 而 t 含有 **wrong** 作为子项, 则 g 在原语义中卡住。

证明。对扩充语义中的求值推导作直接归纳即可。 $\square$

若 $g \xrightarrow{w}^* \text{wrong}$, 考察序列中第一次出现 wrong 的那一步。此前的每一步都是原语义中已有的求值步骤；设此前到达的原始项为 h , 则 $g \xrightarrow{o}^* h$, 而引理 A.5 说明 h 在原语义中卡住。这就得到命题 A.2 从右到左的方向。关于原始值 v 的第二个等价式也由同样的规则包含关系与确定性直接得到：求值序列一旦使用了产生 wrong 的规则，就不可能再以原始值结束。至此证明完毕。

译者注：作者官方勘误要求在命题 A.2 中补入关于原始值 v 的第二个等价式，并修正引理 A.4 条件式子情形中的一段推导；这里已经一并落实。

返回练习 3.5.16

3.5.17 解答

分别证明等价关系的两个方向。每个方向都先建立一个关于多步小步求值的技术引理。

引理 A.6. 若 $t_1 \rightarrow^* t'_1$, 则

$$\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow^* \text{if } t'_1 \text{ then } t_2 \text{ else } t_3$$

其他项构造子也有类似结论。

证明。简单归纳。 $\square$

命题 A.7. 若 $t \Downarrow v$, 则 $t \rightarrow^* v$ 。

证明。对 $t \Downarrow v$ 的推导作归纳，并按最后使用的规则分类。

情形 B-VALUE: $t = v$, 结论立即成立。

情形 B-IFTRUE:

$$t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3, \quad t_1 \Downarrow \text{true}, \quad t_2 \Downarrow v$$

由归纳假设, $t_1 \rightarrow^* \text{true}$ 。再由引理 A.6,

$$\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow^* \text{if true then } t_2 \text{ else } t_3$$

使用 E-IFTRUE 有 $\text{if true then } t_2 \text{ else } t_3 \rightarrow t_2$; 另一次使用归纳假设得到 $t_2 \rightarrow^* v$ 。最后利用 $\rightarrow^*$ 的传递性即可。其他情形类似。 $\square$

引理 A.8. 若

$$\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow^* v,$$

则或者

$$t_1 \rightarrow^* \text{true} \quad \text{且} \quad t_2 \rightarrow^* v,$$

或者

$$t_1 \rightarrow^* \text{false} \quad \text{且} \quad t_3 \rightarrow^* v$$

而且, 这些关于 t_1 与 t_2 (或 t_3) 的求值序列都严格短于给定序列。其他项构造子也有类似结论。

证明。对给定求值序列的长度作归纳。条件式不是值, 所以序列至少有一步。按第一步使用的最后一条规则分类即可。若第一步使用 E-IF, 则

$$\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow \text{if } t'_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow^* v, \quad t_1 \longrightarrow t'_1$$

由归纳假设, t'_1 最终归约到 `true` 且 t_2 归约到 v , 或者 t'_1 最终归约到 `false` 且 t_3 归约到 v 。把起始的一步 $t_1 \longrightarrow t'_1$ 接到相应序列之前, 就得到结论。所得序列显然比原序列短。若第一步使用 E-IFTRUE, 则

$$\text{if true then } t_2 \text{ else } t_3 \longrightarrow t_2 \longrightarrow^* v,$$

结论立即成立。E-IFFALSE 情形同理, 此时使用的是 $t_3 \longrightarrow^* v$ 。□

命题 A.9. 若 $t \longrightarrow^* v$, 则 $t \Downarrow v$ 。

证明。先证明辅助结论: 若 $t \longrightarrow t'$ 且 $t' \Downarrow v$, 则 $t \Downarrow v$ 。对 $t \longrightarrow t'$ 的推导作归纳, 并按最后使用的小步规则分类; 每一情形都可以用相应的大步规则及归纳假设重建 $t \Downarrow v$ 。

现在对 $t \longrightarrow^* v$ 的步数作归纳。零步时 $t = v$, 由 B-VALUE 得到结论。若序列至少有一步, 把它写成

$$t \longrightarrow t' \longrightarrow^* v$$

由归纳假设 $t' \Downarrow v$, 再应用刚才的辅助结论, 得到 $t \Downarrow v$ 。□

译者注: 原书对命题 A.9 直接按小步序列长度归纳, 但作者官方勘误指出该证明在 `succ` 情形中不成立, 因为子项与整项的求值步数可能相同。这里采用勘误建议的辅助引理修复证明。

[返回练习 3.5.17](#)

A.2 第 4 章: 算术表达式的 ML 实现

4.2.1 解答

`eval` 每次递归调用自身时, 都会在调用栈上再压入一个 `try` 处理器。每次单步求值都会产生一次递归调用, 所以调用栈最终会溢出。实质上, 用 `try` 包住对 `eval` 的递归调用后, 这个调用尽管看起来像尾调用, 却不再是尾调用。较好 (但可读性稍差) 的写法是:

```

let rec eval t =
  let t'opt =
    try Some (eval1 t) with NoRuleApplies -> None
  in
  match t'opt with
  | Some t' -> eval t'
  | None -> t

```

Rust 对照实现 (译者增补)

```

pub fn eval_without_exception_handler(mut term: Term) -> Term {
  loop {
    match eval1(&term) {
      Ok(next) => term = next,
      Err(NoRuleApplies) => return term,
    }
  }
}

```

[返回练习 4.2.1](#)

A.3 第 5 章：无类型 lambda 演算

5.2.1 解答

$$or = \lambda b. \lambda c. b \text{ tru } c,$$

$$not = \lambda b. b \text{ fls } tru$$

[返回练习 5.2.1](#)

5.2.2 解答

$$scc2 = \lambda n. \lambda s. \lambda z. n \text{ s } (s \text{ z})$$

[返回练习 5.2.2](#)

5.2.3 解答

$$times2 = \lambda m. \lambda n. \lambda s. \lambda z. m (n s) z$$

也可以写得更紧凑：

$$times3 = \lambda m. \lambda n. \lambda s. m (n s)$$

译者注：从 $times2$ 到 $times3$ 用到了 η 归约 (*eta-reduction*)：若变量 z 不在 F 中自由出现，则 $\lambda z. F z$ 可以简写为 F 。这里取 $F = m(n s)$ ，便可把 $\lambda z. m(n s) z$ 简写为 $m(n s)$ 。后者本身已经是一个等待初始值 z 的函数，因此 z 并未从计算中消失，只是不再显式写出。

返回练习 5.2.3

5.2.4 解答

做法不止一种：

$$power1 = \lambda m. \lambda n. n (times m) c_1,$$

$$power2 = \lambda m. \lambda n. n m$$

这里第一个参数 m 是底数，第二个参数 n 是指数。

译者注：原书初版解答把 m, n 的作用写反；这里已按作者官方勘误交换二者的作用。

返回练习 5.2.4

5.2.5 解答

$$subtract1 = \lambda m. \lambda n. n prd m$$

返回练习 5.2.5

5.2.6 解答

对 $prd c_n$ 求值需要 $O(n)$ 步，因为 prd 用 n 构造出一个含 n 对数字的序列，然后取该序列最后一对的第一分量。

返回练习 5.2.6

5.2.7 解答

下面是一种简单写法：

$$equal = \lambda m. \lambda n. and(iszro(m prd n)) \\ (iszro(n prd m))$$

返回练习 5.2.7

5.2.8 解答

作者原先设想的解答如下：

$$\begin{aligned} nil &= \lambda c. \lambda n. n, \\ cons &= \lambda h. \lambda t. \lambda c. \lambda n. c \ h \ (t \ c \ n), \\ head &= \lambda l. l \ (\lambda h. \lambda t. h) \ fls, \\ tail &= \lambda l. fst(l \ (\lambda x. \lambda p. pair(snd \ p) \ (cons \ x \ (snd \ p))) \ (pair \ nil \ nil)), \\ isnil &= \lambda l. l \ (\lambda h. \lambda t. fls) \ tru \end{aligned}$$

还有一种颇为不同的做法：

$$\begin{aligned} nil &= pair \ tru \ tru, \\ cons &= \lambda h. \lambda t. pair \ fls \ (pair \ h \ t), \\ head &= \lambda z. fst(snd \ z), \\ tail &= \lambda z. snd(snd \ z), \\ isnil &= fst \end{aligned}$$

译者注：第一组解答沿用题目规定的右折叠编码。可以把 lcn 读作：“用二元函数 c 和空表对应值 n 对列表 l 做右折叠。”空表没有元素，所以 $nilcn = n$ ；在尾表 t 前添加元素 h 后，应先得到尾表的折叠结果 tcn ，再计算 $ch(tcn)$ ，这就解释了 nil 与 $cons$ 的定义。

要取得表头，解答把 c 选为 $\lambda h. \lambda t. h$ ：它保留当前元素 h ，丢弃右侧已经折叠出的结果 t ，所以对非空列表最终留下第一个元素； fls 只是空表情况下的默认结果。要判断列表是否为空，则把空表对应值选为 tru ，并让 c 每遇到一个元素都返回 fls ：空表不会调用 c ，因而得到真；非空表至少调用一次 c ，因而得到假。

$tail$ 使用了与数值前驱函数相同的“落后一步”技巧。暂用普通列表记法表示每个编码对应的内容；对 $[x, y, z]$ 从右向左折叠时，数对状态依次变化为

$$([], []) \mapsto ([], [z]) \mapsto ([z], [y, z]) \mapsto ([y, z], [x, y, z])$$

更新函数 $\lambda x. \lambda p. pair(snd \ p) \ (cons \ x \ (snd \ p))$ 把状态“(上一步结果, 当前结果)”更新为“(当前结果, $x ::$ 当前结果)”。全部元素处理完后，第二分量是原列表，第一分量恰好少了最前面的元素；因此取 fst 就得到尾表。对空表，初始状态 $([], [])$ 不变，所以结果仍为空表。

第二组解答没有使用右折叠编码，而是采用“标签加数据”的表示：第一分量用 tru 或 fls 区分空表与非空表；非空表的第二分量再保存数对 (h, t) ，即表头和表尾。因此 $isnil$ 只需取第一分量， $head$ 与 $tail$ 则分别从内部数对中取第一、第二分量。

返回练习 5.2.8

5.2.9 解答

原定义使用原始 if 而不是 $test$ ，是为了避免条件式两个分支总被求值；若两边都求值，阶乘就会发散。改用 $test$ 时，可以把两个分支包在无实际作用的 λ 抽象中，以阻止这种发散。抽象是值，所以按值调用策略不会进入其内部，而是把它们原封不动地传给 $test$ ；

test 选择一个分支并把它返回。随后把整个 *test* 表达式应用于一个哑参数 (例如 c_0), 即可迫使被选中的分支求值:

$$\begin{aligned} ff &= \lambda f. \lambda n. test(iszro\ n) (\lambda x. c_1) (\lambda x. times\ n\ (f\ (prd\ n)))\ c_0, \\ factorial &= fix\ ff \end{aligned}$$

例如,

$$equal\ c_6\ (factorial\ c_3) \longrightarrow^* \lambda x. \lambda y. x$$

[返回练习 5.2.9](#)

5.2.10 解答

下面这个递归函数可以完成转换:

$$\begin{aligned} cn &= \lambda f. \lambda m. \text{if iszero } m \text{ then } c_0 \text{ else } scc(f\ (pred\ m)), \\ churchnat &= fix\ cn \end{aligned}$$

快速测试如下:

$$equal(churchnat\ 4)\ c_4 \longrightarrow^* \lambda x. \lambda y. x$$

[返回练习 5.2.10](#)

5.2.11 解答

$$\begin{aligned} ff &= \lambda f. \lambda l. test(isnil\ l) (\lambda x. c_0) (\lambda x. plus(head\ l) (f\ (tail\ l)))\ c_0, \\ sumlist &= fix\ ff, \\ l &= cons\ c_2\ (cons\ c_3\ (cons\ c_4\ nil)) \end{aligned}$$

于是

$$equal(sumlist\ l)\ c_9 \longrightarrow^* \lambda x. \lambda y. x$$

当然, 也可以不用 *fix*:

$$sumlist' = \lambda l. l\ plus\ c_0,$$

并且

$$equal(sumlist'\ l)\ c_9 \longrightarrow^* \lambda x. \lambda y. x$$

[返回练习 5.2.11](#)

5.3.3 解答

对 t 的大小作归纳。假设所需性质对比 t 小的项成立; 只要能证明它对 t 本身成立, 就能得出该性质对所有项成立。分三种情形。

情形： $t = x$ 。

$$|\text{FV}(t)| = |\{x\}| = 1 = \text{size}(t)$$

情形： $t = \lambda x. t_1$ 。由归纳假设 $|\text{FV}(t_1)| \leq \text{size}(t_1)$ ，所以

$$\begin{aligned} |\text{FV}(t)| &= |\text{FV}(t_1) \setminus \{x\}| \\ &\leq |\text{FV}(t_1)| \leq \text{size}(t_1) < \text{size}(t) \end{aligned}$$

情形： $t = t_1 t_2$ 。由归纳假设， $|\text{FV}(t_1)| \leq \text{size}(t_1)$ 且 $|\text{FV}(t_2)| \leq \text{size}(t_2)$ 。于是

$$\begin{aligned} |\text{FV}(t)| &= |\text{FV}(t_1) \cup \text{FV}(t_2)| \\ &\leq |\text{FV}(t_1)| + |\text{FV}(t_2)| \\ &\leq \text{size}(t_1) + \text{size}(t_2) < \text{size}(t) \end{aligned}$$

返回练习 5.3.3

5.3.6 解答

完全 β 归约。 规则如下：

$$\begin{array}{c} \frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2} \text{E-APP1} \quad \frac{t_2 \longrightarrow t'_2}{t_1 t_2 \longrightarrow t_1 t'_2} \text{E-APP2} \\[2ex] \frac{t_1 \longrightarrow t'_1}{\lambda x. t_1 \longrightarrow \lambda x. t'_1} \text{E-ABS} \quad \frac{}{(\lambda x. t_{12}) t_2 \longrightarrow [x \mapsto t_2] t_{12}} \text{E-APPABS} \end{array}$$

注意，这里没有使用值的句法范畴。

正规序。 一种写法是

$$\begin{array}{c} \frac{na_1 \longrightarrow t'_1}{na_1 t_2 \longrightarrow t'_1 t_2} \text{E-APP1} \quad \frac{t_2 \longrightarrow t'_2}{nanf_1 t_2 \longrightarrow nanf_1 t'_2} \text{E-APP2} \\[2ex] \frac{t_1 \longrightarrow t'_1}{\lambda x. t_1 \longrightarrow \lambda x. t'_1} \text{E-ABS} \quad \frac{}{(\lambda x. t_{12}) t_2 \longrightarrow [x \mapsto t_2] t_{12}} \text{E-APPABS}, \end{array}$$

其中范式、非抽象范式与非抽象项的句法范畴定义为

$$\begin{aligned} nf &::= \lambda x. nf \mid nanf \quad \text{范式,} \\ nanf &::= x \mid nanf \, nf \quad \text{非抽象范式,} \\ na &::= x \mid t_1 t_2 \quad \text{非抽象项} \end{aligned}$$

与其他策略相比，这个定义略显笨拙。通常只说：正规序归约与完全 β 归约相同，但总是优先选择最左、最外层可归约式。

惰性求值。 值仍定义为任意抽象，与按值调用相同；规则为

$$\frac{t_1 \longrightarrow t'_1}{t_1 t_2 \longrightarrow t'_1 t_2} \text{ E-APP1} \qquad \frac{}{(\lambda x. t_{12}) t_2 \longrightarrow [x \mapsto t_2] t_{12}} \text{ E-APPABS}$$

译者注：完全 β 归约中的 E-ABS 在原书初版解答中漏印，已经按作者官方勘误补回。

[返回练习 5.3.6](#)

5.3.8 解答

$$\frac{}{\lambda x. t \Downarrow \lambda x. t} \text{ B-VALUE}$$

$$\frac{t_1 \Downarrow \lambda x. t_{12} \quad t_2 \Downarrow v_2 \quad [x \mapsto v_2] t_{12} \Downarrow v}{t_1 t_2 \Downarrow v} \text{ B-APP}$$

[返回练习 5.3.8](#)

A.4 第 6 章：项的无名表示

6.1.1 解答

$$\begin{aligned} c_0 &= \lambda. \lambda. 0, \\ c_2 &= \lambda. \lambda. 1(10), \\ plus &= \lambda. \lambda. \lambda. \lambda. 31(210), \\ fix &= \lambda. (\lambda. 1(\lambda. (11)0))(\lambda. 1(\lambda. (11)0)), \\ foo &= (\lambda. \lambda. 0)(\lambda. 0) \end{aligned}$$

返回练习 6.1.1

6.1.5 解答

两个函数可以定义如下：

$$\begin{aligned} \text{removenames}_\Gamma(x) &= x \text{ 在 } \Gamma \text{ 中最右一次出现的索引,} \\ \text{removenames}_\Gamma(\lambda x. t_1) &= \lambda. \text{removenames}_{\Gamma, x}(t_1), \\ \text{removenames}_\Gamma(t_1 t_2) &= \text{removenames}_\Gamma(t_1) \text{removenames}_\Gamma(t_2), \\ \text{restorenames}_\Gamma(k) &= \Gamma \text{ 中索引为 } k \text{ 的名字,} \\ \text{restorenames}_\Gamma(\lambda. t_1) &= \lambda x. \text{restorenames}_{\Gamma, x}(t_1), \\ &\quad \text{其中 } x \text{ 是第一个不属于 } \text{dom}(\Gamma) \text{ 的名字,} \\ \text{restorenames}_\Gamma(t_1 t_2) &= \text{restorenames}_\Gamma(t_1) \text{restorenames}_\Gamma(t_2) \end{aligned}$$

对项作直接的结构归纳，即可证明 *removenames* 与 *restorenames* 所要求的两个性质。

译者注：原书初版解答在两个应用分支中误把下标写成 Γ, x ；此处已按作者官方勘误改为 Γ 。

返回练习 6.1.5

6.2.2 解答

1. $\lambda. \lambda. 1(04)$
2. $\lambda. 03(\lambda. 014)$

返回练习 6.2.2

6.2.5 解答

$$[0 \mapsto 1] (0 (\lambda. \lambda. 2)) = 1 (\lambda. \lambda. 3)$$

$$\text{即 } a (\lambda x. \lambda y. a),$$

$$[0 \mapsto 1 (\lambda. 2)] (0 (\lambda. 1)) = (1 (\lambda. 2)) (\lambda. (2 (\lambda. 3)))$$

$$\text{即 } (a (\lambda z. a)) (\lambda x. a (\lambda z. a)),$$

$$[0 \mapsto 1] (\lambda. 0 2) = \lambda. 0 2$$

$$\text{即 } \lambda b. b a,$$

$$[0 \mapsto 1] (\lambda. 1 0) = \lambda. 2 0$$

$$\text{即 } \lambda a'. a a'$$

译者注：这四项计算的关键是始终在同一个命名上下文中解释索引。对 $\Gamma = a, b$ ，从右向左编号得到 $b \mapsto 0$ 、 $a \mapsto 1$ ；每进入一层 lambda，外部变量的索引都增加 1。下面补出作者解答省略的中间步骤。

1. 具名项 $b (\lambda x. \lambda y. b)$ 转换为 $0 (\lambda. \lambda. 2)$ ，而替换 $[b \mapsto a]$ 转换为 $[0 \mapsto 1]$ 。替换依次进入两层 lambda 时，目标编号与待代入项的索引分别从 0, 1 变为 1, 2，再变为 2, 3：

$$\begin{aligned} [0 \mapsto 1] (0 (\lambda. \lambda. 2)) &= 1 (\lambda. [1 \mapsto 2] (\lambda. 2)) \\ &= 1 (\lambda. \lambda. [2 \mapsto 3] 2) \\ &= 1 (\lambda. \lambda. 3) \end{aligned}$$

外层的 1 与两层 lambda 内的 3 都指向同一个外部变量 a ，因而恢复成具名形式后得到 $a (\lambda x. \lambda y. a)$ 。

2. 待代入项与被替换项分别转换为

$$a (\lambda z. a) \mapsto 1 (\lambda. 2), \quad b (\lambda x. b) \mapsto 0 (\lambda. 1)$$

记 $s = 1 (\lambda. 2)$ 。替换进入 λx 后，必须先把 s 中自由出现的 a 上移一位：

$$\uparrow^1 (s) = 2 (\lambda. 3)$$

因此

$$\begin{aligned} [0 \mapsto s] (0 (\lambda. 1)) &= s (\lambda. [1 \mapsto \uparrow^1 (s)] 1) \\ &= (1 (\lambda. 2)) (\lambda. (2 (\lambda. 3))) \end{aligned}$$

索引的增加并未改变变量：2 和更内层的 3 仍指向外部的 a 。恢复名称后得到 $(a (\lambda z. a)) (\lambda x. a (\lambda z. a))$ 。

3. 在 $\lambda b. b a$ 中，函数体里的 b 由当前 lambda 绑定。它在无名形式 $\lambda. 0 2$ 中表现为索引 0，而进入 lambda 后要寻找的外部 b 已变成索引 1。函数体中没有索引 1，所以替换不改变该项。
4. 在 $\lambda a. b a$ 中， b 是自由的，而 a 由当前 lambda 绑定。无名形式为 $\lambda. 1 0$ 。把外部的 b 替换为外部的 a 时，待代入的 a 必须从索引 1 上移为 2，以免被当前 lambda 捕获，故结果为 $\lambda. 2 0$ 。恢复名称时为避免与外部的 a 冲突，可把绑定变量改名为 a' ，得到 $\lambda a'. a a'$ 。

6.2.8 解答

若 Γ 是命名上下文, 记 $\Gamma(x)$ 为从右向左数时 x 在 Γ 中的索引。所需证明的性质是

$$\text{removenames}_{\Gamma}([x \mapsto s]t) = [\Gamma(x) \mapsto \text{removenames}_{\Gamma}(s)](\text{removenames}_{\Gamma}(t))$$

证明对 t 作归纳, 使用定义 5.3.5 与 6.2.4、若干简单计算, 以及关于 *removenames* 和项上其他基本操作的几个简单引理。抽象情形中, 约定 5.3.4 起关键作用。

[返回练习 6.2.8](#)

6.3.1 解答

索引变成负数, 唯一可能是最后的负移位作用于一个需要移动的索引 0。但中间结果中不会留下这样的变量: 替换操作已经消除了函数体中原来指向形参的变量; 而代入项在替换前已经向上移动了一位, 所以其中的自由变量索引都至少为 1。因此, 最后的负移位不会产生负索引。

译者注: 这里所说的索引 0, 特指相对于整个中间结果是自由的、因而会被顶层移位处理的变量。代入项内部仍可能出现由其自身 *lambda* 抽象绑定的索引 0, 例如 $\lambda.0$; 移位操作进入该 *lambda* 后会提高截点, 所以不会移动这个受约束变量, 更不会把它变成 -1 。

[返回练习 6.3.1](#)

6.3.2 解答

使用索引与使用层级的两种表示等价, 其证明可见 Lescanne 与 Rouyer-Degli (1995)。de Bruijn 层级还见 de Bruijn (1972) 以及 Filinski (1999, §5.2) 的讨论。

[返回练习 6.3.2](#)

A.5 第 8 章: 带类型的算术表达式

8.3.5 解答

不能; 删去这条规则会破坏进展性。如果我们确实不愿定义 *pred* 0, 就必须换一种方式处理: 例如, 程序尝试求 0 的前驱时抛出异常 (见第十四章), 或者细化 *pred* 的类型, 明确它只能合法地应用于严格为正的数; 后一种做法或许可以使用交类型 (§15.7) 或依赖类型 (§30.5)。

[返回练习 8.3.5](#)

8.3.6 解答

反例是

if false then true else 0

该项不是良类型的, 却求值为良类型项 0。

[返回练习 8.3.6](#)**8.3.7 解答**

大步语义的类型保持性与小步语义中的性质相似：若一个良类型项求值为某个最终值，则该值与原项具有相同类型；证明也与前面的证明相似。

另一方面，大步语义下的进展性提出了强得多的要求：每个良类型项都能求值为某个最终值，也就是说，良类型项的求值总会终止。对算术表达式而言，这恰好成立；但对更有意思的语言，例如含一般递归的语言（参见§11.11），它往往不成立。这样的语言根本没有这种形式的进展性：从大步语义的角度看，求值到达错误状态与求值不终止之间没有可见区别。这也是语言理论研究者通常更偏爱小步语义的原因之一。

另一种选择，是像练习 8.3.8 那样，在大步语义中显式加入转移到 `wrong` 的规则。例如，Abadi 与 Cardelli 在其对象演算的操作语义中采用了这种风格（Abadi 与 Cardelli, 1996, 第 87 页）。

[返回练习 8.3.7](#)**8.3.8 解答**

扩充后的语义中不再有卡住状态。对每个不是值的项，求值规则要么给出一次普通求值步骤，要么明确把它转移到 `wrong`；当然，还需要证明这些规则确实覆盖了所有非值项。因此，“一个项要么是值，要么能迈出一步”现在对错误项也成立，进展性本身便无法再区分安全程序与错误程序。

安全性的信息转而来自主体归约定理，也就是保持性。保持性要求：若一个良类型项迈出一步，所得项仍然是良类型的。由于 `wrong` 没有类型，假如某个良类型项能够一步转移到 `wrong`，保持性就会要求 `wrong` 具有类型，从而产生矛盾。因此，良类型项不可能一步转移到 `wrong`。证明扩充后语义的保持性时，必须逐一排除那些通向 `wrong` 的规则；这部分论证实际上就是原进展性证明所做的工作。

[返回练习 8.3.8](#)**A.6 第 9 章：简单类型 lambda 演算****9.2.1 解答**

因为类型表达式的集合是空的：类型的抽象语法没有基础情形。

译者注：这里的类型文法只有 $T ::= T \rightarrow T$ 。这条规则可以把两个已经存在的类型组合成函数类型，却不能凭空产生第一个类型。由于文法中没有 `Bool`、`Nat` 或类型变量之类的基础情形，按归纳定义生成的类型集合只能是空集。也不能令 $T = T \rightarrow T$ 来得到一个类型，因为这里的类型必须是由文法有限步构造出的有限表达式；这种自我引用需要借助后文介绍的递归类型。既然不存在类型 T ，就不可能形成任何类型判断 $\Gamma \vdash t : T$ ，因而不存在良类型项。

[返回练习 9.2.1](#)

9.2.3 解答

一个满足要求的上下文是

$$\Gamma = f : \text{Bool} \rightarrow \text{Bool} \rightarrow \text{Bool}, x : \text{Bool}, y : \text{Bool}$$

一般地, 对任意类型 S, T , 以下形式的上下文都满足要求:

$$\Gamma = f : S \rightarrow T \rightarrow \text{Bool}, x : S, y : T$$

这种推理正是第二十二章所介绍的类型重构算法的核心。

[返回练习 9.2.3](#)

9.3.2 解答

反设项 xx 具有类型 T 。由反演引理, 左侧子项 x 必须具有某个类型 $T_1 \rightarrow T_2$, 右侧子项 (同样是 x) 必须具有类型 T_1 。再使用反演引理的变量情形可知, $x : T_1 \rightarrow T_2$ 与 $x : T_1$ 都必须来自 Γ 中的假设。由于 Γ 中至多有一个关于 x 的绑定, 必有

$$T_1 \rightarrow T_2 = T_1$$

若把类型的大小定义为其树形表示中的节点数, 则上述等式要求两侧大小相等; 但

$$|T_1 \rightarrow T_2| = 1 + |T_1| + |T_2| > |T_1|$$

矛盾。

译者注: 这里的 $|T|$ 不是绝对值或集合的基数, 而是类型 T 的语法树所包含的节点数。类型 $T_1 \rightarrow T_2$ 的语法树以箭头构造子为根节点, 左右子树分别是 T_1 和 T_2 , 所以其节点总数为 $1 + |T_1| + |T_2|$ 。其中额外的 1 来自根部的箭头节点。由于每个有限类型都至少包含一个节点, 这个总数必然大于 $|T_1|$ 。

若允许类型无限大, 就能为方程 $T_1 \rightarrow T_2 = T_1$ 构造解。[第二十章](#)会详细回到这一点。

[返回练习 9.3.2](#)

9.3.3 解答

假设 $\Gamma \vdash t : S$ 且 $\Gamma \vdash t : T$ 。对 $\Gamma \vdash t : T$ 的推导作归纳, 证明 $S = T$ 。

情形 T-VAR:

此时 $t = x$, 且 $x : T \in \Gamma$ 。由反演引理第 1 项, 任何 $\Gamma \vdash t : S$ 的推导也只能以 T-VAR 结束, 故 $S = T$ 。

情形 T-ABS:

此时

$$t = \lambda y : T_2. t_1, \quad T = T_2 \rightarrow T_1, \quad \Gamma, y : T_2 \vdash t_1 : T_1$$

由反演引理第 2 项, 任何 $\Gamma \vdash t : S$ 的推导也必须以 T-ABS 结束, 并且含有结论 $\Gamma, y : T_2 \vdash t_1 : S_1$ 的子推导, 其中 $S = T_2 \rightarrow S_1$ 。对子推导 $\Gamma, y : T_2 \vdash t_1 : T_1$ 使用归纳假设, 得到 $S_1 = T_1$, 于是立即有 $S = T$ 。

情形 T-APP、T-TRUE、T-FALSE、T-IF:

类似。

返回定理 9.3.3

9.3.9 解答

对 $\Gamma \vdash t : T$ 的推导作归纳。归纳的每一步都假设所需性质对所有子推导成立: 若某个子推导证明 $\Gamma \vdash s : S$, 且 $s \longrightarrow s'$, 则 $\Gamma \vdash s' : S$ 。然后按推导最后使用的规则分类。

情形 T-VAR:

此时 $t = x$, 且 $x : T \in \Gamma$ 。这种情形不可能发生, 因为没有任何求值规则以变量为左端。

情形 T-ABS:

此时 $t = \lambda x : T_1. t_2$ 。这种情形也不可能发生。

情形 T-APP:

此时

$$t = t_1 t_2, \quad \Gamma \vdash t_1 : T_{11} \rightarrow T_{12}, \quad \Gamma \vdash t_2 : T_{11}, \quad T = T_{12}$$

检查图9-1中左端为应用的求值规则, $t \longrightarrow t'$ 只能由 E-APP1、E-APP2 或 E-APPABS 导出。分别讨论。

子情形 E-APP1:

有 $t_1 \longrightarrow t'_1$ 且 $t' = t'_1 t_2$ 。原类型推导含有结论 $\Gamma \vdash t_1 : T_{11} \rightarrow T_{12}$ 的子推导; 对子推导应用归纳假设, 得到 $\Gamma \vdash t'_1 : T_{11} \rightarrow T_{12}$ 。再结合 $\Gamma \vdash t_2 : T_{11}$, 使用 T-APP 即得 $\Gamma \vdash t' : T$ 。

子情形 E-APP2:

此时 $t_1 = v_1$, $t_2 \longrightarrow t'_2$, 且 $t' = v_1 t'_2$ 。证明类似。

子情形 E-APPABS:

此时

$$t_1 = \lambda x : T_{11}. t_{12}, \quad t_2 = v_2, \quad t' = [x \mapsto v_2] t_{12}$$

用反演引理解构 $\lambda x : T_{11}. t_{12}$ 的类型推导, 得到 $\Gamma, x : T_{11} \vdash t_{12} : T_{12}$ 。结合替换引理 (引理9.3.8), 可得 $\Gamma \vdash t' : T_{12}$ 。

布尔常量与条件表达式的情形与定理 8.3.3 相同。

返回定理 9.3.9

9.3.10 解答

项

$$(\lambda x : \text{Bool} \rightarrow \text{Bool}. \lambda y : \text{Bool}. y)(\lambda z : \text{Bool}. \text{true true})$$

不是良类型的，但按值调用可以把它归约为良类型项 $\lambda y : \text{Bool}. y$ 。

[返回练习 9.3.10](#)

9.4.1 解答

T-TRUE 与 T-FALSE 是引入规则，T-IF 是消去规则。T-ZERO 与 T-SUCC 是引入规则，T-PRED 与 T-ISZERO 是消去规则。

判断 succ 与 pred 究竟是引入形式还是消去形式，需要稍作思考，因为二者既可以看作构造数字，也可以看作使用数字。关键观察是：当 pred 与 succ 相遇时，它们构成一个可归约式；iszero 也类似。

[返回练习 9.4.1](#)

A.7 第 11 章：简单扩展

11.2.1 解答

$$\begin{aligned} t_1 &= (\lambda x : \text{Unit}. x) \text{ unit}, \\ t_{i+1} &= (\lambda f : \text{Unit} \rightarrow \text{Unit}. f (f \text{ unit})) (\lambda x : \text{Unit}. t_i) \end{aligned}$$

译者注：作者的构造把 t_i 包装成函数 $g_i = \lambda x : \text{Unit}. t_i$ 。lambda 抽象本身是值，因此构造 g_i 时不会求值它的函数体 t_i ；每次调用 g_i ， β 规约才会取出函数体。又因为 x 不在 t_i 中自由出现，代入实参不会改变函数体，所以规约结果恰好是 t_i ：

$$\begin{aligned} g_i \text{ unit} &= (\lambda x : \text{Unit}. t_i) \text{ unit} \\ &\longrightarrow [x \mapsto \text{unit}] t_i \\ &= t_i \end{aligned}$$

因此，每调用一次 g_i ，都会触发一次对 t_i 的完整求值。再用 g_i 改写上面的定义，第一步规约为

$$t_{i+1} \longrightarrow g_i (g_i \text{ unit})$$

按照按值求值的次序，内层的 $g_i \text{ unit}$ 先执行一次 t_i ；得到 unit 后，外层调用又执行一次 t_i 。若 E_i 表示 t_i 求值到范式所需的步数，那么这两次求值共需 $2E_i$ 步。此外还有三次 β 规约：第一步把外层函数应用于 g_i ，第二步调用内层的 g_i ，第三步调用外层的 g_i 。因此

$$E_1 = 1, \quad E_{i+1} = 2E_i + 3, \quad E_i = 2^{i+1} - 3$$

由此可见，求值步数呈指数增长。另一方面， t_{i+1} 的书写形式中只出现一次 t_i ，其余部分的大小固定。若用 S_i 表示项的大小，则存在常数 c 使 $S_{i+1} = S_i + c$ ，所以 $S_i = O(i)$ 。这个构造的关键在于：项本身的大小只线性增长，函数应用却会在求值过程中把前一项的计算重复两次。

[返回练习 11.2.1](#)

11.3.2 解答

$$\frac{\Gamma \vdash t_2 : T_2}{\Gamma \vdash \lambda_ : T_1. t_2 : T_1 \rightarrow T_2} \text{ T-WILDCARD} \quad (\lambda_ : T_{11}. t_{12}) v_2 \longrightarrow t_{12} \text{ E-WILDCARD}$$

证明这两条规则可由语法糖定义导出，与定理11.3.1的证明完全相同。

返回练习 11.3.2

11.4.1 解答

第一问很直接。把类型断言展开为

$$t \text{ as } T \stackrel{\text{def}}{=} (\lambda x : T. x) t$$

即可直接由抽象和应用的规则导出类型断言的类型规则与求值规则。

若把类型断言的求值规则改为急切版本，就需要更细致的展开方式，使 t 等到断言本身被消去之后才开始求值。例如，可定义

$$t \text{ as } T \stackrel{\text{def}}{=} (\lambda x : \text{Unit} \rightarrow T. x \text{ unit})(\lambda y : \text{Unit}. t) \quad \text{其中 } y \notin \text{FV}(t)$$

译者注：这两种展开方式的区别在于求值顺序。第一种展开 $(\lambda x : T. x) t$ 把 t 作为普通参数传入恒等函数。按照按值求值的规则，必须先把 t 求值为某个值 v ，然后才能进行外层的 β 规约：

$$(\lambda x : T. x) t \longrightarrow^* (\lambda x : T. x) v \longrightarrow v$$

这正好对应正式规则的次序：先在类型标注内部求值，得到值后再去掉标注。

第二种展开先把 t 包进 $\lambda y : \text{Unit}. t$ 。lambda 抽象本身已经是值，所以按值求值不会立即进入函数体计算 t 。这种把尚未求值的表达式包进函数、留待以后触发的结构称为延迟计算封装。外层应用可以立即进行两次 β 规约：

$$\begin{aligned} & (\lambda x : \text{Unit} \rightarrow T. x \text{ unit})(\lambda y : \text{Unit}. t) \\ & \longrightarrow (\lambda y : \text{Unit}. t) \text{ unit} \longrightarrow t \end{aligned}$$

因此，只有在表示类型标注的两层包装都被消去之后， t 才开始求值。这与急切规则的求值顺序相符，但高层语言的一步对应内部语言的两步。条件 $y \notin \text{FV}(t)$ 保证新增的 lambda 抽象不会意外绑定 t 中原本自由的 y 。

这里选用 Unit 并不重要，换成任何类型都可以。

这种展开在直观上正确，却不能逐字满足定理11.3.1中的性质：展开后的类型断言需要两步才消失，而高层规则 E-ASCRIBE 只走一步。可把展开看成一种简单编译；源语言的一步往往需要目标语言中的多步来模拟。因此，应把要求放宽为

$$t \longrightarrow_E t' \implies e(t) \longrightarrow_I^* e(t')$$

反方向也要谨慎表述。展开后的断言走完第一步时，中间项既不是原高层项的展开，也不是消去断言后那个高层项的展开；不过，这一步总能再走若干步，抵达另一个高层项的展开。形式化地，若 $e(t) \longrightarrow_I s$ ，则存在 t' ，使得

$$t \longrightarrow_E t' \quad \text{且} \quad s \longrightarrow_I^* e(t')$$

返回练习 11.4.1

11.5.1 解答

需要加入以下分支：

```
let rec eval1 ctx t =
  match t with
  ...
  | TmLet(fi, x, v1, t2) when isval ctx v1 ->
    termSubstTop v1 t2
  | TmLet(fi, x, t1, t2) ->
    let t1' = eval1 ctx t1 in
    TmLet(fi, x, t1', t2)
  | ...

let rec typeof ctx t =
  match t with
  ...
  | TmLet(fi, x, t1, t2) ->
    let tyT1 = typeof ctx t1 in
    let ctx' = addbinding ctx x (VarBind tyT1) in
    typeof ctx' t2
  | ...
```

Rust 对照实现（译者增补）

```
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Type {
    Unit,
    Arrow(Box<Type>, Box<Type>),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Term {
    Unit,
    Var(usize),
    Abs(Type, Box<Term>),
    App(Box<Term>, Box<Term>),
    Let(Box<Term>, Box<Term>),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum Error {
    NoRuleApplies,
    UnboundVariable(usize),
}
```



```

    ExpectedFunction(Type),
    ParameterMismatch { expected: Type, actual: Type },
}

fn is_value(term: &Term) -> bool {
    matches!(term, Term::Unit | Term::Abs(_, _))
}

pub fn eval1(term: &Term) -> Result<Term, Error> {
    match term {
        // E-AppAbs: 函数与参数都已成为值，执行一次顶层替换。
        Term::App(function, argument)
            if matches!(function.as_ref(), Term::Abs(_, _)) && is_value(argument)
        =>
        {
            let Term::Abs(_, body) = function.as_ref() else {
                unreachable!("the guard above requires an abstraction")
            };
            Ok(substitute_top(argument, body))
        }
        // E-App1: 函数位置尚未成为值，先让函数走一步。
        Term::App(function, argument) if !is_value(function) => Ok(Term::App(
            Box::new(eval1(function)?),
            Box::new(**argument).clone()),
        )),
        // E-App2: 函数已经是值，接着求值参数。
        Term::App(function, argument) => Ok(Term::App(
            Box::new(**function).clone(),
            Box::new(eval1(argument)?),
        )),
        // E-LetV: 绑定项已经是值，代入 let 主体。
        Term::Let(bound, body) if is_value(bound) => Ok(substitute_top(bound,
        => body)),
        // E-Let: 绑定项尚未成为值，只让它先走一步。
        Term::Let(bound, body) => Ok(Term::Let(
            Box::new(eval1(bound)?),
            Box::new(**body).clone()),
        )),
        _ => Err(Error::NoRuleApplies),
    }
}

// 把代入项放进函数体或 let 主体时，需要按当前绑定深度移动其中的自由变量。
fn shift(term: &Term, distance: isize, cutoff: usize) -> Term {

```

```

match term {
  Term::Unit => Term::Unit,
  Term::Var(index) if *index >= cutoff => {
    Term::Var(index.checked_add_signed(distance).expect("valid shift"))
  }
  Term::Var(index) => Term::Var(*index),
  Term::Abs(parameter, body) => Term::Abs(
    parameter.clone(),
    Box::new(shift(body, distance, cutoff + 1)),
  ),
  Term::App(function, argument) => Term::App(
    Box::new(shift(function, distance, cutoff)),
    Box::new(shift(argument, distance, cutoff)),
  ),
  Term::Let(bound, body) => Term::Let(
    Box::new(shift(bound, distance, cutoff)),
    Box::new(shift(body, distance, cutoff + 1)),
  ),
}
}

fn substitute(term: &Term, replacement: &Term, variable: usize, cutoff: usize) ->
↳ Term {
  match term {
    Term::Unit => Term::Unit,
    Term::Var(index) if *index == variable + cutoff => {
      let distance = isize::try_from(cutoff).expect("cutoff fits in isize");
      shift(replacement, distance, 0)
    }
    Term::Var(index) => Term::Var(*index),
    Term::Abs(parameter, body) => Term::Abs(
      parameter.clone(),
      Box::new(substitute(body, replacement, variable, cutoff + 1)),
    ),
    Term::App(function, argument) => Term::App(
      Box::new(substitute(function, replacement, variable, cutoff)),
      Box::new(substitute(argument, replacement, variable, cutoff)),
    ),
    Term::Let(bound, body) => Term::Let(
      Box::new(substitute(bound, replacement, variable, cutoff)),
      Box::new(substitute(body, replacement, variable, cutoff + 1)),
    ),
  }
}
}

```

```

fn substitute_top(replacement: &Term, body: &Term) -> Term {
    let lifted = shift(replacement, 1, 0);
    let substituted = substitute(body, &lifted, 0, 0);
    shift(&substituted, -1, 0)
}

pub fn type_of(context: &[Type], term: &Term) -> Result<Type, Error> {
    match term {
        Term::Unit => Ok(Type::Unit),
        Term::Var(index) => context
            .get(*index)
            .cloned()
            .ok_or(Error::UnboundVariable(*index)),
        Term::Abs(parameter, body) => {
            let mut body_context = context.to_vec();
            body_context.insert(0, parameter.clone());
            let result = type_of(&body_context, body)?;
            Ok(Type::Arrow(Box::new(parameter.clone()), Box::new(result)))
        }
        Term::App(function, argument) => {
            let function_type = type_of(context, function)?;
            let argument_type = type_of(context, argument)?;
            match function_type {
                Type::Arrow(parameter, result) if *parameter == argument_type =>
                    ↪ Ok(*result),
                Type::Arrow(parameter, _) => Err(Error::ParameterMismatch {
                    expected: *parameter,
                    actual: argument_type,
                }),
                other @ Type::Unit => Err(Error::ExpectedFunction(other)),
            }
        }
    }

    // T-Let: 先取得绑定项的类型, 再用它扩展主体的上下文。
    Term::Let(bound, body) => {
        let bound_type = type_of(context, bound)?;
        let mut body_context = context.to_vec();
        body_context.insert(0, bound_type);
        type_of(&body_context, body)
    }
}

```

返回练习 11.5.1

11.5.2 解答

这个定义存在两个问题。第一，它改变了求值次序：规则 E-LETV 与 E-LET 规定按值调用，即在 $\text{let } x = t_1 \text{ in } t_2$ 中，必须先把 t_1 归约为值，才能用它替换 x ，继而开始处理 t_2 。

第二，尽管这种翻译保持了类型规则 T-LET 的有效性——这可直接由替换引理（引理 9.3.8）得到——它却不保持“不是良类型的”这一性质。例如，不是良类型的项

$$\text{let } x = \text{unit unit in unit}$$

会被翻译成良类型项 unit ：因为 let 体 unit 中没有出现 x ，不是良类型的子项 unit unit 便直接消失了。

返回练习 11.5.2

11.8.2 解答

一种为记录模式加入类型的方案如下。除了通常的项类型判断，还需要定义一种专门分析模式的类型判断 $\vdash p : T \Rightarrow \Delta$ 。从算法角度看，它以模式 p 和待匹配值的类型 T 为输入；若成功，就返回上下文 Δ ，列出模式中各变量应获得的类型。规则 T-LET 在检查 let 体 t_2 时，把该上下文加入当前上下文 Γ 。以下始终假设，记录模式中不同字段绑定的变量集合互不相交。

新增的模式语法为

$$p ::= x \mid \{l_i = p_i\}^{i \in 1..n}$$

模式类型判断与推广后的 let 类型规则如下：

$$\begin{array}{c} \frac{}{\vdash x : T \Rightarrow x : T} \text{P-VAR} \quad \frac{\vdash p_i : T_i \Rightarrow \Delta_i \text{ 对每个 } i}{\vdash \{l_i = p_i\}^{i \in 1..n} : \{l_i : T_i\}^{i \in 1..n} \Rightarrow \Delta_1, \dots, \Delta_n} \text{P-RCD} \\[2ex] \frac{\Gamma \vdash t_1 : T_1 \quad \vdash p : T_1 \Rightarrow \Delta \quad \Gamma, \Delta \vdash t_2 : T_2}{\Gamma \vdash \text{let } p = t_1 \text{ in } t_2 : T_2} \text{T-LET} \end{array}$$

图 A-1：带类型的记录模式

译者注：P-Var 与 P-Rcd 是图 11-8 中 M-Var 与 M-Rcd 的静态对应，也要按模式结构递归配合。检查模式 $\{a = x, b = y\}$ 时，外层使用 P-Rcd，它的两个前提再分别使用 P-Var。区别在于，运行时的 M 规则产生实际替换 σ ，这里的 P 规则只计算各模式变量的类型，并把它们汇总为上下文 Δ 。

还可以放宽记录模式规则：模式不必列出记录的全部字段，只需写出希望提取的那些字段：

$$\frac{\{l_i \mid i \in 1..n\} \subseteq \{k_j \mid j \in 1..m\} \quad \text{对每个 } i, \text{ 存在 } j \text{ 使 } l_i = k_j \text{ 且 } \vdash p_i : T_j \Rightarrow \Delta_i}{\vdash \{l_i = p_i\}^{i \in 1..n} : \{k_j : T_j\}^{j \in 1..m} \Rightarrow \Delta_1, \dots, \Delta_n} \text{P-RCD'}$$

译者注：这条规则同时使用两套下标，分别枚举模式字段和完整记录类型的字段：

- n 是模式中实际写出的字段数， $i \in 1..n$ 逐一枚举这些字段； l_i 是第 i 个字段的标签， p_i 是它的子模式；
- m 是被匹配记录类型的总字段数， $j \in 1..m$ 逐一枚举这些字段； k_j 是第 j 个字段的标签， T_j 是它的类型；
- $\{l_i \mid i \in 1..n\}$ 是模式使用的标签集合， $\{k_j \mid j \in 1..m\}$ 是记录类型提供的全部标签；子集条件要求前者中的每个标签都出现在后者中；
- 条件 $l_i = k_j$ 表示在记录类型中找到了与当前模式字段同名的字段；
- Δ_i 是子模式 p_i 引入的变量类型绑定，最后把所有 Δ_i 合并为结果上下文。

因此，横线下方的判断可以读作：“模式 $\{l_i = p_i\}$ 用来匹配类型为 $\{k_j : T_j\}$ 的记录时，会引入上下文 $\Delta_1, \dots, \Delta_n$ 。”例如，对模式 $\{b = x\}$ 和记录类型 $\{a : \text{Nat}, b : \text{Bool}, c : \text{Nat}\}$ ，有 $n = 1$ 、 $l_1 = b$ 、 $p_1 = x$ ，以及 $m = 3$ 、 $k_2 = b$ 、 $T_2 = \text{Bool}$ 。因此按标签找到 $l_1 = k_2$ ，再由 P-Var 得到 $\vdash x : \text{Bool} \Rightarrow x : \text{Bool}$ ，最终输出上下文 $x : \text{Bool}$ 。

采用这条规则后，记录投影不必再作为核心语言中单独定义的基本构造，因为它可以用模式匹配和 let 表达，作为下列语法糖：

$$t.l \stackrel{\text{def}}{=} \text{let } \{l = x\} = t \text{ in } x$$

扩展系统的保持性证明与简单类型 lambda 演算几乎相同，只需再证明一个把模式类型关系与运行时匹配联系起来的引理。若替换 σ 与上下文 Δ 有相同的定义域，记 $\Gamma \vdash \sigma : \Delta$ 表示：对每个 $x \in \text{dom}(\Delta)$ ，都有 $\Gamma \vdash \sigma(x) : \Delta(x)$ 。

译者注：这里的 Δ 是“变量到类型”的表， σ 是“变量到实际值”的表。例如，模式类型判断可能得到

$$\Delta = (x : \text{Nat}, y : \text{Bool})$$

而运行时匹配得到

$$\sigma = [x \mapsto 5, y \mapsto \text{true}]$$

二者有相同的定义域，表示它们记录的是同一组变量： $\text{dom}(\Delta) = \text{dom}(\sigma) = \{x, y\}$ 。其中， $\Delta(x)$ 是静态分析为 x 规定的类型， $\sigma(x)$ 是运行时实际绑定给 x 的值。因此 $\Gamma \vdash \sigma : \Delta$ 在这个例子中就是同时要求 $\Gamma \vdash 5 : \text{Nat}$ 和 $\Gamma \vdash \text{true} : \text{Bool}$ ：运行时得到的每个值都符合静态分析为相应变量预测的类型。

于是：

若 $\Gamma \vdash t : T$ 且 $\vdash p : T \Rightarrow \Delta$ ，则 $\text{match}(p, t) = \sigma$ ，并且 $\Gamma \vdash \sigma : \Delta$ 。

还需要把通常的替换引理（引理9.3.8）推广为：

若 $\Gamma, \Delta \vdash t : T$ 且 $\Gamma \vdash \sigma : \Delta$ ，则 $\Gamma \vdash \sigma t : T$ 。

有了这两个引理，T-LET 情形的保持性证明便与从前相同。进展性方面，模式类型判断保证：当 t_1 已求值为值时，规则 E-LETV 中的 $match(p, t_1)$ 一定有定义，不会因为匹配失败而卡住。

[返回练习 11.8.2](#)

11.9.1 解答

$$\text{Bool} \stackrel{\text{def}}{=} \text{Unit} + \text{Unit},$$

$$\text{true} \stackrel{\text{def}}{=} \text{inl unit},$$

$$\text{false} \stackrel{\text{def}}{=} \text{inr unit},$$

$$\text{if } t_0 \text{ then } t_1 \text{ else } t_2 \stackrel{\text{def}}{=} \text{case } t_0 \text{ of inl } x_1 \Rightarrow t_1 \mid \text{inr } x_2 \Rightarrow t_2$$

其中另取互不相同的变量 x_1, x_2 ，并要求它们都不在 t_0, t_1, t_2 中自由出现。

[返回练习 11.9.1](#)

11.11.1 解答

```
equal =
  fix
    (lambda eq:Nat -> Nat -> Bool.
      lambda m:Nat. lambda n:Nat.
        if iszero m then iszero n
        else if iszero n then false
        else eq (pred m) (pred n));
  ► equal : Nat -> Nat -> Bool

plus = fix (lambda p:Nat -> Nat -> Nat.
  lambda m:Nat. lambda n:Nat.
    if iszero m then n else succ (p (pred m) n));
  ► plus : Nat -> Nat -> Nat

times = fix (lambda t:Nat -> Nat -> Nat.
  lambda m:Nat. lambda n:Nat.
    if iszero m then 0 else plus n (t (pred m) n));
  ► times : Nat -> Nat -> Nat

factorial = fix (lambda f:Nat -> Nat.
  lambda m:Nat.
    if iszero m then 1 else times m (f (pred m)));
  ► factorial : Nat -> Nat
```

```
factorial 5;
► 120 : Nat
```

[返回练习 11.11.1](#)

11.11.2 解答

```
letrec plus : Nat -> Nat -> Nat =
  lambda m:Nat. lambda n:Nat.
    if iszero m then n else succ (plus (pred m) n) in
letrec times : Nat -> Nat -> Nat =
  lambda m:Nat. lambda n:Nat.
    if iszero m then 0 else plus n (times (pred m) n) in
letrec factorial : Nat -> Nat =
  lambda m:Nat.
    if iszero m then 1 else times m (factorial (pred m)) in
factorial 5;
► 120 : Nat
```

[返回练习 11.11.2](#)

11.12.1 解答

出人意料的是，进展性并不成立。例如，表达式 `head[T] nil[T]` 已经卡住：没有求值规则适用，但它又不是值。在完整的程序语言中，通常会让这个表达式抛出异常，而不是卡住；第十四章会讨论异常。

[返回练习 11.12.1](#)

11.12.2 解答

并非所有类型标注都能省略：若擦去 `nil` 上的标注，类型唯一性定理就不再成立。类型检查器看到 `nil` 时，知道必须赋予它 `List T` 这一形式的类型，却无法在不作某种猜测的情况下决定 `T` 是什么。当然，更复杂的类型算法——例如 OCaml 编译器使用的算法——恰恰会进行这种推断。第二十二章会回到这一点。

[返回练习 11.12.2](#)

A.8 第 12 章：规范化

12.1.1 解答

问题出在应用情形。设我们要证明 $t_1 t_2$ 可规范化。由归纳假设， t_1 与 t_2 都可规范化；记其范式分别为 v_1 与 v_2 。由类型关系的反演引理 9.3.1， v_1 具有某个函数类型 $T_{11} \rightarrow T_{12}$ 。再由典范形式引理（引理 9.3.4）， v_1 必为 $\lambda x : T_{11}. t_{12}$ 。

可是, 把 $t_1 t_2$ 归约到 $(\lambda x : T_{11}. t_{12}) v_2$ 并没有得到范式, 因为此时 E-APPABS 仍然适用, 产生 $[x \mapsto v_2] t_{12}$ 。要完成证明, 就必须说明这个新项也可规范化; 而结构归纳在这里无法继续, 因为替换可能按照 x 在 t_{12} 中出现的次数复制 v_2 , 使新项比原项 $t_1 t_2$ 更大。

[返回练习 12.1.1](#)

12.1.7 解答

在定义 12.1.2 中增加两项:

$$\begin{aligned} \mathcal{R}_{\text{Bool}}(t) &\iff t \text{ 的求值会在有限步内终止,} \\ \mathcal{R}_{T_1 \times T_2}(t) &\iff t \text{ 的求值会在有限步内终止, 且} \\ &\quad \mathcal{R}_{T_1}(t.1) \text{ 与 } \mathcal{R}_{T_2}(t.2) \end{aligned}$$

引理 12.1.4 的证明需要增加一个情形。设 $T = T_1 \times T_2$ 。对于“仅当”方向, 假设 $\mathcal{R}_T(t)$ 且 $t \rightarrow t'$, 需证 $\mathcal{R}_T(t')$ 。由 $\mathcal{R}_T$ 的定义可知 $\mathcal{R}_{T_1}(t.1)$ 与 $\mathcal{R}_{T_2}(t.2)$ 。规则 E-PROJ1 和 E-PROJ2 给出 $t.1 \rightarrow t'.1$ 与 $t.2 \rightarrow t'.2$; 对二者使用归纳假设, 得到 $\mathcal{R}_{T_1}(t'.1)$ 与 $\mathcal{R}_{T_2}(t'.2)$, 再由定义得到 $\mathcal{R}_T(t')$ 。“如果”方向相同。

最后, 要为引理 12.1.5 的证明补上各条新类型规则的情形。

情形 T-IF:

此时

$$t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3, \quad \Gamma \vdash t_1 : \text{Bool}, \quad \Gamma \vdash t_2 : T, \quad \Gamma \vdash t_3 : T$$

其中 $\Gamma = x_1 : T_1, \dots, x_n : T_n$ 。令 $\sigma = [x_1 \mapsto v_1] \cdots [x_n \mapsto v_n]$ 。由归纳假设, $\mathcal{R}_{\text{Bool}}(\sigma t_1)$ 、 $\mathcal{R}_T(\sigma t_2)$ 和 $\mathcal{R}_T(\sigma t_3)$ 。第一项说明 σt_1 的求值会在有限步内终止; 它又具有布尔类型, 所以其最终值只能是 true 或 false。因此条件表达式 σt 最终会选择 σt_2 或 σt_3 。两者均已满足 $\mathcal{R}_T$, 再由引理 12.1.4 对求值保持 $\mathcal{R}_T$ 的结论, 得到 $\mathcal{R}_T(\sigma t)$ 。

情形 T-TRUE 与 T-FALSE:

立即成立。

情形 T-PAIR:

此时 $t = \{t_1, t_2\}$, 且 $T = T_1 \times T_2$ 。由归纳假设, 对 $i = 1, 2$ 都有 $\mathcal{R}_{T_i}(\sigma t_i)$, 所以两个分量的求值都会在有限步内终止; 按值求值因而也会把 $\{\sigma t_1, \sigma t_2\}$ 求值为一个值。记 σt_i 的范式为 w_i ; 由引理 12.1.4, $\mathcal{R}_{T_i}(w_i)$ 。投影的求值规则给出

$$\{\sigma t_1, \sigma t_2\}.i \longrightarrow^* w_i$$

再次使用引理 12.1.4, 可得 $\mathcal{R}_{T_i}(\{\sigma t_1, \sigma t_2\}.i)$ 。于是由定义得到 $\mathcal{R}_{T_1 \times T_2}(\{\sigma t_1, \sigma t_2\})$, 也就是 $\mathcal{R}_{T_1 \times T_2}(\sigma t)$ 。

情形 T-PROJ1:

由 $\mathcal{R}_{T_1 \times T_2}$ 的定义立即得到。

情形 T-PROJ2:

类似。

[返回练习 12.1.7](#)

A.9 第 13 章：引用

13.1.1 解答



[返回练习 13.1.1](#)

13.1.2 解答

不行：现在，凡是用不同于 *update* 所接收索引的索引调用 *lookup*，都会发散。原因是，在用新函数覆盖 a 之前，必须先取出 a 中原先保存的值；否则，等到真正执行查找时，读到的将是刚写入的新函数，而不是原函数。

[返回练习 13.1.2](#)

13.1.3 解答

设语言提供原始操作 *free*：它接收引用单元，释放相应存储，并且下一个新分配的引用单元会复用同一块内存。于是程序

```
let r = ref 0 in
let s = r in
free r;
let t = ref true in
t := false;
succ (!s)
```

会求值到卡住项 `succ false`。别名在这里至关重要：即使余下表达式中不再自由出现变量 r ，同一位置仍可能由别名 s 访问，所以不能简单规定“若余下表达式中自由出现 r ，则 `free r` 非法”来发现无效释放。

在 C 这类手工管理内存的语言中，很容易写出同类例子；这里的 `ref` 相当于 C 的 `malloc`，`free` 则是 C 中 `free` 的简化版本。

[返回练习 13.1.3](#)

13.3.1 解答

垃圾收集的一种简单形式化方法如下。

1. 令位置集合 L 为有限集，以此刻画内存容量有限。分配规则 E-REFV 还应要求新位置 $\ell \in L$ 。把 L 限定为有限集并不是定义垃圾收集所必需的；这样做是为了让第 5 项中的“内存耗尽”成为一个需要处理的真实情形。
2. 定义存储中的位置可达性。记 $locations(t)$ 为项 t 中提到的位置集合。若 $\ell' \in locations(\mu(\ell))$ ，就说在存储 μ 中 ℓ' 从 ℓ 一步可达；若存在从 ℓ 到 ℓ' 的有限位置序列，且每个位置都从前一位置一步可达，就说 ℓ' 从 ℓ 可达。最后，把存储 μ 中从 $locations(t)$ 可达的位置集合记作 $reachable(t, \mu)$ 。
3. 用关系 $t \mid \mu \rightarrow_{gc} t \mid \mu'$ 描述垃圾收集：

$$\frac{\mu' = \mu \text{ 在 } reachable(t, \mu) \text{ 上的限制}}{t \mid \mu \rightarrow_{gc} t \mid \mu'} \text{ E-GC}$$

也就是说， $\text{dom}(\mu') = reachable(t, \mu)$ ，而且 μ' 在该定义域中的取值与 μ 相同。

4. 把求值序列定义为普通求值步骤与垃圾收集步骤交错组成的序列：

$$\rightarrow^* \stackrel{gc}{\text{def}} (\rightarrow \cup \rightarrow_{gc})^*$$

不能把 E-GC 直接加入普通的单步求值关系；垃圾收集只能在能看见整个待求值项的最外层执行。例如，若允许在应用左侧的求值过程中收集垃圾，就可能复用仍由应用右侧提到的位置，因为只看左侧时看不见这些位置仍然可达。

5. 证明上述改动除了引入“内存耗尽”的可能性之外，不会改变最终结果。更具体地说：

- a. 假设在有限位置集 L 上、允许垃圾收集时有

$$t \mid \mu \xrightarrow{gc}^* t' \mid \mu''$$

那么，在无限存储中不使用垃圾收集时，存在存储 μ' ，使得

$$t \mid \mu \rightarrow^* t' \mid \mu', \quad \text{dom}(\mu'') \subseteq \text{dom}(\mu'), \quad \mu'|_{\text{dom}(\mu'')} = \mu''$$

也就是说，两次执行得到同一个最终项；不带垃圾收集的存储可能保留更多位置，但两个存储在共同定义域上的取值相同。

- b. 反过来，假设在无限存储中不使用垃圾收集时有

$$t \mid \mu \rightarrow^* t' \mid \mu'$$

那么，在有限位置集 L 上允许垃圾收集时，以下两种情况必有一种发生：

- i. 存在存储 μ'' ，使得

$$t \mid \mu \xrightarrow{gc}^* t' \mid \mu'', \quad \text{dom}(\mu'') \subseteq \text{dom}(\mu'), \quad \mu'|_{\text{dom}(\mu'')} = \mu''$$

即带垃圾收集的执行也得到同一个最终项，而最终存储只保留仍然需要的位置；

ii. 求值在某个状态 $t'' \mid \mu''$ 耗尽内存：下一步必须分配新位置，但

$$\text{reachable}(t'', \mu'') = L$$

所有可用位置都仍然可达，因而没有位置可以回收或重新分配。

译者注：作者官方勘误说明，原书本题第 5 项的 5a 与 5b 排写混乱：两条命题比较的应当是“有限位置集上带垃圾收集的执行”与“无限存储中不带垃圾收集的执行”。上文保留原书 5a、5b 的双向模拟结构，并按这项勘误明确两边的运行环境；5b(i) 中存储一致性的范围也按勘误修正为 μ'' 的定义域。

类型安全性还要求同步处理存储类型。若垃圾收集把 μ 限制为 μ' ，就应把相应的存储类型 Σ 也限制到 $\text{dom}(\mu')$ ；否则，原保持性陈述只允许存储类型增长，无法描述垃圾收集删除位置之后的状态。进展性与保持性的陈述和证明都要使用这一同步限制后的存储类型。

这种简化模型忽略了真实存储的若干方面，例如不同种类的值通常占用不同大小的内存，以及终结器和弱指针等更高级的语言特性。操作语义中更精细的垃圾收集处理可见 Morrisett 等人 (1995)。

[返回练习 13.3.1](#)

13.4.1 解答

```
let r1 = ref (lambda x:Nat. 0) in
let r2 = ref (lambda x:Nat. (!r1) x) in
(r1 := (lambda x:Nat. (!r2) x);
 r2)
```

[返回练习 13.4.1](#)

13.5.2 解答

设 μ 是只有一个位置 ℓ 的存储：

$$\mu = (\ell \mapsto \lambda x : \text{Unit}. (!\ell) x)$$

并令 Γ 为空上下文。则 μ 对以下两个存储类型都是良类型的：

$$\Sigma_1 = \ell : \text{Unit} \rightarrow \text{Unit},$$

$$\Sigma_2 = \ell : \text{Unit} \rightarrow (\text{Unit} \rightarrow \text{Unit})$$

[返回练习 13.5.2](#)

13.5.8 解答

这个系统中存在不能规范化的良类型项。练习 13.1.2 已经给出一例。还有一例是

$$t_1 = \lambda r : \text{Ref}(\text{Unit} \rightarrow \text{Unit}). (r := \lambda x : \text{Unit}. (!r) x; (!r) \text{unit}),$$

$$t_2 = \text{ref}(\lambda x : \text{Unit}. x)$$

把 t_1 应用于 t_2 ，会得到一个发散的良类型项。

更一般地，可以按以下步骤借助引用定义任意递归函数；某些函数式语言实现确实使用这种技术。

1. 分配引用单元，并用适当类型的占位函数初始化：

```
factref = ref (lambda _:Nat. 0);
► factref : Ref (Nat -> Nat)
```

2. 定义所需函数的主体，通过引用单元的内容进行递归调用：

```
factbody =
  lambda n:Nat.
    if iszero n then 1 else times n ((!factref) (pred n));
► factbody : Nat -> Nat
```

3. 把真正的函数体“回填”到引用单元：

```
factref := factbody;
```

4. 取出引用单元的内容并使用：

```
fact = !factref;
fact 5;
► 120 : Nat
```

[返回练习 13.5.8](#)

A.10 第 14 章：异常

14.1.1 解答

若在 `error` 上标注它预期具有的类型，就会破坏类型保持性。例如，以下项是良类型的：

$$(\lambda x : \text{Nat}. x)((\lambda y : \text{Bool}. 5)(\text{error as Bool}))$$

这里 `error as  $T$` 表示带类型标注的异常语法。但该项经过一次单步求值后会得到不是良类型的项

$$(\lambda x : \text{Nat}. x)(\text{error as Bool})$$

求值规则把错误从发生处一路向上传播到程序最外层；在传播途中，我们需要把它视为具有不同的类型。规则 T-ERROR 的灵活性正是为此而设。

[返回练习 14.1.1](#)

14.3.1 解答

《Standard ML 定义》(Milner、Tofte、Harper 与 MacQueen, 1997; Milner 与 Tofte, 1991b) 形式化了 `exn` 类型。Harper 与 Stone (2000) 给出了一种相关的处理方式。

[返回练习 14.3.1](#)

14.3.2 解答

见 Leroy 与 Pessaux (2000)。

[返回练习 14.3.2](#)

14.3.3 解答

见 Harper 等人 (1993)。

[返回练习 14.3.3](#)

A.11 第 15 章：子类型化

15.2.1 解答

先用排列规则把 y 字段移到最前面，再用宽度规则丢掉其余字段，最后以传递规则连接两步：

$$\begin{array}{c}
 \frac{}{\{x : \text{Nat}, y : \text{Nat}, z : \text{Nat}\} <: \{y : \text{Nat}, x : \text{Nat}, z : \text{Nat}\}} \text{S-RCDPERM} \\
 \frac{}{\{y : \text{Nat}, x : \text{Nat}, z : \text{Nat}\} <: \{y : \text{Nat}\}} \text{S-RCDWIDTH} \\
 \hline
 \{x : \text{Nat}, y : \text{Nat}, z : \text{Nat}\} <: \{y : \text{Nat}\} \quad \text{S-TRANS}
 \end{array}$$

[返回练习 15.2.1](#)

15.2.2 解答

不是。仍沿用正文中的缩写 R_x 与 R_{xy} 。例如，可以先用包容规则把函数 f 的类型从 $R_x \rightarrow \text{Nat}$ 改成 $R_{xy} \rightarrow \text{Nat}$ ，再应用它：

$$\begin{array}{c}
 \frac{}{\vdash f : R_x \rightarrow \text{Nat}} \\
 \frac{}{R_{xy} <: R_x} \text{S-RCDWIDTH} \quad \frac{}{\text{Nat} <: \text{Nat}} \text{S-REFL} \\
 \hline
 R_x \rightarrow \text{Nat} <: R_{xy} \rightarrow \text{Nat} \quad \text{S-ARROW} \\
 \hline
 \vdash f : R_{xy} \rightarrow \text{Nat} \quad \text{T-SUB} \\
 \frac{}{\vdash xy : R_{xy}} \\
 \hline
 \vdash f xy : \text{Nat} \quad \text{T-APP}
 \end{array}$$

另一棵推导树先把实参 xy 的类型从 R_{xy} 提升为 R_x ：

$$\begin{array}{c}
 \frac{}{\vdash f : R_x \rightarrow \text{Nat}} \\
 \frac{}{\vdash xy : R_{xy}} \text{T-RCD} \\
 \frac{}{R_{xy} <: R_x} \text{S-RCDWIDTH} \quad \frac{}{R_x <: R_x} \text{S-REFL} \\
 \hline
 R_{xy} <: R_x \quad \text{S-TRANS} \\
 \hline
 \vdash xy : R_x \quad \text{T-SUB} \\
 \hline
 \vdash f xy : \text{Nat} \quad \text{T-APP}
 \end{array}$$

第二种做法还能任意插入更多自反与传递步骤。因此，本演算中每个可导出的类型判断实际上都有无穷多棵不同的推导树。

[返回练习 15.2.2](#)

15.2.3 解答

1. 共有六个：

$$\{a : \text{Top}, b : \text{Top}\}, \quad \{b : \text{Top}, a : \text{Top}\}, \quad \{a : \text{Top}\}, \quad \{b : \text{Top}\}, \quad \{\}, \quad \text{Top}$$

2. 例如，令

$$S_0 = \{\}, \quad S_1 = \{a : \text{Top}\}, \quad S_2 = \{a : \text{Top}, b : \text{Top}\}, \quad \dots$$

每增加一个字段便得到 $S_{i+1} <: S_i$ 。

3. 例如令 $T_i = S_i \rightarrow \text{Top}$ 。箭头类型的参数位置逆变，所以 $T_i <: T_{i+1}$ ，形成无限上升链。

返回练习 15.2.3**15.2.4 解答**

第一问不存在。若这样的类型存在，它只能是箭头类型或记录类型，不可能是 **Top**。然而，记录类型不可能成为任意箭头类型的子类型，箭头类型也不可能成为任意记录类型的子类型。

第二问同样不存在。若箭头类型 $T_1 \rightarrow T_2$ 是所有箭头类型的超类型，根据箭头规则的逆变前提，其参数类型 T_1 必须是每个其他类型 S_1 的子类型；第一问已经说明这种类型不存在。

返回练习 15.2.4**15.2.5 解答**

若保留现有求值语义，加入这条规则并不安全。它会给出 $\text{Nat} \times \text{Bool} <: \text{Nat}$ ，于是本来会卡住的项

$$\text{succ}(5, \text{true})$$

也能通过类型检查，违反进展性定理。在§15.6介绍的强制转换语义下，这条规则是安全的；不过，即使采用该语义，它也会给子类型检查算法带来一些困难。

返回练习 15.2.5**15.3.1 解答**

向子类型关系加入错误的类型对，既可能破坏保持性，也可能破坏进展性。例如，若加入公理

$$\{x : \{\}\} <: \{x : \text{Top} \rightarrow \text{Top}\}$$

便可推出 $t = (\{x = \{\}\}.x)\{\}$ 具有类型 Top 。但 $t \longrightarrow \{\}\{\}$ ，而结果无法赋型，因而违反保持性。若加入公理

$$\{\} <: \text{Top} \rightarrow \text{Top}$$

则项 $\{\}\{\}$ 可以赋型，却既不是值也不能求值，因而违反进展性。

反过来，若从子类型关系中删去某些判断 $S <: T$ ，使更少的类型构成子类型关系，并不会破坏进展性或保持性。进展性定理只在前提中提到类型关系；限制子类型关系，继而限制类型关系，只会减少需要考虑的良类型项。保持性方面，即使删去 S-TRANS 也不会有问题，因为 T-SUB 可以恢复它在类型推导中的作用。例如，原先可以写成

$$\frac{\Gamma \vdash t : S \quad \frac{S <: U \quad U <: T}{S <: T} \text{S-TRANS}}{\Gamma \vdash t : T} \text{T-SUB}$$

的推导，总可以改写为连续两次包容：

$$\frac{\frac{\Gamma \vdash t : S \quad S <: U}{\Gamma \vdash t : U} \text{T-SUB} \quad U <: T}{\Gamma \vdash t : T} \text{T-SUB}$$

返回练习 15.3.1

15.3.2 解答

1. 对 $S <: T_1 \rightarrow T_2$ 的子类型推导作归纳。检查 图15-1和15-3中的规则，末条规则只能是 S-REFL、S-TRANS 或 S-ARROW。

若为 S-REFL，则 $S = T_1 \rightarrow T_2$ ，结论由自反性立即得到。若为 S-ARROW，其两个前提恰好就是所需结论。

若为 S-TRANS，则存在 U ，使两个子推导分别以 $S <: U$ 和 $U <: T_1 \rightarrow T_2$ 为结论。对第二个子推导使用归纳假设，得到 $U = U_1 \rightarrow U_2$ 、 $T_1 <: U_1$ 和 $U_2 <: T_2$ 。现在已知 U 是箭头类型，可以对第一个子推导使用归纳假设，得到 $S = S_1 \rightarrow S_2$ 、 $U_1 <: S_1$ 和 $S_2 <: U_2$ 。最后两次使用传递性，得到 $T_1 <: S_1$ 与 $S_2 <: T_2$ 。

2. 对 $S <: \{l_i : T_i\}^{i \in 1..n}$ 的子类型推导作归纳。末条规则只能是 S-REFL、S-TRANS、S-RCDWIDTH、S-RCDDEPTH 或 S-RCDPERM。除传递情形外，结论都可以直接从相应规则的前提读出。

若末条规则为 S-TRANS，则存在记录类型 $U = \{u_a : U_a\}^{a \in 1..o}$ ，且有 $S <: U$ 和 $U <: \{l_i : T_i\}^{i \in 1..n}$ 。对后一个子推导使用归纳假设，可知每个 l_i 都是某个 u_a ，且对应字段满足 $U_a <: T_i$ 。再对前一个子推导使用归纳假设，得到 $S = \{k_j : S_j\}^{j \in 1..m}$ ，每个 u_a 都是某个 k_j ，且对应字段满足 $S_j <: U_a$ 。标签集合的包含关系具有传递

性，字段类型的子类型关系也具有传递性，所以每个 l_i 都出现在 S 中，相应字段满足 $S_j <: T_i$ 。

[返回引理 15.3.2](#)

15.3.6 解答

两部分都对类型推导作归纳。这里只写第一部分；第二部分相同。

检查类型规则可知，以 $\vdash v : T_1 \rightarrow T_2$ 为结论的推导，末条规则只能是 T-ABS 或 T-SUB。若为 T-ABS，结论直接成立。若为 T-SUB，从前提得到 $\vdash v : S$ 和 $S <: T_1 \rightarrow T_2$ 。由子类型关系的反演引理， S 必为 $S_1 \rightarrow S_2$ 。于是可以对前一条类型前提的推导使用归纳假设，得到 v 是 lambda 抽象。

[返回引理 15.3.6](#)

A.12 第 16 章：子类型化的元理论

16.1.2 解答

第一项。对 S 的结构作归纳。若 $S = \text{Top}$ ，直接使用 S-Top；若 $S = S_1 \rightarrow S_2$ ，先根据归纳假设分别构造 $S_1 <: S_1$ 与 $S_2 <: S_2$ 的无自反规则推导，再应用 S-ARROW。记录类型的情形相同：先对各字段类型使用归纳假设，再应用 S-RCD。因此，任何 $S <: S$ 的推导都不必使用 S-REFL。

第二项。先利用第一项，把原推导中的每次 S-REFL 都换成不使用自反规则的推导。接下来只需证明：任何不含 S-REFL 的 $S <: T$ 推导，都能进一步改造成不含 S-TRANS 的推导。

把待改造的推导视为一棵树，并对它的大小作强归纳。这里按大小而不按推导结构归纳，是因为在箭头和记录情形中，我们会把原推导的若干部分重新组合成新的推导；这些新推导不是原推导中现成的子树，但只要规模严格小于原推导，强归纳假设仍然适用。

若原推导的末条规则不是 S-TRANS，就分别对它的每个直接子推导使用归纳假设，消去其中的传递规则。末条规则本身也不是传递规则，因此重新组合之后，整个推导便不再含有 S-TRANS。

下面只需处理末条规则为 S-TRANS 的情况。此时两个直接子推导的结论分别是 $S <: U$ 和 $U <: T$ 。下面按照这两个子推导各自的末条规则分类讨论；情形名称中斜线左侧表示 $S <: U$ 推导的末条规则，斜线右侧表示 $U <: T$ 推导的末条规则。各情形的目标都是绕过中间类型 U ，重新构造 $S <: T$ 的无传递规则推导。

情形任意规则 / S-Top。右侧推导以 S-Top 结束，所以 $T = \text{Top}$ 。无论 S 是什么，直接使用 S-Top 就得到 $S <: T$ 。

情形 S-Top / 任意规则。左侧推导以 S-Top 结束，所以 $U = \text{Top}$ 。先对右侧子推导使用归纳假设，设它已经不含传递规则。检查其末条规则：自反规则已经消去，箭头规则和记录规则又都要求左侧类型分别是箭头或记录，因此只有 S-Top 可能适用。于是 $T = \text{Top}$ ，结论仍由 S-Top 得到。

情形 S-ARROW / S-ARROW。写 $S = S_1 \rightarrow S_2$ 、 $U = U_1 \rightarrow U_2$ 、 $T = T_1 \rightarrow T_2$ 。两个子推导给出

$$U_1 <: S_1, \quad S_2 <: U_2, \quad T_1 <: U_1, \quad U_2 <: T_2$$

为了直接应用 S-ARROW，需要得到 $T_1 <: S_1$ 和 $S_2 <: T_2$ 。分别把上面两列中的推导用 S-TRANS 接起来，即可暂时构造出这两个判断的推导。它们都严格小于原来以传递规则结束的整个推导，所以归纳假设可以再次消去其中的 S-TRANS。最后应用 S-ARROW，便得到不含传递规则的 $S_1 \rightarrow S_2 <: T_1 \rightarrow T_2$ 推导。

译者注：这里的规模递减可以按规则节点数直接核对。原推导包含四份处理参数和结果类型的内部推导，外面还有两次 S-ARROW 和最末端的一次 S-TRANS，共多出三个规则节点。重新配对以后，两份新推导合起来仍使用原来的四份内部推导，但只新加两次 S-TRANS，总节点数已经少了一个；分别来看，每份新推导只使用其中两份内部推导，当然也都严格小于原推导。归纳假设正是分别用于这两份较小的新推导，所以临时加入传递规则并不会形成循环论证。

情形 $S\text{-RCD} / S\text{-RCD}$ 。证明相同：对目标记录中的每个字段，先用 $S\text{-TRANS}$ 连接左右两个记录子推导给出的字段类型关系。所得新推导都严格小于原推导，因此可以用归纳假设消去其中的传递规则；最后应用 $S\text{-RCD}$ ，重新组成不含传递规则的记录子类型推导。

其余组合只有 $S\text{-ARROW} / S\text{-RCD}$ 和 $S\text{-RCD} / S\text{-ARROW}$ ；它们会要求中间类型 U 同时是箭头类型与记录类型，因此不可能发生。

返回引理 16.1.2

16.1.3 解答

加入 Bool 后，引理第一项应改为：“每个类型 S 的 $S <: S$ 都能在不使用 $S\text{-REFL}$ 的情况下导出，只有 $S = \text{Bool}$ 时例外。”另一种做法是在子类型定义中显式加入

$$\text{Bool} <: \text{Bool}$$

再证明扩充后的系统可以删去 $S\text{-REFL}$ 。

译者注： Bool 在这里成为例外，只因为它是没有内部结构的基本类型。删去 $S\text{-REFL}$ 后， $S\text{-TOP}$ 只能导出 $\text{Bool} <: \text{Top}$ ， $S\text{-ARROW}$ 只适用于箭头类型， $S\text{-RCD}$ 只适用于记录类型；因此没有规则能够导出 $\text{Bool} <: \text{Bool}$ 。每加入一种新的基本类型，都要为它保留这样的自身子类型规则，或者继续使用通用的 $S\text{-REFL}$ 。

返回练习 16.1.3

16.2.5 解答

要证明的是：对任意声明式推导 $\Gamma \vdash t : T$ ，都能构造一份算法推导 $\Gamma \vdash_{\mathbf{A}} t : S$ ，且 $S <: T$ 。对给定的声明式推导作归纳，按其最后使用的规则分为以下情形。在每个情形中，归纳假设均应用于末条规则的直接子推导。

情形 T-Var。已知 $x : T \in \Gamma$ 。由 TA-VAR 得到 $\Gamma \vdash_{\mathbf{A}} x : T$ 。取 $S = T$ ；由子类型关系的自反性， $S <: T$ 。

情形 T-Abs。声明式推导的直接子推导为

$$\Gamma, x : T_1 \vdash t_2 : T_2$$

结论为 $\Gamma \vdash \lambda x : T_1. t_2 : T_1 \rightarrow T_2$ 。归纳假设给出某个 $S_2 <: T_2$ ，使 $\Gamma, x : T_1 \vdash_{\mathbf{A}} t_2 : S_2$ 。使用 TA-ABS 得到

$$\Gamma \vdash_{\mathbf{A}} \lambda x : T_1. t_2 : T_1 \rightarrow S_2$$

又因为 $T_1 <: T_1$ 且 $S_2 <: T_2$ ，规则 $S\text{-ARROW}$ 给出

$$T_1 \rightarrow S_2 <: T_1 \rightarrow T_2$$

所以算法规则为这个抽象算出了声明式类型的一个子类型。

情形 T-App。声明式推导的两个直接子推导为

$$\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}$$

结论为 $\Gamma \vdash t_1 t_2 : T_{12}$ 。分别应用归纳假设，得到

$$\Gamma \vdash_A t_1 : S_1, \quad S_1 <: T_{11} \rightarrow T_{12},$$

$$\Gamma \vdash_A t_2 : S_2, \quad S_2 <: T_{11}$$

由子类型反演， S_1 必须也是箭头类型。因此可以写成 $S_1 = S_{11} \rightarrow S_{12}$ ，并且有

$$T_{11} <: S_{11} \quad \text{和} \quad S_{12} <: T_{12}$$

实参的算法类型满足 $S_2 <: T_{11}$ ，而参数类型又满足 $T_{11} <: S_{11}$ ，所以传递性给出

$$S_2 <: S_{11}$$

再由算法子类型关系的完备性，可将该判断改写为 $\vdash_A S_2 <: S_{11}$ 。这正是 TA-APP 所需的子类型前提，因而可以导出

$$\Gamma \vdash_A t_1 t_2 : S_{12}$$

前面已经得到 $S_{12} <: T_{12}$ ，因此应用情形完成。

情形 T-Rcd。对每个字段的直接子推导 $\Gamma \vdash t_i : T_i$ 应用归纳假设，得到某个 $S_i <: T_i$ ，使 $\Gamma \vdash_A t_i : S_i$ 。使用 TA-RCD 可得

$$\Gamma \vdash_A \{l_i = t_i\}^{i \in 1..n} : \{l_i : S_i\}^{i \in 1..n}$$

由记录子类型规则，每个 $S_i <: T_i$ 共同给出

$$\{l_i : S_i\}^{i \in 1..n} <: \{l_i : T_i\}^{i \in 1..n}$$

因此记录情形成立。

情形 T-Proj。设声明式前提为

$$\Gamma \vdash t_1 : \{l_i : T_i\}^{i \in 1..n}$$

而结论是 $\Gamma \vdash t_1.l_j : T_j$ 。归纳假设给出 $\Gamma \vdash_A t_1 : S$ ，且

$$S <: \{l_i : T_i\}^{i \in 1..n}$$

由子类型反演， S 必须是一个记录类型，并且至少包含字段 $l_j : S_j$ ，其中 $S_j <: T_j$ 。因此 TA-PROJ 导出

$$\Gamma \vdash_A t_1.l_j : S_j$$

而 $S_j <: T_j$ ，正好得到所需结论。

情形 T-Sub。设末条规则的直接子推导为 $\Gamma \vdash t : U$ ，并且 $U <: T$ 。归纳假设给出某个 $S <: U$ ，使 $\Gamma \vdash_A t : S$ 。由子类型关系的传递性， $S <: T$ 。同一份算法推导因此就满足定理结论。

返回定理 16.2.5

16.2.6 解答

若删去 S-ARROW, 项 $\lambda x : \{a : \text{Nat}\}. x$ 在声明式规则下同时具有

$$\{a : \text{Nat}\} \rightarrow \{a : \text{Nat}\} \quad \text{和} \quad \{a : \text{Nat}\} \rightarrow \text{Top}$$

两种类型。删去箭头子类型规则后, 这两个类型不可比较, 也不存在同时位于二者之下的类型, 所以该项没有最小类型。

[返回练习 16.2.6](#)

16.3.2 解答

以下两个算法相互递归。并算法总会返回一个类型; 交算法在不存在交时失败。

$$S \sqcup T = \begin{cases} \text{Bool} & \text{若 } S = T = \text{Bool} \\ M_1 \rightarrow J_2 & \begin{array}{l} \text{若 } S = S_1 \rightarrow S_2, \quad T = T_1 \rightarrow T_2, \\ S_1 \sqcap T_1 = M_1, \quad S_2 \sqcup T_2 = J_2 \end{array} \\ \{j_i : J_i\}^{i \in 1..q} & \begin{array}{l} \text{若 } S = \{k_j : S_j\}^{j \in 1..m}, \quad T = \{l_i : T_i\}^{i \in 1..n}, \\ \{j_i\}^{i \in 1..q} = \{k_j\}^{j \in 1..m} \cap \{l_i\}^{i \in 1..n}, \\ S_j \sqcup T_i = J_i \quad \text{对每个 } j_i = k_j = l_i \end{array} \\ \text{Top} & \text{其他情形} \end{cases}$$

$$S \sqcap T = \begin{cases} S & \text{若 } T = \text{Top} \\ T & \text{若 } S = \text{Top} \\ \text{Bool} & \text{若 } S = T = \text{Bool} \\ J_1 \rightarrow M_2 & \begin{array}{l} \text{若 } S = S_1 \rightarrow S_2, \quad T = T_1 \rightarrow T_2, \\ S_1 \sqcup T_1 = J_1, \quad S_2 \sqcap T_2 = M_2 \end{array} \\ \{m_i : M_i\}^{i \in 1..q} & \begin{array}{l} \text{若 } S = \{k_j : S_j\}^{j \in 1..m}, \quad T = \{l_i : T_i\}^{i \in 1..n}, \\ \{m_i\}^{i \in 1..q} = \{k_j\}^{j \in 1..m} \cup \{l_i\}^{i \in 1..n}, \\ S_j \sqcap T_i = M_i \quad \text{对每个 } m_i = k_j = l_i, \\ M_i = S_j \quad \text{若 } m_i = k_j \text{ 只出现在 } S \text{ 中}, \\ M_i = T_i \quad \text{若 } m_i = l_i \text{ 只出现在 } T \text{ 中} \end{array} \\ \text{fail} & \text{其他情形} \end{cases}$$

译者注：箭头类型的并在参数位置使用交，正是参数逆变的结果。设 $U_1 \rightarrow U_2$ 是 $S_1 \rightarrow S_2$ 与 $T_1 \rightarrow T_2$ 的公共超类型。由 S-ARROW，前一个子类型判断要求 $U_1 <: S_1$ 且 $S_2 <: U_2$ ，后一个要求 $U_1 <: T_1$ 且 $T_2 <: U_2$ 。因此，公共超类型的参数类型 U_1 必须是 S_1, T_1 的共同下界，结果类型 U_2 则必须是 S_2, T_2 的共同上界。为了得到最小的公共函数超类型，前者应取最大公共下界，即交；后者应取最小公共上界，即并。这就得到

$$(S_1 \rightarrow S_2) \sqcup (T_1 \rightarrow T_2) = (S_1 \sqcap T_1) \rightarrow (S_2 \sqcup T_2)$$

例如，若某个函数可能具有 $\{x : \text{Bool}\} \rightarrow \text{Bool}$ 或 $\{y : \text{Bool}\} \rightarrow \text{Bool}$ 中的任一类型，那么只有同时含有 x 和 y 字段的实参才肯定能被两种函数接受。因此，这两个函数类型的并是

$$\{x : \text{Bool}, y : \text{Bool}\} \rightarrow \text{Bool}$$

求交时情形对偶：参数位置取并，结果位置取交。

记录交中，公共字段取字段类型的交；只出现在一侧的字段原样保留。任何递归调用失败时，整次交计算失败。递归调用时两种输入类型的总大小严格减小，因此算法不会发散。

并算法的箭头分支会调用交算法计算参数类型的交；若这次调用失败，并算法就继续落入最后的默认分支，返回 Top。例如，

$$(\text{Bool} \rightarrow \text{Top}) \sqcup (\{\} \rightarrow \text{Top}) = \text{Top}$$

两种算法都不会发散，因为每次递归调用都会减小输入 S, T 的总大小。交算法虽然可能失败，但只要两个输入存在共同下界，它就一定成功。

引理 A.10. 若 $L <: S$ 且 $L <: T$ ，则存在某个 M ，使 $S \sqcap T = M$ 。

证明. 对 S 的大小归纳（等价地，也可对 T 的大小归纳），并按 S, T 的外层形式分类。若其中一个是 Top，交算法的前两个分支之一适用，结果分别为另一个输入。若 S, T 的外层形式不同，则子类型反演引理会对共同下界 L 的形式提出互不相容的要求。例如， S 是箭头类型时， L 也必须是箭头类型； T 是记录类型时， L 又必须是记录类型。^a因此只剩三种情形。

若 $S = T = \text{Bool}$ ，交算法的第三个分支立即返回 Bool。

若 $S = S_1 \rightarrow S_2$ 且 $T = T_1 \rightarrow T_2$ ，并算法的全定义性保证第一次递归调用 $S_1 \sqcup T_1$ 返回某个 J_1 。子类型反演又说明 $L = L_1 \rightarrow L_2$ ，并且 $L_2 <: S_2, L_2 <: T_2$ 。于是 L_2 是 S_2, T_2 的共同下界，归纳假设保证 $S_2 \sqcap T_2$ 成功并返回某个 M_2 。所以 $S \sqcap T = J_1 \rightarrow M_2$ 。

最后，设 $S = \{k_j : S_j\}^{j \in 1..m}$ ， $T = \{l_i : T_i\}^{i \in 1..n}$ 。由子类型反演， L 必是记录类型，而且包含 S, T 中出现的全部标签。对同时出现在 S, T 中的每个标签， L 中对应字段类型都是两侧字段类型的共同子类型；归纳假设因此保证交算法对所有公共字段的递归调用都成功。只出现在一侧的字段无需递归求交，故整次记录求交成功。□

下面验证这些定义确实计算出并与交。论证分成两部分：[命题A.11](#)说明求得的交是 S, T 的下界、求得的并是它们的上界；[命题A.12](#)说明求得的交大于每个共同下界，求得的并小于每个共同上界。

命题 A.11.

1. 若 $S \sqcup T = J$, 则 $S <: J$ 且 $T <: J$ 。
2. 若 $S \sqcap T = M$, 则 $M <: S$ 且 $M <: T$ 。

证明. 对 $S \sqcup T = J$ 或 $S \sqcap T = M$ 的“推导”大小作直接归纳; 这里的大小就是为了算出 J 或 M 而对两个定义进行的递归调用次数。□

命题 A.12.

1. 设 $S \sqcup T = J$, 且对某个 U 有 $S <: U$ 与 $T <: U$ 。则 $J <: U$ 。
2. 设 $S \sqcap T = M$, 且对某个 L 有 $L <: S$ 与 $L <: T$ 。则 $L <: M$ 。

证明. 两部分同时对 S, T 的大小归纳; 给定 S, T 后, 依次证明两部分。

先证第一项, 并按 U 的形式分类。若 $U = \text{Top}$, 则任何 J 都满足 $J <: U$ 。若 $U = \text{Bool}$, 子类型反演说明 $S = T = \text{Bool}$, 所以 $J = \text{Bool}$ 。

若 $U = U_1 \rightarrow U_2$, 子类型反演给出 $S = S_1 \rightarrow S_2$ 、 $T = T_1 \rightarrow T_2$, 并且

$$U_1 <: S_1, \quad U_1 <: T_1, \quad S_2 <: U_2, \quad T_2 <: U_2$$

由归纳假设, S_1, T_1 的交 M_1 位于 U_1 之上, 即 $U_1 <: M_1$; S_2, T_2 的并 J_2 位于 U_2 之下, 即 $J_2 <: U_2$ 。应用 S-ARROW, 得到 $M_1 \rightarrow J_2 <: U_1 \rightarrow U_2$ 。

若 U 是记录类型, 子类型反演说明 S, T 也都是记录类型; S, T 的标签集合都包含 U 的标签集合, 而且 U 每个字段的类型都是两侧对应字段类型的超类型。因此, S, T 的并至少包含 U 的全部标签; 对每个公共字段, 归纳假设说明并中的字段类型是 U 对应字段类型的子类型。由 S-RCD 得 $J <: U$ 。

再证第二项, 并按 S, T 的形式分类。若其中一个是 Top , 交就是另一个输入, 结论立即成立。两个非最大类型具有不同外层形式的情形, 由 [引理A.10](#)的论证可知不可能发生。若二者都是 Bool , 结论同样立即成立。

若 $S = S_1 \rightarrow S_2$ 、 $T = T_1 \rightarrow T_2$, 子类型反演给出 $L = L_1 \rightarrow L_2$, 并且

$$S_1 <: L_1, \quad T_1 <: L_1, \quad L_2 <: S_2, \quad L_2 <: T_2$$

由归纳假设, S_1, T_1 的并 J_1 满足 $J_1 <: L_1$, S_2, T_2 的交 M_2 满足 $L_2 <: M_2$ 。再由 S-ARROW 得 $L_1 \rightarrow L_2 <: J_1 \rightarrow M_2$ 。

若 S, T 都是记录类型, 子类型反演说明 L 也是记录类型, 并包含 S, T 的全部标签 (还可能更多), 对应字段类型满足所需子类型关系。对交 M 中的每个标签 m_l , 分三种情况。若它同时出现在 S, T 中, 其在 M 中的类型是两侧字段类型的交, 归纳假设说明 L 中对对应字段类型是它的子类型。若 m_l 只出现在 S 中, 则它在 M 中沿用 S 中的类型, 而我们已知 L 中对对应字段类型更小。只出现在 T 中的情况相同。于是由 S-RCD 得 $L <: M$ 。□

返回命题 16.3.2

^a严格地说, [引理15.3.2](#)没有讨论 Bool 。补上的布尔情形只需说明: Bool 的子类型只有它自身。

16.3.3 解答

最小类型是 Top ，即 Bool 与 $\{\}$ 的并。不过，这个项能够通过类型检查可视为语言的一个弱点： Top 型值没有可供使用的操作，程序员通常不会有意写出这样的条件式。一种做法是从系统中删除 Top ，并把求并改成偏操作；另一种做法是保留 Top ，但让类型检查器在项的最小类型为 Top 时发出警告。

返回练习 16.3.3

16.3.4 解答

处理不变的 Ref 很直接，只需在并、交算法中分别加入一条分支：

$$\begin{aligned}\text{Ref } S_1 \sqcup \text{Ref } T_1 &= \text{Ref } T_1 \quad \text{若 } S_1 <: T_1 \text{ 且 } T_1 <: S_1 \\ \text{Ref } S_1 \sqcap \text{Ref } T_1 &= \text{Ref } T_1 \quad \text{若 } S_1 <: T_1 \text{ 且 } T_1 <: S_1\end{aligned}$$

译者注：这里沿用§15.5中 Ref 的不变子类型规则。一个 $\text{Ref } S_1$ 既能读取又能写入：要把它当作 $\text{Ref } T_1$ 使用，读取要求 $S_1 <: T_1$ ，写入则要求 $T_1 <: S_1$ 。因此，只有在两个元素类型互为子类型时， $\text{Ref } S_1$ 与 $\text{Ref } T_1$ 才能相互替代；此时任取其中一个，都可作为二者的并或交。若两者不等价，直接取 $\text{Ref}(S_1 \sqcup T_1)$ 会扩大允许写入的值的范围，不再类型安全。此时求并会落入默认分支得到 Top ，求交则可能失败。

把 Ref 分解为 Source 和 Sink 后会遇到严重困难：子类型关系不再总有并或交。例如， $\text{Ref}\{a : \text{Nat}, b : \text{Bool}\}$ 与 $\text{Ref}\{a : \text{Nat}\}$ 同时是 $\text{Source}\{a : \text{Nat}\}$ 和 $\text{Sink}\{a : \text{Nat}, b : \text{Bool}\}$ 的子类型，但后两个类型没有共同下界。

最简单的解决方式之一，是只加入 Source 或只加入 Sink 。许多应用只需其中一个。例如，§18.12中的类实现只需 Source ；并发语言中的服务器进程则可能只需要向外传递发送能力，即 Sink ；接收能力只由服务器自身持有。

若系统只加入 Source ，并算法在加入上面的 Ref 分支后，再加入以下四种组合，仍然是完备的：

$$\begin{aligned}\text{Ref } S_1 \sqcup \text{Ref } T_1 &= \text{Source } J_1 \quad \text{若 } S_1 \sqcup T_1 = J_1 \\ \text{Source } S_1 \sqcup \text{Source } T_1 &= \text{Source } J_1 \quad \text{若 } S_1 \sqcup T_1 = J_1 \\ \text{Ref } S_1 \sqcup \text{Source } T_1 &= \text{Source } J_1 \quad \text{若 } S_1 \sqcup T_1 = J_1 \\ \text{Source } S_1 \sqcup \text{Ref } T_1 &= \text{Source } J_1 \quad \text{若 } S_1 \sqcup T_1 = J_1\end{aligned}$$

交算法也加入类似分支。

另一种方案由 Hennessy 与 Riely (1998) 提出：让引用类型接受两个参数。 $\text{Ref } S \ T$ 允许写入 S 型值、读出 T 型值；它对第一个参数逆变，对第二个参数协变。此时可把 $\text{Sink } S$ 定义为 $\text{Ref } S \ \text{Top}$ ，把 $\text{Source } T$ 定义为 $\text{Ref } \text{Bot } T$ 。

返回练习 16.3.4

16.4.1 解答

需要。合适的规则是

$$\frac{\Gamma \vdash_A t_1 : T_1 \quad T_1 = \text{Bot} \quad \Gamma \vdash_A t_2 : T_2 \quad \Gamma \vdash_A t_3 : T_3 \quad T_2 \sqcup T_3 = T}{\Gamma \vdash_A \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \text{TA-IF}$$

另一条看似自然的候选规则会直接把整个条件式赋为 Bot:

$$\frac{\Gamma \vdash_A t_1 : T_1 \quad T_1 = \text{Bot} \quad \Gamma \vdash_A t_2 : T_2 \quad \Gamma \vdash_A t_3 : T_3}{\Gamma \vdash_A \text{if } t_1 \text{ then } t_2 \text{ else } t_3 : \text{Bot}} \text{TA-IF}$$

这条规则从运行行为看是安全的: Bot 没有值, 条件项不可能求得 true 或 false, 因而整个条件式不会产生普通结果。但它可能给某些项赋予声明式规则无法赋予的类型, 从而破坏算法类型关系的可靠性定理 [定理16.2.4](#)。

译者注: 例如, 设上下文中有 $x : \text{Bot}$, 考虑 $\text{if } x \text{ then true else false}$ 。上面的候选规则会把整个条件式赋为 Bot。声明式系统可以先由 $\text{Bot} <: \text{Bool}$ 把 x 当作布尔值, 再根据两个分支得到整个条件式的类型 Bool; 它无法反向把该类型降为 Bot。因此, 这个项并非完全无法由声明式系统赋型; 问题在于, 候选算法规则赋予它的 Bot 无法由声明式规则推导出来。

[返回练习 16.4.1](#)

A.13 第 17 章：子类型化的 ML 实现

17.3.1 解答

只需把命题16.3.2的证明练习中给出的算法写成程序：

```
let rec join tyS tyT =
  match (tyS,tyT) with
  | (TyArr(tyS1,tyS2),TyArr(tyT1,tyT2)) ->
    (try TyArr(meet tyS1 tyT1, join tyS2 tyT2)
     with Not_found -> TyTop)
  | (TyBool,TyBool) ->
    TyBool
  | (TyRecord(fS), TyRecord(fT)) ->
    let labelsS = List.map (fun (li,_) -> li) fS in
    let labelsT = List.map (fun (li,_) -> li) fT in
    let commonLabels =
      List.find_all (fun l -> List.mem l labelsT) labelsS in
    let commonFields =
      List.map (fun li ->
        let tySi = List.assoc li fS in
        let tyTi = List.assoc li fT in
        (li, join tySi tyTi))
        commonLabels in
    TyRecord(commonFields)
  | _ ->
    TyTop

and meet tyS tyT =
  match (tyS,tyT) with
  | (_,TyTop) -> tyS
  | (TyTop,_) -> tyT
  | (TyArr(tyS1,tyS2),TyArr(tyT1,tyT2)) ->
    TyArr(join tyS1 tyT1, meet tyS2 tyT2)
  | (TyBool,TyBool) ->
    TyBool
  | (TyRecord(fS), TyRecord(fT)) ->
    let labelsS = List.map (fun (li,_) -> li) fS in
    let labelsT = List.map (fun (li,_) -> li) fT in
    let allLabels =
      List.append
        labelsS
        (List.find_all
          (fun l -> not (List.mem l labelsS)) labelsT) in
    let allFields =
```

```

    List.map (fun li ->
        if not(List.mem li labelsS) then
            (li, List.assoc li fT)
        else if not(List.mem li labelsT) then
            (li, List.assoc li fS)
        else
            let tySi = List.assoc li fS in
            let tyTi = List.assoc li fT in
            (li, meet tySi tyTi))
    allLabels in
    TyRecord(allFields)
| _ ->
    raise Not_found

```

再为条件式补上类型检查分支：

```

| TmTrue(fi) ->
    TyBool
| TmFalse(fi) ->
    TyBool
| TmIf(fi,t1,t2,t3) ->
    if subtype (typeof ctx t1) TyBool then
        join (typeof ctx t2) (typeof ctx t3)
    else
        error fi "guard of conditional not a boolean"

```

相应的 Rust 对照实现如下。为使代码能够独立阅读，先给出所需的类型定义：

Rust 对照实现（译者增补）

```

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum JoinType {
    Bool,
    Top,
    Arrow(Box<JoinType>, Box<JoinType>),
    Record(Vec<(String, JoinType)>),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum JoinTerm {
    True,
    False,
    If(Box<JoinTerm>, Box<JoinTerm>, Box<JoinTerm>),
}

```

```

Typed(JoinType),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum JoinTypeError {
    ExpectedBoolean(JoinType),
}

/// 计算两个类型的最小公共超类型（并）。
/// 结果同时是两个输入类型的超类型，并且在所有公共超类型中尽可能小；
/// 条件式用它合并两个分支的类型。
pub fn join(left: &JoinType, right: &JoinType) -> JoinType {
    match (left, right) {
        (JoinType::Bool, JoinType::Bool) => JoinType::Bool,
        (JoinType::Arrow(left_in, left_out), JoinType::Arrow(right_in,
↪ right_out)) => {
            meet(left_in, right_in).map_or(JoinType::Top, |input| {
                JoinType::Arrow(Box::new(input), Box::new(join(left_out,
↪ right_out)))
            })
        }
        (JoinType::Record(left_fields), JoinType::Record(right_fields)) =>
↪ JoinType::Record(
            left_fields
                .iter()
                .filter_map(|(label, left_type)| {
                    right_fields
                        .iter()
                        .find(|(right_label, _)| right_label == label)
                        .map(|(_, right_type)| (label.clone(), join(left_type,
↪ right_type)))
                })
                .collect(),
            ),
        _ => JoinType::Top,
    }
}

/// 计算两个类型的最大公共子类型（交）。
/// 结果同时是两个输入类型的子类型，并且在所有公共子类型中尽可能大；
/// 当前类型语言无法表示这样的类型时返回 `None`。
pub fn meet(left: &JoinType, right: &JoinType) -> Option<JoinType> {
    match (left, right) {

```

```

        (JoinType::Top, other) | (other, JoinType::Top) =>
↪ Some(other.clone()),
        (JoinType::Bool, JoinType::Bool) => Some(JoinType::Bool),
        (JoinType::Arrow(left_in, left_out), JoinType::Arrow(right_in,
↪ right_out)) => {
            Some(JoinType::Arrow(
                Box::new(join(left_in, right_in)),
                Box::new(meet(left_out, right_out)?),
            ))
        }
        (JoinType::Record(left_fields), JoinType::Record(right_fields)) => {
            let mut result = left_fields.clone();
            for (label, right_type) in right_fields {
                if let Some((), result_type) = result
                    .iter_mut()
                    .find(|(result_label, _)| result_label == label)
                {
                    *result_type = meet(result_type, right_type)?;
                } else {
                    result.push((label.clone(), right_type.clone()));
                }
            }
            Some(JoinType::Record(result))
        }
        _ => None,
    }
}

```

/// 计算布尔值与条件式扩展中的项类型。

/// 条件式的守卫必须具有 `Bool` 类型，整个条件式的类型是两个分支类型的并。

```

pub fn extended_type_of(term: &JoinTerm) -> Result<JoinType, JoinTypeError> {
    match term {
        JoinTerm::True | JoinTerm::False => Ok(JoinType::Bool),
        JoinTerm::If(guard, then_term, else_term) => {
            let guard_type = extended_type_of(guard)?;
            if guard_type != JoinType::Bool {
                return Err(JoinTypeError::ExpectedBoolean(guard_type));
            }
            Ok(join(
                &extended_type_of(then_term)?,
                &extended_type_of(else_term)?,
            ))
        }
    }
}

```

```
JoinTerm::Typed(ty) => Ok(ty.clone()),  
  }  
}
```

[返回练习 17.3.1](#)

17.3.2 解答

见原书配套源码中的 `rcdsubbot` 实现。

译者注：该实现把 `TyBot` 加入类型语法，并让 `TyBot` 成为每种类型的子类型；应用和投影还按照 §16.4 处理函数项或记录项类型为 `TyBot` 的情形。

[返回练习 17.3.2](#)

A.14 第 18 章：案例研究——命令式对象

18.6.1 解答

```
DecCounter =
  {get:Unit->Nat, inc:Unit->Unit, reset:Unit->Unit,
   dec:Unit->Unit};

decCounterClass =
  lambda r:CounterRep.
    let super = resetCounterClass r in
    {get = super.get,
     inc = super.inc,
     reset = super.reset,
     dec = lambda _:Unit. r.x := pred(!(r.x))};
```

[返回练习 18.6.1](#)

18.7.1 解答

```
BackupCounter2 =
  {get:Unit->Nat, inc:Unit->Unit,
   reset:Unit->Unit, backup:Unit->Unit,
   reset2:Unit->Unit, backup2:Unit->Unit};

BackupCounterRep2 = {x:Ref Nat, b:Ref Nat, b2:Ref Nat};

backupCounterClass2 =
  lambda r:BackupCounterRep2.
    let super = backupCounterClass r in
    {get = super.get,
     inc = super.inc,
     reset = super.reset,
     backup = super.backup,
     reset2 = lambda _:Unit. r.x := !(r.b2),
     backup2 = lambda _:Unit. r.b2 := !(r.x)};
```

[返回练习 18.7.1](#)

18.11.1 解答

先让 get 和 set 都递增访问计数：

```
instrCounterClass =
  lambda r:InstrCounterRep.
```

```

lambda self:Unit->InstrCounter.
lambda _:Unit.
  let super = setCounterClass r self unit in
  {get = lambda _:Unit.
    (r.a := succ(!(r.a)); super.get unit),
   set = lambda i:Nat.
    (r.a := succ(!(r.a)); super.set i),
   inc = super.inc,
   accesses = lambda _:Unit. !(r.a)};

```

加入 reset 的子类如下:

```

ResetInstrCounter =
  {get:Unit->Nat, set:Nat->Unit, inc:Unit->Unit,
   accesses:Unit->Nat, reset:Unit->Unit};

resetInstrCounterClass =
  lambda r:InstrCounterRep.
  lambda self:Unit->ResetInstrCounter.
  lambda _:Unit.
    let super = instrCounterClass r self unit in
    {get = super.get,
     set = super.set,
     inc = super.inc,
     accesses = super.accesses,
     reset = lambda _:Unit. r.x := 0};

```

最后加入备份功能, 并给出对象生成器:

```

BackupInstrCounter =
  {get:Unit->Nat, set:Nat->Unit, inc:Unit->Unit,
   accesses:Unit->Nat, backup:Unit->Unit, reset:Unit->Unit};

BackupInstrCounterRep =
  {x:Ref Nat, a:Ref Nat, b:Ref Nat};

backupInstrCounterClass =
  lambda r:BackupInstrCounterRep.
  lambda self:Unit->BackupInstrCounter.
  lambda _:Unit.
    let super = resetInstrCounterClass r self unit in
    {get = super.get,
     set = super.set,
     inc = super.inc,
     accesses = super.accesses,

```



```

    reset = lambda _:Unit. r.x := !(r.b),
    backup = lambda _:Unit. r.b := !(r.x)};

newBackupInstrCounter =
  lambda _:Unit.
    let r = {x=ref 1, a=ref 0, b=ref 0} in
      fix (backupInstrCounterClass r) unit;

```

返回练习 18.11.1

18.13.1 解答

可以利用引用的别名关系检验对象身份。每创建一个对象，就分配一条专属的 `Ref Nat` 身份标记；同一对象返回同一条引用，不同对象返回不同引用。本语言虽不能直接比较引用，却可以通过先写后读判断它们是否指向同一个单元。

内部表示用 `id` 实例变量保存这条标记。实例变量本身也用引用单元表示，所以 `id` 字段为 `Ref (Ref Nat)`：外层引用是实例变量，内层引用才是身份标记。

```

IdCounterRep = {x:Ref Nat, id:Ref (Ref Nat)};
IdCounter =
  {get:Unit->Nat, inc:Unit->Unit, id:Unit->(Ref Nat)};
idCounterClass =
  lambda r:IdCounterRep.
    {get = lambda _:Unit. !(r.x),
     inc = lambda _:Unit. r.x := succ(!(r.x)),
     id = lambda _:Unit. !(r.id)};

```

`id` 方法解引用外层的实例变量，返回其中保存的身份标记。`sameObject` 接受两个具有 `id` 方法的对象，并检查它们返回的引用是否指向同一个单元：

```

sameObject =
  lambda a:{id:Unit->(Ref Nat)}.
  lambda b:{id:Unit->(Ref Nat)}.
    ((b.id unit) := 1;
     (a.id unit) := 0;
     iszero (!(b.id unit)));

```

具体做法是：先通过对象 `b` 的身份标记写入 1，再通过对象 `a` 的身份标记写入 0，最后读取 `b` 的身份标记。若两者是同一对象，后一次写入会把同一单元改为 0，函数返回 `true`；若它们是不同对象，`b` 的单元仍保存 1，函数返回 `false`。

译者注：双层引用并非判断对象身份所必需。作者在这里沿用本章的实例变量编码：`id` 的值是一条 `Ref Nat` 身份标记，而实例变量本身又用引用单元保存，因而得到 `Ref (Ref Nat)`。若直接把身份标记放进内部表示，可将字段改为 `id:Ref Nat`，并令 `id` 方法返回 `r.id`；`sameObject` 无须改变。

返回练习 18.13.1

A.15 第 19 章：案例研究——Featherweight Java

19.4.1 解答

每项类声明都必须带有 `extends` 子句，而这些子句又不得形成环。因此，从任何类开始沿 `extends` 链不断向上查找，最终都必定到达 `Object`。所以无需另设规则，已有规则就能导出每个类都是 `Object` 的子类型。

[返回练习 19.4.1](#)

19.4.2 解答

一个显而易见的改进，是把类型转换的三条类型规则合并成一条：

$$\frac{\Gamma \vdash t_0 : D}{\Gamma \vdash (C)t_0 : C} \text{ T-CAST}$$

并取消“荒谬转换”这一概念。另一项改进是省略构造函数，因为它们本来就不执行任何实质计算。

[返回练习 19.4.2](#)

19.4.6 解答

1. 在 FJ 中加入接口的形式化定义并不困难。
2. 假设声明以下接口：

```
interface A {}
interface B {}
interface C extends A,B {}
interface D extends A,B {}
```

此时 A 与 B 都是 C、D 的共同上界，但二者之间没有最小者，因而 C、D 没有最小上界。

3. 不采用条件表达式通常使用的算法式规则

$$\frac{\Gamma \vdash t_1 : \text{boolean} \quad \Gamma \vdash t_2 : E_2 \quad \Gamma \vdash t_3 : E_3}{\Gamma \vdash t_1 ? t_2 : t_3 : E_2 \vee E_3}$$

Java 使用以下两条受限规则：

$$\frac{\Gamma \vdash t_1 : \text{boolean} \quad \Gamma \vdash t_2 : E_2 \quad \Gamma \vdash t_3 : E_3 \quad E_2 <: E_3}{\Gamma \vdash t_1 ? t_2 : t_3 : E_3}$$

$$\frac{\Gamma \vdash t_1 : \text{boolean} \quad \Gamma \vdash t_2 : E_2 \quad \Gamma \vdash t_3 : E_3 \quad E_3 <: E_2}{\Gamma \vdash t_1 ? t_2 : t_3 : E_2}$$

这些规则看起来合理，却无法与 FJ 所用的小步操作语义良好配合：类型保持性实际上不成立，很容易构造反例。

译者注：这里所说的是求值前后保留完全相同类型的通常形式。设 $B <: A$ ，则

$$\text{true} ? \text{new } B() : \text{new } A()$$

由第一条规则取得类型 A ，却会单步求值为 $\text{new } B()$ 。算法式类型关系只能为后者算出最小类型 B ；由于系统没有 T-Sub，无法再导出它具有类型 A ，所以通常形式的保持性失败。[定理19.5.1](#)采用了较弱的结论，允许求值后的类型 C' 是原类型 C 的子类型；在这种表述下，上述例子得到的 $B <: A$ 正好满足要求。

返回练习 19.4.6

19.4.7 解答

有些出人意料的是，处理 `super` 比处理 `self` 更困难，因为解释 `super` 时，必须记住当前正在执行的方法体最初定义在哪个类中。至少有两种办法可以保存这项信息：

1. 给项添加标注，指明应从哪里开始查找其中的 `super`；
2. 增加一个预处理步骤，重写整个类表：把对 `super` 的引用改写成对 `this` 的引用，同时使用经过改名的方法名来记录它们来自哪个类。

返回练习 19.4.7

19.5.1 解答

证明主定理前，先建立几个辅助引理。与以往一样，关键是把类型判断与替换联系起来的引理 A.14。

译者注：这份证明分为两层。引理 A.13 保证继承不会改变已有方法的类型；引理 A.14 说明把类型符合要求的实参替换进项以后，所得项仍然良类型；引理 A.15 与 A.16 则为方法调用准备环境和方法体所需的条件。主证明随后对单步求值规则作归纳。除方法调用与类型转换外，各情形基本都是对发生求值的子项使用归纳假设，再应用原来的外层类型规则。

引理 A.13 若 $\text{mtype}(m, D) = \overline{C} \rightarrow C_0$ ，则对每个 $C <: D$ 都有 $\text{mtype}(m, C) = \overline{C} \rightarrow C_0$ 。证明。对 $C <: D$ 的推导作直接归纳。若 $CT(C) = \text{class } C \text{ extends } E\{\dots\}$ ，则分两种情况。若 m 未在 C 中定义，方法类型查找会继续到父类 E ；若 m 在 C 中重新定义，类表的良构条件要求它保持从 E 继承的方法签名。因此两种情况下都有 $\text{mtype}(m, C) = \text{mtype}(m, E)$ ，再应用归纳假设即可。

引理 A.14 (项替换保持类型) 若

$$\Gamma, \bar{x} : \overline{B} \vdash t : D, \quad \Gamma \vdash \bar{s} : \overline{A}, \quad \overline{A} <: \overline{B},$$

则存在 $C <: D$ ，使

$$\Gamma \vdash [\bar{x} \mapsto \bar{s}]t : C$$

证明。对 $\Gamma, \bar{x} : \bar{B} \vdash t : D$ 的推导作归纳。基本思路与带子类型化 lambda 演算完全相同，细节略有差异；最后两种情形最值得注意。

情形 T-Var: $t = x$ ，且 $x : D \in \Gamma, \bar{x} : \bar{B}$ 。若 $x \notin \{\bar{x}\}$ ，替换不改变 x ，结论立即成立。若 $x = x_i$ 且 $D = B_i$ ，则替换结果为 s_i ；由 $\Gamma \vdash s_i : A_i$ 及 $A_i <: B_i$ ，取 $C = A_i$ 即可。

情形 T-Field: $t = t_0.f_i$ ，且

$$\Gamma, \bar{x} : \bar{B} \vdash t_0 : D_0, \quad \text{fields}(D_0) = \bar{C} \bar{f}, \quad D = C_i$$

由归纳假设，存在 $C_0 <: D_0$ ，使 $\Gamma \vdash [\bar{x} \mapsto \bar{s}]t_0 : C_0$ 。因为 $C_0 <: D_0$ ， C_0 会继承 D_0 的全部字段，并且这些字段的标签和类型保持不变； C_0 只能在其后增加字段。因此对某个字段类型序列 $\bar{D}$ ，有

$$\text{fields}(C_0) = \text{fields}(D_0), \bar{D} \bar{g}$$

所以 f_i 在 C_0 中仍有类型 C_i 。由 T-Field 得

$$\Gamma \vdash ([\bar{x} \mapsto \bar{s}]t_0).f_i : C_i$$

情形 T-Invk: $t = t_0.m(\bar{t})$ ，且

$$\begin{aligned} \Gamma, \bar{x} : \bar{B} \vdash t_0 : D_0, \quad \text{mtype}(m, D_0) = \bar{E} \rightarrow D, \\ \Gamma, \bar{x} : \bar{B} \vdash \bar{t} : \bar{D}, \quad \bar{D} <: \bar{E} \end{aligned}$$

由归纳假设，存在 $C_0, \bar{C}$ ，使

$$\begin{aligned} \Gamma \vdash [\bar{x} \mapsto \bar{s}]t_0 : C_0, \quad C_0 <: D_0, \\ \Gamma \vdash [\bar{x} \mapsto \bar{s}]\bar{t} : \bar{C}, \quad \bar{C} <: \bar{D} \end{aligned}$$

由引理 A.13, $\text{mtype}(m, C_0) = \bar{E} \rightarrow D$ ；再由子类型关系的传递性， $\bar{C} <: \bar{E}$ 。于是 T-Invk 给出

$$\Gamma \vdash ([\bar{x} \mapsto \bar{s}]t_0).m([\bar{x} \mapsto \bar{s}]\bar{t}) : D$$

情形 T-New: $t = \text{new } D(\bar{t})$ ，且

$$\text{fields}(D) = \bar{D} \bar{f}, \quad \Gamma, \bar{x} : \bar{B} \vdash \bar{t} : \bar{C}, \quad \bar{C} <: \bar{D}$$

归纳假设给出某个 $\bar{E} <: \bar{C}$ ，满足 $\Gamma \vdash [\bar{x} \mapsto \bar{s}]\bar{t} : \bar{E}$ 。由传递性， $\bar{E} <: \bar{D}$ ，再用 T-New 即得结论。

情形 T-UCast: $t = (D)t_0$ ，且 $\Gamma, \bar{x} : \bar{B} \vdash t_0 : C, C <: D$ 。归纳假设给出 $E <: C$ ，满足 $\Gamma \vdash [\bar{x} \mapsto \bar{s}]t_0 : E$ 。由传递性 $E <: D$ ，再用 T-UCast 得 $\Gamma \vdash (D)([\bar{x} \mapsto \bar{s}]t_0) : D$ 。

情形 T-DCast: $t = (D)t_0$ ，且 $\Gamma, \bar{x} : \bar{B} \vdash t_0 : C, D <: C, D \neq C$ 。归纳假设给出 $E <: C$ ，满足 $\Gamma \vdash [\bar{x} \mapsto \bar{s}]t_0 : E$ 。若 $E <: D$ ，使用 T-UCast；否则，若 $D <: E$ ，使用 T-DCast（此时由 $E \not<: D$ 可知 $D \neq E$ ）；若两个方向都不存在子类型关系，则由 T-SCast 得到结果，并产生一条荒谬转换警告。

情形 T-SCast: $t = (D)t_0$, 且 $\Gamma, \bar{x} : \bar{B} \vdash t_0 : C$ 、 $D \not\prec C$ 、 $C \not\prec D$ 。归纳假设给出 $E \prec C$, 满足 $\Gamma \vdash [\bar{x} \mapsto \bar{s}]t_0 : E$ 。首先, $D \not\prec E$: 否则由 $E \prec C$ 和传递性就会得到 $D \prec C$, 与前提矛盾。其次, $E \not\prec D$: 若同时有 $E \prec C$ 和 $E \prec D$, 由于 FJ 采用单继承, E 的所有祖先类位于同一条继承链上, 因而 C 与 D 必有一个是另一个的子类型, 同样与前提矛盾。因此可用 T-SCast 得到结果, 并产生荒谬转换警告。

引理 A.15 (弱化) 若 $\Gamma \vdash t : C$, 则 $\Gamma, x : D \vdash t : C$ 。

证明。对类型推导作直接归纳。

引理 A.16 若

$$mtype(m, C_0) = \bar{D} \rightarrow D, \quad mbody(m, C_0) = (\bar{x}, t),$$

则存在 D_0 及 $C \prec D$, 使

$$C_0 \prec D_0, \quad \bar{x} : \bar{D}, \text{this} : D_0 \vdash t : C$$

证明。对 $mbody(m, C_0)$ 的推导作归纳。基例中 m 就定义在 C_0 内; 类表良构保证 T-Method 已经导出 $\bar{x} : \bar{D}, \text{this} : C_0 \vdash t : C$, 此时取 $D_0 = C_0$ 即可。归纳步骤中, 若 m 未在 C_0 中定义, 方法体查找会转向其父类 E , 方法类型查找同样满足 $mtype(m, C_0) = mtype(m, E)$ 。因此可以对 $mbody(m, E)$ 应用归纳假设; 再结合 $C_0 \prec E$ 与传递性, 就得到所需的 $C_0 \prec D_0$ 。

现在证明保持性定理。对 $t \longrightarrow t'$ 的推导作归纳, 并按最后使用的规则分类。倒数第二个子情形 (T-DCast) 可能产生新的荒谬转换警告。

情形 E-ProjNew: 设

$$t = (\text{new } C_0(\bar{v})).f_i, \quad t' = v_i, \quad fields(C_0) = \bar{D} \bar{f}$$

从 t 的形式可知, $\Gamma \vdash t : C$ 的最后一条规则必为 T-Field, 其前提是 $\Gamma \vdash \text{new } C_0(\bar{v}) : D_0$, 且 $C = D_i$ 。后一判断的最后一条规则又必为 T-New, 其前提包括 $\Gamma \vdash \bar{v} : \bar{C}$ 与 $\bar{C} \prec \bar{D}$, 并且 $D_0 = C_0$ 。特别地, $\Gamma \vdash v_i : C_i$, 而 $C_i \prec D_i = C$, 结论成立。

情形 E-InvkNew: 设

$$\begin{aligned} t &= (\text{new } C_0(\bar{v})).m(\bar{u}), \\ t' &= [\bar{x} \mapsto \bar{u}, \text{this} \mapsto \text{new } C_0(\bar{v})]t_0, \\ mbody(m, C_0) &= (\bar{x}, t_0) \end{aligned}$$

$\Gamma \vdash t : C$ 的最后两条规则必为 T-Invk 与 T-New, 其前提包括

$$\Gamma \vdash \text{new } C_0(\bar{v}) : C_0, \quad \Gamma \vdash \bar{u} : \bar{C}, \quad \bar{C} \prec \bar{D}, \quad mtype(m, C_0) = \bar{D} \rightarrow C$$

由引理 A.16, 存在 D_0 与 $B \prec C$, 满足 $C_0 \prec D_0$ 且 $\bar{x} : \bar{D}, \text{this} : D_0 \vdash t_0 : B$ 。由引理 A.15 可把 Γ 加入环境; 再用引理 A.14 同时替换形参与 **this**, 得到某个 $E \prec B$, 使 $\Gamma \vdash t' : E$ 。由传递性 $E \prec C$, 完成此情形。

情形 E-CastNew: 设

$$t = (D)(\text{new } C_0(\bar{v})), \quad C_0 <: D, \quad t' = \text{new } C_0(\bar{v})$$

$\Gamma \vdash t : C$ 的推导只能以 T-UCast 结束; 若以 T-SCast 或 T-DCast 结束, 就会与 $C_0 <: D$ 矛盾。T-UCast 的前提给出 $\Gamma \vdash \text{new } C_0(\bar{v}) : C_0$ 与 $D = C$, 所以取 $C' = C_0$ 即可。

对于同余规则, 对发生求值的子项使用归纳假设, 再重新应用相应的外层类型规则即可。类型转换稍有不同: 子项求值后可能得到更具体的类型, 因而适用的转换规则也可能改变。这里只展示 E-Cast。设 $t = (D)t_0$ 、 $t' = (D)t'_0$ 、 $t_0 \longrightarrow t'_0$ 。根据原推导最后使用的类型规则, 分三种子情形。

子情形 T-UCast: 已知 $\Gamma \vdash t_0 : C_0$ 、 $C_0 <: D$ 、 $D = C$ 。归纳假设给出某个 $C'_0 <: C_0$, 使 $\Gamma \vdash t'_0 : C'_0$ 。由传递性 $C'_0 <: C$, 所以 T-UCast 导出 $\Gamma \vdash (C)t'_0 : C$, 且不增加荒谬转换警告。

子情形 T-DCast: 已知 $\Gamma \vdash t_0 : C_0$ 、 $D <: C_0$ 、 $D = C$ 。归纳假设给出 $C'_0 <: C_0$, 使 $\Gamma \vdash t'_0 : C'_0$ 。若 $C'_0 <: C$, 使用 T-UCast; 否则, 若 $C <: C'_0$, 使用 T-DCast (此时 $C \neq C'_0$); 这两种情况都不增加荒谬转换警告。若两个方向都不存在子类型关系, 则用 T-SCast, 并产生一条荒谬转换警告。

子情形 T-SCast: 已知 $\Gamma \vdash t_0 : C_0$ 、 $D \not<: C_0$ 、 $C_0 \not<: D$ 、 $D = C$ 。归纳假设给出 $C'_0 <: C_0$, 使 $\Gamma \vdash t'_0 : C'_0$ 。若 $C <: C'_0$, 由 $C'_0 <: C_0$ 和传递性会得到 $C <: C_0$, 与原前提矛盾, 所以 $C \not<: C'_0$ 。若 $C'_0 <: C$, 那么 C'_0 同时以 C_0 和 C 为祖先; 由单继承可知 C_0 与 C 必可比较, 也与原前提矛盾, 所以 $C'_0 \not<: C$ 。因此 T-SCast 导出 $\Gamma \vdash (C)t'_0 : C$, 并保留荒谬转换警告。

返回定理 19.5.1

A.16 第 20 章：递归类型

20.1.1 解答

```

Tree =  $\mu$ X. <leaf:Unit, node:{Nat,X,X}>;
leaf = <leaf=unit> as Tree;
► leaf : Tree

node = lambda n:Nat. lambda t1:Tree. lambda t2:Tree.
      <node={n,t1,t2}> as Tree;
► node : Nat -> Tree -> Tree -> Tree

isleaf = lambda l:Tree. case l of
      <leaf=u> ==> true | <node=p> ==> false;
► isleaf : Tree -> Bool

label = lambda l:Tree. case l of
      <leaf=u> ==> 0 | <node=p> ==> p.1;
► label : Tree -> Nat

left = lambda l:Tree. case l of
      <leaf=u> ==> leaf | <node=p> ==> p.2;
► left : Tree -> Tree

right = lambda l:Tree. case l of
      <leaf=u> ==> leaf | <node=p> ==> p.3;
► right : Tree -> Tree

append = fix (lambda f:NatList->NatList->NatList.
      lambda l1:NatList. lambda l2:NatList.
        if isnil l1 then l2
        else cons (hd l1) (f (tl l1) l2));
► append : NatList -> NatList -> NatList

preorder = fix (lambda f:Tree->NatList. lambda t:Tree.
      if isleaf t then nil
      else cons (label t)
        (append (f (left t)) (f (right t))));
► preorder : Tree -> NatList

t1 = node 1 leaf leaf;
t2 = node 2 leaf leaf;
t3 = node 3 t1 t2;
t4 = node 4 t3 t3;

```

```
l = preorder t4;

hd l;           ► 4 : Nat
hd (tl l);      ► 3 : Nat
hd (tl (tl l)); ► 1 : Nat
```

[返回练习 20.1.1](#)

20.1.2 解答

```
fib = fix (lambda f:Nat->Nat->Stream.
  lambda m:Nat. lambda n:Nat. lambda _:Unit.
    {n, f n (plus m n)}) 0 1;
► fib : Stream
```

译者注：这个流当前返回 n ，并把下一状态更新为相邻的两个数 $(n, m+n)$ ，所以从 $(0, 1)$ 开始会依次产生 $1, 1, 2, 3, 5, \dots$ 。

[返回练习 20.1.2](#)

20.1.3 解答

```
Counter =  $\mu$ C. {get:Nat, inc:Unit->C, dec:Unit->C,
  reset:Unit->C, backup:Unit->C};

c = let create =
  fix (lambda cr:{x:Nat,b:Nat}->Counter.
    lambda s:{x:Nat,b:Nat}.
      {get = s.x,
       inc = lambda _:Unit.
         cr {x=succ(s.x),b=s.b},
       dec = lambda _:Unit.
         cr {x=pred(s.x),b=s.b},
       backup = lambda _:Unit.
         cr {x=s.x,b=s.x},
       reset = lambda _:Unit.
         cr {x=s.b,b=s.b}})
  in create {x=0,b=0};
► c : Counter
```

译者注：状态记录中的 x 是当前值， b 是备份值。

[返回练习 20.1.3](#)

20.1.4 解答

```

D =  $\mu$ X. <nat:Nat, bool:Bool, fn:X $\rightarrow$ X>;

lam = lambda f:D $\rightarrow$ D. <fn=f> as D;
ap = lambda f:D. lambda a:D. case f of
    <nat=n> ==> divergeD unit
  | <bool=b> ==> divergeD unit
  | <fn=g> ==> g a;

ifd = lambda b:D. lambda t:D. lambda e:D. case b of
    <nat=n> ==> divergeD unit
  | <bool=b> ==> (if b then t else e)
  | <fn=g> ==> divergeD unit;

tru = <bool=true> as D;
fls = <bool=false> as D;

ifd fls one zro;          ▶ <nat=0> as D : D
ifd fls one fls;         ▶ <bool=false> as D : D

```

即使第二个条件表达式在原来的无类型对象语言中会被视为类型不一致，它仍能编码成 D 的值。这里构造的是表示无类型项的数据结构：对象语言（被表示的语言）是无类型演算，元语言（用来进行表示的语言）则是带递归类型的简单类型 lambda 演算。能够在带类型的元语言中表示无类型对象语言，并不比用 ML 数据结构表示本书各章的带类型或无类型 lambda 项更令人意外。

[返回练习 20.1.4](#)

20.1.5 解答

```

lam = lambda f:D $\rightarrow$ D. <fn=f> as D;
ap = lambda f:D. lambda a:D. case f of
    <nat=n> ==> divergeD unit
  | <fn=g> ==> g a
  | <rcd=r> ==> divergeD unit;

rcd = lambda fields:Nat $\rightarrow$ D. <rcd=fields> as D;
prj = lambda f:D. lambda n:Nat. case f of
    <nat=m> ==> divergeD unit
  | <fn=g> ==> divergeD unit
  | <rcd=r> ==> r n;

```

```
myrcd = rcd (lambda n:Nat.
    if iszero n then zro
    else if iszero (pred n) then one
    else divergeD unit);
```

译者注：myrcd 是对上述记录构造与字段投影编码的示例。记录在这里由一个 $\text{Nat} \rightarrow D$ 型查找函数表示：标签 0 映射到 `zro`，标签 1 映射到 `one`，访问其他标签则通过 `divergeD unit` 进入发散；外层的 `rcd` 再把这个函数封装成 D 型记录。因此，`prj myrcd 0` 得到 `zro`，`prj myrcd 1` 得到 `one`。原书的 `myrcd` 定义把第一处判断写成 `iszero 0`，这会使判断恒为真，全部字段都返回 `zro`。结合下一分支对 `pred n` 的检查和本题要按字段编号选择值的目的，这里把它修正为 `iszero n`。作者官方勘误未收录这一处。

[返回练习 20.1.5](#)

20.2.1 解答

下面把几个较有代表性的例子改写为同构递归形式。

```
Hungry =  $\mu$ A. Nat->A;
f = fix (lambda f:Nat->Hungry. lambda n:Nat.
    fold [Hungry] f);
ff = fold [Hungry] f;
ff1 = (unfold [Hungry] ff) 0;
ff2 = (unfold [Hungry] ff1) 2;
```

译者注：这个例子集中展示了从等递归写法改成同构递归写法时，需要在哪些位置显式加入 `fold` 与 `unfold`。由

$$\text{Hungry} = \mu A. \text{Nat} \rightarrow A$$

可知，`Hungry` 展开一层后的类型是

$$U = \text{Nat} \rightarrow \text{Hungry}$$

等递归系统直接把 `U` 与 `Hungry` 视为相同类型，所以正文中的原版本可以在两者之间直接切换。同构递归系统把它们视为两个不同类型，程序必须显式使用以下两个操作在两者之间转换：

$$\text{fold}[\text{Hungry}] : U \rightarrow \text{Hungry} \quad \text{unfold}[\text{Hungry}] : \text{Hungry} \rightarrow U$$

先看 `fix` 的参数。为避免与外层定义的 `f` 混淆，暂时把这个递归参数改名为 `self`，并把传给 `fix` 的生成器写成

$$G = \lambda \text{self} : U. \lambda n : \text{Nat}. \text{fold}[\text{Hungry}] \text{ self}$$

其类型可以逐层读出：`self` 具有类型 `U = Nat → Hungry`；`fold[Hungry] self` 因而具有类型 `Hungry`；所以 `λn : Nat. fold[Hungry] self` 具有类型 `Nat → Hungry = U`。生成器 `G` 的输入与输出于是都是 `U`，即 `G : U → U`，正好可以传给 `fix`，最终得到 `fix G : U`。

函数体中的第一处 `fold` 不能省略。若仍像等递归版本那样直接返回 `self`，内层抽象将具有类型

$$\text{Nat} \rightarrow U = \text{Nat} \rightarrow (\text{Nat} \rightarrow \text{Hungry})$$

而需要的输出类型是 `U = Nat → Hungry`。等递归系统会自动把其中的 `U` 当作 `Hungry`；同构递归系统不会这样做。把 `self` 写成 `fold[Hungry] self` 后，函数体才真正具有 `Hungry` 类型，整个内层抽象才具有所需的类型 `U`。

现在再逐行看后面的定义。`f = fix G` 的类型是 `U`，也就是 `Nat → Hungry`，还不是 `Hungry`；因此需要

$$\text{ff} = \text{fold}[\text{Hungry}] \text{ f}$$

把它转为 `Hungry`。同构递归系统也不会直接把 `ff` 当作函数，所以不能直接写 `ff 0`。必须先将它展开：

$$\text{unfold}[\text{Hungry}] \text{ ff} : U = \text{Nat} \rightarrow \text{Hungry}$$

现在传入 `0`，所得 `ff1` 具有类型 `Hungry`。若还要传入下一个自然数，就必须再次展开 `ff1`；这正是 `ff2 = (unfold[Hungry] ff1) 2` 的作用，所得 `ff2` 仍具有类型 `Hungry`。

从求值过程也能看出它为何可以永远继续接收参数：

$$\text{unfold}[\text{Hungry}] \text{ ff} \longrightarrow \text{f}, \quad \text{f } 0 \longrightarrow^* \text{fold}[\text{Hungry}] \text{ f} = \text{ff}$$

传入一个自然数后，又得到同一个 `Hungry` 值；继续传入有限多个自然数，每一步都会重复这一过程。三类新增操作的职责由此分明：函数体中的 `fold` 保证每次调用返回 `Hungry`，外层的 `fold` 把初始函数转成 `Hungry`，每次调用前的 `unfold` 则把它重新转成可以接收自然数的函数。

```
fixT = lambda f:T->T.
  (lambda x:(μA.A->T).
    f ((unfold [μA.A->T] x) x))
  (fold [μA.A->T]
    (lambda x:(μA.A->T).
      f ((unfold [μA.A->T] x) x)))));
D = μX. X->X;
```

```
lam = lambda f:D->D. fold [D] f;
ap = lambda f:D. lambda a:D. (unfold [D] f) a;

Counter =  $\mu$ C. {get:Nat, inc:Unit->C};
c = let create =
    fix (lambda cr:{x:Nat}->Counter. lambda s:{x:Nat}.
        fold [Counter]
            {get = s.x,
             inc = lambda _:Unit. cr {x=succ(s.x)}})
    in create {x=0};
c1 = (unfold [Counter] c).inc unit;
(unfold [Counter] c1).get;
```

[返回练习 20.2.1](#)

A.17 第 21 章：递归类型的元理论

21.1.7 解答

$$\begin{array}{llll}
 E_2(\emptyset) & = & \{a\} & E_2(\{a, b\}) & = & \{a, c\} \\
 E_2(\{a\}) & = & \{a\} & E_2(\{a, c\}) & = & \{a, b\} \\
 E_2(\{b\}) & = & \{a\} & E_2(\{b, c\}) & = & \{a, b\} \\
 E_2(\{c\}) & = & \{a, b\} & E_2(\{a, b, c\}) & = & \{a, b, c\}
 \end{array}$$

译者注：等式 $E_2(\{a\}) = \{a\}$ 表示：把集合 $\{a\}$ 作为输入，依次检查定义 E_2 的三条规则。第一条规则没有前提，能够生成 a ；第二条规则要求输入中有 c ，第三条规则要求输入中同时有 a 和 b ，因而都不能使用。最终输出中只有 a ，所以 $E_2(\{a\}) = \{a\}$ 。

E_2 -闭集合为 $\{a\}$ 和 $\{a, b, c\}$ ； E_2 -一致集合为 $\emptyset$ 、 $\{a\}$ 和 $\{a, b, c\}$ 。所以

$$\mu E_2 = \{a\}, \quad \nu E_2 = \{a, b, c\}.$$

返回练习 21.1.7

21.1.9 解答

为推出自然数上的普通归纳原理，定义

$$F(X) = \{0\} \cup \{i+1 \mid i \in X\}.$$

把谓词 P 看成所有满足 P 的自然数组成的集合。设 $P(0)$ 成立，并且 $P(i) \Rightarrow P(i+1)$ 。若 $X \subseteq P$ ，那么 $F(X)$ 中的 0 属于 P ；其中每个 $i+1$ 也因 $i \in X \subseteq P$ 和归纳步骤而属于 P ，所以 $F(X) \subseteq P$ 。特别地，取 $X = P$ ，便有 $F(P) \subseteq P$ ，即 P 是 F -闭的。归纳原理给出 $\mu F \subseteq P$ 。另一方面，任何 F -闭集合都必须包含 0，并且只要包含 i 就必须包含 $i+1$ ，所以 $\mu F = \mathbb{N}$ 。因此 $P(n)$ 对所有 $n \in \mathbb{N}$ 成立。

对字典序归纳，定义

$$F(X) = \{(m, n) \mid \forall (m', n') < (m, n). (m', n') \in X\}.$$

把谓词 P 看成所有满足 P 的自然数对组成的集合。题设是：若某个数对的所有字典序前驱都属于 P ，那么该数对也属于 P 。因此，若 $X \subseteq P$ ，任取 $(m, n) \in F(X)$ ，它的所有字典序前驱都属于 X ，从而也属于 P ；题设于是给出 $(m, n) \in P$ ，所以 $F(X) \subseteq P$ 。特别地， $F(P) \subseteq P$ ，即 P 是 F -闭的，归纳原理给出 $\mu F \subseteq P$ 。

最后证明 $\mu F = \mathbb{N} \times \mathbb{N}$ 。首先， $\mathbb{N} \times \mathbb{N}$ 是 F -闭的。再反设它的某个真子集 Y 也是 F -闭的。由于字典序是良基的，不属于 Y 的数对中存在字典序最小者，记为 (m, n) 。它的所有字典序前驱都属于 Y ，所以 $(m, n) \in F(Y)$ ；但按选取方式， $(m, n) \notin Y$ ，这与 $F(Y) \subseteq Y$ 矛盾。因此， $\mathbb{N} \times \mathbb{N}$ 的任何真子集都不可能是 F -闭的，故最小的 F -闭集合就是 $\mathbb{N} \times \mathbb{N}$ 。

译者注：这里定义生成函数 F ，是为了把具体的归纳规则接到 [推论21.1.8](#)的抽象归纳原理上。普通归纳中的 $\{0\}$ 表示起点“0 是自然数”， $\{i+1 \mid i \in X\}$ 表示后继规则“已有 i 便可生成 $i+1$ ”。从空集反复应用 F ，会依次得到 $\{0\}$ 、 $\{0, 1\}$ 、 $\{0, 1, 2\}$ 等集合，最小不动点正是 $\mathbb{N}$ 。

字典序归纳的生成函数采用同样的思路。条件 $(m, n) \in F(X)$ 表示：所有字典序小于 (m, n) 的数对都已经属于 X ，因而现在可以处理 (m, n) 。例如，若 $X = \{(0, 0)\}$ ，则 $(0, 1)$ 的唯一字典序前驱是 $(0, 0)$ ，所以 $(0, 1) \in F(X)$ ； $(0, 0)$ 没有前驱，也属于 $F(X)$ 。但 $(0, 2)$ 还有前驱 $(0, 1) \notin X$ ，因此尚不属于 $F(X)$ 。

证明末尾的“真子集”按集合包含关系比较，“最小数对”则按字典序比较。若一个真子集 Y 漏掉的字典序最小数对是 (m, n) ，那么它的所有字典序前驱都已属于 Y ，故 $(m, n) \in F(Y)$ ；但 $(m, n) \notin Y$ ，所以 $F(Y) \not\subseteq Y$ ， Y 不可能是 F -闭的。

返回练习 21.1.9

21.2.2 解答

先定义一个足够宽的候选全集。这里的树是偏函数 $T : \{1, 2\}^* \rightarrow \{\rightarrow, \times, \text{Top}\}$ ，只要求根节点 $T(\epsilon)$ 有定义，并且只要 $T(\pi, \sigma)$ 有定义，其父节点 $T(\pi)$ 就有定义。除此之外暂不限制节点标记；例如，标为 **Top** 的节点也可以有非平凡子节点。把所有这样的树组成的集合取作全集 U 。与[§21.2](#)相同，符号 **Top**、 $\times$ 和 $\rightarrow$ 也表示相应的树构造操作。定义

$$F(X) = \{\text{Top}\} \cup \{T_1 \times T_2 \mid T_1, T_2 \in X\} \cup \{T_1 \rightarrow T_2 \mid T_1, T_2 \in X\}$$

由定义可见 $\mathcal{T} \subseteq U$ ，所以可以在同一全集中比较 $\mathcal{T}$ 与 νF ，以及 $\mathcal{T}_f$ 与 μF 。下面分别验证这两个等式。

先证 $\mathcal{T} = \nu F$ 。

包含关系 $\mathcal{T} \subseteq \nu F$ 。任取 $T \in \mathcal{T}$ 。把 $X = \mathcal{T}$ 代入 F 的定义，得到

$$F(\mathcal{T}) = \{\text{Top}\} \cup \{T_1 \times T_2 \mid T_1, T_2 \in \mathcal{T}\} \cup \{T_1 \rightarrow T_2 \mid T_1, T_2 \in \mathcal{T}\}$$

若 $T = \text{Top}$ ，它属于右侧第一部分。若 $T = T_1 \times T_2$ 或 $T = T_1 \rightarrow T_2$ ，则两棵直接子树仍是合法树，因而 $T_1, T_2 \in \mathcal{T}$ ，所以 T 分别属于右侧第二或第三部分。三种情况下都有 $T \in F(\mathcal{T})$ ，故 $\mathcal{T} \subseteq F(\mathcal{T})$ 。这说明 $\mathcal{T}$ 是 F -一致的；再由余归纳原理得到 $\mathcal{T} \subseteq \nu F$ 。

包含关系 $\nu F \subseteq \mathcal{T}$ 。任取 $T \in \nu F$ 。因为 νF 是不动点，所以 $\nu F = F(\nu F)$ ，从而 $T \in F(\nu F)$ 。逐项查看 F 的定义可知，树的根节点必由三种构造之一生成：它或者是没有子节点的 **Top**，或者标为 $\times$ 或 $\rightarrow$ ，并有两棵仍属于 νF 的直接子树。对直接子树可以重复同一论证。形式上，沿任意路径 π 对其长度作归纳，便可逐层验证[定义21.2.1](#)第三、第四项条件。因此 T 是合法树类型，即 $T \in \mathcal{T}$ 。

再证 $\mathcal{T}_f = \mu F$ 。

包含关系 $\mu F \subseteq \mathcal{T}_f$ 。把 $X = \mathcal{T}_f$ 代入 F 的定义：第一部分生成单节点有限树 **Top**，第二、第三部分分别用两棵有限树构造积类型树和箭头类型树，结果仍是有限树。所以 $F(\mathcal{T}_f) \subseteq \mathcal{T}_f$ ，即 $\mathcal{T}_f$ 是 F -闭的。归纳原理于是给出 $\mu F \subseteq \mathcal{T}_f$ 。

包含关系 $\mathcal{T}_f \subseteq \mu F$ 。对有限树的大小作归纳；这里用满足 $T(\pi)$ 有定义的最长路径 π 的长度衡量树的大小。单节点树 **Top** 属于 $F(\mu F) = \mu F$ 。对于更大的有限树，其根标为 $\times$

或 $\rightarrow$ ，两棵直接子树都更小。由归纳假设，两棵子树属于 μF ；再按 F 的定义，整棵树也属于 $F(\mu F) = \mu F$ 。因此 $\mathcal{T}_f \subseteq \mu F$ 。

[返回练习 21.2.2](#)

21.3.3 解答

类型对 $(\text{Top}, \text{Top} \times \text{Top})$ 不属于 $\nu \mathcal{S}$ 。从 $\mathcal{S}$ 的定义可见，它不属于任何 $\mathcal{S}(X)$ ，所以不存在包含它的 $\mathcal{S}$ -一致集合；特别地， $\nu \mathcal{S}$ 也不包含它。

[返回练习 21.3.3](#)

21.3.4 解答

无限类型的反例。任取无限树类型 T 。下面证明 $(T, T) \in \nu \mathcal{S}$ ，但 $(T, T) \notin \mu \mathcal{S}$ 。令

$$R = \{(Q, Q) \mid Q \text{ 是 } T \text{ 的子树}\}.$$

先验证 $R \subseteq \mathcal{S}(R)$ 。任取 $(Q, Q) \in R$ ，按 Q 的根节点分类。若 $Q = \text{Top}$ ，则 (Q, Q) 由 $\mathcal{S}$ 的第一部分直接生成。若 $Q = Q_1 \times Q_2$ ，则两棵直接子树仍是 T 的子树，故 $(Q_1, Q_1), (Q_2, Q_2) \in R$ ，积类型分支生成 (Q, Q) 。若 $Q = Q_1 \rightarrow Q_2$ ，箭头类型分支同样由这两个类型对生成 (Q, Q) 。因此 R 是 $\mathcal{S}$ -一致的。余归纳给出 $R \subseteq \nu \mathcal{S}$ ；又因为 T 是自身的子树， $(T, T) \in R$ ，所以 $(T, T) \in \nu \mathcal{S}$ 。

回到 (T, T) 。无限树 T 至少有一条无限路径。沿这条路径，每遇到 $\rightarrow$ 或 $\times$ 节点，证明当前子树是自身子类型的推导都必须继续进入同一棵直接子树，因而形成一条无限推导分支，无法得到有限推导。例如，若 $T = \text{Top} \times T$ ，积类型规则会把 $T <: T$ 化为 $\text{Top} <: \text{Top}$ 和同一个目标 $T <: T$ 。所以 $(T, T) \notin \mu \mathcal{S}$ 。

译者注：令 D 为所有具有有限子类型推导的类型对组成的关系。下面说明 $D = \mu \mathcal{S}$ 。

先证 $\mu \mathcal{S} \subseteq D$ 。以 D 中的判断为前提应用一条规则，只需把该规则接在若干份有限推导之下，所得推导仍然有限。因此 $\mathcal{S}(D) \subseteq D$ ，即 D 是 $\mathcal{S}$ -闭的。根据 Knaster–Tarski 定理， $\mu \mathcal{S}$ 是所有 $\mathcal{S}$ -闭关系的交，因而包含于每一个闭关系；特别地， $\mu \mathcal{S} \subseteq D$ 。

再证 $D \subseteq \mu \mathcal{S}$ 。任取 D 中的类型对，对其有限推导的高度作归纳。若最后使用的是无前提规则，其结论直接属于 $\mathcal{S}(\mu \mathcal{S}) = \mu \mathcal{S}$ 。否则，最后一条规则的各个前提都有更矮的推导，由归纳假设均属于 $\mu \mathcal{S}$ ；应用最后一条规则，结论便属于 $\mathcal{S}(\mu \mathcal{S}) = \mu \mathcal{S}$ 。因此 $D \subseteq \mu \mathcal{S}$ ，最终得到 $D = \mu \mathcal{S}$ 。所以，只能依靠无限循环相互支持、无法形成有限推导的判断，不会进入 $\mu \mathcal{S}$ 。

有限类型的情形。不存在属于 $\nu \mathcal{S}_f$ 却不属于 $\mu \mathcal{S}_f$ 的有限类型对；事实上，两个不动点相等。按最小不动点的定义，它包含于 $\mathcal{S}_f$ 的任意不动点；特别地， $\mu \mathcal{S}_f \subseteq \nu \mathcal{S}_f$ 。故只需证明反向包含关系 $\nu \mathcal{S}_f \subseteq \mu \mathcal{S}_f$ 。任取 $(S, T) \in \nu \mathcal{S}_f$ ，对 S, T 的大小之和归纳。由于 $\nu \mathcal{S}_f = \mathcal{S}_f(\nu \mathcal{S}_f)$ ，可以检查 $\mathcal{S}_f$ 中生成该类型对的分支：若 $T = \text{Top}$ ，结论由无前提规则直接属于 $\mu \mathcal{S}_f$ ；若 S, T 都是积类型，两个分量对应的类型对规模更小，由归纳假设属于 $\mu \mathcal{S}_f$ ，再应用积类型分支即可；箭头类型的情形相同，只需注意参数位置的类型对方向相反。于是 $(S, T) \in \mu \mathcal{S}_f$ ，两个不动点相等。

[返回练习 21.3.4](#)

21.3.8 解答

定义树类型上的恒等关系

$$I = \{(T, T) \mid T \in \mathcal{T}\}.$$

若能证明 I 是 $\mathcal{S}$ -一致的, 余归纳便给出 $I \subseteq \nu\mathcal{S}$, 即自反性。任取 $(T, T) \in I$, 对 T 的根分类。若 $T = \text{Top}$, 按定义 $(T, T) \in \mathcal{S}(I)$ 。若 $T = T_1 \times T_2$, 由于 $(T_1, T_1), (T_2, T_2) \in I$, 积类型分支给出 $(T_1 \times T_2, T_1 \times T_2) \in \mathcal{S}(I)$ 。箭头情形相同。

[返回练习 21.3.8](#)

21.4.2 解答

依余归纳原理, 只需证明 $U \times U$ 是 F^{TR} -一致的。任取 $(x, y) \in U \times U$, 再任选 $z \in U$ 。有 $(x, z), (z, y) \in U \times U$, 所以按 F^{TR} 的定义, $(x, y) \in F^{\text{TR}}(U \times U)$ 。

[返回练习 21.4.2](#)

21.5.2 解答

检查可逆性, 只需查看 $\mathcal{S}_f$ 与 $\mathcal{S}$ 的定义, 并确认每个 $\mathcal{G}_{(\mathcal{S}, T)}$ 至多含一个极小成员。两个定义的每一项都明确规定了可支持元素的形式和支持集内容, 所以可直接写出 $\text{support}_{\mathcal{S}_f}$ 与 $\text{support}_{\mathcal{S}}$; 其形式与 [定义21.8.4](#)之后的 $\text{support}_{\mathcal{S}_\mu}$ 相同, 只去掉两个 μ 分支。

译者注: 把作者省略的分类写开如下。对 $\mathcal{F} \in \{\mathcal{S}_f, \mathcal{S}\}$, 其支持函数为

$$\text{support}_{\mathcal{F}}(S, T) = \begin{cases} \emptyset, & T = \text{Top}, \\ \{(S_1, T_1), (S_2, T_2)\}, & S = S_1 \times S_2, T = T_1 \times T_2, \\ \{(T_1, S_1), (S_2, T_2)\}, & S = S_1 \rightarrow S_2, T = T_1 \rightarrow T_2, \\ \uparrow, & \text{其他情形}. \end{cases}$$

两者的公式相同; $\mathcal{S}_f$ 的参数只取有限树类型, $\mathcal{S}$ 的参数则可以取无限树类型。右侧为 Top 时无需任何前提, 所以最小支持集为空集。积类型必须同时支持两个对应分量; 箭头类型必须支持逆向的参数类型对 (T_1, S_1) 和正向的结果类型对 (S_2, T_2) 。其余外层形状无法由任何分支生成, 故支持函数无定义。这几种形状互不重叠, 而且每种形状所需的前提集合都唯一, 因此 $\mathcal{S}_f$ 与 $\mathcal{S}$ 都可逆。

[返回练习 21.5.2](#)

21.5.4 解答

相应推导规则是

$$\frac{i}{h} \quad \frac{a}{b} \quad \frac{c}{d} \quad \frac{b}{e} \quad \frac{d}{f} \quad \frac{e}{g} \quad \frac{f}{g} \quad \frac{g}{g}.$$

由这些规则可直接计算题目给出的两个集合; 图中的 $\mu E = \{c, f, g\}$ 与 $\nu E = \{a, b, c, d, e, f, g\}$ 也分别满足最小、最大不动点的定义。

[返回练习 21.5.4](#)

21.5.6 解答

逆命题不成立。 $\nu F \setminus \mu F$ 中的元素也可能通向一条无限链，而非环。例如定义

$$F : \mathcal{P}(\mathbb{N}) \rightarrow \mathcal{P}(\mathbb{N}), \quad F(X) = \{0\} \cup \{n \mid n+1 \in X\}.$$

则 $\mu F = \{0\}$, $\nu F = \mathbb{N}$ 。对每个 $n > 0$, 有 $\text{support}(n) = \{n+1\}$, 所以从 n 出发得到没有环的无限链。

[返回练习 21.5.6](#)

21.5.13 解答

先证部分正确性。两项都对一次算法运行的递归结构作归纳。

1. 若算法因 $X = \emptyset$ 返回真, 则 $X \subseteq \mu F$ 显然。若因为 $\text{lfp}(\text{support}(X))$ 返回真, 归纳假设给出 $\text{support}(X) \subseteq \mu F$, 再由 [引理21.5.8](#) 得到 $X \subseteq \mu F$ 。
2. 若因 $\text{support}(X) \uparrow$ 返回假, [引理21.5.8](#) 给出 $X \not\subseteq \mu F$ 。否则递归调用返回假, 归纳假设给出 $\text{support}(X) \not\subseteq \mu F$, 再由同一引理得到结论。

下面刻画保证算法对有限输入终止的一类生成函数。对有限状态生成函数 F , 令偏函数 $\text{height}_F : U \rightarrow \mathbb{N}$ 为满足以下条件的最小偏函数:^a

$$\text{height}(x) = \begin{cases} 0, & \text{support}(x) = \emptyset, \\ 0, & \text{support}(x) \uparrow, \\ 1 + \max\{\text{height}(y) \mid y \in \text{support}(x)\}, & \text{support}(x) \neq \emptyset. \end{cases}$$

若 x 位于可达环中, 或依赖环中元素, 则高度无定义。若 height_F 是全函数, 称 F 具有有限高度。若 $y \in \text{support}(x)$ 且二者高度都有定义, 则 $\text{height}(y) < \text{height}(x)$ 。

若 F 是有限状态的, 并且具有有限高度, 则 $\text{lfp}(X)$ 对每个有限输入都终止。递归调用中的集合始终有限, 而

$$h(Y) = \max\{\text{height}(y) \mid y \in Y\}$$

每次严格减小且非负, 可作为终止度量。

[返回练习 21.5.13](#)

^a也可以把这个定义改写为一个最小不动点: 在表示偏函数的关系上定义适当的单调函数, 再取其最小不动点。

21.8.5 解答

$\mathcal{S}_d$ 与 $\mathcal{S}_\mu$ 的定义相同, 只是最后一项不要求 $T \neq \mu X.T_1$ 和 $T \neq \text{Top}$ 。它不可逆, 因为 $\mathcal{G}_{(\mu X.\text{Top}, \mu Y.\text{Top})}$ 含有两个彼此不包含的极小生成集

$$\{(\text{Top}, \mu Y.\text{Top})\}, \quad \{(\mu X.\text{Top}, \text{Top})\}.$$

可将这里的两个极小生成集，与同一类型对在 $\mathcal{S}_\mu$ 下的生成集作比较。

$\mathcal{S}_\mu$ 的最后一项是 $\mathcal{S}_d$ 最后一项的限制，其他各项相同，因此 $\nu\mathcal{S}_\mu \subseteq \nu\mathcal{S}_d$ 。反向包含关系由余归纳和下面的引理得到；该引理说明 $\nu\mathcal{S}_d$ 是 $\mathcal{S}_\mu$ -一致的。

引理 A.17. 对任意 μ -类型 S, T ，若 $(S, T) \in \nu\mathcal{S}_d$ ，则 $(S, T) \in \mathcal{S}_\mu(\nu\mathcal{S}_d)$ 。

证明概要. 令 $k = \mu\text{-height}(S)$ ，对 k 作归纳。归纳把 $(S, T) \in \nu\mathcal{S}_d$ 的任意推导变换为同一判断在 $\nu\mathcal{S}_\mu$ 中的推导。左 μ -折叠规则的限制保证，变换后的每段折叠序列都先连续应用若干次左折叠，再连续应用若干次右折叠。 $\square$

译者注：这项重排是要把 $\mathcal{S}_d$ 中允许任意穿插的折叠步骤，改造成符合 $\mathcal{S}_\mu$ 限制的推导。按推导树从前提向结论读，一旦右折叠在右侧加入最外层 μ ， $\mathcal{S}_\mu$ 左折叠分支要求“右侧不以 μ 开头”的旁条件便不再满足；因此每段折叠只能先做左折叠，再做右折叠。重排的难点只来自这项左折叠限制；右折叠没有相应的旁条件。沿推导树从结论追溯前提时，每处理一次左折叠，左侧类型开头连续出现的 μ 绑定便少一个，所以只需对这个数量 k 归纳。

[返回练习 21.8.5](#)

21.9.2 解答

$$\frac{}{T \sqsubseteq T} \quad \frac{S \sqsubseteq T_1}{S \sqsubseteq T_1 \times T_2} \quad \frac{S \sqsubseteq T_2}{S \sqsubseteq T_1 \times T_2} \\ \frac{S \sqsubseteq T_1}{S \sqsubseteq T_1 \rightarrow T_2} \quad \frac{S \sqsubseteq T_2}{S \sqsubseteq T_1 \rightarrow T_2} \quad \frac{S \sqsubseteq [X \mapsto \mu X.T]T}{S \sqsubseteq \mu X.T}.$$

有一点值得注意：TD 与本章此前考察的生成函数不同，它不可逆。例如判断 $B \sqsubseteq (A \times B) \rightarrow (B \times C)$ 有两个生成集 $\{B \sqsubseteq A \times B\}$ 与 $\{B \sqsubseteq B \times C\}$ ，二者互不包含。

[返回练习 21.9.2](#)

21.9.7 解答

除以 μ 开头的类型外，BU 的规则与 21.9.2 解答中的 TD 规则相同； μ 分支改为

$$\frac{S \preceq T}{[X \mapsto \mu X.T]S \preceq \mu X.T}.$$

[返回练习 21.9.7](#)

21.11.1 解答

例子很多。对几乎任意 T ， $\mu X.T$ 与 $[X \mapsto \mu X.T]T$ 就是一个简单例子。更有意思的例子是

$$\mu X.\text{Nat} \times (\text{Nat} \times X) \quad \text{和} \quad \mu X.\text{Nat} \times X.$$

[返回练习 21.11.1](#)

A.18 第 22 章：类型重构

22.3.9 解答

主要的算法约束生成规则如下：

$$\begin{array}{c}
 \frac{\Gamma(x) = T}{\Gamma \vdash_F x : T \mid_F \emptyset} \text{CT-VAR} \qquad \frac{\Gamma, x : T_1 \vdash_F t_2 : T_2 \mid_{F'} C \quad x \notin \text{dom}(\Gamma)}{\Gamma \vdash_F \lambda x : T_1. t_2 : T_1 \rightarrow T_2 \mid_{F'} C} \text{CT-ABS} \\
 \\
 \frac{\Gamma \vdash_F t_1 : T_1 \mid_{F'} C_1 \quad \Gamma \vdash_{F'} t_2 : T_2 \mid_{F''} C_2 \quad F'' = X, F'''}{\Gamma \vdash_F t_1 t_2 : X \mid_{F'''} C_1 \cup C_2 \cup \{T_1 = T_2 \rightarrow X\}} \text{CT-APP}
 \end{array}$$

其余规则类似。

原规则与算法规则的等价性可写成两项。

1. **可靠性**：若 $\Gamma \vdash_F t : T \mid_{F'} C$ ，并且 Γ, t 中出现的变量均不在 F 中，则 $\Gamma \vdash t : T \mid_{F \setminus F'} C$ 。
2. **完备性**：若 $\Gamma \vdash t : T \mid_X C$ ，则存在由 X 中名称排列成的序列 F ，使 $\Gamma \vdash_F t : T \mid_{\emptyset} C$ 。

两项都对推导直接归纳。可靠性的应用情形使用以下事实：若 Γ, t 中出现的类型变量不在 F 中，且 $\Gamma \vdash_F t : T \mid_{F'} C$ ，那么 T 或 C 中出现的每个类型变量，都出现在 Γ, t 中，或属于 $F \setminus F'$ 。完备性的应用情形使用以下事实：若 $\Gamma \vdash_F t : T \mid_{F'} C$ ，且 G 是一列新鲜变量名，则 $\Gamma \vdash_{F,G} t : T \mid_{F',G} C$ 。

[返回练习 22.3.9](#)

22.3.10 解答

用类型对的列表表示约束集后，约束生成算法就是把 [解答 22.3.9](#) 中的推导规则逐项写成程序：

```

let rec recon ctx nextuvar t = match t with
| TmVar(fi,i,_) ->
    let tyT = getTypeFromContext fi ctx i in
    (tyT, nextuvar, [])
| TmAbs(fi,x,tyT1,t2) ->
    let ctx' = addbinding ctx x (VarBind(tyT1)) in
    let (tyT2,nextuvar2,constr2) = recon ctx' nextuvar t2 in
    (TyArr(tyT1,tyT2), nextuvar2, constr2)
| TmApp(fi,t1,t2) ->
    let (tyT1,nextuvar1,constr1) = recon ctx nextuvar t1 in
    let (tyT2,nextuvar2,constr2) = recon ctx nextuvar1 t2 in
    let NextUVar(tyX,nextuvar') = nextuvar2() in
    let newconstr = [(tyT1,TyArr(tyT2,TyId(tyX)))] in
    (TyId(tyX), nextuvar',

```

```

    List.concat [newconstr; constr1; constr2])
| TmZero(fi) -> (TyNat, nextuvar, [])
| TmSucc(fi,t1) ->
    let (tyT1,nextuvar1,constr1) = recon ctx nextuvar t1 in
    (TyNat,nextuvar1,(tyT1,TyNat)::constr1)
| TmPred(fi,t1) ->
    let (tyT1,nextuvar1,constr1) = recon ctx nextuvar t1 in
    (TyNat,nextuvar1,(tyT1,TyNat)::constr1)
| TmIsZero(fi,t1) ->
    let (tyT1,nextuvar1,constr1) = recon ctx nextuvar t1 in
    (TyBool,nextuvar1,(tyT1,TyNat)::constr1)
| TmTrue(fi) -> (TyBool,nextuvar, [])
| TmFalse(fi) -> (TyBool,nextuvar, [])
| TmIf(fi,t1,t2,t3) ->
    let (tyT1,nextuvar1,constr1) = recon ctx nextuvar t1 in
    let (tyT2,nextuvar2,constr2) = recon ctx nextuvar1 t2 in
    let (tyT3,nextuvar3,constr3) = recon ctx nextuvar2 t3 in
    let newconstr = [(tyT1,TyBool); (tyT2,tyT3)] in
    (tyT3,nextuvar3,
    List.concat [newconstr; constr1; constr2; constr3])

```

Rust 对照复用练习 22.3.10 题干后已经展示的 Type 与 FreshVariables。这里的 Context 是变量名到类型的映射，Constraints 是待求解的类型等式列表。核心约束生成函数如下：

Rust 对照实现（译者增补）

```

pub type Constraints = Vec<(Type, Type)>;
pub type Context = BTreeMap<String, Type>;

/// 为项生成候选结果类型，以及使该项可赋型所需满足的类型等式。
pub fn generate_constraints(
    context: &Context,
    term: &Term,
    fresh: &mut FreshVariables,
) -> Result<(Type, Constraints), TypeError> {
    match term {
        Term::True | Term::False => Ok((Type::Bool, vec! [])),
        Term::Zero => Ok((Type::Nat, vec! [])),
        Term::Successor(argument) | Term::Predecessor(argument) => {
            let (ty, mut constraints) = generate_constraints(context,
↪ argument, fresh)?;
            constraints.push((ty, Type::Nat));
        }
    }
}

```

```

        Ok((Type::Nat, constraints))
    }
    Term::IsZero(argument) => {
        let (ty, mut constraints) = generate_constraints(context,
↪ argument, fresh)?;
        constraints.push((ty, Type::Nat));
        Ok((Type::Bool, constraints))
    }
    Term::If(guard, then_term, else_term) => {
        let (guard_type, mut constraints) = generate_constraints(context,
↪ guard, fresh)?;
        let (then_type, then_constraints) = generate_constraints(context,
↪ then_term, fresh)?;
        let (else_type, else_constraints) = generate_constraints(context,
↪ else_term, fresh)?;
        constraints.extend(then_constraints);
        constraints.extend(else_constraints);
        constraints.push((guard_type, Type::Bool));
        constraints.push((then_type.clone(), else_type));
        Ok((then_type, constraints))
    }
    Term::Variable(name) => context
        .get(name)
        .cloned()
        .map(|ty| (ty, vec![]))
        .ok_or_else(|| TypeError::UnknownVariable(name.clone())),
    Term::Abstraction {
        parameter,
        annotation: Some(parameter_type),
        body,
    } => {
        let parameter_type = parameter_type.clone();
        let mut body_context = context.clone();
        body_context.insert(parameter.clone(), parameter_type.clone());
        let (body_type, constraints) = generate_constraints(&body_context,
↪ body, fresh)?;
        Ok((
            Type::Arrow(Box::new(parameter_type), Box::new(body_type)),
            constraints,
        ))
    }
    Term::Application(function, argument) => {

```

```

    let (function_type, mut constraints) =
↪ generate_constraints(context, function, fresh)?;
    let (argument_type, argument_constraints) =
        generate_constraints(context, argument, fresh)?;
    constraints.extend(argument_constraints);
    let result_type = fresh.fresh();
    constraints.push((
        function_type,
        Type::Arrow(Box::new(argument_type),
↪ Box::new(result_type.clone()))),
        ));
    Ok((result_type, constraints))
}
// Unit、引用、赋值和 let 等后续扩展由完整工程继续处理。
extension => generate_extension_constraints(context, extension,
↪ fresh),
}
}

```

返回练习 22.3.10

22.3.11 解答

由图11-12的 T-Fix 可直接得到：

$$\frac{\Gamma \vdash t_1 : T_1 \mid_{X_1} C_1 \quad X \text{ 不出现在 } X_1, \Gamma, t_1 \text{ 中}}{\Gamma \vdash \text{fix } t_1 : X \mid_{X_1 \cup \{X\}} C_1 \cup \{T_1 = X \rightarrow X\}} \text{CT-Fix}$$

它先重构 t_1 的类型 T_1 ，再以新鲜类型变量 X 要求 $T_1 = X \rightarrow X$ ，最终把 X 作为 $\text{fix } t_1$ 的类型。 letrec 的规则可由此规则和 letrec 的派生形式定义得到。

返回练习 22.3.11

22.4.3 解答

$\{X = \text{Nat}, Y = X \rightarrow X\}$	$[X \mapsto \text{Nat}, Y \mapsto \text{Nat} \rightarrow \text{Nat}]$
$\{\text{Nat} \rightarrow \text{Nat} = X \rightarrow Y\}$	$[X \mapsto \text{Nat}, Y \mapsto \text{Nat}]$
$\{X \rightarrow Y = Y \rightarrow Z, Z = U \rightarrow W\}$	$[X \mapsto U \rightarrow W, Y \mapsto U \rightarrow W, Z \mapsto U \rightarrow W]$
$\{\text{Nat} = \text{Nat} \rightarrow Y\}$	不可合一
$\{Y = \text{Nat} \rightarrow Y\}$	不可合一
$\emptyset$	$[]$

返回练习 22.4.3

22.4.6 解答

本题首先需要一种替换的表示。可选的表示有很多；一种简单方案是复用 [解答 22.3.10](#) 中的 `constr` 数据类型：替换就是一个约束集，但要求其中每个类型对的左侧都是待求解的类型变量。下面依次给出五个定义：`substinty` 用一个类型替换单个类型变量；`appliesubst` 把整个替换作用于类型；`substinconstr` 把一项替换作用于约束集中的所有类型；`occursin` 执行防止循环依赖的“出现检查”；最后，`unify` 按图22-2的伪代码求解约束。像往常一样，`unify` 还接收文件位置 `fi` 和字符串 `msg`，用于在合一失败时打印错误信息。

```
let substinty tyX tyT tyS =
  let rec f tyS = match tyS with
    | TyArr(tyS1,tyS2) -> TyArr(f tyS1,f tyS2)
    | TyNat -> TyNat
    | TyBool -> TyBool
    | TyId(s) -> if s=tyX then tyT else TyId(s)
  in f tyS

let appliesubst constr tyT =
  List.fold_left
    (fun tyS (TyId(tyX),tyC2) -> substinty tyX tyC2 tyS)
    tyT (List.rev constr)

let substinconstr tyX tyT constr =
  List.map
    (fun (tyS1,tyS2) ->
      (substinty tyX tyT tyS1, substinty tyX tyT tyS2))
    constr

let occursin tyX tyT =
  let rec o tyT = match tyT with
    | TyArr(tyT1,tyT2) -> o tyT1 || o tyT2
    | TyNat -> false
    | TyBool -> false
    | TyId(s) -> s=tyX
  in o tyT

let unify fi ctx msg constr =
  let rec u constr = match constr with
    | [] -> []
    | (tyS,TyId(tyX))::rest ->
```

```

    if tyS=TyId(tyX) then u rest
    else if occursin tyX tyS then error fi (msg ^ ": circular
    ↪ constraints")
    else List.append (u (substinconstr tyX tyS rest))
                      [(TyId(tyX),tyS)]
| (TyId(tyX),tyT)::rest ->
    if tyT=TyId(tyX) then u rest
    else if occursin tyX tyT then error fi (msg ^ ": circular
    ↪ constraints")
    else List.append (u (substinconstr tyX tyT rest))
                      [(TyId(tyX),tyT)]
| (TyNat,TyNat)::rest -> u rest
| (TyBool,TyBool)::rest -> u rest
| (TyArr(tyS1,tyS2),TyArr(tyT1,tyT2))::rest ->
    u ((tyS1,tyT1)::(tyS2,tyT2)::rest)
| (tyS,tyT)::rest -> error fi "Unsolvable constraints"
in u constr

```

Rust 对照继续复用练习 22.3.10 题干后展示的 Type。下面先给出合一器依赖的替换、错误类型及四个辅助函数，再给出 unify 本身。

Rust 对照实现（译者增补）

```

pub type Substitution = BTreeMap<String, Type>;
pub type Constraints = Vec<(Type, Type)>;
pub type Context = BTreeMap<String, Type>;

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum TypeError {
    UnknownVariable(String),
    CannotUnify(Type, Type),
    OccursCheck { variable: String, within: Type },
    UnsupportedInReconbase(&'static str),
}

/// 收集类型中出现的所有类型变量。
fn free_variables(ty: &Type, variables: &mut BTreeSet<String>) {
    match ty {
        Type::Variable(name) => {
            variables.insert(name.clone());
        }
        Type::Arrow(domain, codomain) => {
            free_variables(domain, variables);

```



```

        free_variables(codomain, variables);
    }
    Type::Reference(element) => free_variables(element, variables),
    Type::Bool | Type::Nat | Type::Unit => {}
}
}

/// 递归应用替换, 并沿 `X -> Y -> Nat` 这样的替换链一直展开。
fn apply(substitution: &Substitution, ty: &Type) -> Type {
    match ty {
        Type::Variable(name) => substitution.get(name).map_or_else(
            || ty.clone(),
            |replacement| apply(substitution, replacement),
        ),
        Type::Arrow(domain, codomain) => Type::Arrow(
            Box::new(apply(substitution, domain)),
            Box::new(apply(substitution, codomain)),
        ),
        Type::Reference(element) =>
        ↪ Type::Reference(Box::new(apply(substitution, element))),
        Type::Bool | Type::Nat | Type::Unit => ty.clone(),
    }
}

/// 复合两个替换: 先应用 `older`, 再应用 `newer`。
fn compose(newer: &Substitution, older: &Substitution) -> Substitution {
    let mut composed = older
        .iter()
        .map(|(name, ty)| (name.clone(), apply(newer, ty)))
        .collect::<Substitution>();
    composed.extend(newer.clone());
    composed
}

/// 把一个已经求解的变量等式应用于其余所有约束。
fn substitute_constraint(
    variable: &str,
    replacement: &Type,
    constraints: Constraints,
) -> Constraints {
    let one = Substitution::from([(variable.to_owned(),
    ↪ replacement.clone())]);
    constraints

```

```

        .into_iter()
        .map(|(left, right)| (apply(&one, &left), apply(&one, &right)))
        .collect()
    }

    /// 计算有限类型等式集的最一般合一子。
    ///
    /// 本章重构的是有限简单类型，因此出现检查会拒绝
    /// `X = X -> Nat` 这样的循环等式。
    pub fn unify(mut constraints: Constraints) -> Result<Substitution, TypeError>
    ↪ {
        let mut solution = Substitution::new();
        // 每个待处理约束都已经应用了此前求得的替换。
        while let Some((left, right)) = constraints.pop() {
            if left == right {
                continue;
            }
            match (left, right) {
                (Type::Variable(variable), ty) | (ty, Type::Variable(variable)) =>
                ↪ {
                    let mut variables = BTreeSet::new();
                    free_variables(&ty, &mut variables);
                    if variables.contains(&variable) {
                        return Err(TypeError::OccursCheck {
                            variable,
                            within: ty,
                        });
                    }
                    constraints = substitute_constraint(&variable, &ty,
                ↪ constraints);
                    let step = Substitution::from([(variable, ty)]);
                    solution = compose(&step, &solution);
                }
                (Type::Arrow(s1, s2), Type::Arrow(t1, t2)) => {
                    constraints.push((*s1, *t1));
                    constraints.push((*s2, *t2));
                }
                (Type::Reference(s), Type::Reference(t)) => constraints.push((*s,
                ↪ *t)),
                (left, right) => return Err(TypeError::CannotUnify(left, right)),
            }
        }
    }

```

```
Ok(solution)
}
```

这个教学版合一器没有投入太多工作改善错误信息。实际编译器中，清楚解释类型错误往往是实现类型重构语言时最困难的工程问题之一；可参见 Wand (1986)。

[返回练习 22.4.6](#)

22.5.6 解答

把类型重构直接扩展到记录并不容易。主要困难是投影应生成什么约束。朴素规则会写成

$$\frac{\Gamma \vdash t : T \mid_{\mathcal{X}} C}{\Gamma \vdash t.l_i : X \mid_{\mathcal{X} \cup \{X\}} C \cup \{T = \{l_i : X\}\}}$$

这条规则要求 $t.l_i$ 的接收者恰好具有单字段记录类型 $\{l_i : X\}$ ，因而会错误地拒绝含有额外字段的记录。

Wand (1987) 提出的办法后来由 Wand (1988, 1989b)、Rémy (1989, 1990) 等人进一步发展。其做法是增加一类行变量。它的取值不是单个类型，而是整行“字段标签及相应类型”。令 $\oplus$ 合并字段互不相交的两行，则投影可生成约束

$$\frac{\Gamma \vdash t : T \mid_{\mathcal{X}} C}{\Gamma \vdash t.l_i : X \mid_{\mathcal{X} \cup \{X, \sigma, \rho\}} C \cup \{T = \{\rho\}, \rho = (l_i : X) \oplus \sigma\}} \text{CT-PROJ}$$

含义是：只要 t 的记录行 ρ 包含 $l_i : X$ ，其余字段由 σ 表示，投影结果就具有类型 X 。新的约束含有满足结合律和交换律的 $\oplus$ ，已经不再是普通类型等式；求解时需要一种等式合一算法。

[返回练习 22.5.6](#)

A.19 第 23 章：全称类型

23.4.3 解答

先定义列表连接，再用它反转列表：

```
append = λX. fix(λapp : List X → List X → List X. λl1 : List X. λl2 : List X.
  if isnil[X]l1 then l2
  else cons[X](head[X]l1)(app(tail[X]l1)l2)),
reverse = λX. fix(λrev : List X → List X. λl : List X.
  if isnil[X]l then nil[X]
  else append[X] (rev(tail[X]l)) (cons[X](head[X]l)(nil[X])))
```

类型检查器分别输出

append : ∀X. List X → List X → List X reverse : ∀X. List X → List X

[返回练习 23.4.3](#)

23.4.5 解答

and = λb : CBool. λc : CBool. λX. λt : X. λf : X. b[X](c[X]t f)f

[返回练习 23.4.5](#)

23.4.6 解答

iszro = λn : CNat. n[CBool](λb : CBool. fls)tru

[返回练习 23.4.6](#)

23.4.8 解答

```
pairNat = λn1 : CNat. λn2 : CNat. λX. λf : CNat → CNat → X. f n1 n2,
fstNat = λp : PairNat. p[CNat](λn1 : CNat. λn2 : CNat. n1),
sndNat = λp : PairNat. p[CNat](λn1 : CNat. λn2 : CNat. n2)
```

[返回练习 23.4.8](#)

23.4.9 解答

$$\begin{aligned} \text{zz} &= \text{pairNat } c0 \ c0, \\ f &= \lambda p : \text{PairNat}. \text{pairNat } (\text{sndNat } p) \ (\text{cplus } c1 \ (\text{sndNat } p)), \\ \text{prd} &= \lambda m : \text{CNat}. \text{fstNat } (m[\text{PairNat}]f\text{zz}) \end{aligned}$$

译者注：原书解答迭代时保存 $(i-1, i)$ ，最后取第一分量；这与题目提示建议保存 $(i, i-1)$ 再取第二分量的方向不同，但计算结果相同。

[返回练习 23.4.9](#)

23.4.10 解答

$$\begin{aligned} \text{vpred} &= \lambda n : \text{CNat}. \lambda X. \lambda s : X \rightarrow X. \lambda z : X. \\ &\quad (n[(X \rightarrow X) \rightarrow X] (\lambda p : (X \rightarrow X) \rightarrow X. \lambda q : X \rightarrow X. \\ &\quad \quad q(p\ s)) (\lambda x : X \rightarrow X. z)) (\lambda x : X. x) \end{aligned}$$

其类型为 $\text{CNat} \rightarrow \text{CNat}$ 。

作者感谢 Michael Levin 使他了解到这个例子。 [返回练习 23.4.10](#)

23.4.11 解答

$$\text{head} = \lambda X. \lambda \text{default} : X. \lambda l : \text{List } X. l[X] (\lambda hd : X. \lambda tl : X. hd) \text{default}$$

[返回练习 23.4.11](#)

23.4.12 解答

本题最难的部分是插入函数。下面的解答把给定列表 l 用作折叠函数，折叠过程中同时构造两份新列表：第一份与原列表相同，第二份则已经包含新元素 e 。从右向左处理 l 的每个元素 hd 时，折叠函数收到 hd 和右侧元素已经构成的列表对。它比较 e 与 hd ：若 $e \leq hd$ ，则 e 应当位于第二份结果列表的开头，所以把 e 加到尚未包含它的第一份列表

之前；若 $e > hd$ ，则 e 应当出现在第二份列表更靠后的地方，只需把 hd 加到已经完成插入的第二份列表之前。

```
insert = λX.λleq : X → X → Bool.λl : List X.λe : X.
  let res = l[Pair(List X)(List X)]
  (λhd : X.λacc : Pair(List X)(List X).
    let rest = fst[List X][List X] acc in
    let newrest = cons[X] hd rest in
    let restwithe = snd[List X][List X] acc in
    let newrestwithe =
      if leq e hd then cons[X] e (cons[X] hd rest)
      else cons[X] hd restwithe in
    pair[List X][List X] newrest newrestwithe)
  (pair[List X][List X](nil[X])(cons[X] e (nil[X])))
  in snd[List X][List X] res
```

类型检查器输出

```
insert : ∀X. (X → X → Bool) → List X → X → List X
```

接着需要一个自然数比较函数。这里使用基本自然数，因此用 `fix` 定义递归；若改用 Church 自然数，就可以完全避开 `fix`：

```
leqnat = fix(λf : Nat → Nat → Bool. λm : Nat. λn : Nat.
  if iszero m then true
  else if iszero n then false
  else f (pred m) (pred n))
```

类型检查器输出

```
leqnat : Nat → Nat → Bool
```

最后，把原列表中的元素逐一插入新列表：

```
sort = λX.λleq : X → X → Bool.
  λl : List X. l[List X]
  (λhd : X.λrest : List X. insert[X] leq rest hd)(nil[X])
```

类型检查器输出

```
sort : ∀X. (X → X → Bool) → List X → List X
```

为验证排序函数，先构造乱序列表

```
l = cons[Nat] 9 (cons[Nat] 2 (cons[Nat] 6 (cons[Nat] 4 (nil[Nat]))))
```

再令

$$l = \text{sort}[\text{Nat}] \text{ leqnat } l$$

并定义按下标读取列表的函数：

$$\begin{aligned} \text{nth} = & \lambda X. \lambda \text{default} : X. \text{fix}(\lambda f : \text{List } X \rightarrow \text{Nat} \rightarrow X. \lambda l : \text{List } X. \lambda n : \text{Nat}. \\ & \text{if iszero } n \text{ then head}[X] \text{ default } l \\ & \text{else } f (\text{tail}[X] l) (\text{pred } n)) \end{aligned}$$

类型检查器输出

$$\text{nth} : \forall X. X \rightarrow \text{List } X \rightarrow \text{Nat} \rightarrow X$$

于是

$$\begin{aligned} \text{nth}[\text{Nat}] 0 l 0 & \longrightarrow^* 2, & \text{nth}[\text{Nat}] 0 l 1 & \longrightarrow^* 4, \\ \text{nth}[\text{Nat}] 0 l 2 & \longrightarrow^* 6, & \text{nth}[\text{Nat}] 0 l 3 & \longrightarrow^* 9, \\ \text{nth}[\text{Nat}] 0 l 4 & \longrightarrow^* 0 \end{aligned}$$

Reynolds (1985) 曾以一项精彩工作证明：可以在 System F 中实现良类型的排序算法；他的算法与这里给出的版本略有不同。[返回练习 23.4.12](#)

23.5.1 解答

证明结构几乎与[定理9.3.9](#)相同。E-TAppTabs 情形还需要 [引理9.3.8](#)的类型替换结论：

$$\Gamma, X, \Delta \vdash t : T \implies \Gamma, [X \mapsto S] \Delta \vdash [X \mapsto S] t : [X \mapsto S] T$$

保留尾部上下文 Δ 才能得到足够强的归纳假设；否则 T-Abs 情形无法处理在 X 之后引入的项变量绑定。[返回定理 23.5.1](#)

23.5.2 解答

证明与[定理9.3.5](#)相同，只需在 [引理9.3.4](#)的典范形式结论中增加：

$$\text{若值 } v \text{ 具有类型 } \forall X. T_{12}, \text{ 则 } v = \lambda X. t_{12}$$

主证明的类型应用情形会使用这一结论。[返回定理 23.5.2](#)

23.6.3 解答

除最后一步需要看出怎样把前面的结果拼在一起导出矛盾外，各步都只是归纳或等式计算。Pawel Urzyczyn 建议了这个论证的结构。

1. 对 t 作直接的结构归纳，并使用赋型反演引理。
2. 证明下面这个更强的命题：若

$$t = \lambda \bar{Y}. (r[\bar{B}])$$

其中 r 不一定暴露, 且 $\text{erase}(t) = m$ 、 $\Gamma \vdash t : T$, 则存在

$$s = \lambda \bar{X}. (u[\bar{A}])$$

使 u 暴露、 $\text{erase}(s) = m$ 且 $\Gamma \vdash s : T$ 。归纳对象是 r 外层的类型抽象和类型应用的总数。

基例中没有这样的外层构造, 故 r 本身已暴露。归纳步骤中, r 的最外层若为类型应用 $r_1[R]$, 便把 R 加到 $\bar{B}$ 中, 再用归纳假设。若最外层为 $\lambda Z.r_1$, 分两种情况:

(a) 若 $\bar{B}$ 为空, 把 Z 加到 $\bar{Y}$ 中, 再用归纳假设。

(b) 若 $\bar{B}$ 非空, 写成 $\bar{B} = B_0 \bar{B}'$, 于是

$$t = \lambda \bar{Y}. ((\lambda Z.r_1)[B_0][\bar{B}'])$$

其中含有一处 E-TAppTabs 红项。归约后得到

$$t' = \lambda \bar{Y}. ([Z \mapsto B_0]r_1[\bar{B}'])$$

而 $[Z \mapsto B_0]r_1$ 的外层类型抽象与类型应用严格少于 r 。保持性定理说明 t' 与 t 类型相同, 再应用归纳假设即可。

3. 直接使用赋型反演引理。
4. 依次使用第 2、3、2 步, 并作等式计算即可。
5. 直接使用第 2 步和赋型反演引理。
6. 对 T_1 的大小归纳。基例中 T_1 是类型变量; 它必须来自 $\overline{X_1 X_2}$, 否则会有

$$[\overline{X_1 X_2 \mapsto A}](\forall \bar{Y}. T_1) = \forall \bar{Y}. W = \forall Z. (\forall \bar{Y}. W) \rightarrow ([\overline{X_1 \mapsto B}]T_2),$$

但左边没有箭头, 右边至少有一个箭头, 不可能相等。其他类型构造情形直接由归纳假设得到。

7. 反设 Ω 可赋型。由第 1、3 步, 存在暴露应用 $o = s u$, 并且

$$\begin{array}{ll} \text{erase}(s) = \lambda x. x x & \text{erase}(u) = \lambda y. y y \\ \Gamma \vdash s : U \rightarrow V & \Gamma \vdash u : U \end{array}$$

第 2 步给出

$$s' = \lambda \bar{R}. (s_0[\bar{E}]), \quad u' = \lambda \bar{V}. (u_0[\bar{F}]),$$

其中 s_0, u_0 暴露, 并且分别仍擦除为 $\lambda x. x x$ 和 $\lambda y. y y$, 且 $\Gamma \vdash s' : U \rightarrow V$ 、 $\Gamma \vdash u' : U$ 。

因为 s' 的类型是箭头类型, $\bar{R}$ 必为空; 又因为 s_0, u_0 暴露且擦除结果以普通抽象开头, 二者本身必须以普通抽象开头, 所以 $\bar{E}, \bar{F}$ 也为空。于是

$$o' = s' u' = s_0(\lambda \bar{V}. u_0) = (\lambda x : T_x. w)(\lambda \bar{V}. \lambda y : T_y. v),$$

其中 $\text{erase}(w) = xx$ 、 $\text{erase}(v) = yy$ 。由反演引理, $U = T_x$, 且

$$\Gamma, x : T_x \vdash w : W, \quad \Gamma, \bar{V}, y : T_y \vdash v : P$$

对左边的判断使用第 4 步。第 5 步排除了 $T_x = \forall \bar{X}. X_i$ 的情形, 所以

$$T_x = \forall \bar{X}_1 \bar{X}_2. T_1 \rightarrow T_2$$

并满足第 4 步的类型方程。第 6 步遂说明 $\text{leftmost-leaf}(T_1) = X_i$, 其中 $X_i \in \bar{X}_1 \bar{X}_2$ 。

对右边的判断同样使用第 4 步。若落入第 4(a) 步, 则

$$T_y = \forall \bar{Y}. Y_i \quad (Y_i \in \bar{Y})$$

若落入第 4(b) 步, 则

$$T_y = \forall \bar{Y}_1 \bar{Y}_2. S_1 \rightarrow S_2$$

并且对某些类型序列 $\bar{C}, \bar{D}$, 有

$$[\overline{Y_1 Y_2 \mapsto C}] S_1 = [\overline{Y_1 \mapsto D}] (\forall Z'. S_1 \rightarrow S_2)$$

第一种情形立即给出 T_y 的最左叶为 $Y_i \in \bar{Y}$; 第二种情形由第 6 步给出 $Y_i \in \bar{Y}_1 \bar{Y}_2$ 。

另一方面, 由 o' 的结构和反演引理,

$$\forall \bar{V}. T_y \rightarrow P = T_x = \forall \bar{X}_1 \bar{X}_2. T_1 \rightarrow T_2,$$

因而特别有 $T_y = T_1$ 。同一个类型的最左叶于是既等于 $X_i \in \bar{X}_1 \bar{X}_2$, 又等于上面两种情形之一给出的 Y_i 。变量 X_i 与 Y_i 由不同位置的量词绑定, 不可能相同, 矛盾。因此 Ω 不可赋型。

[返回练习 23.6.3](#)

23.7.1 解答

$\text{let } r = \lambda X. \text{ref}(\lambda x : X. x) \text{ in } (r[\text{Nat}] := (\lambda x : \text{Nat}. \text{succ } x); !(r[\text{Bool}]) \text{ true})$

译者注：按本章的类型传递语义，每次调用 $r[T]$ 都进入类型抽象体，因此会新建一个引用单元；两个实例不会共享同一单元。若采用朴素的完全擦除，类型抽象会消失，**ref** 的分配就被提前到 **let** 右侧，只执行一次，这才会重现§22.7中的不安全共享。按值擦除 erase_v 把类型抽象保留为虚设函数，仍会令每次实例化分别执行分配。

[返回练习 23.7.1](#)

A.20 第 24 章：存在类型

24.1.1 解答

p_6 同时提供常量 a 和函数 f ，但客户只能反复把 f 应用于 a ，最后仍得到无法观察的抽象 X 值。 p_7 允许用 f 构造 X 值，却仍没有观察它们的操作。 p_8 的两个字段都可使用，但接口没有隐藏任何与表示有关的信息，存在包也就失去作用。[返回练习 24.1.1](#)

24.2.1 解答

```
stackADT = {*List Nat, {new = nil[Nat],
    push = λn : Nat.λs : List Nat.cons[Nat] n s,
    top = λs : List Nat.head[Nat] s,
    pop = λs : List Nat.tail[Nat] s,
    isempty = λs : List Nat.isnil[Nat] s}}
as {∃ Stack,
    {new : Stack,
        push : Nat → Stack → Stack,
        top : Stack → Nat,
        pop : Stack → Stack,
        isempty : Stack → Bool}}
```

检查器给出

```
? stackADT : {∃ Stack, {new : Stack, push : Nat → Stack → Stack,
    top : Stack → Nat, pop : Stack → Stack,
    isempty : Stack → Bool}}
```

```
let {Stack, stack} = stackADT in
stack.top (stack.push 5 (stack.push 3 stack.new));
? 5 : Nat
```

[返回练习 24.2.1](#)

24.2.2 解答

```
counterADT = {*Ref Nat, {new = λ_ : Unit.ref 1,
    get = λr : Ref Nat. !r,
    inc = λr : Ref Nat. r := succ(!r)}}
as {∃ Counter,
    {new : Unit → Counter, get : Counter → Nat, inc : Counter → Unit}}
```

检查器给出

$$\begin{aligned} ? \text{ counterADT} : \{ & \exists \text{ Counter}, \\ & \{ \text{new} : \text{Unit} \rightarrow \text{Counter}, \\ & \text{get} : \text{Counter} \rightarrow \text{Nat}, \\ & \text{inc} : \text{Counter} \rightarrow \text{Unit} \} \} \end{aligned}$$

[返回练习 24.2.2](#)

24.2.3 解答

令 `zeroCounter` 为零值计数器对象，则可取

$$\begin{aligned} \text{FlipFlop} = \{ & \exists X, \{ \text{state} : X, \\ & \text{methods} : \{ \text{read} : X \rightarrow \text{Bool}, \\ & \text{toggle} : X \rightarrow X, \text{reset} : X \rightarrow X \} \} \}, \\ f = \{ & * \text{Counter}, \{ \text{state} = \text{zeroCounter}, \\ & \text{methods} = \{ \text{read} = \lambda s : \text{Counter}. \text{iseven}(\text{sendget } s), \\ & \text{toggle} = \lambda s : \text{Counter}. \text{sendinc } s, \\ & \text{reset} = \lambda s : \text{Counter}. \text{zeroCounter} \} \} \} \\ & \text{as FlipFlop} \end{aligned}$$

检查器给出 $? f : \text{FlipFlop}$ 。[返回练习 24.2.3](#)

24.2.4 解答

$$\begin{aligned} c = \{ & * \text{Ref Nat}, \{ \text{state} = \text{ref } 5, \\ & \text{methods} = \{ \text{get} = \lambda x : \text{Ref Nat}. !x, \\ & \text{inc} = \lambda x : \text{Ref Nat}. (x := \text{succ}(!x); x) \} \} \} \text{ as Counter} \end{aligned}$$

[返回练习 24.2.4](#)

24.2.5 解答

这一类型确实能在单个对象内部实现并集，却几乎无法调用。要调用某对象的 `union` : $X \rightarrow X \rightarrow X$ ，必须提供两个同一表示类型 X 的状态。打开两个不同对象会分别引入两个互不相同的抽象类型，第二个对象的状态不能传给第一个对象的方法；允许这样传递也不安全，因为二者表示可能完全不同。因此，该接口至多允许一个集合与自身求并。[返](#)

[回练习 24.2.5](#)

24.3.2 解答

至少要证明翻译保持赋型和计算。若以 $\llbracket - \rrbracket$ 表示翻译，则应有

$$\Gamma \vdash t : T \implies \llbracket \Gamma \rrbracket \vdash \llbracket t \rrbracket : \llbracket T \rrbracket$$

以及

$$t \longrightarrow^* t' \implies \llbracket t \rrbracket \longrightarrow^* \llbracket t' \rrbracket$$

这两项容易验证。反向性质则不成立：翻译可能把源语言中错误或卡住的项映射为目标语言中良类型且可继续求值的项。具体反例是

$$(\{\ast\text{Nat}, 0\} \text{ as } \{\exists X, X\})[\text{Bool}]$$

源项试图把存在包当作全称值作类型应用，因而既非良类型也无法求值；编码后的目标项却是良类型的，而且不会卡住。[返回练习 24.3.2](#)

24.3.3 解答

作者没有见过这一编码的完整书面版本。它似乎可以做到，但不会是逐个局部构造展开的简单句法糖；翻译必须一次处理整个程序。[返回练习 24.3.3](#)

A.21 第 25 章: System F 的 ML 实现

25.2.1 解答

记 $\uparrow_c^d(T)$ 为在截止位置 c 之上把自由类型变量上移 d 位:

$$\begin{aligned}\uparrow_c^d(k) &= \begin{cases} k+d & k \geq c, \\ k & k < c, \end{cases} \\ \uparrow_c^d(T_1 \rightarrow T_2) &= \uparrow_c^d(T_1) \rightarrow \uparrow_c^d(T_2), \\ \uparrow_c^d(\forall X.T_2) &= \forall X. \uparrow_{c+1}^d(T_2), \\ \uparrow_c^d(\exists X.T_2) &= \exists X. \uparrow_{c+1}^d(T_2).\end{aligned}$$

顶层移位简写为 $\uparrow^d(T) = \uparrow_0^d(T)$ 。 [返回练习 25.2.1](#)

译者注: 在书中的无名运行时表示里, 每个变量节点还保存了该位置预期的上下文长度; 移位时, 这个长度也要同步增加 d 。

25.4.1 解答

这次移位是为类型变量 X 腾出上下文位置。把 v_{12} 代入 t_2 所得的结果应当在 Γ, X 中具有良好的作用域, 而原来的 v_{12} 只相对于 Γ 定义。 [返回练习 25.4.1](#)

译者注: 具体顺序是: 先把 v_{12} 上移一位, 再替换项变量; 随后 `tytermSubstTop` 代入实际类型并删除 X 的上下文位置, 使最终结果重新相对于 Γ 解释。

A.22 第 26 章：有界量化

26.2.3 解答

Abadi、Cardelli 与 Viswanathan (1996) 的对象编码中确实需要完全 $F_{<}$ 的量词规则；Abadi 与 Cardelli (1996) 也讨论了这一例子。[返回练习 26.2.3](#)

26.3.4 解答

$$\begin{aligned} \text{spluzz} &= \lambda n : \text{SZero}.\lambda m : \text{SZero}.\lambda X.\lambda S <: X.\lambda Z <: X.\lambda s : X \rightarrow S.\lambda z : Z. \\ &\quad n[X][S][Z]s (m[X][S][Z]sz), \\ \text{spluspn} &= \lambda n : \text{SPos}.\lambda m : \text{SNat}.\lambda X.\lambda S <: X.\lambda Z <: X.\lambda s : X \rightarrow S.\lambda z : Z. \\ &\quad n[X][S][X]s (m[X][S][Z]sz) \end{aligned}$$

```
? spluspn : SPos -> SNat -> SPos
```

[返回练习 26.3.4](#)

26.3.5 解答

$$\begin{aligned} \text{SBool} &= \forall X.\forall T <: X.\forall F <: X.T \rightarrow F \rightarrow X, \\ \text{STrue} &= \forall X.\forall T <: X.\forall F <: X.T \rightarrow F \rightarrow T, \\ \text{SFalse} &= \forall X.\forall T <: X.\forall F <: X.T \rightarrow F \rightarrow F \end{aligned}$$

```
tru = lambda X. lambda T<:X. lambda F<:X.
      lambda t:T. lambda f:F. t;
? tru : STrue

fls = lambda X. lambda T<:X. lambda F<:X.
      lambda t:T. lambda f:F. f;
? fls : SFalse
```

$$\begin{aligned} \text{notft} &= \lambda b : \text{SFalse}.\lambda X.\lambda T <: X.\lambda F <: X.\lambda t : T.\lambda f : F. b[X][F][T]ft, \\ \text{nottf} &= \lambda b : \text{STrue}.\lambda X.\lambda T <: X.\lambda F <: X.\lambda t : T.\lambda f : F. b[X][F][T]ft \end{aligned}$$

```
? notft : SFalse -> STrue
? nottf : STrue -> SFalse
```

[返回练习 26.3.5](#)

26.4.3 解答

第 (1)、(2) 项需要在抽象和类型抽象情形中使用排列；第 (3)、(4) 项则是在量词情形中使用排列。[返回练习 26.4.3](#)

26.4.5 解答

第 1 项对子类型推导归纳。S-Refl、S-Top 立即成立；S-Trans、S-Arrow、S-All 直接使用归纳假设。S-TVar 有两个子情形。若最后查询的变量 $Y \neq X$ ，原上界绑定在收窄后的上下文中仍存在。若 $Y = X$ ，原结论使用 $X <: Q$ ；收窄后先由 S-TVar 得 $X <: P$ ，再由弱化把假设 $\Gamma \vdash P <: Q$ 搬入扩展上下文，最后用 S-Trans 得 $X <: Q$ 。

第 2 项对赋型推导归纳，并在 T-TApp 的子类型前提以及 T-Sub 情形使用第 1 项。[返回](#)

[引理 26.4.5](#)**26.4.11 解答**

四项均对子类型推导归纳。以箭头情形为例：S-Refl、S-Top 直接给出结论；S-TVar 和 S-All 不可能产生箭头左侧；S-Arrow 的两个子推导正是所需关系。若末步为 S-Trans，先对左子推导使用归纳假设，得到中间类型为 Top 或某个箭头。前者再用第 4 项可知目标也是 Top；后者再对右子推导使用归纳假设，并用两次传递性连接参数和结果位置的关系。全称类型情形同理。[返回练习 26.4.11](#)

26.5.1 解答

完全版本允许两个表示类型的上界不同，并令上界协变：

$$\frac{\Gamma \vdash S_1 <: T_1 \quad \Gamma, X <: S_1 \vdash S_2 <: T_2}{\Gamma \vdash \{\exists X <: S_1, S_2\} <: \{\exists X <: T_1, T_2\}} \text{S-SOME (完全)}$$

[返回练习 26.5.1](#)**26.5.2 解答**

没有子类型时恰有四种：

```
{*Nat, {a=5,b=7}} as {Some X, {a:Nat,b:Nat}};
{*Nat, {a=5,b=7}} as {Some X, {a:X,b:Nat}};
{*Nat, {a=5,b=7}} as {Some X, {a:Nat,b:X}};
{*Nat, {a=5,b=7}} as {Some X, {a:X,b:X}};
```

加入子类型和有界量化后还会出现许多选择，例如：

```
{*Nat, {a=5,b=7}} as {Some X, {a:Nat}};
{*Nat, {a=5,b=7}} as {Some X, {b:X}};
{*Nat, {a=5,b=7}} as {Some X, {a:Top,b:X}};
{*Nat, {a=5,b=7}} as {Some X, Top};
{*Nat, {a=5,b=7}} as {Some X<:Nat, {a:X,b:X}};
{*Nat, {a=5,b=7}} as {Some X<:Nat, {a:Top,b:X}};
```

[返回练习 26.5.2](#)

26.5.3 解答

一种办法是把重置计数器 ADT 嵌套在普通计数器 ADT 中:

```
counterADT =
  {*Nat,
   {new = 1,
    get = lambda i:Nat. i,
    inc = lambda i:Nat. succ(i),
    rcADT =
      {*Nat,
       {new = 1,
        get = lambda i:Nat. i,
        inc = lambda i:Nat. succ(i),
        reset = lambda i:Nat. 1}}}
   as {Some ResetCounter<:Nat,
       {new:ResetCounter,
        get:ResetCounter->Nat,
        inc:ResetCounter->ResetCounter,
        reset:ResetCounter->ResetCounter}}}}
  as {Some Counter,
      {new:Counter,
       get:Counter->Nat,
       inc:Counter->Counter,
       rcADT:
        {Some ResetCounter<:Counter,
         {new:ResetCounter,
          get:ResetCounter->Nat,
          inc:ResetCounter->ResetCounter,
          reset:ResetCounter->ResetCounter}}}}};
? counterADT
: {Some Counter,
  {new:Counter, get:Counter->Nat, inc:Counter->Counter,
   rcADT:
    {Some ResetCounter<:Counter,
     {new:ResetCounter, get:ResetCounter->Nat,
      inc:ResetCounter->ResetCounter,
      reset:ResetCounter->ResetCounter}}}}
```

依次打开这两个包后,余下程序的上下文会含有类型变量绑定 `Counter <: Top` 与 `ResetCounter <: Counter`,以及项变量绑定 `counter : {...}` 与 `resetCounter : {...}`:

```
let {Counter, counter} = counterADT in
let {ResetCounter, resetCounter} = counter.rcADT in
counter.get
```

```
(counter.inc
  (resetCounter.reset
    (resetCounter.inc resetCounter.new)));
? 2 : Nat
```

[返回练习 26.5.3](#)

26.5.4 解答

只需在[第24.3节](#)的编码中显然的位置加入上界。在类型层面得到：

$$\{\exists X <: S, T\} \stackrel{\text{def}}{=} \forall Y. (\forall X <: S. T \rightarrow Y) \rightarrow Y$$

项层面的改动直接由此得到。[返回练习 26.5.4](#)

A.23 第 27 章：案例研究——再论命令式对象

27.1 解答

一种做法如下：

```

setCounterClass =
  lambda M<:SetCounter. lambda R<:CounterRep.
  lambda self:Ref (R->M). lambda r:R.
    {get = lambda _:Unit. !(r.x),
     set = lambda i:Nat. r.x:=i,
     inc = lambda _:Unit.
       (!self r).set (succ(!self r).get unit)}};
? setCounterClass
: All M<:SetCounter. All R<:CounterRep.
  (Ref (R->M)) -> R -> SetCounter

instrCounterClass =
  lambda M<:InstrCounter. lambda R<:InstrCounterRep.
  lambda self:Ref (R->M). lambda r:R.
    let super = setCounterClass [M] [R] self in
    {get = (super r).get,
     set = lambda i:Nat.
       (r.a:=succ(!(r.a)); (super r).set i),
     inc = (super r).inc,
     accesses = lambda _:Unit. !(r.a)};
? instrCounterClass
: All M<:InstrCounter. All R<:InstrCounterRep.
  (Ref (R->M)) -> R -> InstrCounter

newInstrCounter =
  let m = ref
    (lambda r:InstrCounterRep. error as InstrCounter) in
  let m' = instrCounterClass
    [InstrCounter] [InstrCounterRep] m in
  (m := m';
   lambda _:Unit.
     let r = {x=ref 1, a=ref 0} in m' r);
? newInstrCounter : Unit -> InstrCounter

```

[返回练习 27.1](#)

A.24 第 28 章：有界量化的元理论

28.2.3 解答

T-TAbs 情形要用一次 S-Refl 提供完全 S-All 新增的上界前提。T-TApp 情形中，完全系统的子类型反演给出

$$N_1 = \forall X <: N_{11}.N_{12}, \quad \Gamma \vdash T_{11} <: N_{11}, \quad \Gamma, X <: T_{11} \vdash N_{12} <: T_{12}.$$

由原来的 $\Gamma \vdash T_2 <: T_{11}$ 和传递性得到 $\Gamma \vdash T_2 <: N_{11}$ ，这证成了 TA-TApp 的实参上界前提，从而得到算法赋型判断

$$\Gamma \vdash_A t_1[T_2] : [X \mapsto T_2]N_{12}$$

最后仍用类型替换保持子类型，得到

$$\Gamma \vdash [X \mapsto T_2]N_{12} <: [X \mapsto T_2]T_{12} = T.$$

[返回练习 28.2.3](#)

28.5.1 解答

[定理28.3.5](#)（具体来说，其中的 S-All 情形）对完全 $F_{<}$ 不成立。

译者注：原书此处写作 S-All；但[定理28.3.5](#)考察的是算法子类型关系，其证明中的相应分支是 SA-All。这里应当是在指完全系统的 SA-All 规则。

[返回练习 28.5.1](#)

28.5.6 解答

有界量词和无界量词必须分别比较，不能增加二者之间的子类型规则，否则原来的问题会重新出现。没有 Top 时，该受限系统的子类型关系可判定；详细算法见 Katiyar 与 Sankar (1992)。

但这不是对实际语言的稳定解决方案。加入带宽度子类型的记录后，空记录 $\{\}$ 会在记录类型中扮演类似最大类型的角色，修改后的 Ghelli 例子仍能令检查器发散。具体地，令

$$T = \forall X <: \{\}. \neg \{a : \forall Y <: X. \neg Y\}$$

输入

$$X_0 <: \{a : T\} \vdash_A X_0 <: \{a : \forall X_1 <: X_0. \neg X_1\}$$

会使子类型检查器发散。

Martin Hofmann 帮助作者完成了这个例子；Katiyar 与 Sankar (1992) 也作出了同样的观察。[返回练习 28.5.6](#)

28.6.3 解答

作者数得九个公共子类型：

$$\begin{aligned} \forall X <: Y' \rightarrow Z. Y \rightarrow Z' & \quad \forall X <: Y' \rightarrow Z. \text{Top} \rightarrow Z' & \quad \forall X <: Y' \rightarrow Z. X \\ \forall X <: Y' \rightarrow \text{Top}. Y \rightarrow Z' & \quad \forall X <: Y' \rightarrow \text{Top}. \text{Top} \rightarrow Z' & \quad \forall X <: Y' \rightarrow \text{Top}. X \\ \forall X <: \text{Top}. Y \rightarrow Z' & \quad \forall X <: \text{Top}. \text{Top} \rightarrow Z' & \quad \forall X <: \text{Top}. X. \end{aligned}$$

其中

$$\forall X <: Y' \rightarrow Z. Y \rightarrow Z' \quad \text{和} \quad \forall X <: Y' \rightarrow Z. X$$

都是 S, T 的下界，却没有既是二者公共超类型、又仍为 S, T 下界的类型，所以不存在交。若要找没有并的例子，可取 $S \rightarrow \text{Top}$ 与 $T \rightarrow \text{Top}$ ；上述两个没有合适公共超类型的类型也可构成另一例。[返回练习 28.6.3](#)

28.7.1 解答

同时定义 $R_{X,\Gamma}(T)$ 与 $L_{X,\Gamma}(T)$ ：前者是最小的 X -自由超类型，后者是最大的 X -自由子类型； R 总有结果， L 可能失败。为避免版面过密，[图A-2](#)省略下标 X, Γ 。两项定义的旁条件不同：每逢使用 L ，都必须检查结果是否有定义，写作 $L(T) \neq \text{fail}$ ；由于存在 Top ， R 总有定义。定义的正确性由 Ghelli 与 Pierce (1998) 证明。

$$\begin{aligned} R(\forall Y <: S. T) &= \begin{cases} \forall Y <: S. R(T) & X \notin \text{FV}(S), \\ \text{Top} & X \in \text{FV}(S) \end{cases} & L(\forall Y <: S. T) &= \begin{cases} \forall Y <: S. L(T) & L(T) \neq \text{fail}, \\ & X \notin \text{FV}(S), \\ \text{fail} & \text{其他情况} \end{cases} \\ R(\{\exists Y <: S, T\}) &= \begin{cases} \{\exists Y <: S, R(T)\} & X \notin \text{FV}(S), \\ \text{Top} & X \in \text{FV}(S) \end{cases} & L(\{\exists Y <: S, T\}) &= \begin{cases} \{\exists Y <: S, L(T)\} & L(T) \neq \text{fail}, \\ & X \notin \text{FV}(S), \\ \text{fail} & \text{其他情况} \end{cases} \\ R(S \rightarrow T) &= \begin{cases} L(S) \rightarrow R(T) & L(S) \neq \text{fail}, \\ \text{Top} & L(S) = \text{fail} \end{cases} & L(S \rightarrow T) &= \begin{cases} R(S) \rightarrow L(T) & L(T) \neq \text{fail}, \\ \text{fail} & L(T) = \text{fail} \end{cases} \\ R(X) &= T \quad \text{其中 } X <: T \in \Gamma, & L(X) &= \text{fail}, \\ R(Y) &= Y \quad \text{其中 } Y \neq X, & L(Y) &= Y \quad \text{其中 } Y \neq X, \\ R(\text{Top}) &= \text{Top} & L(\text{Top}) &= \text{Top} \end{aligned}$$

图 A-2: 给定类型的最小 X -自由超类型与最大 X -自由子类型

[返回练习 28.7.1](#)

28.7.2 解答

把完全 $F_{<}$ 的子类型问题翻译到只含有界存在量词的系统。记 $\neg S = S \rightarrow \text{Top}$, 并定义

$$\begin{aligned} \llbracket X \rrbracket &= X, \\ \llbracket \text{Top} \rrbracket &= \text{Top}, \\ \llbracket T_1 \rightarrow T_2 \rrbracket &= \llbracket T_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket, \\ \llbracket \forall X <: T_1. T_2 \rrbracket &= \neg \{ \exists X <: \llbracket T_1 \rrbracket, \neg \llbracket T_2 \rrbracket \}. \end{aligned}$$

对上下文的翻译定义为

$$\llbracket X_1 <: T_1, \dots, X_n <: T_n \rrbracket = X_1 <: \llbracket T_1 \rrbracket, \dots, X_n <: \llbracket T_n \rrbracket,$$

对子类型判断的翻译定义为

$$\llbracket \Gamma \vdash S <: T \rrbracket = \llbracket \Gamma \rrbracket \vdash \llbracket S \rrbracket <: \llbracket T \rrbracket.$$

这个翻译满足

$$\Gamma \vdash S <: T \iff \llbracket \Gamma \rrbracket \vdash \llbracket S \rrbracket <: \llbracket T \rrbracket.$$

[返回练习 28.7.2](#)

译者附录：原书未解答练习参考解答

本附录收录原书以 $\nrightarrow$ 标明、未在附录 A 中提供解答的练习。以下参考解答均为译者补充材料，不属于原书正文或作者提供的解答。

第 2 章：数学预备知识

练习 2.2.6

要证明 R' 是 R 的自反闭包，需要核对三个条件：它包含 R ，它是自反关系，并且它是满足前两个条件的最小关系。

(1) 由

$$R' = R \cup \{(s, s) \mid s \in S\}$$

立即得到 $R \subseteq R'$ 。

(2) 任取 $s \in S$ 。根据右侧集合的定义， $(s, s) \in \{(s, s) \mid s \in S\}$ ，因而 $(s, s) \in R'$ 。所以 R' 是自反关系。

(3) 设 R'' 是 S 上任意一个包含 R 的自反关系。由 $R \subseteq R''$ ，可知 R'' 包含 R 中的所有有序对；又因为 R'' 自反，所以对每个 $s \in S$ 都有 $(s, s) \in R''$ 。因此

$$R \cup \{(s, s) \mid s \in S\} \subseteq R'',$$

即 $R' \subseteq R''$ 。

由此， R' 是包含 R 的最小自反关系，所以它正是 R 的自反闭包。

[返回练习 2.2.6](#)

练习 2.2.7

先注意 $R_i \subseteq R_{i+1}$ ，所以序列 $R_0, R_1, \dots$ 单调递增。

(1) 因为 $R = R_0 \subseteq \bigcup_i R_i = R^+$ ，所以 R^+ 包含 R 。

(2) 证明 R^+ 具有传递性。若 $(s, t) \in R^+$ 且 $(t, u) \in R^+$ ，则存在 i, j ，使 $(s, t) \in R_i$ 且 $(t, u) \in R_j$ 。令 $k = \max(i, j)$ 。由单调性，两对都属于 R_k ，所以按 R_{k+1} 的定义， $(s, u) \in R_{k+1} \subseteq R^+$ 。

(3) 证明最小性。这里的“最小”按关系之间的包含关系理解。设 Q 是任意一个包含 R 的传递关系；我们需要证明

$$R^+ \subseteq Q$$

因为 $R^+ = \bigcup_i R_i$ ，只要对 i 作归纳，证明每一层都满足 $R_i \subseteq Q$ 即可。

基例。由 $R_0 = R$ 以及假设 $R \subseteq Q$ ，立即得到 $R_0 \subseteq Q$ 。

归纳步骤。假设 $R_i \subseteq Q$ ，下面证明 $R_{i+1} \subseteq Q$ 。任取 $(s, u) \in R_{i+1}$ 。根据 R_{i+1} 的定义，分为两种情形：

- a. 若 $(s, u) \in R_i$ ，则由归纳假设直接得到 $(s, u) \in Q$ 。
- b. 否则，存在某个 t ，使 $(s, t) \in R_i$ 且 $(t, u) \in R_i$ 。由归纳假设， $(s, t), (t, u) \in Q$ ；再由 Q 的传递性，得到 $(s, u) \in Q$ 。

两种情形下都有 $(s, u) \in Q$ ，所以 $R_{i+1} \subseteq Q$ 。归纳可知，对所有 i 都有 $R_i \subseteq Q$ ，因而

$$R^+ = \bigcup_i R_i \subseteq Q$$

由于 Q 是任意一个包含 R 的传递关系， R^+ 包含于所有这样的关系，所以它是其中最小的一个。

所以 R^+ 是包含 R 的最小传递关系，即 R 的传递闭包。

[返回练习 2.2.7](#)

练习 2.2.8

定义 S 上的关系

$$Q = \{(s, t) \in S \times S \mid P(s) \Rightarrow P(t)\}$$

关系 Q 是自反的，因为 $P(s) \Rightarrow P(s)$ ；它也是传递的，因为

$$P(r) \Rightarrow P(s), \quad P(s) \Rightarrow P(t) \implies P(r) \Rightarrow P(t)$$

“ P 被 R 保持”恰好说明 $R \subseteq Q$ 。而 R^* 是包含 R 的最小自反传递关系，所以 $R^* \subseteq Q$ 。因此，若 $(s, t) \in R^*$ 且 $P(s)$ 成立，就有 $P(t)$ 成立；也就是说， P 被 R^* 保持。

[返回练习 2.2.8](#)

第 3 章：无类型算术表达式

练习 3.5.18

要强制采用“先 then 分支，再 else 分支，最后守卫”的次序，可以把条件式规则改成：

$$\frac{t_2 \longrightarrow t'_2}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \longrightarrow \text{if } t_1 \text{ then } t'_2 \text{ else } t_3} \text{ E-IFTHEN}$$

$$\frac{t_3 \longrightarrow t'_3}{\text{if } t_1 \text{ then } v_2 \text{ else } t_3 \longrightarrow \text{if } t_1 \text{ then } v_2 \text{ else } t'_3} \text{ E-IFELSE}$$

$$\frac{t_1 \longrightarrow t'_1}{\text{if } t_1 \text{ then } v_2 \text{ else } v_3 \longrightarrow \text{if } t'_1 \text{ then } v_2 \text{ else } v_3} \text{ E-IF}$$

最后保留两个计算规则，但要求两个分支都已经是值：

$$\text{if true then } v_2 \text{ else } v_3 \longrightarrow v_2 \quad (\text{E-IFTRUE}),$$

$$\text{if false then } v_2 \text{ else } v_3 \longrightarrow v_3 \quad (\text{E-IFFALSE})$$

这些值限制决定了求值次序：只要 t_2 还能归约，就只能使用 E-IFTHEN；等 t_2 成为值后，才轮到 t_3 ；两个分支都成为值后，才允许归约守卫。尤其不能把原来的 E-IFTRUE、E-IFFALSE 原样保留，否则守卫一旦已经是布尔值，就会跳过尚未求值的分支。

[返回练习 3.5.18](#)

第 4 章：算术表达式的 ML 实现

练习 4.2.2

大步求值器直接递归求出各前提的最终值，而不是先生成一个中间项、再从头调用 eval：

```
let rec eval t =
  match t with
  | TmTrue _ -> t
  | TmFalse _ -> t
  | TmZero _ -> t
  | TmIf(_, t1, t2, t3) ->
    (match eval t1 with
     | TmTrue _ -> eval t2
     | TmFalse _ -> eval t3
     | _ -> raise NoRuleApplies)
  | TmSucc(fi, t1) ->
    let v1 = eval t1 in
    if isnumericval v1 then TmSucc(fi, v1)
```

```

    else raise NoRuleApplies
  | TmPred(_, t1) ->
    (match eval t1 with
     | TmZero _ -> TmZero(dummyinfo)
     | TmSucc(_, nv1) when isnumericval nv1 -> nv1
     | _ -> raise NoRuleApplies)
  | TmIsZero(_, t1) ->
    (match eval t1 with
     | TmZero _ -> TmTrue(dummyinfo)
     | TmSucc(_, nv1) when isnumericval nv1 ->
       TmFalse(dummyinfo)
     | _ -> raise NoRuleApplies)

```

Rust 对照实现 (译者增补)

```

pub fn eval_big(term: &Term) -> StepResult {
  match term {
    Term::True(_) | Term::False(_) | Term::Zero(_) => Ok(term.clone()),
    Term::If(_, guard, then_term, else_term) => match eval_big(guard)? {
      Term::True(_) => eval_big(then_term),
      Term::False(_) => eval_big(else_term),
      _ => Err(NoRuleApplies),
    },
    Term::Succ(info, inner) => {
      let value = eval_big(inner)?;
      is_numeric_value(&value)
        .then(|| Term::Succ(*info, Box::new(value)))
        .ok_or(NoRuleApplies)
    },
    Term::Pred(_, inner) => match eval_big(inner)? {
      Term::Zero(_) => Ok(Term::Zero(DUMMY_INFO)),
      Term::Succ(_, numeric) if is_numeric_value(&numeric) =>
        ↪ Ok(*numeric),
      _ => Err(NoRuleApplies),
    },
    Term::IsZero(_, inner) => match eval_big(inner)? {
      Term::Zero(_) => Ok(Term::True(DUMMY_INFO)),
      Term::Succ(_, numeric) if is_numeric_value(&numeric) =>
        ↪ Ok(Term::False(DUMMY_INFO)),
      _ => Err(NoRuleApplies),
    },
  }
}

```

条件式只求值被守卫选中的分支，这与练习 3.5.17 的大步规则一致。若某个前提的求值结果是一个不符合规则要求的范式，就抛出 `NoRuleApplies`，表示原项卡住。

[返回练习 4.2.2](#)

第 5 章：无类型 lambda 演算

练习 5.3.7

保留练习 3.5.16 为布尔值与自然数给出的全部 `wrong` 规则，并把 lambda 抽象加入两类“错误操作数”：

$$badnat ::= \text{wrong} \mid \text{true} \mid \text{false} \mid \lambda x. t,$$

$$badbool ::= \text{wrong} \mid nv \mid \lambda x. t$$

这是必要的，因为在 λ_{NB} 中，抽象也是值，但它既不能作为数值运算的操作数，也不能作为条件式守卫。

对应用再定义非函数值

$$badfun ::= \text{true} \mid \text{false} \mid nv$$

并加入三条规则：

$$\text{wrong } t_2 \longrightarrow \text{wrong} \quad (\text{E-APP-WRONG1}),$$

$$v_1 \text{ wrong} \longrightarrow \text{wrong} \quad (\text{E-APP-WRONG2}),$$

$$badfun v_2 \longrightarrow \text{wrong} \quad (\text{E-APP-WRONG})$$

第一条处理函数位置求值失败的情形；第二条处理按值调用在参数位置求值失败的情形；第三条处理两边都已成为值、但左边并非抽象的情形。它们与原来的 `E-APP1`、`E-APP2`、`E-APPABS` 配合，保持“先函数、后参数、最后应用”的次序。这样，原语义中所有卡住的 λ_{NB} 项都会在扩充语义中求值到 `wrong`，反之亦然；证明可按练习 3.5.16 的结构逐项扩展。

[返回练习 5.3.7](#)

第 6 章：项的无名表示

练习 6.1.4

对每个 $n, i \in \mathbb{N}$ ，递归定义

$$S_{n,0} = \emptyset,$$

$$S_{n,i+1} = \{0, \dots, n-1\}$$

$$\cup \{\lambda. t \mid t \in S_{n+1,i}\}$$

$$\cup \{t_1 t_2 \mid t_1, t_2 \in S_{n,i}\},$$

并令

$$S_n = \bigcup_i S_{n,i}$$

先同时对 n 证明 $S_{n,i} \subseteq S_{n,i+1}$ 。基例为空集；归纳步骤逐一检查变量、抽象和应用三种生成方式即可。

现在证明 $\{S_n\}$ 满足定义 6.1.2 的三条闭包条件。若 $0 \leq k < n$ ，则 $k \in S_{n,1}$ 。若 $t \in S_{n+1}$ ，则 t 属于某个 $S_{n+1,i}$ ，故 $\lambda.t \in S_{n,i+1}$ 。若 $t_1, t_2 \in S_n$ ，取一个同时不小于二者所在阶段的 i ，利用累积性可得 $t_1, t_2 \in S_{n,i}$ ，于是 $t_1 t_2 \in S_{n,i+1}$ 。

还需证明最小性。设集合族 $\{U_n\}$ 满足定义 6.1.2 的三条规则。对 i 作归纳，同时证明所有 n 都有 $S_{n,i} \subseteq U_n$ 。基例显然；归纳步骤中，变量由第一条规则进入 U_n ，抽象由对 $S_{n+1,i}$ 的归纳假设和第二条规则进入 U_n ，应用则由对两个子项的归纳假设和第三条规则进入 U_n 。因此 $S_n \subseteq U_n$ 。

所以 $\{S_n\}$ 是满足三条规则的最小集合族，与定义 6.1.2 中的 $\{T_n\}$ 相同。

[返回练习 6.1.4](#)

练习 6.2.3

证明一个稍强的、显式记录当前绑定深度的命题。设 $t \in T_{n+c}$ ，并把索引 $k \geq c$ 的出现视为相对于外层上下文的自由变量，其外层索引为 $k - c$ 。若 $d < 0$ 时这些外层索引都不小于 $|d|$ ，则

$$\uparrow_c^d(t) \in T_{\max(n+d,0)+c}$$

对 t 的结构作归纳。

- 若 $t = k$ ，当 $k < c$ 时移位不改变 k ，而 $k < c \leq \max(n+d,0) + c$ 。当 $k \geq c$ 时，结果为 $k + d$ 。若 $d \geq 0$ ，显然 $c \leq k + d < n + d + c$ ；若 $d < 0$ ，前提 $k - c \geq |d|$ 保证 $k + d \geq c$ ，而 $k < n + c$ 保证 $k + d < n + d + c$ 。因此结果属于目标项集。
- 若 $t = \lambda.t_1$ ，则 $t_1 \in T_{n+c+1}$ 。对 t_1 使用归纳假设，截点改为 $c + 1$ ，得到

$$\uparrow_{c+1}^d(t_1) \in T_{\max(n+d,0)+c+1}$$

再用定义 6.1.2 的抽象规则，得到 $\lambda. \uparrow_{c+1}^d(t_1) \in T_{\max(n+d,0)+c}$ 。

- 若 $t = t_1 t_2$ ，分别对两个子项使用归纳假设，再用应用规则即可。

在上述结论中取 $c = 0$ ，就得到题目要求的

$$\uparrow^d(t) \in T_{\max(n+d,0)}$$

[返回练习 6.2.3](#)

练习 6.2.6

对 t 的结构作归纳；归纳命题同时对所有 n 成立。

- 若 $t = k$ ，则 $k < n$ 。若 $k = j$ ，替换结果为 $s \in T_n$ ；否则结果仍为 $k \in T_n$ 。
- 若 $t = \lambda. t_1$ ，则 $t_1 \in T_{n+1}$ ，而

$$[j \mapsto s]t = \lambda. [j + 1 \mapsto \uparrow^1(s)]t_1$$

由移位性质， $\uparrow^1(s) \in T_{n+1}$ ，且 $j + 1 \leq n + 1$ 。对 t_1 使用归纳假设，得到

$$[j + 1 \mapsto \uparrow^1(s)]t_1 \in T_{n+1}$$

再用抽象规则，整个结果属于 T_n 。

- 若 $t = t_1 t_2$ ，归纳假设分别给出 $[j \mapsto s]t_1 \in T_n$ 与 $[j \mapsto s]t_2 \in T_n$ ，再用应用规则即可。

[返回练习 6.2.6](#)

练习 6.2.7

可以从“进入一个绑定子时，外部自由变量的编号都要让出一个位置”这一不变量重新推导两个定义。

先定义在截点 c 之上移位：

$$\begin{aligned} \uparrow_c^d(k) &= \begin{cases} k, & k < c, \\ k + d, & k \geq c, \end{cases} \\ \uparrow_c^d(\lambda. t_1) &= \lambda. \uparrow_{c+1}^d(t_1), \\ \uparrow_c^d(t_1 t_2) &= \uparrow_c^d(t_1) \uparrow_c^d(t_2) \end{aligned}$$

变量分支只移动当前绑定深度之外的变量；进入抽象时，截点加一。

再定义替换。变量 j 命中时换成 s ，其他变量保持不变；进入抽象时，目标变量在函数体中的编号变成 $j + 1$ ，而 s 的自由变量也必须整体上移一位：

$$\begin{aligned} [j \mapsto s]k &= \begin{cases} s, & k = j, \\ k, & k \neq j, \end{cases} \\ [j \mapsto s](\lambda. t_1) &= \lambda. [j + 1 \mapsto \uparrow^1(s)]t_1, \\ [j \mapsto s](t_1 t_2) &= ([j \mapsto s]t_1)([j \mapsto s]t_2) \end{aligned}$$

自检时可用两个关键例子： $[0 \mapsto s](\lambda. 1)$ 应把外层变量替换为上移后的 s ，而 $[0 \mapsto s](\lambda. 0)$ 必须保持抽象内部的受约束变量 0 不变。这两点分别检验了“进入抽象时移动替换项”和“不能误替换受约束变量”。

[返回练习 6.2.7](#)

第 7 章：无类型 lambda 演算的 ML 实现

练习 7.3.1

按值调用的大步求值器先把函数位置求值为抽象，再把参数求值为值，执行一次顶层替换，最后递归求值替换结果：

```
let rec eval ctx t =
  match t with
  | TmAbs _ -> t
  | TmApp(_, t1, t2) ->
    let v1 = eval ctx t1 in
    let v2 = eval ctx t2 in
    (match v1 with
     | TmAbs(_, _, t12) ->
       eval ctx (termSubstTop v2 t12)
     | _ -> raise NoRuleApplies)
  | _ -> raise NoRuleApplies
```

Rust 对照实现 (译者增补)

```
pub fn eval_big(context: &Context, term: &Term) -> Result<Term, EvalError> {
  match term {
    Term::Abs(_, _, _) => Ok(term.clone()),
    Term::App(_, function, argument) => {
      let function_value = eval_big(context, function)?;
      let argument_value = eval_big(context, argument)?;
      let Term::Abs(_, _, body) = function_value else {
        return Err(EvalError::NoRuleApplies);
      };
      let reduced = term_substitute_top(&argument_value, &body)?;
      eval_big(context, &reduced)
    }
    Term::Var(_, _, _) => Err(EvalError::NoRuleApplies),
  }
}
```

对闭项而言，这段程序与练习 5.3.8 的规则逐项对应：TmAbs 对应 B-VALUE，TmApp 分支对应 B-APP 的三个前提。若对含自由变量的开项求值，变量没有大步规则，因而抛出 NoRuleApplies。

[返回练习 7.3.1](#)

第 8 章：带类型的算术表达式

练习 8.2.3

证明目标是：若 $t : T$ ，则 t 的每个子项都具有某个类型。对 $t : T$ 的类型推导作归纳。归纳命题取为：对推导中的每个结论 $s : S$ ，项 s 以及它的所有子项都是良类型的。

基础情形。若最后一条规则是 T-TRUE、T-FALSE 或 T-ZERO，则 t 没有真子项；而 t 本身已由该规则获得类型，所以结论成立。

归纳情形。其余每条类型规则都先给直接子项建立类型，再为整个项建立类型。例如：

- 若最后使用 T-IF，三个前提分别说明 t_1, t_2, t_3 良类型。由归纳假设，它们各自的所有子项也良类型；条件项的子项恰好是这三个直接子项及其子项，所以结论成立。
- 若最后使用 T-SUCC、T-PRED 或 T-ISZERO，唯一的直接子项 t_1 由规则前提获得类型；由归纳假设， t_1 的所有子项也良类型。

所有类型规则均已覆盖，故每个良类型项的每个子项都是良类型的。

[返回练习 8.2.3](#)

练习 8.3.4

要证：若 $t : T$ 且 $t \rightarrow t'$ ，则 $t' : T$ 。这次对 $t \rightarrow t'$ 的求值推导作归纳。归纳假设是：对当前求值推导的每个直接子推导 $s \rightarrow s'$ ，只要 $s : S$ ，就有 $s' : S$ 。按最后使用的求值规则分类。

- E-IFTRUE：此时 $t = \text{if true then } t_2 \text{ else } t_3$ 且 $t' = t_2$ 。由 $t : T$ 的反演引理， $t_2 : T$ ，故结论成立。E-IFFALSE 相同。
- E-IF：此时 $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$ 、 $t_1 \rightarrow t'_1$ ，且 $t' = \text{if } t'_1 \text{ then } t_2 \text{ else } t_3$ 。由反演引理， $t_1 : \text{Bool}$ 且 $t_2 : T$ 、 $t_3 : T$ 。对子推导 $t_1 \rightarrow t'_1$ 使用归纳假设，得到 $t'_1 : \text{Bool}$ ；再用 T-IF 即得 $t' : T$ 。
- E-SUCC、E-PRED、E-ISZERO：都先由反演引理得到被求值子项的类型，再对子推导使用归纳假设，最后用相应类型规则重建 $t' : T$ 。
- E-PREDZERO： $t = \text{pred } 0$ 、 $t' = 0$ 。由 $t : T$ 的反演引理得到 $T = \text{Nat}$ ，而 T-ZERO 给出 $t' : \text{Nat}$ 。
- E-PREDSUCC： $t = \text{pred (succ } nv_1)$ 、 $t' = nv_1$ 。由反演引理得到 $T = \text{Nat}$ 且 $nv_1 : \text{Nat}$ ，所以 $t' : T$ 。
- E-ISZEROZERO：结果为 true；E-ISZEROSUCC：结果为 false。两种情形都由反演引理得到 $T = \text{Bool}$ ，再由常量的类型规则得到结论。

这些正是算术语言的全部求值规则，故保持性成立。

[返回练习 8.3.4](#)

第 9 章：简单类型 lambda 演算

练习 9.2.2

令 $\Gamma = f : \text{Bool} \rightarrow \text{Bool}$ 。第一棵推导树为

$$\begin{array}{c}
 \frac{f : \text{Bool} \rightarrow \text{Bool} \in \Gamma}{\Gamma \vdash f : \text{Bool} \rightarrow \text{Bool}} \text{T-VAR} \\
 \frac{\Gamma \vdash f : \text{Bool} \rightarrow \text{Bool}}{\Gamma \vdash \text{false} : \text{Bool}} \text{T-FALSE} \\
 \frac{}{\Gamma \vdash \text{true} : \text{Bool}} \text{T-TRUE} \quad \frac{}{\Gamma \vdash \text{false} : \text{Bool}} \text{T-FALSE} \\
 \frac{\Gamma \vdash \text{true} : \text{Bool} \quad \Gamma \vdash \text{false} : \text{Bool}}{\Gamma \vdash \text{if false then true else false} : \text{Bool}} \text{T-IF} \\
 \frac{\Gamma \vdash \text{if false then true else false} : \text{Bool}}{\Gamma \vdash f (\text{if false then true else false}) : \text{Bool}} \text{T-APP}
 \end{array}$$

第二棵树的最外层是 T-ABS。令 $\Gamma' = \Gamma, x : \text{Bool}$ ，则其函数体的推导为

$$\begin{array}{c}
 \frac{f : \text{Bool} \rightarrow \text{Bool} \in \Gamma'}{\Gamma' \vdash f : \text{Bool} \rightarrow \text{Bool}} \text{T-VAR} \\
 \frac{x : \text{Bool} \in \Gamma'}{\Gamma' \vdash x : \text{Bool}} \text{T-VAR} \\
 \frac{}{\Gamma' \vdash \text{false} : \text{Bool}} \text{T-FALSE} \quad \frac{x : \text{Bool} \in \Gamma'}{\Gamma' \vdash x : \text{Bool}} \text{T-VAR} \\
 \frac{\Gamma' \vdash \text{false} : \text{Bool} \quad \Gamma' \vdash x : \text{Bool}}{\Gamma' \vdash \text{if } x \text{ then false else } x : \text{Bool}} \text{T-IF} \\
 \frac{\Gamma' \vdash \text{if } x \text{ then false else } x : \text{Bool}}{\Gamma' \vdash f (\text{if } x \text{ then false else } x) : \text{Bool}} \text{T-APP}
 \end{array}$$

因此

$$\frac{\Gamma' \vdash f (\text{if } x \text{ then false else } x) : \text{Bool}}{\Gamma \vdash \lambda x : \text{Bool}. f (\text{if } x \text{ then false else } x) : \text{Bool} \rightarrow \text{Bool}} \text{T-ABS}$$

[返回练习 9.2.2](#)

第 11 章：简单扩展

第 11.2 节未编号练习

答案是可数无限多个。事实上，即使把类型固定为

$$(\text{Unit} \rightarrow \text{Unit}) \rightarrow \text{Unit} \rightarrow \text{Unit}$$

也能构造无限多个不同的闭范式。对每个 $n \in \mathbb{N}$ ，定义

$$c_n = \lambda f : \text{Unit} \rightarrow \text{Unit}. \lambda x : \text{Unit}. f^n x$$

其中 $f^0 x = x$ ，而 $f^{n+1} x = f(f^n x)$ 。省略上面已经给出的类型标注后，开头几项是

$$\begin{aligned} c_0 &= \lambda f. \lambda x. x, \\ c_1 &= \lambda f. \lambda x. f x, \\ c_2 &= \lambda f. \lambda x. f(f x), \\ c_3 &= \lambda f. \lambda x. f(f(f x)) \end{aligned}$$

这些项只有变量 f 和 x ，且二者都被 lambda 抽象绑定，所以它们都是闭项。每个应用的函数位置都是变量 f ，而不是 lambda 抽象，因此没有 β 可归约式，所有 c_n 都是范式。不同的 n 对应不同层数的 f 应用，所以这些范式在语法上两两不同。这说明闭范式至少有可数无限多个；另一方面，每个 lambda 项都是有限字符串，所有项的集合至多可数，因此闭范式恰好有可数无限多个。

这组项正是以 `Unit` 为底层类型写出的 Church 数。尽管它们在语法上不同，但在没有副作用的 `Unit` 语言中，给定函数和参数后，最终可观察到的结果仍只能是 `unit`。因此，这个例子也展示了“语法不同”与“可观察行为不同”并不是一回事。

[返回第 11.2 节脚注谜题](#)

练习 11.8.1

可把记录投影规则显式写成

$$\frac{j \in 1..n \quad l = l_j}{\{l_i = v_i\}^{i \in 1..n}. l \longrightarrow v_j} \text{E-PROJRC D}$$

这里还隐含记录标签两两不同这一良构条件。与正文中的简写相比，这一版本把“所投影标签 l 恰好是第 j 个字段的标签”写进了前提，因此无需再靠文字约定解释下标 j 从何而来。

[返回练习 11.8.1](#)

第 15 章：子类型化

练习 15.5.1

证明目标。若 $\Gamma \vdash t : T$ 且 $t \longrightarrow t'$ ，则 $\Gamma \vdash t' : T$ 。

对给定的类型推导作归纳。原有规则的情形与保持性定理 [定理15.3.5](#)相同，只需检查末条类型规则为 T-DOWNCAST 的新增情形。此时

$$t = t_0 \text{ as } T, \quad \Gamma \vdash t_0 : S$$

现在分析 $t \longrightarrow t'$ 最后使用的求值规则。由于 t 的最外层是类型转换，这一步只可能由下面两类规则导出。

第一，内部的 t_0 还能继续求值。若 $t_0 \longrightarrow t'_0$ ，则 E-ASCRIBE1 给出

$$t_0 \text{ as } T \longrightarrow t'_0 \text{ as } T$$

因而 $t' = t'_0 \text{ as } T$ 。归纳假设给出 $\Gamma \vdash t'_0 : S$ ，再应用 T-DOWNCAST，得到 $\Gamma \vdash t' : T$ 。

第二，内部已经是值 v ，且运行时检查确认 $\vdash v : T$ 。此时 E-DOWNCAST 给出

$$v \text{ as } T \longrightarrow v$$

所以 $t' = v$ 。空上下文中的类型判断可由弱化引理放入任意上下文，因此 $\Gamma \vdash v : T$ ，即 $\Gamma \vdash t' : T$ 。若运行时检查失败，则没有求值规则适用，也就不存在题目所假定的 t' ；因此不形成第三种情形。

所以新增的向下类型转换规则不破坏保持性。

[返回练习 15.5.1](#)

练习 15.5.2

1. 删去第一个前提后，只凭 $T_1 <: S_1$ 就可以推出 $\text{Ref } S_1 <: \text{Ref } T_1$ 。取 $S_1 = \text{Top}$ 、 $T_1 = \text{Nat}$ ，便会错误地得到 $\text{Ref Top} <: \text{Ref Nat}$ 。于是下面的程序能够通过类型检查：

```
(λr : Ref Nat. succ(!r))(ref (true as Top))
```

运行时，引用单元实际保存的是 `true`，读取后得到 `succ true`，求值卡住。缺失的第一个前提本来负责保证“从该引用中读出的实际值确实可以作为 `Nat` 型值使用”。

2. 删去第二个前提后，只凭 $S_1 <: T_1$ 就可以推出上述引用子类型关系。取 $S_1 = \text{Nat}$ 、 $T_1 = \text{Top}$ ，便会错误地得到 $\text{Ref Nat} <: \text{Ref Top}$ 。考虑

```
let r = ref 0 in
let u = (λs : Ref Top. s := true) r in
succ(!r)
```

参数 s 的静态类型允许把 $\text{true} : \text{Top}$ 写入引用，但 r 的原始类型仍是 Ref Nat 。随后读取 r 并取后继，又得到 succ true 。缺失的第二个前提本来负责保证“通过被视为 Ref Top 的引用写入的值，也符合该引用单元实际要求的 Nat 类型”。

[返回练习 15.5.2](#)

练习 15.5.3

下面的 Java 程序能够通过静态类型检查，却会在第三行抛出 `ArrayStoreException`：

```
String[] strings = new String[1];
Object[] objects = strings;
objects[0] = new Object();
```

第二行利用数组协变，把 `String[]` 当作 `Object[]`。第三行按照变量 `objects` 的静态类型是合法的，但这块数组在运行时仍是 `String[]`，不能存入普通 `Object`，所以动态检查失败。

[返回练习 15.5.3](#)

练习 15.6.3

先为源语言中的每个记录类型规定一种固定的字段顺序，例如按标签的字典序排列。这里约定：空记录翻译为空元组；只有一个字段的记录仍翻译成一元组，不直接用其中的字段值代替整个元组。设记录类型 $R = \{l_i : T_i\}^{i \in 1..n}$ 按该顺序写成 $l_1, \dots, l_n$ ，把它翻译为元组类型

$$\llbracket R \rrbracket = \{\llbracket T_i \rrbracket\}^{i \in 1..n}$$

记录项相应翻译为同一顺序的元组，投影 $t.l_i$ 翻译为对 $\llbracket t \rrbracket$ 的第 i 个元组投影。其余类型和项沿用正文的翻译。

子类型推导生成的强制转换按规则定义：

- 自反规则生成恒等函数；传递规则生成函数复合；
- 宽度规则生成一个元组投影函数：它从源记录对应的元组中取出目标记录要求的字段，并用这些字段组成新的元组；
- 排列规则两端采用同一规范字段顺序，因此生成恒等函数；
- 深度规则对每个分量分别应用相应字段类型的强制转换，再组成转换结果的元组；
- 箭头规则和最大类型规则与正文相同。

例如，若 $R = \{a : \text{Nat}, b : \text{Bool}\}$ ，则宽度推导 $R <: \{a : \text{Nat}\}$ 生成的转换是

$$\lambda p : \{\text{Nat}, \text{Bool}\}. \{p.1\}$$

这里的 $\{p.1\}$ 是一元元组，类型为 $\{\mathbf{Nat}\}$ ；花括号是元组构造的一部分。互为排列的记录类型经过规范化后翻译成同一元组类型，所以相应强制转换不需要重排运行时值。最后，对类型推导作结构归纳，证明 $[\Gamma] \vdash [\mathcal{D}] : [T]$ 。普通类型规则直接使用归纳假设；T-SUB 情形还需证明每条子类型推导生成的转换具有类型 $[S] \rightarrow [T]$ 。对该子类型推导再作结构归纳即可；上面逐条给出的元组投影和分量转换恰好覆盖所有新增情形。因此，定理15.6.2在元组目标语言中仍然成立。

[返回练习 15.6.3](#)

第 16 章：子类型化的元理论

练习 16.2.1

证明分为两种情形。

情形 T-Rcd. 考虑一棵以 T-RCD 结束的推导。它的第 i 个前提为 $\Gamma \vdash t_i : T_i$ ，表示记录中标签 l_i 对应的字段项 t_i 具有类型 T_i 。对每个字段分别定义 S_i ：若该前提的最后一条规则是 T-SUB，就取包容前的类型 S_i ，使 $\Gamma \vdash t_i : S_i$ 且 $S_i <: T_i$ ；若最后一条规则不是 T-SUB，则令 $S_i = T_i$ ，并由自反性得到 $S_i <: T_i$ 。先对所有字段应用 T-RCD，得到

$$\Gamma \vdash \{l_i = t_i\}^{i \in 1..n} : \{l_i : S_i\}^{i \in 1..n}$$

再由记录子类型规则得到 $\{l_i : S_i\}^{i \in 1..n} <: \{l_i : T_i\}^{i \in 1..n}$ ，最后在 T-RCD 之后统一使用一次 T-SUB。

情形 T-Proj. 考虑一棵以 T-PROJ 结束、且其前提的最后一条规则是 T-SUB 的推导。设包容前已经得到 $\Gamma \vdash t : S$ ，再由 $S <: \{l_i : T_i\}^{i \in 1..n}$ 推出 $\Gamma \vdash t : \{l_i : T_i\}^{i \in 1..n}$ ，使 T-PROJ 能够导出 $\Gamma \vdash t.l_j : T_j$ 。由子类型反演， S 本身也必须是记录类型，其中包含字段 $l_j : S_j$ ，且 $S_j <: T_j$ 。因此，可以直接从 $\Gamma \vdash t : S$ 使用 T-PROJ 得到 $\Gamma \vdash t.l_j : S_j$ ，再把 T-SUB 作为整个投影推导的最后一条规则，得到 $\Gamma \vdash t.l_j : T_j$ 。

两种情形都把原先位于某个前提末尾的 T-SUB 越过 T-RCD 或 T-PROJ，使它成为导出相同结论的最后一条规则。

[返回练习 16.2.1](#)

练习 16.2.3

取

$$s = (\lambda r : \{x : \mathbf{Nat}\}. r) \{x = 0, y = 1\} \quad \text{和} \quad t = \{x = 0, y = 1\}$$

有 $s \rightarrow t$ 。算法类型规则给 s 的类型是 $S = \{x : \mathbf{Nat}\}$ 。具体地说，抽象中的参数标注使函数具有类型

$$\{x : \mathbf{Nat}\} \rightarrow \{x : \mathbf{Nat}\}$$

实参的算法类型虽然是 $\{x : \text{Nat}, y : \text{Nat}\}$ ，却是参数类型 $\{x : \text{Nat}\}$ 的子类型，所以 TA-APP 可以使用，并以函数的结果类型 S 作为整个应用的类型。

求值时，函数体只是变量 r ，所以一次 β 归约会原样返回实参；参数上的类型标注只限制函数体可以怎样使用 r ，并不会在运行时删除字段 y 。因此求值结果

$$t = \{x = 0, y = 1\}$$

的算法类型为

$$T = \{x : \text{Nat}, y : \text{Nat}\}$$

由记录宽度子类型规则， $T <: S$ ；反向的 $S <: T$ 不成立。因此，单步求值使算法算出的最小类型从 S 变成了更精确的 T ，算法类型关系不满足“求值前后保持同一类型”这一通常形式的保持性定理。

声明式关系仍然满足通常的保持性。它先给 t 赋予类型 T ，再由 $T <: S$ 使用 T-SUB，仍可得到 $\vdash t : S$ 。因此这里失效的只是算法类型的精确相等，类型安全并未受到影响。

[返回练习 16.2.3](#)

第 17 章：子类型化的 ML 实现

练习 17.3.3

下面的实现让子类型检查失败时携带两类信息：比较进行到记录的哪一层，以及该处究竟是缺少字段，还是两侧类型的构造不同。subtype_error 保存这些数据，Subtype_error 则把数据包装成可由 raise 抛出的异常。这样既保留了结构化错误信息，也符合 OCaml 对异常值类型的要求。

typeof 完整处理记录、投影、变量、抽象和应用。应用分支先分别求出函数与实参的类型，再从两个类型的最外层开始调用 check_subtype；递归检查进入记录字段或函数类型内部时，会逐步记录所经过的位置。投影和变量分支也报告缺失字段、越界索引及上下文长度。错误消息会打印出错误路径，以及实际类型和预期类型。完整源码还实现错误消息格式化，并用测试覆盖成功检查、嵌套字段缺失、类型构造不匹配、变量越界、投影缺少字段、投影非记录项和应用非函数项。下面省略本章正文已经展示过的项与类型定义，连续列出错误数据、带路径的子类型检查和完整类型检查器：

OCaml 实现

```
type subtype_error =
  | MissingField of string list * string * ty * ty
  | ShapeMismatch of string list * ty * ty

exception Subtype_error of subtype_error

type type_error =
```

```

| UnboundVariable of int * int
| MissingProjectionField of string * ty
| ProjectionFromNonRecord of ty
| SubtypingFailed of subtype_error
| ExpectedFunction of ty

exception Type_error of type_error

let rec check_subtype path ty_s ty_t =
  if ty_s = ty_t then ()
  else
    match (ty_s, ty_t) with
    | _, TyTop -> ()
    | TyArr (s1, s2), TyArr (t1, t2) ->
      check_subtype ("parameter" :: path) t1 s1;
      check_subtype ("result" :: path) s2 t2
    | TyRecord fields_s, TyRecord fields_t ->
      List.iter
        (fun (label, ty_ti) ->
          match List.assoc_opt label fields_s with
          | None ->
            raise
              (Subtype_error
                (MissingField (List.rev path, label, ty_s, ty_t)))
          | Some ty_si -> check_subtype (label :: path) ty_si ty_ti)
        fields_t
    | _ ->
      raise
        (Subtype_error (ShapeMismatch (List.rev path, ty_s, ty_t)))

let rec typeof context term =
  match term with
  | TmRecord fields ->
    TyRecord
      (List.map (fun (label, field) -> (label, typeof context field))
        ↪ fields)
  | TmProj (record, label) -> (
    match typeof context record with
    | TyRecord fields -> (
      match List.assoc_opt label fields with
      | Some field_type -> field_type
      | None ->

```

```

        raise (Type_error (MissingProjectionField (label, TyRecord
            ↪ fields))))
    | other -> raise (Type_error (ProjectionFromNonRecord other)))
| TmVar index -> (
    match List.nth_opt context index with
    | Some ty -> ty
    | None -> raise (Type_error (UnboundVariable (index, List.length
        ↪ context))))
| TmAbs (parameter_type, body) ->
    TyArr (parameter_type, typeof (parameter_type :: context) body)
| TmApp (fn, argument) ->
    let function_type = typeof context fn in
    let argument_type = typeof context argument in
    (match function_type with
    | TyArr (parameter_type, result_type) ->
        (try
            check_subtype [] argument_type parameter_type;
            result_type
        with Subtype_error detail ->
            raise (Type_error (SubtypingFailed detail)))
    | other -> raise (Type_error (ExpectedFunction other)))

```

Rust 对照实现沿用本章正文的 Type、Term 和 Context，同样把诊断数据、带路径的子类型检查和类型检查器放在一份连续实现中。

Rust 辅助实现（译者增补）

```

/// 描述子类型检查和类型检查失败的准确位置与原因。
///
/// 与只返回 `false` 的子类型判断不同，这些变体会保存沿嵌套记录或函数类型
/// 走过的路径，以及出错位置的实际类型和预期类型。
#[derive(Clone, Debug, PartialEq, Eq)]
pub enum DiagnosticError {
    /// 源记录缺少目标记录要求的字段。
    MissingSubtypeField {
        path: Vec<String>,
        label: String,
        source: Type,
        target: Type,
    },
    /// 对应位置的类型构造不同，例如一侧是记录而另一侧是函数。
    ShapeMismatch {
        path: Vec<String>,

```



```

        source: Type,
        target: Type,
    },
    /// de Bruijn 索引超出了当前上下文。
    UnboundVariable { index: usize, context_len: usize },
    /// 投影所选字段不存在。
    MissingProjectionField { label: String, record: Type },
    /// 投影的左侧不是记录。
    ExpectedRecord(Type),
    /// 应用的左侧不是函数。
    ExpectedFunction(Type),
    /// 应用实参不满足函数参数类型；内部错误保留具体的子类型失败原因。
    ParameterMismatch(Box<DiagnosticError>),
}

/// 检查 `source <: target`，并在首次失败的位置返回结构化诊断。
///
/// `path` 记录递归检查已经进入的字段或函数位置；调用入口通常传入空切片。
/// 函数参数沿逆变方向递归，函数结果和记录字段沿协变方向递归。
pub fn check_subtype(path: &[String], source: &Type, target: &Type) ->
    ↪ Result<(), DiagnosticError> {
    if source == target {
        return Ok(());
    }
    match (source, target) {
        (_, Type::Top) => Ok(()),
        (Type::Arrow(source_in, source_out), Type::Arrow(target_in,
    ↪ target_out)) => {
            check_subtype(&extend_path(path, "parameter"), target_in,
    ↪ source_in)?;
            check_subtype(&extend_path(path, "result"), source_out,
    ↪ target_out)
        }
        (Type::Record(source_fields), Type::Record(target_fields)) => {
            for (label, target_type) in target_fields {
                let (_, source_type) = source_fields
                    .iter()
                    .find(|(source_label, _)| source_label == label)
                    .ok_or_else(|| DiagnosticError::MissingSubtypeField {
                        path: path.to_vec(),
                        label: label.clone(),
                        source: source.clone(),

```

```

        target: target.clone(),
    })?;
    check_subtype(&extend_path(path, label.clone()), source_type,
↪ target_type)?;
    }
    Ok(())
}
_ => Err(DiagnosticError::ShapeMismatch {
    path: path.to_vec(),
    source: source.clone(),
    target: target.clone(),
}),
}
}

/// 检查一个应用项并返回其结果类型。
///
/// 这里先分别计算函数与实参的类型，再用带路径的子类型检查验证实参；失败时
/// `ParameterMismatch` 会保留内部的字段路径或类型构造差异。
fn diagnostic_application_type(
    context: &Context,
    function: &Term,
    argument: &Term,
) -> Result<Type, DiagnosticError> {
    let function_type = diagnostic_type_of(context, function)?;
    let argument_type = diagnostic_type_of(context, argument)?;
    match function_type {
        Type::Arrow(parameter_type, result_type) => {
            check_subtype(&[], &argument_type, &parameter_type)
                .map_err(|error|
↪ DiagnosticError::ParameterMismatch(Box::new(error)))?;
            Ok(*result_type)
        }
        other => Err(DiagnosticError::ExpectedFunction(other)),
    }
}

```

返回练习 17.3.3

练习 17.3.4

强制转换语义把隐含的子类型使用编译成普通函数调用。先区分源语言类型与目标语言类型。类型翻译记作 $\llbracket T \rrbracket$ ，其中

$$\llbracket \text{Top} \rrbracket = \text{Unit}, \quad \llbracket S \rightarrow T \rrbracket = \llbracket S \rrbracket \rightarrow \llbracket T \rrbracket, \quad \llbracket \{l_i : T_i\}^{i \in 1..n} \rrbracket = \{l_i : \llbracket T_i \rrbracket\}^{i \in 1..n}$$

因此，`coerce S T` 在确认 $S <: T$ 后，生成的函数具有类型 $\llbracket S \rrbracket \rightarrow \llbracket T \rrbracket$ ，并非未经翻译的 $S \rightarrow T$ 。特别地，目标为 `Top` 时生成 $\lambda x : \llbracket S \rrbracket. \text{unit}$ ，其结果确实具有 `Unit` 类型。函数类型的参数位置反向生成强制转换，结果位置沿原方向生成；记录类型则只重建目标类型要求的字段。

例如，把

$$\{x = \{\}, y = \{\}\}$$

传给只接受 $\{x : \text{Top}\}$ 的函数时，类型检查使用记录宽度子类型化。翻译器会插入一个显式转换函数，其目标语言形式为

$$\lambda r : \{x : \text{Unit}, y : \text{Unit}\}. \{x = r.x\}$$

这个函数丢弃字段 y ，把实参变成函数实际要求的记录。翻译后的程序因而不需要子类型规则。

项翻译同时返回目标类型与目标项。应用 $t_1 t_2$ 若得到源类型 $S_1 \rightarrow S_2$ 和 T_1 ，便在实参与函数之间插入从 $\llbracket T_1 \rrbracket$ 到 $\llbracket S_1 \rrbracket$ 的强制转换。抽象中的类型标注、上下文中保存的类型以及记录字段类型也都经过同一类型翻译。

下面只列出本题新增的核心 OCaml 实现：源语言与目标语言的语法、类型翻译、强制转换构造及项翻译。目标语言的移位、替换与求值沿用第七章的算法，完整配套源码中保留这些实现和端到端测试。

OCaml 实现

```
type ty =
  | TyTop
  | TyArr of ty * ty
  | TyRecord of (string * ty) list

type term =
  | TmRecord of (string * term) list
  | TmProj of term * string
  | TmVar of int
  | TmAbs of ty * term
  | TmApp of term * term

type target_ty =
  | TyUnit
  | TyTargetArr of target_ty * target_ty
  | TyTargetRecord of (string * target_ty) list

type target_term =
  | TmUnit
  | TmTargetRecord of (string * target_term) list
  | TmTargetProj of target_term * string
  | TmTargetVar of int
  | TmTargetAbs of target_ty * target_term
  | TmTargetApp of target_term * target_term

exception Translation_error of string

let rec translate_type = function
  | TyTop -> TyUnit
  | TyArr (parameter, result) ->
    TyTargetArr (translate_type parameter, translate_type result)
  | TyRecord fields ->
    TyTargetRecord
      (List.map
         (fun (label, field_type) -> (label, translate_type field_type))
         fields)

let rec coerce ty_s ty_t =
  if ty_s = ty_t then TmTargetAbs (translate_type ty_s, TmTargetVar 0)
  else
    match (ty_s, ty_t) with
```

```

| _, TyTop -> TmTargetAbs (translate_type ty_s, TmUnit)
| TyArr (s1, s2), TyArr (t1, t2) ->
  let parameter_coercion = coerce t1 s1 in
  let result_coercion = coerce s2 t2 in
  let coerced_argument =
    TmTargetApp (parameter_coercion, TmTargetVar 0)
  in
  let function_result =
    TmTargetApp (TmTargetVar 1, coerced_argument)
  in
  TmTargetAbs
    ( translate_type ty_s,
      TmTargetAbs
        ( translate_type t1,
          TmTargetApp (result_coercion, function_result) ) )
| TyRecord fields_s, TyRecord fields_t ->
  let fields =
    List.map
      (fun (label, ty_ti) ->
        match List.assoc_opt label fields_s with
        | None ->
          raise
            (Translation_error ("missing source field " ^ label))
        | Some ty_si ->
          ( label,
            TmTargetApp
              ( coerce ty_si ty_ti,
                TmTargetProj (TmTargetVar 0, label) ) ) )
      fields_s
  in
  TmTargetAbs (translate_type ty_s, TmTargetRecord fields)
| _ -> raise (Translation_error "no subtype coercion exists")

```

完整源码随后还有一个普通的源语言类型检查函数 `typeof`；它不参与强制转换构造，且结构与本章正文的类型检查器相同，因而在这里略去。接下来完整列出项翻译函数：

```

let rec translate context term =
  match term with
  | TmRecord fields ->
    let translated =
      List.map
        (fun (label, field) ->

```

```

        let field_type, field_term = translate context field in
        ((label, field_type), (label, field_term)))
    fields
in
( TyRecord (List.map fst translated),
  TmTargetRecord (List.map snd translated) )
| TmProj (record, label) -> (
  let record_type, record_term = translate context record in
  match record_type with
  | TyRecord fields -> (
    match List.assoc_opt label fields with
    | Some field_type ->
      (field_type, TmTargetProj (record_term, label))
    | None -> raise (Translation_error ("missing field " ^ label))
  | _ -> raise (Translation_error "projection expected a record"))
| TmVar index -> (
  match List.nth_opt context index with
  | Some ty -> (ty, TmTargetVar index)
  | None -> raise (Translation_error "unbound variable"))
| TmAbs (parameter_type, body) ->
  let body_type, body_term = translate (parameter_type :: context) body in
  ( TyArr (parameter_type, body_type),
    TmTargetAbs (translate_type parameter_type, body_term) )
| TmApp (fn, argument) -> (
  let function_type, function_term = translate context fn in
  let argument_type, argument_term = translate context argument in
  match function_type with
  | TyArr (parameter_type, result_type) ->
    let argument_coercion = coerce argument_type parameter_type in
    ( result_type,
      TmTargetApp
        ( function_term,
          TmTargetApp (argument_coercion, argument_term) ) )
  | _ -> raise (Translation_error "application expected a function"))

```

Rust 对照实现采用相同结构：

Rust 辅助实现（译者增补）

```

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum TargetType {
    Unit,
    Arrow(Box<TargetType>, Box<TargetType>),

```

```

    Record(Vec<(String, TargetType)>),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum TargetTerm {
    Unit,
    Variable(usize),
    Abstraction(TargetType, Box<TargetTerm>),
    Application(Box<TargetTerm>, Box<TargetTerm>),
    Record(Vec<(String, TargetTerm)>),
    Projection(Box<TargetTerm>, String),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum TranslationError {
    NotSubtype { source: Type, target: Type },
    UnboundVariable(usize),
    MissingField(String),
    ExpectedRecord(Type),
    ExpectedFunction(Type),
}

/// 把源语言类型递归映射为不含子类型关系的目标语言类型。
///
/// 翻译结果用于目标项的类型标注、目标语言上下文，以及 `coerce` 所生成函数的
/// 输入和输出类型。这个函数只改变类型的表示，不构造实际的值转换；`Top` 被
/// 擦除为只有一个值的 `Unit`，箭头与记录则递归翻译其组成部分。实际的值转换
/// 由 `coerce` 生成。
pub fn translate_type(source: &Type) -> TargetType {
    match source {
        Type::Top => TargetType::Unit,
        Type::Arrow(input, output) => TargetType::Arrow(
            Box::new(translate_type(input)),
            Box::new(translate_type(output)),
        ),
        Type::Record(fields) => TargetType::Record(
            fields
                .iter()
                .map(|(label, ty)| (label.clone(), translate_type(ty)))
                .collect(),
        ),
    }
}

```

```

}

/// 根据 `source <: target` 构造一个显式强制转换函数。
///
/// 返回的目标项具有 `translate_type(source) -> translate_type(target)` 类型。
/// 若源类型并非目标类型的子类型，则返回 `TranslationError::NotSubtype`。
pub fn coerce(source: &Type, target: &Type) -> Result<TargetTerm,
↳ TranslationError> {
    let source_target_type = translate_type(source);
    // 相同类型之间的强制转换就是恒等函数。
    if source == target {
        return Ok(TargetTerm::Abstraction(
            source_target_type,
            Box::new(TargetTerm::Variable(0)),
        ));
    }
    match (source, target) {
        // `Top` 翻译成 `Unit`，因此转换只需丢弃输入并返回 `unit`。
        (_, Type::Top) => Ok(TargetTerm::Abstraction(
            source_target_type,
            Box::new(TargetTerm::Unit),
        )),
        (Type::Arrow(source_in, source_out), Type::Arrow(target_in,
↳ target_out)) => {
            // 要把原函数 f : source_in -> source_out 包装成
            // target_in -> target_out，箭头子类型规则要求：
            //
            //   target_in <: source_in
            //   source_out <: target_out
            //
            // 因此生成的包装函数具有以下结构：
            //
            //   lambda f. lambda x.
            //       output_coercion (f (input_coercion x))
            //
            // 新函数收到 target_in 型参数 x，先把它转换成原函数能够接受的
            // source_in；原函数返回 source_out 后，再转换成承诺的 target_out。
            let input_coercion = coerce(target_in, source_in)?;
            let output_coercion = coerce(source_out, target_out)?;

            // 两层抽象依次绑定 f 和 x。在最内层函数体中，Variable(0) 是 x，
            // Variable(1) 是外层的 f。

```



```

Ok(TargetTerm::Abstraction(
  translate_type(source),
  Box::new(TargetTerm::Abstraction(
    translate_type(target_in),
    Box::new(TargetTerm::Application(
      Box::new(output_coercion),
      Box::new(TargetTerm::Application(
        Box::new(TargetTerm::Variable(1)),
        Box::new(TargetTerm::Application(
          Box::new(input_coercion),
          Box::new(TargetTerm::Variable(0)),
        )),
      )),
    )),
  )),
))
}
(Type::Record(source_fields), Type::Record(target_fields)) => {
  // 目标语言没有记录子类型规则，因此不能直接把带有额外字段的源记录
  // 当作目标记录使用。这里生成如下形式的包装函数：
  //
  //   lambda r. {
  //     l1 = c1 (r.l1),
  //     ...
  //   }
  //
  // 输出必须恰好提供目标类型要求的字段，所以遍历 target_fields;
  // 源记录的额外字段被丢弃，这对应记录宽度子类型化。
  let translated_fields = target_fields
    .iter()
    .map(|(label, target_type)| {
      // 每个目标字段都必须能在源记录中找到同名字段。
      let (_, source_type) = source_fields
        .iter()
        .find(|(source_label, _)| source_label == label)
        .ok_or_else(||
↳ TranslationError::MissingField(label.clone()))?;

      // 保留下来的字段也可能需要从 source_type 转成 target_type;
      // 这对应记录深度子类型化。
      let field_coercion = coerce(source_type, target_type)?;
      Ok((

```

```

        label.clone(),
        // 当前只有一层 lambda, Variable(0) 就是源记录 r。
        // 这一项表示 c_i (r.l_i)。
        TargetTerm::Application(
            Box::new(field_coercion),
            Box::new(TargetTerm::Projection(
                Box::new(TargetTerm::Variable(0)),
                label.clone(),
            )),
        ),
    ))
    })
    .collect::

```

/// 翻译函数应用，并在实参外显式插入所需的强制转换。

///

/// 返回值同时给出目标语言中的结果类型和应用项。其余项构造由

/// `translate\_term` 逐层递归翻译。

```

fn translate_application(
    context: &Context,
    function: &Term,
    argument: &Term,
) -> Result<(TargetType, TargetTerm), TranslationError> {
    let source_function_type =
        type_of(context, function).map_err(|_|
    ↪ TranslationError::ExpectedFunction(Type::Top))?;
    let source_argument_type =
        type_of(context, argument).map_err(|_|
    ↪ TranslationError::ExpectedFunction(Type::Top))?;
    let (_, function_term) = translate_term(context, function)?;
    let (_, argument_term) = translate_term(context, argument)?;
}

```

```

match source_function_type {
  Type::Arrow(parameter_type, result_type) => {
    // 把实参从其实际类型显式转换为函数要求的参数类型。
    let argument_coercion = coerce(&source_argument_type,
↪ &parameter_type)?;
    Ok((
      translate_type(&result_type),
      TargetTerm::Application(
        Box::new(function_term),
        Box::new(TargetTerm::Application(
          Box::new(argument_coercion),
          Box::new(argument_term),
        )),
      ),
    ))
  }
  other => Err(TranslationError::ExpectedFunction(other)),
}
}

```

端到端测试构造上述宽记录应用，检查翻译后的项仍为良类型，并最终求值为 `unit:`
 Rust 辅助实现（译者增补）

```

#[test]
fn coercion_translation_is_typed_and_evaluates() {
  let function = Term::Abstraction(
    "r".into(),
    x_top_record(),
    Box::new(Term::Projection(Box::new(Term::Variable(0)), "x".into())),
  );
  let argument = Term::Record(vec![
    ("x".into(), Term::Record(Vec::new())),
    ("y".into(), Term::Record(Vec::new())),
  ]);
  let source = Term::Application(Box::new(function), Box::new(argument));
  let (translated_type, translated_term) = translate_term(&Vec::new(),
↪ &source).unwrap();
  assert_eq!(translated_type, TargetType::Unit);
  assert_eq!(
    target_type_of(&[], &translated_term),
    Some(TargetType::Unit)
  );
}

```

```
assert_eq!(target_eval(&translated_term), TargetTerm::Unit);
}
```

[返回练习 17.3.4](#)

第 18 章：案例研究——命令式对象

练习 18.6.2

在项的语法中增加记录扩展构造：

$$t ::= \dots \mid t \text{ with } \{l_i = t_i\}^{i \in 1..n}$$

右侧列表中的标签互不重复；若某个标签已经出现在基记录中，新字段会替换旧字段，否则追加为新字段。

求值采用从左到右的按值次序。先求值基记录；基记录成为值后，再依次求值扩展列表中的各字段。所有部分都是值时执行合并：

$$\frac{t \longrightarrow t'}{t \text{ with } F \longrightarrow t' \text{ with } F} \text{ E-WITHBASE}$$

$$\frac{t_i \longrightarrow t'_i}{v \text{ with } \{l_1 = v_1, \dots, l_i = t_i, \dots, l_n = t_n\} \longrightarrow v \text{ with } \{l_1 = v_1, \dots, l_i = t'_i, \dots, l_n = t_n\}} \text{ E-WITHFIELD}$$

其中最后一条规则只在 $v, v_1, \dots, v_{i-1}$ 已经是值时使用。若 $v = \{k_j = w_j\}^{j \in 1..m}$ ，合并规则为

$$\frac{}{\{k_j = w_j\}^{j \in 1..m} \text{ with } \{l_i = v_i\}^{i \in 1..n} \longrightarrow \{k_j = w_j \mid k_j \notin \{l_1, \dots, l_n\}\}, \{l_i = v_i\}^{i \in 1..n}} \text{ E-WITH}$$

结论右侧表示先保留未被覆盖的旧字段，再放入全部新字段。

类型规则先推导基记录与每个新字段的类型，再以同样方式合并字段类型：

$$\frac{\Gamma \vdash t : \{k_j : T_j\}^{j \in 1..m} \quad \Gamma \vdash t_i : S_i \quad (i \in 1..n)}{\Gamma \vdash t \text{ with } \{l_i = t_i\}^{i \in 1..n} : \{k_j : T_j \mid k_j \notin \{l_1, \dots, l_n\}\}, \{l_i : S_i\}^{i \in 1..n}} \text{ T-WITH}$$

它允许覆盖字段时改变字段类型，因为扩展结果是一条新记录，不会修改原记录。若希望构造只表达“增加字段而不覆盖”，再加旁条件 $\{l_i\} \cap \{k_j\} = \emptyset$ 即可。正文中的例子由此写成 `super with {reset = ...}`，无需重复列出 `get` 与 `inc`。

[返回练习 18.6.2](#)

第 19 章：案例研究——Featherweight Java

练习 19.4.3

FJ 原本把对象值直接写成 $\text{new } C(\bar{v})$ 。这样的值没有独立身份；若要修改字段，必须像引用演算那样另设存储。下面给出一种完整扩展。

在项和值中加入位置 l 和赋值：

$$t ::= \dots \mid l \mid t.f := t \quad v ::= l$$

存储 μ 把位置映射到已完成初始化的对象 $\text{new } C(\bar{v})$ 。求值判断改写为 $(t, \mu) \longrightarrow (t', \mu')$ 。对象创建先按从左到右的次序求值全部实参，再分配新位置：

$$\frac{l \notin \text{dom}(\mu)}{(\text{new } C(\bar{v}), \mu) \longrightarrow (l, \mu[l \mapsto \text{new } C(\bar{v})])} \text{E-NEWSTORE}$$

字段读取改为从存储取值：

$$\frac{\mu(l) = \text{new } C(\bar{v}) \quad \text{fields}(C) = \bar{C} \bar{f}}{(l.f_i, \mu) \longrightarrow (v_i, \mu)} \text{E-FIELDSTORE}$$

方法调用仍使用 $mbody$ ，但现在以接收者位置替换 **this**：

$$\frac{\mu(l) = \text{new } C(\bar{v}) \quad mbody(m, C) = (\bar{x}, t_0)}{(l.m(\bar{u}), \mu) \longrightarrow ([\bar{x} \mapsto \bar{u}, \text{this} \mapsto l]t_0, \mu)} \text{E-INVKSTORE}$$

类型转换必须查询接收者位置中保存对象的运行期类：

$$\frac{\mu(l) = \text{new } C(\bar{v}) \quad C <: D}{((D)l, \mu) \longrightarrow (l, \mu)} \text{E-CASTSTORE}$$

若 $C \not<: D$ ，这项转换没有可用规则，仍按 FJ 的约定卡住。缺少这条规则会使对象一经分配成位置后，所有本来可以成功的类型转换都无法继续。

赋值先求值接收者，再求值右侧；两者均为值后，按照以下规则更新对应字段：

$$\frac{\mu(l) = \text{new } C(v_1, \dots, v_i, \dots, v_n) \quad \text{fields}(C) = C_1 f_1, \dots, C_n f_n}{(l.f_i := v, \mu) \longrightarrow (v, \mu[l \mapsto \text{new } C(v_1, \dots, v, \dots, v_n)])} \text{E-ASSIGN}$$

规则结论右侧配置的第一分量是 v ，表示赋值表达式本身的求值结果就是刚写入的值；第二分量则是完成字段更新后的存储。这与 Java 赋值表达式的行为一致。若希望赋值只是一条命令，也可加入 **Unit** 并返回 **unit**。

所有同余规则都要改成携带存储。可以用求值上下文一次写全：在 定义19.5.3的上下文语法中加入 $E.f := t$ 与 $l.f := E$ ，并使用

$$\frac{(t, \mu) \longrightarrow (t', \mu')}{(E[t], \mu) \longrightarrow (E[t'], \mu')} \text{E-CONTEXTSTORE}$$

这里的上下文仍规定从左到右求值，因此字段访问、方法调用、对象创建、类型转换和赋值都会把每一步产生的新存储继续传给外围项。

类型层面加入存储类型 Σ ，其中 $\Sigma(l) = C$ 表示位置 l 存放类 C 的对象。位置和赋值规则为

$$\frac{\Sigma(l) = C}{\Gamma \mid \Sigma \vdash l : C} \text{ T-LOC}$$

$$\frac{\Gamma \mid \Sigma \vdash t_0 : C_0 \quad \text{fields}(C_0) = \overline{C} \ \overline{f} \quad \Gamma \mid \Sigma \vdash t_1 : D \quad D <: C_i}{\Gamma \mid \Sigma \vdash t_0.f_i := t_1 : C_i} \text{ T-ASSIGN}$$

还需定义 $\Sigma \vdash \mu$ ：二者定义域相同，并且每个位置中对象的全部字段值都符合该类声明的字段类型。保持性定理随后与第十三章相同：一步求值可能扩展存储和存储类型，但已有位置的类型不变；字段更新保持存储良类型。

返回练习 19.4.3

练习 19.4.4

加入 Java 中用于抛出异常的 `throw`，以及用于捕获异常的 `try-catch` 构造：

$$t ::= \dots \mid \text{throw } t \mid \text{try } t \text{ catch } (C \ x) \ h$$

并约定可抛出的对象都是某个根类 `Throwable` 的子类。异常本身仍是普通对象值。当被抛出的项已经求值为对象值 v 时，`throw v` 表示异常已经产生；接下来会跳过外层尚未完成的计算，向外寻找类型匹配的 `catch`。

先求值 `throw` 的参数与 `try` 的受保护项：

$$\frac{t \longrightarrow t'}{\text{throw } t \longrightarrow \text{throw } t'} \text{ E-THROW} \quad \frac{t \longrightarrow t'}{\text{try } t \text{ catch } (C \ x) \ h \longrightarrow \text{try } t' \text{ catch } (C \ x) \ h} \text{ E-TRY}$$

受保护项正常得到值时，处理器被丢弃：

$$\text{try } v \text{ catch } (C \ x) \ h \longrightarrow v$$

若得到 `throw v`，令 D 为 v 的运行类。若 $D <: C$ ，执行处理器并以 v 替换 x ；否则继续向外传播：

$$\frac{v = \text{new } D(\overline{v}) \quad D <: C}{\text{try } (\text{throw } v) \text{ catch } (C \ x) \ h \longrightarrow [x \mapsto v]h} \text{ E-CATCH}$$

$$\frac{v = \text{new } D(\overline{v}) \quad D \not<: C}{\text{try } (\text{throw } v) \text{ catch } (C \ x) \ h \longrightarrow \text{throw } v} \text{ E-MISS}$$

此外，对字段访问、方法调用、对象创建、类型转换及其参数位置，都要加入异常传播规则；一旦当前求值位置产生 `throw v`，就停止求值同一构造中尚未处理的部分，并把异常传到外层。

`throw t` 一旦执行，就不会向当前计算返回普通值，而会中断正常求值并向外寻找处理器；因此，它不可能产生一个类型不符合外层预期的正常结果。这使它可以出现在任何预期类型的位置，例如类型声明为 C 但总是抛出异常的方法体。相应的类型规则让 `throw t` 可以取得任意预期类型：

$$\frac{\Gamma \vdash t : D \quad D <: \text{Throwable}}{\Gamma \vdash \text{throw } t : C} \text{ T-THROW}$$

前提只要求被抛出的对象具有 **Throwable** 的子类型；结论中的 C 与异常对象的类型 D 无关，可以由 `throw t` 所在位置的类型要求决定。

处理器的两个正常出口必须具有相容类型。采用 FJ 的算法式风格，可要求处理器结果是受保护项结果类型的子类型，或反过来，再选择较大的类型；采用带包容规则的声明式系统，则可先把两个分支都提升到共同超类型 T ：

$$\frac{\Gamma \vdash t : T \quad \Gamma, x : C \vdash h : T \quad C <: \text{Throwable}}{\Gamma \vdash \text{try } t \text{ catch } (C \ x) \ h : T} \text{ T-TRY}$$

这样，异常传播不会改变外围项期望的类型；进展性则把最终结果扩展为三类：普通值、未处理的 `throw v`，或失败的类型转换。

[返回练习 19.4.4](#)

练习 19.4.5

可以加入包容规则，但类型转换不能直接读取一个已经经过包容提升的类型。先记原书的算法判断为 $\Gamma \vdash_{\min} t : S$ ：它为项计算唯一的最小类型 S 。再定义声明式判断 $\Gamma \vdash_d t : T$ ，并加入

$$\frac{\Gamma \vdash_d t : S \quad S <: T}{\Gamma \vdash_d t : T} \text{ T-SUB}$$

对方法调用、对象创建和方法体检查，可以删去原规则中单列的子类型前提，直接借助 T-Sub 把实参或方法体提升到声明所要求的类型。例如：

$$\frac{\Gamma \vdash_d t_0 : C_0 \quad \text{mtype}(m, C_0) = \bar{D} \rightarrow C \quad \Gamma \vdash_d \bar{t} : \bar{D}}{\Gamma \vdash_d t_0.m(\bar{t}) : C} \text{ T-INVKD}$$

$$\frac{\text{fields}(C) = \bar{D} \ \bar{f} \quad \Gamma \vdash_d \bar{t} : \bar{D}}{\Gamma \vdash_d \text{new } C(\bar{t}) : C} \text{ T-NEWD}$$

若实参的最小类型为 $C_i <: D_i$ ，先由其声明式规则取得 C_i ，再用 T-Sub 提升为 D_i ，即可满足这些规则。

向上、向下和荒谬转换必须依据同一个确定的源类型分类；否则，T-Sub 可以先改变被转换项的类型，再让同一转换落入另一条规则。因此，三条转换规则都以算法检查器算出的最小类型为准：

$$\frac{\Gamma \vdash_{\min} t_0 : D \quad D <: C}{\Gamma \vdash_d (C)t_0 : C} \text{ T-UCastD}$$

$$\frac{\Gamma \vdash_{\min} t_0 : D \quad C <: D \quad C \neq D}{\Gamma \vdash_d (C)t_0 : C} \text{ T-DCastD}$$

第三条荒谬转换规则不再另列；它也使用前提 $\Gamma \vdash_{\min} t_0 : D$ ，并要求 C 与 D 互不为子类型。

实现仍先运行原书的算法检查器求最小类型；声明式推导只说明此后允许把结果用于任何超类型位置。若算法为项 t 算出最小类型 S ，声明式系统便可将 t 赋予 S 或 S 的任意超类型 T ；反过来，声明式系统能够赋予 t 的每个类型 T 都满足 $S <: T$ 。这样，T-Sub 只会增加项可以取得的超类型，不会改变类型转换的判断。例如，虽然 $\text{new } B() : B$ 可以通过 T-Sub 提升为 `Object`，转换 $(A)\text{new } B()$ 仍依据最小类型 B 检查，因此依旧是荒谬转换，不会被误判为从 `Object` 到 A 的普通向下转换。

[返回练习 19.4.5](#)

练习 19.5.5

下面给出一个可执行的 FJ 实现。完整程序由类表、静态检查和求值三部分组成；完整源码位于 `code/book-snippets/src/chapter19.rs`。这道工程题的实现较长，书中不再逐页展开 Rust 源码，而是交代数据模型、关键检查、求值顺序和端到端例子，使读者可以先理解实现结构，需要时再到工程中逐函数核对。

一、数据模型与类表。项的数据类型直接对应图19-1中的五种形式：变量、字段访问、方法调用、对象创建和类型转换；类声明则保存类名、超类、字段、构造函数与方法。类表是从类名到类声明的映射，并提供四项基本查询：

$\text{subtype}(C, D)$ 从 C 沿超类链向上查找，判断是否能到达 D 。

$\text{fields}(C)$ 先取得继承字段，再追加 C 自己声明的字段。

$\text{mtype}(m, C)$ 沿继承链查找方法 m ，返回形参类型序列与结果类型。

$\text{mbody}(m, C)$ 按相同次序查找方法，返回形参名序列与方法体。

载入类表时还要检查类名和成员名不重复、引用的类已经定义、继承图无环、子类不遮蔽继承字段，以及覆盖方法的形参与结果类型和原方法完全相同。

二、类声明检查。构造函数的形参必须依次列出继承字段与当前类字段；传给 `super` 的实参必须恰好是继承字段；随后每个当前类字段都由同名形参赋给 `this.f`。检查方法时，在包含各形参和 `this` 的环境中求出方法体类型，确认它是声明结果类型的子类型，再检查方法覆盖是否满足上述约束。

三、项的类型检查。类型检查函数按图19-4递归处理项。变量从环境取类型；字段访问先求出接收者类型，再从该类的完整字段序列中查找字段；方法调用先查询方法签名，然后逐项检查实参与形参；对象创建则用完整字段序列检查构造实参。类型转换依据源类与目标类的子类型关系分为向上转换、向下转换和荒谬转换；实现记录荒谬转换警告，但仍按原 FJ 规则返回目标类型。

四、单步求值。求值函数按图19-3从左到右寻找下一项待求值的子项。接收者已经成为 $\text{new } C(\bar{v})$ 后，字段访问按字段在完整字段序列中的位置取值；方法调用查询方法体，并同时完成替换 $\{x_i \mapsto v_i, \text{this} \mapsto v_0\}$ 。这里必须使用同时替换，否则某个实参中恰好出现另一形参名时，后续替换会错误地再次改写它。运行期对象的类是目标类的子类型时，类型转换可以删去；否则该项卡住。多步解释器反复执行一次单步求值，直到得到对象值或再无规则可用。

五、端到端例子。正文中的 `Pair.setfst` 调用依次求值为

$$\begin{aligned} & \text{new Pair}(\text{new } A(), \text{new } B()).\text{setfst}(\text{new } B()) \\ \longrightarrow & \text{new Pair}(\text{new } B(), (\text{new Pair}(\text{new } A(), \text{new } B())).\text{snd}) \\ \longrightarrow & \text{new Pair}(\text{new } B(), \text{new } B()) \end{aligned}$$

第一步把实参替换给 `newfst`，并把接收者替换给 `this`；第二步再从接收者对象中投影出 `snd` 字段。这条路径同时经过方法查询、同步替换和字段投影，因而可以作为整份实现的最小端到端检查。

配套测试还覆盖合法与失败的类型转换、荒谬转换警告、构造函数和方法参数错误、非法覆盖、继承环，以及重复或保留名称。完整源码参加 `make rust-check` 的编译和测试；书中只保留理解实现所需的结构、算法与完整求值路径。

[返回练习 19.5.5](#)

练习 19.5.6

GJ 风格扩展的核心，是让类、方法和类型都可以带有受上界约束的类型变量。下面说明如何扩展练习19.5.5的实现，并列出新增机制的关键代码。

语法。类型从类名扩展为类型变量或带实参的类类型：

$$T ::= X \mid C(\bar{T})$$

类与方法声明分别增加类型形参 $\langle \bar{X} <: \bar{N} \rangle$ ，对象创建、方法调用和类型转换也携带相应的类型实参。上界 N 可以提到先前声明的类型变量，从而表达 F-有界约束，例如 $X <: \text{Comparable}\langle X \rangle$ 。

Rust 辅助实现（译者增补）

```
#[derive(Clone, Debug, PartialEq, Eq, Hash)]
pub enum GType {
```

```

    Variable(String),
    Class(String, Vec<GType>),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub struct TypeParameter {
    pub name: String,
    pub bound: GType,
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub enum GTerm {
    Variable(String),
    Field(Box<GTerm>, String),
    Invoke(Box<GTerm>, String, Vec<GType>, Vec<GTerm>),
    New(GType, Vec<GTerm>),
    Cast(GType, Box<GTerm>),
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub struct GMethod {
    pub type_parameters: Vec<TypeParameter>,
    pub result: GType,
    pub name: String,
    pub parameters: Vec<(GType, String)>,
    pub body: GTerm,
}

#[derive(Clone, Debug, PartialEq, Eq)]
pub struct GClass {
    pub name: String,
    pub type_parameters: Vec<TypeParameter>,
    pub superclass: GType,
    pub fields: Vec<(GType, String)>,
    pub methods: Vec<GMethod>,
}

```

环境。在变量环境 Γ 之外增加类型变量环境 Δ ，记录每个类型变量的上界。先定义类型良构判断 $\Delta \vdash T$ OK：类类型的实参数量必须正确，且把实际类型代入声明上界后，每个实参都必须是对应上界的子类型。

类或方法的形式类型变量换成实际类型实参时，递归使用下面的类型替换函数。

Rust 辅助实现（译者增补）

```
fn substitute_type(ty: &GType, substitution: &HashMap<String, GType>) -> GType
↪ {
    match ty {
        GType::Variable(name) => substitution
            .get(name)
            .cloned()
            .unwrap_or_else(|| ty.clone()),
        GType::Class(name, arguments) => GType::Class(
            name.clone(),
            arguments
                .iter()
                .map(|argument| substitute_type(argument, substitution))
                .collect(),
        ),
    }
}
```

子类型。增加变量规则

$$\frac{X <: N \in \Delta}{\Delta \vdash X <: N}$$

类继承规则需要先把类声明中的形式类型变量替换为实际类型参数。例如，若

class $C\langle \overline{X} <: \overline{N} \rangle$ extends $D\langle \overline{U} \rangle \{ \dots \}$,

则

$$C\langle \overline{T} \rangle <: [\overline{X} \mapsto \overline{T}]D\langle \overline{U} \rangle$$

类型参数本身保持不变；GJ 的类类型并不会仅因实参之间有子类型关系就自动形成协变关系。

下面的实现同时处理类型变量的上界、沿继承链进行的子类型查询，以及类类型实参是否满足声明上界的检查。

Rust 辅助实现（译者增补）

```
pub fn is_subtype(
    &self,
    source: &GType,
    target: &GType,
    bounds: &HashMap<String, GType>,
) -> bool {
    self.is_subtype_avoiding_cycles(source, target, bounds, &mut
↪ HashSet::new())
```

```

}

fn is_subtype_avoiding_cycles(
    &self,
    source: &GType,
    target: &GType,
    bounds: &HashMap<String, GType>,
    visited: &mut HashSet<GType>,
) -> bool {
    if source == target {
        return true;
    }
    if !visited.insert(source.clone()) {
        return false;
    }
    match source {
        GType::Variable(name) => bounds.get(name).is_some_and(|bound| {
            self.is_subtype_avoiding_cycles(bound, target, bounds, visited)
        }),
        GType::Class(name, arguments) if name != "Object" => {
            let Ok(class) = self.class(name) else {
                return false;
            };
            let substitution: HashMap<_, _> = class
                .type_parameters
                .iter()
                .map(|parameter| parameter.name.clone())
                .zip(arguments.iter().cloned())
                .collect();
            self.is_subtype_avoiding_cycles(
                &substitute_type(&class.superclass, &substitution),
                target,
                bounds,
                visited,
            )
        }
        GType::Class(_, _) => false,
    }
}

pub fn check_type(&self, ty: &GType, bounds: &HashMap<String, GType>) ->
    ↪ Result<(), GjError> {

```

```

match ty {
  GType::Variable(name) => bounds
    .contains_key(name)
    .then_some(())
    .ok_or_else(|| GjError::UnknownType(name.clone())),
  GType::Class(name, arguments) if name == "Object" &&
↪ arguments.is_empty() => Ok(()),
  GType::Class(name, arguments) => {
    let class = self.class(name)?;
    if class.type_parameters.len() != arguments.len() {
      return Err(GjError::WrongTypeArity {
        expected: class.type_parameters.len(),
        actual: arguments.len(),
      });
    }
    for argument in arguments {
      self.check_type(argument, bounds)?;
    }
    let substitution: HashMap<_, _> = class
      .type_parameters
      .iter()
      .map(|parameter| parameter.name.clone())
      .zip(arguments.iter().cloned())
      .collect();
    for (parameter, argument) in
↪ class.type_parameters.iter().zip(arguments) {
      let bound = substitute_type(&parameter.bound, &substitution);
      if !self.is_subtype(argument, &bound, bounds) {
        return Err(GjError::BoundViolation {
          argument: argument.clone(),
          bound,
        });
      }
    }
    Ok(())
  }
}

```

字段与方法查询。 *fields*、*mtype* 和 *mbody* 沿继承链查找时，都必须累积并应用类型替换。泛型方法调用还要先检查显式类型实参满足方法的上界，再把它们代入形参类型、返

回类型与方法体中的类型标注。它们和上一题的查询框架相同，主要差别就是多了这一步替换，故不再重复列出整段实现。

类型检查与求值。类型判断改写为 $\Delta; \Gamma \vdash t : T$ 。对象创建和方法调用先检查类型实参，再按替换后的字段或形参类型检查项实参。运行时仍可采用 FJ 的求值规则：类型参数不参与字段选择和方法体替换，可以在执行前擦除；但证明擦除正确，需要先证明类型替换保持良构与赋型，再证明项替换保持赋型。

这里采用完整的擦除执行路径。建立泛型类表时先检查类型形参、上界、继承、字段、方法签名和方法体；字段或方法的接收者是类型变量时，沿 Δ 中的上界查询成员。执行程序前，把泛型类声明、字段、方法签名和方法体全部擦除为 FJ 类表，为每个类生成 FJ 的标准构造函数。若类型实参代入后，某个字段或方法的结果类型比其擦除后声明的结果类型更具体，擦除过程会在成员访问外插入必要的 FJ 类型转换；例如，`Box<A>.get()` 的声明在擦除后返回 `Object`，用于需要 `A` 的位置时便插入到 `A` 的转换。最后以同样方式擦除待求值项，并交给上一题的 FJ 解释器实际执行。

下面的代码给出擦除的关键步骤。`erase_type` 递归把类型变量替换成其上界的擦除；该函数只有这一条递归和类名保持不变两种情形，故不再单独列出。`erase_typed_term` 先保留源项的静态类型，再在字段访问或方法调用的擦除结果过于宽泛时插入 FJ 转换。它调用的字段类型与方法结果类型查询只负责沿擦除后的继承链查找声明，前文已经说明其算法，故不再重复列出。对象创建和原有转换只需递归擦除类型参数。

Rust 辅助实现（译者增补）

```
fn insert_erasure_cast(term: Term, actual: &str, expected: String) -> Term {
    if actual == expected {
        term
    } else {
        Term::Cast(expected, Box::new(term))
    }
}

/// 擦除一个良类型 GJ 项；若泛型字段或方法结果的擦除声明更一般，
/// 则插入 FJ 所需的类型转换。
fn erase_typed_term(
    &self,
    bounds: &HashMap<String, GType>,
    context: &HashMap<String, GType>,
    term: &GTerm,
) -> Result<(Term, GType), GjError> {
    let source_type = self.type_of(bounds, context, term)?;
    let erased = match term {
        GTerm::Variable(name) => Term::Variable(name.clone()),
        GTerm::Field(receiver, field) => {
            let receiver_type = self.type_of(bounds, context, receiver)?;
```

```

        let (erased_receiver, _) = self.erase_typed_term(bounds, context,
↪ receiver)?;
        let raw_type = self.erased_field_type(&receiver_type, field,
↪ bounds)?;
        let raw = Term::Field(Box::new(erased_receiver), field.clone());
        Self::insert_erasure_cast(raw, &raw_type, erase_type(&source_type,
↪ bounds))
    }
    GTerm::Invoke(receiver, name, _, arguments) => {
        let receiver_type = self.type_of(bounds, context, receiver)?;
        let (erased_receiver, _) = self.erase_typed_term(bounds, context,
↪ receiver)?;
        let erased_arguments = arguments
            .iter()
            .map(|argument| {
                self.erase_typed_term(bounds, context, argument)
                    .map(|(erased, _)| erased)
            })
            .collect::<Result<_, _>>()?;
        let raw_type =
            self.erased_method_result(&erase_type(&receiver_type, bounds),
↪ name)?;
        let raw = Term::Invoke(Box::new(erased_receiver), name.clone(),
↪ erased_arguments);
        Self::insert_erasure_cast(raw, &raw_type, erase_type(&source_type,
↪ bounds))
    }
    GTerm::New(ty, arguments) => Term::New(
        erase_type(ty, bounds),
        arguments
            .iter()
            .map(|argument| {
                self.erase_typed_term(bounds, context, argument)
                    .map(|(erased, _)| erased)
            })
            .collect::<Result<_, _>>()?,
    ),
    GTerm::Cast(ty, subject) => Term::Cast(
        erase_type(ty, bounds),
        Box::new(self.erase_typed_term(bounds, context, subject)?.0),
    ),
};

```

```
Ok((erased, source_type))
}
```

验证顺序。先测试无泛型 FJ 程序在扩展后行为不变；再测试单一泛型容器、带界类型变量、泛型方法、继承时改变类型实参、递归上界，以及违反上界的拒绝案例。还要拒绝重定义 `Object` 和改变类型形参上界的非法方法覆盖，并用泛型返回值处于更具体类型上下文的程序检验转换插入。最后比较擦除前后的求值结果。这样既覆盖 GJ 扩展新增的静态机制，也验证它没有改变 FJ 原有程序的可观察行为。

完整实现中的 `GenericTable::check_classes` 检查类和方法声明；`fields`、`method`、`type_of` 与 `check_arguments` 完成成员查询和类型检查；`erase_method`、`erase_table` 与 `eval_erased` 把整个程序交给 FJ 解释器执行。测试覆盖无泛型程序回归、泛型容器和方法、上界成员查询、改变类型实参的继承、递归上界、上界违反、非法覆盖，以及擦除后需要插入转换的方法体和链式成员调用。

[返回练习 19.5.6](#)

第 20 章：递归类型

练习 20.2.2

要证明的是图 20-1 中的同构递归演算满足以下两项性质。

进展性。若闭项 t 良类型，则 t 要么是值，要么存在 t' 使 $t \longrightarrow t'$ 。证明对类型推导作归纳。旧句法形式沿用简单类型 `lambda` 演算的证明，只需处理两种新增项。

若最后一条规则是 T-FOLD，则项形如 $\text{fold}[U]t_1$ 。由归纳假设， t_1 要么可以求值一步，此时用 E-FOLD 推进；要么已经是值 v_1 ，此时 $\text{fold}[U]v_1$ 本身就是值。

若最后一条规则是 T-UNFOLD，则项形如 $\text{unfold}[U]t_1$ 。由归纳假设，若 t_1 可以求值一步，就用 E-UNFOLD。若 t_1 已经是值，还需使用新增的典范形式事实：闭的 $U = \mu X.T$ 型值只能形如 $\text{fold}[U]v$ 。于是 E-UNFLDFLD 可用，整个项可以继续求值。

保持性。若 $\Gamma \vdash t : T$ 且 $t \longrightarrow t'$ ，则 $\Gamma \vdash t' : T$ 。证明对单步求值推导作归纳。两个同余规则直接使用归纳假设，再分别应用 T-FOLD 或 T-UNFOLD。

关键情形是

$$\text{unfold}[S](\text{fold}[U]v_1) \longrightarrow v_1.$$

因为原项良类型，T-UNFOLD 给出 $S = \mu X.T_1$ ，并且 $\text{fold}[U]v_1 : S$ 。再反演 T-FOLD，可知折叠标注必须是同一个递归类型 $U = S$ ，且

$$\Gamma \vdash v_1 : [X \mapsto S]T_1.$$

这正是原项经 T-UNFOLD 得到的结果类型，因此规约后的 v_1 保持原类型。

证明中最值得注意的两点，是为递归类型补充典范形式引理，以及在 E-UNFLDFLD 情形中借助良类型性证明两个运行时未比较的类型标注实际相同。

[返回练习 20.2.2](#)

第 22 章：类型重构

22.2.3 解答

记题中项为 t 。要让 $(xz)(yz)$ 可赋型， xz 必须得到一个函数，而 yz 必须得到该函数所需的参数。下面三组彼此不同的替换都满足这一要求：

$$\begin{aligned}\sigma_1 &= [X \mapsto \text{Nat} \rightarrow \text{Bool} \rightarrow \text{Nat}, Y \mapsto \text{Nat} \rightarrow \text{Bool}, Z \mapsto \text{Nat}], \\ \sigma_2 &= [X \mapsto \text{Bool} \rightarrow \text{Nat} \rightarrow \text{Bool}, Y \mapsto \text{Bool} \rightarrow \text{Nat}, Z \mapsto \text{Bool}], \\ \sigma_3 &= [X \mapsto Z \rightarrow A \rightarrow B, Y \mapsto Z \rightarrow A].\end{aligned}$$

相应结果类型分别是主类型的自然数实例、布尔实例，以及最一般的

$$(Z \rightarrow A \rightarrow B) \rightarrow (Z \rightarrow A) \rightarrow Z \rightarrow B.$$

第三组保留了 Z, A, B ，因而比前两组更一般：再令 $(Z, A, B) = (\text{Nat}, \text{Bool}, \text{Nat})$ ，就得到第一组；再令 $(Z, A, B) = (\text{Bool}, \text{Nat}, \text{Bool})$ ，就得到第二组。

[返回练习 22.2.3](#)

22.3.3 解答

先在上下文 $x : X, y : Y, z : Z$ 中处理最内层的三个应用：

1. xz 引入新鲜变量 A ，生成约束 $X = Z \rightarrow A$ ，结果类型为 A ；
2. yz 引入新鲜变量 B ，生成约束 $Y = Z \rightarrow B$ ，结果类型为 B ；
3. $(xz)(yz)$ 引入新鲜变量 C ，生成约束 $A = B \rightarrow C$ ，结果类型为 C 。

因此，函数体 $(xz)(yz)$ 的暂定类型为 C 。再由内向外应用三次 CT-Abs，依次得到

$$Z \rightarrow C, \quad Y \rightarrow Z \rightarrow C, \quad X \rightarrow Y \rightarrow Z \rightarrow C.$$

于是整个项的暂定结果类型以及生成的新鲜变量集合和约束集可以取为

$$\begin{aligned}S &= X \rightarrow Y \rightarrow Z \rightarrow C, \\ \mathcal{X} &= \{A, B, C\}, \\ C_0 &= \{X = Z \rightarrow A, Y = Z \rightarrow B, A = B \rightarrow C\}.\end{aligned}$$

推导树中 x, z, y, z 四个变量出现分别由四次 CT-Var 得到；随后应用三次 CT-App 构造三个应用，再应用三次 CT-Abs 包回三层 lambda 抽象，最终得到结论 $\vdash t : S \mid_{\mathcal{X}} C_0$ 。

[返回练习 22.3.3](#)

22.5.2 解答

合一解答 22.3.3 得到的约束：先由 $A = B \rightarrow C$ ，再代入 $X = Z \rightarrow A$ 、 $Y = Z \rightarrow B$ 。可取主合一子

$$[X \mapsto Z \rightarrow B \rightarrow C, Y \mapsto Z \rightarrow B, A \mapsto B \rightarrow C].$$

把它作用于结果类型 S ，得到主类型

$$(Z \rightarrow B \rightarrow C) \rightarrow (Z \rightarrow B) \rightarrow Z \rightarrow C.$$

变量名本身无关紧要；把 Z, B, C 一致地改名，仍是同一个主类型方案。

[返回练习 22.5.2](#)

22.5.5 解答

实现分为三层。第一层按图22-1遍历项，返回暂定结果类型和约束；第二层调用图22-2的合一算法；第三层把主合一子作用于暂定结果类型。关键不变量是：每次处理应用时，为其结果创建的类型变量都不曾出现过。无标注抽象要到 §22.6 才加入语言，本题仍沿用显式标注的 lambda 抽象。

书内已经依次给出实际 Rust 实现：图 22-1 后的类型与项定义、解答 22.3.10 中的约束生成器和 解答 22.4.6 中的合一器。把约束生成与合一串联起来、再把所得替换作用于暂定结果类型的主类型入口如下：

Rust 辅助实现（译者增补）

```
/// 先生成约束再进行合一，从而重构项的主类型。
pub fn principal_type(context: &Context, term: &Term) -> Result<Type,
↳ TypeError> {
    let mut fresh = FreshVariables::default();
    for ty in context.values() {
        fresh.reserve_type(ty);
    }
    fresh.reserve_term_annotations(term);
    let (schematic_type, constraints) = generate_constraints(context, term,
↳ &mut fresh)?;
    let substitution = unify(constraints)?;
    Ok(apply(&substitution, &schematic_type))
}
```

这三层合在一起，即构成题目要求的完整实现；这里不再重复刊载前两层源码。

同一份源码中的端到端测试覆盖题目给出的三组典型交互、自应用的出现检查失败、多步替换的传递应用、显式标注与自动生成变量同名时的回归情形，以及布尔值、自然数和条件项的约束生成。

[返回练习 22.5.5](#)

22.5.7 解答

本题只修改练习22.5.5的 `reconbase` 检查器，因此处理的仍是带显式形参类型标注的简单语言；无标注抽象和 `let` 多态分别到§22.6和 §22.7再加入。

增量版本不再先收集整棵语法树的约束、最后统一求解，而是在处理每个语法构造时立即合一新产生的约束。对外的入口仍返回整个项的主类型；递归辅助函数 *infer* 则返回一对 (σ, T) ： σ 是处理当前子项时求得的替换， T 是在当前已知约束下得到的最一般类型；这里约定，返回的 T 已经应用了同一次调用求出的 σ 。处理后续子项前，必须先把 σ 作用于上下文，使后续步骤能够使用刚刚获得的类型信息。

以应用 $t_1 t_2$ 为例，处理顺序可以明确写成：

$$\begin{aligned} \text{infer}(\Gamma, t_1) &= (\sigma_1, T_1), \\ \text{infer}(\sigma_1 \Gamma, t_2) &= (\sigma_2, T_2). \end{aligned}$$

接着创建全新的结果类型变量 X ，并求下列等式的最一般合一子 σ_3 ：

$$\sigma_2(T_1) = T_2 \rightarrow X.$$

左侧先应用 σ_2 ，是因为重构 t_2 时得到的新信息也可能约束 t_1 的类型。整个应用最终返回

$$(\sigma_3 \circ \sigma_2 \circ \sigma_1, \sigma_3(X)).$$

复合顺序记录了信息产生的先后次序：后求得的替换必须作用于此前替换的结果。

这里的中间类型只是当前已处理约束下的最一般类型，后续约束仍可能使它更具体。最一般性得以保留，是因为每次合一都选取最一般合一子：任何满足新约束的其他替换，都可以写成这个最一般合一子与另一替换的复合。因此，把它复合到已有替换时，只会施加满足新约束所必需的限制；处理到根节点时，所有约束都已纳入，所得类型便是整个项的主类型。例如， $(\lambda x : Y. x) 0$ 先为函数得到 $Y \rightarrow Y$ ，再由应用产生 $Y \rightarrow Y = \text{Nat} \rightarrow X$ ；最一般合一子只令 $Y = X = \text{Nat}$ ，于是整个应用的类型为 Nat ，无需重新遍历函数体。

下面给出 `reconbase` 全部语法分支的增量实现。递归辅助函数负责逐层处理项，并把替换返回给上一层，用来更新后续子项所处的上下文。最外层没有后续子项，`principal_type_incremental` 因而只返回已经更新完毕的类型。

Rust 辅助实现（译者增补）

```
/// 把替换作用于单态类型上下文中的每个绑定。
fn apply_monomorphic_context(substitution: &Substitution, context: &Context)
    -> Context {
    context
        .iter()
        .map(|(name, ty)| (name.clone(), apply(substitution, ty)))
        .collect()
}
```

```

/// 为 `reconbase` 的项增量求解约束，并返回替换与当前主类型。
///
/// 后处理的子项总是在前面所得替换更新过的上下文中检查。本函数只处理
/// 第 22.5 节 `reconbase` 支持的语法；遇到无标注抽象、`let` 等本题尚未
/// 处理的形式时，会明确报告不支持。
#[allow(clippy::too_many_lines)]
fn infer_reconbase_incremental(
    context: &Context,
    term: &Term,
    fresh: &mut FreshVariables,
) -> Result<(Substitution, Type), TypeError> {
    match term {
        Term::True | Term::False => Ok((Substitution::new(), Type::Bool)),
        Term::Zero => Ok((Substitution::new(), Type::Nat)),
        Term::Variable(name) => context
            .get(name)
            .cloned()
            .map(|ty| (Substitution::new(), ty))
            .ok_or_else(|| TypeError::UnknownVariable(name.clone())),
        Term::Abstraction {
            parameter,
            annotation: Some(parameter_type),
            body,
        } => {
            let mut body_context = context.clone();
            body_context.insert(parameter.clone(), parameter_type.clone());
            let (substitution, body_type) =
                infer_reconbase_incremental(&body_context, body, fresh)?;
            Ok((
                substitution.clone(),
                Type::Arrow(
                    Box::new(apply(&substitution, parameter_type)),
                    Box::new(apply(&substitution, &body_type)),
                ),
            ))
        },
        Term::Application(function, argument) => {
            let (function_substitution, function_type) =
                infer_reconbase_incremental(context, function, fresh)?;
            let argument_context =
                ↪ apply_monomorphic_context(&function_substitution, context);

```

```

    let (argument_substitution, argument_type) =
        infer_reconbase_incremental(&argument_context, argument,
↪ fresh)?;

    let result_type = fresh.fresh();
    let application_substitution = unify(vec![(
        apply(&argument_substitution, &function_type),
        Type::Arrow(Box::new(argument_type),
↪ Box::new(result_type.clone()))),
        ])?;
    let substitution = compose(
        &application_substitution,
        &compose(&argument_substitution, &function_substitution),
    );
    // result_type 是本分支刚创建的新鲜变量，前两个替换不可能约束它；
    // 只需应用当前合一所得的替换，就能得到整个应用的结果类型。
    Ok((substitution, apply(&application_substitution, &result_type)))
}

Term::Successor(argument) | Term::Predecessor(argument) => {
    let (substitution, argument_type) =
        infer_reconbase_incremental(context, argument, fresh)?;
    let numeric = unify(vec![(argument_type, Type::Nat)])?;
    Ok((compose(&numeric, &substitution), Type::Nat))
}

Term::IsZero(argument) => {
    let (substitution, argument_type) =
        infer_reconbase_incremental(context, argument, fresh)?;
    let numeric = unify(vec![(argument_type, Type::Nat)])?;
    Ok((compose(&numeric, &substitution), Type::Bool))
}

Term::If(guard, then_term, else_term) => {
    let (guard_substitution, guard_type) =
        infer_reconbase_incremental(context, guard, fresh)?;
    let guard_check = unify(vec![(guard_type, Type::Bool)])?;
    let before_then = compose(&guard_check, &guard_substitution);

    let then_context = apply_monomorphic_context(&before_then,
↪ context);
    let (then_substitution, then_type) =
        infer_reconbase_incremental(&then_context, then_term, fresh)?;
    let before_else = compose(&then_substitution, &before_then);

```

```

        let else_context = apply_monomorphic_context(&before_else,
↪ context);
        let (else_substitution, else_type) =
            infer_reconbase_incremental(&else_context, else_term, fresh)?;
        let branch_check = unify(vec![(
            apply(&else_substitution, &then_type),
            else_type.clone(),
        )])?;
        let substitution = compose(&branch_check,
↪ &compose(&else_substitution, &before_else));
        Ok((substitution, apply(&branch_check, &else_type)))
    }
    Term::Abstraction {
        annotation: None, ..
    } => Err(TypeError::UnsupportedInReconbase(
        "unannotated lambda abstraction",
    )),
    Term::Unit
    | Term::Reference(_)
    | Term::Dereference(_)
    | Term::Assignment(_, _)
    | Term::Sequence(_, _)
    | Term::Let { .. } => Err(TypeError::UnsupportedInReconbase(
        "construct introduced after Section 22.5",
    )),
    },
}
}

/// 增量重构 `reconbase` 项的主类型。
pub fn principal_type_incremental(context: &Context, term: &Term) ->
↪ Result<Type, TypeError> {
    let mut fresh = FreshVariables::default();
    // 生成新变量前，先登记上下文以及整棵项的标注中已经使用的变量名。
    for ty in context.values() {
        fresh.reserve_type(ty);
    }
    fresh.reserve_term_annotations(term);
    // 辅助函数已经把累计替换应用于返回类型；最外层不再需要该替换。
    let (_, ty) = infer_reconbase_incremental(context, term, &mut fresh)?;
    Ok(ty)
}

```

返回练习 22.5.7

22.7.1 解答

在练习22.5.7的增量重构器上增加类型方案 $\forall X_1 \dots X_n. T$ 。处理 `let` 时，先重构右侧并把当前替换作用于上下文；计算右侧类型中自由、但上下文中不自由的变量，把它们泛化后绑定给 `let` 变量。处理变量时，每次查询类型方案都以全新的类型变量实例化所有量化变量。

若语言包含引用等副作用，泛化步骤还必须实行值限制：只有右侧是句法值时才泛化；其他右侧以不量化的单态方案加入上下文。下面先给出在本节中新增加的类型方案、泛化、实例化和 `let` 分支等完整实现，再给出计算主类型的入口。

Rust 辅助实现（译者增补）

```
/// 一个类型方案由受全称量化的类型变量集合及其类型体组成。
#[derive(Clone, Debug, PartialEq, Eq)]
pub struct TypeScheme {
    quantified: BTreeSet<String>,
    ty: Type,
}

impl TypeScheme {
    fn monomorphic(ty: Type) -> Self {
        Self {
            quantified: BTreeSet::new(),
            ty,
        }
    }
}

pub type SchemeContext = BTreeMap<String, TypeScheme>;

fn type_variables(ty: &Type) -> BTreeSet<String> {
    let mut variables = BTreeSet::new();
    free_variables(ty, &mut variables);
    variables
}

fn scheme_variables(scheme: &TypeScheme) -> BTreeSet<String> {
    type_variables(&scheme.ty)
        .difference(&scheme.quantified)
        .cloned()
        .collect()
}
```



```

fn context_variables(context: &SchemeContext) -> BTreeSet<String> {
    context.values().flat_map(scheme_variables).collect()
}

fn apply_scheme(substitution: &Substitution, scheme: &TypeScheme) ->
    ↪ TypeScheme {
    // 只替换类型方案中的自由变量。若方案是 forall X. X -> Y, 而外部替换为
    // {X := Nat, Y := Bool}, 就必须忽略 X := Nat, 因为 X 已由 forall X 绑定;
    // 最终结果应是 forall X. X -> Bool。
    let filtered = substitution
        .iter()
        .filter(|(name, _)| !scheme.quantified.contains(*name))
        .map(|(name, ty)| (name.clone(), ty.clone()))
        .collect();
    TypeScheme {
        quantified: scheme.quantified.clone(),
        ty: apply(&filtered, &scheme.ty),
    }
}

fn apply_context(substitution: &Substitution, context: &SchemeContext) ->
    ↪ SchemeContext {
    context
        .iter()
        .map(|(name, scheme)| (name.clone(), apply_scheme(substitution,
    ↪ scheme)))
        .collect()
}

fn instantiate(scheme: &TypeScheme, fresh: &mut FreshVariables) -> Type {
    // 每次使用多态变量时, 都以一组全新的类型变量替换量化变量。
    let substitution = scheme
        .quantified
        .iter()
        .map(|name| (name.clone(), fresh.fresh()))
        .collect();
    apply(&substitution, &scheme.ty)
}

fn generalize(context: &SchemeContext, ty: Type) -> TypeScheme {
    // 只量化类型中自由、但上下文中不自由的变量, 避免捕获环境施加的约束。

```

```

    let context_variables = context_variables(context);
    let quantified = type_variables(&ty)
        .difference(&context_variables)
        .cloned()
        .collect();
    TypeScheme { quantified, ty }
}

fn is_syntactic_value(term: &Term) -> bool {
    match term {
        Term::True | Term::False | Term::Zero | Term::Unit | Term::Abstraction
    ↪ { .. } => true,
        Term::Successor(argument) => is_syntactic_value(argument),
        Term::Variable(_)
        | Term::Predecessor(_)
        | Term::IsZero(_)
        | Term::If(_, _, _)
        | Term::Application(_, _)
        | Term::Reference(_)
        | Term::Dereference(_)
        | Term::Assignment(_, _)
        | Term::Sequence(_, _)
        | Term::Let { .. } => false,
    }
}

/// 依照本节概述的 let 多态算法逐步推断主类型。
///
/// 每次递归调用同时返回子项的类型，以及检查该子项时求得的最一般替换；
/// 检查后续子项前，先把这个替换应用于上下文。
#[allow(clippy::too_many_lines)]
pub fn infer_incremental(
    context: &SchemeContext,
    term: &Term,
    fresh: &mut FreshVariables,
) -> Result<(Substitution, Type), TypeError> {
    match term {
        Term::True | Term::False => Ok((Substitution::new(), Type::Bool)),
        Term::Zero => Ok((Substitution::new(), Type::Nat)),
        Term::Unit => Ok((Substitution::new(), Type::Unit)),
        // 查询多态变量时实例化其类型方案；不同的查询得到彼此独立的新变量。
        Term::Variable(name) => context

```

```

        .get(name)
        .map(|scheme| (Substitution::new(), instantiate(scheme, fresh)))
        .ok_or_else(|| TypeError::UnknownVariable(name.clone())),
Term::Abstraction {
    parameter,
    annotation,
    body,
} => {
    // 无标注形参先取得新类型变量；函数体返回的替换也必须作用于形参类型。
    let parameter_type = annotation.clone().unwrap_or_else(||
↪ fresh.fresh());
    let mut body_context = context.clone();
    body_context.insert(
        parameter.clone(),
        TypeScheme::monomorphic(parameter_type.clone()),
    );
    let (body_substitution, body_type) =
↪ infer_incremental(&body_context, body, fresh)?;
    Ok((
        body_substitution.clone(),
        Type::Arrow(
            Box::new(apply(&body_substitution, &parameter_type)),
            Box::new(body_type),
        ),
    ))
}
Term::Application(function, argument) => {
    // 先处理函数，并把所得替换作用于上下文后再处理实参。
    let (function_substitution, function_type) =
        infer_incremental(context, function, fresh)?;
    let argument_context = apply_context(&function_substitution,
↪ context);
    let (argument_substitution, argument_type) =
        infer_incremental(&argument_context, argument, fresh)?;

    // X 表示整个应用的结果。实参阶段可能进一步约束函数类型，所以先把
    // argument_substitution 作用于 function_type，再执行合一。
    let result_type = fresh.fresh();
    let application_substitution = unify(vec![(
        apply(&argument_substitution, &function_type),
        Type::Arrow(Box::new(argument_type),
↪ Box::new(result_type.clone()))],

```

```

    ))?;

    // 后得到的替换作用在外层，依次吸收实参和函数阶段获得的信息。
    let substitution = compose(
        &application_substitution,
        &compose(&argument_substitution, &function_substitution),
    );
    // result_type 是本分支刚创建的新鲜变量，前两个替换不可能约束它；
    // 只需应用当前合一所得的替换，就能得到整个应用的结果类型。
    Ok((substitution, apply(&application_substitution, &result_type)))
}

Term::Reference(argument) => {
    let (substitution, ty) = infer_incremental(context, argument,
↪ fresh)?;
    Ok((substitution, Type::Reference(Box::new(ty))))
}

Term::Dereference(argument) => {
    let (substitution, ty) = infer_incremental(context, argument,
↪ fresh)?;
    // 解引用要求实参具有 Ref X；合一同时求出所指元素的类型 X。
    let result = fresh.fresh();
    let dereference = unify(vec![(ty,
↪ Type::Reference(Box::new(result.clone())))])?;
    Ok((
        compose(&dereference, &substitution),
        apply(&dereference, &result),
    ))
}

Term::Assignment(target, value) => {
    // 先处理赋值目标；处理右值前，必须用目标产生的替换更新上下文。
    let (target_substitution, target_type) =
↪ infer_incremental(context, target, fresh)?;
    let value_context = apply_context(&target_substitution, context);
    let (value_substitution, value_type) =
↪ infer_incremental(&value_context, value, fresh)?;

    // 右值阶段也可能约束目标类型，因此先更新 target_type，再要求它为
↪ Ref T。
    let assignment = unify(vec![(
        apply(&value_substitution, &target_type),
        Type::Reference(Box::new(value_type)),
    )])?;

```

```

Ok((
  compose(
    &assignment,
    &compose(&value_substitution, &target_substitution),
  ),
  Type::Unit,
))
})
Term::Sequence(first, second) => {
  // 序列的第一项必须为 Unit; 确认后再在更新后的上下文中处理第二项。
  let (first_substitution, first_type) = infer_incremental(context,
↪ first, fresh)?;
  let unit = unify(vec![(first_type, Type::Unit)])?;
  let before_second = compose(&unit, &first_substitution);
  let second_context = apply_context(&before_second, context);
  let (second_substitution, second_type) =
    infer_incremental(&second_context, second, fresh)?;
  Ok((compose(&second_substitution, &before_second), second_type))
}
Term::Successor(argument) | Term::Predecessor(argument) => {
  let (substitution, ty) = infer_incremental(context, argument,
↪ fresh)?;
  let numeric = unify(vec![(ty, Type::Nat)])?;
  Ok((compose(&numeric, &substitution), Type::Nat))
}
Term::IsZero(argument) => {
  let (substitution, ty) = infer_incremental(context, argument,
↪ fresh)?;
  let numeric = unify(vec![(ty, Type::Nat)])?;
  Ok((compose(&numeric, &substitution), Type::Bool))
}
Term::If(guard, then_term, else_term) => {
  // 守卫必须为 Bool; 这一检查产生的替换对两个分支都可见。
  let (guard_substitution, guard_type) = infer_incremental(context,
↪ guard, fresh)?;
  let guard_check = unify(vec![(guard_type, Type::Bool)])?;
  let after_guard = compose(&guard_check, &guard_substitution);
  let branch_context = apply_context(&after_guard, context);
  let (then_substitution, then_type) =
    infer_incremental(&branch_context, then_term, fresh)?;

  // 后一分支在前一分支更新过的上下文中检查, 最后再合一两个分支类型。

```

```

        let else_context = apply_context(&then_substitution,
↪ &branch_context);
        let (else_substitution, else_type) =
            infer_incremental(&else_context, else_term, fresh)?;
        let branch_check = unify(vec![(
            apply(&else_substitution, &then_type),
            else_type.clone(),
        )])?;
        let substitution = compose(
            &branch_check,
            &compose(
                &else_substitution,
                &compose(&then_substitution, &after_guard),
            ),
        );
        Ok((substitution, apply(&branch_check, &else_type)))
    }
    Term::Let { name, value, body } => {
        // 先把绑定项产生的替换作用于上下文和它自己的类型。
        let (value_substitution, value_type) = infer_incremental(context,
↪ value, fresh)?;
        let value_context = apply_context(&value_substitution, context);
        let value_type = apply(&value_substitution, &value_type);

        // 有引用时采用值限制：只有句法值可以泛化，其他项保持单态。
        let scheme = if is_syntactic_value(value) {
            generalize(&value_context, value_type)
        } else {
            TypeScheme::monomorphic(value_type)
        };
        let mut body_context = value_context;
        body_context.insert(name.clone(), scheme);
        let (body_substitution, body_type) =
↪ infer_incremental(&body_context, body, fresh)?;
        Ok((compose(&body_substitution, &value_substitution), body_type))
    }
}

/// 使用 ML 风格的 `let` 泛化重构主类型。
pub fn let_principal_type(context: &SchemeContext, term: &Term) ->
↪ Result<Type, TypeError> {

```

```
let mut fresh = FreshVariables::default();
// 生成新变量前，先登记上下文以及整棵项的标注中已经使用的变量名。
for scheme in context.values() {
    fresh.reserve_type(&scheme.ty);
}
fresh.reserve_term_annotations(term);
// 辅助函数已经把累计替换应用于返回类型；最外层不再需要该替换。
let (_, ty) = infer_incremental(context, term, &mut fresh)?;
Ok(ty)
}
```

返回练习 22.7.1

第 23 章：全称类型

23.4.1 解答

对 `id`，在 $\Gamma, X, x : X$ 下由 T-Var 有 $x : X$ ，连续使用 T-Abs 和 T-TAbs 得

$$\vdash \lambda X. \lambda x : X. x : \forall X. X \rightarrow X$$

对 `double`，固定 X 并令 $f : X \rightarrow X, a : X$ 。两次 T-App 依次给出 $f a : X$ 和 $f(f a) : X$ ；再依次抽象 a, f, X ，得到

$$\vdash \text{double} : \forall X. (X \rightarrow X) \rightarrow X \rightarrow X$$

对 `selfApp`，上下文中

$$x : \forall X. X \rightarrow X$$

由 T-TApp，在类型 $\forall X. X \rightarrow X$ 处实例化 x ，得到

$$x[\forall X. X \rightarrow X] : (\forall X. X \rightarrow X) \rightarrow (\forall X. X \rightarrow X)$$

再把原来的 x 作为实参应用，即得结果类型 $\forall X. X \rightarrow X$ 。外层 T-Abs 给出题中函数类型。

最后看 `quadruple`。在固定的 X 下，先实例化 `double`[X]，得到

$$(X \rightarrow X) \rightarrow X \rightarrow X$$

再把外层 `double` 的类型变量实例化为 $X \rightarrow X$ ：

$$\text{double}[X \rightarrow X] : ((X \rightarrow X) \rightarrow X \rightarrow X) \rightarrow (X \rightarrow X) \rightarrow X \rightarrow X$$

一次 T-App 正好允许它接收 `double`[X]，结果为 $(X \rightarrow X) \rightarrow X \rightarrow X$ 。最后对 X 使用 T-TAbs，得到题中全称类型。[返回练习 23.4.1](#)

23.4.2 解答

固定 X, Y 后， $f : X \rightarrow Y, l : \text{List } X$ 。空分支 `nil`[Y] 具有类型 $\text{List } Y$ 。非空分支中

$$\text{head}[X]l : X, \quad f(\text{head}[X]l) : Y,$$

递归调用

$$m(\text{tail}[X]l) : \text{List } Y$$

因此 `cons`[Y] 的两个实参类型正确，两个条件分支都具有 $\text{List } Y$ 。`fix` 的函数体由此具有 $\text{List } X \rightarrow \text{List } Y$ ，再抽象 f, Y, X 即得所示全称类型。[返回练习 23.4.2](#)

23.4.4 解答

可采用插入排序。允许 `fix` 时，完整定义如下：

```

insert = λX.λleq : X → X → Bool.
      fix(λins : X → List X → List X. λx : X.λl : List X.
        if isnil[X] l then cons[X] x (nil[X])
        else if leq x (head[X] l)
          then cons[X] x l
          else cons[X] (head[X] l) (ins x (tail[X] l))),
sort = λX.λleq : X → X → Bool.
      fix(λs : List X → List X. λl : List X.
        if isnil[X] l then nil[X]
        else insert[X] leq (head[X] l) (s (tail[X] l)))

```

其中

$$\text{insert} : \forall X. (X \rightarrow X \rightarrow \text{Bool}) \rightarrow X \rightarrow \text{List } X \rightarrow \text{List } X$$

沿列表递归，在第一个满足比较条件的位置插入新元素；`sort` 再递归排序表尾，并把表头插入已经排好的表尾。若要求纯 System F，则可使用 [解答 23.4.12](#) 的折叠编码。返回

[练习 23.4.4](#)

23.4.7 解答

对 `ctimes`，固定 X 和 $s : X \rightarrow X$ 后， $m[X]s : X \rightarrow X$ 。Church 数 n 把这个函数迭代 n 次；每次又会把 s 迭代 m 次，所以总计迭代 mn 次。

对 `cexp`， $m[X] : (X \rightarrow X) \rightarrow X \rightarrow X$ ，因此可把它看成类型 $X \rightarrow X$ 上的“迭代器”。令 n 在 $X \rightarrow X$ 这一类型上迭代 $m[X]$ ，便把“乘以 m 次迭代次数”的操作再迭代 n 次，得到 m^n 次应用。逐层使用 `T-TApp` 可得两项均返回 `CNat`。返回[练习 23.4.7](#)

第 24 章：存在类型

24.3.1 解答

先只看目标类型

$$\forall Y. (\forall X. \llbracket T \rrbracket \rightarrow Y) \rightarrow Y$$

要构造它，必须先抽象 Y ，再接收 $f : \forall X. \llbracket T \rrbracket \rightarrow Y$ 。手中只有隐藏类型 S 和值 $t : [X \mapsto S]T$ ，所以先把 f 在 $\llbracket S \rrbracket$ 处实例化，再应用于 $\llbracket t \rrbracket$ ：

$$\lambda Y. \lambda f : \forall X. \llbracket T \rrbracket \rightarrow Y. f[\llbracket S \rrbracket][\llbracket t \rrbracket]$$

开包时反过来，把包的翻译应用于所需结果类型 $\llbracket T_2 \rrbracket$ ，再把原来的包体翻译写成一个能对任意隐藏类型 X 和相应值 $x : \llbracket T_{11} \rrbracket$ 产生 $\llbracket T_2 \rrbracket$ 的函数：

$$\llbracket t_1 \rrbracket[\llbracket T_2 \rrbracket](\lambda X. \lambda x : \llbracket T_{11} \rrbracket. \llbracket t_2 \rrbracket)$$

这正是正文中的两条翻译。[返回练习 24.3.1](#)

第 26 章：有界量化

26.2.1 解答

S-Arrow 要求参数逆变、结果协变。目标 $B \rightarrow Y <: X \rightarrow B$ 因而分解为

$$\Gamma \vdash X <: B \quad \text{和} \quad \Gamma \vdash Y <: B$$

第一项由上下文 $X <: B$ 和 S-TVar 得到；第二项先由 $Y <: X$ 、 $X <: B$ 各用一次 S-TVar，再用 S-Trans 得到。最后应用 S-Arrow。[返回练习 26.2.1](#)

26.2.2 解答

取 $A <: B$ 。完全规则可推出

$$\forall X <: B. X <: \forall X <: A. B,$$

因为右侧只要求在较小的 A 范围上接受类型实参，而左侧能接受所有 B 的子类型；在 $X <: A$ 下又有 $X <: B$ 。核规则要求两边上界完全相同，因而不能推出。把量词体的 X, B 换成其他满足协变关系的类型，还可得到更多例子。[返回练习 26.2.2](#)

26.3.1 解答

展开序对：

$$\forall X. (S_1 \rightarrow S_2 \rightarrow X) \rightarrow X \quad \text{与} \quad \forall X. (T_1 \rightarrow T_2 \rightarrow X) \rightarrow X$$

量词上界同为 Top，可用核 S-All。量词体都是箭头；最外层参数位置逆变，所以只需证明

$$T_1 \rightarrow T_2 \rightarrow X <: S_1 \rightarrow S_2 \rightarrow X$$

再两次使用 S-Arrow，正好分别需要 $S_1 <: T_1$ 、 $S_2 <: T_2$ ，结果位置 $X <: X$ 由 S-Refl 得到。[返回练习 26.3.1](#)

26.3.6 解答

这个系统仍保留 System F 的无界多态，并非要用子类型取代全称类型；区别只是量词变量不能带上界。因而，章首的 f 与 f_2 仍可按原样使用， f_{poly} 也仍是普通的 System F 多态恒等函数。[§26.3](#)中的多态序对编码同样可用。新增的量词体协变规则还足以推出 $\text{Pair } S_1 S_2 <: \text{Pair } T_1 T_2$ ，所以基于柔性元组的记录编码及其宽度、深度子类型性质也能保留。

本章动机中的 $f2poly$ 不能照原样表达。函数体要访问 $x.a$ ，必须知道类型参数 X 满足 $X <: \{a : \text{Nat}\}$ ；无界量词无法把这项假设带进函数体。细化的 Church 数 SNat 、 SZero 、 SPos 也依赖 $S <: X$ 和 $Z <: X$ 等量词上界，因而不能保留这些精确类型。[§26.5](#)的有界

存在类型当然也不属于这个较简单的系统。简言之，它能表达“不依赖量词上界”的序对和记录例子，却不能表达需要在多态函数体内使用上界信息的例子。[返回练习 26.3.6](#)

26.4.16 解答

完全系统的证明框架不变，但所有涉及 S-All 的辅助引理都必须同时处理变化的上界。下面列出需要调整的地方。

首先，排列、弱化、删除无关项变量与项替换的陈述不变。收窄也仍有原来的两项陈述。其证明新增的关键情形是完全 S-All：若

$$\Gamma, X <: Q, \Delta \vdash \forall Y <: S_1.S_2 <: \forall Y <: T_1.T_2,$$

最后一条规则给出

$$\Gamma, X <: Q, \Delta \vdash T_1 <: S_1 \quad \text{和} \quad \Gamma, X <: Q, \Delta, Y <: T_1 \vdash S_2 <: T_2$$

分别对这两个更小的推导使用收窄的归纳假设，再应用完全 S-All 即可。类型替换保持子类型的 S-All 情形也完全相同：先对上界之间的前提作替换，再对扩展了 $Y <: [X \mapsto P]T_1$ 的量词体前提作替换。类型替换保持赋型随后仍由这个引理处理 T-Sub 与 T-TApp。

其次，右向左反演引理的全称类型一项必须改成：若

$$\Gamma \vdash S <: \forall X <: U_1.T_2,$$

则 S 是类型变量，或 $S = \forall X <: S_1.S_2$ ，并且

$$\Gamma \vdash U_1 <: S_1 \quad \text{且} \quad \Gamma, X <: U_1 \vdash S_2 <: T_2$$

因此，[引理26.4.12](#)的第 2 项也不再得到 $U_1 = S_1$ ，而只得到 $\Gamma \vdash U_1 <: S_1$ 。抽象体原来在 $\Gamma, X <: S_1$ 下赋型；先用收窄把上下文改为 $\Gamma, X <: U_1$ ，便可继续使用后面的替换引理。

在保持性证明的类型 beta 归约情形中，T-TApp 已知实际类型实参 $T_2 <: U_1$ 。上一步又给出 $U_1 <: S_1$ ，所以由传递性有 $T_2 <: S_1$ ；这正是把 T_2 代入原类型抽象体所需的前提。随后按核系统的证明一样应用类型替换保持赋型、类型替换保持子类型和 T-Sub。

典范形式引理只需把“两个量词上界相同”改成上面的逆变关系；它仍能推出闭的全称类型值一定是类型抽象。进展性只用这个外层形式，不要求两个上界相同，因而其余步骤不变。由此，完全 $F_{<}$ 同样满足保持性和进展性。[返回练习 26.4.16](#)

第 28 章：有界量化的元理论

28.4.3 解答

可判定。把题中的规则直接改写为算法规则：检查两个全称类型时，依次递归检查 $S_1 <: T_1$ 、 $T_1 <: S_1$ ，再在 $X <: T_1$ 下检查量词体 $S_2 <: T_2$ 。其他算法规则沿用核 $F_{<}$ 。

该过程会终止。先把记录字段按标签排成固定次序；这样，两个互为子类型的记录上界就有相同的规范表示，也有相同的权重。检查上界时，递归调用只处理两个全称类型的组成部分；检查量词体时，可以统一用该规范表示作为 (X) 的上界。以定义 28.3.4 的权重之和度量，这三个递归调用都严格小于原来的全称类型判断。与完全 SA-All 不同，这条规则不会在量词体中换入一个仅单向相关、可能越来越复杂的新上界，因而不会产生 §28.5 中的无限上界链。

可靠性直接来自题中声明式规则。完备性按声明式推导归纳：反身和传递情形分别使用算法关系的反身性、传递性；量词情形的三个前提正好由归纳假设得到。记录字段的排列会使两个上界语法不同，但记录子类型算法能分别证明它们互为子类型，所以题中给出的两个全称类型可以被算法接受。由此，该系统的子类型关系可判定。返回练习 28.4.3

28.5.2 解答

按定义， $\neg S = \forall X <: S.X$ ， $\neg T = \forall X <: T.X$ 。完全 $F_{<}$ 的 S-All 规则给出

$$\Gamma \vdash \forall X <: S.X <: \forall X <: T.X$$

当且仅当

$$\Gamma \vdash T <: S \quad \text{且} \quad \Gamma, X <: T \vdash X <: X.$$

第二个判断总可由反身性得到，所以整个判断成立恰好等价于 $\Gamma \vdash T <: S$ 。这说明外层全称量词的上界处于逆变位置：放进 $\neg(\cdot)$ 后，原来的子类型方向会反转。返回事实 28.5.2

28.5.4 解答

先用一项关于正负出现的单调性性质。若上下文含有 $X <: U$ ，则对类型结构归纳可得：

1. 若 X 在类型 A 中只正出现，则 $X <: U \vdash A <: [X \mapsto U]A$ ；
2. 若 X 在类型 A 中只负出现，则 $X <: U \vdash [X \mapsto U]A <: A$ 。

变量情形直接使用 $X <: U$ ；箭头的参数位置会交换正负，结果位置保持正负；全称类型的上界位置交换正负，量词体保持正负。因此各个归纳步骤正好对应 S-Arrow 和 S-All 的逆变、协变方向。

现在证明两个方向。若 $X <: U \vdash S <: T$ ，在这份推导中以 U 替换 X ；绑定 $X <: U$ 所需的前提变为 $U <: U$ ，由反身性成立。类型替换引理于是给出

$$\vdash [X \mapsto U]S <: [X \mapsto U]T.$$

反过来，假设上式成立。先用弱化把它放到上下文 $X <: U$ 中。由于 X 在 S 中只正出现、在 T 中只负出现，上面的单调性性质分别给出

$$S <: [X \mapsto U]S \quad \text{和} \quad [X \mapsto U]T <: T.$$

把这两个判断与假设用传递性连接起来，便得到 $X <: U \vdash S <: T$ 。[返回事实 28.5.4](#)